FROM SENSOR SIGNAL TO SYSTEM BEHAVIOUR

HYDROPONICS FIRMWARE EXPLAINED

C, FreeRTOS and Embedded Architecture



ESP32-S3 / SEVEN SENSORS / COMPLETE ORIGINAL SOURCE

HYDROPONICS FIRMWARE EXPLAINED

C, FreeRTOS and Embedded Architecture

A practical introduction through a complete ESP32-S3 controller.

From C declarations to sensor drivers, task ownership and relay decisions.

Guided explanations, precise diagrams and complete original source in one book.

PREFACE

Architecture becomes clear when the data has a path.

A hydroponics controller brings several kinds of time into one program. An ADC returns a voltage quickly. A temperature chip converts for hundreds of milliseconds. A gas sensor expects a regular command cadence. A relay decision must remain responsive while those operations continue. FreeRTOS provides execution contexts; the application architecture decides how they cooperate.

This book explains that cooperation through a complete firmware implementation. It starts with C records and ownership, follows each sensor from its physical signal to a Reading, then connects those observations to alarm policy, power transitions, saved settings and device communication. The aim is to make every important handoff visible: what the producer knows, what the consumer receives and which task owns the state in between.

The opening chapters assume basic familiarity with variables, conditions and loops. Pointers, structures, callbacks, task stacks and queues are developed where the firmware needs them. Each sensor has its own chapter, including its original source and bring-up program. Shared definitions are linked within the PDF so the explanation and implementation remain in one document.

Guided source readings focus on the statements whose order changes behaviour. A guard must reject bad data before conversion. A result must be retained before its consumer is notified. A worker must release the buses before its replacement receives permission. These are concrete reasons for code placement, not rules to memorise without context.

The source listings preserve every original line in the stated scope. Reading notes and line numbers sit outside the code. Small numerical examples expose the conversions, while the original native tests provide worked inputs and expected outcomes. Hardware measurements and host tests answer different questions; the reference section records which evidence supports the implementation.

The four parts build one connected view. Foundations establish the contracts. Sensor Architectures explains how observations are made. System Services shows how the controller acts and changes state. Integration and Verification follows complete events across those boundaries.

CONTENTS

The book at a glance

PART I / FOUNDATIONS

C records, time, task ownership and shared interfaces

01 Firmware and C Foundations 12

02 Shared Data and Time 40

03 RTOS: Tasks, Startup and Acquisition 57

04 Shared Driver, I2C and ADC Helpers 138

PART II / SENSOR ARCHITECTURES

Seven devices, seven paths from a signal to a Reading

05 DFR0026: Light Measurement 160

06 BME280: Air Temperature, Humidity and Pressure 173

07 DS18B20: Water Temperature 191

08 pH-4502C: pH Measurement 211

09 SEN0244: TDS Measurement 225

10 SGP30: TVOC and CO2eq 241

11 HC-SR04: Ultrasonic Distance 264

CONTENTS

The book at a glance / continued

PART III / SYSTEM SERVICES

Output policy, power, saved settings and communication

12 Alarm Rules, Lamp Policy and Relay Output 282

13 Power Modes, Rail Validity and Ownership Handover 317

14 Saved Configuration and Sensor Calibration 351

15 Device Communication: Protocol, BLE and Wi-Fi 385

PART IV / INTEGRATION AND VERIFICATION

Builds, original tests and complete system paths

16 Building, Bring-up and Native Tests 459

17 Complete System Walkthroughs 520

REFERENCE

Appendix A Board reference, glossary and sources 526

Complete source directory 534

Source integrity note 540

Navigation

The following pages list each numbered section. Every contents entry opens its destination. Chapter openings repeat their local contents, and PDF bookmarks reach the guided readings and complete original files.

DETAILED CONTENTS / 1 OF 6

Foundations: C and Data

01 Firmware and C Foundations 12

- 1.1 Reading the architecture through its code 13
- 1.2 The controller at a glance 14
- 1.3 Layers and responsibilities 15
- 1.4 C syntax needed to read this project 16
- 1.5 static: lifetime, scope and private names 17
- 1.6 Pointers, records and the C driver object model 18
- 1.7 const qualifies an access path 19
- 1.8 Memory lifetime: stack, static storage and heap 20
- 1.9 volatile makes accesses observable to the compiler 21
- 1.10 Headers, extern, inline and compiler directives 22
- 1.11 Guards, control flow and operators 23
- 1.12 Bits, byte order and numeric scale 24
- 1.13 Integer widths, casts and arithmetic boundaries 25
- 1.14 Arrays, strings, padding and byte copies 26
- 1.15 Build configuration and the Arduino bridge 27

02 Shared Data and Time 40

- 2.1 A Reading is more than a number 41
- 2.2 Health, quality and freshness answer different questions 42
- 2.3 Two clocks, two jobs 43

DETAILED CONTENTS / 2 OF 6

Foundations: RTOS and Drivers

03 RTOS: Tasks, Startup and Acquisition 57

3.1 What the RTOS contributes 58
3.2 A lease is permission to touch the shared drivers 59
3.3 A task is a function plus saved execution state 60
3.4 Priorities and affinity on two cores 61
3.5 RTOS types: handles, control blocks and ticks 62
3.6 Task creation and static memory 63
3.7 Delays, notifications and queues 64
3.8 Startup establishes dependencies and ownership 65
3.9 One worker owns all seven drivers 66
3.10 Absolute schedules avoid slow drift 67
3.11 How a pass starts and advances measurements 68
3.12 Why blocking work and echo capture cannot overlap freely 69
3.13 Results, frame guards and failure recovery 70
3.14 Latest state is not a history queue 71
3.15 Alarm delivery: store, queue, then notify 72
3.16 Safety is a task wrapper around a pure decision core 73
3.17 Cross-core access needs more than volatile 74
3.18 Choose synchronization by the contract it preserves 75
3.19 Logging is a bounded side channel 76
3.20 Progress supervision checks completed work 77
3.21 Memory supervision makes assumptions visible 78
3.22 Watchdogs recover from work that stops returning 79

04 Shared Driver, I2C and ADC Helpers 138

4.1 The common driver lifecycle 139
4.2 The I2C HAL: short transactions, one owner 140
4.3 The ADC HAL and the analog signal chain 141
4.4 Spaced windows and trimmed means 142

DETAILED CONTENTS / 3 OF 6

Sensors: Light, Climate and pH

05 DFR0026: Light Measurement 160

5.1 Light: ten small samples make one result 161
5.2 Light: from pin voltage to calibrated lux 162

06 BME280: Air Temperature, Humidity and Pressure 173

6.1 BME280: the simple reading path 174
6.2 BME280: identify, reset and configure 175
6.3 BME280: poll, compensate and publish 176

07 DS18B20: Water Temperature 191

7.1 DS18B20: send, leave, return 192
7.2 1-Wire: precise bits without occupying the whole conversion 193
7.3 DS18B20: a conversion that spans the frame 194

08 pH-4502C: pH Measurement 211

8.1 pH: first voltage, then a fitted line 212
8.2 pH: recover voltage, then apply a calibration 213
8.3 pH: the result decision order 214

DETAILED CONTENTS / 4 OF 6

Sensors: TDS, Gas and Distance

09 SEN0244: TDS Measurement 225

9.1 TDS: a voltage plus a water temperature 226
9.2 TDS: the polynomial and temperature dependency 227
9.3 TDS: filter, limits and undetectable faults 228

10 SGP30: TVOC and CO2eq 241

10.1 SGP30: a sensor with memory and timing rules 242
10.2 SGP30: command words, CRC and cadence 243
10.3 SGP30: a small state machine within one driver 244
10.4 SGP30: humidity and baseline determine health 245

11 HC-SR04: Ultrasonic Distance 264

11.1 HC-SR04: time the edges, then do the arithmetic 265
11.2 HC-SR04: measure a pulse, then convert distance 266
11.3 HC-SR04: completion and timeout are different outcomes 267

DETAILED CONTENTS / 5 OF 6

System Services

12 Alarm Rules, Lamp Policy and Relay Output 282

12.1 Alarm rules turn accepted samples into events 283
12.2 Lamp control: hysteresis, dwell and explicit exceptions 284
12.3 Latches, relay output and what feedback is missing 285

13 Power Modes, Rail Validity and Ownership Handover 317

13.1 Rail epochs prevent accepting a measurement across a transition 318
13.2 Power mode is a state machine, not a long delay 319
13.3 The power ISR records the urgent fact and defers policy 320
13.4 Power-sense diagnostics are a separate application 321
13.5 Power-sense analysis helpers, step by step 322

14 Saved Configuration and Sensor Calibration 351

14.1 Configuration: one owner for settings and flash 352
14.2 A/B slots, revisions and a checksum 353
14.3 Calibration capture does not take over the ADC 354
14.4 A mutex protects the ultrasonic echo from flash work 355

15 Device Communication: Protocol, BLE and Wi-Fi 385

15.1 Device communication crosses service boundaries 386
15.2 Bytes, messages and the bounded JSON codec 387
15.3 State, telemetry, events and command outcomes 388
15.4 Reading the two C++ adapters 389
15.5 Device-side BLE: short callbacks, one transport owner 390
15.6 A command crosses tasks without moving ownership 391
15.7 Wi-Fi joins in steps and backs off after failure 392

DETAILED CONTENTS / 6 OF 6

Integration and Reference

16 Building, Bring-up and Native Tests 459

16.1 Production and bring-up build different entry points 460
16.2 What each sensor bring-up program adds 461
16.3 Combined climate and water tests teach dependencies 462
16.4 Native tests as executable examples 463
16.5 The serial capture helper is a host-side tool 464

17 Complete System Walkthroughs 520

17.1 From darkness to a relay command 521
17.2 From an overtemperature sample to an interlock 522
17.3 A saved light calibration through the whole controller 523
17.4 Failure examples explain the architecture's boundaries 524
17.5 Implementation limits to keep visible during development 525

Appendix A Reference 526

A.1 Board profile and configuration reference 527
A.2 Glossary: language and execution 528
A.3 Glossary: measurements and state 529
A.4 Language reference: keywords and declarations 530
A.5 Sources and reproducibility 531
A.6 Sources and verification record 532
A.7 Sources for C, compiler and adapter semantics 533
Complete source directory 534

PART I / FOUNDATIONS

CHAPTER 01

Firmware and C Foundations

A controller is easier to understand when its responsibilities have names. C structures describe the data; headers define the contracts; folders separate hardware access from decisions. These boundaries explain how the rest of the program fits together.

In this chapter

1. Distinguish types, qualifiers, scope and memory lifetime.
2. Read pointers, headers, guards and byte operations.
3. Connect C contracts and build settings to the architecture.

Chapter contents

1.1 Reading the architecture through its code 13
1.2 The controller at a glance 14
1.3 Layers and responsibilities 15
1.4 C syntax needed to read this project 16
1.5 static: lifetime, scope and private names 17
1.6 Pointers, records and the C driver object model 18
1.7 const qualifies an access path 19
1.8 Memory lifetime: stack, static storage and heap 20
1.9 volatile makes accesses observable to the compiler 21
1.10 Headers, extern, inline and compiler directives 22
1.11 Guards, control flow and operators 23
1.12 Bits, byte order and numeric scale 24
1.13 Integer widths, casts and arithmetic boundaries 25
1.14 Arrays, strings, padding and byte copies 26
1.15 Build configuration and the Arduino bridge 27

Guided source readings begin on page 35.

Complete original listings begin on page 28. Shared files are linked in the explanations; Appendix A contains the full filename index.

CHAPTER 1

1.1 Reading the architecture through its code

A sensor value becomes useful only after software establishes what it measures, when it was acquired and whether its conditions were valid. This controller carries those facts through a common Reading record. The architecture protects that record as it moves from a device protocol to a relay decision.

Follow a complete path

The first chapters establish C data structures, time and task ownership. The light chapter then follows one complete measurement from ADC setup to a published result. The remaining sensor chapters add digital transactions, compensation, long conversions and interrupt timing to that same lifecycle.

Read the reason before the statements

Each chapter begins with the responsibility being implemented. Guided readings then place original statements beside their explanation. The discussion follows the input, guard, state update and next consumer, showing why validation precedes conversion and why retained data precedes notification.

Not every nearby declaration has an execution dependency. A type declaration must be visible where it is used; two independent constants can often appear in either order. The explanations distinguish these language requirements from the runtime ordering that protects shared state.

Keep the source beside the explanation

Complete original files follow the guided readings within the same chapter. Line numbers refer to those files. A grey > marks a visual continuation of a long line; it is not a source character. Internal links open shared helpers and the source directory without leaving the PDF.

The desktop Bluetooth app and its separate demo are outside scope. Device-side BLE remains part of the architecture because commands pass through Configuration and Safety. An embedded archive preserves the exact printed source bytes, and the final reference section records the revision and verification scope.

CHAPTER 1

1.2 The controller at a glance

The controller has seven physical sensors and ten measured metrics. Nine application tasks are created by a successful production boot. Two of them are acquisition workers, but only the worker holding the lease may access the shared drivers and buses.

Area	Behaviour in this source snapshot
Measurement	Normal and low-power workers share one serial measurement engine.
Safety	Owns the relay, consumes alarms and reset commands, checks missing inputs and power.
Power	GPIO8 edge detection, rail epochs and an acknowledged ownership handover.
Configuration	Queued commands, A/B NVS records, sensor selection and calibration.
Device communication	Secure BLE command/state/telemetry link; Wi-Fi station with bounded retry.
Observation	Logger, progress stamps, Supervisor, event journal and task-watchdog subscriptions.

What remains unfinished

There is no MQTT publisher, TCP data server or tank-volume conversion. The task name WifiMqtt is retained, but its body currently manages Wi-Fi association only. The desktop Bluetooth app and its separate demo firmware are outside this book.

The ordinary alarm rules still ship disabled. Lamp automatic control is enabled, with pin-voltage thresholds of 40 mV and 120 mV and 30 s minimum ON/OFF dwell. Stored, valid user calibrations can replace the nominal pH, TDS and light conversions.

Follow executable paths

Several source comments describe earlier stages. For example, main now starts Configuration, both acquisition workers, PowerCoordinator, BLE and Wi-Fi. A comment saying C-only also needs qualification: most logic is C, with two small C++ adapters for NimBLE and Arduino Wi-Fi.

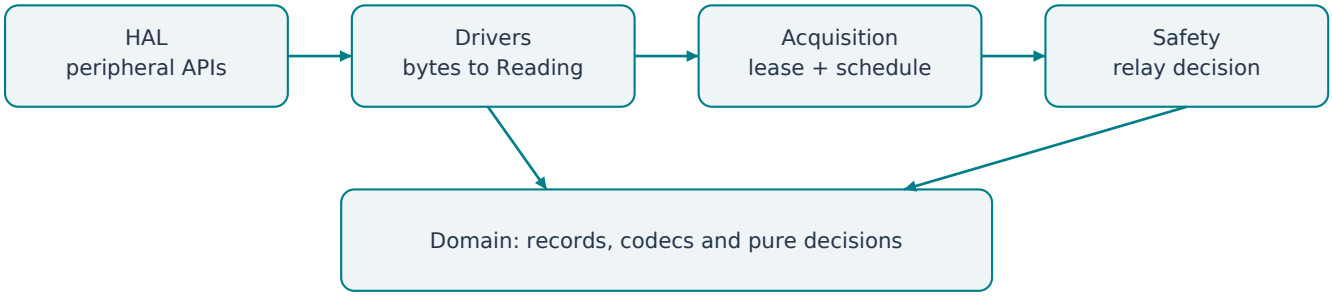
The next section follows a temperature through these layers. The final references identify the source revision and verification results.

Original source in this PDF: src/main.c (p. 82); include/policy_config.h (p. 286); src/services/acquisition.h (p. 89); src/transports/wifi_mqtt.c (p. 453)

CHAPTER 1

1.3 Layers and responsibilities

The folders express a direction of responsibility. Domain code knows values and decisions. The HAL knows the ESP32 peripheral APIs. Drivers translate device protocols into Reading records. Services organise work and own state. Transports carry external communication without owning the sensor buses or the relay.



Configuration and transports exchange copies through the service boundaries.

Follow a temperature through the layers

`i2cBusReadRegs()` retrieves bytes from the BME280. `bme280Poll()` unpacks them and applies factory coefficients. Acquisition copies the resulting Reading into its working Snapshot, evaluates matching rules, and publishes a queue copy. Safety receives alarms and peeks the snapshot. Only Safety decides whether to call the relay HAL.

This is why a sensor driver does not publish MQTT messages or change GPIO15. Such actions would connect measurement timing to network delays or create two independent relay writers. The source explicitly prohibits task creation, transport includes and relay decisions inside sensor drivers.

Headers and implementation files

A `.h` file declares the types and functions other modules may use; the matching `.c` implements them. `#include "domain/reading.h"` makes the Reading declaration visible at compile time. It does not execute that module. The linker later joins compiled object files and resolves calls.

`domain/` avoids Arduino and FreeRTOS headers. Consequently, the same algorithms run on a PC with a fake time value in the tests. This separation makes a threshold decision easy to verify without simulating an ADC or connecting an ESP32.

Original source in this PDF: src/drivers/sensor_driver.h (p. 144); src/domain/types.h (p. 44); platformio.ini (p. 28)

CHAPTER 1

1.4 C syntax needed to read this project

A declaration describes the type of a value. `uint8_t` is an unsigned 8-bit integer; `uint32_t` is an unsigned 32-bit integer. `float` stores an approximate real number. `bool` stores a logical value. A `typedef` gives an existing type a name that expresses its role: `MonoMs` is a `uint64_t` used for monotonic milliseconds.

```
// Source: src/domain/types.h
typedef uint64_t MonoMs;
typedef uint8_t SensorId;
```

The enumeration in types.h supplies named integer constants. The separate 8-bit typedef keeps fields compact. An enum constant is not a runtime object. `SENSOR_DS18B20` has value 1 because values increase from the explicit zero.

Structures and arrays

A `struct` groups fields into one record. `Reading out[4]` reserves four records; valid indices are 0 through 3. `out[k]` selects one. `Snapshot.readings[METRIC_COUNT]` holds ten records indexed by metric identity, making access predictable without a search.

`sizeof(s_stack)` gives storage in bytes. `sizeof(array) / sizeof(array[0])` counts elements when the expression still refers to an actual array. It does not count elements after an array has decayed to a pointer parameter.

Initialization supplies the first value

A declaration gives an object its type and name. An initializer such as `uint32_t count = 0;` supplies its initial value. An ordinary local without initialization does not start with a defined value. The function must establish its value before reading it. Static-storage objects receive zero initialization when no explicit initializer is given; the next section explains their lifetime.

A typedef adds an alias, not a new distinct C type. SensorId and MetricId both use uint8_t, so their names express intent without making every accidental interchange a type error. The enum constants identify the supported sensors and metrics.

`bool`, true and false come from stdbool.h in the C build. A bool stores a logical value. The underlying C type is _Bool; nonzero scalar values convert to true. The fixed-width integer aliases come from stdint.h.

The next sections separate lifetime, const access and volatile observations before connecting them to task ownership.

Original source in this PDF: src/domain/types.h (p. 44); src/services/acquisition.c (p. 96); include/policy_config.h (p. 286)

CHAPTER 1

1.5 static: lifetime, scope and private names

Three questions describe a C object: where its name is visible, how long the object exists and whether another compiled file can refer to that name. These are scope, storage duration and linkage. `static` affects different questions depending on where it appears. [R12]

A file-scope static object survives function calls

```
// Source: src/services/power_coordinator.c
static TaskHandle_t s_task = NULL;
```

This handle exists for the program's lifetime. Static storage persists across calls during one run; it does not by itself survive a reboot. The initializer gives it the null-pointer value before application execution. Internal linkage keeps the name private to this translation unit: this source file after its headers have been included. Another module obtains the handle through the public API instead of naming s_task directly.

A file-scope object without static also has static storage duration. The difference is usually external linkage, which allows a matching declaration in another file to name it. Removing static does not turn a global object into a stack local.

A local static remembers state between calls

```
// Teaching example: not a function from the repository.
uint32_t countVisits(void) {
  static uint32_t count;
  count++;
  return count;
}
```

In C, count is initialized to zero before execution begins. Three calls return 1, 2 and 3. Its name is visible only inside the function, but the object survives all three returns. With an ordinary initialized local, each call would begin with a fresh value.

A static function has a private name

`static bool gatedSetBlob(...)` defines a helper callable by name within configuration.c. Here static controls function linkage; it does not give the function a private stack or prevent simultaneous calls.

Static storage supplies one persistent object, not one copy per task. Both acquisition workers share the same file-scope driver objects. The lease, rather than the static keyword, keeps their access exclusive. A private C name can still identify an object whose address is passed elsewhere.

Original source in this PDF: src/services/power_coordinator.c (p. 334); src/services/configuration.c (p. 374); src/services/acquisition.c (p. 96)

CHAPTER 1

1.6 Pointers, records and the C driver object model

A pointer stores an address. `&object` obtains an address; `*pointer` accesses the object there. `r->health` is shorthand for `(*r).health`. `NULL` represents no valid object. A parameter such as `Reading* out` lets a function fill the caller's storage.

```
// Source: src/drivers/sensor_driver.h
struct SensorDriver {
  const SensorDriverOps* ops;
};
static inline DriverStatus sensorDriverStart(
    SensorDriver* d, MonoMs now) {
  return d->ops->start(d, now);
}
```

`ops` points to a table of function pointers. `start` selects a function from that table, then calls it with the driver address and current time. BME280's table selects `bme280Start`; DS18B20's table selects `ds18b20Start`. Acquisition uses the same call for both.

Why base must be the first member

Each concrete driver begins with `SensorDriver base`. The address of the structure and its first member are the same, so a pointer to that base can be cast back to the original concrete driver type. Helpers such as `asBme280()` perform this cast. The technique works only when the pointer really belongs to that driver and the first-member layout is preserved.

A constructor such as `bme280DriverConstruct()` is a normal C function. It assigns the operations table, copies configuration and resets results. It does not allocate memory and does not yet communicate with the chip. The later `init()` call performs hardware configuration.

Copy versus shared access

`self->config_ = *config` copies the configuration record, but any bus pointer inside it still refers to the existing bus object. `*out = s_stats` copies all fields into a caller-owned record. A FreeRTOS queue similarly copies the configured number of bytes; it does not automatically follow pointers inside those bytes.

The project avoids pointer-bearing Reading payloads. Therefore a queued Reading or Snapshot is a self-contained value whose contents do not depend on a producer's stack remaining alive.

Original source in this PDF: src/drivers/sensor_driver.h (p. 144); src/drivers/bme280.h (p. 177); src/drivers/bme280.c (p. 182)

CHAPTER 1

1.7 const qualifies an access path

A pointer and the object it points to are separate objects. Placing const before the pointed-to type limits writes through that pointer. Placing const after the star prevents reassignment of the pointer itself. [R12]

Declaration pattern	What the declaration permits
`const Reading* r`	r may point elsewhere; fields cannot be changed through r.
`Reading* const r`	r keeps its initial address; fields may be changed through r.
`const Reading* const r`	Neither reassignment nor field writes through r are permitted.

Read a declaration from the named object

In `const Reading* r`, r is a pointer to a const-qualified Reading. `Reading const* r` means the same thing. An initialized constant pointer can still be copied into another pointer variable; const constrains this declared object, not every possible alias.

`const SensorDriverOps* ops` gives a driver read-only access to its operation table. The driver object can remain mutable: its count, pending state and result change during measurement. A configuration parameter can likewise be read-only while the function changes its own driver state.

Const does not freeze a changing system

A pointer to const does not stop a different permitted writer from updating the underlying non-const object. It does not create a snapshot or lock. A consumer needing a coherent current record obtains a queue copy or uses the record's synchronization protocol.

An object actually defined as const must not be modified by casting away the qualifier. A cast can silence a type restriction; it cannot make that write valid. Const also does not promise a particular RAM or flash location. Placement depends on the object, compiler and linker.

The caller's pointer is passed by value

A function receiving `const Reading* r` receives a copy of the address. Reassigning its local r does not change the caller's pointer. Writing through a permitted non-const pointer can change the caller's object. A `Reading**` parameter is needed when a function is to update a caller-owned pointer variable itself.

Original source in this PDF: src/drivers/sensor_driver.h (p. 144); src/domain/alarm.h (p. 288); src/drivers/bme280.h (p. 177)

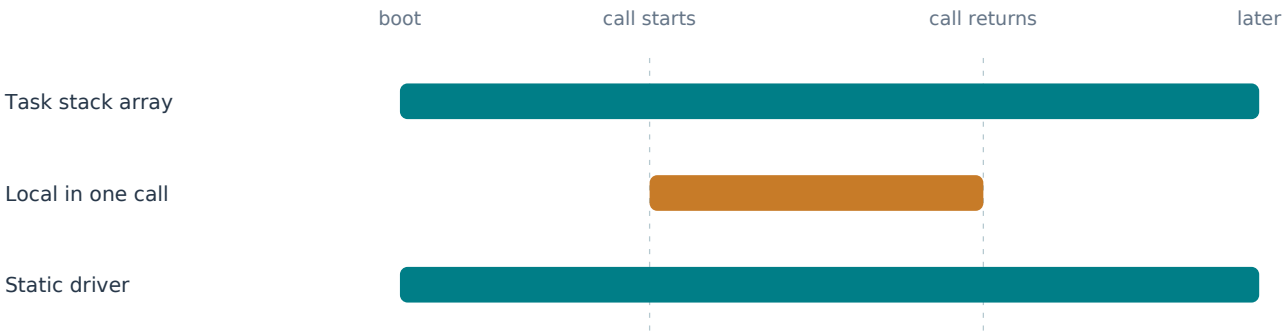
CHAPTER 1

1.8 Memory lifetime: stack, static storage and heap

Lifetime determines when an object's address can be used. Scope determines where its name is visible. A pointer can outlive the local object it once named.

Storage	Role in this controller
Automatic local	Per-call objects, normally on the calling task's stack.
Static object	Persistent drivers, queue buffers and task storage.
Allocated object	Heap blocks reserved at runtime until released.

The heap is a pool for runtime allocation. malloc reserves a block; free releases it. Framework allocators perform similar work internally. Returning from a caller does not itself release a heap block.



Reserved stack storage survives the shorter-lived objects placed within it.

The task stack array has static lifetime. Objects placed within it can have shorter call lifetimes. Reserved stack memory does not keep a returned function's locals alive. The diagram compares lifetimes, not memory addresses.

A blocked call is still active

Blocking preserves execution context and active locals. Returning ends the relevant automatic lifetimes. A task can wait with locals; a returned function cannot leave a valid pointer to its expired local buffer.

A private stack is not an access barrier

Tasks use separate stacks in this firmware's shared address space. Invalid pointers and out-of-bounds writes can corrupt another task's memory. Core affinity neither copies globals nor restricts pointer access.

Copy before the producer reuses storage

logPrintf builds a local LogRecord and sends it to a queue that copies its bytes. Logger later receives independent storage. A queue of pointers would copy only addresses and require a separate lifetime protocol. Reading and Snapshot carry self-contained values instead.

Original source in this PDF: src/diagnostics/logger.c (p. 125); src/services/snapshot_store.c (p. 88); src/services/acquisition.c (p. 96)

CHAPTER 1

1.9 volatile makes accesses observable to the compiler

A compiler can reuse an ordinary value when the program's visible operations allow it. Firmware also observes changes made asynchronously, outside a task's normal flow, by an interrupt or peripheral. An interrupt temporarily redirects a core to an ISR, an interrupt service routine that handles the event. Volatile marks accesses that the implementation must treat as observable. It is a type qualifier, not a scheduling operation. [R13]

```
// Source: src/hal/pulse_capture.h, selected members
volatile CaptureState state_;  // written last by the ISR
volatile uint32_t riseUs_;
volatile uint32_t fallUs_;
```

These fields connect task code with the echo ISR. A task reading state_ must perform the volatile access rather than assume an earlier value remains sufficient. Volatile does not mean that storage bypasses every CPU cache or that a read measures a voltage.

An access is not a complete synchronization protocol

`counter++` reads, adds and writes. Volatile alone does not make that sequence indivisible. Two writers can read the same old value and lose an increment. It also does not guarantee an indivisible 64-bit access on this 32-bit target.

GCC does not use a volatile flag as a general ordering barrier for ordinary memory. Publishing ordinary payload fields and then setting a volatile ready flag therefore needs more reasoning than the keyword alone. The pulse HAL deliberately makes the captured fields volatile and writes state last; its complete protocol still depends on ownership, interrupt placement and target access properties.

Read each use with its producer and consumer

The power ISR changes s_isrArmed asynchronously. The power task uses that observation while the rail pair is protected separately by a task/ISR spinlock. Progress timestamps use another lock, and BLE link facts use C++ atomics. These choices preserve different contracts.

Adding volatile to a shared Snapshot would neither copy it coherently nor replace the queue. The RTOS chapter explains the synchronization mechanisms that provide those stronger guarantees.

Original source in this PDF: src/hal/pulse_capture.h (p. 268); src/hal/pulse_capture.c (p. 271); src/services/power_coordinator.c (p. 334); src/services/rail_state.c (p. 324); src/diagnostics/progress.c (p. 129)

CHAPTER 1

1.10 Headers, extern, inline and compiler directives

A translation unit is one source file after preprocessing. Header declarations tell that unit which types and functions exist. A declaration is sufficient for compiling a call; linking must later find the required definition.

Extern declares a shared name

`extern uint32_t counter;` declares an externally linked object without defining its storage. A corresponding definition belongs in one source file. This is a teaching example; the controller generally exposes shared state through functions and copied records. An extern declaration neither locks the object nor creates a second copy.

A function prototype such as `bool sensorReady(void);` declares a function returning bool with no parameters. In C, `(void)` explicitly states that parameter list. A void return type means that the function returns no value. `void*` instead means a generic object pointer whose use requires the correct type and lifetime.

Static inline keeps a small wrapper in a header

```
// Source: src/drivers/sensor_driver.h
static inline DriverStatus sensorDriverStart(SensorDriver* d, MonoMs now) {
  return d->ops->start(d, now);
}
```

Static gives the header definition internal linkage in each including unit. Inline permits inline expansion, where the compiler substitutes the function body at a call. It does not require that optimization. The wrapper still performs an ordinary operation-table dispatch, and its return passes the selected driver's status back to acquisition. [R14]

Directives and SDK attributes act at build time

`#include` inserts declarations; `#define` defines a macro; `#if`, `#ifdef` and `#endif` select compiled text. `#pragma once` is the supported compiler directive that prevents repeated inclusion of a header within one translation unit. These lines are not task instructions executed at runtime.

`IRAM_ATTR` is an SDK macro used as a function attribute to place code in instruction RAM. It does not itself select interrupt priority or make every called function and referenced datum safe while flash cache is unavailable. Interrupt allocation and the entire call path determine that property. [R15]

Original source in this PDF: src/drivers/sensor_driver.h (p. 144); src/hal/pulse_capture.c (p. 271); src/hal/arduino_entry.h (p. 80); platformio.ini (p. 28)

CHAPTER 1

1.11 Guards, control flow and operators

Control-flow statements decide which operation executes next. Their placement matters whenever later code assumes a condition has already been checked.

```
// Source: src/domain/config_blob.c
if (b == NULL || b->magic != CONFIG_BLOB_MAGIC || b->schema != CONFIG_BLOB_SCHEMA) return false;
```

Logical OR, `||`, checks left to right and stops at the first true operand. If b is null, its fields are never read. Moving a field access before the null check would remove that protection. Logical AND, `&&`, stops at the first false operand. A nonzero scalar is true in an if condition; zero is false.

Loops repeat work; they do not imply a wait

A for loop repeats according to its initialization, condition and update. `for (;;)` has no stopping condition and forms the task's outer loop. A while loop checks its condition before each iteration. Neither syntax blocks a task by itself; an RTOS wait inside the body provides blocking. A do-while loop executes its body before checking. The project's `do { ... } while (0)` macros group several statements into one statement-shaped expansion.

Return, break and continue have different destinations

`return` leaves the current function, optionally giving its caller a result. A driver's return does not terminate the acquisition task that called it. `break` leaves the nearest enclosing loop or switch. `continue` skips the rest of the nearest loop iteration and proceeds according to that loop's control expression.

`switch` selects a case from a state value. Cases without a return or break can fall through. `default` handles a value not named by the cases. SGP30 uses these branches to distinguish an outstanding measurement reply from a baseline reply.

Logical operators differ from bit operations

`&`, `|` and `^` combine integer bits; `~` complements them. They do not short-circuit. Acquisition's `progress |= tryPoll(...)` always calls tryPoll and combines its result with the accumulated flag. Replacing that expression with a short-circuit OR can skip a required driver call once progress is already true.

`=` assigns; `==` compares; `!=` tests inequality. The conditional expression `test ? a : b` evaluates only the selected alternative. The relay HAL uses it to map a logical command through the configured polarity.

The same character can have another role in context: `&object` obtains an address, `*pointer` accesses its object, and `a & mask` performs bitwise AND. Parentheses expose grouping when those roles appear together.

Original source in this PDF: src/domain/config_blob.c (p. 360); src/services/acquisition.c (p. 96); src/drivers/sgp30.c (p. 254); src/hal/relay.c (p. 314)

CHAPTER 1

1.12 Bits, byte order and numeric scale

Firmware often receives bytes rather than physical units. Bitwise operations select fields or build register words. `1u << 3` produces a mask for bit 3. `status & mask` tests that bit. `flags | kQualityLate` adds a quality flag without removing existing flags.

```
// Source: src/domain/sgp30_codec.c
out[i] = (uint16_t)((w[0] << 8) | w[1]);
```

This combines a high byte and low byte. If they are `0x01` and `0x90`, the result is `0x0190`, or 400. SGP30 sends the most significant byte first. DS18B20 sends its temperature low byte first, so its decoder combines them in the opposite order.

Signed values need a signed interpretation

DS18B20's two bytes form a signed 16-bit temperature count. Casting the combined unsigned word to `int16_t` preserves the intended negative representation on the target. The count is divided by 16, because each least significant bit represents 0.0625 degC at 12-bit resolution. Example: raw 401 becomes 25.0625 degC; raw -162 becomes -10.125 degC.

Fixed point is an integer with an agreed scale

BME280's compensation functions keep integer precision internally. Temperature is in hundredths of a degree, pressure in Q24.8 Pa and humidity in Q22.10 %RH. Dividing by 100, 256 and 1024 gives the published floats. Q8.8 absolute humidity for SGP30 means an unsigned integer representing grams per cubic metre multiplied by 256.

These scales are part of the interface, not optional formatting. A raw pressure value interpreted without the division by 256 would be wrong by a factor of 256.

CRC checks communication, not chemistry

A CRC is a short check value computed from message bytes. The receiver recomputes it and rejects a mismatch. This detects many transmission errors. CRC checks byte integrity. Calibration, installation and the identity of the measured fluid require separate checks. DS18B20 and SGP30 use different initial values and bit processing even though both use a polynomial related to 0x31.

Original source in this PDF: src/domain/ds18b20_codec.c (p. 201); src/domain/sgp30_codec.c (p. 248); src/drivers/bme280.c (p. 182)

CHAPTER 1

1.13 Integer widths, casts and arithmetic boundaries

Signed integers represent negative and nonnegative values; unsigned integers represent nonnegative values. Fixed-width names make interfaces explicit. `uint8_t` spans 0-255, `int16_t` spans -32768 to 32767, and `uint32_t` spans 0-4294967295 on this target. `uint64_t` carries long monotonic times. The fixed-width types come from stdint.h; they are typedef names, not C keywords. The widths of int and long depend on the implementation. A char holds one C byte and is used for byte strings.

`size_t` is the unsigned type produced by sizeof and used for object sizes. `uintptr_t` is an available unsigned integer type that can preserve an object-pointer value when converted to an integer and back. Acquisition uses it when passing a small worker identity through the task API's void pointer parameter; that convention does not allocate an object at the numeric address.

A cast converts a value; it does not validate it

Converting 300 to uint8_t produces 44 because the unsigned conversion reduces it modulo 256. Converting a representable positive float to an integer discards its fractional part. Light adds 0.5f before its nonnegative pin voltage is converted, giving nearest-integer rounding on that path.

A cast cannot repair a wrong address, extend an object's lifetime or check whether a byte count fits a buffer. Pointer validity and numeric range must be established before the operation that depends on them.

The operand types determine the arithmetic

`7 / 2` produces integer 3. `7 / 2.0f` produces 3.5f. `2000.0f` in the ultrasonic formula keeps the distance calculation floating point. The f suffix makes a fractional literal float; without it the literal is double. Both are approximate real-number types. In a variadic printf call, which accepts additional arguments after its format string, float arguments are promoted to double; the logging casts make that interface explicit. The u suffix selects an unsigned integer literal, as in `1u << bit`.

Small integer types can be promoted to int before arithmetic. Choosing an 8-bit destination does not mean an expression executes entirely with 8-bit arithmetic. Intermediate widths matter in byte unpacking and BME280 compensation.

Wrap is deliberate only where the code allows it

Unsigned arithmetic wraps modulo its width. Subtracting two nearby 32-bit microsecond timestamps can therefore recover a short interval across wrap. Signed overflow has undefined behaviour: C specifies no valid result, so predictable wrap cannot be assumed. A shift count must also remain within the promoted operand's width. These rules explain the explicit unsigned masks and wider compensation intermediates.

Original source in this PDF: src/domain/types.h (p. 44); src/services/acquisition.c (p. 96); src/domain/ultrasonic.c (p. 274); src/drivers/bme280.c (p. 182); src/drivers/light.c (p. 168)

CHAPTER 1

1.14 Arrays, strings, padding and byte copies

A C array owns a fixed sequence of elements. Its valid indices run from zero to one less than its element count. C does not insert a bounds check for an arbitrary array access; the function's guards must establish that the index is valid.

Sizeof measures storage, not valid content

When applied to an actual array, `sizeof(a) / sizeof(a[0])` counts its elements. An array parameter such as `uint8_t data[]` is adjusted to a pointer parameter. Inside that function, sizeof(data) is the pointer's size, so the caller must supply the buffer length separately.

A structure can contain padding between or after its members to meet alignment requirements. sizeof includes that padding. The project's `_Static_assert` declarations reject unexpected Reading and Snapshot sizes at compile time. Unlike a runtime assert, a failing static assertion prevents the build.

Offsets and copies operate on bytes

`offsetof(ConfigBlob, crc)` gives the byte offset of the crc member. Configuration checksums the bytes before that offset, including intervening padding. This is why initialization and byte-preserving copies matter to its stored format.

memcpy copies a requested number of bytes between non-overlapping regions; memset writes a byte pattern. Neither understands pointer ownership or performs a deep copy. A copied structure's pointer fields still refer to the same external objects. A raw structure layout also depends on the compiler and target, so device communication encodes explicit JSON fields.

A string needs its terminating byte

A C string ends at a zero character, written `'\0'`. A buffer holding 20 single-byte characters needs at least 21 bytes. strlen counts bytes before that terminator, while sizeof on the buffer counts its total storage. A UTF-8 character can occupy several bytes; the protocol frame length counts bytes. Reading beyond an unterminated buffer is invalid.

NULL is a null pointer constant; it is not the string terminator. snprintf bounds its output and reports the length that would have been needed. The protocol writer separately tracks capacity and overflow so it can reject a truncated message before transmission.

Original source in this PDF: src/domain/reading.h (p. 46); src/domain/config_blob.c (p. 360); src/domain/protocol_codec.c (p. 402); src/diagnostics/logger.c (p. 125)

CHAPTER 1

1.15 Build configuration and the Arduino bridge

`platformio.ini` selects the toolchain, framework, board and source files. The environment name `esp32dev` is retained from an older board; its actual board setting is `esp32-s3-devkitc-1`. The board setting, not the environment's nickname, determines the target.

Setting	Meaning
`espressif32@6.12.0`	Pins the PlatformIO platform release.
`framework = arduino`	Supplies Arduino-ESP32 and its bundled ESP-IDF/FreeRTOS.
`qio_opi`	QIO flash with octal PSRAM memory arrangement.
`default_16MB.csv`	Selects the flash partition layout.
`-I src`	Adds src to header search paths.
`BOARD_HAS_PSRAM`	Enables board-specific framework support; does not move every static array to PSRAM.

NimBLE-Arduino 2.5.1 and ArduinoJson 6.21.5 are pinned. Production enables FW_ENABLE_BLE and FW_ENABLE_WIFI. NimBLE is used through a C++ adapter; JSON is built by the bounded C codec rather than ArduinoJson. Wi-Fi has a separate small C++ adapter.

How C setup() can be called from C++

The framework calls `setup()` and `loop()` from C++ code, where names are normally mangled into linker symbols. `hal/arduino_entry.h` attaches the expected symbol names to the C declarations:

```
// Source: src/hal/arduino_entry.h
void setup(void) __asm__("_Z5setupv");
void loop(void) __asm__("_Z4loopv");
```

This GNU compiler extension solves a linkage problem. It does not change how setup executes. Every alternative bring-up entry point includes the same bridge header.

The pinned platform uses the Arduino-ESP32 2.0.17 / ESP-IDF 4.4 family. The task and watchdog explanations therefore use the ESP-IDF 4.4 API. ESP-IDF 5 changes some signatures, including task-watchdog initialisation.

`CFG_CPU_MHZ_NORMAL` is 160, but no active call applies it. A configuration constant alone does not establish the actual running CPU frequency. Inspect the built framework configuration or measure it when timing depends on that value.

Original source in this PDF: platformio.ini (p. 28); src/hal/arduino_entry.h (p. 80); include/app_config.h (p. 36)

ORIGINAL SOURCE / CHAPTER 1 / 1 OF 4

platformio.ini

Original lines 1-41 | Code and comments unchanged; long lines wrap visually

Each environment selects a target and entry point. Test environments replace main.c, while native excludes hardware code.

```
; Hydroponics Monitoring System - new firmware (ESP32-S3-WROOM-1 N16R8)
; Env names kept from the WROOM-32D era so existing commands still work.
; Architecture: "Firmware Architecture.pdf" (Rev 3). Team guide: README.md.
; Language: all project sources are C (gnu11 on the host). The Arduino-ESP32
; framework stays the platform; its C HAL (esp32-hal-*) and FreeRTOS are used.
;
;   pio run -e esp32dev            production image (src/main.c)
;   pio run -e test_bme280 -t upload -t monitor
;                                  one sensor driver + its bring-up main
;   pio test -e native             host unit tests for src/domain (needs gcc)
;
; Versions are pinned to the ESP-Demo baseline (§13). Do not bump them
; without a compile + API check and a note in README.md.

[platformio]
default_envs = esp32dev

; ------------------------------------------------------------------ shared ----
[esp32_base]
platform = espressif32@6.12.0
framework = arduino
board = esp32-s3-devkitc-1
; N16R8: 16 MB QIO flash + 8 MB octal PSRAM (GPIO35-37 reserved, as board_profile.h).
board_build.arduino.memory_type = qio_opi
board_build.flash_size = 16MB
board_upload.flash_size = 16MB
board_build.partitions = default_16MB.csv
monitor_speed = 115200
; NimBLE-Arduino (C++) is used through the C shim src/transports/ble_gatt.cpp.
; ArduinoJson is not used by project code (JSON is built in domain/protocol_codec).
lib_deps =
    h2zero/NimBLE-Arduino@2.5.1
    bblanchon/ArduinoJson@6.21.5
build_flags =
    -I src
    -D CORE_DEBUG_LEVEL=1
    -D BOARD_HAS_PSRAM
test_ignore =
    bringup
    native

```

Return to guided explanation, p. 13

ORIGINAL SOURCE / CHAPTER 1 / 2 OF 4

platformio.ini

Original lines 42-82 | Code and comments unchanged; long lines wrap visually

Each environment selects a target and entry point. Test environments replace main.c, while native excludes hardware code.

```
; ------------------------------------------------------------ production ----
; Production images run the BLE transport (NimBLE) and the Wi-Fi station.
; Bring-up envs define neither, so they never link the radios.
[production]
build_flags =
    ${esp32_base.build_flags}
    -D FW_ENABLE_BLE=1
    -D FW_ENABLE_WIFI=1

[env:esp32dev]
extends = esp32_base
build_flags = ${production.build_flags}
build_src_filter = +<*>

; Acceptance test (§19 "Stall injection"): production image whose task blocks
; forever after 60 s. Expect: Supervisor WARN at ~2 s (acq) / ~1 s (safety),
; lamp OFF within 3 s (acq only), task-watchdog reset at ~5 s, then
; "reset reason: TASK_WDT" on the next boot. Never deploy these.
[env:stall_acq]
extends = env:esp32dev
build_flags =
    ${production.build_flags}
    -D FW_INJECT_STALL_ACQ=60000

[env:stall_safety]
extends = env:esp32dev
build_flags =
    ${production.build_flags}
    -D FW_INJECT_STALL_SAFETY=60000

; Acceptance test (cut BLE frame): every 5th Telemetry frame stops after its
; first chunk, as if NimBLE buffers stayed full. Expect "ble: frame cut ...
; link dropped to resync", a clean reconnect and no decode error in the app.
; Never deploy.
[env:ble_tear]
extends = env:esp32dev
build_flags =
    ${production.build_flags}
    -D FW_INJECT_BLE_TEAR=5

; Acceptance test (§19 "Temperature alarm"): a test rule trips on every valid
```

Return to guided explanation, p. 13

ORIGINAL SOURCE / CHAPTER 1 / 3 OF 4

platformio.ini

Original lines 83-123 | Code and comments unchanged; long lines wrap visually

Each environment selects a target and entry point. Test environments replace main.c, while native excludes hardware code.

```
; BME280 sample (4 per second, every frame phase) and de-energises the relay.
; Safety logs submit->action (us) and sample->action (ms) every 20 TRIPs.
; Target: sample -> Safety action <= 10 ms. Never deploy.
[env:alarm_latency]
extends = env:esp32dev
build_flags =
    ${production.build_flags}
    -D FW_ALARM_LATENCY_TEST=1

; ---------------------------------------------------- sensor bring-up ----
; Each env links every production module (drivers, hal, domain, ...) but
; replaces src/main.c with a single test/bringup/<sensor>_main.c (R9).
[bringup_base]
extends = esp32_base
build_flags =
    ${esp32_base.build_flags}
    -I test/bringup
    -D FW_BRINGUP=1

[env:test_bme280]
extends = bringup_base
build_src_filter = +<*> -<main.c> +<../test/bringup/bme280_main.c>

[env:test_ds18b20]
extends = bringup_base
build_src_filter = +<*> -<main.c> +<../test/bringup/ds18b20_main.c>

[env:test_light]
extends = bringup_base
build_src_filter = +<*> -<main.c> +<../test/bringup/light_main.c>

; Light level -> relay module (GPIO15) -> grow lamp, hysteresis + dwell.
[env:test_lamp]
extends = bringup_base
build_src_filter = +<*> -<main.c> +<../test/bringup/lamp_main.c>

[env:test_hcsr04]
extends = bringup_base
build_src_filter = +<*> -<main.c> +<../test/bringup/hcsr04_main.c>

[env:test_ph]
```

Return to guided explanation, p. 13

ORIGINAL SOURCE / CHAPTER 1 / 4 OF 4

platformio.ini

Original lines 124-165 | Code and comments unchanged; long lines wrap visually

Each environment selects a target and entry point. Test environments replace main.c, while native excludes hardware code.

```
124 extends = bringup_base
125 build_src_filter = +<*> -<main.c> +<../test/bringup/ph_main.c>
126
127 [env:test_tds]
128 extends = bringup_base
129 build_src_filter = +<*> -<main.c> +<../test/bringup/tds_main.c>
130
131 [env:test_sgp30]
132 extends = bringup_base
133 build_src_filter = +<*> -<main.c> +<../test/bringup/sgp30_main.c>
134
135 ; Shared I2C bus: BME280 + SGP30 together, BME280 humidity -> SGP30 compensation.
136 [env:test_climate]
137 extends = bringup_base
138 build_src_filter = +<*> -<main.c> +<../test/bringup/climate_main.c>
139
140 ; Water probes: DS18B20 + TDS together, DS18B20 water temperature -> TDS compensation.
141 [env:test_water]
142 extends = bringup_base
143 build_src_filter = +<*> -<main.c> +<../test/bringup/water_main.c>
144
145 ; External-power sense on GPIO8: edges (ISR, us) + ADC trace (1 kHz) on plug/unplug.
146 ; Commissioning gate for BOARD_POWER_SENSE_VERIFIED. DTR/RTS released so that
147 ; reopening the monitor after an unplug does not reset the board.
148 [env:test_power_sense]
149 extends = bringup_base
150 build_src_filter = +<*> -<main.c> +<../test/bringup/power_sense_main.c>
151 monitor_dtr = 0
152 monitor_rts = 0
153
154 ; ------------------------------------------------------- host unit tests ----
155 ; Only src/domain is compiled: it must stay free of Arduino/FreeRTOS headers.
156 [env:native]
157 platform = native
158 build_flags =
159     -I src
160     -I include
161     -std=gnu11
162 build_src_filter = -<*> +<domain/>
163 test_build_src = yes
164 test_filter = native/*
165 test_ignore = bringup
```

Return to guided explanation, p. 13

ORIGINAL SOURCE / CHAPTER 1 / 1 OF 3

include/board_profile.h

Original lines 1-35 | Code and comments unchanged; long lines wrap visually

These are the actual GPIO, divider and rail assignments for this board profile.

```
1  // Board profile: pins, rails and divider values for the ESP32-S3-WROOM-1 N16R8
2  // board (decision D16, supersedes D02).
3  // Source: Firmware Architecture Rev 3 §5 (Sensor <-> ESP32-S3 Signal Pinout,
4  // 22 Oct 2026, + Power Circuit Diagram). FROZEN CONTRACT - change only with a
5  // hardware-verified reason.
6  //
7  // Pure C (no Arduino headers) so domain code and native tests can use it.
8  // Anything not on the drawing stays BOARD_PIN_UNASSIGNED, and a pin on the
9  // drawing whose behaviour is not yet measured keeps its feature disabled
10 // (see boardProfileValidate()).
11 // Reserved on the N16R8: GPIO35-37 (octal PSRAM), GPIO19/20 (native USB),
12 // strapping pins GPIO0/3/45/46.
13 #pragma once
14 #include <stdbool.h>
15 #include <stdint.h>
16
17 #define BOARD_PIN_UNASSIGNED ((int8_t)-1)
18
19 typedef uint8_t Rail;
20 enum {
21   RAIL_3V3 = 0,         // retained 3.3 V supply, kept on battery
22   RAIL_5V_SWITCHED = 1  // external 5 V branches, lost on battery
23 };
24
25 // Bit mask used by the acquisition lease (allowed rails for a worker).
26 #define BOARD_RAIL_MASK_3V3 ((uint8_t)(1u << 0))
27 #define BOARD_RAIL_MASK_5V ((uint8_t)(1u << 1))
28
29 // ---- Shared I2C bus (retained 3.3 V; 4.7 kOhm pull-ups to 3.3 V) ------------
30 #define BOARD_PIN_I2C_SDA ((int8_t)21)
31 #define BOARD_PIN_I2C_SCL ((int8_t)47)
32 #define BOARD_I2C_CLOCK_HZ ((uint32_t)400000)
33
34 // ---- BME280 (retained 3.3 V, I2C) -------------------------------------------
35 #define BOARD_BME280_I2C_ADDRESS ((uint8_t)0x76)      // SDO = GND
```

Return to guided explanation, p. 13

ORIGINAL SOURCE / CHAPTER 1 / 2 OF 3

include/board_profile.h

Original lines 36-70 | Code and comments unchanged; long lines wrap visually

These are the actual GPIO, divider and rail assignments for this board profile.

```
36 #define BOARD_BME280_I2C_ADDRESS_ALT ((uint8_t)0x77)  // SDO = VDDIO
37
38 // ---- SGP30 module (retained 3.3 V, I2C, TVOC / CO2eq) -----------------------
39 #define BOARD_SGP30_I2C_ADDRESS ((uint8_t)0x58)  // fixed
40
41 // ---- DS18B20 (retained 3.3 V, 1-Wire, external supply, 4.7 kOhm pull-up) ----
42 #define BOARD_PIN_ONE_WIRE ((int8_t)14)
43
44 // ---- DFR0026 ambient light (retained 3.3 V, ADC1_CH0) -----------------------
45 // S -> 10 kOhm series -> GPIO1, 1 uF to GND (tau ~10 ms).
46 #define BOARD_PIN_LIGHT_ADC ((int8_t)1)
47
48 // ---- SEN0244 TDS (filtered external 5 V, ADC1_CH5, 0-2.3 V nominal) ---------
49 // A -> GPIO6, 100 nF to GND.
50 #define BOARD_PIN_TDS_ADC ((int8_t)6)
51
52 // ---- pH-4502C (filtered external 5 V, ADC1_CH6, divider 10k top / 15k bottom)
53 // PO -> 10k/15k -> GPIO7 + 1 uF to GND. DO and TO not connected.
54 #define BOARD_PIN_PH_ADC ((int8_t)7)
55 #define BOARD_PH_DIVIDER_TOP_OHM ((uint32_t)10000)
56 #define BOARD_PH_DIVIDER_BOTTOM_OHM ((uint32_t)15000)
57
58 // ---- HC-SR04 (external 5 V, ECHO through 1k/2k divider) ---------------------
59 #define BOARD_PIN_HCSR04_TRIG ((int8_t)17)
60 #define BOARD_PIN_HCSR04_ECHO ((int8_t)18)
61
62 // ---- External-power sense (ADC1_CH7; 220 Ohm over 2 x 220 Ohm from ext. 5 V) -
63 // ~3.33 V at 5.00 V. Used as a digital edge input; the ADC can check levels.
64 #define BOARD_PIN_EXTERNAL_POWER_SENSE ((int8_t)8)
65 #define BOARD_POWER_SENSE_DIVIDER_TOP_OHM ((uint32_t)220)
66 #define BOARD_POWER_SENSE_DIVIDER_BOTTOM_OHM ((uint32_t)440)
67 // Measured 25 Oct 2026 with test_power_sense (breadboard, 6 losses / 5 power-ons,
68 // README "External-power sense bring-up"): LOW 0-53 mV, HIGH >= 3.05 V (ADC
69 // clipped; <= 3.6 V by calculation only, 3.17-3.50 V for a 4.75-5.25 V supply).
70 // Switches at ~1.65 V falling / ~1.95 V rising. Power loss chatters 479-928
```

Return to guided explanation, p. 13

ORIGINAL SOURCE / CHAPTER 1 / 3 OF 3

include/board_profile.h

Original lines 71-105 | Code and comments unchanged; long lines wrap visually

boardPinAssigned (L88-88): These are the actual GPIO, divider and rail assignments for this board profile.
boardProfileValidate (L97-105): These are the actual GPIO, divider and rail assignments for this board profile.

```
71  // edges over 5-14 ms; power-on 3-16 edges over < 0.2 ms. First falling edge
72  // 59-220 ms after the rail starts to drop (+ rail already ~2.5 V).
73  // Requirements on WP9: the ISR disables itself on the first edge and the power
74  // task re-reads the level after BOARD_POWER_SENSE_QUALIFY_MS before acting and
75  // re-arming; the rail epoch also rejects 5 V readings that ended just before it.
76  #define BOARD_POWER_SENSE_VERIFIED true
77  #define BOARD_POWER_SENSE_QUALIFY_MS ((uint32_t)20)
78
79  // ---- Relay 1 (SRD-05VDC, coil on external 5 V) ------------------------------
80  // GPIO15 -> 220 Ohm -> IRLZ44N gate (low side), 10 kOhm gate pull-down,
81  // 1N5822 flyback. High = coil energised. Contact use and load still open (§12).
82  #define BOARD_PIN_RELAY1 ((int8_t)15)
83  #define BOARD_RELAY1_ACTIVE_LOW false
84
85  // ---- Open hardware gates (§20) - NOT on the drawing --------------------------
86  #define BOARD_PIN_RAIL_5V_FEEDBACK BOARD_PIN_UNASSIGNED
87
88  static inline bool boardPinAssigned(int8_t pin) { return pin != BOARD_PIN_UNASSIGNED; }
89
90  typedef struct BoardValidation {
91    bool powerSenseAssigned;
92    bool railFeedbackAssigned;
93    bool relayAssigned;
94    bool productionReady;  // false => report a non-production build status
95  } BoardValidation;
96
97  static inline BoardValidation boardProfileValidate(void) {
98    BoardValidation v;
99    v.powerSenseAssigned = boardPinAssigned(BOARD_PIN_EXTERNAL_POWER_SENSE);
100   v.railFeedbackAssigned = boardPinAssigned(BOARD_PIN_RAIL_5V_FEEDBACK);
101   v.relayAssigned = boardPinAssigned(BOARD_PIN_RELAY1);
102   // Minimum gate for power modes: the sense pin exists AND has been measured.
103   v.productionReady = v.powerSenseAssigned && BOARD_POWER_SENSE_VERIFIED;
104   return v;
105 }
```

Return to guided explanation, p. 13

GUIDED SOURCE READING / CHAPTER 1

Read a configuration line as a choice with consequences

Before following tasks, establish the numbers they use. A header centralizes policy shared by several files, so acquisition and supervision do not each invent their own meaning for a frame or stale input. The cast on a constant fixes the intended integer type; it does not start a timer.

Original excerpt: include/app_config.h (p. 36) / lines 11-20. The complete unchanged file follows.

```
11  #define CFG_FRAME_PERIOD_MS ((uint32_t)1000)         // ordinary sensors
12  #define CFG_BME280_PERIOD_MS ((uint32_t)250)         // fast climate check
13  #define CFG_BME280_BUDGET_MS ((uint32_t)12)          // conversion + read
14  #define CFG_DS18B20_CONVERSION_MAX_MS ((uint32_t)750)  // 12-bit
15  #define CFG_DS18B20_BUDGET_MS ((uint32_t)20)
16  #define CFG_HCSR04_TIMEOUT_MS ((uint32_t)30)
17  #define CFG_ANALOG_GROUP_BUDGET_MS ((uint32_t)5)
18  #define CFG_SNAPSHOT_BUDGET_MS ((uint32_t)10)
19  #define CFG_FRAME_RESERVATION_MS ((uint32_t)150)
20  #define CFG_SERVICE_STEP_TARGET_MS ((uint32_t)5)
```

The frame period releases ordinary measurement work every 1000 ms. The next period constant releases BME280 every 250 ms. Their separation allows four climate opportunities in one ordinary frame without requiring four starts for every other sensor.

The 750 ms DS18B20 conversion constant describes elapsed sensor time. Its separate 20 ms budget represents a different scheduling concern. Neither line allocates CPU time automatically: the functions that read these constants decide how to wait or admit a call. This distinction is why configuration should be read together with its call sites.

Notice that changing the order of independent #define lines would not change execution. In contrast, changing a period changes every deadline calculated from it. Some source placement groups related declarations for readability; later walkthroughs identify the statement orders that are required by data or hardware dependencies.

Continue to the task table in the full listing. Each row is a record, not a task-creation call. The startup code must pass those fields to FreeRTOS before a task exists.

ORIGINAL SOURCE / CHAPTER 1 / 1 OF 2

include/app_config.h

Original lines 1-46 | Code and comments unchanged; long lines wrap visually

These constants describe periods, queues and task placement. A constant only affects a path that uses it.

```
// Application timing, queue and task budgets.
// Source: Firmware Architecture §6 (tasks), §7 (schedule), §8 (staleness),
// §9 (queues), §11 (power). These are DESIGN BUDGETS, not measurements:
// replace with measured values as each stage is validated.
//
// Pure C (no Arduino headers).
#pragma once
#include <stdint.h>

// ---- Schedule (§7) ---------------------------------------------------------
#define CFG_FRAME_PERIOD_MS ((uint32_t)1000)         // ordinary sensors
#define CFG_BME280_PERIOD_MS ((uint32_t)250)         // fast climate check
#define CFG_BME280_BUDGET_MS ((uint32_t)12)          // conversion + read
#define CFG_DS18B20_CONVERSION_MAX_MS ((uint32_t)750)  // 12-bit
#define CFG_DS18B20_BUDGET_MS ((uint32_t)20)
#define CFG_HCSR04_TIMEOUT_MS ((uint32_t)30)
#define CFG_ANALOG_GROUP_BUDGET_MS ((uint32_t)5)
#define CFG_SNAPSHOT_BUDGET_MS ((uint32_t)10)
#define CFG_FRAME_RESERVATION_MS ((uint32_t)150)
#define CFG_SERVICE_STEP_TARGET_MS ((uint32_t)5)

// ---- Staleness (§8) --------------------------------------------------------
#define CFG_STALE_BME280_MS ((uint32_t)1000)
#define CFG_STALE_ORDINARY_MS ((uint32_t)3000)

// ---- Queues (§9) -----------------------------------------------------------
#define CFG_ALARM_DETAIL_QUEUE_LEN ((uint32_t)16)
#define CFG_ALARM_JOURNAL_LEN ((uint32_t)64)
#define CFG_COMMAND_QUEUE_LEN ((uint32_t)8)
#define CFG_TELEMETRY_BACKLOG_LEN ((uint32_t)60)
#define CFG_LOG_QUEUE_LEN ((uint32_t)32)

// ---- Power transitions (§11) -----------------------------------------------
#define CFG_POWER_LOSS_DEBOUNCE_MS ((uint32_t)50)
#define CFG_POWER_RETURN_STABLE_MS ((uint32_t)500)
#define CFG_TRANSITION_ACK_MS ((uint32_t)200)
#define CFG_NETWORK_SHUTDOWN_ACK_MS ((uint32_t)2000)

// ---- Network (§15) ---------------------------------------------------------
#define CFG_MQTT_BACKOFF_INITIAL_MS ((uint32_t)1000)
#define CFG_MQTT_BACKOFF_MAX_MS ((uint32_t)60000)
#define CFG_BLE_STATE_PERIOD_MS ((uint32_t)2000)
#define CFG_BLE_TELEMETRY_PERIOD_MS ((uint32_t)3000)

// ---- Safety / supervision (§10, §16) ---------------------------------------
#define CFG_SAFETY_HEALTH_CHECK_MS ((uint32_t)100)
```

Return to guided explanation, p. 35

ORIGINAL SOURCE / CHAPTER 1 / 2 OF 2

include/app_config.h

Original lines 47-92 | Code and comments unchanged; long lines wrap visually

cfgTaskSpec (L72-89): These constants describe periods, queues and task placement. A constant only affects a path that uses it.

```
47 #define CFG_SUPERVISOR_PERIOD_MS ((uint32_t)1000)
48 #define CFG_TASK_WATCHDOG_MS ((uint32_t)5000)
49
50 // ---- Tasks (§6). Higher number = higher priority. Stack in bytes. --------------
51 typedef uint8_t TaskId;
52 enum {
53   TASK_ID_SAFETY = 0,
54   TASK_ID_POWER_COORDINATOR,
55   TASK_ID_ACQUISITION_NORMAL,
56   TASK_ID_ACQUISITION_LOW_POWER,
57   TASK_ID_WIFI_MQTT,
58   TASK_ID_BLE_TRANSPORT,
59   TASK_ID_CONFIGURATION,
60   TASK_ID_SUPERVISOR,
61   TASK_ID_SERIAL_LOGGER,
62   TASK_ID_COUNT
63 };
64
65 typedef struct TaskSpec {
66   const char* name;
67   uint8_t core;
68   uint8_t priority;
69   uint32_t stackBytes;
70 } TaskSpec;
71
72 static inline const TaskSpec* cfgTaskSpec(TaskId id) {
73   // Indexed by TaskId. Keep order in sync with the enum above.
74   static const TaskSpec kTaskSpecs[TASK_ID_COUNT] = {
75       // Core 1 since 28 Oct 2026 (D14 fallback): the Wi-Fi driver's start-up
76       // starved Safety on core 0 for 40-110 ms; on core 1 the worst case is
77       // 145 us. Frozen-header change made on the project owner's decision.
78       {"Safety", 1, 6, 3 * 1024},
79       {"PowerCoord", 0, 5, 4 * 1024},
80       {"AcqNormal", 1, 4, 4 * 1024},
81       {"AcqLowPower", 0, 4, 4 * 1024},
82       {"WifiMqtt", 0, 2, 6 * 1024},
83       {"BleTransport", 0, 2, 6 * 1024},
84       {"Configuration", 0, 2, 4 * 1024},
85       {"Supervisor", 0, 1, 3 * 1024},
86       {"SerialLogger", 1, 1, 2 * 1024},
87   };
88   return &kTaskSpecs[id];
89 }
90
91 // ---- CPU frequency (§13) - 80 MHz low power is only a candidate ----------------
92 #define CFG_CPU_MHZ_NORMAL ((uint32_t)160)
```

Return to guided explanation, p. 35

ORIGINAL SOURCE / CHAPTER 1 / 1 OF 1

.gitignore

Original lines 1-13 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
.pio/
.vscode/
__pycache__/
tools/.ble_bench_state.json
# Desktop app (app/): environments and build work files. app/dist/ (the
# Windows .exe and the Linux binary) IS committed.
app/.venv/
app/.venv-linux/
app/build/
app/.pytest_cache/
app/*.spec
app/link.log
*.pyc
```

Return to guided explanation, p. 13

ORIGINAL SOURCE / CHAPTER 1 / 1 OF 1

.gitattributes

Original lines 1-2 | Code and comments unchanged; long lines wrap visually

Git export policy excludes app/dist from source archives; this file is configuration, not MCU runtime code.

```
1 # Shell scripts must keep LF endings, or bash on Linux / WSL rejects them.
2 *.sh text eol=lf
```

Return to guided explanation, p. 13

PART I / FOUNDATIONS

CHAPTER 02

Shared Data and Time

A temperature value needs a unit, a measurement interval and a health verdict before another task can act on it. The Reading record keeps those facts together. Time checks then determine whether an accepted observation is still current.

In this chapter

1. Read the fields of Reading and Snapshot as a data contract.
2. Distinguish health, quality and freshness.
3. Use monotonic deadlines without confusing milliseconds and ticks.

Chapter contents

2.1 A Reading is more than a number 41
2.2 Health, quality and freshness answer different questions 42
2.3 Two clocks, two jobs 43

Guided source readings begin on page 48.

Complete original listings begin on page 44. Shared files are linked in the explanations; Appendix A contains the full filename index.

CHAPTER 2

2.1 A Reading is more than a number

A value without its unit, age and health cannot support a reliable decision. The Reading record carries all of these together. The compiled layout is asserted to be 56 bytes; a Snapshot holding ten Readings is asserted to be 584 bytes. Padding for alignment explains why summing just visible field widths is insufficient.

Field group	Why it exists
`sensor`, `metric`, `unit`	Identifies the source, meaning and numeric unit.
`health`, `qualityFlags`	States usability and extra limitations.
`value`, `raw`, `hasRaw`	Keeps converted output and an optional diagnostic input.
`intervalStartMs`, `intervalEndMs`	Describes when the acquisition happened.
`lastGoodMs`, `sequence`	Tracks usable results independently of failures.
`powerEpoch`	Identifies supply history; acquisition and consumers reject invalid or changed-epoch 5V results.
`configRevision`, `calibrationRevision`	Records which settings produced the result.

Seven sensors, ten meanings

BME280 yields air temperature, humidity and pressure. DS18B20 yields water temperature. DFR0026 yields light voltage, or lux after a stored fit. HC-SR04 yields water-surface distance. pH-4502C yields pH or module voltage depending on calibration. SEN0244 yields TDS. SGP30 yields TVOC and estimated CO2eq.

`raw` is not universally ADC counts. Analog drivers store filtered pin millivolts. DS18B20 stores signed temperature counts. HC-SR04 stores pulse microseconds. BME280 stores raw register counts. The unit and driver contract determine the interpretation.

`readingReset()` clears storage, sets identity, and sets STARTING. A zero float left in that record is not a measured zero. Most failure paths preserve an earlier numeric value while changing health; consumers must inspect health rather than displaying the old number as current.

`_Static_assert` checks the layout during compilation. This protects RAM calculations and queue assumptions. It is not a portable wire-format definition: raw structs contain implementation-specific alignment, padding and float representation. The current transports encode explicit JSON fields rather than sending raw structures.

Original source in this PDF: src/domain/reading.h (p. 46); src/domain/reading.c (p. 49); src/domain/types.h (p. 44)

CHAPTER 2

2.2 Health, quality and freshness answer different questions

Health is the main verdict. A quality flag adds detail, such as saturation or lack of compensation. Freshness asks whether time has made an otherwise valid reading too old.

Health	Practical meaning
STARTING	No usable result has arrived.
WARMING	The device is still settling or initialising its algorithm.
OK	The driver accepts the value in its declared units.
STALE	A formerly OK record has exceeded the consumer's age limit.
FAULT / TIMEOUT	A transaction, decoding, range or completion check failed.
UNPOWERED	Driver or acquisition power checks judge supply/signal unavailable.
UNCALIBRATED	A result exists, but its physical accuracy is not established.
DEPENDENCY_INVALID	A required compensation input is unusable.

Actionable means fresh and OK

`readingIsActionable()` checks only stored health. `readingIsActionableAt()` calls `readingHealthAt()` and also checks the age of an OK record. The latter is the right predicate for generic numeric alarm rules.

Example: an OK air sample ends at 1000 ms. Its stale limit is 1000 ms. At 2000 ms the sample is still accepted; at 2001 ms it is stale, because the code uses a strict greater-than test. Ordinary metrics use 3000 ms.

`readingHealthAt()` returns an effective health; it does not rewrite the stored record. It only ages OK records. A FAULT remains FAULT, and an UNCALIBRATED record remains UNCALIBRATED even when old. Safety therefore performs a separate age check for light, whose voltage readings are deliberately UNCALIBRATED.

The light exception is explicit

Generic rules reject UNCALIBRATED values. Lamp control deliberately accepts UNCALIBRATED millivolts because it uses relative voltage thresholds, not lux. This exception belongs to the lamp policy; it does not make uncalibrated pH or TDS eligible for arbitrary alarm thresholds.

Quality flags are independent bits: saturated, uncompensated, partial-filter and late. Each driver sets only the flags relevant to its path. An OK HC-SR04 distance still has the uncompensated flag because its sound speed is fixed.

Original source in this PDF: src/domain/reading.c (p. 49); src/domain/alarm.c (p. 291); src/domain/safety_core.c (p. 307); src/domain/lamp_control.c (p. 301)

CHAPTER 2

2.3 Two clocks, two jobs

`monoNowMs()` returns `esp_timer_get_time() / 1000` as an unsigned 64-bit value. This is elapsed time since boot, not calendar time. A network clock correction must not change a 30 s dwell or make a sample suddenly look younger.

FreeRTOS also has a tick counter. Tick counts control task blocking. Millisecond deadlines describe sensor work. `pdMS_TO_TICKS()` translates between the two. These clocks have different precision and must not be mixed without conversion.

```
// Source: src/hal/clock.c
MonoMs monoNowMs(void) {
  return (MonoMs)(esp_timer_get_time() / 1000);
}
```

Waiting until a deadline

Acquisition computes `due - now` only when `due > now`, casts that nonnegative interval to the required width and converts it to ticks. If due is already reached, it uses zero wait and does work immediately. A positive time that rounds to zero ticks also returns immediately; the loop must re-evaluate time rather than assuming it slept.

The one-sensor bring-up harness explicitly forces at least one tick for positive poll waits. Production uses timed notifications directly. Behaviour therefore depends on the actual framework tick configuration, especially for 1 ms ultrasonic polls. A requested millisecond is not a universal guarantee of a 1 ms sleep.

Microseconds for pulses, milliseconds for policy

HC-SR04 edge timestamps use the low 32 bits of the microsecond timer. Unsigned subtraction correctly obtains a short interval even across one wrap. The wrap period is 2^32 microseconds, about 71.6 min; this is safe for a pulse measured over tens of milliseconds, not an unlimited history duration.

The monotonic 64-bit millisecond clock is used for sample ages and schedules. A millisecond timestamp cannot resolve a 50 us task response. That is why the alarm-latency build records submit-to-action in microseconds and sample-to-action separately in milliseconds.

Original source in this PDF: src/hal/clock.c (p. 53); src/hal/pulse_capture.c (p. 271); src/services/acquisition.c (p. 96); src/services/safety.c (p. 115)

ORIGINAL SOURCE / CHAPTER 2 / 1 OF 2

src/domain/types.h

Original lines 1-29 | Code and comments unchanged; long lines wrap visually

monoReached (L14-14): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.

```
1  // Basic shared types. Domain layer: pure C, no Arduino/FreeRTOS includes.
2  //
3  // Enumerations are stored as uint8_t (the typedefs below) with the values in
4  // an anonymous enum. This keeps every record exactly the size it had with the
5  // C++ `enum class X : uint8_t` types (queue item sizes, RAM).
6  #pragma once
7  #include <stdbool.h>
8  #include <stdint.h>
9
10 // Monotonic milliseconds since boot. Used for every age and deadline.
11 // Never wall-clock/NTP time (§8). 64-bit so it does not wrap in practice.
12 typedef uint64_t MonoMs;
13
14 static inline bool monoReached(MonoMs now, MonoMs deadline) { return now >= deadline; }
15
16 typedef uint8_t SensorId;
17 enum {
18   SENSOR_BME280 = 0,
19   SENSOR_DS18B20,
20   SENSOR_LIGHT,   // DFR0026
21   SENSOR_HCSR04,
22   SENSOR_PH,      // pH-4502C
23   SENSOR_TDS,     // SEN0244
24   SENSOR_SGP30,   // TVOC / CO2eq (replaced the MQ-7, D18)
25   SENSOR_COUNT
26 };
27
28 // One entry per published metric. A sensor may produce several metrics.
29 typedef uint8_t MetricId;
```

Return to guided explanation, p. 41

ORIGINAL SOURCE / CHAPTER 2 / 2 OF 2

src/domain/types.h

Original lines 30-57 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
30 enum {
31   METRIC_AIR_TEMPERATURE = 0,  // BME280
32   METRIC_AIR_HUMIDITY,         // BME280
33   METRIC_AIR_PRESSURE,         // BME280
34   METRIC_WATER_TEMPERATURE,    // DS18B20
35   METRIC_LIGHT_LEVEL,          // DFR0026 (mV until calibrated, then lux)
36   METRIC_WATER_DISTANCE,       // HC-SR04
37   METRIC_PH,                   // pH-4502C
38   METRIC_TDS,                  // SEN0244
39   METRIC_TVOC,                 // SGP30, ppb
40   METRIC_CO2EQ,                // SGP30, ppm (estimate, not measured CO2)
41   METRIC_COUNT
42 };
43
44 typedef uint8_t Unit;
45 enum {
46   UNIT_NONE = 0,
47   UNIT_DEG_C,
48   UNIT_PERCENT_RH,
49   UNIT_PASCAL,
50   UNIT_MILLIVOLT,
51   UNIT_MILLIMETRE,
52   UNIT_PH,
53   UNIT_PPM,
54   UNIT_ADC_RAW,
55   UNIT_PPB,
56   UNIT_LUX   // light after an on-device fit (WP8 calibration). Appended: values above
 >
unchanged
57 };
```

Return to guided explanation, p. 41

ORIGINAL SOURCE / CHAPTER 2 / 1 OF 2

src/domain/reading.h

Original lines 1-39 | Code and comments unchanged; long lines wrap visually

Keep identity, units, health, age and numbers together. Current state is not just a float.

```
1  // Reading contract and latest-snapshot record (Firmware Architecture §8).
2  // FROZEN CONTRACT - every driver, service and transport depends on it.
3  // Domain layer: pure C, no Arduino/FreeRTOS includes.
4  #pragma once
5  #include <stdbool.h>
6  #include <stdint.h>
7  #include "domain/types.h"
8
9  typedef uint8_t Health;
10 enum {
11   HEALTH_STARTING = 0,        // no valid result yet
12   HEALTH_WARMING,             // heater/filter/first conversion settling
13   HEALTH_OK,                  // fresh, valid, declared units/calibration
14   HEALTH_STALE,               // last good value exceeded its age limit
15   HEALTH_FAULT,               // transport, CRC, range or conversion error
16   HEALTH_TIMEOUT,             // operation did not complete in its budget
17   HEALTH_UNPOWERED,           // rail off or power epoch changed during measurement
18   HEALTH_UNCALIBRATED,        // acquisition works, physical units not trustworthy
19   HEALTH_DEPENDENCY_INVALID   // e.g. TDS without a valid water temperature
20 };
21
22 // Additional quality bits (combine with Health).
23 enum QualityFlag {
24   kQualityNone = 0,
25   kQualitySaturated = 1u << 0,     // ADC at/near rail
26   kQualityUncompensated = 1u << 1, // compensation input missing
27   kQualityFilterPartial = 1u << 2, // filter window not full
28   kQualityLate = 1u << 3,          // completed after its deadline
29 };
30
31 // Fixed-size record, one per metric. Plain data: safe to copy through queues.
32 typedef struct Reading {
33   SensorId sensor;
34   MetricId metric;
35   Unit unit;
36   Health health;
37   uint16_t qualityFlags;      // QualityFlag bits
38   bool hasRaw;
39   float value;                // in `unit`; meaningless unless health == Ok/Uncalibrated
```

Return to guided explanation, p. 41

ORIGINAL SOURCE / CHAPTER 2 / 2 OF 2

src/domain/reading.h

Original lines 40-78 | Code and comments unchanged; long lines wrap visually

Keep identity, units, health, age and numbers together. Current state is not just a float.

```
  int32_t raw;                // ADC counts / mV / raw register value if hasRaw
  MonoMs intervalStartMs;     // acquisition interval
  MonoMs intervalEndMs;
  MonoMs lastGoodMs;          // last usable result (OK or UNCALIBRATED); 0 = never
  uint32_t sequence;          // per-metric sample counter
  uint32_t powerEpoch;        // rail epoch when measured (§12)
  uint16_t configRevision;
  uint16_t calibrationRevision;
} Reading;

// Latest-snapshot record published by the acquisition lease holder (§8, §9).
// A coherent copy, NOT a simultaneous measurement of all sensors.
typedef struct Snapshot {
  uint32_t frameSequence;
  MonoMs publishedMs;
  uint32_t powerEpoch;
  Reading readings[METRIC_COUNT];  // indexed by MetricId
} Snapshot;

// Initialise a record to STARTING with identity fields set.
void readingReset(Reading* r, SensorId sensor, MetricId metric, Unit unit);

// True if the stored health is OK. Does NOT check age: a consumer that acts on
// the value (Safety, control rules) must use readingIsActionableAt() instead.
bool readingIsActionable(const Reading* r);

// True only for fresh, valid, calibrated data (eligible for numeric rules):
// stored health OK and not older than staleLimitMs at `now` (§10).
bool readingIsActionableAt(const Reading* r, MonoMs now, uint32_t staleLimitMs);

// Returns Stale if an Ok reading is older than staleLimitMs at `now`,
// otherwise the reading's current health.
Health readingHealthAt(const Reading* r, MonoMs now, uint32_t staleLimitMs);

const char* healthName(Health h);
const char* metricName(MetricId m);
const char* unitName(Unit u);

void snapshotReset(Snapshot* s);
```

Return to guided explanation, p. 41

GUIDED SOURCE READING / CHAPTER 2

Initialize identity before a value can be trusted

The configuration tells the controller when to work. A Reading tells it what that work produced. readingReset establishes a record that is safe to inspect before the first measurement completes.

Original excerpt: src/domain/reading.c (p. 49) / lines 11-19. The complete unchanged file follows.

```
11  void readingReset(Reading* r, SensorId sensor, MetricId metric, Unit unit) {
12    memset(r, 0, sizeof(*r));
13    r->sensor = sensor;
14    r->metric = metric;
15    r->unit = unit;
16    r->health = HEALTH_STARTING;
17    r->qualityFlags = kQualityNone;
18    r->hasRaw = false;
19  }
```

memset clears the entire record first. This removes old sequence, timestamp, raw and value fields. The identity assignments come afterwards because clearing later would erase the sensor, metric and unit just written. sizeof(*r) means the size of the pointed-to Reading, not the size of its pointer.

HEALTH_STARTING is then assigned explicitly. A cleared float is numerically zero, but zero temperature or zero distance would be a plausible measurement. STARTING tells consumers that this zero was produced by initialization. hasRaw is false for the same reason: no ADC or register value has been observed yet.

The function operates through a pointer so it changes the caller's record. The caller supplies sensor, metric and unit because a shared initializer cannot infer which physical quantity this particular record will hold.

Below this function, readingHealthAt answers a different question: whether an existing OK observation has become too old. Reset establishes initial meaning; age checking updates a consumer's interpretation as time passes. That distinction prepares the queue walkthrough, where consumers receive copies rather than a permanently live value.

ORIGINAL SOURCE / CHAPTER 2 / 1 OF 3

src/domain/reading.c

Original lines 1-42 | Code and comments unchanged; long lines wrap visually

readingReset (L11-19): Clear the record and set identity plus STARTING. A zero float is not yet a measurement.
readingIsActionable (L21-21): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.

```
#include "domain/reading.h"

#include <string.h>

// The records must keep the layout they had in C++ (enum class : uint8_t):
// they are copied through queues and sized into RAM budgets.
_Static_assert(sizeof(Reading) == 56, "Reading layout changed");
// 528 -> 584 bytes on 22 Oct 2026: MQ-7 CO signal replaced by SGP30 TVOC + CO2eq.
_Static_assert(sizeof(Snapshot) == 584, "Snapshot layout changed");

void readingReset(Reading* r, SensorId sensor, MetricId metric, Unit unit) {
  memset(r, 0, sizeof(*r));
  r->sensor = sensor;
  r->metric = metric;
  r->unit = unit;
  r->health = HEALTH_STARTING;
  r->qualityFlags = kQualityNone;
  r->hasRaw = false;
}

bool readingIsActionable(const Reading* r) { return r->health == HEALTH_OK; }

Health readingHealthAt(const Reading* r, MonoMs now, uint32_t staleLimitMs) {
  if (r->health != HEALTH_OK) return r->health;
  if (now > r->intervalEndMs && (now - r->intervalEndMs) > staleLimitMs) {
    return HEALTH_STALE;
  }
  return HEALTH_OK;
}

bool readingIsActionableAt(const Reading* r, MonoMs now, uint32_t staleLimitMs) {
  return readingHealthAt(r, now, staleLimitMs) == HEALTH_OK;
}

const char* healthName(Health h) {
  switch (h) {
    case HEALTH_STARTING: return "STARTING";
    case HEALTH_WARMING: return "WARMING";
    case HEALTH_OK: return "OK";
    case HEALTH_STALE: return "STALE";
    case HEALTH_FAULT: return "FAULT";
    case HEALTH_TIMEOUT: return "TIMEOUT";
```

Return to guided explanation, p. 48

ORIGINAL SOURCE / CHAPTER 2 / 2 OF 3

src/domain/reading.c

Original lines 43-84 | Code and comments unchanged; long lines wrap visually

healthName (L35-48): Translate an identity/state into a human-readable label for diagnostics.
metricName (L50-65): Translate an identity/state into a human-readable label for diagnostics.

```
43      case HEALTH_UNPOWERED: return "UNPOWERED";
44      case HEALTH_UNCALIBRATED: return "UNCALIBRATED";
45      case HEALTH_DEPENDENCY_INVALID: return "DEPENDENCY_INVALID";
46    }
47    return "?";
48  }
49
50  const char* metricName(MetricId m) {
51    switch (m) {
52      case METRIC_AIR_TEMPERATURE: return "air_temperature";
53      case METRIC_AIR_HUMIDITY: return "air_humidity";
54      case METRIC_AIR_PRESSURE: return "air_pressure";
55      case METRIC_WATER_TEMPERATURE: return "water_temperature";
56      case METRIC_LIGHT_LEVEL: return "light_level";
57      case METRIC_WATER_DISTANCE: return "water_distance";
58      case METRIC_PH: return "ph";
59      case METRIC_TDS: return "tds";
60      case METRIC_TVOC: return "tvoc";
61      case METRIC_CO2EQ: return "co2eq";
62      case METRIC_COUNT: break;
63    }
64    return "?";
65  }
66
67  const char* unitName(Unit u) {
68    switch (u) {
69      case UNIT_NONE: return "";
70      case UNIT_DEG_C: return "degC";
71      case UNIT_PERCENT_RH: return "%RH";
72      case UNIT_PASCAL: return "Pa";
73      case UNIT_MILLIVOLT: return "mV";
74      case UNIT_MILLIMETRE: return "mm";
75      case UNIT_PH: return "pH";
76      case UNIT_PPM: return "ppm";
77      case UNIT_ADC_RAW: return "raw";
78      case UNIT_PPB: return "ppb";
79      case UNIT_LUX: return "lux";
80    }
81    return "?";
82  }
83
84  static SensorId metricSensor(MetricId m) {
```

Return to guided explanation, p. 48

ORIGINAL SOURCE / CHAPTER 2 / 3 OF 3

src/domain/reading.c

Original lines 85-125 | Code and comments unchanged; long lines wrap visually

metricSensor (L84-99): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.
metricUnit (L103-118): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.

```
85    switch (m) {
86      case METRIC_AIR_TEMPERATURE:
87      case METRIC_AIR_HUMIDITY:
88      case METRIC_AIR_PRESSURE: return SENSOR_BME280;
89      case METRIC_WATER_TEMPERATURE: return SENSOR_DS18B20;
90      case METRIC_LIGHT_LEVEL: return SENSOR_LIGHT;
91      case METRIC_WATER_DISTANCE: return SENSOR_HCSR04;
92      case METRIC_PH: return SENSOR_PH;
93      case METRIC_TDS: return SENSOR_TDS;
94      case METRIC_TVOC:
95      case METRIC_CO2EQ: return SENSOR_SGP30;
96      case METRIC_COUNT: break;
97    }
98    return SENSOR_COUNT;
99  }
100
101 // The unit each driver publishes when calibrated. pH and light may publish
102 // mV while uncalibrated; the Reading's own unit field says which.
103 static Unit metricUnit(MetricId m) {
104   switch (m) {
105     case METRIC_AIR_TEMPERATURE: return UNIT_DEG_C;
106     case METRIC_AIR_HUMIDITY: return UNIT_PERCENT_RH;
107     case METRIC_AIR_PRESSURE: return UNIT_PASCAL;
108     case METRIC_WATER_TEMPERATURE: return UNIT_DEG_C;
109     case METRIC_LIGHT_LEVEL: return UNIT_MILLIVOLT;
110     case METRIC_WATER_DISTANCE: return UNIT_MILLIMETRE;
111     case METRIC_PH: return UNIT_PH;
112     case METRIC_TDS: return UNIT_PPM;
113     case METRIC_TVOC: return UNIT_PPB;
114     case METRIC_CO2EQ: return UNIT_PPM;
115     case METRIC_COUNT: break;
116   }
117   return UNIT_NONE;
118 }
119
120 void snapshotReset(Snapshot* s) {
121   memset(s, 0, sizeof(*s));
122   for (uint8_t i = 0; i < METRIC_COUNT; ++i) {
123     readingReset(&s->readings[i], metricSensor((MetricId)i), (MetricId)i,
  > metricUnit((MetricId)i));
124   }
125 }
```

Return to guided explanation, p. 48

ORIGINAL SOURCE / CHAPTER 2 / 1 OF 1

src/hal/clock.h

Original lines 1-5 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // Monotonic clock (HAL). The only source of "now" for ages and deadlines.
2  #pragma once
3  #include "domain/types.h"
4
5  MonoMs monoNowMs(void);
```

Return to guided explanation, p. 41

ORIGINAL SOURCE / CHAPTER 2 / 1 OF 1

src/hal/clock.c

Original lines 1-5 | Code and comments unchanged; long lines wrap visually

monoNowMs (L5-5): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.

```
1  #include "hal/clock.h"
2
3  #include <esp_timer.h>
4
5  MonoMs monoNowMs(void) { return (MonoMs)(esp_timer_get_time() / 1000); }
```

Return to guided explanation, p. 41

ORIGINAL SOURCE / CHAPTER 2 / 1 OF 1

src/domain/sensor_set.h

Original lines 1-42 | Code and comments unchanged; long lines wrap visually

sensorMaskHas (L20-22): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.

```
1  // Fixed sensor set as the app sees it (FirmwareContract.md v2, "Sensors").
2  // Pure C. Work package WP8 (Stage 5).
3  //
4  // Sensors are fixed by the board (pins in board_profile.h). The app can only
5  // enable or disable them; the enabled set is a SensorMask persisted by
6  // Configuration. Keys are the protocol names, stable across firmware versions.
7  #pragma once
8  #include <stdbool.h>
9  #include <stddef.h>
10 #include <stdint.h>
11
12 #include "domain/reading.h"
13 #include "domain/types.h"
14
15 typedef uint8_t SensorMask;  // bit per SensorId
16 #define SENSOR_MASK_ALL ((SensorMask)((1u << SENSOR_COUNT) - 1u))
17 // Sensors on the external, switched 5 V rail (§14), gated on the rail epoch.
18 #define SENSOR_MASK_RAIL_5V ((SensorMask)((1u << SENSOR_HCSR04) | (1u << SENSOR_PH) | (1u <<
 > SENSOR_TDS)))
19
20 static inline bool sensorMaskHas(SensorMask mask, SensorId s) {
21   return s < SENSOR_COUNT && (mask & (1u << s)) != 0;
22 }
23
24 // Protocol key ("bme280", "ds18b20", "light", "hcsr04", "ph", "tds", "sgp30").
25 const char* sensorKey(SensorId s);
26 const char* sensorLabel(SensorId s);
27
28 // What disabling this sensor does to the others (app warning), "" if nothing.
29 const char* sensorDependsNote(SensorId s);
30
31 // SENSOR_COUNT if the key (len bytes, not NUL-terminated) is unknown.
32 SensorId sensorFromKey(const char* key, size_t len);
33
34 // Sensor that produces a metric; SENSOR_COUNT for an invalid metric.
35 SensorId sensorOfMetric(MetricId m);
36
37 // Consumer-side rail check (§12), on a snapshot copy: acquisition checks the
38 // epoch when it accepts a result, but the rail can fail after that. Every 5 V
39 // reading measured in another epoch than railEpoch, or any 5 V reading while
40 // the rail is not valid now, becomes UNPOWERED (never shown, captured or
41 // acted on). STARTING readings are left as they are.
42 void snapshotGateRail(Snapshot* s, bool railValid, uint32_t railEpoch);
```

Return to guided explanation, p. 41

ORIGINAL SOURCE / CHAPTER 2 / 1 OF 2

src/domain/sensor_set.c

Original lines 1-31 | Code and comments unchanged; long lines wrap visually

sensorKey (L23-23): Map sensor keys and metrics, record dependency notes and invalidate copied 5V readings against current rail history.
sensorLabel (L25-25): Map sensor keys and metrics, record dependency notes and invalidate copied 5V readings against current rail history.

```
#include "domain/sensor_set.h"

#include <string.h>

typedef struct SensorInfo {
  const char* key;
  const char* label;
  const char* dependsNote;
} SensorInfo;

// Indexed by SensorId.
static const SensorInfo kSensors[SENSOR_COUNT] = {
    {"bme280", "BME280 air temperature, humidity, pressure",
     "SGP30 loses humidity compensation"},
    {"ds18b20", "DS18B20 water temperature", "TDS loses temperature compensation"},
    {"light", "DFR0026 light level", "Automatic lamp stays OFF"},
    {"hcsr04", "HC-SR04 water level", ""},
    {"ph", "pH-4502C solution pH", ""},
    {"tds", "SEN0244 TDS", ""},
    {"sgp30", "SGP30 TVOC / CO2eq", ""},
};

const char* sensorKey(SensorId s) { return s < SENSOR_COUNT ? kSensors[s].key : "?"; }

const char* sensorLabel(SensorId s) { return s < SENSOR_COUNT ? kSensors[s].label : "?"; }

const char* sensorDependsNote(SensorId s) {
  return s < SENSOR_COUNT ? kSensors[s].dependsNote : "";
}

SensorId sensorFromKey(const char* key, size_t len) {
```

Return to guided explanation, p. 41

ORIGINAL SOURCE / CHAPTER 2 / 2 OF 2

src/domain/sensor_set.c

Original lines 32-63 | Code and comments unchanged; long lines wrap visually

sensorFromKey (L31-37): Map sensor keys and metrics, record dependency notes and invalidate copied 5V readings against current rail history.
snapshotGateRail (L39-46): On a consumer copy, mark 5V metrics UNPOWERED if supply is invalid now or their stored epoch is older.

```
32    for (uint8_t i = 0; i < SENSOR_COUNT; ++i) {
33      const char* k = kSensors[i].key;
34      if (strlen(k) == len && memcmp(k, key, len) == 0) return (SensorId)i;
35    }
36    return SENSOR_COUNT;
37  }
38
39  void snapshotGateRail(Snapshot* s, bool railValid, uint32_t railEpoch) {
40    for (uint8_t m = 0; m < METRIC_COUNT; ++m) {
41      Reading* r = &s->readings[m];
42      if (!sensorMaskHas(SENSOR_MASK_RAIL_5V, sensorOfMetric((MetricId)m))) continue;
43      if (r->health == HEALTH_STARTING || r->health == HEALTH_UNPOWERED) continue;
44      if (!railValid || r->powerEpoch != railEpoch) r->health = HEALTH_UNPOWERED;
45    }
46  }
47
48  SensorId sensorOfMetric(MetricId m) {
49    switch (m) {
50      case METRIC_AIR_TEMPERATURE:
51      case METRIC_AIR_HUMIDITY:
52      case METRIC_AIR_PRESSURE: return SENSOR_BME280;
53      case METRIC_WATER_TEMPERATURE: return SENSOR_DS18B20;
54      case METRIC_LIGHT_LEVEL: return SENSOR_LIGHT;
55      case METRIC_WATER_DISTANCE: return SENSOR_HCSR04;
56      case METRIC_PH: return SENSOR_PH;
57      case METRIC_TDS: return SENSOR_TDS;
58      case METRIC_TVOC:
59      case METRIC_CO2EQ: return SENSOR_SGP30;
60      case METRIC_COUNT: break;
61    }
62    return SENSOR_COUNT;
63  }
```

Return to guided explanation, p. 41

PART I / FOUNDATIONS

CHAPTER 03

RTOS: Tasks, Startup and Acquisition

FreeRTOS lets a task wait while another task runs. The architecture still needs rules for who may touch a bus, which records cross task boundaries and what survives a missed wake. This chapter derives those rules from startup and the running service loops.

In this chapter

1. Follow task creation, blocking, priority and core affinity.
2. Trace a measurement through the exclusive acquisition worker.
3. Connect snapshots, alarms, logging and supervision to their RTOS mechanisms.

Chapter contents

3.1 What the RTOS contributes 58
3.2 A lease is permission to touch the shared drivers 59
3.3 A task is a function plus saved execution state 60
3.4 Priorities and affinity on two cores 61
3.5 RTOS types: handles, control blocks and ticks 62
3.6 Task creation and static memory 63
3.7 Delays, notifications and queues 64
3.8 Startup establishes dependencies and ownership 65
3.9 One worker owns all seven drivers 66
3.10 Absolute schedules avoid slow drift 67
3.11 How a pass starts and advances measurements 68
3.12 Why blocking work and echo capture cannot overlap freely 69
3.13 Results, frame guards and failure recovery 70
3.14 Latest state is not a history queue 71
3.15 Alarm delivery: store, queue, then notify 72
3.16 Safety is a task wrapper around a pure decision core 73
3.17 Cross-core access needs more than volatile 74
3.18 Choose synchronization by the contract it preserves 75
3.19 Logging is a bounded side channel 76
3.20 Progress supervision checks completed work 77
3.21 Memory supervision makes assumptions visible 78
3.22 Watchdogs recover from work that stops returning 79

Guided source readings begin on page 81.

Complete original listings begin on page 80. Shared files are linked in the explanations; Appendix A contains the full filename index.

CHAPTER 3

3.1 What the RTOS contributes

FreeRTOS chooses which task runs. It does not know that a Reading is a temperature or that the relay controls a lamp. The application gives those operations meaning.

Step	What the actual firmware does
1. Make startup safe	main.c writes inactive outputs before creating workers.
2. Create workers	Static task creation supplies a stack, control block, priority and core.
3. Wait for a deadline or event	Acquisition waits until a driver/job time; Safety waits for an alarm or 100 ms.
4. Advance one step	start sends a command; poll checks or completes an operation.
5. Pass copies	Snapshot and event records move through queues; ownership stays clear.
6. Report completed passes	Progress timestamps and watchdog resets follow the task's pass.

A wait does not erase local state

When acquisition blocks, its local variables and call position remain in its task context. The sensor can continue converting while the CPU runs another task. On wake, the code reads the clock again and checks what is actually due.

Follow the implemented wait

Acquisition marks progress and feeds its watchdog after each runPass call. These operations report that the worker completed a pass; they do not assert that a particular sensor produced a good reading. The worker converts a positive deadline difference directly to ticks. If the conversion rounds to zero, it immediately rechecks the clock and due work. The later worker-loop reading connects these statements to blocking and progress reporting.

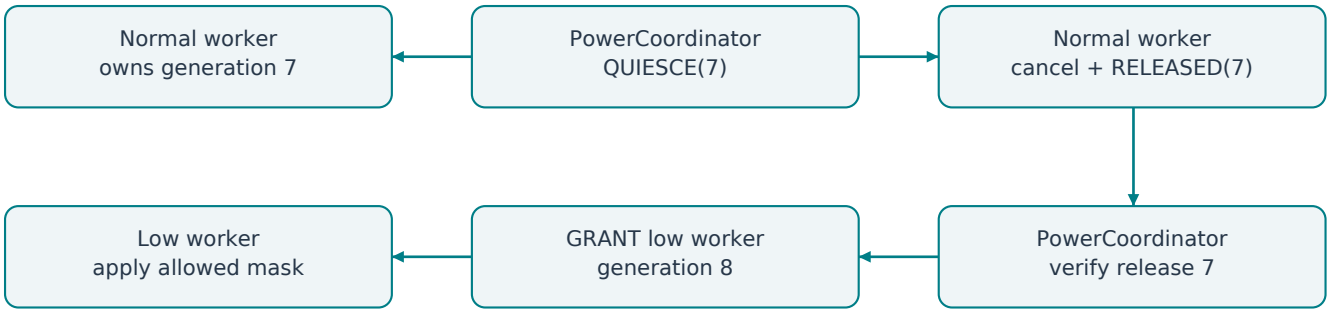
Where to look first

Read main.c, then acqTask and safetyTask. Next read runPass, tryStart and tryPoll to see how the lease holder interleaves devices. The remaining RTOS service files explain state copying, alarm delivery, logging and supervision. Pure alarm/lamp decisions are explained in their own later chapter.

CHAPTER 3

3.2 A lease is permission to touch the shared drivers

Imagine one key for a shared workshop. Both workers exist, but only the key holder enters. The lease is that permission; it is not a second set of sensors or a queue of measurements.



Until release 7 arrives: wait or report DEGRADED; never grant a rival.

Why a generation number is needed

Suppose normal worker owns generation 7. PowerCoordinator asks generation 7 to quiesce. Only RELEASED(7) satisfies that handover. An older acknowledgement cannot authorize a new owner. After release, the next grant increments the generation. Shared ownership fields change under s_leaseMux.

What release means

The worker cancels in-flight operations and releases any ping gate. When the 5V rail is down, preparePowerOff keeps HC-SR04 TRIG LOW and invalidates affected initialization. The worker unsubscribes from the task watchdog before blocking without ownership. It must not access a driver again until granted another lease.

What acquire means

The new holder joins the watchdog, restarts the job grid and applies configuration limited by its allowed sensor mask. A sensor requested by configuration but excluded by power publishes UNPOWERED. Reenabled drivers initialize again and show WARMING until a result is available.

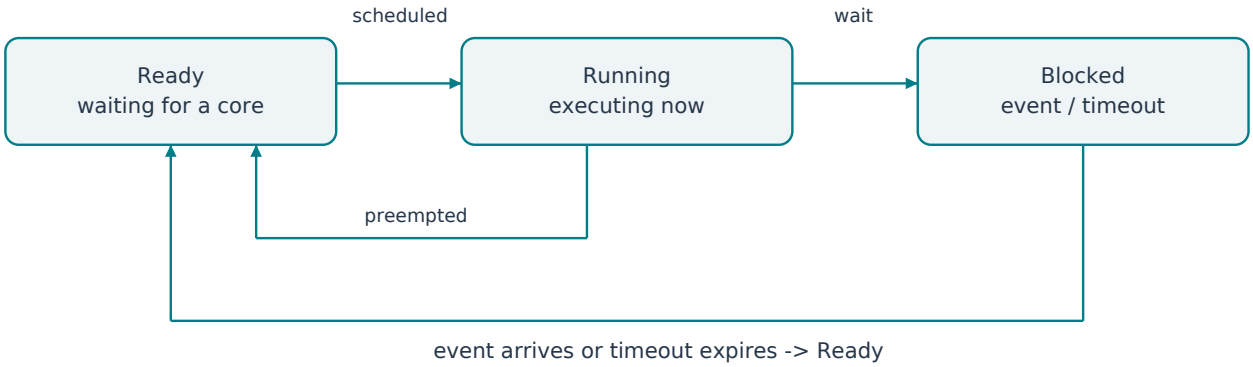
The state machine reports DEGRADED after a missing acknowledgement but keeps waiting. Timeout is a diagnostic outcome, not permission to create two owners. That choice protects buses and shared state even when it costs continued measurement.

Original source in this PDF: src/services/acquisition.c (p. 96); src/domain/power_mode.c (p. 328)

CHAPTER 3

3.3 A task is a function plus saved execution state

An ordinary call runs and returns to its caller. A task entry function normally runs an endless loop. FreeRTOS gives each task its own stack and saves enough CPU state to pause it and resume at the interrupted statement. Preemption does not restart the function.



Running, Ready and Blocked

Running means a core is currently executing the task. Ready means it could run but a scheduler choice prevents it. Blocked means it is waiting for an event or timeout and is not competing for CPU time. Suspended is a separate state reached by explicit suspension; this application mainly uses blocking.

When acquisition waits for the next scheduled poll, its stack and driver state remain intact. Logger or idle work can then run on core 1. When the timeout expires, acquisition becomes Ready. Its priority decides when it becomes Running.

A busy loop that repeatedly asks whether a sensor is ready stays Ready/Running. It wastes core time and can starve lower-priority tasks. `taskYIELD()` requests another scheduling choice but does not make a higher-priority task Blocked. This source avoids that pattern by splitting device work into start and poll stages.

A time delay is not reserved CPU time

If a task requests a 10 ms delay, the timeout makes it eligible again after the required tick interval. It may resume later because more urgent eligible work is executing. A sensor deadline therefore requires bounded calls and sensible priority placement as well as a delay API.

The infinite loops must not return. Arduino already starts FreeRTOS before calling application setup; this program must not call `vTaskStartScheduler()` itself. [R2, R3]

Original source in this PDF: src/services/acquisition.c (p. 96); src/services/safety.c (p. 115); src/diagnostics/logger.c (p. 125)

CHAPTER 3

3.4 Priorities and affinity on two cores

A Ready task can execute only on an allowed core. On that core, higher-priority Ready work normally runs before lower-priority work. Blocking lets another task use the processor. Response time also depends on core affinity, critical sections, SDK work and the duration of the operation already running. [R2]

Application task	Core / priority	Stack bytes
Safety	1 / 6	3072
PowerCoord	0 / 5	4096
AcqNormal	1 / 4	4096
AcqLowPower	0 / 4	4096
WifiMqtt	0 / 2	6144
BleTransport	0 / 2	6144
Configuration	0 / 2	4096
Supervisor	0 / 1	3072
SerialLogger	1 / 1	2048

Why Safety moved to core1

The repository records that Wi-Fi startup delayed Safety when it shared core0 with radio work. Safety runs on core1, above AcqNormal. An alarm notification can therefore preempt acquisition after the driver call releases any critical section. Radio/SDK tasks, idle tasks, timers and Arduino loopTask also exist; this table lists application tasks only.

During normal operation AcqLowPower waits without the lease. In low power, AcqNormal waits and AcqLowPower runs on core0. Safety remains on core1 in both modes. Core1 is therefore not idle in low power.

Read the actual creation path

cfgTaskSpec supplies names, placement and budgets. main calls the initialisers that create these tasks. A failed initialisation can leave fewer tasks running. The returned task handle records whether creation succeeded.

Equal-priority tasks may share execution under the port's scheduling rules. Configuration and the two transports are designed to block between steps. Avoid assuming equal priority gives deterministic ordering.

Original source in this PDF: include/app_config.h (p. 36); src/main.c (p. 82); src/services/acquisition.c (p. 96)

CHAPTER 3

3.5 RTOS types: handles, control blocks and ticks

The RTOS headers introduce type names that express kernel roles. They are aliases and structures supplied by the port, not extra C keywords. Opaque means that application code passes the handle to an API instead of interpreting private kernel fields. Their names distinguish references to existing objects from storage that the application must keep alive.

Type	Role in this firmware
`TaskHandle_t`	Opaque reference used to notify or inspect an existing task.
`QueueHandle_t`	Opaque reference used to access an existing queue.
`SemaphoreHandle_t`	Reference used by the semaphore or mutex API.
`StaticTask_t`, `StaticQueue_t`	Caller-supplied storage for kernel control information.
`StackType_t`	Element type used by the port's task-stack storage.
`TickType_t`	Type for kernel tick times and blocking intervals.
`BaseType_t`	Kernel integer type used for many API results and ISR wake flags.

A handle is not the object's complete storage

```
// Source: src/services/snapshot_store.c
static StaticQueue_t s_queueControl;
static uint8_t s_queueStorage[sizeof(Snapshot)];
static QueueHandle_t s_queue = NULL;
```

The control object stores kernel bookkeeping. The byte array stores the queued Snapshot. The handle identifies the queue created from those objects. Copying the handle gives another reference to the same queue; it does not create another mailbox or copy its messages.

NULL means that no queue has been assigned to this handle. It does not mean that an existing queue is empty. A receive or peek result reports whether a message was available. These static backing objects remain alive for every later queue operation.

Read units and return conventions from the API

pdMS_TO_TICKS converts milliseconds to the configured tick scale; it can round a short positive interval to zero. MonoMs remains the application's separate millisecond type. Likewise, StackType_t does not define the units of a task-creation argument: this ESP-IDF port specifies stack depth in bytes.

A BaseType_t result is interpreted using that function's contract, such as comparison with pdTRUE for a successful queue peek. The ISR woken flag instead asks whether an unblocked higher-priority task needs a scheduling decision at interrupt exit. The same integer type can carry different meanings.

Original source in this PDF: src/services/snapshot_store.c (p. 88); src/services/acquisition.c (p. 96); src/services/power_coordinator.c (p. 334); src/services/flash_gate.c (p. 369)

CHAPTER 3

3.6 Task creation and static memory

`xTaskCreateStaticPinnedToCore()` takes an entry function, name, stack depth, argument, priority, stack storage, control-block storage and core. The returned handle identifies the created task; NULL means failure. Static storage must outlive the task.

```
// Source: src/services/acquisition.c, createWorker()
TaskHandle_t t = xTaskCreateStaticPinnedToCore(
    acqTask, spec->name, stackBytes, (void*)(uintptr_t)w,
    spec->priority, stack, tcb, spec->core);
```

The argument identifies which acquisition worker is running the same entry function. The integer passes through uintptr_t, an integer type suitable for pointer conversion, and is decoded at task entry. The two calls supply different stacks and control blocks.

Bytes in this ESP-IDF port

Here stack depth is specified in bytes. The nine declared application stacks total 36 KiB (36,864 bytes). This is only stack payload: control blocks, queues, drivers, protocol buffers, SDK tasks and heap allocations add more. Do not copy this byte-count pattern to a FreeRTOS port whose stack depth is measured in words. [R2, R3]

Large task-private objects are static to protect small stacks. Static means persistent storage, not automatic thread safety. Each buffer still needs one owner or a defined synchronization method.

A task can run immediately

Dependencies must exist before task creation. Configuration publishes defaults or loaded settings before Safety and acquisition start. Safety channels exist before acquisition can submit an alarm. Both acquisition workers initially wait for a grant, so task creation alone does not authorize sensor access.

The framework and radio libraries can allocate dynamically. PSRAM does not remove internal-heap requirements. Supervisor measures unused stack minima and internal heap separately.

Original source in this PDF: src/services/acquisition.c (p. 96); src/services/configuration.c (p. 374); src/services/safety.c (p. 115); include/app_config.h (p. 36)

CHAPTER 3

3.7 Delays, notifications and queues

The application uses different waiting mechanisms because different information must survive the wait.

Mechanism	What it conveys here
`vTaskDelayUntil()`	A periodic release tied to an absolute tick grid.
`ulTaskNotifyTake()`	Wake on a signal or the next scheduled deadline.
`xQueueReceive()`	Wait for a copied item that must be consumed.
`xQueuePeek()`	Read a copied current state without consuming it.
`xQueueOverwrite()`	Replace the current value of a length-one mailbox.

A notification is a wake signal

`safetySubmitEvent()` stores alarm information first, then calls `xTaskNotifyGive()`. Safety waits with `ulTaskNotifyTake(pdTRUE, timeout)`: a notification can wake it before the 100 ms periodic health check. `pdTRUE` clears the accumulated notification count on a successful take.

Safety must not interpret that count as the number of alarms. Several submissions may merge into one wake, and a timeout can wake it with no alarm. The queue and sticky store hold the meaning; the notification only asks the receiver to inspect them.

Periodic wait versus relative wait

Supervisor uses `vTaskDelayUntil(&last, periodTicks)`. Its next intended release is based on the previous release grid rather than on the time it finishes checking. A relative `vTaskDelay(periodTicks)` would add work duration to each cycle.

`vTaskDelayUntil()` is not the same no-catch-up algorithm used by acquisition's domain scheduler. If Supervisor overruns a period, later calls may return without blocking until the release grid catches up. Acquisition explicitly counts and skips missed jobs instead.

Queue timeout zero has a purpose

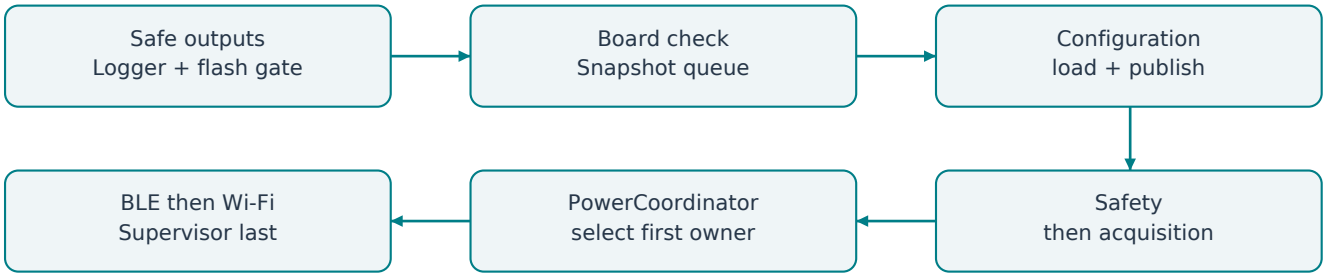
`xQueueSend(queue, &record, 0)` means try immediately and return if full. Alarm and log producers cannot wait indefinitely for slow consumers. What happens on failure depends on the channel: log records are dropped and counted; alarm state has separate retention.

Original source in this PDF: src/services/safety.c (p. 115); src/services/snapshot_store.c (p. 88); src/diagnostics/logger.c (p. 125); src/services/supervisor.c (p. 132)

CHAPTER 3

3.8 Startup establishes dependencies and ownership

Production setup creates dependencies in order. It writes safe outputs before starting workers, then returns. loop blocks indefinitely; it is not the sensor loop.



Both acquisition workers exist; a grant permits measurement.

Creation order establishes dependencies. Tasks can run immediately.

Read startup in groups

First, gpioApplySafeDefaults commands the active-high relay and ultrasonic TRIG LOW. Logger and the flash-write gate are initialized. The board profile reports its pin/power-sense checks, and the length-one Snapshot queue is created.

Configuration then loads NVS and publishes settings before returning. Safety starts with those settings and commands OFF. Acquisition constructs drivers and creates both workers. AcqNormal begins the HALs on core1, but neither worker measures until PowerCoordinator grants ownership.

PowerCoordinator selects the first worker from the sense level. BLE and Wi-Fi start afterwards. Supervisor starts last. Workers may already be running while later initialisers execute, so this order establishes dependencies rather than one simultaneous start.

Startup failures

setup logs failures and continues. Failure to create Safety leaves the relay at its startup OFF level. If PowerCoordinator task creation fails, its fallback restores the current rail epoch and grants the normal worker all sensors. That fallback preserves sensing but has no automatic power transitions.

If the power task exists but its sense ISR cannot be armed, it polls GPIO8. Polling cannot observe a short pulse between ticks. A failed board gate is reported but is not a strict software startup interlock.

The board uses a normally-open lamp contact and an active-high transistor stage. Software assumes that wiring; relayIsOn reports a command, not measured contact feedback.

Original source in this PDF: src/main.c (p. 82); src/hal/gpio_safe.c (p. 312); src/services/configuration.c (p. 374); src/services/power_coordinator.c (p. 334)

CHAPTER 3

3.9 One worker owns all seven drivers

Acquisition has two task stacks and one shared set of drivers, buses, scheduling records and working Snapshot. The lease names the worker allowed to touch that state. During normal power AcqNormal owns it; during low power AcqLowPower owns it. The inactive worker blocks.

DriverRun keeps each sensor's place

The generic driver pointer selects the device. initialised records successful setup; pending means a measurement is unfinished; wantStart requests this period's work. nextPollMs gives the next scheduled call, retryAtMs enforces backoff, and lastFailure suppresses repeated logs. disabled excludes a sensor from scheduling. startEpoch records the supply history for a 5V window.

initBudgetMs and stepBudgetMs help admit blocking calls. They do not install an interrupt that can stop a hung function. The task watchdog is a later recovery layer.

Ownership changes at a pass boundary

PowerCoordinator requests quiescence for the current generation. The holder wakes, cancels pending work, releases an ultrasonic flash gate if held, makes dead-rail interfaces safe, leaves the task watchdog and publishes RELEASED. Only then is another worker granted a new generation.

The new holder joins the watchdog, restarts the two absolute schedules from now and forces a configuration application. It intersects the saved sensor-enabled mask with the lease's allowed mask. No task outside the holder calls a sensor driver to capture or calibrate it.

The same measurement engine

Both workers use the 1000 ms ordinary frame and 250 ms BME280 grid. Low power retains BME280, DS18B20, light and SGP30. HC-SR04, pH and TDS are outside its lease. A sensor can also be disabled in saved configuration independently of its power allowance.

The lease lock orders ownership fields across cores. A grant is refused while an owner remains. A missing release acknowledgement causes DEGRADED rather than authorizing simultaneous bus access.

Original source in this PDF: src/services/acquisition.c (p. 96); src/services/acquisition.h (p. 89); src/domain/power_mode.h (p. 325)

CHAPTER 3

3.10 Absolute schedules avoid slow drift

`scheduleJobInit()` stores a period, budget and first deadline. A zero period is replaced by 1 ms. `scheduleJobDue()` tests whether now has reached the deadline. `scheduleEarliestDue()` returns the earliest deadline in a nonempty table.

```
// Source: src/domain/schedule.c
const MonoMs missed = lateness / job->periodMs;
job->nextDueMs += (missed + 1) * job->periodMs;
```

The deadline moves along its existing grid. It does not become `now + period`. This preserves the intended release pattern after a delayed run.

Worked timing example

A BME280 job is due at 250 ms and has a 250 ms period. The worker reaches it at 780 ms. Lateness is 530 ms, so `missed = 530 / 250 = 2`. The job runs once, counts two skipped releases and sets its next due time to 1000 ms.

The missed releases are not replayed as a burst. Replaying would spend time reading several nearly simultaneous samples while further delaying current work. The current result describes the present environment; skipped counters preserve the scheduling history.

The scheduler records every run, maximum and latest lateness, late count and skipped count. Lateness strictly above 5 ms increments `late`; smaller jitter still contributes to the maximum. A measured maximum of 4 ms therefore does not imply zero timing variation.

Admission is between driver calls

`scheduleFitsBefore(now, budget, deadline)` checks `now + budget <= deadline`. It can defer a long ordinary step that would cross a BME280 release. It cannot preempt an already-running driver function. Admission protects the next release only when the declared budget covers the call duration.

The job's `budgetMs` field is stored, but the active service separately uses DriverRun budgets in `blockingAllowed()`. A named budget is not automatically enforced everywhere that struct appears.

Original source in this PDF: src/domain/schedule.h (p. 84); src/domain/schedule.c (p. 85); src/services/acquisition.c (p. 96)

CHAPTER 3

3.11 How a pass starts and advances measurements

`runPass()` first handles frame and BME280 boundaries. It then performs at most eight iterations that advance ready work. Each iteration refreshes current time, tries the fast BME280 start, polls due drivers, and attempts ordinary starts in a fixed order.

Order at an ordinary frame	Reason
DS18B20	Starts the long internal conversion before other frame work.
SGP30	Maintains the approximately one-second gas algorithm cadence.
HC-SR04	Starts after 1-Wire calls that would disturb echo timestamps.
TDS, pH, light	Starts analog windows that can overlap in elapsed time.

tryStart()

If init backoff has not elapsed, the service skips this period. If a blocking call is not currently allowed, the job remains wanted and later jobs in the fixed order wait too. Otherwise init runs. A failed init is logged on change, sets a 5000 ms retry deadline and stops that period's start attempt.

After successful init, the service refreshes time because init may take tens of milliseconds. `prepareInputs()` copies current humidity information into SGP30 or current water temperature into TDS. Then start is called. DRIVER_OK marks the measurement pending and requests an immediate first poll. The first poll commonly just returns a future time.

tryPoll()

A poll runs only for a pending driver whose next time is reached and whose blocking budget is allowed. DRIVER_PENDING updates nextPollMs; a returned time at or before now is moved to now + 1 ms. Any completed result goes through `finishRun()`.

`progress |= ...` deliberately evaluates the right-hand side even if progress is already true. A short-circuit logical OR in that position could accidentally stop later driver calls from executing. After an iteration with no progress, the pass ends and the task computes a new wait.

At most eight iterations bounds the number of service advances per pass. It does not bound a broken function that never returns, nor does it make every iteration equal in duration.

Original source in this PDF: src/services/acquisition.c (p. 96)

CHAPTER 3

3.12 Why blocking work and echo capture cannot overlap freely

The worker labels a step blocking when its declared budget exceeds 2 ms. `blockingAllowed()` rejects such a step while BME280 has a pending conversion, while HC-SR04 has a pending echo, or when the step would cross the next BME280 slot. BME280 itself gets a special path after the echo check.

DS18B20 start and completed read use bit-banged 1-Wire traffic. Some bit slots disable local interrupts. If an echo edge occurs during that window, the ISR timestamps it late. A 70 us edge error corresponds to about 12 mm at 343 m/s, so protecting the timing window preserves distance accuracy even when average CPU load is low.

`pingAllowed()` also rejects an ultrasonic trigger while a DS18B20 start is wanted, or while a DS18B20 poll is due within the 30 ms echo timeout plus 2 ms margin. This avoids starting an echo just before a scheduled 1-Wire read.

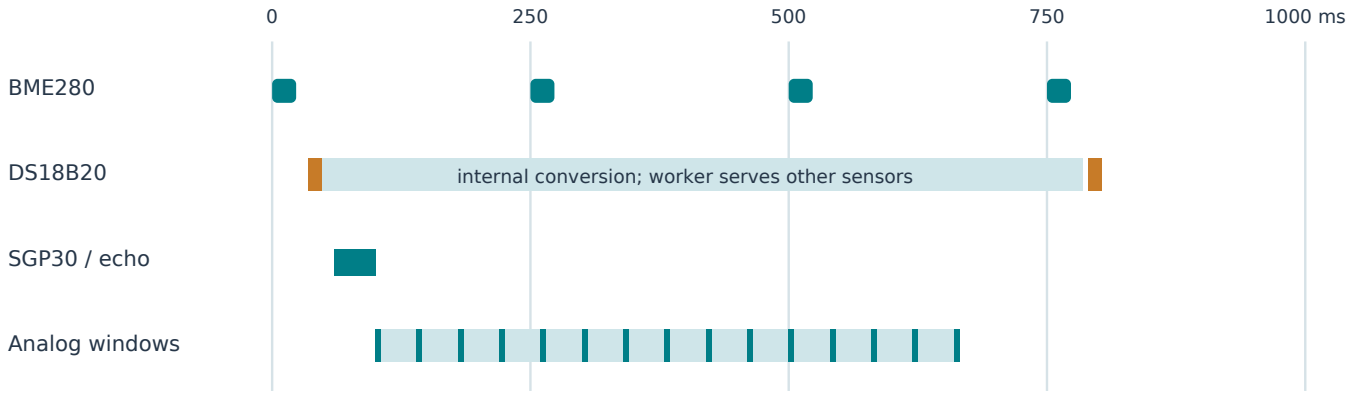


Illustration of ordering and overlap, not a measured trace.

This is an illustrative steady-state sequence, not a measured trace. BME280 starts around each quarter-second boundary and is read after about 10 ms. DS18B20 starts after the first fast climate result, converts independently, and is read at least 750 ms after its recorded start, subject to admission rules. Analog samples occupy brief calls across their longer windows.

A dependency can belong to the previous frame

TDS copies water temperature at its start. Its 560 ms nominal window usually finishes before the current DS18B20 conversion. The previous frame's accepted temperature can still be fresh under the 3000 ms limit. The first frame has no such temperature, so DEPENDENCY_INVALID is expected.

The SGP30 humidity setter similarly receives whatever accepted BME280 data is in the working snapshot when SGP30 starts. It copies derived compensation state; it does not arrange simultaneous sampling of the two chips.

Original source in this PDF: src/services/acquisition.c (p. 96); src/drivers/ds18b20.c (p. 206); src/drivers/tds.c (p. 235)

CHAPTER 3

3.13 Results, frame guards and failure recovery

finishRun clears pending state, releases the HC-SR04 flash gate, records changed failure/recovery state and stores results. DRIVER_READY still requires checking Reading.health.

A new observation gets a new sequence

storeResults asks for up to four Readings. For a 5V sensor, it rejects the value as UNPOWERED if the rail is invalid or the epoch differs from the one saved at start. It then assigns sequence = previous stored sequence + 1, stamps the applied configuration revision and current epoch, evaluates matching rules and publishes the Snapshot.

Failures also get a new sequence. Otherwise a FAULT after an OK sample could reuse the old sequence and be mistaken for a retransmitted observation. Sequence identifies an observation. Health states whether that observation succeeded.

A guard is a failed result

At the next frame or BME280 boundary, pending work is cancelled. guardTimeout publishes TIMEOUT with the late flag and ends the interval now. If rail history changed, UNPOWERED takes precedence. Some other fields can still contain an older driver record; the failed health prevents treating the number as a completed measurement.

Settings are applied between frames

applyConfiguration copies the published ConfigBlob, applies new calibrations and updates enabled state. Disabled runs are cancelled; newly enabled runs reinitialize and publish WARMING until a result arrives. Acquisition acknowledges the revision after applying it. This preserves a single driver owner while commands arrive elsewhere.

Recovery has several levels

An init failure backs off 5 s. A not-present completion requests reinitialization on a later due time. Init failure itself does not call storeResults, so a prior value may age instead. A READY result with bad health does not increment the same counter as a driver-status failure. Health describes the observation, age determines whether a previously accepted observation is still fresh, and counters/logs explain the service path that produced it.

Original source in this PDF: src/services/acquisition.c (p. 96); src/services/configuration.c (p. 374); src/services/rail_state.c (p. 324)

CHAPTER 3

3.14 Latest state is not a history queue

`snapshot_store.c` creates a length-one queue of Snapshot records. Acquisition uses `xQueueOverwrite()`. Safety and Supervisor use zero-wait `xQueuePeek()`. They obtain complete copied snapshots without taking the producer's working object away from it.

```
// Source: src/services/snapshot_store.c
s_queue = xQueueCreateStatic(1, sizeof(Snapshot),
                             s_queueStorage, &s_queueControl);
```

If acquisition publishes A, B and C before Supervisor reads, Supervisor receives C. Skipping A and B is intentional: the channel means current state. A 584-byte payload is copied, so the consumer cannot see half of one application write and half of the next through this API.

Coherent does not mean simultaneous

The Snapshot contains the latest result for each metric. Air temperature may be a few milliseconds old while TDS spans more than half a second and water temperature was obtained in a different part of the frame. `publishedMs` describes publication time, not a common sampling instant.

`startFrame()` republishes the current working snapshot and increments frameSequence before all new frame results exist. Individual Reading timestamps and sequences remain the way to tell whether a particular measurement changed.

Why alarms have another channel

A brief threshold trip could appear and disappear between two snapshot reads. A latest-state mailbox cannot preserve that event. Acquisition therefore evaluates enabled rules as soon as a result is stored, sends an AlarmEvent and wakes Safety. The detail queue and sticky store retain information independently of the current snapshot.

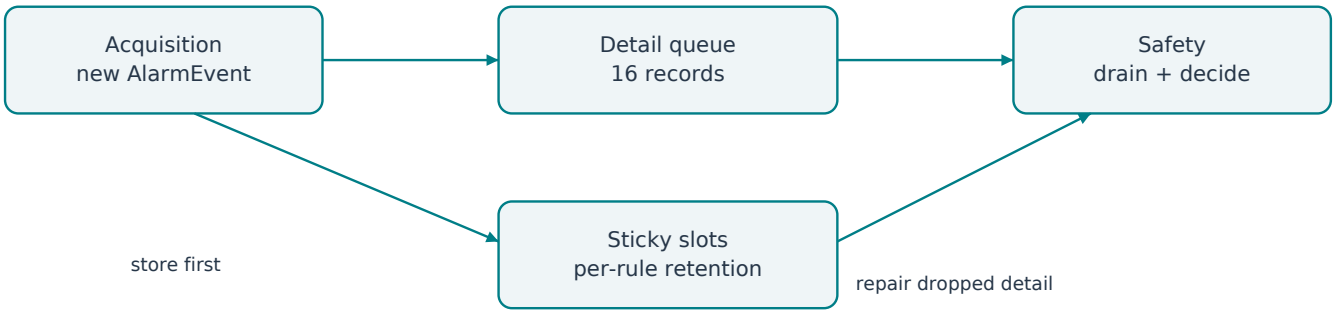
This distinction recurs throughout the architecture: a mailbox answers what is true now, an event channel records that something happened, and a notification asks a task to look. Confusing these meanings produces missed transitions even when every API call is thread-safe.

Original source in this PDF: src/services/snapshot_store.c (p. 88); src/services/acquisition.c (p. 96); src/services/safety.c (p. 115)

CHAPTER 3

3.15 Alarm delivery: store, queue, then notify

Safety receives ordinary events through a 16-entry detail queue. Per-rule sticky slots retain a worst TRIP and latest CLEAR if the zero-wait detail send fails. Notification wakes the consumer after the retained data is ready.



Notification wakes Safety after retained data has been written.

The in-flight counter closes a race

safetySubmitEvent calls stickyBegin under a short lock, sends to the queue outside the lock, then calls stickyEnd under the lock. Begin increments inFlight. End records a failed send and decrements it. Safety's stickyCollect leaves a slot untouched while inFlight is nonzero.

Safety can drain the queue between the producer's send attempt and its final lost flag. Clearing retention in that gap could erase the only TRIP copy. Deferring collection until the send outcome is known preserves it for the next pass. The notification is sent after stickyEnd.

Collection is bounded

Safety drains up to 16 details, notes which rules supplied a queued TRIP, then collects missing retained state. It may append at most two events per rule. The static buffer has 16 + 2 times ruleCount entries. No queue call waits while holding the sticky spinlock.

Delivery retention and history are different

Sticky retention protects a decision under overflow; it cannot preserve every transition. The 64-entry event journal records processed TRIP, CLEAR and successful RESET events for communication. Its reader cursor can report a gap if overwritten.

The host suite includes a focused in-flight sticky case. Its input sequence demonstrates the ordering rule above; the testing chapter shows how to read such a case alongside its implementation.

Original source in this PDF: src/services/safety.c (p. 115); src/domain/alarm_sticky.c (p. 293); src/domain/event_journal.c (p. 296)

CHAPTER 3

3.16 Safety is a task wrapper around a pure decision core

safetyInit creates alarm-detail, command and status queues, initializes the journal and pure SafetyCore, writes relay OFF and creates the priority6 task on core1. Large buffers are static and task-private.

One Safety pass

Safety wakes on notification or a 100 ms timeout. It drains alarms and commands, peeks the latest Snapshot, gates 5V readings against the current rail epoch and reads the enabled-sensor mask. It then calls the pure safetyCoreStep and writes the relay only if the output changes.

The pure core applies events, checks freshness, processes commands, then computes output. TRIP sets active and optionally latched; CLEAR removes active without removing the latch. A TRIP-and-CLEAR batch still forces a relay-deenergise action for that pass.

Missing inputs participate in the output decision

For an enabled relay-deenergise rule, an unusable input forces OFF with reason INPUT_MISSING. A warning-only rule may be stale without creating that actuator hold. A deliberately disabled sensor is skipped for stale-rule reporting unless its rule remains latched. This differs from merely displaying a stale mask.

An invalid 5V rail also holds the relay OFF with reason UNPOWERED. Interlock has first reason priority, then missing rule input, then power, then disabled lamp auto-control. The hold bypasses ordinary lamp dwell.

Commands return actual outcomes

Reset commands have a deadline, rule ID and reply tag. A command from a closed session is dropped before application; expired commands are refused. A latch reset is accepted only while the rule is inactive. The task journals accepted resets and returns outcomes through a short callback to the transport.

After action, Safety journals processed events, overwrites its latest status, marks progress and feeds its own watchdog. A status field remains a software observation, not independent physical feedback.

Original source in this PDF: src/services/safety.c (p. 115); src/domain/safety_core.c (p. 307); src/domain/safety_core.h (p. 303)

CHAPTER 3

3.17 Cross-core access needs more than volatile

FreeRTOS queue operations safely copy their records between tasks. Other shared objects use a `portMUX_TYPE` spinlock. Entering the critical section excludes another core using that lock and protects short updates.

Shared object	Protection in this source
Snapshot and SafetyStatus	FreeRTOS length-one queues.
Alarm detail events	FreeRTOS queue.
Sticky alarm slots	s_stickyMux critical sections.
Acquisition statistics	s_statsMux critical sections.
Progress timestamps	s_mux critical sections.
RailState record	s_railMux task/ISR critical sections.
Log-drop counter	s_dropMux critical sections.

Why protect a 64-bit timestamp

The ESP32-S3 CPU is 32-bit. A 64-bit value can require multiple machine accesses. Without coordination, a reader on the other core could see one half before an update and the other half after it. progress.c takes a lock around both reads and writes, returning one coherent MonoMs.

`volatile` asks the compiler to perform accesses rather than assume a cached value remains sufficient. It does not make compound operations atomic, create a cross-core ownership protocol or protect the lifetime of referenced data. The sensor drivers avoid most of these issues through single-task ownership.

Critical sections must be short

Do not block, print or wait for a conversion while holding a spinlock. The source generally copies small records or increments counters inside the lock, then computes/logs outside. Disabling interrupts on one core alone is insufficient to exclude a task running on the other core. [R2]

The sticky queue protocol and power-sense diagnostic rings need more careful reasoning than a single locked record. Their surrounding producer/consumer order must also be correct. Thread-safe individual operations do not automatically make a multi-step event-reconstruction algorithm atomic.

Original source in this PDF: src/diagnostics/progress.c (p. 129); src/services/rail_state.c (p. 324); src/services/safety.c (p. 115); src/services/acquisition.c (p. 96)

CHAPTER 3

3.18 Choose synchronization by the contract it preserves

Shared data needs a rule for ownership and for what a reader must observe together. The correct mechanism follows from that rule, rather than from adding a qualifier to every variable.

Mechanism	Contract used by this firmware
Queue copy	Transfer a complete record or publish a latest-state copy.
Mutex	One task owns a resource across a longer operation; another task can block or try.
Spinlock critical section	Protect a short shared update, including against a participating task on the other core.
Atomic operation	Access or update one atomic object with the language's defined synchronization semantics.

A read-modify-write must remain together

An ordinary counter increment can lose a concurrent update. A lock around the complete increment excludes a second participating writer. In the BLE adapter, `s_generation.fetch_add(1)` is an atomic increment instead. A separate atomic load followed by a separate store would still be two operations and could lose an intervening increment.

Atomic does not mean that a multi-field record becomes one snapshot. Reading s_secure and s_generation separately does not lock the connection between those reads. Session checks and the transport's surrounding protocol provide the application meaning.

The critical section protects the pair

RailState contains both validity and epoch. Its producer and consumers use the same spinlock so a read obtains one coherent pair. Merely making each field volatile would not preserve that relationship. Progress uses the same principle for a 64-bit timestamp.

Waiting changes the choice of primitive

The flash gate is a mutex because an NVS commit can take time. Acquisition attempts it without waiting and defers a ping if it is busy. A spinlock must not be held across that commit or a task-blocking call. Task notifications wake an owner but carry neither a protected record nor resource ownership by themselves.

A function is reentrant when another invocation can run before the first finishes without corrupting either invocation's state. A static scratch buffer is shared across calls, so a second invocation can overwrite the first invocation's data. A mutex can serialize task callers; an ISR needs a protocol valid in interrupt context. Static, const and volatile do not establish reentrancy on their own.

Original source in this PDF: src/services/rail_state.c (p. 324); src/diagnostics/progress.c (p. 129); src/services/flash_gate.c (p. 369); src/transports/ble_gatt.cpp (p. 429)

CHAPTER 3

3.19 Logging is a bounded side channel

`logPrintf()` formats a fixed LogRecord with a level byte and 128-character text array. `vsnprintf()` bounds the text; long messages are truncated. It then attempts to enqueue without waiting. The 32-entry queue protects producers from a slow UART.

`serialLoggerTask()` waits for a record indefinitely, prints a level tag and text, reports newly observed drop counts, and stamps progress after the output completes. This task is the normal application log-output owner.

Formatting still costs the producer

Queueing removes serial transmission from the caller's critical path. It does not remove vsnprintf work, stack use or float formatting cost from the producer. `LogRecord rec` is local to logPrintf, so each calling task needs enough stack headroom for that path.

At 115200 bit/s with approximately 10 bits per character, a 128-character message alone takes about 11.1 ms of line time. Thirty-two full-size records can represent hundreds of milliseconds of UART backlog. Dropping and counting prevents that backlog from blocking acquisition.

The boot fallback is different

Before s_task exists, logPrintf writes directly with printf. This supports early boot and bring-up, which may not call logInit. If logger creation fails, later workers still use that blocking fallback. Therefore the header's nonblocking description applies to the functioning queued path, not every failure case.

`logDroppedCount()` protects the drop counter with a lock. `logQueueDepth()` asks how many records wait. `logTaskHandle()` exposes a handle for Supervisor stack checks. `tagOf()` maps numeric levels to E, W, I and D; invalid levels print a question mark.

Limits of the status line

The Supervisor assembles a multi-metric status into the same 128-character text limit. Later columns can be truncated. It also prints stored health without applying an age check in cell(). A stopped producer can leave a plausible number in a diagnostic line even though Safety's separate freshness logic has already forced the lamp OFF.

Original source in this PDF: src/diagnostics/logger.c (p. 125); src/diagnostics/logger.h (p. 123); src/services/supervisor.c (p. 132)

CHAPTER 3

3.20 Progress supervision checks completed work

progressMark records completed useful work, and progressLastMs returns a locked 64-bit timestamp. Supervisor wakes every second on an absolute tick grid. It observes; it does not call drivers or feed another task's watchdog.

Observed work	No-progress deadline
Safety	1000 ms
Active acquisition worker	2000 ms
Logger, when records are queued	2000 ms
Configuration	3000 ms
BLE transport	2000 ms
Wi-Fi task	2000 ms
PowerCoordinator	1000 ms

Inactive does not mean stalled

The acquisition entry dynamically selects the lease holder's task ID. An inactive worker owes no progress. If no worker owns the lease during handover, this check is skipped; PowerCoordinator's acknowledgement deadline reports a stuck transition instead.

There is a 5000 ms boot grace. The first late check logs once per stall episode; recovery logs once. Deadlines are evaluated at the next one-second Supervisor pass, not at an exact hardware interrupt instant.

Configuration marks progress while waiting normally for the flash gate or an application acknowledgement. Its bounded wait should not look like an unexplained hang. Logger is considered idle when its queue is empty, even if it is currently printing the last dequeued record.

Counters tell a different story

Supervisor reports deltas for late/skipped jobs, guard cancellations, detail overflows and dropped logs. Its period's DEGRADED label clears after the status report. It is not an alarm latch and does not certify calibration or hardware correctness.

Original source in this PDF: src/services/supervisor.c (p. 132); src/services/supervisor.h (p. 130); src/diagnostics/progress.c (p. 129)

CHAPTER 3

3.21 Memory supervision makes assumptions visible

Supervisor inspects all nine application task handles, stack high-water marks and internal heap every 10 s. It reports periodically every 60 s and warns when a live task has less than 512 free stack bytes or the internal-heap historical minimum is below 20000 bytes.

A minimum is about paths already executed

The stack high-water mark is the smallest unused area observed so far. A rare parser, security or formatting path can require more stack later. The largest free heap block identifies the largest contiguous allocation available. Total free heap can be larger because it includes separate fragments.

The comment suggests 25 percent stack headroom, but the implemented warning is an absolute 512 bytes. For Safety's 3072-byte stack, 25 percent would be 768 bytes. Do not treat the two criteria as identical.

Static buffers preserve stack space

Protocol frames, calibration requests, snapshots and Safety's event arrays are static. This makes RAM cost visible and avoids large local arrays on small stacks. They remain mutable state with an explicit owner; moving a shared helper's local buffer to static would require additional synchronization.

The repository contains several historical heap/stack runs from earlier feature stages. Those historical figures describe earlier images. The radio, configuration and low-power tasks add storage and execution paths; memory sizing requires measurements of the complete image, including uncommon paths.

Supervisor's observation itself uses bounded log records. The first line reports the main seven tasks, and a second line reports AcqLowPower and PowerCoord. A truncated log is a presentation limitation, not missing allocated stack storage.

Original source in this PDF: src/services/supervisor.c (p. 132); include/app_config.h (p. 36); src/services/configuration.c (p. 374); src/transports/ble_transport.c (p. 436)

CHAPTER 3

3.22 Watchdogs recover from work that stops returning

Safety, the active acquisition worker, PowerCoordinator, Configuration, BLE and Wi-Fi subscribe themselves to the task watchdog. Each resets its own subscription after completed work. The inactive acquisition worker unsubscribes when releasing the lease, because waiting indefinitely is then expected. [R4]

The application names a 5000 ms budget but does not call esp_task_wdt_init to set timeout or panic policy. The effective setting belongs to the framework build. Supervisor can warn early; it must not conceal a peer's failure by feeding its subscription.

Intentional failure images

stall_acq blocks acquisition after the configured threshold. Safety can still age light data and force OFF. stall_safety blocks the relay owner, so the previous command may remain until reset. alarm_latency uses a synthetic rule; ble_tear cuts selected telemetry frames to test resynchronization. These are acceptance images, not the normal production behaviour.

What the repository measured

The README records an older no-radio run of 37-96 us submit-to-action and later radio-load runs. When Safety shared core0 with Wi-Fi startup, isolated delays reached 40-110 ms. The task table therefore places Safety on core1.

With Safety on core1, the README records startup runs at up to 145 us submit-to-action, and a connected Wi-Fi/BLE run with a worst case of 152 us and sample-to-action at most 1 ms. These are project bench records, not new measurements or universal worst-case guarantees. They also predate physical validation of the new power/calibration paths.

The timing ends at software action, not relay contact motion. Sampling interval, filtering, persistence, task delivery and mechanical response are separate contributors to a physical response time.

Original source in this PDF: src/services/acquisition.c (p. 96); src/services/safety.c (p. 115); src/services/configuration.c (p. 374); src/services/power_coordinator.c (p. 334); platformio.ini (p. 28)

ORIGINAL SOURCE / CHAPTER 3 / 1 OF 1

src/hal/arduino_entry.h

Original lines 1-9 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // Arduino entry points defined in C.
2  // The Arduino-ESP32 core (cores/esp32/main.cpp) calls setup() and loop() with
3  // C++ linkage, i.e. the mangled symbols _Z5setupv and _Z4loopv. These
4  // declarations give the C definitions those symbol names. Include this header
5  // in every file that defines setup()/loop() (src/main.c, test/bringup/*.c).
6  #pragma once
7
8  void setup(void) __asm__("_Z5setupv");
9  void loop(void) __asm__("_Z4loopv");
```

Return to guided explanation, p. 58

GUIDED SOURCE READING / CHAPTER 3

Boot order is the first concurrency lesson

setup appears sequential, but each task creation can start another execution stream. Read the function as a dependency chain: every service must receive usable state before it can run.

Original excerpt: src/main.c (p. 82) / lines 41-55. The complete unchanged file follows.

```
41    // 4. Static queues.
42    if (!snapshotStoreInit()) logPrintf(LOG_LEVEL_ERROR, "snapshot store init failed");
43
44    // 5. Configuration: loads NVS and publishes before the tasks below start.
45    // On failure the compiled defaults apply (all sensors on).
46    if (!configurationInit()) logPrintf(LOG_LEVEL_ERROR, "configuration init failed - compiled
 >  defaults");
47
48    // 6-10. Tasks. Both acquisition workers start without the lease;
49    // PowerCoordinator grants the first one from the sense level (WP9).
50    if (!safetyInit()) logPrintf(LOG_LEVEL_ERROR, "safety init failed - relay stays OFF");
51    if (!acquisitionInit()) logPrintf(LOG_LEVEL_ERROR, "acquisition init failed");
52    if (!powerCoordinatorInit()) logPrintf(LOG_LEVEL_ERROR, "power coordinator init failed -
 >  normal mode, no transitions");
53    // Sensing and Safety run without it: a failed radio only costs the app link.
54    if (!bleTransportInit()) logPrintf(LOG_LEVEL_ERROR, "BLE transport init failed - no app
 >  link");
55    // Never waits for an access point; without credentials it stays idle.
```

snapshotStoreInit comes before the workers because acquisition publishes through that queue and Safety reads it. Creating a task first would let it run before its communication channel exists. Configuration follows, loading or constructing settings and publishing them before Safety or acquisition reads a first configuration.

Safety starts before acquisition so an alarm producer has a receiver. Acquisition creates both workers, but the lease keeps measurement inactive until PowerCoordinator grants an owner. This extra gate lets task creation and permission to touch hardware be separate events.

Each if checks an initializer's result and logs failure. The check does not undo earlier work, and it does not abort setup. Read what a branch actually controls: an error log is different from a startup interlock. Safe GPIO defaults were established earlier, which is why a missing Safety task leaves the relay commanded OFF.

BLE, Wi-Fi and Supervisor appear later because they consume service state rather than establish the measurement contract. loop finally blocks, leaving application work to these tasks. The next walkthrough follows the acquisition task to see what it does with that permission.

ORIGINAL SOURCE / CHAPTER 3 / 1 OF 2

src/main.c

Original lines 1-32 | Code and comments unchanged; long lines wrap visually

setup (L25-58): Establish safe outputs, logger, gate, board check and queues; load configuration; start Safety, both acquisition workers, power, radios and supervision.

```
1  // Production entry point - bootstrap ONLY (Firmware Architecture §5).
2  // No sensing, reconnect or policy work belongs here or in loop().
3  // Order (RTOS Architecture §5): safe outputs, logger, flash gate, board gate,
4  // queues, Configuration (publishes the applied config that Safety and
5  // acquisition read on their first pass), Safety BEFORE acquisition (the
6  // first alarm event always has a receiver), PowerCoordinator (grants the
7  // acquisition lease), the BLE transport, Wi-Fi, then Supervisor. A failed step is logged and
>  boot continues; the relay is
8  // already de-energised and only Safety can change it.
9  #include <Arduino.h>
10
11 #include "board_profile.h"
12 #include "diagnostics/logger.h"
13 #include "hal/arduino_entry.h"
14 #include "hal/gpio_safe.h"
15 #include "services/acquisition.h"
16 #include "services/configuration.h"
17 #include "services/flash_gate.h"
18 #include "services/power_coordinator.h"
19 #include "services/safety.h"
20 #include "services/snapshot_store.h"
21 #include "services/supervisor.h"
22 #include "transports/ble_transport.h"
23 #include "transports/wifi_mqtt.h"
24
25 void setup(void) {
26   // 1. Safe output defaults before anything else.
27   gpioApplySafeDefaults();
28
29   // 2. Serial console: UART0 at 115200 (CONFIG_ESP_CONSOLE_UART_BAUDRATE).
30   logInit();
31   logPrintf(LOG_LEVEL_INFO, "hydroponics firmware - stage 6 (BLE app link, calibration,
>  low-power mode)");
32   if (!flashGateInit()) logPrintf(LOG_LEVEL_ERROR, "flash gate init failed - pings
>  unguarded");
```

Return to guided explanation, p. 81

ORIGINAL SOURCE / CHAPTER 3 / 2 OF 2

src/main.c

Original lines 33-63 | Code and comments unchanged; long lines wrap visually

setup (L25-58): Establish safe outputs, logger, gate, board check and queues; load configuration; start Safety, both acquisition workers, power, radios and supervision.
loop (L60-63): Park the Arduino loop task. The application work happens in explicitly created tasks.

```
33
34    // 3. Board gate: pins assigned and power sense measured.
35    BoardValidation board = boardProfileValidate();
36    logPrintf(LOG_LEVEL_INFO, "board gate: power sense %s, relay %s -> %s",
37              board.powerSenseAssigned ? "yes" : "NO",
38              board.relayAssigned ? "yes" : "NO",
39              board.productionReady ? "pass" : "FAIL");
40
41    // 4. Static queues.
42    if (!snapshotStoreInit()) logPrintf(LOG_LEVEL_ERROR, "snapshot store init failed");
43
44    // 5. Configuration: loads NVS and publishes before the tasks below start.
45    // On failure the compiled defaults apply (all sensors on).
46    if (!configurationInit()) logPrintf(LOG_LEVEL_ERROR, "configuration init failed - compiled
 >  defaults");
47
48    // 6-10. Tasks. Both acquisition workers start without the lease;
49    // PowerCoordinator grants the first one from the sense level (WP9).
50    if (!safetyInit()) logPrintf(LOG_LEVEL_ERROR, "safety init failed - relay stays OFF");
51    if (!acquisitionInit()) logPrintf(LOG_LEVEL_ERROR, "acquisition init failed");
52    if (!powerCoordinatorInit()) logPrintf(LOG_LEVEL_ERROR, "power coordinator init failed -
 >  normal mode, no transitions");
53    // Sensing and Safety run without it: a failed radio only costs the app link.
54    if (!bleTransportInit()) logPrintf(LOG_LEVEL_ERROR, "BLE transport init failed - no app
 >  link");
55    // Never waits for an access point; without credentials it stays idle.
56    if (!wifiMqttInit()) logPrintf(LOG_LEVEL_ERROR, "Wi-Fi init failed - Bluetooth only");
57    if (!supervisorInit()) logPrintf(LOG_LEVEL_ERROR, "supervisor init failed");
58  }
59
60  void loop(void) {
61    // Compatibility stub: the Arduino loop task blocks forever.
62    vTaskDelay(portMAX_DELAY);
63  }
```

Return to guided explanation, p. 81

ORIGINAL SOURCE / CHAPTER 3 / 1 OF 1

src/domain/schedule.h

Original lines 1-44 | Code and comments unchanged; long lines wrap visually

scheduleJobDue (L29-31): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.
scheduleFitsBefore (L42-44): Ask whether a declared call budget fits before another job's deadline. It cannot interrupt a call that hangs.

```
1  // Absolute-deadline job table for the acquisition engine (§9). Pure C.
2  // Work package WP7 (Stage 4). Host-tested; the acquisition task only calls it.
3  //
4  // Each job has a period and an absolute next deadline on the monotonic clock.
5  // A job that runs late keeps its grid: the next deadline is the first grid
6  // point after `now`. Missed grid points are counted as skipped and are never
7  // replayed in a burst ("skip, never catch up").
8  #pragma once
9  #include <stdbool.h>
10 #include <stdint.h>
11 #include "domain/types.h"
12
13 // Lateness up to this is release jitter, not a late run (§19: <= 5 ms).
14 #define SCHEDULE_LATE_TOLERANCE_MS ((uint32_t)5)
15
16 typedef struct ScheduleJob {
17   uint32_t periodMs;
18   uint32_t budgetMs;   // worst-case worker time for one step of this job
19   MonoMs nextDueMs;
20   uint32_t runs;
21   uint32_t late;       // runs started more than SCHEDULE_LATE_TOLERANCE_MS late
22   uint32_t skipped;    // whole periods that passed without a run
23   uint32_t maxLateMs;  // largest lateness seen (jitter for on-time runs)
24   uint32_t lastLateMs;
25 } ScheduleJob;
26
27 void scheduleJobInit(ScheduleJob* job, uint32_t periodMs, uint32_t budgetMs, MonoMs
 > firstDueMs);
28
29 static inline bool scheduleJobDue(const ScheduleJob* job, MonoMs now) {
30   return monoReached(now, job->nextDueMs);
31 }
32
33 // Record a run that starts at `now` (call only when due) and move the
34 // deadline to the next grid point strictly after `now`.
35 void scheduleJobMarkRun(ScheduleJob* job, MonoMs now);
36
37 // Earliest deadline over `count` jobs (count >= 1).
38 MonoMs scheduleEarliestDue(const ScheduleJob* jobs, uint8_t count);
39
40 // true if a step of `budgetMs` started at `now` ends no later than `deadline`
41 // (§9: defer an ordinary step that would cross the next BME280 slot).
42 static inline bool scheduleFitsBefore(MonoMs now, uint32_t budgetMs, MonoMs deadline) {
43   return now + budgetMs <= deadline;
44 }
```

Return to guided explanation, p. 58

ORIGINAL SOURCE / CHAPTER 3 / 1 OF 1

src/domain/schedule.c

Original lines 1-33 | Code and comments unchanged; long lines wrap visually

scheduleJobInit (L3-12): Set the period and first deadline and clear timing counters. A zero period is replaced with 1 ms.
scheduleJobMarkRun (L14-25): Count lateness/skips and advance along the original absolute grid without replaying missed jobs.

```
1  #include "domain/schedule.h"
2
3  void scheduleJobInit(ScheduleJob* job, uint32_t periodMs, uint32_t budgetMs, MonoMs
>  firstDueMs) {
4    job->periodMs = periodMs > 0 ? periodMs : 1;
5    job->budgetMs = budgetMs;
6    job->nextDueMs = firstDueMs;
7    job->runs = 0;
8    job->late = 0;
9    job->skipped = 0;
10   job->maxLateMs = 0;
11   job->lastLateMs = 0;
12 }
13
14 void scheduleJobMarkRun(ScheduleJob* job, MonoMs now) {
15   const MonoMs lateness = now > job->nextDueMs ? now - job->nextDueMs : 0;
16   const MonoMs missed = lateness / job->periodMs;  // grid points already passed
17   const uint32_t lateMs = lateness > UINT32_MAX ? UINT32_MAX : (uint32_t)lateness;
18
19   ++job->runs;
20   job->lastLateMs = lateMs;
21   if (lateMs > job->maxLateMs) job->maxLateMs = lateMs;
22   if (lateMs > SCHEDULE_LATE_TOLERANCE_MS) ++job->late;
23   job->skipped += missed > UINT32_MAX ? UINT32_MAX : (uint32_t)missed;
24   job->nextDueMs += (missed + 1) * job->periodMs;
25 }
26
27 MonoMs scheduleEarliestDue(const ScheduleJob* jobs, uint8_t count) {
28   MonoMs earliest = jobs[0].nextDueMs;
29   for (uint8_t i = 1; i < count; ++i) {
30     if (jobs[i].nextDueMs < earliest) earliest = jobs[i].nextDueMs;
31   }
32   return earliest;
33 }
```

Return to guided explanation, p. 58

ORIGINAL SOURCE / CHAPTER 3 / 1 OF 1

src/services/snapshot_store.h

Original lines 1-10 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // Latest-snapshot mailbox (§8, §9). Length-one queue.
2  // Writer: the acquisition lease holder only (xQueueOverwrite).
3  // Readers: Safety, BLE, MQTT peek with zero wait - never consume.
4  #pragma once
5  #include <stdbool.h>
6  #include "domain/reading.h"
7
8  bool snapshotStoreInit(void);
9  void snapshotPublish(const Snapshot* snapshot);
10 bool snapshotPeek(Snapshot* out);  // false if nothing published yet
```

Return to guided explanation, p. 58

GUIDED SOURCE READING / CHAPTER 3

Why the queue is created with length one

Startup creates the Snapshot channel before measurement begins. Its three public operations implement one precise promise: keep the newest complete state available to several readers.

Original excerpt: src/services/snapshot_store.c (p. 88) / lines 10-15. The complete unchanged file follows.

```
10  bool snapshotStoreInit(void) {
11    if (s_queue == NULL) {
12      s_queue = xQueueCreateStatic(1, sizeof(Snapshot), s_queueStorage, &s_queueControl);
13    }
14    return s_queue != NULL;
15  }
```

The first argument to xQueueCreateStatic is 1: one queued item. The second is sizeof(Snapshot), so the queue copies a whole 584-byte record. s_queueStorage holds payload bytes, while s_queueControl holds the kernel's bookkeeping. They are file-static because both must remain alive after this initializer returns.

The NULL check makes initialization repeatable: a second call reuses the queue rather than constructing another channel. Returning a bool lets setup report creation failure before relying on that handle.

snapshotPublish uses xQueueOverwrite because the channel means current state. If A, B and C arrive before a reader runs, retaining C is the intended result. A length-16 FIFO would instead accumulate old states and make a slow consumer increasingly late.

snapshotPeek uses zero wait and leaves the item in place. Safety can read it without making Supervisor's next read fail. A missing item returns false; it does not become a valid zero-filled Snapshot. Queue copying also means later writes to acquisition's working record cannot silently alter the reader's local copy.

This queue deliberately loses intermediate versions. Alarm events need a different channel because an overwritten TRIP could matter even after current temperature has returned to normal.

ORIGINAL SOURCE / CHAPTER 3 / 1 OF 1

src/services/snapshot_store.c

Original lines 1-24 | Code and comments unchanged; long lines wrap visually

snapshotStoreInit (L10-15): Create a static length-one queue for complete Snapshot copies.
snapshotPublish (L17-19): Overwrite latest state; intermediate versions are intentionally not kept as history.

```
1  #include "services/snapshot_store.h"
2
3  #include <freertos/FreeRTOS.h>
4  #include <freertos/queue.h>
5
6  static StaticQueue_t s_queueControl;
7  static uint8_t s_queueStorage[sizeof(Snapshot)];
8  static QueueHandle_t s_queue = NULL;
9
10 bool snapshotStoreInit(void) {
11   if (s_queue == NULL) {
12     s_queue = xQueueCreateStatic(1, sizeof(Snapshot), s_queueStorage, &s_queueControl);
13   }
14   return s_queue != NULL;
15 }
16
17 void snapshotPublish(const Snapshot* snapshot) {
18   if (s_queue != NULL) xQueueOverwrite(s_queue, snapshot);
19 }
20
21 bool snapshotPeek(Snapshot* out) {
22   if (s_queue == NULL || out == NULL) return false;
23   return xQueuePeek(s_queue, out, 0) == pdTRUE;
24 }
```

Return to guided explanation, p. 87

ORIGINAL SOURCE / CHAPTER 3 / 1 OF 2

src/services/acquisition.h

Original lines 1-41 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // Acquisition engine: the one serial owner of all sensor buses.
2  // Work package WP7 (Stage 4). Architecture §6, §7, §9, §12.
3  //
4  // Stage 4 (decision D15) had ONE worker, AcqNormal (core 1, priority 4), owning
5  // every driver and bus. It runs an absolute-deadline schedule (1 s frame +
6  // BME280 every 250 ms, domain/schedule), stamps each completed reading,
7  // evaluates the alarm rules for it, submits TRIP/CLEAR to Safety at once and
8  // publishes the snapshot after every completed reading and at each frame.
9  //
10 // WP9 (§14): two workers share the drivers and the schedule, and only the
11 // lease holder runs. AcqNormal (core 1) holds it with external power and runs
12 // every sensor; AcqLowPower (core 0) holds it in low power and runs only the
13 // 3.3 V sensors (ACQ_LOW_POWER_SENSORS). PowerCoordinator moves the lease:
14 //   acquisitionRequestQuiesce(gen)  the holder stops at its next pass
15 //                                   boundary, cancels pending steps, makes the
16 //                                   5 V interfaces safe if the rail is down
17 //                                   (preparePowerOff: HC-SR04 TRIG low), leaves
18 //                                   the task watchdog and releases;
19 //   acquisitionReleasedGeneration() the acknowledgement;
20 //   acquisitionGrant(worker, gen, allowed)  the next holder joins the
21 //                                   watchdog, restarts the schedule and applies
22 //                                   the configuration limited to `allowed`.
23 // A sensor outside `allowed` publishes UNPOWERED; when it comes back it is
24 // initialised again and shows WARMING until its first result.
25 // Rail gating (§14): a 5 V sensor (HC-SR04, pH, TDS) starts only while the
26 // rail is valid and its result is kept only if the rail is still valid with
27 // the same epoch at the end; otherwise it is published UNPOWERED.
28 // HALs begin in AcqNormal on core 1 (the GPIO ISR service is installed there).
29 #pragma once
30 #include <freertos/FreeRTOS.h>
31 #include <freertos/task.h>
32 #include <stdbool.h>
33 #include <stdint.h>
34
35 #include "app_config.h"
36 #include "domain/power_mode.h"
37 #include "domain/sensor_set.h"
38 #include "domain/types.h"
39
40 // Retained 3.3 V supply: the sensors that keep running in low power (§15).
41 #define ACQ_LOW_POWER_SENSORS                                                  \
```

Return to guided explanation, p. 58

ORIGINAL SOURCE / CHAPTER 3 / 2 OF 2

src/services/acquisition.h

Original lines 42-83 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
  ((SensorMask)((1u << SENSOR_BME280) | (1u << SENSOR_DS18B20) | (1u << SENSOR_LIGHT) | \
                (1u << SENSOR_SGP30)))

bool acquisitionInit(void);  // construct drivers, create both workers; no lease yet

// Lease protocol, called only by PowerCoordinator (§14). Grant refuses (false)
// while another worker still holds the lease: never two owners.
bool acquisitionGrant(PowerWorker worker, uint32_t generation, SensorMask allowed);
bool acquisitionRequestQuiesce(uint32_t generation);
uint32_t acquisitionReleasedGeneration(void);  // 0 = nothing released yet

// True once AcqNormal has begun the buses (and the GPIO ISR service on core 1).
bool acquisitionHalsReady(void);

// Task id of the lease holder for the progress check; TASK_ID_COUNT if none.
TaskId acquisitionActiveTaskId(void);

// Counters for Supervisor (copied under a lock; zero-wait).
typedef struct AcquisitionStats {
  uint32_t frames;
  uint32_t frameLate;      // frame releases more than SCHEDULE_LATE_TOLERANCE_MS late
  uint32_t frameSkipped;
  uint32_t frameJitterMaxMs;
  uint32_t bmeLate;
  uint32_t bmeSkipped;
  uint32_t guardTimeouts;  // steps cancelled by the frame / slot guard
  uint32_t driverFailures; // start/poll results other than OK/PENDING/READY
  uint32_t ruleEvents;     // TRIP + CLEAR submitted to Safety
} AcquisitionStats;

void acquisitionGetStats(AcquisitionStats* out);

// SGP30 baseline status name ("none", "learning", "restored", "valid"),
// refreshed every frame. Zero wait, any task.
const char* acquisitionSgp30BaselineName(void);

// The lease holder's task (AcqNormal if none), e.g. to wake it for a new
// configuration. NULL before acquisitionInit().
TaskHandle_t acquisitionTaskHandle(void);
// For Supervisor's stack check.
TaskHandle_t acquisitionNormalTaskHandle(void);
TaskHandle_t acquisitionLowPowerTaskHandle(void);
```

Return to guided explanation, p. 58

GUIDED SOURCE READING / CHAPTER 3

Read the worker loop in the order it gains responsibility

acqTask runs in both acquisition tasks. Its argument identifies the normal or low-power worker; the surrounding lease decides which one is allowed to execute the shared engine.

Original excerpt: src/services/acquisition.c (p. 96) / lines 593-605. The complete unchanged file follows.

```
593      if (!holding) {
594        ulTaskNotifyTake(pdTRUE, portMAX_DELAY);  // a grant (or a stale wake-up)
595        if (!s_halsReady || !holdsLease(me)) continue;
596        holding = true;
597        acquireLease(me, monoNowMs());
598      }
599      if (quiesceRequested()) {
600        releaseLease(me);
601        holding = false;
602        continue;
603      }
604      const MonoMs now = monoNowMs();
605      const MonoMs due = nextWake(now);
```

The first wait blocks a worker without ownership. A notification only asks it to check: s_halsReady and holdsLease must still pass before holding becomes true. This is why receiving a notification is not equivalent to receiving permission.

acquireLease subscribes the holder to the watchdog, restarts deadlines and requests a fresh configuration application. Those actions occur after the permission check. Performing them on every stray wake would make the inactive worker modify shared scheduling state.

The quiesce check follows before new work. If PowerCoordinator has requested release, the holder cancels pending operations, acknowledges its generation and returns to the blocked state. Putting a new sensor start before that check would prolong a handover that was already requested.

Further down, nextWake computes a timed notification wait. On wake the code checks quiescence again, then calls runPass with newly read time. The second check covers a request arriving while the worker slept. Progress and watchdog reset come after the pass, so they describe a completed return through the worker rather than an intention to work.

Follow runPass next: FreeRTOS selects this task, while runPass selects a driver operation inside it.

GUIDED SOURCE READING / CHAPTER 3

Why the pass starts with boundaries and the fast lane

The worker is running and owns the drivers. It now needs to choose useful calls without spending the entire second inside the slowest sensor.

Original excerpt: src/services/acquisition.c (p. 96) / lines 464-480. The complete unchanged file follows.

```
464  static void runPass(MonoMs now) {
465    if (scheduleJobDue(&s_jobs[JOB_FRAME], now)) startFrame(now);
466    if (scheduleJobDue(&s_jobs[JOB_BME280], now)) startBmeSlot(now);
467
468    for (uint8_t iter = 0; iter < 8; ++iter) {
469      bool progress = false;
470      now = monoNowMs();
471      if (s_runs[RUN_BME280].wantStart) progress |= tryStart(RUN_BME280, now);
472      for (uint8_t r = 0; r < RUN_COUNT; ++r) progress |= tryPoll(r, monoNowMs());
473      for (uint8_t i = 0; i < FRAME_ORDER_COUNT; ++i) {
474        const uint8_t r = kFrameOrder[i];
475        if (!s_runs[r].wantStart) continue;
476        if (!tryStart(r, monoNowMs())) break;  // keep the order: later ones wait too
477        progress = true;
478      }
479      if (!progress) break;
480    }
```

The two boundary tests first release any frame or BME280 work that is now due. Without these tests, polling could continue while a new period was never admitted. A frame boundary also applies settings and handles unfinished prior work before new starts.

Each iteration rereads time because earlier calls take time. BME280 gets the first start opportunity; then every pending driver gets a due-poll opportunity. Ordinary starts follow kFrameOrder, which puts 1-Wire work before an ultrasonic ping.

progress uses bitwise OR assignment with a bool. The right-hand side is evaluated even if progress is already true, so a successful earlier poll does not skip later sensors. A short-circuit expression such as progress || tryPoll would stop evaluating calls after the first success.

The break after a blocked ordinary start preserves the chosen order. The final no-progress test avoids repeating a pass that cannot advance anything. The outer bound of eight iterations limits repeated dispatch; it does not interrupt a driver function that has stopped returning.

The result is concurrency inside one owner: many chips or sampling windows can be in flight, but individual bus calls remain serial. The following start/poll functions show how each driver participates.

GUIDED SOURCE READING / CHAPTER 3

Admission checks belong before the hardware command

tryStart is the entry point for one wanted measurement. Its early returns are deliberate: conditions that make an operation invalid are checked before spending bus time or claiming that it began.

Original excerpt: src/services/acquisition.c (p. 96) / lines 295-304. The complete unchanged file follows.

```
295    if (onFiveVoltRail(run)) {
296      const RailState rail = railStateRead();
297      if (!rail.valid5v) {  // no 5 V: no init, no conversion of a dead output
298        markSensor(run, HEALTH_UNPOWERED, now);
299        run->wantStart = false;
300        return true;
301      }
302      run->startEpoch = rail.epoch;
303    }
304    if (!run->initialised) {
```

For pH, TDS and HC-SR04, railStateRead supplies one coherent validity/epoch pair. Invalid power produces an UNPOWERED observation and clears wantStart for this period. Returning true here means the service handled that requested start; it does not mean a physical measurement succeeded.

On valid power the service records startEpoch before the window begins. At completion, matching that saved history is what distinguishes uninterrupted power from a supply that failed and recovered during sampling. Saving only the final epoch would lose that evidence.

Initialization and retry checks follow. A driver that previously failed init waits until retryAtMs; a long init also needs a free scheduling slot. After init, time is refreshed because initialization itself may consume tens of milliseconds.

Immediately before HC-SR04 starts, the worker tries the flash gate. It then copies compensation inputs, clears wantStart and calls the driver. DRIVER_OK makes the operation pending; a later poll performs completion. None of these checks can repair a command already sent, which is why validation and admission appear ahead of the start call.

GUIDED SOURCE READING / CHAPTER 3

A pending result tells the worker when to return

After start, the acquisition engine must distinguish a driver that can advance now from one whose chip or filter still needs elapsed time.

Original excerpt: src/services/acquisition.c (p. 96) / lines 338-349. The complete unchanged file follows.

```
338  static bool tryPoll(uint8_t runId, MonoMs now) {
339    DriverRun* run = &s_runs[runId];
340    if (!run->pending || now < run->nextPollMs) return false;
341    if (!blockingAllowed(runId, run->stepBudgetMs, now)) return false;
342    const PollResult pr = sensorDriverPoll(run->driver, now);
343    if (pr.status == DRIVER_PENDING) {
344      run->nextPollMs = pr.nextPollMs > now ? pr.nextPollMs : now + 1;
345      return true;
346    }
347    finishRun(run, monoNowMs(), pr.status);
348    return true;
349  }
```

The first condition rejects a run that is not pending or not due. The next rejects a call whose blocking budget would disturb protected work. Neither path calls the driver. These guards are placed before sensorDriverPoll because they decide whether touching this device is appropriate now.

PENDING means the operation still exists. The service retains pending and records the returned nextPollMs. If that time is already due, it substitutes now + 1 rather than repeatedly asking for progress at the same instant. This avoids one class of immediate retry loop.

Any other status reaches finishRun. That helper ends pending tracking, releases a ping gate if relevant and stores the driver's verdict. The timestamp is read again after poll returns, so publication reflects completion rather than only the time before the call.

The bool returned by tryPoll belongs to the service's progress loop, while pr.status belongs to the driver's lifecycle and Reading.health belongs to data usability. They answer three different questions. Keeping them separate is what lets a READY result correctly carry FAULT or UNCALIBRATED without confusing the scheduler.

GUIDED SOURCE READING / CHAPTER 3

Publish only after identity and validity are assigned

A completed driver record still needs system metadata. storeResults adds that metadata before the record becomes visible through the Snapshot queue.

Original excerpt: src/services/acquisition.c (p. 96) / lines 203-211. The complete unchanged file follows.

```
203      r->sequence = s_work.readings[r->metric].sequence + 1u;
204      r->powerEpoch = epoch;
205      r->configRevision = s_appliedRevision;
206      s_work.readings[r->metric] = *r;
207      evaluateRules(r, now);
208    }
209    s_work.publishedMs = now;
210    s_work.powerEpoch = epoch;
211    snapshotPublish(&s_work);
```

Sequence comes from the previously stored metric, not solely from the driver's count of good samples. A failed observation therefore gets a new identity too. Otherwise a newly failed result could be mistaken for a retransmission of an earlier successful sample.

powerEpoch and configRevision then attach the supply history and settings used by this acquisition owner. The copy into s_work happens after those assignments so the working Snapshot contains the annotated version. Updating only the temporary out array after that copy would leave stored metadata unchanged.

evaluateRules receives the completed record immediately, before the whole Snapshot is published. Alarm submission therefore does not wait for a slower telemetry task or for the next ordinary frame. The alarm retains its own sample, so later Snapshot overwrite cannot erase the event.

Finally publishedMs and the Snapshot epoch are updated and snapshotPublish copies the complete record set. These assignments do not make all sensors simultaneous; each Reading still carries its own measurement interval. The ordering establishes a consistent publication, not a common physical sample time.

ORIGINAL SOURCE / CHAPTER 3 / 1 OF 15

src/services/acquisition.c

Original lines 1-51 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
#include "services/acquisition.h"

#include <esp_task_wdt.h>
#include <freertos/FreeRTOS.h>
#include <freertos/task.h>
#include <stdint.h>
#include <string.h>

#include "app_config.h"
#include "board_profile.h"
#include "diagnostics/logger.h"
#include "diagnostics/progress.h"
#include "domain/alarm.h"
#include "domain/schedule.h"
#include "drivers/bme280.h"
#include "drivers/ds18b20.h"
#include "drivers/hcsr04.h"
#include "drivers/light.h"
#include "drivers/ph.h"
#include "drivers/sgp30.h"
#include "drivers/tds.h"
#include "hal/adc_channel.h"
#include "hal/clock.h"
#include "hal/i2c_bus.h"
#include "hal/onewire_bus.h"
#include "hal/pulse_capture.h"
#include "policy_config.h"
#include "services/configuration.h"
#include "services/flash_gate.h"
#include "services/rail_state.h"
#include "services/safety.h"
#include "services/snapshot_store.h"

// ---- Buses and drivers: owned by the lease holder only (§11, §14) ------------
static I2cBus s_i2c = I2C_BUS_INIT;
static OneWireBus s_oneWire = ONE_WIRE_BUS_INIT;
static AdcChannel s_adcLight = ADC_CHANNEL_INIT;
static AdcChannel s_adcTds = ADC_CHANNEL_INIT;
static AdcChannel s_adcPh = ADC_CHANNEL_INIT;
static PulseCapture s_capture = PULSE_CAPTURE_INIT;

static Bme280Config s_bmeConfig = {&s_i2c, BOARD_BME280_I2C_ADDRESS};
static Sgp30Config s_sgpConfig = {&s_i2c, BOARD_SGP30_I2C_ADDRESS};
static Ds18b20Config s_dsConfig = {&s_oneWire};
// Calibrations come from the applied configuration: copied in
// acquisitionInit(), then pushed to the drivers whenever a new revision is
// applied (on-device calibration, applyConfiguration()).
static LightConfig s_lightConfig = {&s_adcLight, {1.0f, 0.0f, 0, false}};
static TdsConfig s_tdsConfig = {&s_adcTds, {1.0f, 0.0f, 0, false}};
static PhConfig s_phConfig = {&s_adcPh, BOARD_PH_DIVIDER_TOP_OHM,
BOARD_PH_DIVIDER_BOTTOM_OHM,
                              {0.0f, 0.0f, 0, false},
```

Return to guided explanation, p. 95

ORIGINAL SOURCE / CHAPTER 3 / 2 OF 15

src/services/acquisition.c

Original lines 52-100 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
52                                  true};  // compiled nominal fit: published UNCALIBRATED
53  static Hcsr04Config s_hcConfig = {&s_capture, CFG_HCSR04_TIMEOUT_MS};
54
55  static Bme280Driver s_bme;
56  static Sgp30Driver s_sgp;
57  static Ds18b20Driver s_ds;
58  static LightDriver s_light;
59  static TdsDriver s_tds;
60  static PhDriver s_ph;
61  static Hcsr04Driver s_hc;
62
63  // ---- Per-driver stepping (init -> start -> poll), as test/bringup/water_main.c
64  enum { RUN_BME280 = 0, RUN_DS18B20, RUN_SGP30, RUN_HCSR04, RUN_TDS, RUN_PH, RUN_LIGHT,
 >  RUN_COUNT };
65
66  typedef struct DriverRun {
67    SensorDriver* driver;
68    uint32_t initBudgetMs;   // worst-case blocking of init()
69    uint32_t stepBudgetMs;   // worst-case blocking of one start()/poll(); > 2 = "blocking"
70    bool initialised;
71    bool pending;            // started, not yet READY or failed
72    bool wantStart;          // due this frame, not started yet
73    MonoMs nextPollMs;
74    MonoMs retryAtMs;        // earliest init() retry after a failure
75    DriverStatus lastFailure;  // DRIVER_OK = none; logged on change only
76    bool disabled;           // off by the configuration (WP8) or the lease (WP9): never
 >  scheduled
77    uint32_t startEpoch;     // 5 V sensors: rail epoch when the measurement started
78  } DriverRun;
79
80  // Worst-case blocking from each driver header (§16 budgets).
81  static DriverRun s_runs[RUN_COUNT] = {
82      [RUN_BME280] = {&s_bme.base, 30, 1, false, false, false, 0, 0, DRIVER_OK},
83      [RUN_DS18B20] = {&s_ds.base, 50, 13, false, false, false, 0, 0, DRIVER_OK},
84      [RUN_SGP30] = {&s_sgp.base, 45, 1, false, false, false, 0, 0, DRIVER_OK},
85      [RUN_HCSR04] = {&s_hc.base, 1, 1, false, false, false, 0, 0, DRIVER_OK},
86      [RUN_TDS] = {&s_tds.base, 1, 1, false, false, false, 0, 0, DRIVER_OK},
87      [RUN_PH] = {&s_ph.base, 1, 1, false, false, false, 0, 0, DRIVER_OK},
88      [RUN_LIGHT] = {&s_light.base, 1, 1, false, false, false, 0, 0, DRIVER_OK},
89  };
90
91  // One-second jobs, started in this order at each frame (§9): the 1-Wire work
92  // comes before the HC-SR04 ping because 1-Wire slots mask interrupts and
93  // would delay the echo timestamps (onewire_bus.h).
94  static const uint8_t kFrameOrder[] = {RUN_DS18B20, RUN_SGP30, RUN_HCSR04, RUN_TDS, RUN_PH,
 >  RUN_LIGHT};
95  #define FRAME_ORDER_COUNT ((uint8_t)(sizeof(kFrameOrder) / sizeof(kFrameOrder[0])))
96
97  enum { JOB_FRAME = 0, JOB_BME280, JOB_COUNT };
98  static ScheduleJob s_jobs[JOB_COUNT];
99
100 // ---- Applied configuration (WP8): read at frame boundaries only ---------------
```

Return to guided explanation, p. 95

ORIGINAL SOURCE / CHAPTER 3 / 3 OF 15

src/services/acquisition.c

Original lines 101-148 | Code and comments unchanged; long lines wrap visually

evaluateRules (L142-159): Evaluate only matching enabled rules. Submit actual TRIP/CLEAR information before asking Safety to wake.

```
101 static uint16_t s_appliedRevision;  // stamped into every Reading as configRevision
102 static ConfigBlob s_cfg;            // task-private copy (static: not on the stack)
103 #define BASELINE_OFFER_PERIOD_MS ((MonoMs)60 * 60 * 1000)  // Configuration rate-limits the
  > write too
104 static MonoMs s_nextBaselineOfferMs;
105 static volatile Sgp30BaselineStatus s_sgpBaseline = SGP30_BASELINE_NONE;  // one writer,
  > word-sized
106
107 // ---- Working snapshot and rule state (task-private) -------------------------
108 static Snapshot s_work;
109 static AlarmRuleState s_ruleState[POLICY_RULE_COUNT];
110
111 static portMUX_TYPE s_statsMux = portMUX_INITIALIZER_UNLOCKED;
112 static AcquisitionStats s_stats;
113
114 static StaticTask_t s_tcb;
115 static StackType_t s_stack[4 * 1024];  // cfgTaskSpec(TASK_ID_ACQUISITION_NORMAL)->stackBytes
116 static TaskHandle_t s_task = NULL;
117 static StaticTask_t s_tcbLow;
118 static StackType_t s_stackLow[4 * 1024];  // cfgTaskSpec(TASK_ID_ACQUISITION_LOW_POWER)->stac
  > kBytes
119 static TaskHandle_t s_taskLow = NULL;
120
121 // ---- Lease (WP9, §14) -------------------------------------------------------
122 // Everything above is shared by the two workers but touched only by the lease
123 // holder; the lock below orders the hand-over between the cores.
124 static portMUX_TYPE s_leaseMux = portMUX_INITIALIZER_UNLOCKED;
125 static PowerWorker s_owner = POWER_WORKER_NONE;
126 static uint32_t s_leaseGen;
127 static SensorMask s_allowed = SENSOR_MASK_ALL;
128 static uint32_t s_quiesceGen;   // requested, for the current lease
129 static uint32_t s_releasedGen;
130 static volatile bool s_halsReady;
131 static bool s_forceApply;       // a new holder applies the configuration at once
132
133 #define STATS_ADD(field, n)        \
134   do {                             \
135     portENTER_CRITICAL(&s_statsMux); \
136     s_stats.field += (n);          \
137     portEXIT_CRITICAL(&s_statsMux);  \
138   } while (0)
139
140 // ---- Bookkeeping after a result ---------------------------------------------
141
142 static void evaluateRules(const Reading* r, MonoMs now) {
143   for (uint8_t i = 0; i < POLICY_RULE_COUNT; ++i) {
144     const AlarmRule* rule = &kPolicyRules[i];
145     if (!rule->enabled || rule->metric != r->metric) continue;
146     const RuleDecision d = evaluateRule(rule, r, now, alarmStaleLimitMs(r->metric),
  > &s_ruleState[i]);
147     if (d != RULE_DECISION_TRIP && d != RULE_DECISION_CLEAR) continue;
148     AlarmEvent e;
```

Return to guided explanation, p. 95

ORIGINAL SOURCE / CHAPTER 3 / 4 OF 15

src/services/acquisition.c

Original lines 149-199 | Code and comments unchanged; long lines wrap visually

evaluateRules (L142-159): Evaluate only matching enabled rules. Submit actual TRIP/CLEAR information before asking Safety to wake.
onFiveVoltRail (L162-164): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.

```
    e.ruleId = i;
    e.decision = d;
    e.sample = *r;
#ifdef FW_ALARM_LATENCY_TEST
    safetyLatencyMarkSubmit();
    s_ruleState[i].tripped = false;  // re-arm: the next sample trips again
#endif
    safetySubmitEvent(&e);  // zero wait; wakes Safety now (§4)
    STATS_ADD(ruleEvents, 1);
  }
}

// Sensors on the external, switched 5 V rail (§14): gated on the rail epoch.
static bool onFiveVoltRail(const DriverRun* run) {
  return sensorMaskHas(SENSOR_MASK_RAIL_5V, sensorDriverId(run->driver));
}

// Every metric of the run's sensor shows `health` now. No new value (telemetry
// sends null), but a new observation: a new sequence, so the app records the
// transition and every repeated failure (contract: one row per seq).
static void markSensor(const DriverRun* run, Health health, MonoMs now) {
  const SensorId s = sensorDriverId(run->driver);
  for (uint8_t m = 0; m < METRIC_COUNT; ++m) {
    if (sensorOfMetric((MetricId)m) != s) continue;
    Reading* r = &s_work.readings[m];
    r->health = health;
    r->intervalStartMs = now;
    r->intervalEndMs = now;
    r->powerEpoch = railStateRead().epoch;
    ++r->sequence;
  }
}

// Stamp, store, check rules, for every reading the driver returned.
static void storeResults(DriverRun* run, MonoMs now, bool guardTimeout) {
  Reading out[4];
  const uint8_t n = sensorDriverReadResult(run->driver, out, 4);
  const RailState rail = railStateRead();
  const uint32_t epoch = rail.epoch;
  // The rail went down (or down and up) during the measurement: the number
  // may come from a collapsing supply, so it is never published (§14).
  const bool railLost = onFiveVoltRail(run) && (!rail.valid5v || rail.epoch !=
> run->startEpoch);
  for (uint8_t k = 0; k < n; ++k) {
    Reading* r = &out[k];
    if (r->metric >= METRIC_COUNT) continue;
    if (guardTimeout) {  // abandoned at its deadline: a new, failed result
      r->health = HEALTH_TIMEOUT;
      r->qualityFlags = (uint16_t)(r->qualityFlags | kQualityLate);
      r->intervalEndMs = now;
    }
    if (railLost) r->health = HEALTH_UNPOWERED;
```

Return to guided explanation, p. 95

ORIGINAL SOURCE / CHAPTER 3 / 5 OF 15

src/services/acquisition.c

Original lines 200-251 | Code and comments unchanged; long lines wrap visually

storeResults (L183-212): Reject a 5V window with changed/invalid rail history; assign a new observation sequence, stamp the applied revision and publish.
noteFailure (L214-219): Log a failure only when its status changes, reducing repeated messages during the same episode.

```
    // Every stored result is a new observation, failures included: the
    // drivers count only usable samples, which would give a FAULT after an OK
    // the OK's sequence (and the app would drop it as a re-send).
    r->sequence = s_work.readings[r->metric].sequence + 1u;
    r->powerEpoch = epoch;
    r->configRevision = s_appliedRevision;
    s_work.readings[r->metric] = *r;
    evaluateRules(r, now);
  }
  s_work.publishedMs = now;
  s_work.powerEpoch = epoch;
  snapshotPublish(&s_work);
}

static void noteFailure(DriverRun* run, DriverStatus st, const char* what) {
  if (st == run->lastFailure) return;  // log transitions only
  run->lastFailure = st;
  logPrintf(LOG_LEVEL_WARN, "acq: %s %s -> %s", sensorDriverName(run->driver), what,
            driverStatusName(st));
}

static void noteRecovered(DriverRun* run) {
  if (run->lastFailure == DRIVER_OK) return;
  run->lastFailure = DRIVER_OK;
  logPrintf(LOG_LEVEL_INFO, "acq: %s OK", sensorDriverName(run->driver));
}

// The HC-SR04 ping holds the flash gate from start to result (flash_gate.h).
static void releasePingGate(const DriverRun* run) {
  if (run == &s_runs[RUN_HCSR04]) flashGateGiveAfterPing();
}

static void finishRun(DriverRun* run, MonoMs now, DriverStatus st) {
  run->pending = false;
  releasePingGate(run);
  if (st == DRIVER_READY) {
    noteRecovered(run);
  } else {
    noteFailure(run, st, "result");
    STATS_ADD(driverFailures, 1);
    if (st == DRIVER_NOT_PRESENT || st == DRIVER_NOT_IMPLEMENTED) {
      run->initialised = false;
      run->retryAtMs = now;  // re-init at its next due time
    }
  }
  storeResults(run, now, false);
}

static void guardTimeout(DriverRun* run, MonoMs now) {
  sensorDriverCancel(run->driver);
  run->pending = false;
  releasePingGate(run);
```

Return to guided explanation, p. 95

ORIGINAL SOURCE / CHAPTER 3 / 6 OF 15

src/services/acquisition.c

Original lines 252-302 | Code and comments unchanged; long lines wrap visually

guardTimeout (L248-255): Cancel work left unfinished at its next boundary and publish a TIMEOUT verdict with the late flag.
isBlocking (L259-259): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.

```
252   noteFailure(run, DRIVER_TIMEOUT, "guard");
253   STATS_ADD(guardTimeouts, 1);
254   storeResults(run, now, true);
255 }
256
257 // ---- Blocking rules (§9) ------------------------------------------------------
258
259 static bool isBlocking(uint32_t budgetMs) { return budgetMs > 2; }
260
261 // May a step blocking the worker for budgetMs run now? Not while a BME280
262 // conversion is pending (its poll would come too late and time out), not
263 // while an HC-SR04 echo is in flight (1-Wire masks interrupts), and only if
264 // it ends before the next BME280 slot.
265 static bool blockingAllowed(uint8_t runId, uint32_t budgetMs, MonoMs now) {
266   if (!isBlocking(budgetMs)) return true;
267   if (runId != RUN_BME280 && s_runs[RUN_BME280].pending) return false;
268   if (s_runs[RUN_HCSR04].pending) return false;
269   if (runId == RUN_BME280) return true;
270   return scheduleFitsBefore(now, budgetMs, s_jobs[JOB_BME280].nextDueMs);
271 }
272
273 // The ping must not overlap a DS18B20 call: defer it if one is due soon.
274 static bool pingAllowed(MonoMs now) {
275   const DriverRun* ds = &s_runs[RUN_DS18B20];
276   if (ds->wantStart) return false;
277   return !ds->pending || ds->nextPollMs > now + CFG_HCSR04_TIMEOUT_MS + 2;
278 }
279
280 // ---- Start and poll -----------------------------------------------------------
281
282 static void prepareInputs(uint8_t runId, MonoMs now) {
283   if (runId == RUN_SGP30) {
284     sgp30DriverSetHumidity(&s_sgp, &s_work.readings[METRIC_AIR_TEMPERATURE],
285                            &s_work.readings[METRIC_AIR_HUMIDITY], now);
286   } else if (runId == RUN_TDS) {
287     tdsDriverSetWaterTemperature(&s_tds, &s_work.readings[METRIC_WATER_TEMPERATURE]);
288   }
289 }
290
291 // Returns false if the start must wait (blocking rule); the job stays wanted.
292 static bool tryStart(uint8_t runId, MonoMs now) {
293   DriverRun* run = &s_runs[runId];
294   if (runId == RUN_HCSR04 && !pingAllowed(now)) return false;
295   if (onFiveVoltRail(run)) {
296     const RailState rail = railStateRead();
297     if (!rail.valid5v) {  // no 5 V: no init, no conversion of a dead output
298       markSensor(run, HEALTH_UNPOWERED, now);
299       run->wantStart = false;
300       return true;
301     }
302     run->startEpoch = rail.epoch;
```

Return to guided explanation, p. 93

ORIGINAL SOURCE / CHAPTER 3 / 7 OF 15

src/services/acquisition.c

Original lines 303-354 | Code and comments unchanged; long lines wrap visually

tryStart (L292-335): Check rail validity and save its epoch, respect init backoff and admission, take the ping gate if needed, then start one operation.
tryPoll (L338-349): Only advance a pending driver when its poll time and blocking rules permit it. PENDING supplies the next poll time.

```
  }
  if (!run->initialised) {
    if (!monoReached(now, run->retryAtMs)) {  // back-off: skip this period
      run->wantStart = false;
      return true;
    }
    if (!blockingAllowed(runId, run->initBudgetMs, now)) return false;
    const DriverStatus st = sensorDriverInit(run->driver);
    if (st != DRIVER_OK) {
      noteFailure(run, st, "init");
      run->retryAtMs = now + POLICY_INIT_RETRY_MS;
      run->wantStart = false;
      return true;
    }
    run->initialised = true;
    logPrintf(LOG_LEVEL_INFO, "acq: %s init OK", sensorDriverName(run->driver));
    now = monoNowMs();  // init may have taken tens of ms
  }
  if (!blockingAllowed(runId, run->stepBudgetMs, now)) return false;
  // No NVS write may overlap the echo window: defer the ping while one runs.
  if (runId == RUN_HCSR04 && !flashGateTryTakeForPing()) return false;
  prepareInputs(runId, now);
  run->wantStart = false;
  const DriverStatus st = sensorDriverStart(run->driver, now);
  if (st == DRIVER_OK) {
    run->pending = true;
    run->nextPollMs = now;
  } else {
    if (st != DRIVER_PENDING) run->initialised = false;
    finishRun(run, now, st);
  }
  return true;
}

// Returns true if the run made progress.
static bool tryPoll(uint8_t runId, MonoMs now) {
  DriverRun* run = &s_runs[runId];
  if (!run->pending || now < run->nextPollMs) return false;
  if (!blockingAllowed(runId, run->stepBudgetMs, now)) return false;
  const PollResult pr = sensorDriverPoll(run->driver, now);
  if (pr.status == DRIVER_PENDING) {
    run->nextPollMs = pr.nextPollMs > now ? pr.nextPollMs : now + 1;
    return true;
  }
  finishRun(run, monoNowMs(), pr.status);
  return true;
}

// ---- Frame and slot boundaries ------------------------------------------------

// pH is provisional (UNCALIBRATED in pH units) until the user fits it on the
// device; the compiled nominal fit is never buffer-verified (D13).
```

Return to guided explanation, p. 93

ORIGINAL SOURCE / CHAPTER 3 / 8 OF 15

src/services/acquisition.c

Original lines 355-402 | Code and comments unchanged; long lines wrap visually

phProvisional (L355-355): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.
applyCalibrations (L359-363): Copy only calibration settings to the three analog drivers; acquisition remains their sole owner.

```
355 static bool phProvisional(const ConfigBlob* cfg) { return (cfg->calOverridden & CAL_BIT_PH)
  > == 0; }
356
357 // Drivers keep a private copy of their config: hand them the applied fits.
358 // A window in progress finishes with the old fit (its x is already taken).
359 static void applyCalibrations(const ConfigBlob* cfg) {
360   phDriverSetCalibration(&s_ph, &cfg->ph, phProvisional(cfg));
361   tdsDriverSetCalibration(&s_tds, &cfg->tds);
362   lightDriverSetCalibration(&s_light, &cfg->light);
363 }
364
365 // Sensor on/off and calibration changes take effect here, between frames, by
366 // this task only: no other task ever touches a driver (§17).
367 static SensorMask allowedSensors(void) {
368   portENTER_CRITICAL(&s_leaseMux);
369   const SensorMask allowed = s_allowed;
370   portEXIT_CRITICAL(&s_leaseMux);
371   return allowed;
372 }
373
374 static void applyConfiguration(MonoMs now) {
375   configurationPeekApplied(&s_cfg);
376   if (s_cfg.revision == s_appliedRevision && !s_forceApply) return;
377   s_forceApply = false;
378   applyCalibrations(&s_cfg);
379   const SensorMask allowed = allowedSensors();
380   for (uint8_t i = 0; i < RUN_COUNT; ++i) {
381     DriverRun* run = &s_runs[i];
382     const SensorId id = sensorDriverId(run->driver);
383     const bool wanted = sensorMaskHas(s_cfg.sensorEnabled, id);
384     const bool enabled = wanted && sensorMaskHas(allowed, id);
385     if (wanted && !enabled) markSensor(run, HEALTH_UNPOWERED, now);  // low power: its rail
  > is off
386     if (!enabled && !run->disabled) {
387       if (run->pending) {
388         sensorDriverCancel(run->driver);  // no preparePowerOff: the shared I2C bus stays in
  > use
389         run->pending = false;
390         releasePingGate(run);
391       }
392       run->disabled = true;
393       run->wantStart = false;
394       logPrintf(LOG_LEVEL_INFO, "acq: %s disabled", sensorDriverName(run->driver));
395     } else if (enabled && run->disabled) {
396       run->disabled = false;
397       run->initialised = false;  // init again at its next due time
398       run->retryAtMs = now;
399       run->lastFailure = DRIVER_OK;
400       markSensor(run, HEALTH_WARMING, now);  // until its first result (§13 restore)
401       logPrintf(LOG_LEVEL_INFO, "acq: %s enabled", sensorDriverName(run->driver));
402     }
```

Return to guided explanation, p. 94

ORIGINAL SOURCE / CHAPTER 3 / 9 OF 15

src/services/acquisition.c

Original lines 403-454 | Code and comments unchanged; long lines wrap visually

applyConfiguration (L374-406): Copy the published revision, update fits and intersect the saved enabled mask with the current power allowance at a frame boundary.
offerBaseline (L408-415): At hourly opportunities, copy a ready SGP30 baseline to Configuration for rate-limited persistence.

```
403   }
404   s_appliedRevision = s_cfg.revision;
405   configurationAckApplied(s_appliedRevision);
406 }
407
408 static void offerBaseline(MonoMs now) {
409   if (now < s_nextBaselineOfferMs) return;
410   s_nextBaselineOfferMs = now + BASELINE_OFFER_PERIOD_MS;
411   uint16_t co2eq, tvoc;
412   if (!s_runs[RUN_SGP30].disabled && sgp30DriverGetBaseline(&s_sgp, &co2eq, &tvoc)) {
413     configurationOfferBaseline(co2eq, tvoc);
414   }
415 }
416
417 static void startFrame(MonoMs now) {
418   applyConfiguration(now);
419   offerBaseline(now);
420   s_sgpBaseline = sgp30DriverBaselineStatus(&s_sgp);
421   ScheduleJob* job = &s_jobs[JOB_FRAME];
422   const uint32_t lateBefore = job->late;
423   const uint32_t skippedBefore = job->skipped;
424   scheduleJobMarkRun(job, now);
425
426   // Frame guard: a one-second job still running from the last frame is
427   // abandoned, never retried inside this frame (§18).
428   for (uint8_t i = 0; i < FRAME_ORDER_COUNT; ++i) {
429     DriverRun* run = &s_runs[kFrameOrder[i]];
430     if (run->pending) guardTimeout(run, now);
431     run->wantStart = !run->disabled;
432   }
433
434   ++s_work.frameSequence;
435   s_work.publishedMs = now;
436   snapshotPublish(&s_work);
437
438   portENTER_CRITICAL(&s_statsMux);
439   ++s_stats.frames;
440   s_stats.frameLate += job->late - lateBefore;
441   s_stats.frameSkipped += job->skipped - skippedBefore;
442   s_stats.frameJitterMaxMs = job->maxLateMs;
443   portEXIT_CRITICAL(&s_statsMux);
444 }
445
446 static void startBmeSlot(MonoMs now) {
447   ScheduleJob* job = &s_jobs[JOB_BME280];
448   const uint32_t lateBefore = job->late;
449   const uint32_t skippedBefore = job->skipped;
450   scheduleJobMarkRun(job, now);
451   if (s_runs[RUN_BME280].pending) guardTimeout(&s_runs[RUN_BME280], now);
452   // A disabled BME280 keeps its slot grid: blocking steps still respect it
453   // (scheduleFitsBefore), which is conservative but never wrong.
454   s_runs[RUN_BME280].wantStart = !s_runs[RUN_BME280].disabled;
```

Return to guided explanation, p. 92

ORIGINAL SOURCE / CHAPTER 3 / 10 OF 15

src/services/acquisition.c

Original lines 455-503 | Code and comments unchanged; long lines wrap visually

startBmeSlot (L446-459): Move the 250 ms deadline and request the next fast climate measurement.
runPass (L464-481): Handle new releases first, then try BME280, due polls and ordinary starts. Repeat only while useful progress occurs, up to eight iterations.

```
  portENTER_CRITICAL(&s_statsMux);
  s_stats.bmeLate += job->late - lateBefore;
  s_stats.bmeSkipped += job->skipped - skippedBefore;
  portEXIT_CRITICAL(&s_statsMux);
}

// One pass: boundaries first, the BME280 fast lane next, then every due poll
// and the frame starts in order. Repeats while something progressed (each
// step is bounded, so the pass is too).
static void runPass(MonoMs now) {
  if (scheduleJobDue(&s_jobs[JOB_FRAME], now)) startFrame(now);
  if (scheduleJobDue(&s_jobs[JOB_BME280], now)) startBmeSlot(now);

  for (uint8_t iter = 0; iter < 8; ++iter) {
    bool progress = false;
    now = monoNowMs();
    if (s_runs[RUN_BME280].wantStart) progress |= tryStart(RUN_BME280, now);
    for (uint8_t r = 0; r < RUN_COUNT; ++r) progress |= tryPoll(r, monoNowMs());
    for (uint8_t i = 0; i < FRAME_ORDER_COUNT; ++i) {
      const uint8_t r = kFrameOrder[i];
      if (!s_runs[r].wantStart) continue;
      if (!tryStart(r, monoNowMs())) break;  // keep the order: later ones wait too
      progress = true;
    }
    if (!progress) break;
  }
}

// Next wake: the earliest job deadline or poll time that is not blocked
// (a blocked step is woken by whatever blocks it, which is in this set).
static MonoMs nextWake(MonoMs now) {
  MonoMs due = scheduleEarliestDue(s_jobs, JOB_COUNT);
  for (uint8_t r = 0; r < RUN_COUNT; ++r) {
    const DriverRun* run = &s_runs[r];
    if (!run->pending) continue;
    const MonoMs t = run->nextPollMs > now ? run->nextPollMs : now + 1;
    if (blockingAllowed(r, run->stepBudgetMs, t) && t < due) due = t;
  }
  return due;
}

// HALs begin in AcqNormal, on core 1: the echo ISR (and with it the GPIO ISR
// service the power-sense ISR shares) is installed on this core.
static void beginHals(void) {
  if (!i2cBusBegin(&s_i2c, BOARD_PIN_I2C_SDA, BOARD_PIN_I2C_SCL, BOARD_I2C_CLOCK_HZ)) {
    logPrintf(LOG_LEVEL_ERROR, "acq: I2C begin failed");
  }
  if (!oneWireBusBegin(&s_oneWire, BOARD_PIN_ONE_WIRE)) logPrintf(LOG_LEVEL_ERROR, "acq:
> 1-Wire begin failed");
  if (!adcChannelBegin(&s_adcLight, BOARD_PIN_LIGHT_ADC)) logPrintf(LOG_LEVEL_ERROR, "acq:
> light ADC failed");
```

Return to guided explanation, p. 92

ORIGINAL SOURCE / CHAPTER 3 / 11 OF 15

src/services/acquisition.c

Original lines 504-550 | Code and comments unchanged; long lines wrap visually

beginHals (L498-509): Initialize shared buses on core1 so the GPIO interrupt service and echo capture live on the intended core.
workerTask (L511-511): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.

```
504   if (!adcChannelBegin(&s_adcTds, BOARD_PIN_TDS_ADC)) logPrintf(LOG_LEVEL_ERROR, "acq: TDS
  >   ADC failed");
505   if (!adcChannelBegin(&s_adcPh, BOARD_PIN_PH_ADC)) logPrintf(LOG_LEVEL_ERROR, "acq: pH ADC
  >   failed");
506   if (!pulseCaptureBegin(&s_capture, BOARD_PIN_HCSR04_TRIG, BOARD_PIN_HCSR04_ECHO)) {
507     logPrintf(LOG_LEVEL_ERROR, "acq: HC-SR04 capture failed");
508   }
509 }
510
511 static TaskHandle_t workerTask(PowerWorker w) { return w == POWER_WORKER_LOW ? s_taskLow :
  >   s_task; }
512
513 static bool holdsLease(PowerWorker me) {
514   portENTER_CRITICAL(&s_leaseMux);
515   const bool mine = s_owner == me;
516   portEXIT_CRITICAL(&s_leaseMux);
517   return mine;
518 }
519
520 static bool quiesceRequested(void) {
521   portENTER_CRITICAL(&s_leaseMux);
522   const bool requested = s_quiesceGen != 0 && s_quiesceGen == s_leaseGen;
523   portEXIT_CRITICAL(&s_leaseMux);
524   return requested;
525 }
526
527 // New holder: join the task watchdog, restart the schedule from now (the
528 // previous holder's deadlines are not ours to count as late) and apply the
529 // configuration within the allowed sensors at the first frame.
530 static void acquireLease(PowerWorker me, MonoMs now) {
531   if (esp_task_wdt_add(NULL) != ESP_OK) logPrintf(LOG_LEVEL_ERROR, "acq: task watchdog add
  >   failed");
532   scheduleJobInit(&s_jobs[JOB_FRAME], CFG_FRAME_PERIOD_MS, CFG_FRAME_RESERVATION_MS, now);
533   scheduleJobInit(&s_jobs[JOB_BME280], CFG_BME280_PERIOD_MS, CFG_BME280_BUDGET_MS, now);
534   for (uint8_t i = 0; i < RUN_COUNT; ++i) {
535     s_runs[i].pending = false;
536     s_runs[i].wantStart = false;
537   }
538   s_forceApply = true;
539   progressMark(me == POWER_WORKER_NORMAL ? TASK_ID_ACQUISITION_NORMAL :
  >   TASK_ID_ACQUISITION_LOW_POWER);
540   logPrintf(LOG_LEVEL_INFO, "acq: %s worker holds the lease (gen %lu, sensors 0x%02x)",
541             me == POWER_WORKER_NORMAL ? "normal" : "low-power", (unsigned long)s_leaseGen,
542             (unsigned)allowedSensors());
543 }
544
545 // QUIESCE -> RELEASED (§14): nothing is left running, the 5 V interfaces are
546 // safe if their rail is down, and the worker never touches a driver again
547 // until the next grant.
548 static void releaseLease(PowerWorker me) {
549   const bool railDown = !railStateRead().valid5v;
550   for (uint8_t i = 0; i < RUN_COUNT; ++i) {
```

Return to guided explanation, p. 92

ORIGINAL SOURCE / CHAPTER 3 / 12 OF 15

src/services/acquisition.c

Original lines 551-597 | Code and comments unchanged; long lines wrap visually

releaseLease (L548-572): Cancel pending work, release the ping gate, prepare dead-rail interfaces, leave the watchdog and acknowledge the released generation.
acqTask (L574-619): Normal worker begins HALs on core1; each worker otherwise waits for a grant, runs only while holder and releases at a pass boundary.

```
551     DriverRun* run = &s_runs[i];
552     if (run->pending) {
553       sensorDriverCancel(run->driver);
554       run->pending = false;
555       releasePingGate(run);
556     }
557     run->wantStart = false;
558     if (railDown && onFiveVoltRail(run)) {
559       sensorDriverPreparePowerOff(run->driver);  // HC-SR04: TRIG held low, no drive into a
  > dead module
560       run->initialised = false;
561     }
562   }
563   snapshotPublish(&s_work);
564   esp_task_wdt_delete(NULL);  // a blocked worker owes the watchdog nothing
565   portENTER_CRITICAL(&s_leaseMux);
566   s_owner = POWER_WORKER_NONE;
567   s_releasedGen = s_leaseGen;
568   s_quiesceGen = 0;
569   portEXIT_CRITICAL(&s_leaseMux);
570   logPrintf(LOG_LEVEL_INFO, "acq: %s worker released (gen %lu)", me == POWER_WORKER_NORMAL ?
  > "normal" : "low-power",
571             (unsigned long)s_releasedGen);
572 }
573
574 static void acqTask(void* arg) {
575   const PowerWorker me = (PowerWorker)(uintptr_t)arg;
576   const TaskId id = me == POWER_WORKER_NORMAL ? TASK_ID_ACQUISITION_NORMAL :
  > TASK_ID_ACQUISITION_LOW_POWER;
577   if (me == POWER_WORKER_NORMAL) {
578     beginHals();
579     s_halsReady = true;
580     portENTER_CRITICAL(&s_leaseMux);
581     const PowerWorker owner = s_owner;
582     portEXIT_CRITICAL(&s_leaseMux);
583     if (owner == POWER_WORKER_LOW && s_taskLow != NULL) xTaskNotifyGive(s_taskLow);  //
  > granted before the HALs
584   }
585   const MonoMs t0 = monoNowMs();
586   (void)t0;
587   bool holding = false;
588
589   // Task watchdog (§18): the holder resets it after each pass. A pass happens
590   // at least every 250 ms (BME280 slot), so 5 s of silence means the task is
591   // stuck inside a call. A worker without the lease is not on the watchdog.
592   for (;;) {
593     if (!holding) {
594       ulTaskNotifyTake(pdTRUE, portMAX_DELAY);  // a grant (or a stale wake-up)
595       if (!s_halsReady || !holdsLease(me)) continue;
596       holding = true;
597       acquireLease(me, monoNowMs());
```

Return to guided explanation, p. 91

ORIGINAL SOURCE / CHAPTER 3 / 13 OF 15

src/services/acquisition.c

Original lines 598-645 | Code and comments unchanged; long lines wrap visually

acqTask (L574-619): Normal worker begins HALs on core1; each worker otherwise waits for a grant, runs only while holder and releases at a pass boundary.
acquisitionTaskHandle (L621-626): Expose the task handle for observation without transferring state ownership.

```
598     }
599     if (quiesceRequested()) {
600       releaseLease(me);
601       holding = false;
602       continue;
603     }
604     const MonoMs now = monoNowMs();
605     const MonoMs due = nextWake(now);
606     ulTaskNotifyTake(pdTRUE, due > now ? pdMS_TO_TICKS((uint32_t)(due - now)) : 0);
607     if (quiesceRequested()) continue;  // released at the top, before any new step
608     runPass(monoNowMs());
609 #ifdef FW_INJECT_STALL_ACQ
610     // Acceptance test (§19 "Stall injection"): block forever after N ms.
611     if (monoNowMs() - t0 >= (MonoMs)(FW_INJECT_STALL_ACQ)) {
612       logPrintf(LOG_LEVEL_WARN, "acq: INJECTED STALL - blocking forever");
613       vTaskDelay(portMAX_DELAY);
614     }
615 #endif
616     progressMark(id);
617     esp_task_wdt_reset();
618   }
619 }
620
621 TaskHandle_t acquisitionTaskHandle(void) {
622   portENTER_CRITICAL(&s_leaseMux);
623   const PowerWorker owner = s_owner;
624   portEXIT_CRITICAL(&s_leaseMux);
625   return owner == POWER_WORKER_LOW ? s_taskLow : s_task;
626 }
627
628 TaskHandle_t acquisitionNormalTaskHandle(void) { return s_task; }
629 TaskHandle_t acquisitionLowPowerTaskHandle(void) { return s_taskLow; }
630
631 static TaskHandle_t createWorker(TaskId id, PowerWorker w, StackType_t* stack, uint32_t
  > stackBytes,
632                                  StaticTask_t* tcb) {
633   const TaskSpec* spec = cfgTaskSpec(id);
634   if (spec->stackBytes != stackBytes) {
635     logPrintf(LOG_LEVEL_WARN, "acq: %s stack %lu B, task table says %lu B", spec->name,
  > (unsigned long)stackBytes,
636               (unsigned long)spec->stackBytes);
637   }
638   TaskHandle_t t = xTaskCreateStaticPinnedToCore(acqTask, spec->name, stackBytes,
  > (void*)(uintptr_t)w,
639                                                  spec->priority, stack, tcb, spec->core);
640   if (t != NULL) {
641     logPrintf(LOG_LEVEL_INFO, "acq: %s started (core %u, prio %u)", spec->name,
  > (unsigned)spec->core,
642               (unsigned)spec->priority);
643   }
644   return t;
645 }
```

Return to guided explanation, p. 91

ORIGINAL SOURCE / CHAPTER 3 / 14 OF 15

src/services/acquisition.c

Original lines 646-692 | Code and comments unchanged; long lines wrap visually

acquisitionInit (L647-678): Copy configured fits, construct seven drivers, stage an accepted stored SGP30 baseline and create both workers without a lease.
acquisitionGrant (L680-698): Under the lease lock, refuse an occupied lease; otherwise save worker, generation and allowed mask, then notify that worker.

```
646
647 bool acquisitionInit(void) {
648   if (s_task != NULL) return true;
649   // Applied configuration: calibrations now, the sensor mask at the first frame.
650   configurationPeekApplied(&s_cfg);
651   s_phConfig.calibration = s_cfg.ph;
652   s_phConfig.calibrationProvisional = phProvisional(&s_cfg);
653   s_tdsConfig.calibration = s_cfg.tds;
654   s_lightConfig.calibration = s_cfg.light;
655   bme280DriverConstruct(&s_bme, &s_bmeConfig);
656   sgp30DriverConstruct(&s_sgp, &s_sgpConfig);
657   ds18b20DriverConstruct(&s_ds, &s_dsConfig);
658   lightDriverConstruct(&s_light, &s_lightConfig);
659   tdsDriverConstruct(&s_tds, &s_tdsConfig);
660   phDriverConstruct(&s_ph, &s_phConfig);
661   hcsr04DriverConstruct(&s_hc, &s_hcConfig);
662   uint16_t co2eq, tvoc;
663   if (configurationStoredBaseline(&co2eq, &tvoc)) {
664     sgp30DriverSetBaseline(&s_sgp, co2eq, tvoc);  // restored at the SGP30's init
665     logPrintf(LOG_LEVEL_INFO, "acq: SGP30 baseline restored from NVS");
666   }
667   s_appliedRevision = 0;  // revisions start at 1: forces applyConfiguration() at the first
  > frame
668   s_nextBaselineOfferMs = BASELINE_OFFER_PERIOD_MS;
669   snapshotReset(&s_work);
670   memset(s_ruleState, 0, sizeof(s_ruleState));
671
672   // No lease yet: PowerCoordinator grants the first one (§14).
673   s_task = createWorker(TASK_ID_ACQUISITION_NORMAL, POWER_WORKER_NORMAL, s_stack,
  > sizeof(s_stack), &s_tcb);
674   s_taskLow = createWorker(TASK_ID_ACQUISITION_LOW_POWER, POWER_WORKER_LOW, s_stackLow,
  > sizeof(s_stackLow),
675                            &s_tcbLow);
676   if (s_taskLow == NULL) logPrintf(LOG_LEVEL_ERROR, "acq: low-power worker missing - no
  > sensing in low power");
677   return s_task != NULL;
678 }
679
680 bool acquisitionGrant(PowerWorker worker, uint32_t generation, SensorMask allowed) {
681   if (worker != POWER_WORKER_NORMAL && worker != POWER_WORKER_LOW) return false;
682   portENTER_CRITICAL(&s_leaseMux);
683   const bool free = s_owner == POWER_WORKER_NONE;
684   if (free) {
685     s_owner = worker;
686     s_leaseGen = generation;
687     s_allowed = (SensorMask)(allowed & SENSOR_MASK_ALL);
688     s_quiesceGen = 0;
689   }
690   portEXIT_CRITICAL(&s_leaseMux);
691   if (!free) {
692     logPrintf(LOG_LEVEL_ERROR, "acq: grant gen %lu refused - the lease is still held",
  > (unsigned long)generation);
```

Return to guided explanation, p. 91

ORIGINAL SOURCE / CHAPTER 3 / 15 OF 15

src/services/acquisition.c

Original lines 693-743 | Code and comments unchanged; long lines wrap visually

acquisitionGrant (L680-698): Under the lease lock, refuse an occupied lease; otherwise save worker, generation and allowed mask, then notify that worker.
acquisitionRequestQuiesce (L700-710): Accept a request only for the current generation, record it under the lease lock and wake the holder.

```
    return false;
  }
  TaskHandle_t t = workerTask(worker);
  if (t != NULL) xTaskNotifyGive(t);
  return true;
}

bool acquisitionRequestQuiesce(uint32_t generation) {
  portENTER_CRITICAL(&s_leaseMux);
  const bool current = s_owner != POWER_WORKER_NONE && s_leaseGen == generation;
  const PowerWorker owner = s_owner;
  if (current) s_quiesceGen = generation;
  portEXIT_CRITICAL(&s_leaseMux);
  if (!current) return false;
  TaskHandle_t t = workerTask(owner);
  if (t != NULL) xTaskNotifyGive(t);  // wakes it from a bounded wait
  return true;
}

uint32_t acquisitionReleasedGeneration(void) {
  portENTER_CRITICAL(&s_leaseMux);
  const uint32_t gen = s_releasedGen;
  portEXIT_CRITICAL(&s_leaseMux);
  return gen;
}

bool acquisitionHalsReady(void) { return s_halsReady; }

TaskId acquisitionActiveTaskId(void) {
  portENTER_CRITICAL(&s_leaseMux);
  const PowerWorker owner = s_owner;
  portEXIT_CRITICAL(&s_leaseMux);
  if (owner == POWER_WORKER_NORMAL) return TASK_ID_ACQUISITION_NORMAL;
  if (owner == POWER_WORKER_LOW) return TASK_ID_ACQUISITION_LOW_POWER;
  return TASK_ID_COUNT;
}

const char* acquisitionSgp30BaselineName(void) {
  switch (s_sgpBaseline) {  // contract v2 spelling (lowercase)
    case SGP30_BASELINE_LEARNING: return "learning";
    case SGP30_BASELINE_RESTORED: return "restored";
    case SGP30_BASELINE_VALID: return "valid";
    default: return "none";
  }
}

void acquisitionGetStats(AcquisitionStats* out) {
  portENTER_CRITICAL(&s_statsMux);
  *out = s_stats;
  portEXIT_CRITICAL(&s_statsMux);
}
```

Return to guided explanation, p. 91

ORIGINAL SOURCE / CHAPTER 3 / 1 OF 2

src/services/safety.h

Original lines 1-35 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // Safety: latched alarms, interlocks, sole relay-output owner.
2  // Work package WP7 (Stage 4). Architecture §11, §12; decisions in
3  // domain/safety_core (host-tested). This file only waits, copies and writes.
4  //
5  // Task: core 1, priority 6 (cfgTaskSpec(TASK_ID_SAFETY); core 1 since the D14
6  // radio-load test, away from the Wi-Fi/BLE/TCP-IP tasks on core 0). Wakes on an alarm
7  // event or every CFG_SAFETY_HEALTH_CHECK_MS. Never waits on network I/O,
8  // NVS or a mutex. Grow-lamp auto control follows POLICY_LAMP_AUTO_ENABLED.
9  #pragma once
10 #include <freertos/FreeRTOS.h>
11 #include <freertos/task.h>
12 #include <stdbool.h>
13 #include <stdint.h>
14
15 #include "domain/alarm.h"
16 #include "domain/event_journal.h"
17 #include "domain/safety_core.h"
18
19 // Creates the detail queue, status mailbox and the task. Relay stays OFF.
20 bool safetyInit(void);
21
22 // Zero-wait, safe from any task (not from an ISR). The per-rule sticky slot
23 // is always updated; the detail queue may be full, which is counted (§9).
24 // Wakes Safety. Returns false only if the detail record was dropped.
25 bool safetySubmitEvent(const AlarmEvent* event);
26
27 // Published after every Safety pass (length-1 mailbox, §8).
28 typedef struct SafetyStatus {
29   MonoMs atMs;
30   bool relayOn;
31   bool lampWantsOn;
32   LampReason lampReason;
33   bool interlock;
34   uint16_t latchedMask;
35   uint16_t activeMask;
```

Return to guided explanation, p. 58

ORIGINAL SOURCE / CHAPTER 3 / 2 OF 2

src/services/safety.h

Original lines 36-69 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
36    uint16_t staleMask;
37    uint32_t detailOverflow;  // alarm detail records dropped since boot
38    uint32_t passes;
39  } SafetyStatus;
40
41  // Zero-wait copy of the latest status. false before the first pass.
42  bool safetyStatusPeek(SafetyStatus* out);
43
44  // For Supervisor's stack check. NULL before safetyInit().
45  TaskHandle_t safetyTaskHandle(void);
46
47  // ---- WP8: bounded remote commands (§12 step 4) ------------------------------
48  // The outcome is reported from the Safety task after the step that handled
49  // the command: the callback must copy and return at once.
50  typedef void (*SafetyReplyFn)(uint32_t tag, SafetyCommandOutcome outcome, void* ctx);
51
52  // Zero wait. false = command queue full (reply "busy"). command->expiresMs
53  // is checked in Safety, never applied late. session is the transport's link
54  // generation at receive time: if sessionNow() differs when Safety handles the
55  // command, it is dropped unapplied and unanswered (NULL = no session check).
56  bool safetySubmitCommand(const SafetyCommand* command, uint32_t session, uint32_t
 >  (*sessionNow)(void),
57                           SafetyReplyFn reply, void* ctx);
58
59  // ---- WP8: alarm event journal (§11) -----------------------------------------
60  // TRIP / CLEAR / reset events with independent reader cursors (BLE, LAN).
61  // Zero wait, any task. A new cursor sees only events appended after it.
62  void safetyJournalCursorInit(JournalCursor* cursor);
63  bool safetyJournalNext(JournalCursor* cursor, JournalEvent* out, bool* gap);
64
65  #ifdef FW_ALARM_LATENCY_TEST
66  // Test build only: acquisition records the esp_timer time (us) at which it
67  // submitted an event; Safety measures until it has applied the action.
68  void safetyLatencyMarkSubmit(void);
69  #endif
```

Return to guided explanation, p. 58

GUIDED SOURCE READING / CHAPTER 3

Store, send and notify are three separate operations

Acquisition has identified an alarm transition. safetySubmitEvent must preserve enough information for the relay owner even if the detail queue is full.

Original excerpt: src/services/safety.c (p. 115) / lines 132-144. The complete unchanged file follows.

```
132    portENTER_CRITICAL(&s_stickyMux);
133    stickyBegin(slot, &kPolicyRules[e->ruleId], e);  // in flight: Safety leaves the slot alone
134    portEXIT_CRITICAL(&s_stickyMux);
135
136    const bool queued = s_detail != NULL && xQueueSend(s_detail, e, 0) == pdTRUE;  // never
  >  inside the portMUX
137    portENTER_CRITICAL(&s_stickyMux);
138    stickyEnd(slot, queued);
139    if (!queued) ++s_detailOverflow;
140    portEXIT_CRITICAL(&s_stickyMux);
141    if (s_task != NULL) xTaskNotifyGive(s_task);  // after stickyEnd(): the next pass sees the
  >  outcome
142    return queued;
143  }
144
```

stickyBegin runs while the per-rule slot is locked. It retains the decision and increments inFlight, announcing that the producer has not yet determined its queue-send outcome. The lock is released before xQueueSend: queue APIs do their own synchronization and do not belong inside this spinlock critical section.

The send uses timeout zero, so a full queue cannot block the acquisition schedule. Its bool is passed to stickyEnd under the same slot lock. A failed send marks retained detail as lost; a successful one leaves the normal queue path responsible for delivery.

Notification comes last. This lets the awakened Safety task see the completed send outcome. If Safety happens to inspect the slot earlier on its periodic wake, stickyCollect sees inFlight and leaves it for a later pass. This protocol closes the gap between independently synchronized queue and slot operations.

The false return describes detail-queue failure. It does not mean the retained TRIP vanished. An API return describes one operation's contract. The queue result must be interpreted together with the retention protocol.

GUIDED SOURCE READING / CHAPTER 3

Why a Safety pass writes the relay before reporting

The notification makes Safety Ready. Once scheduled, it must turn retained inputs into one actuator decision before spending time formatting diagnostics.

Original excerpt: src/services/safety.c (p. 115) / lines 238-253. The complete unchanged file follows.

```
238     const MonoMs now = monoNowMs();
239     const uint8_t n = drainEvents();
240     const uint8_t nCmd = drainCommands();
241     const bool haveSnap = snapshotPeek(&s_snap);
242     const RailState rail = railStateRead();
243     if (haveSnap) snapshotGateRail(&s_snap, rail.valid5v, rail.epoch);  // no 5 V value of an
  >  older epoch
244     configurationPeekApplied(&s_cfg);  // defaults (all on) if Configuration is not running
245     const SensorMask disabled = (SensorMask)(SENSOR_MASK_ALL &
  >  (SensorMask)~s_cfg.sensorEnabled);
246     const SafetyInputs in = {haveSnap ? &s_snap : NULL, s_events, n, now, s_cmds, nCmd,
  >  disabled,
247                              !rail.valid5v};
248     safetyCoreStep(&s_core, &in, &out);
249
250     const bool relayChanged = out.relayOn != relayOn;
251     if (relayChanged) {
252       relayWrite(out.relayOn);
253       relayOn = out.relayOn;
```

The task reads time, drains events and commands, then takes copies of current Snapshot, rail and configuration. snapshotGateRail invalidates old-epoch 5V data in that copy before the pure decision core can use it. This prevents a previously good number from surviving a later power failure in the consumer's interpretation.

SafetyInputs groups these observations without giving the core ownership of RTOS queues. safetyCoreStep can therefore be tested on a PC: it receives records and returns decisions instead of reaching into peripherals.

The relayChanged comparison avoids rewriting an unchanged command. When there is a change, relayWrite comes before the following logs and status reporting. The physical command does not wait for a UART record to print or a network message to leave.

Later, callbacks return command outcomes, journal records preserve processed events, and a length-one queue publishes SafetyStatus. Progress and watchdog reset follow the completed pass. Read this order as acquire inputs, decide, act, report: reporting observes the decision rather than becoming a prerequisite for it.

ORIGINAL SOURCE / CHAPTER 3 / 1 OF 8

src/services/safety.c

Original lines 1-44 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
#include "services/safety.h"

#include <esp_task_wdt.h>
#include <esp_timer.h>
#include <freertos/FreeRTOS.h>
#include <freertos/queue.h>
#include <freertos/task.h>
#include <string.h>

#include "app_config.h"
#include "diagnostics/logger.h"
#include "diagnostics/progress.h"
#include "hal/clock.h"
#include "hal/relay.h"
#include "domain/alarm_sticky.h"
#include "domain/sensor_set.h"
#include "policy_config.h"
#include "services/configuration.h"
#include "services/rail_state.h"
#include "services/snapshot_store.h"

_Static_assert(EVENT_JOURNAL_LEN == CFG_ALARM_JOURNAL_LEN, "journal length must match
app_config.h");

// ---- Sticky per-rule store (§11): survives a full detail queue -------------
// Protocol in domain/alarm_sticky.h; every call is under s_stickyMux.
static portMUX_TYPE s_stickyMux = portMUX_INITIALIZER_UNLOCKED;
static StickySlot s_sticky[POLICY_RULE_COUNT];
static volatile uint32_t s_detailOverflow;  // single writer per increment site, read for
status only

// ---- Queues and task (all static, §16) -------------------------------------
static StaticQueue_t s_detailControl;
static uint8_t s_detailStorage[CFG_ALARM_DETAIL_QUEUE_LEN * sizeof(AlarmEvent)];
static QueueHandle_t s_detail = NULL;

static StaticQueue_t s_statusControl;
static uint8_t s_statusStorage[sizeof(SafetyStatus)];
static QueueHandle_t s_status = NULL;

static StaticTask_t s_tcb;
static StackType_t s_stack[3 * 1024];  // cfgTaskSpec(TASK_ID_SAFETY)->stackBytes
static TaskHandle_t s_task = NULL;

// ---- WP8: command queue and event journal ----------------------------------
typedef struct SafetyRequest {
```

Return to guided explanation, p. 113

ORIGINAL SOURCE / CHAPTER 3 / 2 OF 8

src/services/safety.c

Original lines 45-90 | Code and comments unchanged; long lines wrap visually

safetyLatencyMarkSubmit (L84-89): Diagnostic build only: save the submission microsecond timestamp under a lock.

```
45    SafetyCommand command;
46    uint32_t session;              // transport session generation at receive time
47    uint32_t (*sessionNow)(void);  // NULL = no session check
48    SafetyReplyFn reply;
49    void* ctx;
50  } SafetyRequest;
51
52  static StaticQueue_t s_commandControl;
53  static uint8_t s_commandStorage[SAFETY_MAX_COMMANDS * sizeof(SafetyRequest)];
54  static QueueHandle_t s_commands = NULL;
55
56  static portMUX_TYPE s_journalMux = portMUX_INITIALIZER_UNLOCKED;
57  static EventJournal s_journal;
58
59  // Task-private state: static, never on the 3 KiB stack.
60  static SafetyRequest s_requests[SAFETY_MAX_COMMANDS];
61  static SafetyCommand s_cmds[SAFETY_MAX_COMMANDS];
62  static ConfigBlob s_cfg;
63  static SafetyCore s_core;
64  static Snapshot s_snap;
65  static AlarmEvent s_events[CFG_ALARM_DETAIL_QUEUE_LEN + 2 * POLICY_RULE_COUNT];
66  static SafetyStatus s_statusOut;
67
68  #ifdef FW_ALARM_LATENCY_TEST
69  // ---- Alarm latency test (env:alarm_latency, §19) ----------------------------
70  // submit -> action: esp_timer us from acquisition's submit to the end of the
71  //                   Safety pass that applied the decision (relay written).
72  // sample -> action: ms from the sample's intervalEndMs (BME280 read done).
73  static portMUX_TYPE s_latMux = portMUX_INITIALIZER_UNLOCKED;
74  static int64_t s_submitUs;  // 0 = no event waiting
75
76  typedef struct LatencyStats {
77    uint32_t n;
78    int64_t minUs, maxUs, sumUs;
79    uint32_t minMs, maxMs, sumMs;
80  } LatencyStats;
81  static LatencyStats s_lat;
82  static uint32_t s_latReports;
83
84  void safetyLatencyMarkSubmit(void) {
85    const int64_t t = esp_timer_get_time();
86    portENTER_CRITICAL(&s_latMux);
87    s_submitUs = t;
88    portEXIT_CRITICAL(&s_latMux);
89  }
90
```

Return to guided explanation, p. 113

ORIGINAL SOURCE / CHAPTER 3 / 3 OF 8

src/services/safety.c

Original lines 91-131 | Code and comments unchanged; long lines wrap visually

latencyRecord (L91-124): Diagnostic build only: measure event-to-action timing after the relay decision has been applied.
safetySubmitEvent (L127-143): Mark sticky data in flight, attempt a zero-wait queue send outside the lock, finish retention and notify Safety afterwards.

```
91  static void latencyRecord(const AlarmEvent* events, uint8_t n) {
92    const AlarmEvent* trip = NULL;
93    for (uint8_t i = 0; i < n; ++i) {
94      if (events[i].decision == RULE_DECISION_TRIP) trip = &events[i];
95    }
96    portENTER_CRITICAL(&s_latMux);
97    const int64_t submitted = s_submitUs;
98    s_submitUs = 0;
99    portEXIT_CRITICAL(&s_latMux);
100   if (trip == NULL || submitted == 0) return;
101
102   const int64_t us = esp_timer_get_time() - submitted;
103   const MonoMs now = monoNowMs();
104   const uint32_t ms = now > trip->sample.intervalEndMs ? (uint32_t)(now -
  > trip->sample.intervalEndMs) : 0;
105   if (ms > 5 || us > 5000) {  // one slow trip: when exactly (the 20-trip summary hides it)
106     logPrintf(LOG_LEVEL_WARN, "latency SPIKE at t=%lu ms: submit->action %lld us,
  > sample->action %lu ms",
107               (unsigned long)now, (long long)us, (unsigned long)ms);
108   }
109   if (s_lat.n == 0 || us < s_lat.minUs) s_lat.minUs = us;
110   if (s_lat.n == 0 || us > s_lat.maxUs) s_lat.maxUs = us;
111   if (s_lat.n == 0 || ms < s_lat.minMs) s_lat.minMs = ms;
112   if (s_lat.n == 0 || ms > s_lat.maxMs) s_lat.maxMs = ms;
113   s_lat.sumUs += us;
114   s_lat.sumMs += ms;
115   if (++s_lat.n == 20) {
116     logPrintf(LOG_LEVEL_INFO,
117               "latency #%lu (20 TRIPs): submit->action us min %lld avg %lld max %lld | "
118               "sample->action ms min %lu avg %lu max %lu",
119               (unsigned long)++s_latReports, (long long)s_lat.minUs, (long long)(s_lat.sumUs
  > / 20),
120               (long long)s_lat.maxUs, (unsigned long)s_lat.minMs, (unsigned long)(s_lat.sumMs
  > / 20),
121               (unsigned long)s_lat.maxMs);
122     memset(&s_lat, 0, sizeof(s_lat));
123   }
124 }
125 #endif
126
127 bool safetySubmitEvent(const AlarmEvent* e) {
128   if (e == NULL || e->ruleId >= POLICY_RULE_COUNT) return false;
129   if (e->decision != RULE_DECISION_TRIP && e->decision != RULE_DECISION_CLEAR) return false;
130   StickySlot* slot = &s_sticky[e->ruleId];
131
```

Return to guided explanation, p. 113

ORIGINAL SOURCE / CHAPTER 3 / 4 OF 8

src/services/safety.c

Original lines 132-172 | Code and comments unchanged; long lines wrap visually

safetySubmitEvent (L127-143): Mark sticky data in flight, attempt a zero-wait queue send outside the lock, finish retention and notify Safety afterwards.
drainEvents (L151-165): Drain detail records, then collect dropped per-rule state only when its producer send outcome is no longer in flight.

```
132   portENTER_CRITICAL(&s_stickyMux);
133   stickyBegin(slot, &kPolicyRules[e->ruleId], e);  // in flight: Safety leaves the slot alone
134   portEXIT_CRITICAL(&s_stickyMux);
135
136   const bool queued = s_detail != NULL && xQueueSend(s_detail, e, 0) == pdTRUE;  // never
  >  inside the portMUX
137   portENTER_CRITICAL(&s_stickyMux);
138   stickyEnd(slot, queued);
139   if (!queued) ++s_detailOverflow;
140   portEXIT_CRITICAL(&s_stickyMux);
141   if (s_task != NULL) xTaskNotifyGive(s_task);  // after stickyEnd(): the next pass sees the
  >  outcome
142   return queued;
143 }
144
145 // Drain the detail queue, then, only for a rule whose detail record was
146 // dropped, let its sticky slot re-supply what may be missing: the TRIP (if
147 // none came through the queue) and, if the rule's last decision was CLEAR,
148 // that CLEAR too, so the rule cannot stay active forever. A record that was
149 // queued arrives through the queue, so nothing is processed twice. A slot
150 // whose send is still in flight is collected on the next pass.
151 static uint8_t drainEvents(void) {
152   uint8_t n = 0;
153   while (n < CFG_ALARM_DETAIL_QUEUE_LEN && xQueueReceive(s_detail, &s_events[n], 0) ==
  >  pdTRUE) ++n;
154
155   bool sawTrip[POLICY_RULE_COUNT] = {false};
156   for (uint8_t i = 0; i < n; ++i) {
157     if (s_events[i].ruleId < POLICY_RULE_COUNT && s_events[i].decision == RULE_DECISION_TRIP)
  >  {
158       sawTrip[s_events[i].ruleId] = true;
159     }
160   }
161   portENTER_CRITICAL(&s_stickyMux);
162   n = (uint8_t)(n + stickyCollect(s_sticky, POLICY_RULE_COUNT, sawTrip, &s_events[n]));
163   portEXIT_CRITICAL(&s_stickyMux);
164   return n;
165 }
166
167 bool safetySubmitCommand(const SafetyCommand* command, uint32_t session, uint32_t
  >  (*sessionNow)(void),
168                          SafetyReplyFn reply, void* ctx) {
169   if (s_commands == NULL || command == NULL) return false;
170   const SafetyRequest request = {*command, session, sessionNow, reply, ctx};
171   if (xQueueSend(s_commands, &request, 0) != pdTRUE) return false;
172   if (s_task != NULL) xTaskNotifyGive(s_task);
```

Return to guided explanation, p. 113

ORIGINAL SOURCE / CHAPTER 3 / 5 OF 8

src/services/safety.c

Original lines 173-216 | Code and comments unchanged; long lines wrap visually

safetySubmitCommand (L167-174): Copy a reset request, session checker, callback and tag to a bounded queue and wake Safety.
safetyJournalCursorInit (L176-180): Check dependencies, configure this module and return whether initialisation succeeded.

```
  return true;
}

void safetyJournalCursorInit(JournalCursor* cursor) {
  portENTER_CRITICAL(&s_journalMux);
  journalCursorInit(&s_journal, cursor);
  portEXIT_CRITICAL(&s_journalMux);
}

bool safetyJournalNext(JournalCursor* cursor, JournalEvent* out, bool* gap) {
  portENTER_CRITICAL(&s_journalMux);
  const bool got = journalNext(&s_journal, cursor, out, gap);
  portEXIT_CRITICAL(&s_journalMux);
  return got;
}

static void journal(uint8_t ruleId, JournalKind kind, MonoMs atMs, float value) {
  JournalEvent e;
  memset(&e, 0, sizeof(e));
  e.ruleId = ruleId;
  e.severity = ruleId < POLICY_RULE_COUNT ? kPolicyRules[ruleId].severity : SEVERITY_INFO;
  e.kind = kind;
  e.atMs = atMs;
  e.value = value;
  portENTER_CRITICAL(&s_journalMux);
  journalAppend(&s_journal, &e);
  portEXIT_CRITICAL(&s_journalMux);
}

// A request from a link that has since closed (or reconnected) is dropped
// unapplied and unanswered, as Configuration does (§11).
static uint8_t drainCommands(void) {
  uint8_t n = 0;
  while (n < SAFETY_MAX_COMMANDS && xQueueReceive(s_commands, &s_requests[n], 0) == pdTRUE) {
    const SafetyRequest* r = &s_requests[n];
    if (r->sessionNow != NULL && r->sessionNow() != r->session) {
      logPrintf(LOG_LEVEL_INFO, "safety: rule %u reset from a closed session dropped",
(unsigned)r->command.ruleId);
      continue;
    }
    s_cmds[n] = r->command;
    ++n;
  }
  return n;
}
```

Return to guided explanation, p. 113

ORIGINAL SOURCE / CHAPTER 3 / 6 OF 8

src/services/safety.c

Original lines 217-258 | Code and comments unchanged; long lines wrap visually

safetyTask (L218-302): Drain events/commands, gate a snapshot against live power, step the pure core, write changed relay state and publish/journal outcomes.

```
217
218 static void safetyTask(void* arg) {
219   (void)arg;
220   SafetyOutputs out;
221   bool relayOn = false;
222   LampReason lastReason = LAMP_REASON_NO_DATA;
223   uint32_t passes = 0;
224
225   // Task watchdog (§18): reset by this task only, after each completed pass
226   // (at least every 100 ms). Supervisor never resets it on Safety's behalf.
227   if (esp_task_wdt_add(NULL) != ESP_OK) logPrintf(LOG_LEVEL_ERROR, "safety: task watchdog add
 > failed");
228
229   for (;;) {
230     ulTaskNotifyTake(pdTRUE, pdMS_TO_TICKS(CFG_SAFETY_HEALTH_CHECK_MS));
231 #ifdef FW_INJECT_STALL_SAFETY
232     // Acceptance test (§19 "Stall injection"): block forever after N ms.
233     if (monoNowMs() >= (MonoMs)(FW_INJECT_STALL_SAFETY)) {
234       logPrintf(LOG_LEVEL_WARN, "safety: INJECTED STALL - blocking forever");
235       vTaskDelay(portMAX_DELAY);
236     }
237 #endif
238     const MonoMs now = monoNowMs();
239     const uint8_t n = drainEvents();
240     const uint8_t nCmd = drainCommands();
241     const bool haveSnap = snapshotPeek(&s_snap);
242     const RailState rail = railStateRead();
243     if (haveSnap) snapshotGateRail(&s_snap, rail.valid5v, rail.epoch);  // no 5 V value of an
 > older epoch
244     configurationPeekApplied(&s_cfg);  // defaults (all on) if Configuration is not running
245     const SensorMask disabled = (SensorMask)(SENSOR_MASK_ALL &
 > (SensorMask)~s_cfg.sensorEnabled);
246     const SafetyInputs in = {haveSnap ? &s_snap : NULL, s_events, n, now, s_cmds, nCmd,
 > disabled,
247                              !rail.valid5v};
248     safetyCoreStep(&s_core, &in, &out);
249
250     const bool relayChanged = out.relayOn != relayOn;
251     if (relayChanged) {
252       relayWrite(out.relayOn);
253       relayOn = out.relayOn;
254     }
255 #ifdef FW_ALARM_LATENCY_TEST
256     latencyRecord(s_events, n);  // the action is applied: stop the clock
257 #endif
258     if (relayChanged) {
```

Return to guided explanation, p. 114

ORIGINAL SOURCE / CHAPTER 3 / 7 OF 8

src/services/safety.c

Original lines 259-300 | Code and comments unchanged; long lines wrap visually

safetyTask (L218-302): Drain events/commands, gate a snapshot against live power, step the pure core, write changed relay state and publish/journal outcomes.

```
259       logPrintf(LOG_LEVEL_INFO, "safety: relay %s (%s)", relayOn ? "ON" : "OFF",
260                 lampReasonName(out.lampReason));
261     } else if (out.lampReason != lastReason &&
262                (out.lampReason == LAMP_REASON_STALE || out.lampReason == LAMP_REASON_FAULT ||
263                 out.lampReason == LAMP_REASON_INTERLOCK)) {
264       logPrintf(LOG_LEVEL_WARN, "safety: lamp %s", lampReasonName(out.lampReason));
265     }
266     lastReason = out.lampReason;
267     for (uint8_t i = 0; i < out.resultCount && i < nCmd; ++i) {
268       if (out.results[i].outcome == SAFETY_CMD_APPLIED) {
269         journal(s_cmds[i].ruleId, JOURNAL_KIND_RESET, now, 0.0f);
270         logPrintf(LOG_LEVEL_INFO, "safety: rule %u latch reset", (unsigned)s_cmds[i].ruleId);
271       }
272       if (s_requests[i].reply != NULL) {
273         s_requests[i].reply(out.results[i].tag, out.results[i].outcome, s_requests[i].ctx);
274       }
275     }
276     for (uint8_t i = 0; i < n; ++i) {
277 #ifdef FW_ALARM_LATENCY_TEST
278       if (s_events[i].decision == RULE_DECISION_TRIP && s_events[i].ruleId == 0) continue;
  > // 4 per s
279 #endif
280       journal(s_events[i].ruleId,
281               s_events[i].decision == RULE_DECISION_TRIP ? JOURNAL_KIND_TRIP :
  > JOURNAL_KIND_CLEAR,
282               s_events[i].sample.intervalEndMs, s_events[i].sample.value);
283       logPrintf(LOG_LEVEL_WARN, "safety: rule %u %s (%s = %.2f)",
  > (unsigned)s_events[i].ruleId,
284                 s_events[i].decision == RULE_DECISION_TRIP ? "TRIP" : "CLEAR",
285                 metricName(s_events[i].sample.metric), (double)s_events[i].sample.value);
286     }
287
288     s_statusOut.atMs = now;
289     s_statusOut.relayOn = relayOn;
290     s_statusOut.lampWantsOn = out.lampWantsOn;
291     s_statusOut.lampReason = out.lampReason;
292     s_statusOut.interlock = out.interlock;
293     s_statusOut.latchedMask = out.latchedMask;
294     s_statusOut.activeMask = out.activeMask;
295     s_statusOut.staleMask = out.staleMask;
296     s_statusOut.detailOverflow = s_detailOverflow;
297     s_statusOut.passes = ++passes;
298     xQueueOverwrite(s_status, &s_statusOut);
299     progressMark(TASK_ID_SAFETY);
300     esp_task_wdt_reset();
```

Return to guided explanation, p. 114

ORIGINAL SOURCE / CHAPTER 3 / 8 OF 8

src/services/safety.c

Original lines 301-343 | Code and comments unchanged; long lines wrap visually

safetyTask (L218-302): Drain events/commands, gate a snapshot against live power, step the pure core, write changed relay state and publish/journal outcomes.
safetyTaskHandle (L304-304): Expose the task handle for observation without transferring state ownership.

```
301   }
302 }
303
304 TaskHandle_t safetyTaskHandle(void) { return s_task; }
305
306 bool safetyInit(void) {
307   if (s_task != NULL) return true;
308   s_detail = xQueueCreateStatic(CFG_ALARM_DETAIL_QUEUE_LEN, sizeof(AlarmEvent),
  >  s_detailStorage,
309                                 &s_detailControl);
310   s_status = xQueueCreateStatic(1, sizeof(SafetyStatus), s_statusStorage, &s_statusControl);
311   s_commands = xQueueCreateStatic(SAFETY_MAX_COMMANDS, sizeof(SafetyRequest),
  >  s_commandStorage,
312                                   &s_commandControl);
313   if (s_detail == NULL || s_status == NULL || s_commands == NULL) return false;
314   journalInit(&s_journal);
315
316   // Lamp policy from the applied configuration (compiled defaults in 8.2).
317   configurationPeekApplied(&s_cfg);
318   SafetyConfig config;
319   memset(&config, 0, sizeof(config));
320   config.lampAutoEnabled = s_cfg.lampAutoEnabled;
321   config.lamp = s_cfg.lamp;
322   config.lightStaleMs = POLICY_LIGHT_STALE_MS;
323   safetyCoreInit(&s_core, &config, kPolicyRules, POLICY_RULE_COUNT, monoNowMs());
324   relayWrite(false);
325
326   const TaskSpec* spec = cfgTaskSpec(TASK_ID_SAFETY);
327   if (spec->stackBytes != sizeof(s_stack)) {
328     logPrintf(LOG_LEVEL_WARN, "safety: stack %u B, task table says %lu B",
329               (unsigned)sizeof(s_stack), (unsigned long)spec->stackBytes);
330   }
331   const BaseType_t safetyCore = spec->core;
332   s_task = xTaskCreateStaticPinnedToCore(safetyTask, spec->name, sizeof(s_stack), NULL,
333                                          spec->priority, s_stack, &s_tcb, safetyCore);
334   if (s_task == NULL) return false;
335   logPrintf(LOG_LEVEL_INFO, "safety: started (core %u, prio %u), lamp auto %s, relay OFF",
336             (unsigned)safetyCore, (unsigned)spec->priority, s_cfg.lampAutoEnabled ? "ON" :
  >  "off");
337   return true;
338 }
339
340 bool safetyStatusPeek(SafetyStatus* out) {
341   if (s_status == NULL || out == NULL) return false;
342   return xQueuePeek(s_status, out, 0) == pdTRUE;
343 }
```

Return to guided explanation, p. 114

ORIGINAL SOURCE / CHAPTER 3 / 1 OF 1

src/diagnostics/logger.h

Original lines 1-22 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // Logger: bounded queue to the serial console, drop-and-count, never blocks
2  // callers. Work package WP10 (Stage 4). Architecture §9, §16.
3  //
4  // logPrintf() formats into a fixed record (LOG_TEXT_LEN, longer text is cut)
5  // and sends it with a zero timeout. The SerialLogger task (core 1, priority
6  // 1) prints it, so it only runs while acquisition is Blocked. Before the task
7  // exists (and in bring-up builds, which never call logInit()) logPrintf()
8  // prints directly. Never call it from an ISR.
9  #pragma once
10 #include <stdint.h>
11
12 #define LOG_TEXT_LEN 128
13
14 typedef unsigned char LogLevel;
15 enum { LOG_LEVEL_ERROR = 0, LOG_LEVEL_WARN, LOG_LEVEL_INFO, LOG_LEVEL_DEBUG };
16
17 void logInit(void);  // creates the queue and the SerialLogger task
18 void logPrintf(LogLevel level, const char* fmt, ...)
19     __attribute__((format(printf, 2, 3)));
20 uint32_t logDroppedCount(void);  // records lost to a full queue since boot
21 uint32_t logQueueDepth(void);    // records waiting to be printed (0 before logInit)
22 void* logTaskHandle(void);       // SerialLogger TaskHandle_t, NULL before logInit()
```

Return to guided explanation, p. 58

GUIDED SOURCE READING / CHAPTER 3

Why formatting and printing have different owners

Safety has acted and now wants a diagnostic message. A direct UART print in that path could wait for output, so the logger separates producing a bounded record from transmitting it.

Original excerpt: src/diagnostics/logger.c (p. 125) / lines 66-77. The complete unchanged file follows.

```
66  void logPrintf(LogLevel level, const char* fmt, ...) {
67    LogRecord rec;
68    rec.level = level;
69    va_list args;
70    va_start(args, fmt);
71    vsnprintf(rec.text, sizeof(rec.text), fmt, args);
72    va_end(args);
73
74    if (s_task == NULL) {  // early boot or bring-up: direct, blocking write
75      printf("[%s] %s\n", tagOf(level), rec.text);
76      return;
77    }
```

The local LogRecord has fixed storage. The level is assigned, variadic arguments are opened with va_start, and vsnprintf writes at most the array's capacity. va_end closes that argument traversal. Bounding the buffer prevents a long message from overwriting adjacent memory; it does not make formatting free of CPU cost.

The later queue send uses zero wait. Success transfers a copy to SerialLogger; failure increments the drop counter. No pointer to the local record is retained after logPrintf returns, so the stack record's lifetime is sufficient for the queue copy.

SerialLogger eventually receives and prints the record at low priority. Placing the print there lets higher-priority Safety preempt it on core1. The early-startup fallback prints directly when no logger queue exists, so that fallback has different timing from the normal queued path.

This design chooses bounded producer delay over complete log history. The dropped-record counter makes that choice visible. Alarm retention uses a different mechanism because a lost debug sentence and a lost relay TRIP have different consequences.

ORIGINAL SOURCE / CHAPTER 3 / 1 OF 2

src/diagnostics/logger.c

Original lines 1-46 | Code and comments unchanged; long lines wrap visually

tagOf (L28-31): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.
serialLoggerTask (L33-51): Wait for a record, print it, report drops and mark output progress.

```
1  #include "diagnostics/logger.h"
2
3  #include <freertos/FreeRTOS.h>
4  #include <freertos/queue.h>
5  #include <freertos/task.h>
6  #include <stdarg.h>
7  #include <stdio.h>
8
9  #include "app_config.h"
10 #include "diagnostics/progress.h"
11
12 typedef struct LogRecord {
13   LogLevel level;
14   char text[LOG_TEXT_LEN];
15 } LogRecord;
16
17 static StaticQueue_t s_queueControl;
18 static uint8_t s_queueStorage[CFG_LOG_QUEUE_LEN * sizeof(LogRecord)];
19 static QueueHandle_t s_queue = NULL;
20
21 static StaticTask_t s_tcb;
22 static StackType_t s_stack[2 * 1024];  // cfgTaskSpec(TASK_ID_SERIAL_LOGGER)->stackBytes
23 static TaskHandle_t s_task = NULL;
24
25 static portMUX_TYPE s_dropMux = portMUX_INITIALIZER_UNLOCKED;
26 static uint32_t s_dropped = 0;
27
28 static const char* tagOf(LogLevel level) {
29   static const char* const kTags[] = {"E", "W", "I", "D"};
30   return level < sizeof(kTags) / sizeof(kTags[0]) ? kTags[level] : "?";
31 }
32
33 static void serialLoggerTask(void* arg) {
34   (void)arg;
35   static LogRecord rec;  // single consumer: static keeps it off the 2 KiB stack
36   uint32_t reportedDrops = 0;
37   for (;;) {
38     if (xQueueReceive(s_queue, &rec, portMAX_DELAY) != pdTRUE) continue;
39     fputs("[", stdout);
40     fputs(tagOf(rec.level), stdout);
41     fputs("] ", stdout);
42     fputs(rec.text, stdout);
43     fputs("\n", stdout);
44     const uint32_t dropped = logDroppedCount();
45     if (dropped != reportedDrops) {  // say so in the log itself, once per change
46       printf("[W] logger: %lu records dropped (queue full)\n", (unsigned long)dropped);
```

Return to guided explanation, p. 124

ORIGINAL SOURCE / CHAPTER 3 / 2 OF 2

src/diagnostics/logger.c

Original lines 47-90 | Code and comments unchanged; long lines wrap visually

serialLoggerTask (L33-51): Wait for a record, print it, report drops and mark output progress.
logQueueDepth (L53-53): Report waiting log records; this helps distinguish a quiet logger from a stalled one.

```
      reportedDrops = dropped;
    }
    progressMark(TASK_ID_SERIAL_LOGGER);
  }
}

uint32_t logQueueDepth(void) { return s_queue != NULL ?
> (uint32_t)uxQueueMessagesWaiting(s_queue) : 0; }

void* logTaskHandle(void) { return s_task; }

void logInit(void) {
  if (s_task != NULL) return;
  s_queue = xQueueCreateStatic(CFG_LOG_QUEUE_LEN, sizeof(LogRecord), s_queueStorage,
> &s_queueControl);
  if (s_queue == NULL) return;  // logPrintf keeps printing directly
  const TaskSpec* spec = cfgTaskSpec(TASK_ID_SERIAL_LOGGER);
  s_task = xTaskCreateStaticPinnedToCore(serialLoggerTask, spec->name, sizeof(s_stack), NULL,
                                         spec->priority, s_stack, &s_tcb, spec->core);
}

void logPrintf(LogLevel level, const char* fmt, ...) {
  LogRecord rec;
  rec.level = level;
  va_list args;
  va_start(args, fmt);
  vsnprintf(rec.text, sizeof(rec.text), fmt, args);
  va_end(args);

  if (s_task == NULL) {  // early boot or bring-up: direct, blocking write
    printf("[%s] %s\n", tagOf(level), rec.text);
    return;
  }
  if (xQueueSend(s_queue, &rec, 0) != pdTRUE) {
    portENTER_CRITICAL(&s_dropMux);
    ++s_dropped;
    portEXIT_CRITICAL(&s_dropMux);
  }
}

uint32_t logDroppedCount(void) {
  portENTER_CRITICAL(&s_dropMux);
  const uint32_t n = s_dropped;
  portEXIT_CRITICAL(&s_dropMux);
  return n;
}
```

Return to guided explanation, p. 124

ORIGINAL SOURCE / CHAPTER 3 / 1 OF 1

src/diagnostics/progress.h

Original lines 1-8 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // Per-task progress timestamps for the progress watchdog.
2  // Work package WP10 (Stage 4). Architecture §16.
3  #pragma once
4  #include "app_config.h"
5  #include "domain/types.h"
6
7  void progressMark(TaskId task);  // call after meaningful work
8  MonoMs progressLastMs(TaskId task);
```

Return to guided explanation, p. 58

GUIDED SOURCE READING / CHAPTER 3

A coherent timestamp needs a very short lock

The worker has finished its pass. progressMark records that fact for Supervisor, which may execute on the other core.

Original excerpt: src/diagnostics/progress.c (p. 129) / lines 14-20. The complete unchanged file follows.

```
14  void progressMark(TaskId task) {
15    if (task >= TASK_ID_COUNT) return;
16    const MonoMs now = monoNowMs();
17    portENTER_CRITICAL(&s_mux);
18    s_lastMs[task] = now;
19    portEXIT_CRITICAL(&s_mux);
20  }
```

The time is obtained before entering the lock, keeping the protected interval focused on a small assignment. The lock then protects the array element from a concurrent reader. MonoMs is 64 bits on a 32-bit processor, so an uncoordinated read could otherwise combine halves from different updates.

portEXIT_CRITICAL must follow the write before the function returns. Keeping the lock while logging or waiting would block the other core from reading progress and extend local interrupt masking. Nothing in this short assignment needs that extra duration.

progressLastMs takes the same lock around its copy and returns after releasing it. Sharing a lock, not merely declaring volatile, is what makes writer and reader agree on one complete timestamp. The protected value records when work last returned; it cannot prove that every sensor result was valid.

This distinction explains Supervisor's role later: it detects absence of completed service activity, while Reading.health reports measurement quality. A controller can be progressing normally while reporting a missing sensor.

ORIGINAL SOURCE / CHAPTER 3 / 1 OF 1

src/diagnostics/progress.c

Original lines 1-28 | Code and comments unchanged; long lines wrap visually

progressMark (L14-20): Save a completed-work timestamp under a lock so the other core cannot read a torn 64-bit value.
progressLastMs (L22-28): Return one coherent timestamp for the task Supervisor is checking.

```
1  #include "diagnostics/progress.h"
2
3  #include <freertos/FreeRTOS.h>
4
5  #include "hal/clock.h"
6
7  // 64-bit stamps are written as two 32-bit halves: a reader on the other core
8  // could see a torn value without the lock (§11).
9  // The acquisition worker without the lease marks nothing; Supervisor checks
10 // only the lease holder (acquisitionActiveTaskId(), WP9).
11 static portMUX_TYPE s_mux = portMUX_INITIALIZER_UNLOCKED;
12 static MonoMs s_lastMs[TASK_ID_COUNT];
13
14 void progressMark(TaskId task) {
15   if (task >= TASK_ID_COUNT) return;
16   const MonoMs now = monoNowMs();
17   portENTER_CRITICAL(&s_mux);
18   s_lastMs[task] = now;
19   portEXIT_CRITICAL(&s_mux);
20 }
21
22 MonoMs progressLastMs(TaskId task) {
23   if (task >= TASK_ID_COUNT) return 0;
24   portENTER_CRITICAL(&s_mux);
25   const MonoMs t = s_lastMs[task];
26   portEXIT_CRITICAL(&s_mux);
27   return t;
28 }
```

Return to guided explanation, p. 128

ORIGINAL SOURCE / CHAPTER 3 / 1 OF 1

src/services/supervisor.h

Original lines 1-41 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // Supervisor: progress deadlines, counters, stack/heap health, every 1 s.
2  // Work package WP10 (Stage 4). Architecture §16, §18.
3  //
4  // Core 0, priority 1: it only observes and reports. It never resets the task
5  // watchdog for another task (AcqNormal and Safety reset it for themselves).
6  //   every 1 s   progress age per watched task (warn once per stall episode,
7  //               "progressing again" on recovery); loss/lateness counters
8  //               (frame/BME280 late and skipped, guard timeouts, alarm detail
9  //               overflow, log drops): one WARN line with the deltas
10 //   every 10 s  stack high-water marks and internal heap: WARN when low
11 //   every 60 s  the same as an INFO report
12 //   every 5 s   status line: readings, lamp, counters, "health OK" or
13 //               "DEGRADED <first cause>" for the last 5 s
14 //   boot        reset reason (a TASK_WDT here means a watched task stalled)
15 #pragma once
16 #include <stdbool.h>
17
18 #define SUPERVISOR_STATUS_EVERY_S 5
19 #define SUPERVISOR_MEMORY_EVERY_S 10
20 #define SUPERVISOR_MEMORY_REPORT_EVERY_S 60
21
22 // Progress deadlines. Safety passes every 100 ms, AcqNormal at least every
23 // 250 ms; the task watchdog (5 s) fires well after these warnings.
24 #define SUPERVISOR_SAFETY_DEADLINE_MS 1000u
25 #define SUPERVISOR_ACQ_DEADLINE_MS 2000u
26 #define SUPERVISOR_LOGGER_DEADLINE_MS 2000u
27 // Configuration wakes every 500 ms and marks progress while it waits for a
28 // flash-gate slot or for acquisition to apply a change (<= 3 s).
29 #define SUPERVISOR_CONFIG_DEADLINE_MS 3000u
30 // BleTransport wakes every 50 ms; one frame send retries for at most ~200 ms per chunk.
31 #define SUPERVISOR_BLE_DEADLINE_MS 2000u
32 // WifiMqtt wakes every 50 ms; every socket call is non-blocking.
33 #define SUPERVISOR_WIFI_DEADLINE_MS 2000u
34 // PowerCoord wakes at least every 100 ms (WP9).
35 #define SUPERVISOR_POWER_DEADLINE_MS 1000u
36 #define SUPERVISOR_BOOT_GRACE_MS 5000u
37
38 #define SUPERVISOR_STACK_WARN_BYTES 512u  // §16: keep >= 25 % headroom; warn below this
39 #define SUPERVISOR_HEAP_WARN_BYTES 20000u // internal heap minimum ever
40
41 bool supervisorInit(void);
```

Return to guided explanation, p. 58

GUIDED SOURCE READING / CHAPTER 3

Observe the active worker rather than both task handles

Both acquisition tasks exist, but only one owes measurement progress. Supervisor translates the table's acquisition entry into the current lease holder before computing an age.

Original excerpt: src/services/supervisor.c (p. 132) / lines 60-73. The complete unchanged file follows.

```
60      if (id == TASK_ID_ACQUISITION_NORMAL) {
61        // WP9: whichever worker holds the lease owes progress; nobody during
62        // the hand-over (PowerCoordinator reports a stuck one as DEGRADED).
63        id = acquisitionActiveTaskId();
64        if (id == TASK_ID_COUNT) continue;
65      }
66      const MonoMs last = progressLastMs(id);  // 0 = never stamped: age from boot
67      const MonoMs age = now > last ? now - last : 0;
68      const bool idleLogger = w->onlyWhenBusy && logQueueDepth() == 0;
69      const bool late = !idleLogger && now > SUPERVISOR_BOOT_GRACE_MS && age > w->deadlineMs;
70      if (late) {
71        markDegraded(w->name);
72        if (!w->stalled) {
73          w->stalled = true;
```

acquisitionActiveTaskId selects the responsible worker. TASK_ID_COUNT means no owner during handover, so the check is skipped. Treating the inactive worker's old timestamp as a stall would report normal low-power operation as a fault.

Only after selecting the correct ID does Supervisor read its protected timestamp. now greater than last guards subtraction, while the boot grace prevents early initialization from becoming a stall report.

The logger has another conditional obligation: an empty queue can mean there is nothing to print. This differs from acquisition, which expects periodic work while holding the lease. A single universal heartbeat rule would confuse useful idleness with failure.

When late, markDegraded updates the reporting period and the stalled flag suppresses repeated identical warnings. Recovery clears that episode. These flags belong to observation, not the alarm state machine, and Supervisor never feeds the watched task's watchdog on its behalf.

ORIGINAL SOURCE / CHAPTER 3 / 1 OF 6

src/services/supervisor.c

Original lines 1-46 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  #include "services/supervisor.h"
2
3  #include <esp_heap_caps.h>
4  #include <esp_system.h>
5  #include <freertos/FreeRTOS.h>
6  #include <freertos/task.h>
7  #include <stdio.h>
8  #include <string.h>
9
10 #include "app_config.h"
11 #include "diagnostics/logger.h"
12 #include "diagnostics/progress.h"
13 #include "domain/reading.h"
14 #include "hal/clock.h"
15 #include "services/acquisition.h"
16 #include "services/configuration.h"
17 #include "services/power_coordinator.h"
18 #include "services/safety.h"
19 #include "services/snapshot_store.h"
20 #include "transports/ble_transport.h"
21 #include "transports/wifi_mqtt.h"
22
23 static StaticTask_t s_tcb;
24 static StackType_t s_stack[3 * 1024];  // cfgTaskSpec(TASK_ID_SUPERVISOR)->stackBytes
25 static TaskHandle_t s_task = NULL;
26 static Snapshot s_snap;  // static: 584 B does not belong on a 3 KiB stack
27
28 // ---- Progress deadlines (§18) ----------------------------------------------------
29 typedef struct Watched {
30   TaskId id;
31   const char* name;
32   uint32_t deadlineMs;   // no progress for longer than this = stalled
33   bool onlyWhenBusy;     // SerialLogger: stalled only if records are waiting
34   bool stalled;          // current episode, warned once
35 } Watched;
36
37 static Watched s_watched[] = {
38     {TASK_ID_SAFETY, "safety", SUPERVISOR_SAFETY_DEADLINE_MS, false, false},
39     {TASK_ID_ACQUISITION_NORMAL, "acq", SUPERVISOR_ACQ_DEADLINE_MS, false, false},
40     {TASK_ID_SERIAL_LOGGER, "logger", SUPERVISOR_LOGGER_DEADLINE_MS, true, false},
41     {TASK_ID_CONFIGURATION, "cfg", SUPERVISOR_CONFIG_DEADLINE_MS, false, false},
42     {TASK_ID_BLE_TRANSPORT, "ble", SUPERVISOR_BLE_DEADLINE_MS, false, false},
43     {TASK_ID_WIFI_MQTT, "wifi", SUPERVISOR_WIFI_DEADLINE_MS, false, false},
44     {TASK_ID_POWER_COORDINATOR, "power", SUPERVISOR_POWER_DEADLINE_MS, false, false},
45 };
46 #define WATCHED_COUNT (sizeof(s_watched) / sizeof(s_watched[0]))
```

Return to guided explanation, p. 131

ORIGINAL SOURCE / CHAPTER 3 / 2 OF 6

src/services/supervisor.c

Original lines 47-92 | Code and comments unchanged; long lines wrap visually

markDegraded (L52-54): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.
checkProgress (L56-82): Watch the active acquisition holder and service progress ages; ignore an inactive acquisition task.

```
47
48  // Health summary shown in the status line: OK until something degraded in
49  // the last status period.
50  static char s_degraded[48];
51
52  static void markDegraded(const char* what) {
53    if (s_degraded[0] == '\0') snprintf(s_degraded, sizeof(s_degraded), "%s", what);
54  }
55
56  static void checkProgress(MonoMs now) {
57    for (size_t i = 0; i < WATCHED_COUNT; ++i) {
58      Watched* w = &s_watched[i];
59      TaskId id = w->id;
60      if (id == TASK_ID_ACQUISITION_NORMAL) {
61        // WP9: whichever worker holds the lease owes progress; nobody during
62        // the hand-over (PowerCoordinator reports a stuck one as DEGRADED).
63        id = acquisitionActiveTaskId();
64        if (id == TASK_ID_COUNT) continue;
65      }
66      const MonoMs last = progressLastMs(id);  // 0 = never stamped: age from boot
67      const MonoMs age = now > last ? now - last : 0;
68      const bool idleLogger = w->onlyWhenBusy && logQueueDepth() == 0;
69      const bool late = !idleLogger && now > SUPERVISOR_BOOT_GRACE_MS && age > w->deadlineMs;
70      if (late) {
71        markDegraded(w->name);
72        if (!w->stalled) {
73          w->stalled = true;
74          logPrintf(LOG_LEVEL_WARN, "sup: %s no progress for %lu ms (deadline %lu)", w->name,
75                    (unsigned long)age, (unsigned long)w->deadlineMs);
76        }
77      } else if (w->stalled) {
78        w->stalled = false;
79        logPrintf(LOG_LEVEL_INFO, "sup: %s progressing again", w->name);
80      }
81    }
82  }
83
84  // ---- Counters: warn when a loss / lateness counter goes up ------------------------
85  typedef struct Counters {
86    uint32_t frameLate, frameSkipped, bmeLate, bmeSkipped, guard, alarmOverflow, logDrops;
87  } Counters;
88
89  static Counters s_prev;
90
91  static void readCounters(Counters* c) {
92    AcquisitionStats a;
```

Return to guided explanation, p. 131

ORIGINAL SOURCE / CHAPTER 3 / 3 OF 6

src/services/supervisor.c

Original lines 93-138 | Code and comments unchanged; long lines wrap visually

readCounters (L91-104): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.
checkCounters (L106-132): Compare counters with the previous check and report increases rather than repeating totals.

```
93    acquisitionGetStats(&a);
94    SafetyStatus st;
95    memset(&st, 0, sizeof(st));
96    safetyStatusPeek(&st);
97    c->frameLate = a.frameLate;
98    c->frameSkipped = a.frameSkipped;
99    c->bmeLate = a.bmeLate;
100   c->bmeSkipped = a.bmeSkipped;
101   c->guard = a.guardTimeouts;
102   c->alarmOverflow = st.detailOverflow;
103   c->logDrops = logDroppedCount();
104 }
105
106 static void checkCounters(void) {
107   Counters c;
108   readCounters(&c);
109   if (memcmp(&c, &s_prev, sizeof(c)) != 0) {
110     // One line per second at most; deltas since the last check.
111     static char line[LOG_TEXT_LEN];  // task-private: keeps the 3 KiB stack free
112     size_t used = (size_t)snprintf(line, sizeof(line), "sup: counters up:");
113     const struct { const char* name; uint32_t now, before; } items[] = {
114         {"frameLate", c.frameLate, s_prev.frameLate},
115         {"frameSkip", c.frameSkipped, s_prev.frameSkipped},
116         {"bmeLate", c.bmeLate, s_prev.bmeLate},
117         {"bmeSkip", c.bmeSkipped, s_prev.bmeSkipped},
118         {"guard", c.guard, s_prev.guard},
119         {"alarmOverflow", c.alarmOverflow, s_prev.alarmOverflow},
120         {"logDrops", c.logDrops, s_prev.logDrops},
121     };
122     for (size_t i = 0; i < sizeof(items) / sizeof(items[0]) && used < sizeof(line); ++i) {
123       if (items[i].now == items[i].before) continue;
124       const int n = snprintf(line + used, sizeof(line) - used, " %s +%lu", items[i].name,
125                              (unsigned long)(items[i].now - items[i].before));
126       if (n > 0) used += (size_t)n;
127       markDegraded(items[i].name);
128     }
129     logPrintf(LOG_LEVEL_WARN, "%s", line);
130     s_prev = c;
131   }
132 }
133
134 // ---- Stacks and heap ------------------------------------------------------------
135 static uint32_t stackFree(TaskHandle_t t) {
136   // ESP-IDF: StackType_t is a byte, so the high-water mark is in bytes.
137   return t != NULL ? (uint32_t)uxTaskGetStackHighWaterMark(t) : 0;
138 }
```

Return to guided explanation, p. 131

ORIGINAL SOURCE / CHAPTER 3 / 4 OF 6

src/services/supervisor.c

Original lines 139-181 | Code and comments unchanged; long lines wrap visually

checkMemory (L140-176): Inspect minimum unused task stacks and internal-heap figures. These are observations, not automatic memory repair.
resetReasonName (L179-193): Translate the MCU reset reason into a readable log label.

```
139
140 static void checkMemory(bool report) {
141   const struct { const char* name; TaskHandle_t handle; } tasks[] = {
142       {"safety", safetyTaskHandle()},
143       {"acq", acquisitionNormalTaskHandle()},
144       {"log", (TaskHandle_t)logTaskHandle()},
145       {"sup", s_task},
146       {"cfg", (TaskHandle_t)configurationTaskHandle()},
147       {"ble", (TaskHandle_t)bleTransportTaskHandle()},
148       {"wifi", (TaskHandle_t)wifiMqttTaskHandle()},
149       {"acqlp", acquisitionLowPowerTaskHandle()},
150       {"pwr", (TaskHandle_t)powerCoordinatorTaskHandle()},
151   };
152   uint32_t freeB[9];
153   bool low = false;
154   for (size_t i = 0; i < 9; ++i) {
155     freeB[i] = stackFree(tasks[i].handle);
156     if (tasks[i].handle != NULL && freeB[i] < SUPERVISOR_STACK_WARN_BYTES) {
157       low = true;
158       markDegraded("stack");
159     }
160   }
161   const uint32_t heapFree = (uint32_t)heap_caps_get_free_size(MALLOC_CAP_INTERNAL);
162   const uint32_t heapMin = (uint32_t)heap_caps_get_minimum_free_size(MALLOC_CAP_INTERNAL);
163   const uint32_t largest = (uint32_t)heap_caps_get_largest_free_block(MALLOC_CAP_INTERNAL);
164   const bool heapLow = heapMin < SUPERVISOR_HEAP_WARN_BYTES;
165   if (heapLow) markDegraded("heap");
166   if (report || low || heapLow) {
167     logPrintf(low || heapLow ? LOG_LEVEL_WARN : LOG_LEVEL_INFO,
168               // Fits LOG_TEXT_LEN (128) with every value at 6 digits.
169               "sup: stack B saf %lu acq %lu log %lu sup %lu cfg %lu ble %lu wifi %lu | heap
 > %lu min %lu big %lu",
170               (unsigned long)freeB[0], (unsigned long)freeB[1], (unsigned long)freeB[2],
171               (unsigned long)freeB[3], (unsigned long)freeB[4], (unsigned long)freeB[5],
 > (unsigned long)freeB[6], (unsigned long)heapFree, (unsigned long)heapMin,
172               (unsigned long)largest);
173     logPrintf(low ? LOG_LEVEL_WARN : LOG_LEVEL_INFO, "sup: stack B acqlp %lu pwr %lu",
 > (unsigned long)freeB[7],
174               (unsigned long)freeB[8]);
175   }
176 }
177
178 // ---- Status line ----------------------------------------------------------------
179 static const char* resetReasonName(esp_reset_reason_t r) {
180   switch (r) {
181     case ESP_RST_POWERON: return "POWERON";
```

Return to guided explanation, p. 131

ORIGINAL SOURCE / CHAPTER 3 / 5 OF 6

src/services/supervisor.c

Original lines 182-226 | Code and comments unchanged; long lines wrap visually

resetReasonName (L179-193): Translate the MCU reset reason into a readable log label.
cell (L196-207): Format one metric for diagnostic display; stored health is used here without a fresh age calculation.

```
    case ESP_RST_SW: return "SW";
    case ESP_RST_PANIC: return "PANIC";
    case ESP_RST_INT_WDT: return "INT_WDT";
    case ESP_RST_TASK_WDT: return "TASK_WDT";
    case ESP_RST_WDT: return "WDT";
    case ESP_RST_BROWNOUT: return "BROWNOUT";
    case ESP_RST_DEEPSLEEP: return "DEEPSLEEP";
    case ESP_RST_EXT: return "EXT";
    case ESP_RST_SDIO: return "SDIO";
    default: return "UNKNOWN";
  }
}

// "airT 23.1" for a usable value, "airT FAULT" otherwise.
static int cell(char* buf, size_t len, const char* label, const Reading* r, int decimals) {
  if (r->health == HEALTH_OK || r->health == HEALTH_UNCALIBRATED) {
    return snprintf(buf, len, " %s %.*f%s", label, decimals, (double)r->value,
                    r->health == HEALTH_UNCALIBRATED ? "~" : "");
  }
  // Failures show their raw value when they have one, e.g. HC-SR04 TIMEOUT
  // with raw = us elapsed (ECHO rose, no echo back) vs no raw (never rose).
  if (r->hasRaw && (r->health == HEALTH_FAULT || r->health == HEALTH_TIMEOUT)) {
    return snprintf(buf, len, " %s %s(raw %ld)", label, healthName(r->health), (long)r->raw);
  }
  return snprintf(buf, len, " %s %s", label, healthName(r->health));
}

static void printStatus(MonoMs now) {
  static char line[LOG_TEXT_LEN];  // task-private: keeps the 3 KiB stack free
  size_t used = (size_t)snprintf(line, sizeof(line), "t=%lus", (unsigned long)(now / 1000));
  if (snapshotPeek(&s_snap)) {
    const Reading* r = s_snap.readings;
    struct { const char* label; MetricId m; int dec; } cols[] = {
        {"airT", METRIC_AIR_TEMPERATURE, 1}, {"RH", METRIC_AIR_HUMIDITY, 0},
        {"waterT", METRIC_WATER_TEMPERATURE, 1}, {"light", METRIC_LIGHT_LEVEL, 0},
        {"pH", METRIC_PH, 2}, {"TDS", METRIC_TDS, 0}, {"dist", METRIC_WATER_DISTANCE, 0},
        {"CO2eq", METRIC_CO2EQ, 0}};
    for (size_t i = 0; i < sizeof(cols) / sizeof(cols[0]) && used < sizeof(line); ++i) {
      const int n = cell(line + used, sizeof(line) - used, cols[i].label, &r[cols[i].m],
cols[i].dec);
      if (n > 0) used += (size_t)n;
    }
  } else if (used < sizeof(line)) {
    used += (size_t)snprintf(line + used, sizeof(line) - used, " no snapshot");
  }
  logPrintf(LOG_LEVEL_INFO, "%s", line);
```

Return to guided explanation, p. 131

ORIGINAL SOURCE / CHAPTER 3 / 6 OF 6

src/services/supervisor.c

Original lines 227-271 | Code and comments unchanged; long lines wrap visually

printStatus (L209-243): Build a bounded diagnostic line from current copied state. Long lines can be truncated. **supervisorTask (L245-263)**: Observe progress/counters each second and memory/status at configured intervals. Do not feed peer watchdogs.

```
227
228   SafetyStatus st;
229   AcquisitionStats a;
230   acquisitionGetStats(&a);
231   if (safetyStatusPeek(&st)) {
232     logPrintf(LOG_LEVEL_INFO,
233               "     lamp %s (%s)%s | latched 0x%04x | frames %lu late %lu skip %lu jitter %lu
  >  ms"
234               " | guard %lu fail %lu | %s%s",
235               st.relayOn ? "ON" : "OFF", lampReasonName(st.lampReason),
236               st.lampWantsOn && !st.relayOn ? " wants ON" : "", (unsigned)st.latchedMask,
237               (unsigned long)a.frames, (unsigned long)a.frameLate, (unsigned
  >  long)a.frameSkipped,
238               (unsigned long)a.frameJitterMaxMs, (unsigned long)a.guardTimeouts,
239               (unsigned long)a.driverFailures, s_degraded[0] ? "DEGRADED " : "health OK",
240               s_degraded);
241   }
242   s_degraded[0] = '\0';  // a new period starts
243 }
244
245 static void supervisorTask(void* arg) {
246   (void)arg;
247   logPrintf(LOG_LEVEL_INFO, "reset reason: %s", resetReasonName(esp_reset_reason()));
248   readCounters(&s_prev);
249   TickType_t last = xTaskGetTickCount();
250   uint32_t seconds = 0;
251   for (;;) {
252     vTaskDelayUntil(&last, pdMS_TO_TICKS(CFG_SUPERVISOR_PERIOD_MS));
253     const MonoMs now = monoNowMs();
254     ++seconds;
255     checkProgress(now);
256     checkCounters();
257     if (seconds % SUPERVISOR_MEMORY_EVERY_S == 0) {
258       checkMemory(seconds % SUPERVISOR_MEMORY_REPORT_EVERY_S == 0);
259     }
260     if (seconds % SUPERVISOR_STATUS_EVERY_S == 0) printStatus(now);
261     progressMark(TASK_ID_SUPERVISOR);
262   }
263 }
264
265 bool supervisorInit(void) {
266   if (s_task != NULL) return true;
267   const TaskSpec* spec = cfgTaskSpec(TASK_ID_SUPERVISOR);
268   s_task = xTaskCreateStaticPinnedToCore(supervisorTask, spec->name, sizeof(s_stack), NULL,
269                                          spec->priority, s_stack, &s_tcb, spec->core);
270   return s_task != NULL;
271 }
```

Return to guided explanation, p. 131

PART I / FOUNDATIONS

CHAPTER 04

Shared Driver, I2C and ADC Helpers

Every sensor driver presents the same lifecycle to acquisition. Underneath it, short HAL functions perform peripheral operations and pure helpers filter or convert numbers. This split keeps a device protocol separate from the scheduling policy that calls it.

In this chapter

1. Follow construct, init, start, poll and readResult.
2. Read the shared I2C and ADC operations.
3. Derive a spaced sampling window and a trimmed mean.

Chapter contents

4.1 The common driver lifecycle 139
4.2 The I2C HAL: short transactions, one owner 140
4.3 The ADC HAL and the analog signal chain 141
4.4 Spaced windows and trimmed means 142

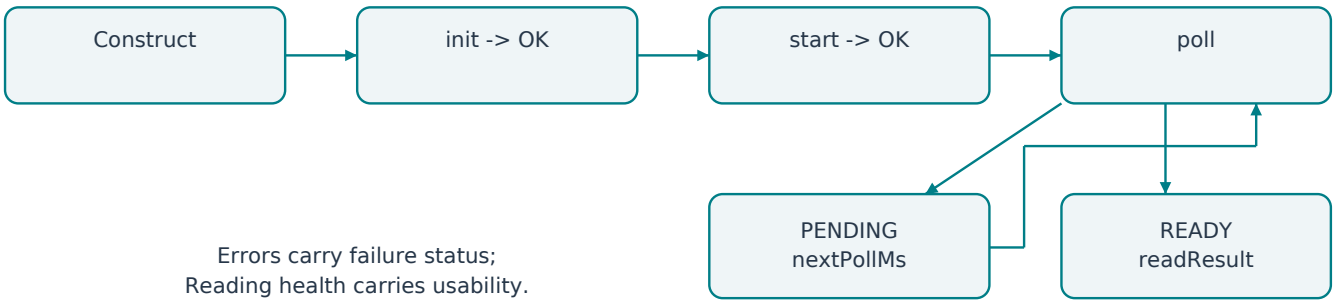
Guided source readings begin on page 143.

Complete original listings begin on page 144. Shared files are linked in the explanations; Appendix A contains the full filename index.

CHAPTER 4

4.1 The common driver lifecycle

Every concrete driver exposes identification, metric count, init, start, poll, readResult, cancel and preparePowerOff through its operations table. The wrapper functions in sensor_driver.h simply dispatch to the right function.



Interpret the two levels of status

`init()` returning DRIVER_OK means configuration succeeded. `start()` returning DRIVER_OK means an operation was accepted. `poll()` returning DRIVER_PENDING gives a future time worth checking. DRIVER_READY means readResult can supply the latest completed verdict. Its Reading health can still be FAULT, WARMING, UNCALIBRATED or DEPENDENCY_INVALID.

For example, TDS can finish its ADC window correctly and report DRIVER_READY while its water-temperature dependency is missing. Reporting transport failure would hide the real issue: sampling worked but compensation could not be trusted.

No hidden owner inside a driver

A driver does not create its own task. It keeps enough state to continue across calls. The acquisition owner chooses when those calls happen. This keeps bus arbitration visible and avoids one task per sensor competing for the same I2C controller.

`cancel()` stops tracking a pending result; it may not physically stop a conversion already running inside the chip. `preparePowerOff()` also places interfaces in an appropriate safe state or invalidates configuration. The production power coordinator invokes an acknowledged handover; acquisition cancels pending work and prepares dead-rail interfaces before release.

The common interface relies on correct pointers and caller-provided output capacity. Some readResult implementations check NULL, while others only check capacity. The current caller supplies valid arrays; this interface is not a fully defensive boundary for arbitrary callers.

Original source in this PDF: src/drivers/sensor_driver.h (p. 144); src/drivers/sensor_driver.c (p. 146)

CHAPTER 4

4.2 The I2C HAL: short transactions, one owner

BME280 and SGP30 share I2C controller 0, SDA GPIO21 and SCL GPIO47 at 400 kHz. The board profile specifies 4.7 kOhm pull-ups to 3.3V. The controller transfers address and data bits; a device's acknowledgement indicates it accepted the addressed transfer.

`i2cBusBegin()` first calls `i2cBusEnd()` if this bus object is already marked started. It then saves the requested pins and clock. The final expression accepts an already initialised controller 0 or initialises that controller with the saved settings. `i2cBusEnd()` deinitialises a started controller and clears the object flag. One acquisition owner keeps this object and controller lifecycle consistent across both I2C sensors.

HAL call	Transaction meaning
`i2cBusProbe()`	Address-only write to test for acknowledgement.
`i2cBusWriteReg()`	Register address followed by one value.
`i2cBusReadRegs()`	Register address, repeated start, then a burst read.
`i2cBusWrite()` / `i2cBusRead()`	Standalone command or reply transfers ending in STOP.

BME280 is register-based, so a register pointer and repeated-start read make sense. SGP30 is command-based and may NACK while processing; its command and later reply are separate bus operations.

Timeouts accumulate across a function

The timeout is 5 ms per transaction. A poll containing two failed transfers can occupy roughly two such budgets, plus overhead and recovery. A 1 ms DriverRun entry is therefore closer to a nominal scheduling estimate than a demonstrated worst-case fault bound for the I2C drivers.

Recovery is bounded

`i2cBusRecover()` first checks SDA. If it is high, the code treats the failure as an absent/busy-device NACK and leaves the shared controller running. If SDA is held low, it releases the controller, generates up to nine SCL pulses, creates a STOP and restarts the controller.

Recovery stops after a bounded pulse sequence. A shorted clock line or failed pull-up can remain faulty. Drivers return failure and let the service schedule later attempts instead of staying in an endless retry loop.

Original source in this PDF: src/hal/i2c_bus.c (p. 149); src/hal/i2c_bus.h (p. 147); include/board_profile.h (p. 32)

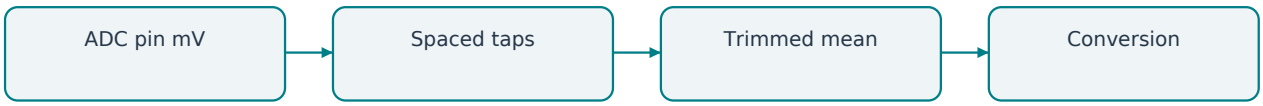
CHAPTER 4

4.3 The ADC HAL and the analog signal chain

Light, TDS and pH share a software sampling pattern but use separate ADC1 pins. `adcChannelBegin()` only accepts GPIO1 through GPIO10, selects 12-bit resolution and 11 dB attenuation, then records the pin. `adcChannelReadMilliVolts()` returns an eFuse-calibrated voltage conversion and a saturation flag.

Input	Pin	Front end	Driver's raw result
Light	GPIO1	10 kOhm series, 1 uF to ground	Filtered pin mV
TDS	GPIO6	Direct signal, 100 nF to ground	Filtered pin mV
pH	GPIO7	10 kOhm / 15 kOhm divider, 1 uF	Filtered pin mV

The ADC reports voltage at the pin. pH conversion must recover the upstream module voltage using the divider ratio. TDS and light do not apply that divider factor.



Light -> mV pH -> divider + line fit TDS -> cubic + temperature

Health and quality are attached after interpretation.

Saturation is a loss of information

The HAL marks calibrated voltages at or above 3050 mV saturated. This is a project threshold near the ESP32-S3 ADC range, not an electrical input-protection mechanism. A clipped ADC cannot report how far above its range the real input went.

Light treats saturation as bright, preserving a lower-bound interpretation. pH and TDS reject saturated windows for quantitative use. The same ADC condition therefore has different domain meanings.

Initialisation and fault limits

Each analog driver performs one throwaway conversion in init, checking the channel and moving one-time calibration cost away from normal polling. The HAL's bool mainly checks whether a valid configured channel exists; a disconnected sensor can still produce a perfectly readable ADC voltage.

ADC calibration corrects the converter's voltage interpretation. It does not calibrate the light-to-lux, electrode-to-pH, or conductivity-to-TDS relationship. These are separate stages with separate evidence.

Original source in this PDF: src/hal/adc_channel.c (p. 153); include/board_profile.h (p. 32); src/drivers/light.c (p. 168); src/drivers/ph.c (p. 219); src/drivers/tds.c (p. 235)

CHAPTER 4

4.4 Spaced windows and trimmed means

Each analog driver starts with one immediate sample. Poll takes one later sample only after the required spacing from the preceding actual sample. A delayed poll shifts the remaining window; it does not collect several rapid samples to pretend that missed times were observed.

Driver	Samples	Spacing	First-to-last span	Trim at each end
Light	10	11 ms	99 ms	2
pH	10	30 ms	270 ms	2
TDS	15	40 ms	560 ms	5

The span is `(N - 1) * spacing`, because the first sample is already taken at start. These are minimum nominal spans; admission delays and actual sampling overhead add time.

trimmedMeanU32()

The domain helper sorts the supplied array in place using insertion sort. For these small fixed windows the implementation is simple and bounded. It drops the lowest and highest trim values and averages the middle values using a 64-bit sum.

Example: ten values sorted as 0, 100, 1499, 1500, 1500, 1500, 1501, 1502, 3100, 4000 become six retained values after trimming two at each end. Their mean is 1500.333 mV. A plain ten-value average would be pulled away by the extremes.

The helper returns zero if there are no retained samples. Current drivers use fixed valid counts. It is not a general error-reporting API for arbitrary filter configurations.

Filtering cannot fix every fault

The analog sample helper remembers whether any tap saturated, even if that tap is later trimmed out. This avoids making an overrange input look trustworthy merely by removing the extreme value.

A window reduces some noise; it does not establish probe calibration or detect all open circuits. Light's 11 ms spacing deliberately walks through a 10 ms flicker period instead of sampling at a fixed flicker phase. The RC front end also smooths the signal; neither mechanism establishes a lux scale.

Original source in this PDF: src/domain/calibration.c (p. 158); src/drivers/light.h (p. 163); src/drivers/ph.h (p. 215); src/drivers/tds.h (p. 231)

GUIDED SOURCE READING / CHAPTER 4

The interface is a table of ordinary function pointers

The service has been calling sensorDriverStart without naming a particular sensor. This header explains how the same service reaches seven different implementations.

Original excerpt: src/drivers/sensor_driver.h (p. 144) / lines 43-57. The complete unchanged file follows.

```
43  typedef struct SensorDriverOps {
44    SensorId (*id)(const SensorDriver* self);
45    const char* (*name)(const SensorDriver* self);
46
47    // Number of Reading records this driver produces per result.
48    uint8_t (*metricCount)(const SensorDriver* self);
49
50    // Probe and configure the device. May be called again after a fault.
51    DriverStatus (*init)(SensorDriver* self);
52
53    // Begin one measurement (e.g. trigger a conversion). Must not block on it.
54    DriverStatus (*start)(SensorDriver* self, MonoMs now);
55
56    // Advance the measurement. Never blocks beyond the documented budget.
57    PollResult (*poll)(SensorDriver* self, MonoMs now);
```

Each field is a function-pointer type: a callable address with a required signature. init accepts the common driver pointer, start additionally accepts time, and poll returns both a lifecycle status and a useful next poll time. The table provides dispatch; it is not a task scheduler.

The concrete driver's base member is first in its struct. Passing its address as SensorDriver* lets a wrapper reach base.ops. Inside a device-specific callback, asLight or asBme280 casts that pointer back to the concrete type. This is valid for the intended first-member layout; an arbitrary unrelated pointer would not be interchangeable.

Read the declaration order as the driver's lifecycle: identify, initialize, start, poll, copy result, cancel, prepare for power loss. The compiler does not enforce that temporal order from the table layout. Acquisition enforces it through initialised, pending and lease state.

Headers expose the contract without exposing every private helper. The matching implementation prints an operations-table initializer that connects these slots to its local functions. Once that connection is understood, the sensor chapters become variations of the same lifecycle rather than seven unrelated programs.

ORIGINAL SOURCE / CHAPTER 4 / 1 OF 2

src/drivers/sensor_driver.h

Original lines 1-47 | Code and comments unchanged; long lines wrap visually

The operations table is a set of callable function addresses. Every driver presents the same interface.

```
// Common sensor driver contract (Firmware Architecture §5, §14).
// FROZEN CONTRACT - all seven drivers implement exactly this interface.
//
// Rules for every driver:
//  - Know hardware, report status. No MQTT/BLE/WiFi includes, no task
//    creation, no power-mode decisions, no relay changes.
//  - No hidden long delays. start() begins an operation; poll() returns
//    Pending + the next time it is worth polling again. The single
//    acquisition worker interleaves sensors using those deadlines.
//  - Worst-case blocking of every call must be documented and bounded.
//  - Configuration (bus, pins, address) is passed to the constructor
//    (<name>DriverConstruct()).
//
// C object model: a driver is a struct whose FIRST member is a SensorDriver.
// SensorDriver holds a pointer to the driver's const operations table (the
// "vtable"). Callers use the sensorDriver*() functions below on a
// SensorDriver*, which dispatch through that table; the driver casts the
// SensorDriver* back to its own type.
#pragma once
#include <stdbool.h>
#include <stdint.h>
#include "domain/reading.h"

typedef uint8_t DriverStatus;
enum {
  DRIVER_OK = 0,          // call accepted (init/start)
  DRIVER_PENDING,         // operation in progress, poll again at nextPollMs
  DRIVER_READY,           // result available via readResult()
  DRIVER_NOT_PRESENT,     // device missing / wrong identity
  DRIVER_FAULT,           // bus, CRC or range error
  DRIVER_TIMEOUT,         // exceeded its budget
  DRIVER_UNPOWERED,       // supply rail not available
  DRIVER_NOT_IMPLEMENTED  // stub (Stage 1 skeleton)
};

typedef struct PollResult {
  DriverStatus status;
  MonoMs nextPollMs;  // meaningful when status == DRIVER_PENDING
} PollResult;

typedef struct SensorDriver SensorDriver;

typedef struct SensorDriverOps {
  SensorId (*id)(const SensorDriver* self);
  const char* (*name)(const SensorDriver* self);

  // Number of Reading records this driver produces per result.
```

Return to guided explanation, p. 143

ORIGINAL SOURCE / CHAPTER 4 / 2 OF 2

src/drivers/sensor_driver.h

Original lines 48-94 | Code and comments unchanged; long lines wrap visually

sensorDriverId (L76-76): Return this module's sensor identity; this does not access hardware.
sensorDriverName (L77-77): Translate an identity/state into a human-readable label for diagnostics.

```
48    uint8_t (*metricCount)(const SensorDriver* self);
49
50    // Probe and configure the device. May be called again after a fault.
51    DriverStatus (*init)(SensorDriver* self);
52
53    // Begin one measurement (e.g. trigger a conversion). Must not block on it.
54    DriverStatus (*start)(SensorDriver* self, MonoMs now);
55
56    // Advance the measurement. Never blocks beyond the documented budget.
57    PollResult (*poll)(SensorDriver* self, MonoMs now);
58
59    // Copy up to maxOut readings of the last completed measurement.
60    // Returns the number written. Health/quality are filled by the driver;
61    // powerEpoch/configRevision are stamped by the acquisition service.
62    uint8_t (*readResult)(SensorDriver* self, Reading* out, uint8_t maxOut);
63
64    // Abandon any pending operation (deadline miss, quiesce request).
65    void (*cancel)(SensorDriver* self);
66
67    // Put the interface pins into a safe state before the rail is removed
68    // (§12: no back-feeding an unpowered module).
69    void (*preparePowerOff)(SensorDriver* self);
70  } SensorDriverOps;
71
72  struct SensorDriver {
73    const SensorDriverOps* ops;
74  };
75
76  static inline SensorId sensorDriverId(const SensorDriver* d) { return d->ops->id(d); }
77  static inline const char* sensorDriverName(const SensorDriver* d) { return d->ops->name(d); }
78  static inline uint8_t sensorDriverMetricCount(const SensorDriver* d) {
79    return d->ops->metricCount(d);
80  }
81  static inline DriverStatus sensorDriverInit(SensorDriver* d) { return d->ops->init(d); }
82  static inline DriverStatus sensorDriverStart(SensorDriver* d, MonoMs now) {
83    return d->ops->start(d, now);
84  }
85  static inline PollResult sensorDriverPoll(SensorDriver* d, MonoMs now) {
86    return d->ops->poll(d, now);
87  }
88  static inline uint8_t sensorDriverReadResult(SensorDriver* d, Reading* out, uint8_t maxOut) {
89    return d->ops->readResult(d, out, maxOut);
90  }
91  static inline void sensorDriverCancel(SensorDriver* d) { d->ops->cancel(d); }
92  static inline void sensorDriverPreparePowerOff(SensorDriver* d) { d->ops->preparePowerOff(d);
 >  }
93
94  const char* driverStatusName(DriverStatus s);
```

Return to guided explanation, p. 143

ORIGINAL SOURCE / CHAPTER 4 / 1 OF 1

src/drivers/sensor_driver.c

Original lines 1-15 | Code and comments unchanged; long lines wrap visually

driverStatusName (L3-15): Translate an identity/state into a human-readable label for diagnostics.

```
1  #include "drivers/sensor_driver.h"
2
3  const char* driverStatusName(DriverStatus s) {
4    switch (s) {
5      case DRIVER_OK: return "OK";
6      case DRIVER_PENDING: return "PENDING";
7      case DRIVER_READY: return "READY";
8      case DRIVER_NOT_PRESENT: return "NOT_PRESENT";
9      case DRIVER_FAULT: return "FAULT";
10     case DRIVER_TIMEOUT: return "TIMEOUT";
11     case DRIVER_UNPOWERED: return "UNPOWERED";
12     case DRIVER_NOT_IMPLEMENTED: return "NOT_IMPLEMENTED";
13   }
14   return "?";
15 }
```

Return to guided explanation, p. 139

ORIGINAL SOURCE / CHAPTER 4 / 1 OF 1

src/hal/i2c_bus.h

Original lines 1-37 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // I2C bus adapter (HAL). Work package WP1 (with BME280).
2  // Single owner: only the acquisition lease holder calls it (§9).
3  // Every transaction must have a bounded timeout; i2cBusRecover() performs a
4  // limited stuck-bus recovery (§16), never a blocking retry loop.
5  //
6  // Worst-case blocking: I2C_BUS_TIMEOUT_MS per transaction (probe/writeReg/
7  // readRegs/write/read); recover() ~0.2 ms of bit-banging plus one begin().
8  #pragma once
9  #include <stdbool.h>
10 #include <stddef.h>
11 #include <stdint.h>
12
13 #define I2C_BUS_TIMEOUT_MS ((uint16_t)5)
14
15 // Treat the members as private to i2c_bus.c. Initialise with I2C_BUS_INIT.
16 typedef struct I2cBus {
17   bool started_;
18   int8_t sda_;
19   int8_t scl_;
20   uint32_t clockHz_;
21 } I2cBus;
22
23 #define I2C_BUS_INIT {false, -1, -1, 0}
24
25 bool i2cBusBegin(I2cBus* bus, int8_t sda, int8_t scl, uint32_t clockHz);
26 bool i2cBusProbe(I2cBus* bus, uint8_t address);
27 bool i2cBusWriteReg(I2cBus* bus, uint8_t address, uint8_t reg, uint8_t value);
28 bool i2cBusReadRegs(I2cBus* bus, uint8_t address, uint8_t reg, uint8_t* buf, size_t len);
29 // Raw transfers for command-based devices (SGP30): each is one transaction
30 // ending in STOP, so a device that NACKs while busy is never held by a
31 // repeated start. A read returns true only if all `len` bytes arrived.
32 bool i2cBusWrite(I2cBus* bus, uint8_t address, const uint8_t* tx, size_t len);
33 bool i2cBusRead(I2cBus* bus, uint8_t address, uint8_t* buf, size_t len);
34 // Call after a failed transaction. Does nothing (returns true) when SDA is
35 // high, i.e. the failure was only a NACK; recovers only a bus held low.
36 bool i2cBusRecover(I2cBus* bus);
37 void i2cBusEnd(I2cBus* bus);
```

Return to guided explanation, p. 139

GUIDED SOURCE READING / CHAPTER 4

A register read is one ordered transaction

Both BME280 and SGP30 use I2C, but their drivers ask for different transactions. This helper implements a register-address write followed by a repeated-start read.

Original excerpt: src/hal/i2c_bus.c (p. 149) / lines 34-40. The complete unchanged file follows.

```
34  bool i2cBusReadRegs(I2cBus* bus, uint8_t address, uint8_t reg, uint8_t* buf, size_t len) {
35    if (!bus->started_ || buf == NULL || len == 0) return false;
36    // Register write, repeated start, read: one bounded transaction.
37    size_t got = 0;
38    i2cWriteReadNonStop(kI2cPort, address, &reg, 1, buf, len, I2C_BUS_TIMEOUT_MS, &got);
39    return got == len;  // a partial or failed read is dropped
40  }
```

The guard checks bus startup, destination pointer and nonzero length before the controller API sees them. got is initialized to zero so a transfer that produces no bytes cannot accidentally inherit a previous success count.

&reg passes the address of the one-byte register selector, and the following 1 declares its length. buf and len describe the caller's destination. The combined operation preserves write-register then read-data ordering without a separate stopped transaction between them.

The return compares actual received bytes with requested length. A partial transfer is rejected before a driver interprets an incomplete register array. Merely checking that some bytes arrived would leave missing bytes as old or uninitialized data.

The byte arrays are local because this API performs the bounded transaction before returning. If an asynchronous DMA interface retained these pointers, their lifetime would need a different design. Read a pointer parameter together with the callee's completion contract, not from its type alone.

ORIGINAL SOURCE / CHAPTER 4 / 1 OF 2

src/hal/i2c_bus.c

Original lines 1-47 | Code and comments unchanged; long lines wrap visually

i2cBusBegin (L12-20): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.
i2cBusProbe (L22-26): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.

```
#include "hal/i2c_bus.h"

#include <Arduino.h>
#include <driver/gpio.h>
#include <esp32-hal-i2c.h>

// Controller 0 (what the C++ version reached through the Arduino `Wire`
// object). Calls go straight to the core's C I2C layer, with the same
// per-transaction timeout Wire.setTimeOut() used to apply.
static const uint8_t kI2cPort = 0;

bool i2cBusBegin(I2cBus* bus, int8_t sda, int8_t scl, uint32_t clockHz) {
  if (bus->started_) i2cBusEnd(bus);
  bus->sda_ = sda;
  bus->scl_ = scl;
  bus->clockHz_ = clockHz;
  // As Wire.begin(): a controller that is already running counts as started.
  bus->started_ = i2cIsInit(kI2cPort) || i2cInit(kI2cPort, sda, scl, clockHz) == ESP_OK;
  return bus->started_;
}

bool i2cBusProbe(I2cBus* bus, uint8_t address) {
  if (!bus->started_) return false;
  // Address-only write: ACK => present.
  return i2cWrite(kI2cPort, address, NULL, 0, I2C_BUS_TIMEOUT_MS) == ESP_OK;
}

bool i2cBusWriteReg(I2cBus* bus, uint8_t address, uint8_t reg, uint8_t value) {
  if (!bus->started_) return false;
  const uint8_t tx[2] = {reg, value};
  return i2cWrite(kI2cPort, address, tx, sizeof(tx), I2C_BUS_TIMEOUT_MS) == ESP_OK;
}

bool i2cBusReadRegs(I2cBus* bus, uint8_t address, uint8_t reg, uint8_t* buf, size_t len) {
  if (!bus->started_ || buf == NULL || len == 0) return false;
  // Register write, repeated start, read: one bounded transaction.
  size_t got = 0;
  i2cWriteReadNonStop(kI2cPort, address, &reg, 1, buf, len, I2C_BUS_TIMEOUT_MS, &got);
  return got == len;  // a partial or failed read is dropped
}

bool i2cBusWrite(I2cBus* bus, uint8_t address, const uint8_t* tx, size_t len) {
  if (!bus->started_ || tx == NULL || len == 0) return false;
  return i2cWrite(kI2cPort, address, tx, len, I2C_BUS_TIMEOUT_MS) == ESP_OK;
}

bool i2cBusRead(I2cBus* bus, uint8_t address, uint8_t* buf, size_t len) {
```

Return to guided explanation, p. 148

ORIGINAL SOURCE / CHAPTER 4 / 2 OF 2

src/hal/i2c_bus.c

Original lines 48-95 | Code and comments unchanged; long lines wrap visually

i2cBusRead (L47-52): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.
i2cBusRecover (L56-90): Leave a merely NACKing bus alone; if SDA is stuck LOW, try bounded clock pulses and a STOP.

```
48    if (!bus->started_ || buf == NULL || len == 0) return false;
49    size_t got = 0;
50    i2cRead(kI2cPort, address, buf, len, I2C_BUS_TIMEOUT_MS, &got);
51    return got == len;  // a partial or failed read is dropped
52  }
53
54  // Release a slave holding SDA low: up to 9 SCL pulses, then a STOP.
55  // Bounded (~0.2 ms); no retry loop.
56  bool i2cBusRecover(I2cBus* bus) {
57    const int8_t sda = bus->sda_;
58    const int8_t scl = bus->scl_;
59    if (sda < 0 || scl < 0) return false;
60    // An idle bus has SDA high: the failure was a NACK (e.g. a device that is
61    // absent or busy), not a stuck bus. Leave the controller running so the
62    // other device on the bus is not disturbed (audit L5).
63    if (bus->started_ && gpio_get_level((gpio_num_t)sda) != 0) return true;
64    if (bus->started_) i2cBusEnd(bus);
65
66    pinMode(sda, INPUT_PULLUP);
67    pinMode(scl, OUTPUT_OPEN_DRAIN);
68    digitalWrite(scl, HIGH);
69    delayMicroseconds(5);
70    for (uint8_t i = 0; i < 9 && digitalRead(sda) == LOW; ++i) {
71      digitalWrite(scl, LOW);
72      delayMicroseconds(5);
73      digitalWrite(scl, HIGH);
74      delayMicroseconds(5);
75    }
76
77    // STOP: SDA low -> SCL high -> SDA high.
78    pinMode(sda, OUTPUT_OPEN_DRAIN);
79    digitalWrite(scl, LOW);
80    digitalWrite(sda, LOW);
81    delayMicroseconds(5);
82    digitalWrite(scl, HIGH);
83    delayMicroseconds(5);
84    digitalWrite(sda, HIGH);
85    delayMicroseconds(5);
86
87    pinMode(sda, INPUT_PULLUP);
88    const bool sdaReleased = digitalRead(sda) == HIGH;
89    return i2cBusBegin(bus, sda, scl, bus->clockHz_) && sdaReleased;
90  }
91
92  void i2cBusEnd(I2cBus* bus) {
93    if (bus->started_ && i2cIsInit(kI2cPort)) i2cDeinit(kI2cPort);
94    bus->started_ = false;
95  }
```

Return to guided explanation, p. 148

ORIGINAL SOURCE / CHAPTER 4 / 1 OF 1

src/hal/adc_channel.h

Original lines 1-24 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // ADC1 channel adapter (HAL). Work package WP3 (shared by light, pH and TDS).
2  // ADC1 only: ADC2 conflicts with Wi-Fi (§3). Returns calibrated millivolts at
3  // the ESP32 pin; divider recovery and units belong to domain/calibration.h.
4  //
5  // 12-bit, 11 dB attenuation (S3 usable range about 0-3100 mV), eFuse calibration.
6  // Worst-case blocking: one conversion per read, well under 1 ms.
7  #pragma once
8  #include <stdbool.h>
9  #include <stdint.h>
10
11 #define ADC_CHANNEL_NO_PIN ((int8_t)-1)
12
13 // Treat the members as private to adc_channel.c. Initialise with ADC_CHANNEL_INIT.
14 typedef struct AdcChannel {
15   int8_t pin_;
16 } AdcChannel;
17
18 #define ADC_CHANNEL_INIT {ADC_CHANNEL_NO_PIN}
19
20 // false if the pin is not an ADC1 pin (GPIO1-10 on the S3); the channel then
21 // stays unusable and every read fails.
22 bool adcChannelBegin(AdcChannel* ch, int8_t pin);
23 // One conversion. false on error. Reports saturation near full scale.
24 bool adcChannelReadMilliVolts(AdcChannel* ch, uint32_t* mv, bool* saturated);
```

Return to guided explanation, p. 139

GUIDED SOURCE READING / CHAPTER 4

Configure the converter before asking it for millivolts

The analog drivers share a small ADC adapter. Its job is to make the ESP32 pin readable in calibrated millivolts, leaving filtering and sensor interpretation to the driver.

Original excerpt: src/hal/adc_channel.c (p. 153) / lines 14-21. The complete unchanged file follows.

```
14  bool adcChannelBegin(AdcChannel* ch, int8_t pin) {
15    ch->pin_ = ADC_CHANNEL_NO_PIN;
16    if (pin < kAdc1FirstPin || pin > kAdc1LastPin) return false;
17    analogReadResolution(12);
18    analogSetPinAttenuation((uint8_t)pin, ADC_11db);
19    ch->pin_ = pin;
20    return true;
21  }
```

Initialization sets pin_ to ADC_CHANNEL_NO_PIN before validating the requested pin. If validation fails, a later read cannot accidentally inherit success from an earlier configuration. The selected channels are on ADC1, keeping these measurements away from the ADC2/Wi-Fi sharing issue.

Resolution and attenuation configure the converter's interpretation. The adapter stores the selected pin only once the required setup is complete. A driver then performs a throwaway read during init so one-time ADC characterization is not first encountered in its regular sampling window.

The read helper fills both a voltage and a saturation flag. The driver preserves that flag across taps; a clipped sample is information about the converter ceiling. A successful read only proves the configured ADC path returned data. It does not prove that a probe is connected or that a voltage-to-pH or voltage-to-lux fit is valid.

That separation determines the following code: acquisition owns when to call, the driver owns the sampling window, and domain helpers own the mathematical conversion. Each layer has enough information for its own decision without writing another layer's state.

ORIGINAL SOURCE / CHAPTER 4 / 1 OF 1

src/hal/adc_channel.c

Original lines 1-32 | Code and comments unchanged; long lines wrap visually

adcChannelBegin (L14-21): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.
adcChannelReadMilliVolts (L23-32): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.

```
1  #include "hal/adc_channel.h"
2
3  #include <Arduino.h>
4  #include <stddef.h>
5
6  // ESP32-S3: ADC1 is GPIO1-10 (ADC2 conflicts with Wi-Fi, §3).
7  static const int8_t kAdc1FirstPin = 1;
8  static const int8_t kAdc1LastPin = 10;
9
10 // At 11 dB (ADC_ATTEN_DB_12 in IDF 5) the S3 measures about 0-3100 mV. Treat a
11 // calibrated result at or above this ceiling as clipped. Characterise per board.
12 static const uint32_t kSaturationMv = 3050;
13
14 bool adcChannelBegin(AdcChannel* ch, int8_t pin) {
15   ch->pin_ = ADC_CHANNEL_NO_PIN;
16   if (pin < kAdc1FirstPin || pin > kAdc1LastPin) return false;
17   analogReadResolution(12);
18   analogSetPinAttenuation((uint8_t)pin, ADC_11db);
19   ch->pin_ = pin;
20   return true;
21 }
22
23 bool adcChannelReadMilliVolts(AdcChannel* ch, uint32_t* mv, bool* saturated) {
24   if (mv) *mv = 0;
25   if (saturated) *saturated = false;
26   if (ch->pin_ < 0) return false;
27   // eFuse-calibrated conversion (esp_adc_cal). One conversion, a few tens of us.
28   const uint32_t value = analogReadMilliVolts((uint8_t)ch->pin_);
29   if (mv) *mv = value;
30   if (saturated) *saturated = value >= kSaturationMv;
31   return true;
32 }
```

Return to guided explanation, p. 152

ORIGINAL SOURCE / CHAPTER 4 / 1 OF 2

src/domain/calibration.h

Original lines 1-27 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // Unit conversion and calibration stages (§3, §14). Pure C.
2  // Work packages: WP5 (pH/TDS), WP3 (light).
3  #pragma once
4  #include <stdbool.h>
5  #include <stdint.h>
6
7  // Recover the module output voltage from the ADC pin voltage through a
8  // resistor divider: Vmodule = Vadc * (Rtop + Rbottom) / Rbottom.
9  // Use MEASURED resistor values once available (§3).
10 float dividerRecoverMv(float adcMv, uint32_t rTopOhm, uint32_t rBottomOhm);
11
12 // Linear fit y = slope * x + offset, with a revision for traceability.
13 typedef struct LinearCalibration {
14   float slope;
15   float offset;
16   uint16_t revision;
17   bool valid;  // false => publish raw with HEALTH_UNCALIBRATED
18 } LinearCalibration;
19
20 float linearApply(const LinearCalibration* c, float x);
21
22 // True if slope and offset are finite numbers. Every fit maker below refuses
23 // non-finite input or output; config_blob refuses a stored non-finite fit.
24 bool linearCalibrationFinite(const LinearCalibration* c);
25
26 // Two-point fit through (x1, y1) and (x2, y2), e.g. module mV read in two pH
27 // buffers. false (and *out left invalid) if the x values are too close.
```

Return to guided explanation, p. 139

ORIGINAL SOURCE / CHAPTER 4 / 2 OF 2

src/domain/calibration.h

Original lines 28-53 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
28 bool linearFromTwoPoints(float x1, float y1, float x2, float y2, uint16_t revision,
29                          LinearCalibration* out);
30
31 // Sort `samples` in place, drop `trim` values at each end and return the mean
32 // of the rest (spike-rejecting filter, pH-4502C tutorial: 10 samples, mean of
33 // the middle 6). Returns 0 if n <= 2 * trim.
34 float trimmedMeanU32(uint32_t* samples, uint8_t n, uint8_t trim);
35
36 // ---- On-device calibration (WP8, Calibration page) ----------------------------
37 // Which stored fits the user made on the device (ConfigBlob.calOverridden).
38 // A fit without its bit is the compiled default and is refreshed at boot.
39 #define CAL_BIT_PH 0x01u
40 #define CAL_BIT_TDS 0x02u
41 #define CAL_BIT_LIGHT 0x04u
42 #define CAL_BITS_ALL 0x07u
43
44 // One-point gain through the origin, y = (y1 / x1) * x (TDS K-factor, light
45 // lux per mV). false if x1 or y1 is not positive or the gain is outside
46 // [minGain, maxGain].
47 bool linearGainFromPoint(float x1, float y1, float minGain, float maxGain, uint16_t revision,
48                          LinearCalibration* out);
49
50 // Two-point pH buffer fit, rejected unless |slope| (pH per module mV) is in
51 // [minAbsSlope, maxAbsSlope] and the buffers are at least 1 pH apart.
52 bool phFitFromBuffers(float mv1, float ph1, float mv2, float ph2, float minAbsSlope, float
 > maxAbsSlope,
53                       uint16_t revision, LinearCalibration* out);
```

Return to guided explanation, p. 139

GUIDED SOURCE READING / CHAPTER 4

Read the filter as sort, select and average

The light, pH and TDS finish functions share trimmedMeanU32. Its purpose is to reduce the influence of extreme taps in a small fixed window.

Original excerpt: src/domain/calibration.c (p. 158) / lines 34-48. The complete unchanged file follows.

```
34  float trimmedMeanU32(uint32_t* samples, uint8_t n, uint8_t trim) {
35    if (n <= 2u * trim) return 0.0f;
36    for (uint8_t i = 1; i < n; ++i) {  // insertion sort: n is ~10
37      const uint32_t v = samples[i];
38      uint8_t j = i;
39      while (j > 0 && samples[j - 1] > v) {
40        samples[j] = samples[j - 1];
41        --j;
42      }
43      samples[j] = v;
44    }
45    uint64_t sum = 0;
46    for (uint8_t i = trim; i < n - trim; ++i) sum += samples[i];
47    return (float)sum / (float)(n - 2u * trim);
48  }
```

The first guard requires at least one retained element after trimming both ends. It occurs before sorting or division; otherwise an invalid trim count could create an empty average.

The insertion-sort loop starts at index 1 because a single element at index 0 is already sorted. v saves the new element before larger values shift right. The j >0 condition is checked before samples[j-1], preventing an access before the array. Storing v after the shifts closes the gap at its proper location.

Only after sorting do the extremes occupy known ends. The sum loop starts at trim and stops before n-trim. For ten taps and trim 2, it averages indices 2 through 7: six values. A uint64_t accumulator provides more headroom than a single 32-bit sample.

The function sorts the caller's array in place. That is appropriate at finish because the driver no longer needs acquisition order. Moving this sort into every poll would repeat work and rearrange an incomplete window. Filtering belongs after the samples exist and before the sensor-specific voltage conversion.

GUIDED SOURCE READING / CHAPTER 4

Derive the line before accepting its coefficients

Calibration turns reference observations into a model. The two-point helper first establishes an invalid output, then accepts a line only when the arithmetic is defined.

Original excerpt: src/domain/calibration.c (p. 158) / lines 16-32. The complete unchanged file follows.

```
16  bool linearFromTwoPoints(float x1, float y1, float x2, float y2, uint16_t revision,
17                           LinearCalibration* out) {
18    out->slope = 1.0f;
19    out->offset = 0.0f;
20    out->revision = revision;
21    out->valid = false;
22    if (!isfinite(x1) || !isfinite(y1) || !isfinite(x2) || !isfinite(y2)) return false;
23    const float dx = x2 - x1;
24    if (!(dx <= -1e-3f || dx >= 1e-3f)) return false;
25    const float slope = (y2 - y1) / dx;
26    const float offset = y1 - slope * x1;
27    if (!isfinite(slope) || !isfinite(offset)) return false;
28    out->slope = slope;
29    out->offset = offset;
30    out->valid = true;
31    return true;
32  }
```

The output starts with neutral coefficients, the requested revision and valid false. Any early return therefore leaves an explicit invalid result rather than partly updated coefficients that a caller might mistake for a completed fit.

Non-finite points are rejected before subtraction and division. dx is the separation between x references. Requiring magnitude at least 0.001 prevents division by zero and avoids fitting through practically coincident input points.

Slope is the output change divided by input change. Offset is then calculated using one reference: y1 minus slope times x1. Computing offset before slope would use an unestablished coefficient. Finite-result checks follow the arithmetic, and valid becomes true only after both coefficients are assigned.

The pH-specific wrapper adds buffer separation and plausible slope magnitude. TDS and light instead use a through-zero one-point gain. The generic mathematical fit and the sensor's plausibility checks belong in separate functions because they answer different questions.

ORIGINAL SOURCE / CHAPTER 4 / 1 OF 2

src/domain/calibration.c

Original lines 1-39 | Code and comments unchanged; long lines wrap visually

dividerRecoverMv (L5-8): Undo the resistor-divider attenuation to recover the module's output voltage.
linearApply (L10-12): Evaluate a stored straight-line calibration: slope times input plus offset.

```
#include "domain/calibration.h"

#include <math.h>

float dividerRecoverMv(float adcMv, uint32_t rTopOhm, uint32_t rBottomOhm) {
  if (rBottomOhm == 0) return 0.0f;
  return adcMv * (float)(rTopOhm + rBottomOhm) / (float)rBottomOhm;
}

float linearApply(const LinearCalibration* c, float x) {
  return c->slope * x + c->offset;
}

bool linearCalibrationFinite(const LinearCalibration* c) { return isfinite(c->slope) &&
isfinite(c->offset); }

bool linearFromTwoPoints(float x1, float y1, float x2, float y2, uint16_t revision,
                         LinearCalibration* out) {
  out->slope = 1.0f;
  out->offset = 0.0f;
  out->revision = revision;
  out->valid = false;
  if (!isfinite(x1) || !isfinite(y1) || !isfinite(x2) || !isfinite(y2)) return false;
  const float dx = x2 - x1;
  if (!(dx <= -1e-3f || dx >= 1e-3f)) return false;
  const float slope = (y2 - y1) / dx;
  const float offset = y1 - slope * x1;
  if (!isfinite(slope) || !isfinite(offset)) return false;
  out->slope = slope;
  out->offset = offset;
  out->valid = true;
  return true;
}

float trimmedMeanU32(uint32_t* samples, uint8_t n, uint8_t trim) {
  if (n <= 2u * trim) return 0.0f;
  for (uint8_t i = 1; i < n; ++i) {  // insertion sort: n is ~10
    const uint32_t v = samples[i];
    uint8_t j = i;
    while (j > 0 && samples[j - 1] > v) {
```

Return to guided explanation, p. 157

ORIGINAL SOURCE / CHAPTER 4 / 2 OF 2

src/domain/calibration.c

Original lines 40-79 | Code and comments unchanged; long lines wrap visually

trimmedMeanU32 (L34-48): Sort the small array, discard extremes and average the retained middle samples. **invalidFit (L50-55)**: Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.

```
      samples[j] = samples[j - 1];
      --j;
    }
    samples[j] = v;
  }
  uint64_t sum = 0;
  for (uint8_t i = trim; i < n - trim; ++i) sum += samples[i];
  return (float)sum / (float)(n - 2u * trim);
}

static void invalidFit(uint16_t revision, LinearCalibration* out) {
  out->slope = 1.0f;
  out->offset = 0.0f;
  out->revision = revision;
  out->valid = false;
}

bool linearGainFromPoint(float x1, float y1, float minGain, float maxGain, uint16_t revision,
                         LinearCalibration* out) {
  invalidFit(revision, out);
  if (!(x1 > 0.0f) || !(y1 > 0.0f) || !isfinite(x1) || !isfinite(y1)) return false;
  const float gain = y1 / x1;
  if (!isfinite(gain) || !(gain >= minGain && gain <= maxGain)) return false;
  out->slope = gain;
  out->valid = true;
  return true;
}

bool phFitFromBuffers(float mv1, float ph1, float mv2, float ph2, float minAbsSlope, float
maxAbsSlope,
                      uint16_t revision, LinearCalibration* out) {
  invalidFit(revision, out);
  const float dph = ph2 - ph1;
  if (!(dph <= -1.0f || dph >= 1.0f)) return false;  // also NaN
  LinearCalibration fit;
  if (!linearFromTwoPoints(mv1, ph1, mv2, ph2, revision, &fit)) return false;
  const float mag = fit.slope < 0.0f ? -fit.slope : fit.slope;
  if (!(mag >= minAbsSlope && mag <= maxAbsSlope)) return false;
  *out = fit;
  return true;
}
```

Return to guided explanation, p. 156

PART II / SENSOR ARCHITECTURES

CHAPTER 05

DFR0026: Light Measurement

The light driver is the first complete sensor example because its signal path is short: read pin voltage, collect ten spaced taps and filter the window. Calibration can change the displayed unit to lux while lamp control continues to use the retained pin voltage.

In this chapter

1. Trace ADC setup, sampling state and completion.
2. Calculate the ten-tap window and trimmed result.
3. Explain why calibrated lux and lamp thresholds use different fields.

Chapter contents

5.1 Light: ten small samples make one result 161
5.2 Light: from pin voltage to calibrated lux 162

Guided source readings begin on page 165.

Complete original listings begin on page 163. Shared files are linked in the explanations; Appendix A contains the full filename index.

CHAPTER 5

5.1 Light: ten small samples make one result

The ADC reads a voltage. This driver filters that voltage. Without a stored fit it publishes millivolts and UNCALIBRATED. With a valid reference fit it publishes lux and OK; raw still preserves pin mV.

Field	Plain meaning
samples_	The small array that collects the voltage readings.
count_	How many taps have been collected.
lastSampleMs_	When the previous actual tap happened.
saturated_	Whether any tap reached the ADC ceiling.
result_	The record copied to the rest of the system.

start takes the first tap. Each later poll can take one tap after 11 ms. After ten taps, the completion path sorts them, removes two values at each end, and averages six. The nominal window spans nine gaps, or 99 ms.

Why READY can still say UNCALIBRATED

READY means the window finished. UNCALIBRATED means no lux mapping has been verified. These statements can both be true. Lamp control deliberately uses this relative millivolt value, while generic numeric alarm rules require OK readings.

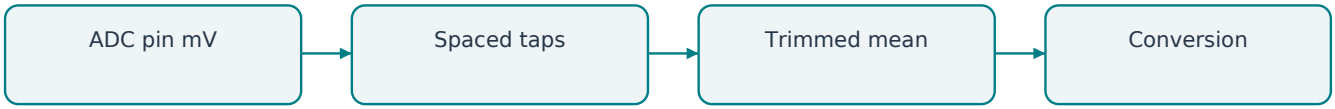
A clipped input means brighter than the ADC can resolve, so the lamp policy treats it as bright. A low input can be real darkness or a disconnected signal, so this driver cannot always detect an unplugged light lead.

The full driver/header and individual bring-up program follow. The ADC and trimmed-mean helper are printed in the shared-interface chapter. Lamp decision code has its own later chapter.

CHAPTER 5

5.2 Light: from pin voltage to calibrated lux

The DFR0026 produces an analog signal. The driver takes ten ADC taps spaced at least 11 ms apart, trims the two highest and two lowest and averages the six middle taps. The minimum nominal sampling span is 99 ms. All taps are short calls, so other work can progress between them.



Light -> mV pH -> divider + line fit TDS -> cubic + temperature

Health and quality are attached after interpretation.

raw always preserves pin voltage

The filtered pin voltage, rounded to integer millivolts, is stored in Reading.raw. A clipped tap sets the saturated quality flag. This is evidence of brightness beyond the measurable scale, not automatically a light-driver fault.

With no valid lux fit, value is pin mV, unit is MILLIVOLT and health is UNCALIBRATED. With a valid stored fit, value is slope times pin mV plus offset, unit is LUX and health is OK. Negative lux is clamped to zero; a non-finite result is FAULT. The fit revision is recorded.

Example: calibrating the displayed scale

An illustrative meter reference of 600 lux at 200 mV gives a through-zero gain of 3 lux/mV. The same 200 mV signal is then displayed as 600 lux. This fit defines a local linear model. Its accuracy depends on the sensor, light spectrum and reference measurements.

Lamp thresholds remain 40/120 pin mV. lampControlUpdate uses raw once value has lux units, so a scale change cannot silently change the relay threshold. The retained pin voltage keeps the control thresholds on the same scale after lux calibration.

What a voltage cannot tell

A disconnected input can resemble darkness and eventually request lamp ON. Filtering cannot distinguish every wiring failure. test_light prints results through the shared harness; test_lamp is a separate relay image. Production on-device calibration is explained in the Configuration chapter.

Original source in this PDF: src/drivers/light.c (p. 168); src/domain/lamp_control.c (p. 301); test/bringup/light_main.c (p. 172)

ORIGINAL SOURCE / CHAPTER 5 / 1 OF 2

src/drivers/light.h

Original lines 1-35 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // dfr0026_light driver. Work package WP3 (Stage 2). Architecture §3, §14.
2  // Source: DFR0026.PDF (DFRobot Analog Ambient Light Sensor V2, pages 1-4).
3  //
4  // Retained 3.3 V rail. S -> 10 kOhm series -> ADC1 pin (GPIO1), 1 uF to GND
5  // (tau ~10 ms). No divider: pin mV = module output mV. More light, higher
6  // voltage. Datasheet range 1-6000 lux, but the output has no lux unit: the FAQ
7  // requires a reference-meter mapping (D13). Without one every result is mV.
8  // With an on-device fit (Calibration page: one reference-meter reading, gain
9  // through zero) the value is lux. raw stays pin mV either way, and the lamp
10 // thresholds use it (lamp_control), so calibrating never moves the lamp.
11 //
12 // Filter: LIGHT_SAMPLES conversions spaced LIGHT_SAMPLE_SPACING_MS apart,
13 // sorted, mean of the middle LIGHT_SAMPLES - 2*LIGHT_TRIM. Mains lamps flicker
14 // at 100 Hz (120 Hz on 60 Hz grids) and the RC only cuts 100 Hz to ~16 %, so
15 // the spacing is deliberately NOT a multiple of 10 ms: 11 ms steps the flicker
16 // phase by 1 ms per tap and the window covers the whole period. The window is
17 // spread over successive poll() calls, never taken as a burst (§9): each tap is
18 // due LIGHT_SAMPLE_SPACING_MS after the previous actual sample.
19 //
20 // Results (poll() returns READY for all; raw = filtered pin mV):
21 //   calibration.valid -> value = lux, UNIT_LUX, HEALTH_OK
22 //   otherwise         -> value = module mV, UNIT_MILLIVOLT, HEALTH_UNCALIBRATED
23 //   any sample clipped -> kQualitySaturated, value kept as a lower bound
24 //                         (bright light on a 3.3 V supply can exceed the S3's
25 //                         ~3.1 V range; this is not a fault)
26 //   ADC error          -> HEALTH_FAULT
27 // No no-signal threshold: ~0 mV is real darkness. With 1 uF on the pin and no
28 // pull-down, an unplugged S line holds its last level while the capacitor leaks;
29 // the driver cannot tell that apart. Characterise it at bring-up.
30 //
31 // Worst-case blocking (one ADC conversion C, well under 1 ms):
32 //   init()    1 C   (throwaway conversion, checks the channel)
33 //   start()   1 C   (first sample of the window)
34 //   poll()    1 C   Pending until the window is full (>= 99 ms)
35 //   others    none
```

Return to guided explanation, p. 161

ORIGINAL SOURCE / CHAPTER 5 / 2 OF 2

src/drivers/light.h

Original lines 36-71 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
36  //
37  // Implements the SensorDriver contract in drivers/sensor_driver.h.
38  #pragma once
39  #include "drivers/sensor_driver.h"
40  #include "hal/adc_channel.h"
41  #include "domain/calibration.h"
42
43  #define LIGHT_SAMPLES 10
44  #define LIGHT_TRIM 2
45  #define LIGHT_SAMPLE_SPACING_MS 11
46
47  typedef struct LightConfig {
48    AdcChannel* adc;                // already begun on BOARD_PIN_LIGHT_ADC
49    LinearCalibration calibration;  // x = pin mV, y = lux; invalid = publish mV
50  } LightConfig;
51
52  // Members after `base` are private to light.c.
53  typedef struct LightDriver {
54    SensorDriver base;  // must stay first
55    LightConfig config_;
56    bool initialised_;
57    bool busy_;
58    bool saturated_;
59    uint8_t count_;
60    uint32_t samples_[LIGHT_SAMPLES];  // pin mV
61    MonoMs startMs_;
62    MonoMs lastSampleMs_;  // next tap is due LIGHT_SAMPLE_SPACING_MS after this
63    Reading result_;
64  } LightDriver;
65
66  // Constructor. Use &driver.base wherever a SensorDriver* is expected.
67  void lightDriverConstruct(LightDriver* self, const LightConfig* config);
68
69  // New lux fit from the applied configuration (on-device calibration, WP8).
70  // Called by the acquisition owner between windows; applies from the next one.
71  void lightDriverSetCalibration(LightDriver* self, const LinearCalibration* calibration);
```

Return to guided explanation, p. 161

GUIDED SOURCE READING / CHAPTER 5

Construction, initialization and start solve different problems

Use the light driver as the first complete analog example. Construction creates a valid software object; initialization checks the ADC path; start begins one new sampling window.

Original excerpt: src/drivers/light.c (p. 168) / lines 40-52. The complete unchanged file follows.

```
40  static DriverStatus lightStart(SensorDriver* d, MonoMs now) {
41    LightDriver* self = asLight(d);
42    if (!self->initialised_) return DRIVER_FAULT;
43    self->count_ = 0;
44    self->saturated_ = false;
45    self->startMs_ = now;
46    if (!sample(self, now)) {
47      self->initialised_ = false;
48      return DRIVER_FAULT;
49    }
50    self->busy_ = true;
51    return DRIVER_OK;
52  }
```

The cast retrieves the concrete driver. The initialised guard comes before sampling, because a configured bus/channel is a precondition for a useful read. count is reset before the first tap so the sample goes into element 0, not after the previous window's final element.

saturated is cleared for the new window, and startMs captures its beginning before the ADC call. sample then reads once, stores samples_[count_++] and records the actual sampling time. Post-increment uses the old index for storage and advances the count afterwards.

Only a successful first sample makes busy true. Setting busy before that test would leave a failed start appearing like an active window unless the error branch carefully undid it. The existing order establishes the state only after its precondition succeeds.

The constructor later installs kLightOps and resets persistent fields once. lightInit instead clears readiness and checks the ADC using a throwaway conversion. Reinitializing a peripheral and resetting the lifetime of a software Reading are deliberately separate operations. The following poll walkthrough shows how this initialized, busy window advances without a long sleep.

GUIDED SOURCE READING / CHAPTER 5

Why a late poll must not catch up ten ADC taps

After the first tap, acquisition can serve other sensors. lightPoll checks the clock each time the worker returns.

Original excerpt: src/drivers/light.c (p. 168) / lines 61-81. The complete unchanged file follows.

```
61    const MonoMs due = self->lastSampleMs_ + LIGHT_SAMPLE_SPACING_MS;
62    if (now < due) {
63      r.status = DRIVER_PENDING;
64      r.nextPollMs = due;
65      return r;
66    }
67    if (!sample(self, now)) {
68      self->busy_ = false;
69      self->initialised_ = false;
70      self->result_.health = HEALTH_FAULT;
71      self->result_.intervalStartMs = self->startMs_;
72      self->result_.intervalEndMs = now;
73      return r;
74    }
75    if (self->count_ < LIGHT_SAMPLES) {
76      r.status = DRIVER_PENDING;
77      r.nextPollMs = self->lastSampleMs_ + LIGHT_SAMPLE_SPACING_MS;
78      return r;
79    }
80    return finish(self, now);
81  }
```

due is based on lastSampleMs, the previous actual tap. If that tap occurred at 100 ms, the next is eligible at 111 ms. A poll at 105 ms returns PENDING and 111 as the next useful time; it performs no ADC access.

Suppose the next call is late and arrives at 130 ms. It takes one tap and records 130, so the following tap is due at 141. Replaying missed taps at 130 would create several nearly simultaneous readings instead of the intended spaced window. That would change the filter's exposure to flicker and noise.

On ADC failure, busy and initialized are cleared and the result is stamped FAULT for this interval. The early return stops the code before it can treat an incomplete array as ten successful taps. If more taps are needed, the function returns another deadline. Only a full count reaches finish.

Ten taps contain nine gaps, so the nominal minimum span is 9 times 11 =99 ms. Most of that time is available to other work. This is why a 99 ms measurement window is different from occupying the CPU for 99 ms.

GUIDED SOURCE READING / CHAPTER 5

Finish preserves the electrical observation before changing units

The sampling window is complete. finish now turns its samples into one observation while keeping the voltage needed by calibration and lamp policy.

Original excerpt: src/drivers/light.c (p. 168) / lines 87-104. The complete unchanged file follows.

```
  87    Reading* r = &self->result_;
  88    r->intervalStartMs = self->startMs_;
  89    r->intervalEndMs = now;
  90    r->hasRaw = true;
  91    r->raw = (int32_t)(pinMv + 0.5f);
  92    // Clipped = brighter than the ADC can show: a lower bound, not a fault.
  93    r->qualityFlags = self->saturated_ ? kQualitySaturated : kQualityNone;
  94    r->calibrationRevision = self->config_.calibration.revision;
  95    if (self->config_.calibration.valid) {
  96      r->unit = UNIT_LUX;
  97      r->value = linearApply(&self->config_.calibration, pinMv);
  98      if (r->value < 0.0f) r->value = 0.0f;
  99      r->health = HEALTH_OK;
 100      if (!isfinite(r->value)) {  // a broken fit is a fault, never a value
 101        r->health = HEALTH_FAULT;
 102        PollResult bad = {DRIVER_READY, now};
 103        return bad;
 104      }
```

The interval records when sampling began and ended. raw is assigned the rounded filtered pin voltage before the calibration branch. It remains in pin mV whether the later value is mV or lux; changing display units cannot erase the electrical input.

The saturated quality bit is attached independently of health. With a valid fit, the code assigns LUX, evaluates slope times voltage plus offset, clamps a negative result and rejects non-finite output. With no fit, it assigns MILLIVOLT and UNCALIBRATED. Units and health therefore describe the particular conversion performed.

lastGoodMs and the driver's sequence advance only after the accepted path. A FAULT returns before those success updates. Acquisition later assigns its own observation sequence to all stored outcomes, so keep the driver's good-result bookkeeping distinct from the service's publication identity.

readResult then copies the finished record into caller storage. cancel drops a partial window, while the calibration setter copies a fit without touching the ADC. Those final functions preserve the same ownership rule: acquisition controls driver calls, and consumers receive records rather than driving sampling themselves.

ORIGINAL SOURCE / CHAPTER 5 / 1 OF 4

src/drivers/light.c

Original lines 1-40 | Code and comments unchanged; long lines wrap visually

asLight (L7-7): Cast the common first-member SensorDriver pointer back to this concrete driver type.
lightId (L11-11): Return this module's sensor identity; this does not access hardware.

```
1  #include "drivers/light.h"
2
3  #include <math.h>
4
5  #include "domain/calibration.h"
6
7  static inline LightDriver* asLight(SensorDriver* d) { return (LightDriver*)d; }
8
9  static PollResult finish(LightDriver* self, MonoMs now);
10
11 static SensorId lightId(const SensorDriver* d) { (void)d; return SENSOR_LIGHT; }
12 static const char* lightName(const SensorDriver* d) { (void)d; return "dfr0026_light"; }
13 static uint8_t lightMetricCount(const SensorDriver* d) { (void)d; return 1; }
14
15 static DriverStatus lightInit(SensorDriver* d) {
16   LightDriver* self = asLight(d);
17   self->initialised_ = false;
18   self->busy_ = false;
19   if (self->config_.adc == 0) return DRIVER_FAULT;
20   // Throwaway conversion: fails if the channel was never begun on an ADC1 pin,
21   // and keeps any one-time ADC characterisation cost out of poll().
22   uint32_t mv = 0;
23   bool saturated = false;
24   if (!adcChannelReadMilliVolts(self->config_.adc, &mv, &saturated)) return DRIVER_FAULT;
25   self->initialised_ = true;
26   return DRIVER_OK;
27 }
28
29 // Take one sample of the window at `now`. false on an ADC error.
30 static bool sample(LightDriver* self, MonoMs now) {
31   uint32_t mv = 0;
32   bool saturated = false;
33   if (!adcChannelReadMilliVolts(self->config_.adc, &mv, &saturated)) return false;
34   self->samples_[self->count_++] = mv;
35   self->lastSampleMs_ = now;
36   self->saturated_ = self->saturated_ || saturated;
37   return true;
38 }
39
40 static DriverStatus lightStart(SensorDriver* d, MonoMs now) {
```

Return to guided explanation, p. 165

ORIGINAL SOURCE / CHAPTER 5 / 2 OF 4

src/drivers/light.c

Original lines 41-80 | Code and comments unchanged; long lines wrap visually

lightStart (L40-52): Begin one operation and record its state; completion is handled later by poll.
lightPoll (L54-81): Advance an in-flight operation only when useful; return PENDING with a future time, READY, or failure.

```
  LightDriver* self = asLight(d);
  if (!self->initialised_) return DRIVER_FAULT;
  self->count_ = 0;
  self->saturated_ = false;
  self->startMs_ = now;
  if (!sample(self, now)) {
    self->initialised_ = false;
    return DRIVER_FAULT;
  }
  self->busy_ = true;
  return DRIVER_OK;
}

static PollResult lightPoll(SensorDriver* d, MonoMs now) {
  LightDriver* self = asLight(d);
  PollResult r = {DRIVER_FAULT, now};
  if (!self->busy_) return r;

  // Spacing is measured from the previous actual sample, never from the window
  // start: after a late poll, catching up would compress the filter taps.
  const MonoMs due = self->lastSampleMs_ + LIGHT_SAMPLE_SPACING_MS;
  if (now < due) {
    r.status = DRIVER_PENDING;
    r.nextPollMs = due;
    return r;
  }
  if (!sample(self, now)) {
    self->busy_ = false;
    self->initialised_ = false;
    self->result_.health = HEALTH_FAULT;
    self->result_.intervalStartMs = self->startMs_;
    self->result_.intervalEndMs = now;
    return r;
  }
  if (self->count_ < LIGHT_SAMPLES) {
    r.status = DRIVER_PENDING;
    r.nextPollMs = self->lastSampleMs_ + LIGHT_SAMPLE_SPACING_MS;
    return r;
  }
  return finish(self, now);
```

Return to guided explanation, p. 165

ORIGINAL SOURCE / CHAPTER 5 / 3 OF 4

src/drivers/light.c

Original lines 81-119 | Code and comments unchanged; long lines wrap visually

lightPoll (L54-81): Advance an in-flight operation only when useful; return PENDING with a future time, READY, or failure.
finish (L83-114): Ten spaced taps become pin mV or calibrated lux; raw remains pin mV for lamp policy and calibration capture.

```
81  }
82
83  static PollResult finish(LightDriver* self, MonoMs now) {
84    self->busy_ = false;
85    const float pinMv = trimmedMeanU32(self->samples_, LIGHT_SAMPLES, LIGHT_TRIM);
86
87    Reading* r = &self->result_;
88    r->intervalStartMs = self->startMs_;
89    r->intervalEndMs = now;
90    r->hasRaw = true;
91    r->raw = (int32_t)(pinMv + 0.5f);
92    // Clipped = brighter than the ADC can show: a lower bound, not a fault.
93    r->qualityFlags = self->saturated_ ? kQualitySaturated : kQualityNone;
94    r->calibrationRevision = self->config_.calibration.revision;
95    if (self->config_.calibration.valid) {
96      r->unit = UNIT_LUX;
97      r->value = linearApply(&self->config_.calibration, pinMv);
98      if (r->value < 0.0f) r->value = 0.0f;
99      r->health = HEALTH_OK;
100     if (!isfinite(r->value)) {  // a broken fit is a fault, never a value
101       r->health = HEALTH_FAULT;
102       PollResult bad = {DRIVER_READY, now};
103       return bad;
104     }
105   } else {
106     r->unit = UNIT_MILLIVOLT;
107     r->value = pinMv;  // no divider: pin mV = module mV
108     r->health = HEALTH_UNCALIBRATED;  // no lux mapping until a reference meter (D13)
109   }
110   r->lastGoodMs = now;  // last usable result (OK or UNCALIBRATED)
111   ++r->sequence;
112   PollResult pr = {DRIVER_READY, now};
113   return pr;
114 }
115
116 static uint8_t lightReadResult(SensorDriver* d, Reading* out, uint8_t maxOut) {
117   if (maxOut < 1) return 0;
118   out[0] = asLight(d)->result_;
119   return 1;
```

Return to guided explanation, p. 167

ORIGINAL SOURCE / CHAPTER 5 / 4 OF 4

src/drivers/light.c

Original lines 120-159 | Code and comments unchanged; long lines wrap visually

lightReadResult (L116-120): Copy the latest Reading record(s) into caller storage. Inspect health even when poll returned READY.
lightCancel (L122-126): Stop software tracking of partial work. A chip conversion can still finish internally.

```
120 }
121
122 static void lightCancel(SensorDriver* d) {
123   LightDriver* self = asLight(d);
124   self->busy_ = false;
125   self->count_ = 0;
126 }
127
128 // Retained rail, and the ADC pin is a high-impedance input behind 10 kOhm:
129 // nothing to drive. Drop any partial window.
130 static void lightPreparePowerOff(SensorDriver* d) { lightCancel(d); }
131
132 static const SensorDriverOps kLightOps = {
133     .id = lightId,
134     .name = lightName,
135     .metricCount = lightMetricCount,
136     .init = lightInit,
137     .start = lightStart,
138     .poll = lightPoll,
139     .readResult = lightReadResult,
140     .cancel = lightCancel,
141     .preparePowerOff = lightPreparePowerOff,
142 };
143
144 void lightDriverConstruct(LightDriver* self, const LightConfig* config) {
145   self->base.ops = &kLightOps;
146   self->config_ = *config;
147   self->initialised_ = false;
148   self->busy_ = false;
149   self->saturated_ = false;
150   self->count_ = 0;
151   self->startMs_ = 0;
152   self->lastSampleMs_ = 0;
153   readingReset(&self->result_, SENSOR_LIGHT, METRIC_LIGHT_LEVEL, UNIT_MILLIVOLT);
154   self->result_.calibrationRevision = config->calibration.revision;
155 }
156
157 void lightDriverSetCalibration(LightDriver* self, const LinearCalibration* calibration) {
158   self->config_.calibration = *calibration;
159 }
```

Return to guided explanation, p. 167

ORIGINAL SOURCE / CHAPTER 5 / 1 OF 1

test/bringup/light_main.c

Original lines 1-40 | Code and comments unchanged; long lines wrap visually

setup (L28-38): Establish safe outputs, logger, gate, board check and queues; load configuration; start Safety, both acquisition workers, power, radios and supervision.
loop (L40-40): Park the Arduino loop task. The application work happens in explicitly created tasks.

```
1  // Bring-up entry point for light (WP3).  pio run -e test_light -t upload -t monitor
2  // Retained 3.3 V rail. S -> 10 kOhm -> GPIO1 (ADC1_CH0), 1 uF to GND.
3  // Replaces src/main.c in this env only; links the production driver.
4  //
5  // Every line prints module mV with "(uncal)": there is no lux mapping (D13).
6  // Procedure (README "DFR0026 light bring-up"), note the mV for each step:
7  //  1. Dark: cover the sensor completely.
8  //  2. Room light, sensor uncovered.
9  //  3. Phone torch close to the lens: find where it clips (~3050 mV at the pin;
10 //     the reading carries kQualitySaturated).
11 //  4. Under LED or fluorescent light: note the spread between successive lines
12 //     (flicker rejection of the 11 ms tap spacing).
13 //  5. Unplug S: watch how the reading decays (the 1 uF holds the last level).
14 #include <Arduino.h>
15 #include <stdio.h>
16
17 #include "app_config.h"
18 #include "board_profile.h"
19 #include "bringup_harness.h"
20 #include "hal/adc_channel.h"
21 #include "drivers/light.h"
22
23 static AdcChannel s_adc = ADC_CHANNEL_INIT;
24 static LightConfig s_config = {&s_adc};
25 static LightDriver s_driver;
26 static BringupContext s_ctx = {&s_driver.base, CFG_FRAME_PERIOD_MS, NULL};
27
28 void setup(void) {
29   lightDriverConstruct(&s_driver, &s_config);
30   delay(200);
31   printf("\n=== bring-up: %s ===\n", sensorDriverName(&s_driver.base));
32   if (!adcChannelBegin(&s_adc, BOARD_PIN_LIGHT_ADC)) {
33     printf("GPIO%d is not an ADC1 pin\n", (int)BOARD_PIN_LIGHT_ADC);
34   }
35   printf("%d taps, %d ms apart, trimmed mean; printing module mV (uncal)\n",
36          LIGHT_SAMPLES, LIGHT_SAMPLE_SPACING_MS);
37   bringupStart(&s_ctx);
38 }
39
40 void loop(void) { vTaskDelay(portMAX_DELAY); }
```

Return to guided explanation, p. 161

PART II / SENSOR ARCHITECTURES

CHAPTER 06

BME280: Air Temperature, Humidity and Pressure

BME280 replaces analog sampling with register transactions and factory correction coefficients. A single forced conversion produces three metrics. Temperature compensation must run first because pressure and humidity depend on its intermediate result.

In this chapter

1. Follow chip identification, reset and register configuration.
2. Separate trigger time, conversion wait and result read.
3. Decode packed values and apply temperature-first compensation.

Chapter contents

6.1 BME280: the simple reading path 174
6.2 BME280: identify, reset and configure 175
6.3 BME280: poll, compensate and publish 176

Guided source readings begin on page 179.

Complete original listings begin on page 177. Shared files are linked in the explanations; Appendix A contains the full filename index.

CHAPTER 6

6.1 BME280: the simple reading path

The chip performs three measurements. The driver sends one trigger, lets the chip convert, then reads and corrects its raw numbers.

Driver step	Read it as
Construct	Set the operation table and three empty Reading records.
init	Check the chip, reset it and load its factory coefficients.
start	Write one register to request a measurement.
poll	Wait until useful, check status and read eight data bytes.
compensateT/P/H	Turn raw counts into physical values using factory coefficients.
readResult	Copy the three records to acquisition.

Temperature is calculated first because pressure and humidity use its intermediate correction, tFine_. This is a calculation dependency inside one driver, not a separate FreeRTOS task.

Example of scale

A compensated temperature integer of 2512 becomes 25.12 degC after division by 100. A pressure result in Q24.8 must be divided by 256 to become Pa. These divisors describe units, not arbitrary averaging.

Read the listing with these fields in mind

busy_ means a conversion is in flight. initialised_ means the driver believes chip settings are installed. startMs_ tells poll when it is worth reading. results_ holds the last verdict for each metric, including faults.

The full header, driver and individual test program are in this chapter. The shared I2C HAL and Reading definition are printed elsewhere in the same PDF and are linked from the source directory. Production fixes the address to 0x76; the individual test can probe 0x77 too.

CHAPTER 6

6.2 BME280: identify, reset and configure

BME280 measures air temperature, relative humidity and pressure. Production uses address 0x76. The bring-up program probes 0x76 and 0x77; production does not automatically select the alternate address. A correctly wired sensor strapped to 0x77 can therefore work in bring-up but fail with the production configuration.

bme280Init()

The function clears its internal initialised/busy flags, reads chip ID register 0xD0 and requires 0x60. This rejects a BMP280, which does not provide humidity. It then writes reset command 0xB6, allows startup time and makes a bounded check that NVM calibration copying has finished.

`readCalibration()` reads two coefficient blocks. `u16le()` and `s16le()` decode unsigned and signed little-endian words. Humidity coefficients H4 and H5 are split across bytes, so the code reconstructs their nibbles and sign. Basic nonzero/non-0xFFFF checks reject obvious empty coefficient storage.

The driver selects x1 oversampling for all three quantities, no IIR filter and forced mode. Forced mode means one measurement per trigger, followed by sleep. Acquisition's 250 ms release provides the cadence; the chip does not independently free-run at 4 Hz. [R5]

bme280Start()

Writing `0x25` to ctrl_meas requests a forced temperature/pressure measurement. The humidity oversampling setting is latched by that write. The driver records startMs and busy, then returns immediately. Conversion time is spent inside the sensor.

```
// Source: src/drivers/bme280.c
static const uint8_t kCtrlMeasForcedX1 =
    (1u << 5) | (1u << 2) | 0x01;
```

The shifts select the x1 oversampling fields; the low bits select forced mode. A failed trigger invalidates internal initialisation so later service recovery can configure the sensor again.

The initialisation budget covers a collection of bus calls and short delays. It is more expensive than a normal measurement trigger and should not be repeated after every successful sample.

Original source in this PDF: src/drivers/bme280.c (p. 182); src/drivers/bme280.h (p. 177); test/bringup/bme280_main.c (p. 189)

CHAPTER 6

6.3 BME280: poll, compensate and publish

`bme280Poll()` first waits until start + 10 ms. Reading the measuring flag immediately after the trigger could see it before conversion begins, so an immediate clear flag is not treated as proof of a fresh result.

After that wait, the driver reads ctrl_hum and status together. If ctrl_hum has returned to its reset setting, the module may have power-cycled. If measuring is still set, poll returns PENDING, or TIMEOUT once the 12 ms configured budget is reached.

When ready, one eight-byte burst reads pressure, temperature and humidity. The burst keeps the chip's shadowed values consistent within that acquisition. Pressure and temperature are reconstructed as 20-bit values; humidity as 16 bits. All-ones results are rejected as implausible bus data.

Three compensation functions

`compensateT()` uses T1-T3 and saves `tFine_`, an intermediate temperature correction. `compensateP()` uses that correction and P1-P9, with 64-bit intermediate arithmetic and a division-by-zero guard. `compensateH()` uses tFine and H1-H6 and clamps output to the valid humidity range.

The output array is temperature, humidity, pressure, matching the Reading identities created by the constructor. Integer outputs are divided by 100, 1024 and 256 respectively. The published pressure is Pa; a display wanting hPa divides by 100.

Status details worth noticing

The poll returns READY even if only pressure fails its range check. That pressure Reading becomes FAULT, while other accepted metrics can remain OK. `fail()` handles whole-operation failure: it changes all three health fields, clears busy/initialised, and preserves prior numeric values and lastGoodMs.

`bme280ReadResult()` copies up to three records. `bme280Cancel()` drops the pending software operation; the forced conversion can finish by itself. `bme280PreparePowerOff()` also demands a new init. The constructor resets results only once, allowing sequences and last-good times to survive later re-initialisation.

The 12 ms timeout is checked while the measuring bit remains set; a late poll finding completed data can still accept it. It is not a strict universal rejection of every sample read after 12 ms.

Original source in this PDF: src/drivers/bme280.c (p. 182); src/drivers/bme280.h (p. 177)

ORIGINAL SOURCE / CHAPTER 6 / 1 OF 2

src/drivers/bme280.h

Original lines 1-27 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // bme280 driver. Work package WP1 (Stage 2). Architecture §4, §7, §14.
2  // Datasheet: Bosch BST-BME280-DS001 rev 1.24.
3  //
4  // Verify chip ID 0x60 (rejects BMP280); read compensation coefficients once
5  // in init(). Forced mode, x1 oversampling T/P/H, IIR off; Bosch integer
6  // compensation (§4.2.3). Produces AirTemperature, AirHumidity, AirPressure.
7  //
8  // Worst-case blocking (I2C_BUS_TIMEOUT_MS = T per transaction):
9  //   init()    ~13 ms + 16 T  (soft reset + bounded NVM-copy wait)
10 //   start()   1 T            (one register write)
11 //   poll()    2 T            (ctrl_hum/status + one 8-byte burst); Pending
12 //                            until start+10 ms (t_max 9.3 ms), Timeout
13 //                            after CFG_BME280_BUDGET_MS
14 //   others    no bus access
15 // Any bus fault, timeout or invalid sample clears the initialised state, so
16 // the next start() returns Fault and the caller re-runs init(). poll() also
17 // detects a module power-cycle (ctrl_hum back to reset 0): it fails the
18 // sample and clears the initialised state; the caller then re-runs init().
19 //
20 // Implements the SensorDriver contract in drivers/sensor_driver.h.
21 #pragma once
22 #include "drivers/sensor_driver.h"
23 #include "hal/i2c_bus.h"
24
25 typedef struct Bme280Config {
26   I2cBus* bus;
27   uint8_t address;  // BOARD_BME280_I2C_ADDRESS
```

Return to guided explanation, p. 174

ORIGINAL SOURCE / CHAPTER 6 / 2 OF 2

src/drivers/bme280.h

Original lines 28-55 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
28 } Bme280Config;
29
30 #define BME280_METRICS 3  // results_ index: 0 T, 1 H, 2 P
31
32 typedef struct Bme280Calib {
33   uint16_t t1;
34   int16_t t2, t3;
35   uint16_t p1;
36   int16_t p2, p3, p4, p5, p6, p7, p8, p9;
37   uint8_t h1, h3;
38   int16_t h2, h4, h5;
39   int8_t h6;
40 } Bme280Calib;
41
42 // Members after `base` are private to bme280.c.
43 typedef struct Bme280Driver {
44   SensorDriver base;  // must stay first
45   Bme280Config config_;
46   Bme280Calib calib_;
47   int32_t tFine_;
48   MonoMs startMs_;
49   bool initialised_;
50   bool busy_;
51   Reading results_[BME280_METRICS];
52 } Bme280Driver;
53
54 // Constructor. Use &driver.base wherever a SensorDriver* is expected.
55 void bme280DriverConstruct(Bme280Driver* self, const Bme280Config* config);
```

Return to guided explanation, p. 174

GUIDED SOURCE READING / CHAPTER 6

Identify, reset, then trust factory coefficients

The BME280 converts register codes using factory calibration. Initialization must establish that the responding chip and its internal setup are the expected ones before regular measurements begin.

Original excerpt: src/drivers/bme280.c (p. 182) / lines 57-70. The complete unchanged file follows.

```
57    uint8_t id = 0;
58    if (!i2cBusReadRegs(bus, address, kRegChipId, &id, 1)) {
59      i2cBusRecover(bus);
60      return DRIVER_FAULT;
61    }
62    if (id != kChipId) return DRIVER_NOT_PRESENT;
63
64    if (!i2cBusWriteReg(bus, address, kRegReset, kResetCmd)) return DRIVER_FAULT;
65    // t_startup <= 2 ms (Table 1). delay(n) may return up to 1 tick early, so
66    // a failed read is "not ready yet" until the bounded wait runs out.
67    delay(3);
68    bool ready = false;
69    for (uint8_t i = 0; i < kResetWaitMaxMs && !ready; ++i) {
70      uint8_t status = kStatusImUpdate;
```

The ID read is attempted before interpreting any calibration bytes. A bus-transfer failure requests bounded bus recovery and returns FAULT; a responding but different ID returns NOT_PRESENT. These distinguish communication failure from the wrong device at that address.

Reset follows identification. The delay and bounded status loop wait for internal startup and NVM coefficient copying. Reading coefficients before that work finishes could feed incorrect constants into every later compensation calculation.

The remaining initializer reads calibration and programs humidity oversampling and filtering. ctrl_hum must be established before the later ctrl_meas write starts forced conversion and latches that setting. Readback checks the selected bits rather than assuming a successful I2C write proves the desired state.

initialised becomes true at the end. Every earlier return leaves it false, so start cannot claim a correctly configured device after only part of initialization succeeded. This pattern is repeated throughout the drivers: invalidate first, perform dependencies, commit readiness last.

GUIDED SOURCE READING / CHAPTER 6

Unpack bytes before applying temperature-dependent corrections

After a forced conversion, the poll routine retrieves one burst containing pressure, temperature and humidity. The next lines reconstruct the chip's packed integers.

Original excerpt: src/drivers/bme280.c (p. 182) / lines 181-195. The complete unchanged file follows.

```
181    const int32_t adcP = ((int32_t)dat[0] << 12) | (dat[1] << 4) | (dat[2] >> 4);
182    const int32_t adcT = ((int32_t)dat[3] << 12) | (dat[4] << 4) | (dat[5] >> 4);
183    const int32_t adcH = ((int32_t)dat[6] << 8) | dat[7];
184
185    // All-ones means a floating bus (impossible as real T/P data: at x1 the low
186    // 4 bits are 0). 0x80000/0x8000 are NOT checked: at x1 they are valid
187    // mid-scale readings (lost settings are detected via ctrl_hum above).
188    if (adcT == 0xFFFFF || adcP == 0xFFFFF || adcH == 0xFFFF) {
189      return fail(self, DRIVER_FAULT, HEALTH_FAULT, now);
190    }
191
192    // Temperature first: it provides tFine_ for P and H.
193    const int32_t tCenti = compensateT(self, adcT);
194    if (tCenti < kTminCenti || tCenti > kTmaxCenti) {
195      return fail(self, DRIVER_FAULT, HEALTH_FAULT, now);
```

Pressure and temperature use 20-bit fields spread across three bytes: the first byte shifts 12 places, the second 4, and the final high nibble fills the low 4 bits. Humidity is a 16-bit pair. Bitwise OR joins non-overlapping fields; it is not arithmetic addition of three independent sensor values.

All-ones checks reject a characteristic unusable pattern before compensation. The temperature function then runs first because it sets tFine, an intermediate used by both pressure and humidity. Reordering pressure ahead of temperature would use a stale tFine from an earlier observation or initialization.

The temperature range check also precedes those dependent calculations. It stops publication when the prerequisite temperature is outside the supported range. Pressure and humidity are then compensated and converted from their fixed-point scales into declared engineering units.

The subsequent arrays deliberately use metric order temperature, humidity, pressure, even though the register burst begins with pressure. Matching values, raws and result identities is more important than retaining byte order. The final loop applies each metric's validity before advancing its last-good information.

GUIDED SOURCE READING / CHAPTER 6

The dense arithmetic is fixed-point scaling

compensateT is short but looks difficult because it implements the manufacturer's correction in scaled integers. Read each intermediate by its inputs and output scale before trying to memorize shifts.

Original excerpt: src/drivers/bme280.c (p. 182) / lines 248-256. The complete unchanged file follows.

```
248  static int32_t compensateT(Bme280Driver* self, int32_t adcT) {
249    const Bme280Calib* c = &self->calib_;
250    const int32_t t1 = c->t1;
251    int32_t var1 = ((((adcT >> 3) - (t1 << 1))) * (int32_t)c->t2) >> 11;
252    int32_t var2 = (((((adcT >> 4) - t1) * ((adcT >> 4) - t1)) >> 12) *
253                    (int32_t)c->t3) >> 14;
254    self->tFine_ = var1 + var2;
255    return (self->tFine_ * 5 + 128) >> 8;
256  }
```

c points to this chip's factory coefficients. t1 is copied into a signed 32-bit local before the arithmetic, and t2/t3 participate as signed coefficients. The two var terms correct different parts of the raw temperature response. Their shifts keep intermediate values on the scale required by the reference equation.

tFine is stored in the driver before returning the final temperature. This assignment has a second consumer: compensateP and compensateH use it immediately afterwards. The return expression scales and rounds tFine into hundredths of a degree Celsius; the later division by 100 produces the float Reading.

Pressure uses 64-bit intermediates because products and offsets exceed safe 32-bit ranges. Its zero-divisor guard occurs before division and returns an invalid code for the caller to reject. Humidity clamps its corrected integer before conversion to percent RH.

These coefficients and scales come from the BME280 compensation algorithm, rather than adjustable plant-control policy. The full listing preserves every term. The engineering task here is to preserve widths, signs, sequence and output units when reading or modifying that implementation.

ORIGINAL SOURCE / CHAPTER 6 / 1 OF 7

src/drivers/bme280.c

Original lines 1-47 | Code and comments unchanged; long lines wrap visually

u16le (L35-35): Follow identity/reset, forced start, delayed poll, raw-byte unpacking, compensation and per-metric health.
s16le (L36-36): Follow identity/reset, forced start, delayed poll, raw-byte unpacking, compensation and per-metric health.

```
#include "drivers/bme280.h"

#include <Arduino.h>
#include <string.h>

#include "app_config.h"

// Registers (datasheet §5.3).
static const uint8_t kRegCalib00 = 0x88;  // ..0xA1, 26 bytes
static const uint8_t kRegChipId = 0xD0;
static const uint8_t kRegReset = 0xE0;
static const uint8_t kRegCalib26 = 0xE1;  // ..0xE7, 7 bytes
static const uint8_t kRegCtrlHum = 0xF2;
static const uint8_t kRegStatus = 0xF3;
static const uint8_t kRegCtrlMeas = 0xF4;
static const uint8_t kRegConfig = 0xF5;
static const uint8_t kRegData = 0xF7;     // ..0xFE, 8 bytes (P, T, H)

static const uint8_t kChipId = 0x60;
static const uint8_t kResetCmd = 0xB6;
static const uint8_t kStatusMeasuring = 1u << 3;
static const uint8_t kStatusImUpdate = 1u << 0;

static const uint8_t kCtrlHumX1 = 0x01;     // osrs_h = x1
static const uint8_t kConfigNoFilter = 0x00;  // IIR off, t_sb unused in forced mode
static const uint8_t kCtrlMeasForcedX1 = (1u << 5) | (1u << 2) | 0x01;  // 0x25

static const uint32_t kFirstPollMs = 10;     // > t_max 9.3 ms at x1/x1/x1 (§9.1)
static const uint8_t kResetWaitMaxMs = 10;   // bounded NVM-copy wait in init()

// Operating range (§1): -40..85 degC, 0..100 %RH, 300..1100 hPa.
static const int32_t kTminCenti = -4000, kTmaxCenti = 8500;
static const float kPminPa = 30000.0f, kPmaxPa = 110000.0f;

static inline uint16_t u16le(const uint8_t* b) { return (uint16_t)(b[0] | (b[1] << 8)); }
static inline int16_t s16le(const uint8_t* b) { return (int16_t)u16le(b); }

static inline Bme280Driver* asBme280(SensorDriver* d) { return (Bme280Driver*)d; }

static bool readCalibration(Bme280Driver* self);
static PollResult fail(Bme280Driver* self, DriverStatus status, Health health, MonoMs now);
static int32_t compensateT(Bme280Driver* self, int32_t adcT);   // 0.01 degC, sets tFine_
static uint32_t compensateP(Bme280Driver* self, int32_t adcP);  // Q24.8 Pa, 0 = invalid
static uint32_t compensateH(Bme280Driver* self, int32_t adcH);  // Q22.10 %RH

static SensorId bme280Id(const SensorDriver* d) { (void)d; return SENSOR_BME280; }
static const char* bme280Name(const SensorDriver* d) { (void)d; return "bme280"; }
```

Return to guided explanation, p. 179

ORIGINAL SOURCE / CHAPTER 6 / 2 OF 7

src/drivers/bme280.c

Original lines 48-94 | Code and comments unchanged; long lines wrap visually

bme280MetricCount (L48-48): Report how many Reading records one result contains.
bme280Init (L50-94): Check dependencies, configure this module and return whether initialisation succeeded.

```
48  static uint8_t bme280MetricCount(const SensorDriver* d) { (void)d; return BME280_METRICS; }
49
50  static DriverStatus bme280Init(SensorDriver* d) {
51    Bme280Driver* self = asBme280(d);
52    self->initialised_ = false;
53    self->busy_ = false;
54    I2cBus* bus = self->config_.bus;
55    const uint8_t address = self->config_.address;
56
57    uint8_t id = 0;
58    if (!i2cBusReadRegs(bus, address, kRegChipId, &id, 1)) {
59      i2cBusRecover(bus);
60      return DRIVER_FAULT;
61    }
62    if (id != kChipId) return DRIVER_NOT_PRESENT;
63
64    if (!i2cBusWriteReg(bus, address, kRegReset, kResetCmd)) return DRIVER_FAULT;
65    // t_startup <= 2 ms (Table 1). delay(n) may return up to 1 tick early, so
66    // a failed read is "not ready yet" until the bounded wait runs out.
67    delay(3);
68    bool ready = false;
69    for (uint8_t i = 0; i < kResetWaitMaxMs && !ready; ++i) {
70      uint8_t status = kStatusImUpdate;
71      ready = i2cBusReadRegs(bus, address, kRegStatus, &status, 1) &&
72              (status & kStatusImUpdate) == 0;
73      if (!ready) delay(1);
74    }
75    if (!ready) return DRIVER_TIMEOUT;
76
77    if (!readCalibration(self)) return DRIVER_FAULT;
78
79    // Sleep mode after reset, so config writes are accepted (§5.4.6).
80    // ctrl_hum takes effect on the ctrl_meas write in start() (§5.4.3).
81    uint8_t check[2] = {0, 0};
82    if (!i2cBusWriteReg(bus, address, kRegCtrlHum, kCtrlHumX1) ||
83        !i2cBusWriteReg(bus, address, kRegConfig, kConfigNoFilter) ||
84        !i2cBusReadRegs(bus, address, kRegCtrlHum, &check[0], 1) ||
85        !i2cBusReadRegs(bus, address, kRegConfig, &check[1], 1)) {
86      return DRIVER_FAULT;
87    }
88    if ((check[0] & 0x07) != kCtrlHumX1 || (check[1] & 0xFC) != kConfigNoFilter) {
89      return DRIVER_FAULT;
90    }
91
92    self->initialised_ = true;
93    return DRIVER_OK;
94  }
```

Return to guided explanation, p. 179

ORIGINAL SOURCE / CHAPTER 6 / 3 OF 7

src/drivers/bme280.c

Original lines 95-142 | Code and comments unchanged; long lines wrap visually

readCalibration (L96-126): Read BME280 factory coefficients and reconstruct signed words and split humidity fields.
bme280Start (L128-139): Begin one operation and record its state; completion is handled later by poll.

```
95
96  static bool readCalibration(Bme280Driver* self) {
97    uint8_t a[26];
98    uint8_t b[7];
99    if (!i2cBusReadRegs(self->config_.bus, self->config_.address, kRegCalib00, a, sizeof(a)) ||
100       !i2cBusReadRegs(self->config_.bus, self->config_.address, kRegCalib26, b, sizeof(b))) {
101     return false;
102   }
103   Bme280Calib* c = &self->calib_;
104   // Table 16. a[0] = 0x88, b[0] = 0xE1.
105   c->t1 = u16le(&a[0]);
106   c->t2 = s16le(&a[2]);
107   c->t3 = s16le(&a[4]);
108   c->p1 = u16le(&a[6]);
109   c->p2 = s16le(&a[8]);
110   c->p3 = s16le(&a[10]);
111   c->p4 = s16le(&a[12]);
112   c->p5 = s16le(&a[14]);
113   c->p6 = s16le(&a[16]);
114   c->p7 = s16le(&a[18]);
115   c->p8 = s16le(&a[20]);
116   c->p9 = s16le(&a[22]);
117   c->h1 = a[25];                                  // 0xA1
118   c->h2 = s16le(&b[0]);                           // 0xE1/0xE2
119   c->h3 = b[2];                                   // 0xE3
120   c->h4 = (int16_t)(((int8_t)b[3] * 16) | (b[4] & 0x0F));  // E4[11:4] E5[3:0]
121   c->h5 = (int16_t)(((int8_t)b[5] * 16) | (b[4] >> 4));    // E6[11:4] E5[7:4]
122   c->h6 = (int8_t)b[6];                           // 0xE7
123
124   // Blank NVM or a bus returning 0x00/0xFF would make these 0/0xFFFF.
125   return c->t1 != 0 && c->p1 != 0 && c->t1 != 0xFFFF && c->p1 != 0xFFFF;
126 }
127
128 static DriverStatus bme280Start(SensorDriver* d, MonoMs now) {
129   Bme280Driver* self = asBme280(d);
130   if (!self->initialised_) return DRIVER_FAULT;
131   if (!i2cBusWriteReg(self->config_.bus, self->config_.address, kRegCtrlMeas,
132                       kCtrlMeasForcedX1)) {
133     self->initialised_ = false;
134     return DRIVER_FAULT;
135   }
136   self->startMs_ = now;
137   self->busy_ = true;
138   return DRIVER_OK;
139 }
140
141 static PollResult bme280Poll(SensorDriver* d, MonoMs now) {
142   Bme280Driver* self = asBme280(d);
```

Return to guided explanation, p. 179

ORIGINAL SOURCE / CHAPTER 6 / 4 OF 7

src/drivers/bme280.c

Original lines 143-189 | Code and comments unchanged; long lines wrap visually

bme280Poll (L141-231): Advance an in-flight operation only when useful; return PENDING with a future time, READY, or failure.

```
143   I2cBus* bus = self->config_.bus;
144   const uint8_t address = self->config_.address;
145   const MonoMs startMs = self->startMs_;
146
147   if (!self->busy_) return fail(self, DRIVER_FAULT, HEALTH_FAULT, now);
148
149   // The measuring bit may not be set yet right after the trigger, so the
150   // first status check waits until the maximum conversion time has passed.
151   if (now < startMs + kFirstPollMs) {
152     PollResult r = {DRIVER_PENDING, startMs + kFirstPollMs};
153     return r;
154   }
155
156   // ctrl_hum + status in one burst. ctrl_hum back at its reset value means the
157   // module was power-cycled and lost its settings: fail, so the caller re-inits.
158   uint8_t cs[2] = {0, 0};
159   if (!i2cBusReadRegs(bus, address, kRegCtrlHum, cs, sizeof(cs))) {
160     i2cBusRecover(bus);
161     return fail(self, DRIVER_FAULT, HEALTH_FAULT, now);
162   }
163   if ((cs[0] & 0x07) != kCtrlHumX1) return fail(self, DRIVER_FAULT, HEALTH_FAULT, now);
164   const uint8_t status = cs[1];
165   if (status & kStatusMeasuring) {
166     if (now - startMs >= CFG_BME280_BUDGET_MS) {
167       return fail(self, DRIVER_TIMEOUT, HEALTH_TIMEOUT, now);
168     }
169     PollResult r = {DRIVER_PENDING, now + 1};
170     return r;
171   }
172
173   // One burst read: shadowing keeps P/T/H consistent only within it (§4.1).
174   uint8_t dat[8];
175   if (!i2cBusReadRegs(bus, address, kRegData, dat, sizeof(dat))) {
176     i2cBusRecover(bus);
177     return fail(self, DRIVER_FAULT, HEALTH_FAULT, now);
178   }
179   self->busy_ = false;
180
181   const int32_t adcP = ((int32_t)dat[0] << 12) | (dat[1] << 4) | (dat[2] >> 4);
182   const int32_t adcT = ((int32_t)dat[3] << 12) | (dat[4] << 4) | (dat[5] >> 4);
183   const int32_t adcH = ((int32_t)dat[6] << 8) | dat[7];
184
185   // All-ones means a floating bus (impossible as real T/P data: at x1 the low
186   // 4 bits are 0). 0x80000/0x8000 are NOT checked: at x1 they are valid
187   // mid-scale readings (lost settings are detected via ctrl_hum above).
188   if (adcT == 0xFFFFF || adcP == 0xFFFFF || adcH == 0xFFFF) {
189     return fail(self, DRIVER_FAULT, HEALTH_FAULT, now);
```

Return to guided explanation, p. 180

ORIGINAL SOURCE / CHAPTER 6 / 5 OF 7

src/drivers/bme280.c

Original lines 190-237 | Code and comments unchanged; long lines wrap visually

bme280Poll (L141-231): Advance an in-flight operation only when useful; return PENDING with a future time, READY, or failure.
fail (L235-245): End the failed operation, stamp a fault verdict and preserve earlier numerical history as non-actionable.

```
  }

  // Temperature first: it provides tFine_ for P and H.
  const int32_t tCenti = compensateT(self, adcT);
  if (tCenti < kTminCenti || tCenti > kTmaxCenti) {
    return fail(self, DRIVER_FAULT, HEALTH_FAULT, now);
  }
  const uint32_t pQ8 = compensateP(self, adcP);
  const uint32_t hQ10 = compensateH(self, adcH);

  const float values[BME280_METRICS] = {
      (float)tCenti / 100.0f,
      (float)hQ10 / 1024.0f,
      (float)pQ8 / 256.0f,
  };
  const int32_t raws[BME280_METRICS] = {adcT, adcH, adcP};
  const bool valid[BME280_METRICS] = {
      true,
      true,  // compensateH clamps to 0..100 %RH
      pQ8 != 0 && values[2] >= kPminPa && values[2] <= kPmaxPa,
  };

  for (uint8_t i = 0; i < BME280_METRICS; ++i) {
    Reading* r = &self->results_[i];
    r->intervalStartMs = startMs;
    r->intervalEndMs = now;
    r->hasRaw = true;
    r->raw = raws[i];
    r->qualityFlags = kQualityNone;
    if (valid[i]) {
      r->health = HEALTH_OK;
      r->value = values[i];
      r->lastGoodMs = now;
      ++r->sequence;
    } else {
      r->health = HEALTH_FAULT;
    }
  }

  PollResult r = {DRIVER_READY, now};
  return r;
}

// Abort the measurement, mark every metric with `health` and force a re-init
// on the next start(). The last value is kept but never reported as Ok.
static PollResult fail(Bme280Driver* self, DriverStatus status, Health health, MonoMs now) {
  self->busy_ = false;
  self->initialised_ = false;
```

Return to guided explanation, p. 180

ORIGINAL SOURCE / CHAPTER 6 / 6 OF 7

src/drivers/bme280.c

Original lines 238-284 | Code and comments unchanged; long lines wrap visually

fail (L235-245): End the failed operation, stamp a fault verdict and preserve earlier numerical history as non-actionable.
compensateT (L248-256): Apply BME280 factory temperature coefficients and save tFine for pressure/humidity.

```
  for (uint8_t i = 0; i < BME280_METRICS; ++i) {
    self->results_[i].health = health;
    self->results_[i].intervalStartMs = self->startMs_;
    self->results_[i].intervalEndMs = now;
  }
  PollResult r = {status, now};
  return r;
}

// Datasheet §4.2.3 (rev 1.1 reference code), fixed-point.
static int32_t compensateT(Bme280Driver* self, int32_t adcT) {
  const Bme280Calib* c = &self->calib_;
  const int32_t t1 = c->t1;
  int32_t var1 = ((((adcT >> 3) - (t1 << 1))) * (int32_t)c->t2) >> 11;
  int32_t var2 = (((((adcT >> 4) - t1) * ((adcT >> 4) - t1)) >> 12) *
                  (int32_t)c->t3) >> 14;
  self->tFine_ = var1 + var2;
  return (self->tFine_ * 5 + 128) >> 8;
}

static uint32_t compensateP(Bme280Driver* self, int32_t adcP) {
  const Bme280Calib* c = &self->calib_;
  int64_t var1 = (int64_t)self->tFine_ - 128000;
  int64_t var2 = var1 * var1 * c->p6;
  var2 = var2 + ((var1 * c->p5) * 131072);              // << 17
  var2 = var2 + ((int64_t)c->p4 * 34359738368LL);       // << 35
  var1 = ((var1 * var1 * c->p3) >> 8) + ((var1 * c->p2) * 4096);  // << 12
  var1 = (((int64_t)1 << 47) + var1) * c->p1 >> 33;
  if (var1 == 0) return 0;  // avoid division by zero
  int64_t p = 1048576 - adcP;
  p = (((p << 31) - var2) * 3125) / var1;
  var1 = ((int64_t)c->p9 * (p >> 13) * (p >> 13)) >> 25;
  var2 = ((int64_t)c->p8 * p) >> 19;
  p = ((p + var1 + var2) >> 8) + ((int64_t)c->p7 * 16);  // << 4; p7 is signed
  return (uint32_t)p;
}

static uint32_t compensateH(Bme280Driver* self, int32_t adcH) {
  const Bme280Calib* c = &self->calib_;
  int32_t v = self->tFine_ - 76800;
  // h4 is signed: multiply, not << 20 (a negative left shift is undefined in C).
  v = (((((adcH << 14) - ((int32_t)c->h4 * 1048576) -
          ((int32_t)c->h5 * v)) + 16384) >> 15) *
       (((((((v * (int32_t)c->h6) >> 10) *
            (((v * (int32_t)c->h3) >> 11) + 32768)) >> 10) + 2097152) *
            (int32_t)c->h2 + 8192) >> 14));
  v = v - (((((v >> 15) * (v >> 15)) >> 7) * (int32_t)c->h1) >> 4);
```

Return to guided explanation, p. 181

ORIGINAL SOURCE / CHAPTER 6 / 7 OF 7

src/drivers/bme280.c

Original lines 285-332 | Code and comments unchanged; long lines wrap visually

compensateH (L275-288): Use tFine and humidity coefficients, clamp to the valid range and return a Q22.10 integer.
bme280ReadResult (L290-296): Copy the latest Reading record(s) into caller storage. Inspect health even when poll returned READY.

```
285   v = v < 0 ? 0 : v;
286   v = v > 419430400 ? 419430400 : v;
287   return (uint32_t)(v >> 12);
288 }
289
290 static uint8_t bme280ReadResult(SensorDriver* d, Reading* out, uint8_t maxOut) {
291   Bme280Driver* self = asBme280(d);
292   if (out == NULL) return 0;
293   const uint8_t n = maxOut < BME280_METRICS ? maxOut : BME280_METRICS;
294   for (uint8_t i = 0; i < n; ++i) out[i] = self->results_[i];
295   return n;
296 }
297
298 // A forced conversion finishes by itself and the chip returns to sleep.
299 static void bme280Cancel(SensorDriver* d) { asBme280(d)->busy_ = false; }
300
301 // The 3.3 V rail stays up in low power (§3); just require a fresh init().
302 static void bme280PreparePowerOff(SensorDriver* d) {
303   Bme280Driver* self = asBme280(d);
304   self->busy_ = false;
305   self->initialised_ = false;
306 }
307
308 static const SensorDriverOps kBme280Ops = {
309     .id = bme280Id,
310     .name = bme280Name,
311     .metricCount = bme280MetricCount,
312     .init = bme280Init,
313     .start = bme280Start,
314     .poll = bme280Poll,
315     .readResult = bme280ReadResult,
316     .cancel = bme280Cancel,
317     .preparePowerOff = bme280PreparePowerOff,
318 };
319
320 // Readings are reset once here so sequence/lastGoodMs survive re-inits.
321 void bme280DriverConstruct(Bme280Driver* self, const Bme280Config* config) {
322   self->base.ops = &kBme280Ops;
323   self->config_ = *config;
324   memset(&self->calib_, 0, sizeof(self->calib_));
325   self->tFine_ = 0;
326   self->startMs_ = 0;
327   self->initialised_ = false;
328   self->busy_ = false;
329   readingReset(&self->results_[0], SENSOR_BME280, METRIC_AIR_TEMPERATURE, UNIT_DEG_C);
330   readingReset(&self->results_[1], SENSOR_BME280, METRIC_AIR_HUMIDITY, UNIT_PERCENT_RH);
331   readingReset(&self->results_[2], SENSOR_BME280, METRIC_AIR_PRESSURE, UNIT_PASCAL);
332 }
```

Return to guided explanation, p. 181

ORIGINAL SOURCE / CHAPTER 6 / 1 OF 2

test/bringup/bme280_main.c

Original lines 1-27 | Code and comments unchanged; long lines wrap visually

chipName (L18-22): Translate an identity/state into a human-readable label for diagnostics.
setup (L24-52): Establish safe outputs, logger, gate, board check and queues; load configuration; start Safety, both acquisition workers, power, radios and supervision.

```
1  // Bring-up entry point for bme280 (WP1).  pio run -e test_bme280 -t upload -t monitor
2  // 3.3 V rail. Expect 3 readings every 250 ms, took <= 12 ms.
3  // Replaces src/main.c in this env only; links the production driver.
4  #include <Arduino.h>
5  #include <stdio.h>
6
7  #include "app_config.h"
8  #include "board_profile.h"
9  #include "bringup_harness.h"
10 #include "hal/i2c_bus.h"
11 #include "drivers/bme280.h"
12
13 static I2cBus s_i2c = I2C_BUS_INIT;
14 static Bme280Config s_config;  // address chosen at runtime in setup()
15 static Bme280Driver s_driver;
16 static BringupContext s_ctx = {&s_driver.base, CFG_BME280_PERIOD_MS, NULL};
17
18 static const char* chipName(uint8_t id) {
19   if (id == 0x60) return "BME280";
20   if (id == 0x56 || id == 0x57 || id == 0x58) return "BMP280 (no humidity!)";
21   return "unknown";
22 }
23
24 void setup(void) {
25   delay(200);
26   printf("\n=== bring-up: bme280 (SDA %d, SCL %d, %lu Hz) ===\n", BOARD_PIN_I2C_SDA,
27          BOARD_PIN_I2C_SCL, (unsigned long)BOARD_I2C_CLOCK_HZ);
```

Return to guided explanation, p. 174

ORIGINAL SOURCE / CHAPTER 6 / 2 OF 2

test/bringup/bme280_main.c

Original lines 28-54 | Code and comments unchanged; long lines wrap visually

setup (L24-52): Establish safe outputs, logger, gate, board check and queues; load configuration; start Safety, both acquisition workers, power, radios and supervision.
loop (L54-54): Park the Arduino loop task. The application work happens in explicitly created tasks.

```
28
29    const bool busOk = i2cBusBegin(&s_i2c, BOARD_PIN_I2C_SDA, BOARD_PIN_I2C_SCL,
30                                   BOARD_I2C_CLOCK_HZ);
31    printf("i2c begin -> %s\n", busOk ? "OK" : "FAILED");
32
33    // Pick whichever address ACKs (SDO strap); default if neither does.
34    uint8_t address = BOARD_BME280_I2C_ADDRESS;
35    const bool ack76 = i2cBusProbe(&s_i2c, BOARD_BME280_I2C_ADDRESS);
36    const bool ack77 = i2cBusProbe(&s_i2c, BOARD_BME280_I2C_ADDRESS_ALT);
37    printf("probe 0x%02X -> %s, 0x%02X -> %s\n", BOARD_BME280_I2C_ADDRESS,
38           ack76 ? "ACK" : "-", BOARD_BME280_I2C_ADDRESS_ALT, ack77 ? "ACK" : "-");
39    if (!ack76 && ack77) address = BOARD_BME280_I2C_ADDRESS_ALT;
40
41    uint8_t id = 0;
42    if (i2cBusReadRegs(&s_i2c, address, 0xD0, &id, 1)) {
43      printf("chip id @0x%02X = 0x%02X (%s)\n", address, id, chipName(id));
44    } else {
45      printf("chip id @0x%02X -> read failed (check wiring/pull-ups)\n", address);
46    }
47
48    s_config.bus = &s_i2c;
49    s_config.address = address;
50    bme280DriverConstruct(&s_driver, &s_config);
51    bringupStart(&s_ctx);
52  }
53
54  void loop(void) { vTaskDelay(portMAX_DELAY); }
```

Return to guided explanation, p. 174

PART II / SENSOR ARCHITECTURES

CHAPTER 07

DS18B20: Water Temperature

DS18B20 spends most of its measurement time converting inside the chip. The worker sends the command, serves other devices and returns after the conversion deadline. Short 1-Wire transactions and a checked scratchpad connect that long physical operation to the common driver interface.

In this chapter

1. Read reset, presence and bit slots on the 1-Wire bus.
2. Follow the 750 ms conversion without a 750 ms CPU wait.
3. Check CRC, decode signed counts and recognise the reset-value guard.

Chapter contents

7.1 DS18B20: send, leave, return 192
7.2 1-Wire: precise bits without occupying the whole conversion 193
7.3 DS18B20: a conversion that spans the frame 194

Guided source readings begin on page 196.

Complete original listings begin on page 195. Shared files are linked in the explanations; Appendix A contains the full filename index.

CHAPTER 7

7.1 DS18B20: send, leave, return

The most useful distinction is between the short bus transaction and the long temperature conversion. The CPU must send timed bits, but it does not need to remain busy while the chip measures.

Step	Meaning
Find the ROM	Identify one specific probe on the shared wire.
Configure 12-bit	Make its count scale and maximum conversion time predictable.
Send Convert T	Ask the probe to measure, then return to acquisition.
Return PENDING	Say when a later poll should occur; do not wait in a loop.
Read scratchpad	Retrieve temperature and integrity/configuration bytes.
Check and convert	Reject bad data, then divide the signed count by 16.

A number to follow

The two temperature bytes 0x91 and 0x01 form 0x0191, or 401 counts. Dividing by 16 gives 25.0625 degC. The CRC must pass before the driver trusts those bytes.

Why the bus code is included here

onewire_bus.c shows how a 1 or 0 is put onto the wire. ds18b20_codec.c checks and interprets bytes. ds18b20.c decides when to send commands and how to label a result. Together, these files explain the complete probe path.

The driver selects the saved ROM on later operations. It starts and later reads the same conversion within a frame; it does not implement the previous-conversion pipeline described in some architecture text. A current TDS window may still use the preceding frame's water temperature.

CHAPTER 7

7.2 1-Wire: precise bits without occupying the whole conversion

DS18B20 uses GPIO14 with an external 4.7 kOhm pull-up to retained 3.3V. It is externally powered, not in parasite-power mode. The HAL configures an open-drain input/output: it drives LOW or releases the line; the resistor produces HIGH.

`oneWireBusReset()` checks that the released line is not stuck LOW, holds it LOW for 480 us, releases it and samples for a presence pulse. Device presence is a protocol response, not a measured water temperature.

Write and read slots

`writeBit()` sends a 1 with a short LOW and a 0 with a longer LOW. `readBit()` briefly pulls LOW, releases and samples while the device can hold the line LOW. Interrupts are disabled only around the timing-sensitive portion of a slot, then re-enabled during the recovery time.

`oneWireBusWriteByte()` and ReadByte process eight bits least significant first. These microsecond operations are intentionally short busy waits. They are different from blocking the worker for the entire 750 ms temperature conversion.

Search and selection

`oneWireBusSearchFirst()` performs the ROM search, choosing the zero branch when devices differ. It returns one ROM and verifies its CRC. The DS18B20 driver then verifies family code 0x28. It does not keep enumerating until a DS18B20 is found if the first device belongs to another family.

`oneWireBusSelect()` sends Match ROM plus the saved eight-byte identity. Subsequent traffic addresses that selected device rather than accidentally reading a second probe. Finding the first probe is sufficient for this system; it is not a general multi-probe discovery service.

CRC and configuration helpers

`oneWireCrc8()` uses an initial CRC of zero and reflected polynomial 0x8C. `ds18b20RomValid()` checks family and ROM CRC. `ds18b20ScratchpadValid()` checks nine-byte scratchpad integrity and fixed configuration bits. `ds18b20Is12Bit()` checks the resolution bits.

Keeping these byte-level algorithms in domain code lets native tests feed known messages and corrupt bytes without hardware. Slot timing still needs target verification.

Original source in this PDF: src/hal/onewire_bus.c (p. 197); src/domain/ds18b20_codec.c (p. 201); src/domain/ds18b20_codec.h (p. 200)

CHAPTER 7

7.3 DS18B20: a conversion that spans the frame

`ds18b20Init()` searches and saves a valid ROM, reads the scratchpad and ensures 12-bit resolution. If needed it writes only the scratchpad configuration and preserves the alarm bytes. It does not issue Copy Scratchpad, avoiding an EEPROM write on each init.

`selectDevice()` performs reset plus Match ROM. `readScratchpad()` selects, sends 0xBE and reads nine bytes. These helpers concentrate the repeated bus sequence in one place.

The actual start/poll sequence

`ds18b20Start()` selects the device and sends Convert T, command 0x44. It records the provided start time and returns. `ds18b20Poll()` returns PENDING until that time + 750 ms. It then reads the scratchpad once and calls finish. The 750 ms maximum belongs to 12-bit conversion; the 1/16 degC count is resolution, not a promise of that measurement accuracy. [R6]

Start contains about 7 ms of 1-Wire traffic and the completed poll about 12 ms according to the driver's declared budget. The timestamp is passed before that traffic completes, so it is not an exact hardware Convert T edge. Target measurements should account for this detail when checking conversion margin.

finish() decides health

A bad scratchpad produces FAULT without trusting raw temperature. A valid scratchpad exposes the signed count. The driver rejects raw 0x0550, the +85 degC power-on value. This is an application-specific assumption that exact +85 degC water is not an accepted result; it is not a universal rule that +85 degC can never be a real DS18B20 measurement.

The result must also be within -55 to +125 degC. Accepted values become OK, update lastGoodMs and increment sequence. A missing presence pulse in start/read publishes FAULT and returns NOT_PRESENT, allowing service re-init.

`ds18b20Cancel()` drops software busy state. `prepareForPowerOff()` cancels without actively driving a retained open-drain bus. `ds18b20DriverGetRom()` copies the discovered identity for diagnostics. Re-init does not erase previously published sample history.

Original source in this PDF: src/drivers/ds18b20.c (p. 206); src/drivers/ds18b20.h (p. 202); src/domain/ds18b20_codec.c (p. 201)

ORIGINAL SOURCE / CHAPTER 7 / 1 OF 1

src/hal/onewire_bus.h

Original lines 1-37 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // 1-Wire bus adapter (HAL). Work package WP2 (with DS18B20).
2  // Externally powered DS18B20 on BOARD_PIN_ONE_WIRE (4.7 kOhm pull-up).
3  //
4  // Bit-banged, open-drain GPIO (the pin only pulls low; the pull-up drives
5  // high). Standard-speed timing from the DS18B20 datasheet (slot 70 us).
6  // Each time slot runs with interrupts off on this core for at most ~70 us;
7  // the reset's 480 us low phase runs with interrupts on.
8  // Worst-case blocking: reset ~1 ms, one byte ~0.6 ms, search ~15 ms.
9  // No parasite power: there is no strong pull-up.
10 // WP7: do not run a 1-Wire transaction on the core that services the HC-SR04
11 // echo ISR while an echo is in flight - a masked edge is timestamped late
12 // (up to ~70 us, ~1 cm).
13 #pragma once
14 #include <stdbool.h>
15 #include <stddef.h>
16 #include <stdint.h>
17
18 // Treat the members as private to onewire_bus.c. Initialise with ONE_WIRE_BUS_INIT.
19 typedef struct OneWireBus {
20   int8_t pin_;
21 } OneWireBus;
22
23 #define ONE_WIRE_BUS_INIT {-1}
24
25 // false if the pin is not a usable GPIO; the bus then stays unusable.
26 bool oneWireBusBegin(OneWireBus* bus, int8_t pin);
27 // true = presence pulse detected. false also if the bus does not idle high
28 // (shorted to GND or missing pull-up).
29 bool oneWireBusReset(OneWireBus* bus);
30 void oneWireBusWriteByte(OneWireBus* bus, uint8_t value);
31 uint8_t oneWireBusReadByte(OneWireBus* bus);
32 // Search ROM [F0h] for the first device (lowest branch at each discrepancy).
33 // Does its own reset. true if a code with a valid CRC was read into rom.
34 bool oneWireBusSearchFirst(OneWireBus* bus, uint8_t rom[8]);
35 // Match ROM [55h] + code. Call right after a successful reset.
36 void oneWireBusSelect(OneWireBus* bus, const uint8_t rom[8]);
37 uint8_t oneWireBusCrc8(const uint8_t* data, size_t len);
```

Return to guided explanation, p. 192

GUIDED SOURCE READING / CHAPTER 7

Mask interrupts only across the timing-sensitive slot

DS18B20 conversion can be deferred, but transmitting one 1-Wire bit requires microsecond timing. readBit makes the short occupied interval visible.

Original excerpt: src/hal/onewire_bus.c (p. 197) / lines 80-90. The complete unchanged file follows.

```
80  static bool readBit(OneWireBus* bus) {
81    portENTER_CRITICAL(&s_mux);
82    busLow(bus);
83    delayMicroseconds(kReadLowUs);
84    busRelease(bus);
85    delayMicroseconds(kReadSampleUs);
86    const bool bit = busLevel(bus);
87    portEXIT_CRITICAL(&s_mux);
88    delayMicroseconds(kReadRestUs);
89    return bit;
90  }
```

The critical section begins before pulling the line LOW. Releasing after the initial 3 us lets the device determine the bit level; the next 10 us places the sample 13 us into the slot. Sampling first or delaying release would read a different part of the electrical waveform.

The observed bool is copied into a local before the lock is released. The remaining 53 us recovery time occurs outside the critical section, so interrupt masking is restricted to the interval that protects the sample timing.

A mutex cannot provide this timing guarantee: another task could preempt the owner while it held a mutex. The short critical section prevents local interrupt-driven stretching of the bit slot. At the system level, acquisition separately avoids overlapping these slots with an ultrasonic echo.

Open-drain configuration means the master drives LOW or releases; the external pull-up supplies HIGH. busRelease therefore does not represent a push-pull HIGH driven against the sensor. Protocol timing and GPIO mode must be understood together.

ORIGINAL SOURCE / CHAPTER 7 / 1 OF 3

src/hal/onewire_bus.c

Original lines 1-42 | Code and comments unchanged; long lines wrap visually

busLow (L34-34): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.
busRelease (L35-35): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.

```
1  #include "hal/onewire_bus.h"
2
3  #include <Arduino.h>
4  #include <driver/gpio.h>
5  #include <string.h>
6
7  #include "domain/ds18b20_codec.h"
8
9  // ESP32-S3 GPIO numbers run 0-48.
10 static const int8_t kMaxGpio = 48;
11
12 static const uint8_t kCmdSearchRom = 0xF0;
13 static const uint8_t kCmdMatchRom = 0x55;
14
15 // Slot timing (us), DS18B20 datasheet AC characteristics: tSLOT 60-120,
16 // tLOW1 1-15, tLOW0 60-120, tRDV 15, tREC >= 1, tRSTL/tRSTH >= 480,
17 // presence low starts 15-60 us after release and lasts 60-240 us.
18 enum {
19   kResetLowUs = 480,
20   kPresenceSampleUs = 70,
21   kResetHighRestUs = 410,  // 70 + 410 = tRSTH
22   kWrite1LowUs = 6,
23   kWrite1RestUs = 64,
24   kWrite0LowUs = 60,
25   kWrite0RestUs = 10,
26   kReadLowUs = 3,
27   kReadSampleUs = 10,  // sampled 13 us into the slot, inside tRDV
28   kReadRestUs = 53
29 };
30
31 // Interrupts off for one slot only (bit timing must not be stretched).
32 static portMUX_TYPE s_mux = portMUX_INITIALIZER_UNLOCKED;
33
34 static inline void busLow(const OneWireBus* bus) { gpio_set_level((gpio_num_t)bus->pin_, 0);
   > }
35 static inline void busRelease(const OneWireBus* bus) { gpio_set_level((gpio_num_t)bus->pin_,
   > 1); }
36 static inline bool busLevel(const OneWireBus* bus) {
37   return gpio_get_level((gpio_num_t)bus->pin_) != 0;
38 }
39
40 bool oneWireBusBegin(OneWireBus* bus, int8_t pin) {
41   bus->pin_ = -1;
42   if (pin < 0 || pin > kMaxGpio) return false;
```

Return to guided explanation, p. 196

ORIGINAL SOURCE / CHAPTER 7 / 2 OF 3

src/hal/onewire_bus.c

Original lines 43-86 | Code and comments unchanged; long lines wrap visually

oneWireBusBegin (L40-54): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.
oneWireBusReset (L56-69): Create a reset pulse and sample for presence using a short timing-critical window.

```
  gpio_config_t io;
  memset(&io, 0, sizeof(io));
  io.pin_bit_mask = 1ULL << (uint8_t)pin;
  io.mode = GPIO_MODE_INPUT_OUTPUT_OD;  // drive low or release; read back the line
  io.pull_up_en = GPIO_PULLUP_DISABLE;  // external 4.7 kOhm to 3.3 V
  io.pull_down_en = GPIO_PULLDOWN_DISABLE;
  io.intr_type = GPIO_INTR_DISABLE;
  if (gpio_config(&io) != ESP_OK) return false;
  bus->pin_ = pin;
  busRelease(bus);
  return true;
}

bool oneWireBusReset(OneWireBus* bus) {
  if (bus->pin_ < 0) return false;
  busRelease(bus);
  if (!busLevel(bus)) return false;  // stuck low: short or no pull-up
  busLow(bus);
  delayMicroseconds(kResetLowUs);
  portENTER_CRITICAL(&s_mux);
  busRelease(bus);
  delayMicroseconds(kPresenceSampleUs);
  const bool presence = !busLevel(bus);
  portEXIT_CRITICAL(&s_mux);
  delayMicroseconds(kResetHighRestUs);
  return presence;
}

static void writeBit(OneWireBus* bus, bool bit) {
  portENTER_CRITICAL(&s_mux);
  busLow(bus);
  delayMicroseconds(bit ? kWrite1LowUs : kWrite0LowUs);
  busRelease(bus);
  portEXIT_CRITICAL(&s_mux);
  delayMicroseconds(bit ? kWrite1RestUs : kWrite0RestUs);
}

static bool readBit(OneWireBus* bus) {
  portENTER_CRITICAL(&s_mux);
  busLow(bus);
  delayMicroseconds(kReadLowUs);
  busRelease(bus);
  delayMicroseconds(kReadSampleUs);
  const bool bit = busLevel(bus);
```

Return to guided explanation, p. 196

ORIGINAL SOURCE / CHAPTER 7 / 3 OF 3

src/hal/onewire_bus.c

Original lines 87-130 | Code and comments unchanged; long lines wrap visually

readBit (L80-90): Start a read slot, release the wire and sample while the device may pull it LOW. **oneWireBusWriteByte (L92-95)**: Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.

```
87    portEXIT_CRITICAL(&s_mux);
88    delayMicroseconds(kReadRestUs);
89    return bit;
90  }
91
92  void oneWireBusWriteByte(OneWireBus* bus, uint8_t value) {
93    if (bus->pin_ < 0) return;
94    for (uint8_t i = 0; i < 8; ++i) writeBit(bus, (value >> i) & 1u);  // LSB first
95  }
96
97  uint8_t oneWireBusReadByte(OneWireBus* bus) {
98    if (bus->pin_ < 0) return 0xFF;
99    uint8_t value = 0;
100   for (uint8_t i = 0; i < 8; ++i) {
101     if (readBit(bus)) value |= (uint8_t)(1u << i);
102   }
103   return value;
104 }
105
106 bool oneWireBusSearchFirst(OneWireBus* bus, uint8_t rom[8]) {
107   if (!oneWireBusReset(bus)) return false;
108   oneWireBusWriteByte(bus, kCmdSearchRom);
109   uint8_t id[8] = {0};
110   for (uint8_t i = 0; i < 64; ++i) {
111     const bool bit = readBit(bus);
112     const bool complement = readBit(bus);
113     if (bit && complement) return false;  // no device took part
114     // Different: every device has `bit` here. Equal (both 0): devices differ,
115     // take the 0 branch, which is the first device in search order.
116     const bool direction = bit != complement ? bit : false;
117     if (direction) id[i / 8] |= (uint8_t)(1u << (i % 8));
118     writeBit(bus, direction);
119   }
120   if (oneWireCrc8(id, 7) != id[7]) return false;
121   memcpy(rom, id, 8);
122   return true;
123 }
124
125 void oneWireBusSelect(OneWireBus* bus, const uint8_t rom[8]) {
126   oneWireBusWriteByte(bus, kCmdMatchRom);
127   for (uint8_t i = 0; i < 8; ++i) oneWireBusWriteByte(bus, rom[i]);
128 }
129
130 uint8_t oneWireBusCrc8(const uint8_t* data, size_t len) { return oneWireCrc8(data, len); }
```

Return to guided explanation, p. 196

ORIGINAL SOURCE / CHAPTER 7 / 1 OF 1

src/domain/ds18b20_codec.h

Original lines 1-35 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // DS18B20 scratchpad decoding and 1-Wire CRC (WP2). Pure C, host-tested.
2  // Datasheet: Maxim DS18B20 (local copy in Electrical System Study/Datasheets/
3  // DS18B20.pdf): Figure 9 (memory map), Figure 10 (configuration register),
4  // Table 1 (temperature format), "CRC Generation".
5  #pragma once
6  #include <stdbool.h>
7  #include <stddef.h>
8  #include <stdint.h>
9
10 #define DS18B20_FAMILY_CODE 0x28
11 #define DS18B20_SCRATCHPAD_LEN 9
12 #define DS18B20_POWER_ON_RAW ((int16_t)0x0550)  // +85 degC, reset value of the register
13 #define DS18B20_CONFIG_12BIT 0x7F               // R1 = R0 = 1, reserved bits as read
14 #define DS18B20_MIN_C (-55.0f)
15 #define DS18B20_MAX_C 125.0f
16
17 // Dallas/Maxim CRC-8 (x^8 + x^5 + x^4 + 1), LSB first, init 0. Running it over
18 // data plus its CRC byte gives 0.
19 uint8_t oneWireCrc8(const uint8_t* data, size_t len);
20
21 // ROM code: CRC over the first 7 bytes matches byte 7 and the family is 28h.
22 // Rejects the all-zero code a bus stuck low produces.
23 bool ds18b20RomValid(const uint8_t rom[8]);
24
25 // Scratchpad CRC matches AND the configuration register has its fixed bits
26 // (bit 7 = 0, bits 0-4 = 1). The second check rejects an all-zero read (bus
27 // stuck low), whose CRC is also 0.
28 bool ds18b20ScratchpadValid(const uint8_t scratchpad[DS18B20_SCRATCHPAD_LEN]);
29
30 // True if the configuration register selects 12-bit resolution.
31 bool ds18b20Is12Bit(uint8_t config);
32
33 // Signed temperature register (bytes 0-1), 1/16 degC per LSB at 12 bits.
34 int16_t ds18b20RawTemperature(const uint8_t scratchpad[DS18B20_SCRATCHPAD_LEN]);
35 float ds18b20RawToDegC(int16_t raw);
```

Return to guided explanation, p. 192

ORIGINAL SOURCE / CHAPTER 7 / 1 OF 1

src/domain/ds18b20_codec.c

Original lines 1-32 | Code and comments unchanged; long lines wrap visually

oneWireCrc8 (L3-15): Recompute the 1-Wire CRC from the received bytes using the reflected polynomial.
ds18b20RomValid (L17-19): Require the DS18B20 family code and a correct ROM CRC.

```
1  #include "domain/ds18b20_codec.h"
2
3  uint8_t oneWireCrc8(const uint8_t* data, size_t len) {
4    uint8_t crc = 0;
5    for (size_t i = 0; i < len; ++i) {
6      uint8_t in = data[i];
7      for (uint8_t b = 0; b < 8; ++b) {
8        const uint8_t mix = (crc ^ in) & 0x01;
9        crc >>= 1;
10       if (mix) crc ^= 0x8C;  // reflected 0x31
11       in >>= 1;
12     }
13   }
14   return crc;
15 }
16
17 bool ds18b20RomValid(const uint8_t rom[8]) {
18   return rom[0] == DS18B20_FAMILY_CODE && oneWireCrc8(rom, 7) == rom[7];
19 }
20
21 bool ds18b20ScratchpadValid(const uint8_t scratchpad[DS18B20_SCRATCHPAD_LEN]) {
22   if (oneWireCrc8(scratchpad, 8) != scratchpad[8]) return false;
23   return (scratchpad[4] & 0x9F) == 0x1F;
24 }
25
26 bool ds18b20Is12Bit(uint8_t config) { return (config & 0x60) == 0x60; }
27
28 int16_t ds18b20RawTemperature(const uint8_t scratchpad[DS18B20_SCRATCHPAD_LEN]) {
29   return (int16_t)(uint16_t)((uint16_t)scratchpad[1] << 8 | scratchpad[0]);
30 }
31
32 float ds18b20RawToDegC(int16_t raw) { return (float)raw / 16.0f; }
```

Return to guided explanation, p. 192

ORIGINAL SOURCE / CHAPTER 7 / 1 OF 2

src/drivers/ds18b20.h

Original lines 1-31 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // ds18b20 driver. Work package WP2 (Stage 2). Architecture §7, §14.
2  // Datasheet: Maxim DS18B20 (local copy in Electrical System Study/Datasheets/
3  // DS18B20.pdf). Domain math (CRC, decoding) in domain/ds18b20_codec.h.
4  //
5  // Retained 3.3 V supply (external power, no parasite mode), 1-Wire on GPIO14
6  // with a 4.7 kOhm pull-up. Produces WaterTemperature in degC.
7  //
8  // init() finds the first device with Search ROM, checks the ROM CRC and the
9  // family code (28h) and keeps that ROM: every later transaction uses Match
10 // ROM, so a second device on the bus is never read by mistake. It then reads
11 // the scratchpad and, if the EEPROM selected a lower resolution, writes 12-bit
12 // to the scratchpad only (no Copy Scratchpad: no EEPROM wear, re-applied on
13 // every init).
14 // start() sends Convert T and returns; poll() is Pending until
15 // CFG_DS18B20_CONVERSION_MAX_MS (750 ms, 12-bit tCONV max) has passed, then
16 // reads the 9-byte scratchpad once.
17 //
18 // Results, highest priority first (poll() returns READY for all of them):
19 //   scratchpad CRC or fixed config bits wrong -> HEALTH_FAULT, no raw
20 //   temperature register = 0550h (+85 degC power-on value) -> HEALTH_FAULT;
21 //                     the device reset without converting, so the next start()
22 //                     fails and init() re-applies the resolution
23 //   outside -55..+125 degC -> HEALTH_FAULT
24 //   otherwise         -> value in degC, HEALTH_OK
25 // raw = temperature register (1/16 degC) whenever the CRC is good.
26 // No presence pulse in start()/poll() -> DRIVER_NOT_PRESENT (re-init needed).
27 // Before the first result readResult() gives HEALTH_STARTING, never 0 degC.
28 //
29 // Worst-case blocking (1-Wire HAL; slots run with interrupts off <= 70 us each):
30 //   init()    ~50 ms  (search ~15 ms, scratchpad read ~12 ms, and when the
31 //                      resolution must be set: write ~9 ms + re-read ~12 ms)
```

Return to guided explanation, p. 192

ORIGINAL SOURCE / CHAPTER 7 / 2 OF 2

src/drivers/ds18b20.h

Original lines 32-62 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
32  //   start()   ~7 ms   (reset, Match ROM, Convert T)
33  //   poll()    ~12 ms  once the conversion time is over (reset, Match ROM,
34  //                      Read Scratchpad, 9 bytes); Pending polls: none
35  //   others    none
36  //
37  // Implements the SensorDriver contract in drivers/sensor_driver.h.
38  #pragma once
39  #include "drivers/sensor_driver.h"
40  #include "hal/onewire_bus.h"
41
42  typedef struct Ds18b20Config {
43    OneWireBus* bus;  // already begun on BOARD_PIN_ONE_WIRE
44  } Ds18b20Config;
45
46  // Members after `base` are private to ds18b20.c.
47  typedef struct Ds18b20Driver {
48    SensorDriver base;  // must stay first
49    Ds18b20Config config_;
50    bool initialised_;
51    bool busy_;
52    bool hasRom_;
53    uint8_t rom_[8];
54    MonoMs startMs_;
55    Reading result_;
56  } Ds18b20Driver;
57
58  // Constructor. Use &driver.base wherever a SensorDriver* is expected.
59  void ds18b20DriverConstruct(Ds18b20Driver* self, const Ds18b20Config* config);
60
61  // ROM code selected by the last successful init(). false before that.
62  bool ds18b20DriverGetRom(const Ds18b20Driver* self, uint8_t rom[8]);
```

Return to guided explanation, p. 192

GUIDED SOURCE READING / CHAPTER 7

A long conversion needs a deadline, not a long task sleep

DS18B20 start selects the saved ROM, sends Convert T and records startMs. The chip then works independently. Its poll function shows how the worker returns without waiting in place.

Original excerpt: src/drivers/ds18b20.c (p. 206) / lines 99-113. The complete unchanged file follows.

```
 99    const MonoMs due = self->startMs_ + CFG_DS18B20_CONVERSION_MAX_MS;
100    if (now < due) {
101      r.status = DRIVER_PENDING;
102      r.nextPollMs = due;
103      return r;
104    }
105    self->busy_ = false;
106    uint8_t sp[DS18B20_SCRATCHPAD_LEN];
107    if (!readScratchpad(self, sp)) {
108      self->initialised_ = false;
109      markNotPresent(self, self->startMs_, now);
110      r.status = DRIVER_NOT_PRESENT;
111      return r;
112    }
113    finish(self, sp, now);
```

The deadline is derived from the actual start and the 750 ms 12-bit conversion allowance. Before due, the function returns PENDING with that deadline. There is no bus transaction in this branch, so acquisition can use the 1-Wire-free interval for other sensors.

After due, busy is cleared before reading the scratchpad. This poll is completing the operation rather than leaving it active for an unintended second finish. A missing presence response invalidates initialization and stamps a new missing-device result instead of reusing the last temperature as current.

readScratchpad returns bytes, while finish decides their validity. The final READY says that a verdict exists, including a possible CRC or range failure. It does not bypass those checks.

The design waits within the same measurement frame for this conversion. TDS commonly completes earlier and uses a recent previous water-temperature observation. That is why the dependency copy and freshness checks appear in TDS rather than assuming both current-frame readings become available together.

GUIDED SOURCE READING / CHAPTER 7

Integrity checks come before signed temperature conversion

The nine scratchpad bytes have arrived. finish must determine whether they can represent a trustworthy conversion before interpreting two of them as temperature.

Original excerpt: src/drivers/ds18b20.c (p. 206) / lines 126-142. The complete unchanged file follows.

```
126    if (!ds18b20ScratchpadValid(sp)) {
127      r->health = HEALTH_FAULT;  // corrupted read; the content is not trusted
128      return;
129    }
130    const int16_t raw = ds18b20RawTemperature(sp);
131    r->hasRaw = true;
132    r->raw = raw;
133    if (raw == DS18B20_POWER_ON_RAW) {
134      // The register still holds its reset value: the device restarted and did
135      // not convert. Water never reads exactly +85 degC here.
136      r->health = HEALTH_FAULT;
137      self->initialised_ = false;  // re-apply the resolution
138      return;
139    }
140    const float degC = ds18b20RawToDegC(raw);
141    if (degC < DS18B20_MIN_C || degC > DS18B20_MAX_C) {
142      r->health = HEALTH_FAULT;
```

The CRC/configuration guard occurs first. If it fails, hasRaw remains false and the function returns FAULT. Converting an unchecked byte pair could turn a corrupted transfer into a plausible temperature.

The raw helper combines the low/high bytes into a signed 16-bit count. raw is then preserved for diagnostics. The explicit +85 degC power-on pattern is rejected by this project because it can indicate a reset without a completed conversion. The driver also requests reinitialization so resolution is reapplied.

Only afterwards does division by 16 produce Celsius and the operating-range test run. An accepted branch updates value, health, lastGoodMs and sequence. Every failure returns before those success assignments, so an old numeric value cannot acquire a new OK verdict merely because its storage still contains a float.

The final getter copies the saved ROM only when hasRom is true. That flag has the same purpose as hasRaw: zero-initialized storage is not evidence that discovery or measurement happened. Initialization establishes identity; each conversion independently establishes a new observation.

ORIGINAL SOURCE / CHAPTER 7 / 1 OF 4

src/drivers/ds18b20.c

Original lines 1-47 | Code and comments unchanged; long lines wrap visually

asDs18b20 (L12-12): Cast the common first-member SensorDriver pointer back to this concrete driver type.
markNotPresent (L18-25): Stamp a new fault outcome for the missing probe instead of leaving an earlier OK verdict current.

```
#include "drivers/ds18b20.h"

#include <string.h>

#include "app_config.h"
#include "domain/ds18b20_codec.h"

static const uint8_t kCmdConvertT = 0x44;
static const uint8_t kCmdReadScratchpad = 0xBE;
static const uint8_t kCmdWriteScratchpad = 0x4E;

static inline Ds18b20Driver* asDs18b20(SensorDriver* d) { return (Ds18b20Driver*)d; }

static void finish(Ds18b20Driver* self, const uint8_t sp[DS18B20_SCRATCHPAD_LEN], MonoMs
now);

// No presence pulse: publish a FAULT verdict, as the other drivers do, so the
// last OK reading is not served as current. lastGoodMs keeps its value.
static void markNotPresent(Ds18b20Driver* self, MonoMs start, MonoMs now) {
  Reading* r = &self->result_;
  r->intervalStartMs = start;
  r->intervalEndMs = now;
  r->qualityFlags = kQualityNone;
  r->hasRaw = false;
  r->health = HEALTH_FAULT;
}

static SensorId ds18b20Id(const SensorDriver* d) { (void)d; return SENSOR_DS18B20; }
static const char* ds18b20Name(const SensorDriver* d) { (void)d; return "ds18b20"; }
static uint8_t ds18b20MetricCount(const SensorDriver* d) { (void)d; return 1; }

// Reset + Match ROM. false if no presence pulse.
static bool selectDevice(Ds18b20Driver* self) {
  OneWireBus* bus = self->config_.bus;
  if (!oneWireBusReset(bus)) return false;
  oneWireBusSelect(bus, self->rom_);
  return true;
}

// Read all 9 scratchpad bytes. false if no presence pulse; the content is
// checked by the caller.
static bool readScratchpad(Ds18b20Driver* self, uint8_t sp[DS18B20_SCRATCHPAD_LEN]) {
  if (!selectDevice(self)) return false;
  oneWireBusWriteByte(self->config_.bus, kCmdReadScratchpad);
  for (uint8_t i = 0; i < DS18B20_SCRATCHPAD_LEN; ++i) {
    sp[i] = oneWireBusReadByte(self->config_.bus);
  }
  return true;
```

Return to guided explanation, p. 204

ORIGINAL SOURCE / CHAPTER 7 / 2 OF 4

src/drivers/ds18b20.c

Original lines 48-95 | Code and comments unchanged; long lines wrap visually

readScratchpad (L41-48): Select the probe and read its nine-byte scratchpad; the caller checks integrity before interpretation.
ds18b20Init (L50-77): Check dependencies, configure this module and return whether initialisation succeeded.

```
48  }
49
50  static DriverStatus ds18b20Init(SensorDriver* d) {
51    Ds18b20Driver* self = asDs18b20(d);
52    self->initialised_ = false;
53    self->busy_ = false;
54    if (self->config_.bus == 0) return DRIVER_FAULT;
55
56    uint8_t rom[8];
57    if (!oneWireBusSearchFirst(self->config_.bus, rom)) return DRIVER_NOT_PRESENT;
58    if (!ds18b20RomValid(rom)) return DRIVER_NOT_PRESENT;  // not a DS18B20
59    memcpy(self->rom_, rom, sizeof(rom));
60    self->hasRom_ = true;
61
62    uint8_t sp[DS18B20_SCRATCHPAD_LEN];
63    if (!readScratchpad(self, sp)) return DRIVER_NOT_PRESENT;
64    if (!ds18b20ScratchpadValid(sp)) return DRIVER_FAULT;
65    if (!ds18b20Is12Bit(sp[4])) {
66      // TH and TL are written back unchanged; the scratchpad only, not EEPROM.
67      if (!selectDevice(self)) return DRIVER_NOT_PRESENT;
68      oneWireBusWriteByte(self->config_.bus, kCmdWriteScratchpad);
69      oneWireBusWriteByte(self->config_.bus, sp[2]);
70      oneWireBusWriteByte(self->config_.bus, sp[3]);
71      oneWireBusWriteByte(self->config_.bus, DS18B20_CONFIG_12BIT);
72      if (!readScratchpad(self, sp)) return DRIVER_NOT_PRESENT;
73      if (!ds18b20ScratchpadValid(sp) || !ds18b20Is12Bit(sp[4])) return DRIVER_FAULT;
74    }
75    self->initialised_ = true;
76    return DRIVER_OK;
77  }
78
79  static DriverStatus ds18b20Start(SensorDriver* d, MonoMs now) {
80    Ds18b20Driver* self = asDs18b20(d);
81    if (!self->initialised_) return DRIVER_FAULT;
82    if (!selectDevice(self)) {
83      self->initialised_ = false;
84      markNotPresent(self, now, now);
85      return DRIVER_NOT_PRESENT;
86    }
87    oneWireBusWriteByte(self->config_.bus, kCmdConvertT);
88    self->startMs_ = now;
89    self->busy_ = true;
90    return DRIVER_OK;
91  }
92
93  static PollResult ds18b20Poll(SensorDriver* d, MonoMs now) {
94    Ds18b20Driver* self = asDs18b20(d);
95    PollResult r = {DRIVER_FAULT, now};
```

Return to guided explanation, p. 204

ORIGINAL SOURCE / CHAPTER 7 / 3 OF 4

src/drivers/ds18b20.c

Original lines 96-142 | Code and comments unchanged; long lines wrap visually

ds18b20Poll (L93-116): Advance an in-flight operation only when useful; return PENDING with a future time, READY, or failure.
finish (L118-149): Follow ROM selection, Convert T, the later scratchpad read, CRC and signed-count conversion.

```
  if (!self->busy_) return r;

  // Fixed worst-case conversion time: no bus traffic while converting.
  const MonoMs due = self->startMs_ + CFG_DS18B20_CONVERSION_MAX_MS;
  if (now < due) {
    r.status = DRIVER_PENDING;
    r.nextPollMs = due;
    return r;
  }
  self->busy_ = false;
  uint8_t sp[DS18B20_SCRATCHPAD_LEN];
  if (!readScratchpad(self, sp)) {
    self->initialised_ = false;
    markNotPresent(self, self->startMs_, now);
    r.status = DRIVER_NOT_PRESENT;
    return r;
  }
  finish(self, sp, now);
  r.status = DRIVER_READY;
  return r;
}

static void finish(Ds18b20Driver* self, const uint8_t sp[DS18B20_SCRATCHPAD_LEN], MonoMs now)
{
  Reading* r = &self->result_;
  r->intervalStartMs = self->startMs_;
  r->intervalEndMs = now;
  r->qualityFlags = kQualityNone;
  r->hasRaw = false;

  // READY in every case below: health carries the verdict.
  if (!ds18b20ScratchpadValid(sp)) {
    r->health = HEALTH_FAULT;  // corrupted read; the content is not trusted
    return;
  }
  const int16_t raw = ds18b20RawTemperature(sp);
  r->hasRaw = true;
  r->raw = raw;
  if (raw == DS18B20_POWER_ON_RAW) {
    // The register still holds its reset value: the device restarted and did
    // not convert. Water never reads exactly +85 degC here.
    r->health = HEALTH_FAULT;
    self->initialised_ = false;  // re-apply the resolution
    return;
  }
  const float degC = ds18b20RawToDegC(raw);
  if (degC < DS18B20_MIN_C || degC > DS18B20_MAX_C) {
    r->health = HEALTH_FAULT;
```

Return to guided explanation, p. 204

ORIGINAL SOURCE / CHAPTER 7 / 4 OF 4

src/drivers/ds18b20.c

Original lines 143-191 | Code and comments unchanged; long lines wrap visually

finish (L118-149): Follow ROM selection, Convert T, the later scratchpad read, CRC and signed-count conversion. **ds18b20ReadResult (L151-155)**: Copy the latest Reading record(s) into caller storage. Inspect health even when poll returned READY.

```
    return;
  }
  r->value = degC;
  r->health = HEALTH_OK;
  r->lastGoodMs = now;
  ++r->sequence;
}

static uint8_t ds18b20ReadResult(SensorDriver* d, Reading* out, uint8_t maxOut) {
  if (maxOut < 1) return 0;
  out[0] = asDs18b20(d)->result_;
  return 1;
}

// A conversion in progress finishes on its own; its result is never read.
static void ds18b20Cancel(SensorDriver* d) { asDs18b20(d)->busy_ = false; }

// Retained rail, and the open-drain pin only ever releases the line between
// slots: nothing to drive. Drop any pending conversion.
static void ds18b20PreparePowerOff(SensorDriver* d) { ds18b20Cancel(d); }

static const SensorDriverOps kDs18b20Ops = {
    .id = ds18b20Id,
    .name = ds18b20Name,
    .metricCount = ds18b20MetricCount,
    .init = ds18b20Init,
    .start = ds18b20Start,
    .poll = ds18b20Poll,
    .readResult = ds18b20ReadResult,
    .cancel = ds18b20Cancel,
    .preparePowerOff = ds18b20PreparePowerOff,
};

void ds18b20DriverConstruct(Ds18b20Driver* self, const Ds18b20Config* config) {
  self->base.ops = &kDs18b20Ops;
  self->config_ = *config;
  self->initialised_ = false;
  self->busy_ = false;
  self->hasRom_ = false;
  memset(self->rom_, 0, sizeof(self->rom_));
  self->startMs_ = 0;
  readingReset(&self->result_, SENSOR_DS18B20, METRIC_WATER_TEMPERATURE, UNIT_DEG_C);
}

bool ds18b20DriverGetRom(const Ds18b20Driver* self, uint8_t rom[8]) {
  if (!self->hasRom_) return false;
  memcpy(rom, self->rom_, sizeof(self->rom_));
  return true;
}
```

Return to guided explanation, p. 205

ORIGINAL SOURCE / CHAPTER 7 / 1 OF 1

test/bringup/ds18b20_main.c

Original lines 1-49 | Code and comments unchanged; long lines wrap visually

printRomIfNew (L24-35): Present already-collected diagnostic information; this does not make the measurement more valid.
setup (L39-47): Establish safe outputs, logger, gate, board check and queues; load configuration; start Safety, both acquisition workers, power, radios and supervision.

```
1  // Bring-up entry point for ds18b20 (WP2).  pio run -e test_ds18b20 -t upload -t monitor
2  // Retained 3.3 V supply, DQ on GPIO14 with a 4.7 kOhm pull-up to 3.3 V.
3  // 12-bit conversion (750 ms max) runs inside the 1 s period.
4  // Replaces src/main.c in this env only; links the production driver.
5  //
6  // Prints the selected ROM code once, then one temperature per second.
7  // "init -> NOT_PRESENT": no presence pulse (DQ open, no supply, missing
8  // pull-up) or the first device is not a DS18B20 (family 28h).
9  #include <Arduino.h>
10 #include <stdio.h>
11 #include <string.h>
12
13 #include "app_config.h"
14 #include "board_profile.h"
15 #include "bringup_harness.h"
16 #include "hal/onewire_bus.h"
17 #include "drivers/ds18b20.h"
18
19 static OneWireBus s_bus = ONE_WIRE_BUS_INIT;
20 static Ds18b20Config s_config = {&s_bus};
21 static Ds18b20Driver s_driver;
22
23 // Print the ROM code whenever init() selected a different one.
24 static void printRomIfNew(SensorDriver* driver) {
25   (void)driver;
26   static uint8_t last[8];
27   static bool printed = false;
28   uint8_t rom[8];
29   if (!ds18b20DriverGetRom(&s_driver, rom)) return;
30   if (printed && memcmp(rom, last, sizeof(rom)) == 0) return;
31   memcpy(last, rom, sizeof(rom));
32   printed = true;
33   printf("# ds18b20 ROM %02X-%02X%02X%02X%02X%02X%02X crc %02X\n", rom[0], rom[6], rom[5],
34          rom[4], rom[3], rom[2], rom[1], rom[7]);
35 }
36
37 static BringupContext s_ctx = {&s_driver.base, CFG_FRAME_PERIOD_MS, printRomIfNew};
38
39 void setup(void) {
40   ds18b20DriverConstruct(&s_driver, &s_config);
41   delay(200);
42   printf("\n=== bring-up: %s ===\n", sensorDriverName(&s_driver.base));
43   if (!oneWireBusBegin(&s_bus, BOARD_PIN_ONE_WIRE)) {
44     printf("GPIO%d cannot be used for 1-Wire\n", (int)BOARD_PIN_ONE_WIRE);
45   }
46   bringupStart(&s_ctx);
47 }
48
49 void loop(void) { vTaskDelay(portMAX_DELAY); }
```

Return to guided explanation, p. 192

PART II / SENSOR ARCHITECTURES

CHAPTER 08

pH-4502C: pH Measurement

The pH probe reaches the ADC through an amplifier module and a voltage divider. The driver first reconstructs module voltage, then applies a fitted line. That order separates an electrical measurement from the chemical quantity inferred from it.

In this chapter

1. Follow the ten-tap analog window and divider correction.
2. Derive the two-point pH line from buffer measurements.
3. Read the branches for unpowered, saturated and unverified results.

Chapter contents

8.1 pH: first voltage, then a fitted line 212
8.2 pH: recover voltage, then apply a calibration 213
8.3 pH: the result decision order 214

Guided source readings begin on page 217.

Complete original listings begin on page 215. Shared files are linked in the explanations; Appendix A contains the full filename index.

CHAPTER 8

8.1 pH: first voltage, then a fitted line

There are three separate operations: measure the ADC pin, undo the resistor divider, and convert module voltage into pH using a calibration line. Keeping them separate makes mistakes easier to locate.

Stage	Example or rule
ADC pin	A filtered voltage, stored as raw millivolts.
Divider recovery	Multiply by (10000 + 15000) / 15000.
Line fit	pH = slope * module_mV + offset.
Health	Decide whether the fitted result is trustworthy enough to use.

Follow one provisional value

1654.8 mV at the pin becomes 2758 mV at the module. The configured provisional fit maps that to pH 7. The result still says UNCALIBRATED because the line has not been checked with buffer solutions. A neat pH number is not evidence of an accurate chemical measurement.

The code collects ten taps 30 ms apart, for a nominal 270 ms window. It rejects saturation and non-finite values. A verified fit outside 0-14 is FAULT; a provisional fit stays UNCALIBRATED so raw capture remains possible. Below 100 mV at the pin, it reports an unpowered/no-signal heuristic. That is an inference from voltage, not a direct measurement of the module's supply.

Two configuration flags

calibration.valid means the code has a usable mathematical line. calibrationProvisional means that line is not yet reference-verified. A valid but provisional line produces pH units with UNCALIBRATED health.

The full header, driver and individual calibration test are in this chapter. Divider and line-fitting helpers are included in the shared-interface chapter. Test calibration constants are separate from production constants.

CHAPTER 8

8.2 pH: recover voltage, then apply a calibration

The pH-4502C module is powered from external 5V. Its PO output reaches GPIO7 through a 10 kOhm top / 15 kOhm bottom divider. DO and TO are unused. The software recovers module voltage before applying a line fit.

```
// Source: src/domain/calibration.c
return adcMv * (float)(rTopOhm + rBottomOhm)
             / (float)rBottomOhm;
```

The nominal recovery factor is 25/15 = 1.6667. Example: 1654.8 mV at the ADC corresponds to 2758 mV at PO. Dividing by the wrong ratio would shift every pH estimate even if the ADC itself were calibrated.

Provisional production fit

Production uses slope -0.0057 pH/mV and sets the offset so 2758 mV maps to pH 7. Thus offset is `7 - (-0.0057 * 2758) = 22.7206`. At 3000 mV the nominal line gives pH 5.6206. These are calculations from configured constants, not independent chemical measurements.

The calibration record is mathematically valid, but `calibrationProvisional` is true. Therefore results in pH units still carry UNCALIBRATED. This distinction prevents a usable formula from being mistaken for a buffer-verified measurement.

Two-point fit

`linearFromTwoPoints()` computes slope `(y2 - y1)/(x2 - x1)` and offset `y1 - slope*x1`, stores a revision and marks valid. It refuses almost identical input voltages because the fit would divide by a near-zero difference.

For example, 2500 mV at pH 6.86 and 3000 mV at pH 4.01 give slope -0.0057 and offset 21.11. Those illustrative points produce a different offset from the provisional production fit. Changing test_ph constants does not change saved production calibration. Production accepts validated buffer fits through Configuration and NVS.

No water-temperature compensation is applied to pH in this implementation. A working linear fit should not be interpreted as complete compensation over every electrode condition or temperature.

Original source in this PDF: src/drivers/ph.c (p. 219); src/domain/calibration.c (p. 158); src/services/acquisition.c (p. 96); test/bringup/ph_main.c (p. 223)

CHAPTER 8

8.3 pH: the result decision order

The pH driver filters ten pin-voltage taps spaced 30 ms apart, recovers module voltage through the divider and then interprets that voltage. Its result verdict follows a deliberate order.

Condition	Outcome
Pin voltage below 100 mV	UNPOWERED heuristic.
Saturated ADC tap	FAULT with saturated quality.
No valid calibration line	Module mV, UNCALIBRATED.
Non-finite fitted pH	FAULT.
Verified fit outside pH 0-14	FAULT.
Provisional nominal fit	pH, UNCALIBRATED, even outside 0-14.
Verified fit in range	pH, OK.

Why provisional out-of-range is retained

The nominal line may not match the particular probe. Rejecting every out-of-range nominal estimate would prevent a calibration capture from accessing an otherwise usable raw voltage. UNCALIBRATED is not generically actionable, so the provisional estimate can remain diagnostic while a measured buffer fit replaces it.

raw always means filtered pin mV. The module voltage used by the fit is recovered with the divider ratio. A stored calibration setter copies a new line and provisional flag into the driver; acquisition calls it at a frame boundary.

Bring-up versus saved calibration

test_ph prints an additional module-mV line whenever raw data exists. This avoids confusion between ADC-pin mV, module mV and the displayed nominal pH. Its local buffer constants still belong to that test image. Production calibration instead stores a ConfigBlob revision in NVS and waits for acquisition to apply it.

The low-voltage UNPOWERED heuristic does not replace rail checking. Production acquisition also checks 5V validity before start and at completion; consumers check their Snapshot copy again against current rail history.

Original source in this PDF: src/drivers/ph.c (p. 219); test/bringup/ph_main.c (p. 223); src/services/acquisition.c (p. 96); src/domain/cal_capture.c (p. 366)

ORIGINAL SOURCE / CHAPTER 8 / 1 OF 2

src/drivers/ph.h

Original lines 1-39 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // ph4502c driver. Work package WP5 (Stage 3). Architecture §3, §7, §14.
2  // Source: PH-Sensor-4502C.pdf (a tutorial, not a manufacturer specification).
3  //
4  // Switched 5 V rail. PO -> 10k/15k divider -> ADC1 pin (GPIO7). DO/TO unused.
5  // Pin mV -> divider recovery -> module mV -> two-point buffer fit -> pH.
6  // Module output is 0-5 V, so at most 3 V at the pin (inside the S3's ~3.1 V
7  // range at 11 dB). With the offset pot set so a shorted BNC reads 2.5 V at PO
8  // (pH 7), the tutorial's nominal slope is about -5.7 pH/V.
9  //
10 // Filter (tutorial sketch): PH_SAMPLES conversions spaced PH_SAMPLE_SPACING_MS
11 // apart, sorted, mean of the middle PH_SAMPLES - 2*PH_TRIM. The window is spread
12 // over successive poll() calls, never taken as a burst (§9): each tap is due
13 // PH_SAMPLE_SPACING_MS after the previous actual sample, so a late poll delays
14 // the rest of the window instead of compressing it.
15 // No temperature compensation: the tutorial reports < 0.01 pH below 30 degC.
16 //
17 // Results:
18 //   calibration.valid  -> value in pH, HEALTH_OK (FAULT outside 0-14)
19 //     + calibrationProvisional -> value in pH, HEALTH_UNCALIBRATED (a nominal
20 //                         fit, e.g. shorted-BNC offset + tutorial slope)
21 //                         (also outside 0-14: the nominal fit may not match the
22 //                         probe yet; capturable for calibration, never actionable)
23 //   otherwise          -> value = module mV, UNIT_MILLIVOLT, HEALTH_UNCALIBRATED
24 //   window mean < PH_NO_SIGNAL_PIN_MV -> HEALTH_UNPOWERED (a 0 V output is a
25 //                         dead rail or open lead, never "very acidic", §10).
26 //                         Also seen before the offset pot is set: factory
27 //                         offset puts pH 7 near 0 V.
28 //   any sample clipped -> kQualitySaturated + HEALTH_FAULT
29 // poll() returns READY for all of these; the health carries the verdict.
30 // raw = filtered pin mV in every case.
31 //
32 // Worst-case blocking (one ADC conversion C, well under 1 ms):
33 //   init()    1 C   (throwaway conversion, checks the channel)
34 //   start()   1 C   (first sample of the window)
35 //   poll()    1 C   Pending until the window is full (>= 270 ms)
36 //   others    none
37 //
38 // Implements the SensorDriver contract in drivers/sensor_driver.h.
39 #pragma once
```

Return to guided explanation, p. 212

ORIGINAL SOURCE / CHAPTER 8 / 2 OF 2

src/drivers/ph.h

Original lines 40-78 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
40 #include "drivers/sensor_driver.h"
41 #include "hal/adc_channel.h"
42 #include "domain/calibration.h"
43
44 #define PH_SAMPLES 10
45 #define PH_TRIM 2
46 #define PH_SAMPLE_SPACING_MS 30
47 // Provisional: below this at the pin the module is treated as unpowered.
48 // pH 14 is ~1.3 V at PO (~0.8 V at the pin), so 100 mV is far from real data.
49 #define PH_NO_SIGNAL_PIN_MV 100
50
51 typedef struct PhConfig {
52   AdcChannel* adc;             // already begun on BOARD_PIN_PH_ADC
53   uint32_t dividerTopOhm;      // BOARD_PH_DIVIDER_TOP_OHM
54   uint32_t dividerBottomOhm;   // BOARD_PH_DIVIDER_BOTTOM_OHM
55   LinearCalibration calibration;  // x = module mV, y = pH
56   bool calibrationProvisional;    // true: fit not checked against buffers
57 } PhConfig;
58
59 // Members after `base` are private to ph.c.
60 typedef struct PhDriver {
61   SensorDriver base;  // must stay first
62   PhConfig config_;
63   bool initialised_;
64   bool busy_;
65   bool saturated_;
66   uint8_t count_;
67   uint32_t samples_[PH_SAMPLES];  // pin mV
68   MonoMs startMs_;
69   MonoMs lastSampleMs_;  // next tap is due PH_SAMPLE_SPACING_MS after this
70   Reading result_;
71 } PhDriver;
72
73 // Constructor. Use &driver.base wherever a SensorDriver* is expected.
74 void phDriverConstruct(PhDriver* self, const PhConfig* config);
75
76 // New fit from the applied configuration (on-device calibration, WP8).
77 // Called by the acquisition owner between windows; applies from the next one.
78 void phDriverSetCalibration(PhDriver* self, const LinearCalibration* calibration, bool
 > provisional);
```

Return to guided explanation, p. 212

GUIDED SOURCE READING / CHAPTER 8

Keep pin voltage, module voltage and pH in that order

pH shares the spaced analog window with light, but its signal passes through a resistor divider and a probe-specific line fit. finish follows that physical chain.

Original excerpt: src/drivers/ph.c (p. 219) / lines 86-98. The complete unchanged file follows.

```
86    const float pinMv = trimmedMeanU32(self->samples_, PH_SAMPLES, PH_TRIM);
87    const float moduleMv = dividerRecoverMv(pinMv, cfg->dividerTopOhm, cfg->dividerBottomOhm);
88
89    Reading* r = &self->result_;
90    r->intervalStartMs = self->startMs_;
91    r->intervalEndMs = now;
92    r->hasRaw = true;
93    r->raw = (int32_t)(pinMv + 0.5f);
94    r->qualityFlags = self->saturated_ ? kQualitySaturated : kQualityNone;
95    r->calibrationRevision = cfg->calibration.revision;
96
97    // READY in every case below: health carries the verdict, raw the level.
98    PollResult pr = {DRIVER_READY, now};
```

The filter first produces pinMv. dividerRecoverMv then multiplies by(10000 +15000)/15000 to recover module voltage. Applying the pH line directly to pin voltage would use the wrong x scale and shift every result.

The Reading preserves the interval and raw pin mV before choosing a converted value. The fit revision identifies which line is being used. This makes a later change in displayed pH distinguishable from a change in the electrical input.

The decision chain below first checks pin voltage below 100 mV, then saturation. Without a valid fit it publishes diagnostic module mV. With a fit it computes pH, rejects non-finite output and rejects out-of-range verified results. A provisional fit remains UNCALIBRATED even when it gives a neat-looking number; nominal coefficients do not establish a buffer measurement.

The pH range exception for provisional fits keeps otherwise useful raw voltage available to calibration capture. It does not make those readings actionable for generic alarm rules. The caller therefore needs both the number's unit and its health to interpret the result.

GUIDED SOURCE READING / CHAPTER 8

A fitted line is a model with a verification state

Once voltage is known, pH conversion is a straight line. The code deliberately separates having a usable mathematical expression from having a reference-verified probe calibration.

Original excerpt: src/drivers/ph.c (p. 219) / lines 116-138. The complete unchanged file follows.

```
116    const float ph = linearApply(&cfg->calibration, moduleMv);
117    r->unit = UNIT_PH;
118    if (!isfinite(ph)) {  // a broken fit is a fault, never a value
119      r->health = HEALTH_FAULT;
120      return pr;
121    }
122    if ((ph < kPhMin || ph > kPhMax) && !cfg->calibrationProvisional) {
123      r->health = HEALTH_FAULT;
124      return pr;
125    }
126    // Out of range on the provisional (nominal) fit: the fit may simply not
127    // match this probe yet. Published UNCALIBRATED (never actionable), so the
128    // Calibration page can still capture it and fix the fit.
129    r->value = ph;
130    ++r->sequence;
131    r->lastGoodMs = now;
132    if (cfg->calibrationProvisional) {  // pH units, but not buffer-verified (D13)
133      r->health = HEALTH_UNCALIBRATED;
134      return pr;
135    }
136    r->health = HEALTH_OK;
137    return pr;
138  }
```

linearApply evaluates slope times recovered module mV plus offset. UNIT_PH is assigned because value now describes pH rather than voltage. The non-finite test prevents NaN or infinity from becoming an accepted measurement.

The following range guard rejects values outside 0-14 only when the fit is not provisional. A nominal line can be wrong for an uncalibrated probe, so retaining its UNCALIBRATED diagnostic output helps capture references. A verified fit outside the physical range instead produces FAULT.

Consider illustrative buffers 2500 mV at pH 6.86 and 3000 mV at pH 4.01. Slope is(4.01-6.86)/(3000-2500) =-0.0057 pH/mV; offset is 6.86 -(-0.0057 times 2500) =21.11. Those coefficients are specific to those points, not automatically the compiled nominal offset.

The setter copies a new fit and provisional flag when acquisition applies a configuration revision. It does not sample the ADC or create a task. This allows calibration to change the conversion model while the acquisition worker retains exclusive control of measurement timing.

ORIGINAL SOURCE / CHAPTER 8 / 1 OF 4

src/drivers/ph.c

Original lines 1-46 | Code and comments unchanged; long lines wrap visually

asPh (L7-7): Cast the common first-member SensorDriver pointer back to this concrete driver type.
phId (L11-11): Return this module's sensor identity; this does not access hardware.

```
1  #include "drivers/ph.h"
2
3  #include <math.h>
4
5  static const float kPhMin = 0.0f, kPhMax = 14.0f;
6
7  static inline PhDriver* asPh(SensorDriver* d) { return (PhDriver*)d; }
8
9  static PollResult finish(PhDriver* self, MonoMs now);
10
11 static SensorId phId(const SensorDriver* d) { (void)d; return SENSOR_PH; }
12 static const char* phName(const SensorDriver* d) { (void)d; return "ph4502c"; }
13 static uint8_t phMetricCount(const SensorDriver* d) { (void)d; return 1; }
14
15 static DriverStatus phInit(SensorDriver* d) {
16   PhDriver* self = asPh(d);
17   self->initialised_ = false;
18   self->busy_ = false;
19   if (self->config_.adc == 0 || self->config_.dividerBottomOhm == 0) return DRIVER_FAULT;
20   // Throwaway conversion: fails if the channel was never begun on an ADC1 pin,
21   // and keeps any one-time ADC characterisation cost out of poll().
22   uint32_t mv = 0;
23   bool saturated = false;
24   if (!adcChannelReadMilliVolts(self->config_.adc, &mv, &saturated)) return DRIVER_FAULT;
25   self->initialised_ = true;
26   return DRIVER_OK;
27 }
28
29 // Take one sample of the window at `now`. false on an ADC error.
30 static bool sample(PhDriver* self, MonoMs now) {
31   uint32_t mv = 0;
32   bool saturated = false;
33   if (!adcChannelReadMilliVolts(self->config_.adc, &mv, &saturated)) return false;
34   self->samples_[self->count_++] = mv;
35   self->lastSampleMs_ = now;
36   self->saturated_ = self->saturated_ || saturated;
37   return true;
38 }
39
40 static DriverStatus phStart(SensorDriver* d, MonoMs now) {
41   PhDriver* self = asPh(d);
42   if (!self->initialised_) return DRIVER_FAULT;
43   self->count_ = 0;
44   self->saturated_ = false;
45   self->startMs_ = now;
46   if (!sample(self, now)) {
```

Return to guided explanation, p. 217

ORIGINAL SOURCE / CHAPTER 8 / 2 OF 4

src/drivers/ph.c

Original lines 47-92 | Code and comments unchanged; long lines wrap visually

phStart (L40-52): Begin one operation and record its state; completion is handled later by poll.
phPoll (L54-81): Advance an in-flight operation only when useful; return PENDING with a future time, READY, or failure.

```
47      self->initialised_ = false;
48      return DRIVER_FAULT;
49    }
50    self->busy_ = true;
51    return DRIVER_OK;
52  }
53
54  static PollResult phPoll(SensorDriver* d, MonoMs now) {
55    PhDriver* self = asPh(d);
56    PollResult r = {DRIVER_FAULT, now};
57    if (!self->busy_) return r;
58
59    // Spacing is measured from the previous actual sample, never from the window
60    // start: after a late poll, catching up would compress the filter taps.
61    const MonoMs due = self->lastSampleMs_ + PH_SAMPLE_SPACING_MS;
62    if (now < due) {
63      r.status = DRIVER_PENDING;
64      r.nextPollMs = due;
65      return r;
66    }
67    if (!sample(self, now)) {
68      self->busy_ = false;
69      self->initialised_ = false;
70      self->result_.health = HEALTH_FAULT;
71      self->result_.intervalStartMs = self->startMs_;
72      self->result_.intervalEndMs = now;
73      return r;
74    }
75    if (self->count_ < PH_SAMPLES) {
76      r.status = DRIVER_PENDING;
77      r.nextPollMs = self->lastSampleMs_ + PH_SAMPLE_SPACING_MS;
78      return r;
79    }
80    return finish(self, now);
81  }
82
83  static PollResult finish(PhDriver* self, MonoMs now) {
84    self->busy_ = false;
85    const PhConfig* cfg = &self->config_;
86    const float pinMv = trimmedMeanU32(self->samples_, PH_SAMPLES, PH_TRIM);
87    const float moduleMv = dividerRecoverMv(pinMv, cfg->dividerTopOhm, cfg->dividerBottomOhm);
88
89    Reading* r = &self->result_;
90    r->intervalStartMs = self->startMs_;
91    r->intervalEndMs = now;
92    r->hasRaw = true;
```

Return to guided explanation, p. 217

ORIGINAL SOURCE / CHAPTER 8 / 3 OF 4

src/drivers/ph.c

Original lines 93-138 | Code and comments unchanged; long lines wrap visually

finish (L83-138): Follow ADC filtering, divider recovery, line fit and the health checks in their actual priority order.

```
  r->raw = (int32_t)(pinMv + 0.5f);
  r->qualityFlags = self->saturated_ ? kQualitySaturated : kQualityNone;
  r->calibrationRevision = cfg->calibration.revision;

  // READY in every case below: health carries the verdict, raw the level.
  PollResult pr = {DRIVER_READY, now};
  if (pinMv < (float)PH_NO_SIGNAL_PIN_MV) {
    r->health = HEALTH_UNPOWERED;
    return pr;
  }
  if (self->saturated_) {
    r->health = HEALTH_FAULT;
    return pr;
  }

  if (!cfg->calibration.valid) {
    r->unit = UNIT_MILLIVOLT;
    r->value = moduleMv;
    r->health = HEALTH_UNCALIBRATED;
    r->lastGoodMs = now;  // last usable result (OK or UNCALIBRATED), as the SGP30
    ++r->sequence;  // a fresh sample, in raw units
    return pr;
  }
  const float ph = linearApply(&cfg->calibration, moduleMv);
  r->unit = UNIT_PH;
  if (!isfinite(ph)) {  // a broken fit is a fault, never a value
    r->health = HEALTH_FAULT;
    return pr;
  }
  if ((ph < kPhMin || ph > kPhMax) && !cfg->calibrationProvisional) {
    r->health = HEALTH_FAULT;
    return pr;
  }
  // Out of range on the provisional (nominal) fit: the fit may simply not
  // match this probe yet. Published UNCALIBRATED (never actionable), so the
  // Calibration page can still capture it and fix the fit.
  r->value = ph;
  ++r->sequence;
  r->lastGoodMs = now;
  if (cfg->calibrationProvisional) {  // pH units, but not buffer-verified (D13)
    r->health = HEALTH_UNCALIBRATED;
    return pr;
  }
  r->health = HEALTH_OK;
  return pr;
}
```

Return to guided explanation, p. 217

ORIGINAL SOURCE / CHAPTER 8 / 4 OF 4

src/drivers/ph.c

Original lines 139-184 | Code and comments unchanged; long lines wrap visually

phReadResult (L140-144): Copy the latest Reading record(s) into caller storage. Inspect health even when poll returned READY.
phCancel (L146-150): Stop software tracking of partial work. A chip conversion can still finish internally.

```
139
140 static uint8_t phReadResult(SensorDriver* d, Reading* out, uint8_t maxOut) {
141   if (maxOut < 1) return 0;
142   out[0] = asPh(d)->result_;
143   return 1;
144 }
145
146 static void phCancel(SensorDriver* d) {
147   PhDriver* self = asPh(d);
148   self->busy_ = false;
149   self->count_ = 0;
150 }
151
152 // The ADC pin is a high-impedance input behind the divider: nothing to drive
153 // before the 5 V rail goes down. Drop any partial window.
154 static void phPreparePowerOff(SensorDriver* d) { phCancel(d); }
155
156 static const SensorDriverOps kPhOps = {
157     .id = phId,
158     .name = phName,
159     .metricCount = phMetricCount,
160     .init = phInit,
161     .start = phStart,
162     .poll = phPoll,
163     .readResult = phReadResult,
164     .cancel = phCancel,
165     .preparePowerOff = phPreparePowerOff,
166 };
167
168 void phDriverConstruct(PhDriver* self, const PhConfig* config) {
169   self->base.ops = &kPhOps;
170   self->config_ = *config;
171   self->initialised_ = false;
172   self->busy_ = false;
173   self->saturated_ = false;
174   self->count_ = 0;
175   self->startMs_ = 0;
176   self->lastSampleMs_ = 0;
177   readingReset(&self->result_, SENSOR_PH, METRIC_PH, UNIT_PH);
178   self->result_.calibrationRevision = config->calibration.revision;
179 }
180
181 void phDriverSetCalibration(PhDriver* self, const LinearCalibration* calibration, bool
  > provisional) {
182   self->config_.calibration = *calibration;
183   self->config_.calibrationProvisional = provisional;
184 }
```

Return to guided explanation, p. 218

ORIGINAL SOURCE / CHAPTER 8 / 1 OF 2

test/bringup/ph_main.c

Original lines 1-42 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // Bring-up entry point for ph (WP5).  pio run -e test_ph -t upload -t monitor
2  // Switched 5 V rail must be ON. PO -> 10k/15k -> GPIO7 (ADC1_CH6).
3  // Replaces src/main.c in this env only; links the production driver.
4  //
5  // Procedure (README "pH-4502C calibration"). Every reading is followed by a
6  // "module NNNN mV" line: the module PO voltage the buffer fit is made from,
7  // whatever the pH line shows (nominal fit, buffer fit or none).
8  //  1. Offset: probe off, BNC shorted. Turn the pot nearest the BNC until the
9  //     module line reads ~2500 mV - that is the module PO voltage (pH 7).
10 //     A factory-set module starts near 0 V and prints "ph=UNPOWERED": check
11 //     the 5 V rail, then keep turning the pot until mV values appear.
12 //  2. Probe in, rinse, pH 6.86 buffer, wait >= 2 min, note the module mV.
13 //     Rinse, pH 4.01 buffer, wait >= 2 min, note the module mV.
14 //  3. Put both mV values below and re-flash: readings switch to pH / OK.
15 // Without buffers: pH comes from the nominal fit below and prints "(uncal)".
16 #include <Arduino.h>
17 #include <stdio.h>
18
19 #include "app_config.h"
20 #include "board_profile.h"
21 #include "bringup_harness.h"
22 #include "hal/adc_channel.h"
23 #include "domain/calibration.h"
24 #include "drivers/ph.h"
25
26 // Module mV measured in each buffer (step 2). 0 = not measured yet: uncalibrated.
27 #define PH_BUFFER_A_PH 6.86f
28 #define PH_BUFFER_A_MV 0.0f
29 #define PH_BUFFER_B_PH 4.01f
30 #define PH_BUFFER_B_MV 0.0f
31 #define PH_CALIBRATION_REVISION 1
32
33 // Nominal fit, used until both buffer values are set. Offset: module mV with the
34 // BNC shorted (pH 7), measured 17 Oct 2026. Re-measure if the offset pot moves.
35 // Slope: tutorial sketch, pH = -5.70 * V. Accuracy roughly +/-0.5-1 pH.
36 #define PH_NOMINAL_MV_AT_PH7 2758.0f
37 #define PH_NOMINAL_PH_PER_MV (-0.0057f)
38
39 static AdcChannel s_adc = ADC_CHANNEL_INIT;
40 static PhConfig s_config = {&s_adc, BOARD_PH_DIVIDER_TOP_OHM, BOARD_PH_DIVIDER_BOTTOM_OHM,
41                             {1.0f, 0.0f, 0, false}, false};  // set in setup()
42 static PhDriver s_driver;
```

Return to guided explanation, p. 212

ORIGINAL SOURCE / CHAPTER 8 / 2 OF 2

test/bringup/ph_main.c

Original lines 43-84 | Code and comments unchanged; long lines wrap visually

printModuleMv (L44-49): Present already-collected diagnostic information; this does not make the measurement more valid.
setup (L53-82): Establish safe outputs, logger, gate, board check and queues; load configuration; start Safety, both acquisition workers, power, radios and supervision.

```
43  // The module mV for the procedure, from the pin mV the driver publishes as raw.
44  static void printModuleMv(SensorDriver* driver) {
45    Reading r;
46    if (sensorDriverReadResult(driver, &r, 1) != 1 || !r.hasRaw) return;
47    printf("    module %.0f mV\n",
48           (double)dividerRecoverMv((float)r.raw, BOARD_PH_DIVIDER_TOP_OHM,
 >  BOARD_PH_DIVIDER_BOTTOM_OHM));
49  }
50
51  static BringupContext s_ctx = {&s_driver.base, CFG_FRAME_PERIOD_MS, printModuleMv};
52
53  void setup(void) {
54    if (PH_BUFFER_A_MV > 0.0f && PH_BUFFER_B_MV > 0.0f) {
55      linearFromTwoPoints(PH_BUFFER_A_MV, PH_BUFFER_A_PH, PH_BUFFER_B_MV, PH_BUFFER_B_PH,
56                          PH_CALIBRATION_REVISION, &s_config.calibration);
57    } else {
58      s_config.calibration.slope = PH_NOMINAL_PH_PER_MV;
59      s_config.calibration.offset = 7.0f - PH_NOMINAL_PH_PER_MV * PH_NOMINAL_MV_AT_PH7;
60      s_config.calibration.revision = 0;
61      s_config.calibration.valid = true;
62      s_config.calibrationProvisional = true;
63    }
64    phDriverConstruct(&s_driver, &s_config);
65    delay(200);
66    printf("\n=== bring-up: %s ===\n", sensorDriverName(&s_driver.base));
67    if (!adcChannelBegin(&s_adc, BOARD_PIN_PH_ADC)) {
68      printf("GPIO%d is not an ADC1 pin\n", (int)BOARD_PIN_PH_ADC);
69    }
70    if (s_config.calibrationProvisional) {
71      printf("nominal pH (not buffer-calibrated): pH 7 at %.0f mV, %.4f pH/mV\n",
72             (double)PH_NOMINAL_MV_AT_PH7, (double)PH_NOMINAL_PH_PER_MV);
73    } else if (s_config.calibration.valid) {
74      // Tutorial nominal is about -0.0057 pH/mV; far from it means a bad buffer reading.
75      printf("calibration rev %u: pH = %.6f * mV + %.3f\n",
76             (unsigned)s_config.calibration.revision, (double)s_config.calibration.slope,
77             (double)s_config.calibration.offset);
78    } else {
79      printf("uncalibrated: printing module PO voltage in mV\n");
80    }
81    bringupStart(&s_ctx);
82  }
83
84  void loop(void) { vTaskDelay(portMAX_DELAY); }
```

Return to guided explanation, p. 212

PART II / SENSOR ARCHITECTURES

CHAPTER 09

SEN0244: TDS Measurement

The TDS driver combines two observations: filtered probe voltage and a copied water temperature. A cubic conversion, temperature correction and reference gain turn those inputs into the reported ppm value. The result is accepted only when its dependencies and range checks pass.

In this chapter

1. Follow the fifteen-tap window and temperature dependency.
2. Calculate the voltage conversion and compensation order.
3. Explain range faults and provisional calibration health.

Chapter contents

9.1 TDS: a voltage plus a water temperature 226
9.2 TDS: the polynomial and temperature dependency 227
9.3 TDS: filter, limits and undetectable faults 228

Guided source readings begin on page 233.

Complete original listings begin on page 229. Shared files are linked in the explanations; Appendix A contains the full filename index.

CHAPTER 9

9.1 TDS: a voltage plus a water temperature

The module voltage depends on conductivity, which changes with temperature. The code therefore needs a second sensor's accepted water-temperature Reading before it can accept a compensated result.

Stage	What it does
ADC window	Collect 15 taps, 40 ms apart.
Filter	Keep the middle five after trimming five at each end.
Cubic	Estimate nominal ppm from module voltage.
Temperature correction	Divide the result by 1 + 0.02 * (T - 25).
Calibration and health	Apply an optional fit and classify the output.

At 1500 mV the cubic gives about 580.35 ppm. At 30 degC the factor is 1.10, so the 25 degC-referred result is about 527.59 ppm. These are formula examples, not reference measurements.

Why the first frame can say DEPENDENCY_INVALID

The TDS driver starts with no water-temperature result. It may finish its 560 ms filter window before the first DS18B20 conversion finishes. Later frames can use a recent previous reading. The copied temperature is checked again for age and health at TDS finish.

No accepted temperature means a diagnostic number may exist, but health prevents treating it as an accepted compensated measurement. No calibration fit means UNCALIBRATED even with valid temperature. Above the supported range or a clipped tap produces FAULT.

The full header, driver, polynomial helper and standalone test are in this chapter. The standalone test can inject a manual temperature for bench work; production uses the real DS18B20 result.

CHAPTER 9

9.2 TDS: the polynomial and temperature dependency

SEN0244 output is sampled directly on GPIO6. The driver converts filtered module millivolts to volts, evaluates the nominal cubic and refers the result to 25 degC.

```
// Source: src/domain/tds_codec.c
const float v = moduleMv / 1000.0f;
const float ppm = (133.42f*v*v*v - 255.86f*v*v
                   + 857.39f*v) * 0.5f;
return ppm / coefficient;
```

The coefficient is `1 + 0.02 * (waterTempC - 25)`. It approximates the conductivity change with temperature. The implementation divides the polynomial's output by this factor; it does not divide voltage before evaluating the nonlinear cubic. This matches the compensation location in DFRobot's GravityTDS library. [R7]

Worked conversion

For 1500 mV, v = 1.5 and the nominal result before compensation is about 580.35 ppm. At 30 degC, the coefficient is 1.10, giving about 527.59 ppm referred to 25 degC. At 20 degC the coefficient is 0.90, giving about 644.83 ppm. These calculations illustrate the formula; they do not establish reference-solution accuracy.

An optional `LinearCalibration` then maps nominal ppm to corrected ppm. In production the fit is invalid, so the nominal curve remains UNCALIBRATED even when compensation is available.

Why water temperature is copied

`tdsDriverSetWaterTemperature()` copies a Reading into private driver state. At finish, `temperatureUsable()` requires HEALTH_OK, age no greater than 3000 ms, and temperature between 0 and 55 degC. It does not accept a convenient default 25 degC without declaring that information is missing.

If temperature is unusable, the code still computes an uncompensated numeric value for diagnostics but marks DEPENDENCY_INVALID and sets the uncompensated flag. Generic alarm rules cannot act on it. An apparently reasonable ppm value is not sufficient evidence of a valid compensated measurement.

Original source in this PDF: src/domain/tds_codec.c (p. 230); src/drivers/tds.c (p. 235); src/drivers/tds.h (p. 231)

CHAPTER 9

9.3 TDS: filter, limits and undetectable faults

TDS uses 15 samples spaced 40 ms apart, trims five at each end and averages the middle five. The 560 ms nominal window fits within the one-second frame while the DS18B20 independently converts.

`tdsInit()`, `tdsStart()`, `sample()` and `tdsPoll()` implement the same incremental ADC lifecycle as pH and light. `finish()` computes ppm first, writes the diagnostic raw/value fields, then decides whether that value can be accepted.

Result precedence

Any saturated tap or ppm above 1000 sets the saturated flag and HEALTH_FAULT. This range guard runs before the missing-temperature check. Next, missing temperature produces DEPENDENCY_INVALID. Otherwise sequence and lastGoodMs advance, with UNCALIBRATED for the nominal curve or OK for a valid calibration.

The 1000 ppm limit comes from the selected module's rated range, not from the numeric capacity of float. Nutrient concentrations above that range require a sensor with an appropriate measurement range. Raising the software limit does not extend the module rating.

Zero does not identify a failed rail

Near-zero voltage can mean low-conductivity water, a probe in air, an open lead or a dead supply. The driver therefore has no pH-style low-signal UNPOWERED threshold. Production acquisition gates the sensor on rail validity and an unchanged epoch. That supplies power-history evidence separately from the analog signal.

`tdsReadResult()` copies the latest record. `tdsCancel()` clears partial-window state; `preparePowerOff()` cancels the passive ADC interface. The constructor resets the temperature dependency to STARTING, ensuring the first TDS sample cannot silently use an invented temperature.

What test_tds changes

The standalone TDS image does not run DS18B20. A positive manual temperature constant enables a callback that repeatedly stamps a synthetic temperature as fresh. This is clearly bring-up-only behaviour. The default zero value means the helper is disabled; it does not mean the water was measured at 0 degC.

Original source in this PDF: src/drivers/tds.c (p. 235); src/drivers/tds.h (p. 231); test/bringup/tds_main.c (p. 239); src/services/acquisition.c (p. 96)

ORIGINAL SOURCE / CHAPTER 9 / 1 OF 1

src/domain/tds_codec.h

Original lines 1-27 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // SEN0244 TDS conversion (WP5). Pure C, host-tested.
2  // Source: DFRobot Gravity Analog TDS Sensor SEN0244 (local copy in
3  // Electrical System Study/Datasheets/SEN0244.PDF) for the curve, and DFRobot's
4  // GravityTDS library for where the temperature compensation is applied:
5  //   ppm(T)      = (133.42 V^3 - 255.86 V^2 + 857.39 V) * 0.5
6  //   coefficient = 1 + 0.02 * (T - 25)
7  //   ppm25       = ppm(T) / coefficient
8  // The datasheet's sample sketch divides the VOLTAGE by the coefficient
9  // instead. Because the curve is not linear, that over-corrects: on 24 Oct 2026
10 // the same tap water at 11.4 / 33.6 / 11.6 degC gave 683 / 584 / 713 ppm with
11 // the sketch's method and 631 / 620 / 653 ppm compensating the ppm (EC), as
12 // the library does. Conductivity is what changes by ~2 %/degC.
13 // The sample's VREF/1024 scaling is not ported: the ADC HAL already returns
14 // calibrated millivolts, and the module output reaches the pin undivided.
15 #pragma once
16
17 #define TDS_FULL_SCALE_PPM 1000.0f   // rated range 0-1000 ppm, +/-10 % F.S.
18 #define TDS_MIN_WATER_TEMP_C 0.0f    // below: frozen water, not a usable input
19 #define TDS_MAX_WATER_TEMP_C 55.0f   // datasheet probe limit
20
21 // Temperature compensation coefficient, 1.0 at 25 degC.
22 float tdsCompensationCoefficient(float waterTempC);
23
24 // Module output (mV) to TDS (ppm) referred to 25 degC: the curve's ppm divided
25 // by `coefficient` (1.0 = no compensation). 0 for a non-positive voltage or
26 // coefficient.
27 float tdsPpmFromModuleMv(float moduleMv, float coefficient);
```

Return to guided explanation, p. 226

ORIGINAL SOURCE / CHAPTER 9 / 1 OF 1

src/domain/tds_codec.c

Original lines 1-12 | Code and comments unchanged; long lines wrap visually

tdsCompensationCoefficient (L3-5): Return the approximate 2%/degC conductivity factor relative to 25 degC.
tdsPpmFromModuleMv (L7-12): Convert millivolts to volts, evaluate the nominal cubic, then correct its result for temperature.

```
1  #include "domain/tds_codec.h"
2
3  float tdsCompensationCoefficient(float waterTempC) {
4    return 1.0f + 0.02f * (waterTempC - 25.0f);
5  }
6
7  float tdsPpmFromModuleMv(float moduleMv, float coefficient) {
8    if (moduleMv <= 0.0f || coefficient <= 0.0f) return 0.0f;
9    const float v = moduleMv / 1000.0f;
10   const float ppm = (133.42f * v * v * v - 255.86f * v * v + 857.39f * v) * 0.5f;
11   return ppm / coefficient;  // compensate the conductivity, not the voltage
12 }
```

Return to guided explanation, p. 226

ORIGINAL SOURCE / CHAPTER 9 / 1 OF 2

src/drivers/tds.h

Original lines 1-39 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // sen0244_tds driver. Work package WP5 (Stage 3). Architecture §8, §14.
2  // Source: DFRobot Gravity Analog TDS Sensor SEN0244 datasheet (wiki + sample).
3  //
4  // Switched 5 V rail. A -> GPIO6 (ADC1_CH5), 100 nF to GND, no divider: pin mV
5  // = module mV. Output 0-2.3 V nominal, inside the S3's ~3.1 V range at 11 dB.
6  // Pin mV -> datasheet cubic -> temperature compensation of the ppm (EC, as
7  // DFRobot GravityTDS; domain/tds_codec.h)
8  // -> optional K-factor (calibration: x = nominal ppm, y = ppm) -> ppm.
9  //
10 // Filter: TDS_SAMPLES conversions spaced TDS_SAMPLE_SPACING_MS apart, sorted,
11 // mean of the middle TDS_SAMPLES - 2*TDS_TRIM (stands in for the sample's
12 // median). The sample takes 30 taps 40 ms apart (~1.2 s), which overruns the
13 // 1 s frame, so the window is shortened to ~560 ms. Spread over successive
14 // poll() calls, never a burst (§9), spacing measured from the previous tap.
15 //
16 // Temperature compensation needs a fresh DS18B20 reading via
17 // tdsDriverSetWaterTemperature(): OK and not older than CFG_STALE_ORDINARY_MS
18 // at the end of the window, within 0-55 degC (probe limit). Otherwise the
19 // value is computed uncompensated - never silently assumed to be 25 degC.
20 //
21 // Results (unit ppm, raw = filtered pin mV in every case), highest first:
22 //   any tap clipped or ppm > 1000 -> kQualitySaturated + HEALTH_FAULT (value
23 //                         kept; above the rated range, never actionable)
24 //   no valid temperature  -> kQualityUncompensated + HEALTH_DEPENDENCY_INVALID
25 //   !calibration.valid    -> HEALTH_UNCALIBRATED (nominal datasheet curve, D13)
26 //   otherwise             -> HEALTH_OK
27 // poll() returns READY for all of these; the health carries the verdict.
28 //
29 // Not detectable here: 0 V is distilled water, a probe in air, an open lead or
30 // a dead rail alike, so there is no UNPOWERED threshold. Rail state is left to
31 // acquisition (rail epoch, WP9).
32 //
33 // Worst-case blocking (one ADC conversion C, well under 1 ms):
34 //   init()    1 C   (throwaway conversion, checks the channel)
35 //   start()   1 C   (first sample of the window)
36 //   poll()    1 C   Pending until the window is full (>= 560 ms)
37 //   others    none
38 //
39 // Implements the SensorDriver contract in drivers/sensor_driver.h.
```

Return to guided explanation, p. 226

ORIGINAL SOURCE / CHAPTER 9 / 2 OF 2

src/drivers/tds.h

Original lines 40-78 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
40 #pragma once
41 #include "drivers/sensor_driver.h"
42 #include "hal/adc_channel.h"
43 #include "domain/calibration.h"
44
45 #define TDS_SAMPLES 15
46 #define TDS_TRIM 5
47 #define TDS_SAMPLE_SPACING_MS 40
48
49 typedef struct TdsConfig {
50   AdcChannel* adc;                // already begun on BOARD_PIN_TDS_ADC
51   LinearCalibration calibration;  // K-factor: x = nominal ppm, y = ppm
52 } TdsConfig;
53
54 // Members after `base` are private to tds.c.
55 typedef struct TdsDriver {
56   SensorDriver base;  // must stay first
57   TdsConfig config_;
58   bool initialised_;
59   bool busy_;
60   bool saturated_;
61   uint8_t count_;
62   uint32_t samples_[TDS_SAMPLES];  // pin mV
63   MonoMs startMs_;
64   MonoMs lastSampleMs_;  // next tap is due TDS_SAMPLE_SPACING_MS after this
65   Reading waterTemperature_;
66   Reading result_;
67 } TdsDriver;
68
69 // Constructor. Use &driver.base wherever a SensorDriver* is expected.
70 void tdsDriverConstruct(TdsDriver* self, const TdsConfig* config);
71
72 // Compensation input from the latest DS18B20 reading (copied, not shared).
73 // Checked for health and age when the window finishes; no ADC access here.
74 void tdsDriverSetWaterTemperature(TdsDriver* self, const Reading* waterTemperature);
75
76 // New K-factor from the applied configuration (on-device calibration, WP8).
77 // Called by the acquisition owner between windows; applies from the next one.
78 void tdsDriverSetCalibration(TdsDriver* self, const LinearCalibration* calibration);
```

Return to guided explanation, p. 226

GUIDED SOURCE READING / CHAPTER 9

Check the dependency when the filter finishes

TDS conversion needs both a filtered voltage and accepted water temperature. Temperature is copied at start, but it must still be usable when the longer analog window finishes.

Original excerpt: src/drivers/tds.c (p. 235) / lines 96-112. The complete unchanged file follows.

```
 96    const bool compensated = temperatureUsable(self, now);
 97    const float coefficient =
 98        compensated ? tdsCompensationCoefficient(self->waterTemperature_.value) : 1.0f;
 99    float ppm = tdsPpmFromModuleMv(pinMv, coefficient);
100    if (cfg->calibration.valid) ppm = linearApply(&cfg->calibration, ppm);
101
102    Reading* r = &self->result_;
103    r->intervalStartMs = self->startMs_;
104    r->intervalEndMs = now;
105    r->hasRaw = true;
106    r->raw = (int32_t)(pinMv + 0.5f);
107    r->unit = UNIT_PPM;
108    r->value = ppm;
109    r->qualityFlags = compensated ? kQualityNone : kQualityUncompensated;
110    r->calibrationRevision = cfg->calibration.revision;
111
112    // READY in every case below: health carries the verdict, raw the level.
```

temperatureUsable checks health, age and 0-55 degC bounds at now. Only then is the temperature coefficient computed from that saved Reading. If unavailable, coefficient 1 permits a diagnostic nominal calculation, while a later health branch marks DEPENDENCY_INVALID.

The voltage-to-ppm cubic runs before the optional calibration. This order matters: the stored gain corrects the nominal temperature-referred ppm, not the millivolt input to a nonlinear polynomial. A gain moved ahead of the cubic would produce a different physical model.

Metadata is written before the verdict branches so even a rejected result retains its interval, raw signal, quality and calibration revision. The uncompensated quality bit explains which calculation path was used.

At 1500 mV, the nominal cubic yields about 580.35 ppm. At 30 degC, coefficient 1.10 gives 527.59 ppm referred to 25 degC. A fitted gain 1.2 would then give 633.11 ppm. Keeping those stages explicit makes unit and calibration errors easier to locate than treating the full expression as one unexplained formula.

GUIDED SOURCE READING / CHAPTER 9

A diagnostic number can exist without an accepted measurement

TDS has calculated ppm, but publishing an OK measurement requires the remaining guards. Their order decides which problem is reported when more than one condition fails.

Original excerpt: src/drivers/tds.c (p. 235) / lines 114-133. The complete unchanged file follows.

```
114    if (!isfinite(ppm)) {  // a broken fit is a fault, never a value
115      r->health = HEALTH_FAULT;
116      return pr;
117    }
118    if (self->saturated_ || ppm > TDS_FULL_SCALE_PPM) {
119      r->qualityFlags |= kQualitySaturated;
120      r->health = HEALTH_FAULT;  // above the rated 0-1000 ppm: never actionable
121      return pr;
122    }
123    if (!compensated) {
124      r->health = HEALTH_DEPENDENCY_INVALID;
125      return pr;
126    }
127    ++r->sequence;
128    r->lastGoodMs = now;  // last usable result (OK or UNCALIBRATED)
129    if (!cfg->calibration.valid) {  // nominal datasheet curve, no reference check (D13)
130      r->health = HEALTH_UNCALIBRATED;
131      return pr;
132    }
133    r->health = HEALTH_OK;
```

Non-finite output returns FAULT first. Saturation or a value above 1000 ppm then returns FAULT with the saturated flag. The range guard precedes the dependency branch, so an over-range result is reported as a range failure even when temperature is also unavailable.

If temperature was not accepted, the function returns DEPENDENCY_INVALID before success bookkeeping. The previously computed number remains diagnostic; consumers must use health to decide whether it can drive a rule or be transmitted as usable data.

Only after the dependency check do sequence and lastGoodMs advance. Without a reference fit, the result is UNCALIBRATED. With one, health becomes OK. Thus filtering, compensation and reference calibration each answer a different question, and passing one does not silently imply the others.

Near-zero voltage has no universal TDS fault meaning: very low conductivity, an open probe and a dead module can overlap electrically. Production handles supply history in acquisition's rail checks. That information belongs outside this voltage-only conversion because the ADC cannot infer the supply epoch from a low number alone.

ORIGINAL SOURCE / CHAPTER 9 / 1 OF 4

src/drivers/tds.c

Original lines 1-46 | Code and comments unchanged; long lines wrap visually

asTds (L8-8): Cast the common first-member SensorDriver pointer back to this concrete driver type.
tdsId (L12-12): Return this module's sensor identity; this does not access hardware.

```
1  #include "drivers/tds.h"
2
3  #include <math.h>
4
5  #include "app_config.h"
6  #include "domain/tds_codec.h"
7
8  static inline TdsDriver* asTds(SensorDriver* d) { return (TdsDriver*)d; }
9
10 static PollResult finish(TdsDriver* self, MonoMs now);
11
12 static SensorId tdsId(const SensorDriver* d) { (void)d; return SENSOR_TDS; }
13 static const char* tdsName(const SensorDriver* d) { (void)d; return "sen0244_tds"; }
14 static uint8_t tdsMetricCount(const SensorDriver* d) { (void)d; return 1; }
15
16 static DriverStatus tdsInit(SensorDriver* d) {
17   TdsDriver* self = asTds(d);
18   self->initialised_ = false;
19   self->busy_ = false;
20   if (self->config_.adc == 0) return DRIVER_FAULT;
21   // Throwaway conversion: fails if the channel was never begun on an ADC1 pin,
22   // and keeps any one-time ADC characterisation cost out of poll().
23   uint32_t mv = 0;
24   bool saturated = false;
25   if (!adcChannelReadMilliVolts(self->config_.adc, &mv, &saturated)) return DRIVER_FAULT;
26   self->initialised_ = true;
27   return DRIVER_OK;
28 }
29
30 // Take one sample of the window at `now`. false on an ADC error.
31 static bool sample(TdsDriver* self, MonoMs now) {
32   uint32_t mv = 0;
33   bool saturated = false;
34   if (!adcChannelReadMilliVolts(self->config_.adc, &mv, &saturated)) return false;
35   self->samples_[self->count_++] = mv;
36   self->lastSampleMs_ = now;
37   self->saturated_ = self->saturated_ || saturated;
38   return true;
39 }
40
41 static DriverStatus tdsStart(SensorDriver* d, MonoMs now) {
42   TdsDriver* self = asTds(d);
43   if (!self->initialised_) return DRIVER_FAULT;
44   self->count_ = 0;
45   self->saturated_ = false;
46   self->startMs_ = now;
```

Return to guided explanation, p. 233

ORIGINAL SOURCE / CHAPTER 9 / 2 OF 4

src/drivers/tds.c

Original lines 47-93 | Code and comments unchanged; long lines wrap visually

tdsStart (L41-53): Begin one operation and record its state; completion is handled later by poll.
tdsPoll (L55-82): Advance an in-flight operation only when useful; return PENDING with a future time, READY, or failure.

```
47    if (!sample(self, now)) {
48      self->initialised_ = false;
49      return DRIVER_FAULT;
50    }
51    self->busy_ = true;
52    return DRIVER_OK;
53  }
54
55  static PollResult tdsPoll(SensorDriver* d, MonoMs now) {
56    TdsDriver* self = asTds(d);
57    PollResult r = {DRIVER_FAULT, now};
58    if (!self->busy_) return r;
59
60    // Spacing is measured from the previous actual sample, never from the window
61    // start: after a late poll, catching up would compress the filter taps.
62    const MonoMs due = self->lastSampleMs_ + TDS_SAMPLE_SPACING_MS;
63    if (now < due) {
64      r.status = DRIVER_PENDING;
65      r.nextPollMs = due;
66      return r;
67    }
68    if (!sample(self, now)) {
69      self->busy_ = false;
70      self->initialised_ = false;
71      self->result_.health = HEALTH_FAULT;
72      self->result_.intervalStartMs = self->startMs_;
73      self->result_.intervalEndMs = now;
74      return r;
75    }
76    if (self->count_ < TDS_SAMPLES) {
77      r.status = DRIVER_PENDING;
78      r.nextPollMs = self->lastSampleMs_ + TDS_SAMPLE_SPACING_MS;
79      return r;
80    }
81    return finish(self, now);
82  }
83
84  // True if the stored water temperature can drive compensation at `now`.
85  static bool temperatureUsable(const TdsDriver* self, MonoMs now) {
86    const Reading* t = &self->waterTemperature_;
87    return readingIsActionableAt(t, now, CFG_STALE_ORDINARY_MS) &&
88           t->value >= TDS_MIN_WATER_TEMP_C && t->value <= TDS_MAX_WATER_TEMP_C;
89  }
90
91  static PollResult finish(TdsDriver* self, MonoMs now) {
92    self->busy_ = false;
93    const TdsConfig* cfg = &self->config_;
```

Return to guided explanation, p. 233

ORIGINAL SOURCE / CHAPTER 9 / 3 OF 4

src/drivers/tds.c

Original lines 94-139 | Code and comments unchanged; long lines wrap visually

finish (L91-135): Follow the spaced ADC window, copied temperature dependency, cubic correction and health decision.
tdsReadResult (L137-141): Copy the latest Reading record(s) into caller storage. Inspect health even when poll returned READY.

```
94    const float pinMv = trimmedMeanU32(self->samples_, TDS_SAMPLES, TDS_TRIM);  // = module mV
95
96    const bool compensated = temperatureUsable(self, now);
97    const float coefficient =
98        compensated ? tdsCompensationCoefficient(self->waterTemperature_.value) : 1.0f;
99    float ppm = tdsPpmFromModuleMv(pinMv, coefficient);
100   if (cfg->calibration.valid) ppm = linearApply(&cfg->calibration, ppm);
101
102   Reading* r = &self->result_;
103   r->intervalStartMs = self->startMs_;
104   r->intervalEndMs = now;
105   r->hasRaw = true;
106   r->raw = (int32_t)(pinMv + 0.5f);
107   r->unit = UNIT_PPM;
108   r->value = ppm;
109   r->qualityFlags = compensated ? kQualityNone : kQualityUncompensated;
110   r->calibrationRevision = cfg->calibration.revision;
111
112   // READY in every case below: health carries the verdict, raw the level.
113   PollResult pr = {DRIVER_READY, now};
114   if (!isfinite(ppm)) {  // a broken fit is a fault, never a value
115     r->health = HEALTH_FAULT;
116     return pr;
117   }
118   if (self->saturated_ || ppm > TDS_FULL_SCALE_PPM) {
119     r->qualityFlags |= kQualitySaturated;
120     r->health = HEALTH_FAULT;  // above the rated 0-1000 ppm: never actionable
121     return pr;
122   }
123   if (!compensated) {
124     r->health = HEALTH_DEPENDENCY_INVALID;
125     return pr;
126   }
127   ++r->sequence;
128   r->lastGoodMs = now;  // last usable result (OK or UNCALIBRATED)
129   if (!cfg->calibration.valid) {  // nominal datasheet curve, no reference check (D13)
130     r->health = HEALTH_UNCALIBRATED;
131     return pr;
132   }
133   r->health = HEALTH_OK;
134   return pr;
135 }
136
137 static uint8_t tdsReadResult(SensorDriver* d, Reading* out, uint8_t maxOut) {
138   if (maxOut < 1) return 0;
139   out[0] = asTds(d)->result_;
```

Return to guided explanation, p. 233

ORIGINAL SOURCE / CHAPTER 9 / 4 OF 4

src/drivers/tds.c

Original lines 140-185 | Code and comments unchanged; long lines wrap visually

tdsReadResult (L137-141): Copy the latest Reading record(s) into caller storage. Inspect health even when poll returned READY.
tdsCancel (L143-147): Stop software tracking of partial work. A chip conversion can still finish internally.

```
  return 1;
}

static void tdsCancel(SensorDriver* d) {
  TdsDriver* self = asTds(d);
  self->busy_ = false;
  self->count_ = 0;
}

// The ADC pin is a high-impedance input (100 nF to GND): nothing to drive
// before the 5 V rail goes down. Drop any partial window.
static void tdsPreparePowerOff(SensorDriver* d) { tdsCancel(d); }

static const SensorDriverOps kTdsOps = {
    .id = tdsId,
    .name = tdsName,
    .metricCount = tdsMetricCount,
    .init = tdsInit,
    .start = tdsStart,
    .poll = tdsPoll,
    .readResult = tdsReadResult,
    .cancel = tdsCancel,
    .preparePowerOff = tdsPreparePowerOff,
};

void tdsDriverConstruct(TdsDriver* self, const TdsConfig* config) {
  self->base.ops = &kTdsOps;
  self->config_ = *config;
  self->initialised_ = false;
  self->busy_ = false;
  self->saturated_ = false;
  self->count_ = 0;
  self->startMs_ = 0;
  self->lastSampleMs_ = 0;
  readingReset(&self->waterTemperature_, SENSOR_DS18B20, METRIC_WATER_TEMPERATURE,
> UNIT_DEG_C);
  readingReset(&self->result_, SENSOR_TDS, METRIC_TDS, UNIT_PPM);
  self->result_.calibrationRevision = config->calibration.revision;
}

void tdsDriverSetWaterTemperature(TdsDriver* self, const Reading* waterTemperature) {
  self->waterTemperature_ = *waterTemperature;
}

void tdsDriverSetCalibration(TdsDriver* self, const LinearCalibration* calibration) {
  self->config_.calibration = *calibration;
}
```

Return to guided explanation, p. 234

ORIGINAL SOURCE / CHAPTER 9 / 1 OF 2

test/bringup/tds_main.c

Original lines 1-33 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // Bring-up entry point for tds (WP5).  pio run -e test_tds -t upload -t monitor
2  // Switched 5 V rail must be ON. A -> GPIO6 (ADC1_CH5), 100 nF to GND.
3  // Replaces src/main.c in this env only; links the production driver.
4  //
5  // Procedure (README "SEN0244 TDS bring-up"):
6  //  1. Probe in the liquid, away from the container wall (datasheet), stir
7  //     gently and wait for the reading to settle. Water below 55 degC only.
8  //  2. Without a water temperature every line is "tds=DEPENDENCY_INVALID
9  //     (raw <pin mV>)": this env runs the TDS driver alone, without the DS18B20.
10 //  3. Measure the water with a thermometer, put it in TDS_MANUAL_WATER_TEMP_C
11 //     and re-flash: lines switch to "<ppm> ppm (uncal)" (nominal curve).
12 // "tds=FAULT (raw ...)" means above 1000 ppm (raw >~ 2160 mV at 25 degC), not
13 // a wiring fault: an ADC error shows as "poll -> FAULT" instead.
14 #include <Arduino.h>
15 #include <stdio.h>
16
17 #include "app_config.h"
18 #include "board_profile.h"
19 #include "bringup_harness.h"
20 #include "hal/adc_channel.h"
21 #include "hal/clock.h"
22 #include "drivers/tds.h"
23
24 // Water temperature from a thermometer, degC. 0 = off (no compensation).
25 // BRING-UP ONLY: stands in for the DS18B20 and is re-stamped as fresh every
26 // frame. Production feeds the real DS18B20 reading.
27 #define TDS_MANUAL_WATER_TEMP_C 0.0f
28
29 static AdcChannel s_adc = ADC_CHANNEL_INIT;
30 static TdsConfig s_config = {&s_adc, {1.0f, 0.0f, 0, false}};  // nominal curve, no K-factor
31 static TdsDriver s_driver;
32
33 // Feed the manual temperature as a fresh OK DS18B20 reading.
```

Return to guided explanation, p. 226

ORIGINAL SOURCE / CHAPTER 9 / 2 OF 2

test/bringup/tds_main.c

Original lines 34-67 | Code and comments unchanged; long lines wrap visually

feedManualTemperature (L34-45): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.
setup (L49-65): Establish safe outputs, logger, gate, board check and queues; load configuration; start Safety, both acquisition workers, power, radios and supervision.

```
34  static void feedManualTemperature(SensorDriver* driver) {
35    (void)driver;
36    Reading t;
37    readingReset(&t, SENSOR_DS18B20, METRIC_WATER_TEMPERATURE, UNIT_DEG_C);
38    const MonoMs now = monoNowMs();
39    t.health = HEALTH_OK;
40    t.value = TDS_MANUAL_WATER_TEMP_C;
41    t.intervalStartMs = now;
42    t.intervalEndMs = now;
43    t.lastGoodMs = now;
44    tdsDriverSetWaterTemperature(&s_driver, &t);
45  }
46
47  static BringupContext s_ctx = {&s_driver.base, CFG_FRAME_PERIOD_MS, NULL};
48
49  void setup(void) {
50    tdsDriverConstruct(&s_driver, &s_config);
51    delay(200);
52    printf("\n=== bring-up: %s ===\n", sensorDriverName(&s_driver.base));
53    if (!adcChannelBegin(&s_adc, BOARD_PIN_TDS_ADC)) {
54      printf("GPIO%d is not an ADC1 pin\n", (int)BOARD_PIN_TDS_ADC);
55    }
56    if (TDS_MANUAL_WATER_TEMP_C > 0.0f) {
57      printf("manual water temperature %.1f degC (bring-up only, not a DS18B20)\n",
58             (double)TDS_MANUAL_WATER_TEMP_C);
59      feedManualTemperature(&s_driver.base);
60      s_ctx.afterResult = feedManualTemperature;
61    } else {
62      printf("no water temperature: readings are DEPENDENCY_INVALID, raw = pin mV\n");
63    }
64    bringupStart(&s_ctx);
65  }
66
67  void loop(void) { vTaskDelay(portMAX_DELAY); }
```

Return to guided explanation, p. 226

PART II / SENSOR ARCHITECTURES

CHAPTER 10

SGP30: TVOC and CO2eq

SGP30 contains an internal air-quality algorithm. Regular measurements, warm-up, humidity input and baseline handling all affect its results. The driver therefore remembers which command is outstanding and advances one command state at a time.

In this chapter

1. Read command words, reply order and word CRCs.
2. Follow measurement and housekeeping states without a long blocking wait.
3. Distinguish TVOC and estimated CO2eq from direct CO2 measurement.

Chapter contents

10.1 SGP30: a sensor with memory and timing rules 242
10.2 SGP30: command words, CRC and cadence 243
10.3 SGP30: a small state machine within one driver 244
10.4 SGP30: humidity and baseline determine health 245

Guided source readings begin on page 247.

Complete original listings begin on page 246. Shared files are linked in the explanations; Appendix A contains the full filename index.

CHAPTER 10

10.1 SGP30: a sensor with memory and timing rules

The gas chip runs an internal algorithm. The firmware must request measurements at approximately one-second intervals and keep track of warm-up, humidity input and baseline state.

Item	Plain meaning
IAQ init	Start/restart the chip's air-quality algorithm.
Measure IAQ	Request one CO2eq/TVOC result.
CRC	Check each two-byte reply word before using it.
Humidity input	Environmental correction derived from BME280.
Baseline	Learned algorithm state that can be restored after a restart.

Two different waits

After one measurement command, poll waits at least 13 ms before reading the reply. After IAQ init, results remain WARMING for the first 15 s. A short conversion wait and a long algorithm warm-up describe different processes.

A new setup without a restored baseline stays LEARNING for 12 h in this driver. The baseline can be kept in RAM across a driver re-init. Configuration loads an eligible stored baseline before initialization and persists changed offers at most hourly. Power-on or unknown downtime discards the old baseline; selected warm-reset reasons permit restore.

Read the state field as a next-step instruction

STEP_MEASURING means wait, then read measurement words. STEP_BASELINE_READ means wait, then read baseline words. STEP_WRITE_FAILED means publish the failed command outcome. STEP_IDLE means no operation is in flight.

CO2eq is an estimated equivalent value, not a direct CO2 measurement. The full driver, header, CRC/humidity helper and standalone test follow. Shared I2C code is included in the same PDF; the climate integration test is included later.

CHAPTER 10

10.2 SGP30: command words, CRC and cadence

SGP30 shares I2C address 0x58 with no configurable alternate address. It returns TVOC in ppb and CO2eq in ppm. CO2eq is an estimate derived from the gas-sensing algorithm, not a direct CO2 measurement. The board profile assumes a suitable module with regulation and level handling, not a bare 1.8V chip connected directly to 3.3V. [R8]

sgp30Init()

The driver requests the feature set, waits for the command duration and checks product-type bits. It sends IAQ init and restores a pending or RAM baseline if available. No baseline means LEARNING. It records warm-up and housekeeping times, clears consecutive-gap state and requests humidity compensation be resent after restart.

Command	Value	Purpose
IAQ init	0x2003	Starts the internal air-quality algorithm.
Measure IAQ	0x2008	Requests CO2eq and TVOC.
Get / set baseline	0x2015 / 0x201E	Transfers learned compensation state.
Set absolute humidity	0x2061	Updates environmental compensation.
Get feature set	0x202F	Checks product identity.

`sendCommand()` sends a big-endian 16-bit command. `sendCommandWords()` appends up to two parameter words, each with its own CRC. `readWords()` receives up to two data words and delegates validation to the codec.

CRC codec

`sgp30Crc8()` starts at 0xFF and processes the polynomial 0x31 most-significant-bit first. `sgp30PackWord()` emits high byte, low byte, CRC. `sgp30UnpackWords()` validates each word's CRC before decoding that word. If any word fails, the caller rejects the entire reply; an earlier decoded element does not make a partial reply acceptable.

A Measure IAQ reply orders CO2eq first and TVOC second. Set baseline uses TVOC first and CO2eq second. Explicit ordering in the helpers avoids silently swapping two valid-looking 16-bit values.

Original source in this PDF: src/drivers/sgp30.c (p. 254); src/domain/sgp30_codec.c (p. 248); include/board_profile.h (p. 32)

CHAPTER 10

10.3 SGP30: a small state machine within one driver

`sgp30Start()` checks timing gaps and sends Measure IAQ. A successful send selects STEP_MEASURING. A failed send selects STEP_WRITE_FAILED so poll can publish a fault without forcing an immediate re-init.

`sgp30Poll()` waits until start + 13 ms before reading the six-byte reply. SGP30 may NACK during conversion, so the code waits for the documented duration instead of repeatedly reading a busy chip. After validating the two words, it stores results and calls finish. [R8]

Housekeeping after the measurement

`finish()` performs at most one housekeeping command. An hourly baseline fetch takes priority; it creates STEP_BASELINE_READ and a second future poll after 11 ms. Otherwise a dirty humidity value can be written. The next scheduled measurement is roughly a second away.

STEP_BASELINE_READ copies a valid baseline into RAM, schedules the next hourly fetch and returns READY. A failed baseline read delays persistence information but does not invalidate the already-stored air-quality sample. STEP_WRITE_FAILED calls fail. An idle poll without a matching start returns FAULT.

Why a single fault does not reset everything

`fail()` marks both Readings FAULT and increments consecutive gaps. Start also notices intervals longer than 1800 ms. After three consecutive gap indications, start invalidates initialisation so the service will re-init. A good measurement resets the gap count.

An IAQ init restarts the 15 s warm-up. Re-initialising on every isolated NACK would keep the sensor warming repeatedly. The driver therefore allows limited transient failure without resetting its algorithm. This is device-specific recovery, unlike BME280's more immediate re-init policy.

The one-second cadence is maintained by the frame scheduler, not a private SGP30 task. Missing releases can affect the gas algorithm even when the CPU later has spare capacity.

Original source in this PDF: src/drivers/sgp30.c (p. 254); src/drivers/sgp30.h (p. 249)

CHAPTER 10

10.4 SGP30: humidity and baseline determine health

sgp30DriverSetHumidity accepts copied temperature and relative-humidity Readings only if both are fresh and actionable. It derives absolute humidity, encodes the 8.8 fixed-point command value and stages it for later bus work. Missing compensation changes result health rather than producing a fresh compensated claim.

Driver-side health

After IAQ initialization there is a 15 s warm-up. Without a restored baseline, the driver learns for 12 h before marking a baseline VALID and requesting its first copy. Later baseline copies are hourly. The gas algorithm expects approximately one measurement per second; repeated excessive gaps can restart it.

The measurement remains a TVOC estimate and a CO2-equivalent estimate. CO2eq is not direct carbon-dioxide measurement. Correct framing or an OK software verdict cannot change that physical interpretation.

Persistent baseline storage and restore

At boot Configuration may load sgp_base from NVS. Acquisition stages the accepted pair before the SGP30 initializes. During operation acquisition offers a ready baseline hourly; Configuration stores changed pairs at most hourly, through the flash gate.

A RAM baseline can survive driver reinitialization. NVS can survive MCU reboot. Neither establishes how long an unpowered sensor has been off, and the program has no wall clock to measure that downtime.

Conservative reset filtering

Configuration restores a stored baseline only for selected warm-reset reasons where the code assumes the sensor supply was retained: software, panic, watchdog and external reset. Power-on, brownout and unknown downtime discard the old key and relearn. This is a reset-reason heuristic; it does not physically measure sensor power continuity.

Baseline persistence and warm-up are separate. A stored pair improves restart state but does not remove the driver's startup timing or validate every sensor/environment combination. The Configuration chapter follows the baseline from this RAM copy into its separately rate-limited storage path.

Original source in this PDF: src/drivers/sgp30.c (p. 254); src/services/configuration.c (p. 374); src/services/acquisition.c (p. 96)

ORIGINAL SOURCE / CHAPTER 10 / 1 OF 1

src/domain/sgp30_codec.h

Original lines 1-28 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // SGP30 word framing and humidity conversion (WP6). Pure C, host-tested.
2  // Datasheet: Sensirion SGP30 v1.0, §6.3 (humidity compensation), §6.6 (CRC).
3  //
4  // Every 16-bit word on the wire is MSB, LSB, CRC-8. The CRC is mandatory in
5  // the write direction; the driver also checks it on every word it reads.
6  #pragma once
7  #include <stdbool.h>
8  #include <stddef.h>
9  #include <stdint.h>
10
11 // CRC-8: polynomial 0x31, init 0xFF, no reflection, no final XOR.
12 // Datasheet test vector: CRC(0xBE, 0xEF) = 0x92.
13 uint8_t sgp30Crc8(const uint8_t* data, size_t len);
14
15 // out = {MSB, LSB, CRC}.
16 void sgp30PackWord(uint16_t word, uint8_t out[3]);
17
18 // Decode nWords 3-byte groups from buf into out[]. false on any CRC mismatch
19 // (out is then not fully written). An all-0xFF floating-bus read fails here.
20 bool sgp30UnpackWords(const uint8_t* buf, uint8_t nWords, uint16_t* out);
21
22 // g/m3 as the sensor's 8.8 fixed-point value, rounded and saturated to
23 // 0x0001..0xFFFF. 0 for a non-positive or NaN input (0 disables compensation).
24 uint16_t sgp30GramsToFixed8p8(float gramsPerM3);
25
26 // Absolute humidity (g/m3) from air temperature and relative humidity, using
27 // the datasheet's Magnus formula. Returns a negative value for invalid input.
28 float sgp30AbsoluteHumidity(float tempC, float rhPercent);
```

Return to guided explanation, p. 242

GUIDED SOURCE READING / CHAPTER 10

Use CRC before turning reply bytes into words

A successful I2C transfer proves bytes arrived. SGP30's word-level CRC supplies an additional integrity check before those bytes become gas or baseline values.

Original excerpt: src/domain/sgp30_codec.c (p. 248) / lines 22-29. The complete unchanged file follows.

```
22  bool sgp30UnpackWords(const uint8_t* buf, uint8_t nWords, uint16_t* out) {
23    for (uint8_t i = 0; i < nWords; ++i) {
24      const uint8_t* w = &buf[3 * i];
25      if (sgp30Crc8(w, 2) != w[2]) return false;
26      out[i] = (uint16_t)((w[0] << 8) | w[1]);
27    }
28    return true;
29  }
```

The word count determines the expected groups of three bytes: high byte, low byte and CRC. For each group, the helper recomputes the CRC over the two data bytes and compares it with the transmitted check byte.

Only a checked pair is combined into an output word. High-byte-first reconstruction differs from the little-endian DS18B20 temperature pair, so byte order belongs to the device protocol rather than the CPU's convenient representation.

If a later word fails, the helper returns false. Earlier output elements may already be filled, but the caller must reject the overall reply. A partially assigned array is not an accepted two-word observation.

The driver keeps that rule by branching on readWords before calling storeResults. This shows why checking a return value is part of data flow: integrity information must control whether conversion and publication happen, rather than merely produce a log beside an otherwise accepted value.

ORIGINAL SOURCE / CHAPTER 10 / 1 OF 1

src/domain/sgp30_codec.c

Original lines 1-46 | Code and comments unchanged; long lines wrap visually

sgp30Crc8 (L5-14): Check each SGP30 word using initial value 0xFF and polynomial 0x31.
sgp30PackWord (L16-20): Write high byte, low byte and the CRC calculated over those two bytes.

```
1  #include "domain/sgp30_codec.h"
2
3  #include <math.h>
4
5  uint8_t sgp30Crc8(const uint8_t* data, size_t len) {
6    uint8_t crc = 0xFF;
7    for (size_t i = 0; i < len; ++i) {
8      crc ^= data[i];
9      for (uint8_t bit = 0; bit < 8; ++bit) {
10       crc = (crc & 0x80) ? (uint8_t)((crc << 1) ^ 0x31) : (uint8_t)(crc << 1);
11     }
12   }
13   return crc;
14 }
15
16 void sgp30PackWord(uint16_t word, uint8_t out[3]) {
17   out[0] = (uint8_t)(word >> 8);
18   out[1] = (uint8_t)(word & 0xFF);
19   out[2] = sgp30Crc8(out, 2);
20 }
21
22 bool sgp30UnpackWords(const uint8_t* buf, uint8_t nWords, uint16_t* out) {
23   for (uint8_t i = 0; i < nWords; ++i) {
24     const uint8_t* w = &buf[3 * i];
25     if (sgp30Crc8(w, 2) != w[2]) return false;
26     out[i] = (uint16_t)((w[0] << 8) | w[1]);
27   }
28   return true;
29 }
30
31 uint16_t sgp30GramsToFixed8p8(float gramsPerM3) {
32   if (!(gramsPerM3 > 0.0f)) return 0;  // also catches NaN
33   const float scaled = gramsPerM3 * 256.0f + 0.5f;
34   if (scaled >= 65535.0f) return 0xFFFF;
35   if (scaled < 1.0f) return 0x0001;  // smallest value that keeps compensation on
36   return (uint16_t)scaled;
37 }
38
39 // dv = 216.7 * (RH/100 * 6.112 * exp(17.62 T / (243.12 + T))) / (273.15 + T)
40 float sgp30AbsoluteHumidity(float tempC, float rhPercent) {
41   if (!(rhPercent >= 0.0f && rhPercent <= 100.0f) || !(tempC > -40.0f && tempC < 85.0f)) {
42     return -1.0f;
43   }
44   const float saturation = 6.112f * expf(17.62f * tempC / (243.12f + tempC));  // hPa
45   return 216.7f * (rhPercent / 100.0f * saturation) / (273.15f + tempC);
46 }
```

Return to guided explanation, p. 247

ORIGINAL SOURCE / CHAPTER 10 / 1 OF 3

src/drivers/sgp30.h

Original lines 1-39 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
// sgp30 driver. Work package WP6 (Stage 2). Architecture §9, §10, §16 (D13, D18).
// Datasheet: Sensirion SGP30 v1.0, May 2020 (E13; local copy in
// Electrical System Study/Datasheets/Sensirion_Gas_Sensors_Datasheet_SGP30.pdf).
//
// Retained 3.3 V supply, shared I2C bus with the BME280 (address 0x58). The
// Adafruit breakout regulates the chip's 1.8 V and shifts the I2C levels.
// Produces Tvoc (ppb) and Co2eq (ppm, an estimate - never a CO2 measurement).
//
// init() checks the feature set (product type 0), sends iaq_init and restores
// a baseline if one is known. Then one measure_iaq per second from the
// absolute frame: start() sends the command, poll() reads 6 bytes once the
// 12 ms conversion is over and checks the CRC-8 of each word. The sensor NACKs
// while it converts, so it is read once, never polled.
//
// Health, highest priority first:
//   FAULT              CRC or bus error on this measurement (also while warming)
//   WARMING            first 15 s after iaq_init (fixed 400 ppm / 0 ppb)
//   DEPENDENCY_INVALID humidity compensation enabled but its input is stale
//   UNCALIBRATED       baseline still LEARNING (no restore, < 12 h) - D13
//   OK                 otherwise; kQualityUncompensated if no humidity input
//
// Unlike the BME280, a single fault does NOT force a re-init: re-init means a
// new iaq_init and 15 s of WARMING. CRC/bus errors and missed seconds (start()
// later than 1.8 s after the previous one) count as gaps; after 3 consecutive
// gaps start() returns Fault and the caller re-runs init(), which re-applies
// the last known baseline.
//
// Baseline persistence belongs to Configuration (WP8, NVS). This driver only
// keeps a RAM copy: it is fetched hourly once VALID or RESTORED, read with
// sgp30DriverGetBaseline() and restored at init from sgp30DriverSetBaseline().
//
// Worst-case blocking (I2C_BUS_TIMEOUT_MS = T per transaction):
//   init()    ~30 ms + 4 T  (feature set, iaq_init, set_iaq_baseline, each
//                            followed by its 10 ms maximum duration)
//   start()   1 T           (one command write)
//   poll()    2 T           (6-byte read + humidity write or baseline command);
//                            Pending until start+13 ms; once an hour Pending
//                            for 11 ms more to read the baseline (1 T)
//   others    no bus access
```

Return to guided explanation, p. 242

ORIGINAL SOURCE / CHAPTER 10 / 2 OF 3

src/drivers/sgp30.h

Original lines 40-77 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
40  //
41  // Implements the SensorDriver contract in drivers/sensor_driver.h.
42  #pragma once
43  #include "drivers/sensor_driver.h"
44  #include "hal/i2c_bus.h"
45
46  typedef struct Sgp30Config {
47    I2cBus* bus;
48    uint8_t address;  // BOARD_SGP30_I2C_ADDRESS
49  } Sgp30Config;
50
51  #define SGP30_METRICS 2  // results_ index: 0 TVOC, 1 CO2eq
52
53  // Baseline compensation state, reported separately from the readings (§10).
54  typedef uint8_t Sgp30BaselineStatus;
55  enum {
56    SGP30_BASELINE_NONE = 0,  // not initialised yet
57    SGP30_BASELINE_LEARNING,  // no baseline restored; learning for 12 h
58    SGP30_BASELINE_RESTORED,  // stored baseline applied after iaq_init
59    SGP30_BASELINE_VALID      // learned for 12 h without a restart
60  };
61
62  // Members after `base` are private to sgp30.c.
63  typedef struct Sgp30Driver {
64    SensorDriver base;  // must stay first
65    Sgp30Config config_;
66    bool initialised_;
67    uint8_t step_;             // measurement step (sgp30.c)
68    uint8_t consecutiveGaps_;
69    uint16_t featureSet_;
70    MonoMs startMs_;
71    MonoMs initMs_;
72    MonoMs lastStartMs_;
73    MonoMs baselineReadyMs_;   // when the baseline answer can be read
74    MonoMs baselineFetchDueMs_;
75    Sgp30BaselineStatus baselineStatus_;
76    MonoMs learningStartMs_;
77    bool hasBaseline_;         // baselineCo2eq_/baselineTvoc_ hold a usable baseline
```

Return to guided explanation, p. 242

ORIGINAL SOURCE / CHAPTER 10 / 3 OF 3

src/drivers/sgp30.h

Original lines 78-116 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
78    uint16_t baselineCo2eq_;
79    uint16_t baselineTvoc_;
80    bool hasPendingBaseline_;  // set by sgp30DriverSetBaseline(), applied at init
81    uint16_t pendingCo2eq_;
82    uint16_t pendingTvoc_;
83    bool humidityEnabled_;     // sgp30DriverSetHumidity() has been called
84    bool humidityValid_;       // its last input was fresh
85    bool humidityDirty_;       // humidity8p8_ not yet written to the chip
86    uint16_t humidity8p8_;     // g/m3 in 8.8 fixed point, 0 = compensation off
87    MonoMs humidityInputMs_;   // intervalEndMs of the older of the two inputs
88    Reading results_[SGP30_METRICS];
89  } Sgp30Driver;
90
91  // Constructor. Use &driver.base wherever a SensorDriver* is expected.
92  void sgp30DriverConstruct(Sgp30Driver* self, const Sgp30Config* config);
93
94  // Humidity compensation input from the latest BME280 temperature and humidity
95  // readings (copied, not shared). Both must be OK and fresh at `now`
96  // (CFG_STALE_BME280_MS); otherwise compensation is switched off on the chip
97  // (written as 0) and the readings report DependencyInvalid. The same happens
98  // if the caller stops calling this and the last input ages past that limit.
99  // Written after the next measurement (changes under ~0.03 g/m3 are ignored);
100 // no bus access here.
101 void sgp30DriverSetHumidity(Sgp30Driver* self, const Reading* airTemperature,
102                             const Reading* airHumidity, MonoMs now);
103
104 // Stored baseline (from NVS) to restore at the next init(). Values in the
105 // order get_iaq_baseline returns them.
106 void sgp30DriverSetBaseline(Sgp30Driver* self, uint16_t co2eq, uint16_t tvoc);
107
108 // Latest baseline worth storing. false while LEARNING (Sensirion: do not
109 // store a baseline before 12 h unless one was restored).
110 bool sgp30DriverGetBaseline(const Sgp30Driver* self, uint16_t* co2eq, uint16_t* tvoc);
111
112 Sgp30BaselineStatus sgp30DriverBaselineStatus(const Sgp30Driver* self);
113 const char* sgp30BaselineStatusName(Sgp30BaselineStatus s);
114
115 // Feature set word read in init() (0 until then), e.g. 0x0022.
116 uint16_t sgp30DriverFeatureSet(const Sgp30Driver* self);
```

Return to guided explanation, p. 242

GUIDED SOURCE READING / CHAPTER 10

A switch statement remembers which reply is expected

SGP30 has several commands with different waits. step is persistent driver state telling the next poll which operation is in flight.

Original excerpt: src/drivers/sgp30.c (p. 254) / lines 132-146. The complete unchanged file follows.

```
132      case STEP_MEASURING: {
133        if (now < self->startMs_ + kFirstReadMs) {
134          PollResult r = {DRIVER_PENDING, self->startMs_ + kFirstReadMs};
135          return r;
136        }
137        uint16_t words[2] = {0, 0};
138        if (!readWords(self, words, 2)) {
139          i2cBusRecover(self->config_.bus);
140          return fail(self, now);
141        }
142        // measure_iaq answers CO2eq first, then TVOC (datasheet §6.3).
143        storeResults(self, words[0], words[1], now);
144        return finish(self, now);
145      }
146
```

The case-specific deadline test comes before reading words. Before 13 ms have elapsed, it returns PENDING and the useful deadline. Once ready, readWords obtains two checked words; a failed transfer goes through recovery and fail rather than passing partly received data onward.

The measurement reply order is CO2eq then TVOC, so those array positions are named explicitly in storeResults. Baseline restore uses a different wire order, TVOC then CO2eq. Treating all two-word commands as interchangeable would swap meanings while retaining apparently valid integers and CRCs.

After storing the gas observation, finish may request one housekeeping command. The ordinary measurement result and the command-processing lifecycle are therefore distinct: a good sample can already exist while a baseline-read reply is pending.

This switch is the sensor equivalent of a saved program position. FreeRTOS saves a task's execution context; step saves which device operation a later call must resume. Neither requires sleeping in a loop until the chip is ready.

GUIDED SOURCE READING / CHAPTER 10

Housekeeping must wait until the measurement reply is consumed

The gas reply has been read. The bus and chip can now accept one maintenance operation before the next approximately one-second measurement.

Original excerpt: src/drivers/sgp30.c (p. 254) / lines 190-208. The complete unchanged file follows.

```
190    if ((self->baselineStatus_ == SGP30_BASELINE_VALID ||
191         self->baselineStatus_ == SGP30_BASELINE_RESTORED) &&
192        monoReached(now, self->baselineFetchDueMs_)) {
193      if (sendCommand(self, kCmdGetIaqBaseline)) {
194        self->step_ = STEP_BASELINE_READ;
195        self->baselineReadyMs_ = now + kCmdMaxMs + 1;
196        PollResult r = {DRIVER_PENDING, self->baselineReadyMs_};
197        return r;
198      }
199      self->baselineFetchDueMs_ = now + kBaselineFetchPeriodMs;
200    } else if (self->humidityDirty_) {
201      const uint16_t word = self->humidity8p8_;
202      if (sendCommandWords(self, kCmdSetAbsoluteHumidity, &word, 1)) {
203        self->humidityDirty_ = false;
204      }
205    }
206
207    PollResult r = {DRIVER_READY, now};
208    return r;
```

The baseline condition requires both a usable baseline status and an elapsed fetch deadline. After the get-baseline command succeeds, step changes to BASELINE_READ and baselineReadyMs records its processing allowance. PENDING returns control so the worker can service other sensors during that allowance.

The else-if gives a due baseline copy priority over a dirty humidity update. Sending both commands immediately would violate the requirement to let the first command finish. Humidity stays dirty and can be sent after a later measurement instead.

If the baseline command fails, the next fetch is delayed rather than discarding the gas observation already stored. The maintenance outcome does not retroactively make the successful measurement transfer fail.

Configuration persistence is another later step: acquisition offers an accepted RAM baseline and Configuration writes it at its rate limit. Keeping chip command timing, RAM state and NVS writes separate avoids putting flash latency into this driver. The full listing continues with humidity age checks and health selection, which determine how the stored gas estimate should be interpreted.

ORIGINAL SOURCE / CHAPTER 10 / 1 OF 8

src/drivers/sgp30.c

Original lines 1-49 | Code and comments unchanged; long lines wrap visually

asSgp30 (L37-37): Cast the common first-member SensorDriver pointer back to this concrete driver type.
sgp30Id (L47-47): Return this module's sensor identity; this does not access hardware.

```
#include "drivers/sgp30.h"

#include <Arduino.h>
#include <string.h>

#include "app_config.h"
#include "domain/sgp30_codec.h"
#include "hal/clock.h"

// Commands (datasheet Table 10).
static const uint16_t kCmdIaqInit = 0x2003;
static const uint16_t kCmdMeasureIaq = 0x2008;
static const uint16_t kCmdGetIaqBaseline = 0x2015;
static const uint16_t kCmdSetIaqBaseline = 0x201E;
static const uint16_t kCmdSetAbsoluteHumidity = 0x2061;
static const uint16_t kCmdGetFeatureSet = 0x202F;

static const uint16_t kProductTypeMask = 0xF000;  // Table 9: SGP30 = 0

// Timing. Driver-local because app_config.h is frozen.
static const uint32_t kCmdMaxMs = 10;           // iaq_init, baseline, humidity, feature set
static const uint32_t kFirstReadMs = 13;        // > measure_iaq max 12 ms, 1 ms clock margin
static const uint32_t kWarmupMs = 15000;        // fixed 400 ppm / 0 ppb after iaq_init
static const uint32_t kGapMs = 1800;            // start() later than this = missed second
static const uint8_t kMaxGaps = 3;              // consecutive gaps before a re-init
static const uint32_t kLearningMs = 12UL * 60 * 60 * 1000;  // no stored baseline
static const uint32_t kBaselineFetchPeriodMs = 60UL * 60 * 1000;
static const uint16_t kHumidityDeadband8p8 = 8;  // ~0.03 g/m3: ignore BME280 noise

enum {
  STEP_IDLE = 0,
  STEP_MEASURING,
  STEP_WRITE_FAILED,  // start() could not send the command; poll() reports it
  STEP_BASELINE_READ
};

static inline Sgp30Driver* asSgp30(SensorDriver* d) { return (Sgp30Driver*)d; }

static bool sendCommand(Sgp30Driver* self, uint16_t cmd);
static bool sendCommandWords(Sgp30Driver* self, uint16_t cmd, const uint16_t* words,
                             uint8_t nWords);
static bool readWords(Sgp30Driver* self, uint16_t* out, uint8_t nWords);
static PollResult fail(Sgp30Driver* self, MonoMs now);
static PollResult finish(Sgp30Driver* self, MonoMs now);
static void storeResults(Sgp30Driver* self, uint16_t co2eq, uint16_t tvoc, MonoMs now);

static SensorId sgp30Id(const SensorDriver* d) { (void)d; return SENSOR_SGP30; }
static const char* sgp30Name(const SensorDriver* d) { (void)d; return "sgp30"; }
static uint8_t sgp30MetricCount(const SensorDriver* d) { (void)d; return SGP30_METRICS; }
```

Return to guided explanation, p. 252

ORIGINAL SOURCE / CHAPTER 10 / 2 OF 8

src/drivers/sgp30.c

Original lines 50-98 | Code and comments unchanged; long lines wrap visually

sgp30Init (L51-102): Check dependencies, configure this module and return whether initialisation succeeded.

```
50
51  static DriverStatus sgp30Init(SensorDriver* d) {
52    Sgp30Driver* self = asSgp30(d);
53    self->initialised_ = false;
54    self->step_ = STEP_IDLE;
55    self->baselineStatus_ = SGP30_BASELINE_NONE;
56
57    // Identity: feature set, product type 0. measure_test is not used (it must
58    // not follow iaq_init and takes 220 ms), nor the General Call reset (it
59    // would reset every device on the shared bus).
60    uint16_t featureSet = 0;
61    if (!sendCommand(self, kCmdGetFeatureSet)) {
62      i2cBusRecover(self->config_.bus);
63      return DRIVER_FAULT;
64    }
65    delay(kCmdMaxMs + 1);  // delay(n) may return up to 1 tick early
66    if (!readWords(self, &featureSet, 1)) {
67      i2cBusRecover(self->config_.bus);
68      return DRIVER_FAULT;
69    }
70    if ((featureSet & kProductTypeMask) != 0) return DRIVER_NOT_PRESENT;
71    self->featureSet_ = featureSet;
72
73    if (!sendCommand(self, kCmdIaqInit)) return DRIVER_FAULT;
74    delay(kCmdMaxMs + 1);  // delay(n) may return up to 1 tick early
75    const MonoMs now = monoNowMs();
76
77    // A stored baseline from Configuration wins over the RAM copy.
78    if (self->hasPendingBaseline_) {
79      self->baselineCo2eq_ = self->pendingCo2eq_;
80      self->baselineTvoc_ = self->pendingTvoc_;
81      self->hasBaseline_ = true;
82      self->hasPendingBaseline_ = false;
83    }
84    if (self->hasBaseline_) {
85      // set_iaq_baseline takes TVOC first, then CO2eq (datasheet §6.3).
86      const uint16_t words[2] = {self->baselineTvoc_, self->baselineCo2eq_};
87      if (!sendCommandWords(self, kCmdSetIaqBaseline, words, 2)) return DRIVER_FAULT;
88      delay(kCmdMaxMs + 1);  // the harness may send measure_iaq right after init()
89      self->baselineStatus_ = SGP30_BASELINE_RESTORED;
90    } else {
91      self->baselineStatus_ = SGP30_BASELINE_LEARNING;
92      self->learningStartMs_ = now;
93    }
94
95    self->initMs_ = now;
96    self->lastStartMs_ = now;
97    self->baselineFetchDueMs_ = now + kBaselineFetchPeriodMs;
98    self->consecutiveGaps_ = 0;
```

Return to guided explanation, p. 252

ORIGINAL SOURCE / CHAPTER 10 / 3 OF 8

src/drivers/sgp30.c

Original lines 99-147 | Code and comments unchanged; long lines wrap visually

sgp30Init (L51-102): Check dependencies, configure this module and return whether initialisation succeeded.
sgp30Start (L104-126): Begin one operation and record its state; completion is handled later by poll.

```
  self->humidityDirty_ = self->humidityEnabled_;  // re-send after a restart
  self->initialised_ = true;
  return DRIVER_OK;
}

static DriverStatus sgp30Start(SensorDriver* d, MonoMs now) {
  Sgp30Driver* self = asSgp30(d);
  if (!self->initialised_) return DRIVER_FAULT;

  if (now - self->lastStartMs_ > kGapMs && self->consecutiveGaps_ < 0xFF) {
    ++self->consecutiveGaps_;
  }
  self->lastStartMs_ = now;
  if (self->consecutiveGaps_ >= kMaxGaps) {
    self->initialised_ = false;  // caller re-runs init(): iaq_init + baseline
    return DRIVER_FAULT;
  }

  self->startMs_ = now;
  if (sendCommand(self, kCmdMeasureIaq)) {
    self->step_ = STEP_MEASURING;
  } else {
    // Reported by poll() as one gap, not as a re-init (that would cost 15 s).
    i2cBusRecover(self->config_.bus);
    self->step_ = STEP_WRITE_FAILED;
  }
  return DRIVER_OK;
}

static PollResult sgp30Poll(SensorDriver* d, MonoMs now) {
  Sgp30Driver* self = asSgp30(d);

  switch (self->step_) {
    case STEP_MEASURING: {
      if (now < self->startMs_ + kFirstReadMs) {
        PollResult r = {DRIVER_PENDING, self->startMs_ + kFirstReadMs};
        return r;
      }
      uint16_t words[2] = {0, 0};
      if (!readWords(self, words, 2)) {
        i2cBusRecover(self->config_.bus);
        return fail(self, now);
      }
      // measure_iaq answers CO2eq first, then TVOC (datasheet §6.3).
      storeResults(self, words[0], words[1], now);
      return finish(self, now);
    }

    case STEP_BASELINE_READ: {
```

Return to guided explanation, p. 252

ORIGINAL SOURCE / CHAPTER 10 / 4 OF 8

src/drivers/sgp30.c

Original lines 148-196 | Code and comments unchanged; long lines wrap visually

sgp30Poll (L128-175): Advance an in-flight operation only when useful; return PENDING with a future time, READY, or failure.
finish (L181-209): Treat step_ as the next operation to perform. Measurement, baseline and humidity commands have different waits.

```
      if (now < self->baselineReadyMs_) {
        PollResult r = {DRIVER_PENDING, self->baselineReadyMs_};
        return r;
      }
      // get_iaq_baseline answers CO2eq first, then TVOC. A failure only
      // delays the next copy: this second's readings are already stored.
      uint16_t words[2] = {0, 0};
      if (readWords(self, words, 2)) {
        self->baselineCo2eq_ = words[0];
        self->baselineTvoc_ = words[1];
        self->hasBaseline_ = true;
      }
      self->baselineFetchDueMs_ = now + kBaselineFetchPeriodMs;
      self->step_ = STEP_IDLE;
      PollResult r = {DRIVER_READY, now};
      return r;
    }

    case STEP_WRITE_FAILED:
      return fail(self, now);

    default: {
      // poll() without a start(): nothing in flight.
      PollResult r = {DRIVER_FAULT, now};
      return r;
    }
  }
}

// After a good measurement the bus is free and the next measure_iaq is ~1 s
// away: do at most one housekeeping command, so the chip is never asked for a
// second command while it is still processing the first (10 ms each). The
// hourly baseline copy goes first; a pending humidity write waits 1 s.
static PollResult finish(Sgp30Driver* self, MonoMs now) {
  self->step_ = STEP_IDLE;

  if (self->baselineStatus_ == SGP30_BASELINE_LEARNING &&
      now - self->learningStartMs_ >= kLearningMs) {
    self->baselineStatus_ = SGP30_BASELINE_VALID;
    self->baselineFetchDueMs_ = now;  // first copy right away
  }

  if ((self->baselineStatus_ == SGP30_BASELINE_VALID ||
       self->baselineStatus_ == SGP30_BASELINE_RESTORED) &&
      monoReached(now, self->baselineFetchDueMs_)) {
    if (sendCommand(self, kCmdGetIaqBaseline)) {
      self->step_ = STEP_BASELINE_READ;
      self->baselineReadyMs_ = now + kCmdMaxMs + 1;
      PollResult r = {DRIVER_PENDING, self->baselineReadyMs_};
```

Return to guided explanation, p. 252

ORIGINAL SOURCE / CHAPTER 10 / 5 OF 8

src/drivers/sgp30.c

Original lines 197-245 | Code and comments unchanged; long lines wrap visually

finish (L181-209): Treat step_ as the next operation to perform. Measurement, baseline and humidity commands have different waits.
storeResults (L211-253): Store gas reply values and choose WARMING, DEPENDENCY_INVALID, UNCALIBRATED or OK from time and dependencies.

```
      return r;
    }
    self->baselineFetchDueMs_ = now + kBaselineFetchPeriodMs;
  } else if (self->humidityDirty_) {
    const uint16_t word = self->humidity8p8_;
    if (sendCommandWords(self, kCmdSetAbsoluteHumidity, &word, 1)) {
      self->humidityDirty_ = false;
    }
  }

  PollResult r = {DRIVER_READY, now};
  return r;
}

static void storeResults(Sgp30Driver* self, uint16_t co2eq, uint16_t tvoc, MonoMs now) {
  self->consecutiveGaps_ = 0;

  // The caller stopped refreshing the humidity input: switch compensation off
  // on the chip (written at the next finish()) instead of keeping it silently.
  if (self->humidityEnabled_ && self->humidityValid_ &&
      now - self->humidityInputMs_ > CFG_STALE_BME280_MS) {
    self->humidityValid_ = false;
    self->humidity8p8_ = 0;
    self->humidityDirty_ = true;
  }

  Health health = HEALTH_OK;
  if (now - self->initMs_ < kWarmupMs) {
    health = HEALTH_WARMING;  // by time, never by the 400/0 values
  } else if (self->humidityEnabled_ && !self->humidityValid_) {
    health = HEALTH_DEPENDENCY_INVALID;
  } else if (self->baselineStatus_ == SGP30_BASELINE_LEARNING) {
    health = HEALTH_UNCALIBRATED;
  }
  const uint16_t quality = (self->humidityEnabled_ && self->humidityValid_)
                               ? kQualityNone
                               : kQualityUncompensated;

  const uint16_t raws[SGP30_METRICS] = {tvoc, co2eq};
  for (uint8_t i = 0; i < SGP30_METRICS; ++i) {
    Reading* r = &self->results_[i];
    r->intervalStartMs = self->startMs_;
    r->intervalEndMs = now;
    r->hasRaw = true;
    r->raw = raws[i];
    r->qualityFlags = quality;
    r->health = health;
    if (health == HEALTH_OK || health == HEALTH_UNCALIBRATED ||
        health == HEALTH_DEPENDENCY_INVALID) {
```

Return to guided explanation, p. 253

ORIGINAL SOURCE / CHAPTER 10 / 6 OF 8

src/drivers/sgp30.c

Original lines 246-294 | Code and comments unchanged; long lines wrap visually

storeResults (L211-253): Store gas reply values and choose WARMING, DEPENDENCY_INVALID, UNCALIBRATED or OK from time and dependencies.
fail (L257-267): End the failed operation, stamp a fault verdict and preserve earlier numerical history as non-actionable.

```
246       r->value = (float)raws[i];
247     }
248     if (health == HEALTH_OK || health == HEALTH_UNCALIBRATED) {
249       r->lastGoodMs = now;
250       ++r->sequence;
251     }
252   }
253 }
254
255 // One rejected measurement: count the gap and mark both metrics FAULT. The
256 // driver stays initialised; start() re-inits only after kMaxGaps in a row.
257 static PollResult fail(Sgp30Driver* self, MonoMs now) {
258   self->step_ = STEP_IDLE;
259   if (self->consecutiveGaps_ < 0xFF) ++self->consecutiveGaps_;
260   for (uint8_t i = 0; i < SGP30_METRICS; ++i) {
261     self->results_[i].health = HEALTH_FAULT;
262     self->results_[i].intervalStartMs = self->startMs_;
263     self->results_[i].intervalEndMs = now;
264   }
265   PollResult r = {DRIVER_FAULT, now};
266   return r;
267 }
268
269 static bool sendCommand(Sgp30Driver* self, uint16_t cmd) {
270   const uint8_t tx[2] = {(uint8_t)(cmd >> 8), (uint8_t)(cmd & 0xFF)};
271   return i2cBusWrite(self->config_.bus, self->config_.address, tx, sizeof(tx));
272 }
273
274 // Command followed by nWords (max 2) parameter words, each with its CRC.
275 static bool sendCommandWords(Sgp30Driver* self, uint16_t cmd, const uint16_t* words,
276                              uint8_t nWords) {
277   uint8_t tx[2 + 2 * 3];
278   if (nWords > 2) return false;
279   tx[0] = (uint8_t)(cmd >> 8);
280   tx[1] = (uint8_t)(cmd & 0xFF);
281   for (uint8_t i = 0; i < nWords; ++i) sgp30PackWord(words[i], &tx[2 + 3 * i]);
282   return i2cBusWrite(self->config_.bus, self->config_.address, tx, 2 + 3 * (size_t)nWords);
283 }
284
285 static bool readWords(Sgp30Driver* self, uint16_t* out, uint8_t nWords) {
286   uint8_t rx[2 * 3];
287   if (nWords > 2) return false;
288   const size_t len = 3 * (size_t)nWords;
289   return i2cBusRead(self->config_.bus, self->config_.address, rx, len) &&
290          sgp30UnpackWords(rx, nWords, out);
291 }
292
293 static uint8_t sgp30ReadResult(SensorDriver* d, Reading* out, uint8_t maxOut) {
294   Sgp30Driver* self = asSgp30(d);
```

Return to guided explanation, p. 253

ORIGINAL SOURCE / CHAPTER 10 / 7 OF 8

src/drivers/sgp30.c

Original lines 295-343 | Code and comments unchanged; long lines wrap visually

sgp30ReadResult (L293-299): Copy the latest Reading record(s) into caller storage. Inspect health even when poll returned READY.
sgp30Cancel (L302-302): Stop software tracking of partial work. A chip conversion can still finish internally.

```
295   if (out == NULL) return 0;
296   const uint8_t n = maxOut < SGP30_METRICS ? maxOut : SGP30_METRICS;
297   for (uint8_t i = 0; i < n; ++i) out[i] = self->results_[i];
298   return n;
299 }
300
301 // A started measurement finishes by itself inside the chip.
302 static void sgp30Cancel(SensorDriver* d) { asSgp30(d)->step_ = STEP_IDLE; }
303
304 // The 3.3 V rail stays up in low power (§3); just require a fresh init(),
305 // which re-applies the RAM baseline.
306 static void sgp30PreparePowerOff(SensorDriver* d) {
307   Sgp30Driver* self = asSgp30(d);
308   self->step_ = STEP_IDLE;
309   self->initialised_ = false;
310 }
311
312 static const SensorDriverOps kSgp30Ops = {
313     .id = sgp30Id,
314     .name = sgp30Name,
315     .metricCount = sgp30MetricCount,
316     .init = sgp30Init,
317     .start = sgp30Start,
318     .poll = sgp30Poll,
319     .readResult = sgp30ReadResult,
320     .cancel = sgp30Cancel,
321     .preparePowerOff = sgp30PreparePowerOff,
322 };
323
324 void sgp30DriverSetHumidity(Sgp30Driver* self, const Reading* airTemperature,
325                             const Reading* airHumidity, MonoMs now) {
326   self->humidityEnabled_ = true;
327   float grams = -1.0f;
328   if (readingIsActionableAt(airTemperature, now, CFG_STALE_BME280_MS) &&
329       readingIsActionableAt(airHumidity, now, CFG_STALE_BME280_MS)) {
330     grams = sgp30AbsoluteHumidity(airTemperature->value, airHumidity->value);
331   }
332   // Stale or invalid input: write 0, which switches compensation off, rather
333   // than let the chip keep an old value silently (§10).
334   const uint16_t value = grams > 0.0f ? sgp30GramsToFixed8p8(grams) : 0;
335   self->humidityValid_ = value != 0;
336   if (self->humidityValid_) {
337     const MonoMs t = airTemperature->intervalEndMs;
338     const MonoMs h = airHumidity->intervalEndMs;
339     self->humidityInputMs_ = t < h ? t : h;  // age of the older input
340   }
341   const uint16_t old = self->humidity8p8_;
342   const uint16_t diff = value > old ? (uint16_t)(value - old) : (uint16_t)(old - value);
343   if ((value == 0) != (old == 0) || diff >= kHumidityDeadband8p8) {
```

Return to guided explanation, p. 253

ORIGINAL SOURCE / CHAPTER 10 / 8 OF 8

src/drivers/sgp30.c

Original lines 344-392 | Code and comments unchanged; long lines wrap visually

sgp30DriverSetHumidity (L324-347): Validate the copied air inputs and stage an absolute-humidity command; no bus access occurs in this setter.
sgp30DriverSetBaseline (L349-353): Stage baseline words for the next driver initialisation.

```
    self->humidity8p8_ = value;
    self->humidityDirty_ = true;
  }
}

void sgp30DriverSetBaseline(Sgp30Driver* self, uint16_t co2eq, uint16_t tvoc) {
  self->pendingCo2eq_ = co2eq;
  self->pendingTvoc_ = tvoc;
  self->hasPendingBaseline_ = true;
}

bool sgp30DriverGetBaseline(const Sgp30Driver* self, uint16_t* co2eq, uint16_t* tvoc) {
  const bool usable = self->hasBaseline_ &&
                      (self->baselineStatus_ == SGP30_BASELINE_VALID ||
                       self->baselineStatus_ == SGP30_BASELINE_RESTORED);
  if (usable) {
    if (co2eq != NULL) *co2eq = self->baselineCo2eq_;
    if (tvoc != NULL) *tvoc = self->baselineTvoc_;
  }
  return usable;
}

Sgp30BaselineStatus sgp30DriverBaselineStatus(const Sgp30Driver* self) {
  return self->baselineStatus_;
}

const char* sgp30BaselineStatusName(Sgp30BaselineStatus s) {
  switch (s) {
    case SGP30_BASELINE_NONE: return "NONE";
    case SGP30_BASELINE_LEARNING: return "LEARNING";
    case SGP30_BASELINE_RESTORED: return "RESTORED";
    case SGP30_BASELINE_VALID: return "VALID";
    default: return "?";
  }
}

uint16_t sgp30DriverFeatureSet(const Sgp30Driver* self) { return self->featureSet_; }

// Readings, baseline and humidity state are reset once here so they survive
// re-inits.
void sgp30DriverConstruct(Sgp30Driver* self, const Sgp30Config* config) {
  memset(self, 0, sizeof(*self));
  self->base.ops = &kSgp30Ops;
  self->config_ = *config;
  self->step_ = STEP_IDLE;
  self->baselineStatus_ = SGP30_BASELINE_NONE;
  readingReset(&self->results_[0], SENSOR_SGP30, METRIC_TVOC, UNIT_PPB);
  readingReset(&self->results_[1], SENSOR_SGP30, METRIC_CO2EQ, UNIT_PPM);
}
```

Return to guided explanation, p. 253

ORIGINAL SOURCE / CHAPTER 10 / 1 OF 2

test/bringup/sgp30_main.c

Original lines 1-35 | Code and comments unchanged; long lines wrap visually

printBaseline (L33-49): Present already-collected diagnostic information; this does not make the measurement more valid.

```
1  // Bring-up entry point for sgp30 (WP6).  pio run -e test_sgp30 -t upload -t monitor
2  // Retained 3.3 V supply, shared I2C bus. Expect 2 readings every 1 s, ~13 ms
3  // each. The first 15 s after init are WARMING (fixed 400 ppm / 0 ppb), never
4  // published. Without a restored baseline the values then print "(uncal)"
5  // (baseline LEARNING, D13) for 12 h; after that the baseline is VALID, the
6  // values turn OK and the baseline words are printed once an hour.
7  // Humidity compensation is not used here (one driver per harness), so OK
8  // values are uncompensated.
9  // Replaces src/main.c in this env only; links the production driver.
10 #include <Arduino.h>
11 #include <stdio.h>
12
13 #include "app_config.h"
14 #include "board_profile.h"
15 #include "bringup_harness.h"
16 #include "hal/i2c_bus.h"
17 #include "drivers/sgp30.h"
18
19 // Baseline to restore at init, as printed by an earlier run ("baseline co2eq
20 // 0x.... tvoc 0x...."). Both 0 = none: the sensor learns from scratch.
21 // Sensirion: a stored baseline is valid for up to 7 days with the sensor off.
22 #define SGP30_RESTORE_CO2EQ 0x0000
23 #define SGP30_RESTORE_TVOC 0x0000
24
25 static I2cBus s_i2c = I2C_BUS_INIT;
26 static Sgp30Config s_config = {&s_i2c, BOARD_SGP30_I2C_ADDRESS};
27 static Sgp30Driver s_driver;
28
29 static Sgp30BaselineStatus s_lastStatus = SGP30_BASELINE_NONE;
30 static uint16_t s_lastCo2eq = 0;
31 static uint16_t s_lastTvoc = 0;
32
33 static void printBaseline(SensorDriver* d) {
34   (void)d;
35   const Sgp30BaselineStatus st = sgp30DriverBaselineStatus(&s_driver);
```

Return to guided explanation, p. 242

ORIGINAL SOURCE / CHAPTER 10 / 2 OF 2

test/bringup/sgp30_main.c

Original lines 36-71 | Code and comments unchanged; long lines wrap visually

printBaseline (L33-49): Present already-collected diagnostic information; this does not make the measurement more valid.
setup (L53-69): Establish safe outputs, logger, gate, board check and queues; load configuration; start Safety, both acquisition workers, power, radios and supervision.

```
36    if (st != s_lastStatus) {
37      printf("[sgp30] baseline %s (feature set 0x%04X)\n", sgp30BaselineStatusName(st),
38             (unsigned)sgp30DriverFeatureSet(&s_driver));
39      s_lastStatus = st;
40    }
41    uint16_t co2eq = 0;
42    uint16_t tvoc = 0;
43    if (sgp30DriverGetBaseline(&s_driver, &co2eq, &tvoc) &&
44        (co2eq != s_lastCo2eq || tvoc != s_lastTvoc)) {
45      printf("[sgp30] baseline co2eq 0x%04X tvoc 0x%04X\n", (unsigned)co2eq, (unsigned)tvoc);
46      s_lastCo2eq = co2eq;
47      s_lastTvoc = tvoc;
48    }
49  }
50
51  static BringupContext s_ctx = {&s_driver.base, CFG_FRAME_PERIOD_MS, printBaseline};
52
53  void setup(void) {
54    sgp30DriverConstruct(&s_driver, &s_config);
55    if (SGP30_RESTORE_CO2EQ != 0 || SGP30_RESTORE_TVOC != 0) {
56      sgp30DriverSetBaseline(&s_driver, SGP30_RESTORE_CO2EQ, SGP30_RESTORE_TVOC);
57    }
58    delay(200);
59    printf("\n=== bring-up: %s (SDA %d, SCL %d) ===\n", sensorDriverName(&s_driver.base),
60           BOARD_PIN_I2C_SDA, BOARD_PIN_I2C_SCL);
61    i2cBusBegin(&s_i2c, BOARD_PIN_I2C_SDA, BOARD_PIN_I2C_SCL, BOARD_I2C_CLOCK_HZ);
62    printf("probe 0x%02X -> %s\n", BOARD_SGP30_I2C_ADDRESS,
63           i2cBusProbe(&s_i2c, BOARD_SGP30_I2C_ADDRESS) ? "ACK" : "-");
64    if (SGP30_RESTORE_CO2EQ != 0 || SGP30_RESTORE_TVOC != 0) {
65      printf("restoring baseline co2eq 0x%04X tvoc 0x%04X\n", (unsigned)SGP30_RESTORE_CO2EQ,
66             (unsigned)SGP30_RESTORE_TVOC);
67    }
68    bringupStart(&s_ctx);
69  }
70
71  void loop(void) { vTaskDelay(portMAX_DELAY); }
```

Return to guided explanation, p. 242

PART II / SENSOR ARCHITECTURES

CHAPTER 11

HC-SR04: Ultrasonic Distance

HC-SR04 measures elapsed time between two echo edges. An interrupt records those edges; the task later notices completion and converts their difference to distance. Separating capture time from poll time explains both the precision and the scheduling constraints.

In this chapter

1. Follow arming, trigger, rising edge and falling edge.
2. Read completion-before-timeout ordering.
3. Convert pulse width to range and identify the tank-geometry boundary.

Chapter contents

11.1 HC-SR04: time the edges, then do the arithmetic 265
11.2 HC-SR04: measure a pulse, then convert distance 266
11.3 HC-SR04: completion and timeout are different outcomes 267

Guided source readings begin on page 270.

Complete original listings begin on page 268. Shared files are linked in the explanations; Appendix A contains the full filename index.

CHAPTER 11

11.1 HC-SR04: time the edges, then do the arithmetic

This sensor's useful information is the width of its ECHO pulse. The interrupt handler records the edges quickly. The normal task later converts their time difference into distance.

Capture state	Meaning
IDLE	Ignore edges; no measurement is armed.
WAITING	Trigger sent; waiting for ECHO to rise.
HIGH	Rising edge recorded; waiting for the fall.
DONE	Both timestamps exist; pulse width is available.

At 343 m/s, a 2000 us round-trip pulse means 343 mm one-way distance. Polling every millisecond only controls how quickly the task notices DONE; the width uses interrupt timestamps.

Why the ISR stays small

It records time and changes state. It does not print, perform floating-point conversion or decide an alarm. This reduces delay to other interrupts and leaves the interpretation visible in the driver.

A missing rising edge gives TIMEOUT without raw width. A rise without a fall gives TIMEOUT with diagnostic elapsed microseconds. A complete pulse can still be FAULT if the calculated distance is outside 20-4000 mm.

The full capture HAL, driver, header, conversion helper and individual test are in this chapter. Distance is not yet water volume or percentage; that would need tank geometry. The fixed sound speed is not compensated for air temperature.

CHAPTER 11

11.2 HC-SR04: measure a pulse, then convert distance

HC-SR04 uses external 5V, TRIG GPIO17 and ECHO GPIO18 through a 1 kOhm / 2 kOhm divider. The trigger is at least 10 us HIGH. The ECHO pulse width represents a round trip of sound; distance is half the travel path.

```
// Source: src/domain/ultrasonic.c
return (float)widthUs * speedMps / 2000.0f;
```

With width 2000 us and nominal speed 343 m/s, distance is 343 mm. The divisor combines the round-trip factor 2 and microseconds-to-millimetres scaling. The driver accepts 20-4000 mm and marks its accepted distances uncompensated because sound speed is fixed.

pulse_capture.c separates edge timing from polling

`pulseCaptureBegin()` configures pins and installs a CHANGE interrupt. Trigger refuses a still-HIGH ECHO, clears old timestamps, arms WAITING and creates the pulse. The ISR records rising time and selects HIGH, then records falling time and selects DONE.

`pulseCapturePoll()` reads the state and, once DONE, returns fall minus rise in microseconds. Polling every millisecond affects how soon software notices completion, not the width's nominal time resolution. The width comes from the edge ISR.

The ISR reads GPIO registers directly and carries `IRAM_ATTR` to place its code in instruction RAM. Its entire call path and accessed storage must remain suitable for ISR execution. The code avoids printf, queues, floating-point conversion and policy work inside this handler.

`pulseCaptureElapsedUs()` gives diagnostic time since trigger. EchoHigh checks the physical input. Cancel disarms; SetSafe also leaves TRIG LOW and ECHO as an input without pull-ups, reducing back-feed risk when the module is unpowered.

Original source in this PDF: src/hal/pulse_capture.c (p. 271); src/drivers/hcsr04.c (p. 278); src/domain/ultrasonic.c (p. 274)

CHAPTER 11

11.3 HC-SR04: completion and timeout are different outcomes

`hcsr04Init()` requires a capture object and nonzero timeout, cancels old state and checks that ECHO is LOW. `hcsr04Start()` records time, triggers and marks busy. A stuck-HIGH ECHO before trigger becomes FAULT and invalidates initialisation.

hcsr04Poll()

DONE is handled first, including on a late poll, because the ISR already captured the width. The domain conversion then yields OK or FAULT depending on the distance range. If capture is unexpectedly neither WAITING nor HIGH, it is cancelled and reported FAULT.

Before the 30 ms deadline, poll requests another check in 1 ms, capped at the deadline. At timeout it cancels first, so a late falling edge cannot complete the next ping accidentally.

Timeout case	Diagnostic Reading
ECHO never rose	TIMEOUT, no raw pulse value.
ECHO rose but did not fall	TIMEOUT with raw elapsed microseconds.
Complete pulse outside range	FAULT with raw width.
Complete pulse inside range	OK distance in mm, uncompensated flag.

`finish()` preserves old numeric value for non-OK outcomes and always stamps the new outcome's health and interval. This avoids presenting timeout as a genuine zero distance. `hcsr04ReadResult()` copies the record, cancel drops pending work, and preparePowerOff also makes the pins safe and requires re-init.

Distance is not automatically water level

The metric is water-surface distance. Converting that into fill height requires a measured sensor-to-bottom geometry; volume may require the tank shape. That conversion is not implemented here. A fill-height or volume display therefore needs an explicit geometric conversion after distance measurement.

The capture state and timestamps use volatile fields for ISR/task visibility. That is a specialised same-core ownership assumption, not a general replacement for locks. Changes to core placement or a multi-core capture consumer need a fresh synchronisation review.

Original source in this PDF: src/drivers/hcsr04.c (p. 278); src/drivers/hcsr04.h (p. 275); src/hal/pulse_capture.h (p. 268)

ORIGINAL SOURCE / CHAPTER 11 / 1 OF 2

src/hal/pulse_capture.h

Original lines 1-31 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // Echo-pulse capture (HAL). Work package WP4 (with HC-SR04).
2  // Use RMT or an audited edge-capture ISR - not pulseIn() (§2, §7).
3  //
4  // Implementation: CHANGE interrupt on ECHO + esp_timer microsecond timestamps
5  // (decision WP4, Oct 2026). The ISR only records the two edges; the width is
6  // read by pulseCapturePoll() from task context. Timestamps are the low 32 bits
7  // of esp_timer_get_time(), so the ISR -> task handoff is single-word writes
8  // (no lock) and widths are a modulo-2^32 subtraction (valid for ~71 min).
9  // Accuracy does not depend on how often poll() is called. ISR latency jitter
10 // is a few us (< 1 mm of distance).
11 //
12 // The timeout policy belongs to the caller (driver budget): this HAL never
13 // gives up by itself. Cancel after a timeout so a late falling edge is never
14 // credited to the next trigger.
15 #pragma once
16 #include <stdbool.h>
17 #include <stdint.h>
18
19 typedef uint8_t CaptureState;
20 enum {
21   CAPTURE_IDLE = 0,  // not armed (begun, cancelled, or nothing triggered)
22   CAPTURE_WAITING,   // triggered, ECHO has not risen yet
23   CAPTURE_DONE,      // rising and falling edge seen, width available
24   CAPTURE_TIMEOUT,   // unused by this HAL (timeouts are the driver's), kept for callers
25   CAPTURE_ERROR,     // not begun
26   CAPTURE_HIGH       // ECHO rose, falling edge not seen yet
27 };
28
29 // Treat the members as private to pulse_capture.c. Initialise with PULSE_CAPTURE_INIT.
30 typedef struct PulseCapture {
31   int8_t trig_;
```

Return to guided explanation, p. 265

ORIGINAL SOURCE / CHAPTER 11 / 2 OF 2

src/hal/pulse_capture.h

Original lines 32-62 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
  int8_t echo_;
  bool begun_;
  volatile CaptureState state_;  // written last by the ISR
  volatile uint32_t riseUs_;
  volatile uint32_t fallUs_;
  uint32_t trigUs_;
} PulseCapture;

#define PULSE_CAPTURE_INIT {-1, -1, false, CAPTURE_IDLE, 0, 0, 0}

// TRIG output low, ECHO input with NO pull (a pull-up would back-feed the
// unpowered 5 V module through the divider), CHANGE interrupt attached.
// false if a pin is invalid.
bool pulseCaptureBegin(PulseCapture* pc, int8_t trigPin, int8_t echoPin);

// 10 us trigger pulse, arm capture. Busy-waits ~12 us (2 us low + 10 us high).
// false (nothing sent) if not begun or ECHO is already high: a stuck line or a
// late echo from the previous ping.
bool pulseCaptureTrigger(PulseCapture* pc);

// Non-blocking. *widthUs = pulse width when DONE, else 0.
CaptureState pulseCapturePoll(PulseCapture* pc, uint32_t* widthUs);

// Microseconds since the last trigger (0 if never triggered).
uint32_t pulseCaptureElapsedUs(const PulseCapture* pc);

// Current ECHO level (false if not begun).
bool pulseCaptureEchoHigh(const PulseCapture* pc);

void pulseCaptureCancel(PulseCapture* pc);                    // disarm, back to IDLE
void pulseCaptureSetSafe(PulseCapture* pc);                    // TRIG low, ECHO input, no
>
pulls (§12)
```

Return to guided explanation, p. 265

GUIDED SOURCE READING / CHAPTER 11

Arm capture before generating the trigger edge

Ultrasonic distance depends on edge timing. The capture HAL prepares a fresh state before emitting the electrical trigger, so an early echo cannot belong to old data.

Original excerpt: src/hal/pulse_capture.c (p. 271) / lines 57-71. The complete unchanged file follows.

```
57  bool pulseCaptureTrigger(PulseCapture* pc) {
58    if (!pc->begun_) return false;
59    pc->state_ = CAPTURE_IDLE;  // ignore edges while re-arming
60    if (echoLevel(pc->echo_)) return false;
61    pc->riseUs_ = 0;
62    pc->fallUs_ = 0;
63    digitalWrite((uint8_t)pc->trig_, LOW);
64    delayMicroseconds(2);
65    pc->trigUs_ = nowUs();
66    pc->state_ = CAPTURE_WAITING;
67    digitalWrite((uint8_t)pc->trig_, HIGH);
68    delayMicroseconds(10);  // datasheet: >= 10 us TTL pulse
69    digitalWrite((uint8_t)pc->trig_, LOW);
70    return true;
71  }
```

The begun guard requires configured pins. CAPTURE_IDLE first tells the ISR to ignore edges while the old timestamps are being replaced. The ECHO-high check rejects an already-active signal rather than starting a second ambiguous measurement.

riseUs and fallUs are cleared, TRIG is held LOW briefly, and trigUs records the current trigger reference. CAPTURE_WAITING is assigned before TRIG goes HIGH. Reversing those two statements could permit a fast rising ECHO while the ISR still believes no capture is armed.

The 10 us trigger pulse is short CPU-occupied timing. The potentially much longer echo wait is handled through ISR state and later polls. Those two kinds of waiting have different scheduling costs even though both belong to one sensor reading.

onEchoEdge stores riseUs before HIGH and fallUs before DONE. State therefore announces that the corresponding timestamp has been written. Poll reads the state before subtracting the two timestamps. AcqNormal installs the GPIO ISR service on core1, and the lease confines HAL calls to one worker. Target validation checks interrupt timing and visibility under the complete running system.

ORIGINAL SOURCE / CHAPTER 11 / 1 OF 2

src/hal/pulse_capture.c

Original lines 1-50 | Code and comments unchanged; long lines wrap visually

nowUs (L12-12): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.
echoLevel (L15-18): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.

```
#include "hal/pulse_capture.h"

#include <Arduino.h>
#include <esp_attr.h>
#include <esp_timer.h>
#include <soc/gpio_reg.h>
#include <stddef.h>

// ESP32-S3 GPIO numbers run 0-48.
static const int8_t kMaxGpio = 48;

static inline uint32_t nowUs(void) { return (uint32_t)esp_timer_get_time(); }

// Direct register read: IRAM-safe, unlike digitalRead().
static inline IRAM_ATTR bool echoLevel(int8_t pin) {
  if (pin < 32) return (REG_READ(GPIO_IN_REG) >> pin) & 1u;
  return (REG_READ(GPIO_IN1_REG) >> (pin - 32)) & 1u;
}

// Only records edges while armed. Rising edge -> HIGH; falling edge while HIGH
// -> DONE. The state is written after the timestamp, so a DONE seen by poll()
// always has a valid fallUs_.
static void IRAM_ATTR onEchoEdge(void* arg) {
  PulseCapture* pc = (PulseCapture*)arg;
  const uint32_t t = nowUs();
  const CaptureState s = pc->state_;
  if (s != CAPTURE_WAITING && s != CAPTURE_HIGH) return;
  if (echoLevel(pc->echo_)) {
    if (s == CAPTURE_WAITING) {
      pc->riseUs_ = t;
      pc->state_ = CAPTURE_HIGH;
    }
  } else if (s == CAPTURE_HIGH) {
    pc->fallUs_ = t;
    pc->state_ = CAPTURE_DONE;
  }
}

bool pulseCaptureBegin(PulseCapture* pc, int8_t trigPin, int8_t echoPin) {
  pc->begun_ = false;
  pc->state_ = CAPTURE_IDLE;
  pc->trig_ = trigPin;
  pc->echo_ = echoPin;
  if (trigPin < 0 || trigPin > kMaxGpio || echoPin < 0 || echoPin > kMaxGpio ||
      trigPin == echoPin) {
    return false;
  }
  digitalWrite((uint8_t)trigPin, LOW);
  pinMode((uint8_t)trigPin, OUTPUT);
  digitalWrite((uint8_t)trigPin, LOW);
```

Return to guided explanation, p. 270

ORIGINAL SOURCE / CHAPTER 11 / 2 OF 2

src/hal/pulse_capture.c

Original lines 51-101 | Code and comments unchanged; long lines wrap visually

pulseCaptureBegin (L39-55): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.
pulseCaptureTrigger (L57-71): Reject a high ECHO, clear old timestamps, arm capture and send a short TRIG pulse.

```
51    pinMode((uint8_t)echoPin, INPUT);  // no pull: no back-feeding the 5 V module
52    attachInterruptArg((uint8_t)echoPin, onEchoEdge, pc, CHANGE);
53    pc->begun_ = true;
54    return true;
55  }
56
57  bool pulseCaptureTrigger(PulseCapture* pc) {
58    if (!pc->begun_) return false;
59    pc->state_ = CAPTURE_IDLE;  // ignore edges while re-arming
60    if (echoLevel(pc->echo_)) return false;
61    pc->riseUs_ = 0;
62    pc->fallUs_ = 0;
63    digitalWrite((uint8_t)pc->trig_, LOW);
64    delayMicroseconds(2);
65    pc->trigUs_ = nowUs();
66    pc->state_ = CAPTURE_WAITING;
67    digitalWrite((uint8_t)pc->trig_, HIGH);
68    delayMicroseconds(10);  // datasheet: >= 10 us TTL pulse
69    digitalWrite((uint8_t)pc->trig_, LOW);
70    return true;
71  }
72
73  CaptureState pulseCapturePoll(PulseCapture* pc, uint32_t* widthUs) {
74    if (widthUs) *widthUs = 0;
75    if (!pc->begun_) return CAPTURE_ERROR;
76    const CaptureState s = pc->state_;
77    if (s == CAPTURE_DONE && widthUs) *widthUs = pc->fallUs_ - pc->riseUs_;
78    return s;
79  }
80
81  uint32_t pulseCaptureElapsedUs(const PulseCapture* pc) {
82    if (pc->trigUs_ == 0) return 0;
83    return nowUs() - pc->trigUs_;
84  }
85
86  bool pulseCaptureEchoHigh(const PulseCapture* pc) {
87    return pc->begun_ && echoLevel(pc->echo_);
88  }
89
90  void pulseCaptureCancel(PulseCapture* pc) { pc->state_ = CAPTURE_IDLE; }
91
92  // The interrupt stays attached (pinMode() keeps the edge type): while IDLE the
93  // ISR ignores edges, and the 2 kOhm divider leg holds ECHO low with the module
94  // unpowered. The capture is usable again as soon as the rail returns.
95  void pulseCaptureSetSafe(PulseCapture* pc) {
96    pc->state_ = CAPTURE_IDLE;
97    if (!pc->begun_) return;
98    digitalWrite((uint8_t)pc->trig_, LOW);  // never drive TRIG high into an unpowered module
99    pinMode((uint8_t)pc->trig_, OUTPUT);
100   pinMode((uint8_t)pc->echo_, INPUT);  // no pulls
101 }
```

Return to guided explanation, p. 270

ORIGINAL SOURCE / CHAPTER 11 / 1 OF 1

src/domain/ultrasonic.h

Original lines 1-21 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // HC-SR04 echo-time conversion (WP4). Pure C, host-tested.
2  // Source: HC-SR04.pdf (module guide) p.2 specifications, p.4 equations 2-3.
3  //
4  // ECHO stays high for the sound's round trip, so distance = width * c / 2.
5  // With c = 343 m/s (20 degC, datasheet) that is 58.3 us per cm. No temperature
6  // compensation yet: c changes ~0.17 %/degC.
7  #pragma once
8  #include <stdbool.h>
9  #include <stdint.h>
10
11 #define ULTRASONIC_NOMINAL_SPEED_MPS 343.0f
12
13 // Datasheet ranging distance 2-400 cm.
14 #define ULTRASONIC_MIN_MM 20.0f
15 #define ULTRASONIC_MAX_MM 4000.0f
16
17 // Echo pulse width (us) -> one-way distance (mm).
18 float ultrasonicEchoUsToMm(uint32_t widthUs, float speedMps);
19
20 // True inside the datasheet ranging distance (inclusive).
21 bool ultrasonicInRangeMm(float mm);
```

Return to guided explanation, p. 265

ORIGINAL SOURCE / CHAPTER 11 / 1 OF 1

src/domain/ultrasonic.c

Original lines 1-10 | Code and comments unchanged; long lines wrap visually

ultrasonicEchoUsToMm (L3-6): Convert round-trip pulse time to one-way millimetres using half the sound-travel distance.
ultrasonicInRangeMm (L8-10): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.

```
1  #include "domain/ultrasonic.h"
2
3  float ultrasonicEchoUsToMm(uint32_t widthUs, float speedMps) {
4    // us * (m/s) = um; one way = half; um -> mm = /1000.
5    return (float)widthUs * speedMps / 2000.0f;
6  }
7
8  bool ultrasonicInRangeMm(float mm) {
9    return mm >= ULTRASONIC_MIN_MM && mm <= ULTRASONIC_MAX_MM;
10 }
```

Return to guided explanation, p. 265

ORIGINAL SOURCE / CHAPTER 11 / 1 OF 2

src/drivers/hcsr04.h

Original lines 1-28 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // hcsr04 driver. Work package WP4 (Stage 3). Architecture §7, §12, §14.
2  // Source: HC-SR04.pdf (module guide, pages 2-5).
3  //
4  // Switched 5 V rail. TRIG (GPIO17) driven at 3.3 V logic; ECHO (GPIO18)
5  // through a 1 kOhm / 2 kOhm divider (5 V -> ~3.33 V). A 10 us trigger starts an
6  // 8-cycle 40 kHz burst; ECHO stays high for the round trip. Distance =
7  // width * 343 m/s / 2 (domain/ultrasonic.h), no temperature compensation.
8  // Datasheet range 2-400 cm, resolution 0.3 cm. One ping per frame (1 s) leaves
9  // old echoes time to die out (datasheet §5: echoes give false short readings).
10 //
11 // Results (poll() returns READY for all of them; the verdict is in health,
12 // raw = echo width or elapsed time in us). A missing distance is never 0:
13 //   echo 20-4000 mm          -> value mm, UNIT_MILLIMETRE, HEALTH_OK
14 //   echo outside 20-4000 mm  -> HEALTH_FAULT, raw = width
15 //   no rising edge by timeout -> HEALTH_TIMEOUT, no raw (stuck low, ECHO
16 //                                unplugged or 5 V rail off: indistinguishable,
17 //                                there is no rail feedback pin)
18 //   rose, no fall by timeout -> HEALTH_TIMEOUT, raw = elapsed (no object in
19 //                                range; clones hold ECHO ~38 ms or longer)
20 // ECHO already high at init()/start() (stuck high, or a late echo still
21 // running) -> DRIVER_FAULT, stored reading HEALTH_FAULT, re-init required.
22 // The timeout is Hcsr04Config.timeoutMs (CFG_HCSR04_TIMEOUT_MS = 30 ms covers
23 // the 23.3 ms round trip at 4 m plus the ~0.5 ms burst lead-in).
24 //
25 // Worst-case blocking:
26 //   init()            none (one GPIO read)
27 //   start()           ~12 us busy-wait (2 us TRIG low + 10 us TRIG high)
28 //   poll()            none (reads the ISR's timestamps); Pending, 1 ms steps,
```

Return to guided explanation, p. 265

ORIGINAL SOURCE / CHAPTER 11 / 2 OF 2

src/drivers/hcsr04.h

Original lines 29-57 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
//                     until the echo ends or timeoutMs
//   cancel(), preparePowerOff()  a few GPIO writes
// The ECHO edge ISR (hal/pulse_capture.c) only stores a timestamp.
//
// preparePowerOff(): TRIG low, ECHO input without pulls (no driving into the
// unpowered module).
//
// Implements the SensorDriver contract in drivers/sensor_driver.h.
#pragma once
#include "drivers/sensor_driver.h"
#include "hal/pulse_capture.h"

typedef struct Hcsr04Config {
  PulseCapture* capture;  // already begun on BOARD_PIN_HCSR04_TRIG / _ECHO
  uint32_t timeoutMs;     // CFG_HCSR04_TIMEOUT_MS
} Hcsr04Config;

// Members after `base` are private to hcsr04.c.
typedef struct Hcsr04Driver {
  SensorDriver base;  // must stay first
  Hcsr04Config config_;
  bool initialised_;
  bool busy_;
  MonoMs startMs_;
  Reading result_;
} Hcsr04Driver;

// Constructor. Use &driver.base wherever a SensorDriver* is expected.
void hcsr04DriverConstruct(Hcsr04Driver* self, const Hcsr04Config* config);
```

Return to guided explanation, p. 265

GUIDED SOURCE READING / CHAPTER 11

Check completed capture before the timeout branch

The HAL timestamps edges independently of the worker's polling interval. hcsr04Poll therefore inspects captured completion before deciding that current wall time is too late.

Original excerpt: src/drivers/hcsr04.c (p. 278) / lines 55-71. The complete unchanged file follows.

```
55    const CaptureState s = pulseCapturePoll(pc, &width);
56    if (s == CAPTURE_DONE) {
57      // Accepted even on a late poll: the width is timestamped by the ISR.
58      const float mm = ultrasonicEchoUsToMm(width, ULTRASONIC_NOMINAL_SPEED_MPS);
59      return finish(self, now, ultrasonicInRangeMm(mm) ? HEALTH_OK : HEALTH_FAULT, true,
60                    width, mm);
61    }
62    if (s != CAPTURE_WAITING && s != CAPTURE_HIGH) {  // capture lost its arming
63      pulseCaptureCancel(pc);
64      return finish(self, now, HEALTH_FAULT, false, 0, 0.0f);
65    }
66
67    const MonoMs deadline = self->startMs_ + self->config_.timeoutMs;
68    if (!monoReached(now, deadline)) {
69      MonoMs next = now + kPollStepMs;
70      if (next > deadline) next = deadline;
71      r.status = DRIVER_PENDING;
```

CAPTURE_DONE returns the measured width immediately, even if this task was scheduled late. The timestamped echo may have completed on time; testing now against the deadline first would wrongly reject that completed capture.

The next guard rejects a capture that is neither waiting nor HIGH. Only an armed, unfinished capture reaches the deadline check. If time remains, the next poll is now +1 ms, capped at the deadline. That 1 ms controls how soon software notices completion; it is not the pulse-width resolution.

At timeout, elapsedUs is saved and capture is cancelled before finishing. A falling edge after cancellation is ignored rather than credited to a later ping. HIGH gives TIMEOUT with a raw elapsed value, while never rising gives TIMEOUT without raw echo data.

A valid 2000 us round trip gives 2000 times 343 /2000 =343 mm. The division by 2000 combines microsecond-to-second, metre-to-millimetre and the half-distance round trip. The driver applies range checks after conversion and stores health independently from any retained old distance value.

ORIGINAL SOURCE / CHAPTER 11 / 1 OF 3

src/drivers/hcsr04.c

Original lines 1-48 | Code and comments unchanged; long lines wrap visually

asHcsr04 (L11-11): Cast the common first-member SensorDriver pointer back to this concrete driver type.
hcsr04Id (L16-16): Return this module's sensor identity; this does not access hardware.

```
#include "drivers/hcsr04.h"

#include <stddef.h>

#include "domain/ultrasonic.h"

// Poll step while the echo is in flight. The width comes from the ISR's us
// timestamps, so this only sets how soon the result is noticed.
static const uint32_t kPollStepMs = 1;

static inline Hcsr04Driver* asHcsr04(SensorDriver* d) { return (Hcsr04Driver*)d; }

static PollResult finish(Hcsr04Driver* self, MonoMs now, Health health, bool hasRaw,
                         uint32_t rawUs, float mm);

static SensorId hcsr04Id(const SensorDriver* d) { (void)d; return SENSOR_HCSR04; }
static const char* hcsr04Name(const SensorDriver* d) { (void)d; return "hcsr04"; }
static uint8_t hcsr04MetricCount(const SensorDriver* d) { (void)d; return 1; }

static DriverStatus hcsr04Init(SensorDriver* d) {
  Hcsr04Driver* self = asHcsr04(d);
  self->initialised_ = false;
  self->busy_ = false;
  if (self->config_.capture == NULL || self->config_.timeoutMs == 0) return DRIVER_FAULT;
  pulseCaptureCancel(self->config_.capture);
  uint32_t width = 0;
  if (pulseCapturePoll(self->config_.capture, &width) == CAPTURE_ERROR) return DRIVER_FAULT;
  // Idle ECHO must be low; high means a stuck line or an echo still running.
  if (pulseCaptureEchoHigh(self->config_.capture)) return DRIVER_FAULT;
  self->initialised_ = true;
  return DRIVER_OK;
}

static DriverStatus hcsr04Start(SensorDriver* d, MonoMs now) {
  Hcsr04Driver* self = asHcsr04(d);
  if (!self->initialised_) return DRIVER_FAULT;
  self->startMs_ = now;
  if (!pulseCaptureTrigger(self->config_.capture)) {
    // ECHO high before the trigger: nothing was sent.
    self->initialised_ = false;
    finish(self, now, HEALTH_FAULT, false, 0, 0.0f);
    return DRIVER_FAULT;
  }
  self->busy_ = true;
  return DRIVER_OK;
}

static PollResult hcsr04Poll(SensorDriver* d, MonoMs now) {
```

Return to guided explanation, p. 277

ORIGINAL SOURCE / CHAPTER 11 / 2 OF 3

src/drivers/hcsr04.c

Original lines 49-96 | Code and comments unchanged; long lines wrap visually

hcsr04Poll (L48-81): Advance an in-flight operation only when useful; return PENDING with a future time, READY, or failure.
finish (L85-103): The capture HAL supplies timed edges. The driver decides range, timeout and Reading health.

```
  Hcsr04Driver* self = asHcsr04(d);
  PollResult r = {DRIVER_FAULT, now};
  if (!self->busy_) return r;
  PulseCapture* pc = self->config_.capture;

  uint32_t width = 0;
  const CaptureState s = pulseCapturePoll(pc, &width);
  if (s == CAPTURE_DONE) {
    // Accepted even on a late poll: the width is timestamped by the ISR.
    const float mm = ultrasonicEchoUsToMm(width, ULTRASONIC_NOMINAL_SPEED_MPS);
    return finish(self, now, ultrasonicInRangeMm(mm) ? HEALTH_OK : HEALTH_FAULT, true,
                  width, mm);
  }
  if (s != CAPTURE_WAITING && s != CAPTURE_HIGH) {  // capture lost its arming
    pulseCaptureCancel(pc);
    return finish(self, now, HEALTH_FAULT, false, 0, 0.0f);
  }

  const MonoMs deadline = self->startMs_ + self->config_.timeoutMs;
  if (!monoReached(now, deadline)) {
    MonoMs next = now + kPollStepMs;
    if (next > deadline) next = deadline;
    r.status = DRIVER_PENDING;
    r.nextPollMs = next;
    return r;
  }

  // Disarm first so a late falling edge is never credited to the next ping.
  const uint32_t elapsedUs = pulseCaptureElapsedUs(pc);
  pulseCaptureCancel(pc);
  if (s == CAPTURE_HIGH) return finish(self, now, HEALTH_TIMEOUT, true, elapsedUs, 0.0f);
  return finish(self, now, HEALTH_TIMEOUT, false, 0, 0.0f);  // never rose
}

// Store one outcome. Only HEALTH_OK carries a value; every other verdict keeps
// the previous value untouched and is reported through health, never as 0.
static PollResult finish(Hcsr04Driver* self, MonoMs now, Health health, bool hasRaw,
                         uint32_t rawUs, float mm) {
  self->busy_ = false;
  Reading* r = &self->result_;
  r->intervalStartMs = self->startMs_;
  r->intervalEndMs = now;
  r->hasRaw = hasRaw;
  r->raw = hasRaw ? (int32_t)rawUs : 0;
  r->qualityFlags = kQualityUncompensated;  // nominal 343 m/s, no air temperature
  r->unit = UNIT_MILLIMETRE;
  r->health = health;
  if (health == HEALTH_OK) {
```

Return to guided explanation, p. 277

ORIGINAL SOURCE / CHAPTER 11 / 3 OF 3

src/drivers/hcsr04.c

Original lines 97-145 | Code and comments unchanged; long lines wrap visually

finish (L85-103): The capture HAL supplies timed edges. The driver decides range, timeout and Reading health. **hcsr04ReadResult (L105-109)**: Copy the latest Reading record(s) into caller storage. Inspect health even when poll returned READY.

```
97      r->value = mm;
98      r->lastGoodMs = now;
99      ++r->sequence;
100   }
101   PollResult pr = {DRIVER_READY, now};
102   return pr;
103 }
104
105 static uint8_t hcsr04ReadResult(SensorDriver* d, Reading* out, uint8_t maxOut) {
106   if (out == NULL || maxOut < 1) return 0;
107   out[0] = asHcsr04(d)->result_;
108   return 1;
109 }
110
111 static void hcsr04Cancel(SensorDriver* d) {
112   Hcsr04Driver* self = asHcsr04(d);
113   self->busy_ = false;
114   if (self->config_.capture != NULL) pulseCaptureCancel(self->config_.capture);
115 }
116
117 // TRIG low and ECHO a plain input before the 5 V rail goes (§12). init()
118 // again once the rail is back.
119 static void hcsr04PreparePowerOff(SensorDriver* d) {
120   Hcsr04Driver* self = asHcsr04(d);
121   hcsr04Cancel(d);
122   if (self->config_.capture != NULL) pulseCaptureSetSafe(self->config_.capture);
123   self->initialised_ = false;
124 }
125
126 static const SensorDriverOps kHcsr04Ops = {
127     .id = hcsr04Id,
128     .name = hcsr04Name,
129     .metricCount = hcsr04MetricCount,
130     .init = hcsr04Init,
131     .start = hcsr04Start,
132     .poll = hcsr04Poll,
133     .readResult = hcsr04ReadResult,
134     .cancel = hcsr04Cancel,
135     .preparePowerOff = hcsr04PreparePowerOff,
136 };
137
138 void hcsr04DriverConstruct(Hcsr04Driver* self, const Hcsr04Config* config) {
139   self->base.ops = &kHcsr04Ops;
140   self->config_ = *config;
141   self->initialised_ = false;
142   self->busy_ = false;
143   self->startMs_ = 0;
144   readingReset(&self->result_, SENSOR_HCSR04, METRIC_WATER_DISTANCE, UNIT_MILLIMETRE);
145 }
```

Return to guided explanation, p. 277

ORIGINAL SOURCE / CHAPTER 11 / 1 OF 1

test/bringup/hcsr04_main.c

Original lines 1-46 | Code and comments unchanged; long lines wrap visually

setup (L32-44): Establish safe outputs, logger, gate, board check and queues; load configuration; start Safety, both acquisition workers, power, radios and supervision.
loop (L46-46): Park the Arduino loop task. The application work happens in explicitly created tasks.

```
1  // Bring-up entry point for hcsr04 (WP4).  pio run -e test_hcsr04 -t upload -t monitor
2  // Switched 5 V rail must be ON. Verify the 1k/2k ECHO divider first.
3  // Replaces src/main.c in this env only; links the production driver.
4  //
5  // Procedure (README "HC-SR04 bring-up"), note the line printed for each step:
6  //  0. Divider gate, before connecting GPIO18: with the module's ECHO wire
7  //     disconnected, put 5 V on the module side of the 1 kOhm and measure ~3.3 V
8  //     at the divider junction (or scope the junction on a running module).
9  //     Never force the module's ECHO output to 5 V: it is a driven output.
10 //  1. Flat target at 10, 50 and 100 cm: "[hcsr04] 1000.xx mm"; compare with a tape.
11 //  2. Point at open space (> 4 m): "water_distance=TIMEOUT (raw <us>)".
12 //  3. Unplug ECHO at the MODULE side (the 2 kOhm keeps GPIO18 low):
13 //     "water_distance=TIMEOUT", no raw. Not at the GPIO18 side: the pin has no
14 //     pull on purpose and would float. Do not test with the 5 V rail off while
15 //     this harness runs - it keeps driving TRIG high into the unpowered module.
16 //  4. Hold ECHO high (to 3.3 V on the GPIO side): "init -> FAULT".
17 //  5. Target closer than 2 cm: FAULT (raw = width) or an unstable value.
18 #include <Arduino.h>
19 #include <stdio.h>
20
21 #include "app_config.h"
22 #include "board_profile.h"
23 #include "bringup_harness.h"
24 #include "hal/pulse_capture.h"
25 #include "drivers/hcsr04.h"
26
27 static PulseCapture s_capture = PULSE_CAPTURE_INIT;
28 static Hcsr04Config s_config = {&s_capture, CFG_HCSR04_TIMEOUT_MS};
29 static Hcsr04Driver s_driver;
30 static BringupContext s_ctx = {&s_driver.base, CFG_FRAME_PERIOD_MS, NULL};
31
32 void setup(void) {
33   hcsr04DriverConstruct(&s_driver, &s_config);
34   delay(200);
35   printf("\n=== bring-up: %s ===\n", sensorDriverName(&s_driver.base));
36   if (!pulseCaptureBegin(&s_capture, BOARD_PIN_HCSR04_TRIG, BOARD_PIN_HCSR04_ECHO)) {
37     printf("pulse capture: invalid pins TRIG=%d ECHO=%d\n", (int)BOARD_PIN_HCSR04_TRIG,
38            (int)BOARD_PIN_HCSR04_ECHO);
39   }
40   printf("TRIG GPIO%d, ECHO GPIO%d, timeout %lu ms, nominal 343 m/s\n",
41          (int)BOARD_PIN_HCSR04_TRIG, (int)BOARD_PIN_HCSR04_ECHO,
42          (unsigned long)CFG_HCSR04_TIMEOUT_MS);
43   bringupStart(&s_ctx);
44 }
45
46 void loop(void) { vTaskDelay(portMAX_DELAY); }
```

Return to guided explanation, p. 265

PART III / SYSTEM SERVICES

CHAPTER 12

Alarm Rules, Lamp Policy and Relay Output

Measurements become actions through explicit policy. Alarm rules emit transitions, latches retain trips and the lamp controller applies hysteresis and dwell. Safety combines those decisions into the single relay command.

In this chapter

1. Derive trip, clear, persistence and latch behaviour.
2. Apply the 40/120 mV light policy and its forced-OFF paths.
3. Separate commanded relay state from physical contact feedback.

Chapter contents

12.1 Alarm rules turn accepted samples into events 283
12.2 Lamp control: hysteresis, dwell and explicit exceptions 284
12.3 Latches, relay output and what feedback is missing 285

Guided source readings begin on page 290.

Complete original listings begin on page 286. Shared files are linked in the explanations; Appendix A contains the full filename index.

CHAPTER 12

12.1 Alarm rules turn accepted samples into events

An AlarmRule names a metric, direction, trip limit, clear limit, persistence, severity, latch policy and actuator action. Acquisition owns one AlarmRuleState per rule and evaluates matching enabled rules immediately after storing a result.

evaluateRule()

First, the reading must be fresh and HEALTH_OK. Otherwise the decision is NOT_ELIGIBLE and any unfinished persistence window is reset. An invalid sample neither trips nor clears an already-tripped rule.

For an ABOVE rule, a value at or above tripThreshold starts or continues persistence. Once enough sample time has elapsed, the function emits TRIP and remembers tripped state. A tripped rule stays tripped until a valid sample is at or below clearThreshold, when it emits CLEAR. BELOW rules mirror the comparisons.

Hysteresis and persistence solve separate problems

Example: trip at 40 degC and clear at 37 degC leaves a 3 degC band. After a trip, 39 degC does not clear it. This prevents repeated transitions near one noisy threshold. A 10 s persistence requirement instead rejects short excursions; it does not define a separate clear level.

Persistence uses Reading.intervalEndMs. Samples at 1000 ms and 11000 ms beyond the limit can satisfy a 10000 ms persistence window if no intervening ineligible sample was evaluated. The function does not independently prove uninterrupted sampling between them. Sample continuity and stale-data policy remain system concerns.

What the function does not enforce

The caller filters enabled state and metric identity. The pure function does not validate the whole rule configuration or implement latching. There is still no command to edit alarm rules; they remain compiled policy. Any future rule-editing interface must validate finite thresholds, hysteresis ordering and supported actions.

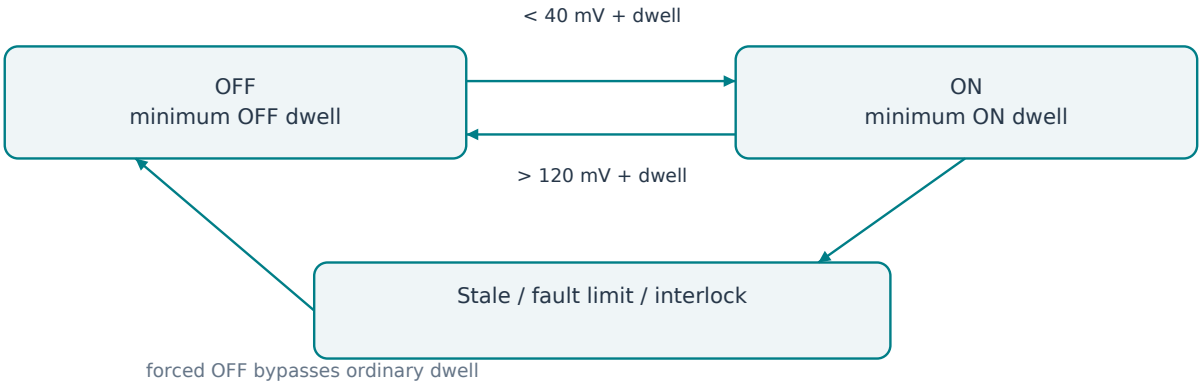
Production defines two examples: a latched air overtemperature relay-deenergise rule and a water-temperature warning with 10 s persistence. Both are disabled. Their 40/37 and 30/28 degC values are placeholders, not commissioned protection limits.

Original source in this PDF: src/domain/alarm.c (p. 291); src/domain/alarm.h (p. 288); include/policy_config.h (p. 286); src/services/acquisition.c (p. 96)

CHAPTER 12

12.2 Lamp control: hysteresis, dwell and explicit exceptions

LampControl starts OFF and records boot time as a state change. `lampControlUpdate()` accepts OK and UNCALIBRATED light readings. Saturation or pin voltage above 120 mV asks for OFF. When value is calibrated lux, the policy takes the pin mV from raw. A value below 40 mV asks for ON. Inside the band, it keeps the current state.



The comparisons are strict. Exactly 40 mV does not request ON; exactly 120 mV does not request OFF. Those boundary cases differ from alarm rules, which use inclusive trip/clear comparisons.

Dwell limits switching frequency

A requested change waits until the current state's minimum 30000 ms hold has elapsed. Dwell is checked on new light results, not by a separate expiration timer inside LampControl. Darkness at startup can therefore turn the lamp on at the first accepted new light update after the OFF dwell, not necessarily at the exact 30.000 s instant.

Example: after ON at 31 s, a bright input at 40 s requests OFF but only 9 s have elapsed. The lamp remains ON and waiting is true. A new bright result at or after 61 s permits OFF. A later dark result must satisfy the OFF dwell before ON returns.

Faults and forced OFF

Five consecutive unusable light results force OFF without waiting for normal dwell. Safety feeds each new result only once by comparing intervalEndMs; ten 100 ms Safety wakes over one failed Reading do not count as ten independent sensor faults.

Safety also checks light age directly. Older than 3000 ms forces OFF even if no new Reading arrives. Alarm interlock, missing input for an enabled relay-interlock rule and invalid 5V power also force OFF. `lampControlForceOff()` changes lastChangeMs only if it actually changes ON to OFF, so repeated force-off calls do not indefinitely restart a dwell while already OFF.

Original source in this PDF: src/domain/lamp_control.c (p. 301); src/domain/safety_core.c (p. 307); include/policy_config.h (p. 286)

CHAPTER 12

12.3 Latches, relay output and what feedback is missing

A latched alarm remembers a trip after the triggering condition clears. `safetyCoreResetLatch()` refuses reset while active is true. A valid recovery observation must therefore arrive before an explicit reset is accepted. The BLE command path queues reset_latch requests to Safety, which checks session, expiry and active state before replying.

The core sets `relayOn = lampAutoEnabled && !hold && lamp.on`. hold combines an alarm interlock, a missing input for an enabled relay-interlock rule, and invalid 5V rail. All must permit energising. The HAL maps the logical bool through BOARD_RELAY1_ACTIVE_LOW; the selected hardware profile is active-high.

```
// Source: src/hal/relay.c
const bool high = BOARD_RELAY1_ACTIVE_LOW ? !on : on;
digitalWrite(BOARD_PIN_RELAY1, high ? HIGH : LOW);
s_on = on;
```

A command is not a measured actuator state

`relayIsOn()` returns s_on, the last software command. There is no contact, coil-current or lamp-current feedback. A failed transistor, unpowered relay coil or welded contact would not be detected by that bool.

The production path has one normal relay owner: Safety. Boot and Safety initialisation establish OFF before ongoing control. `test_lamp` is a separate image with its own local relay helper; it is not a second production task writer.

Status semantics deserve a close read

SafetyOutputs includes lampWantsOn, relayOn, reason and masks. During an interlock, safetyCoreStep forces the internal lamp OFF before assigning lampWantsOn. Despite the field's broader explanatory comment, it therefore does not preserve a pre-interlock ON request in that case. Reason INTERLOCK and the masks explain the actual decision.

Turning `POLICY_LAMP_AUTO_ENABLED` off prevents relay energising while leaving the policy computation visible. It is not a calibration or protection setting for the other sensors. The board pin gate is separately logged and is not included in this relay expression.

Physical safe-state interpretation assumes the specified active-high transistor stage and lamp on the normally-open contact. These software functions do not verify that wiring.

Original source in this PDF: src/domain/safety_core.c (p. 307); src/hal/relay.c (p. 314); src/hal/gpio_safe.c (p. 312); test/bringup/lamp_main.c (p. 315)

ORIGINAL SOURCE / CHAPTER 12 / 1 OF 2

include/policy_config.h

Original lines 1-38 | Code and comments unchanged; long lines wrap visually

This is the active lamp/rule policy. Ordinary alarm rules are disabled; the special latency build changes one rule.

```
1  // Control policy: grow-lamp thresholds and alarm rules (Stage 4, WP7).
2  // NOT a frozen contract: these are placeholders to be replaced by measured /
3  // commissioned values. Pure C (no Arduino headers).
4  //
5  // Lamp: values from the test_lamp bench (README "Lamp relay bring-up"). Set
6  // POLICY_LAMP_OFF_ABOVE_MV above the "night + lamp ON" reading measured at the
7  // final sensor position, or the lamp will switch itself off.
8  #pragma once
9  #include <stdbool.h>
10 #include <stdint.h>
11
12 #include "app_config.h"
13 #include "domain/alarm.h"
14
15 // 1 = Safety drives the relay from the lamp decision. 0 = Safety still
16 // computes and reports the decision but keeps the relay de-energised.
17 #define POLICY_LAMP_AUTO_ENABLED 1
18
19 #define POLICY_LAMP_ON_BELOW_MV 40.0f
20 #define POLICY_LAMP_OFF_ABOVE_MV 120.0f
21 #define POLICY_LAMP_MIN_ON_MS ((uint32_t)30000)   // bench; minutes in real use
22 #define POLICY_LAMP_MIN_OFF_MS ((uint32_t)30000)
23 #define POLICY_LAMP_FAULTS_TO_OFF ((uint8_t)5)
24 // Light results older than this force the lamp OFF (§10 ordinary limit).
25 #define POLICY_LIGHT_STALE_MS CFG_STALE_ORDINARY_MS
26
27 // Analog calibration used by production acquisition (README "pH-4502C
28 // calibration", "SEN0244 TDS bring-up"). Nominal values: pH and TDS stay
29 // HEALTH_UNCALIBRATED until buffer / reference-solution fits replace them.
30 #define POLICY_PH_NOMINAL_MV_AT_PH7 2758.0f    // BNC shorted, 17 Oct 2026
31 #define POLICY_PH_NOMINAL_PH_PER_MV (-0.0057f)  // tutorial sketch slope
32
33 // On-device calibration (Calibration page) plausibility limits. A fit outside
34 // them is refused ("bad_fit"), never stored.
35 #define POLICY_PH_SLOPE_MIN_ABS 0.002f   // pH per module mV (nominal 0.0057)
36 #define POLICY_PH_SLOPE_MAX_ABS 0.02f
37 #define POLICY_TDS_K_MIN 0.5f            // K = reference ppm / nominal ppm
38 #define POLICY_TDS_K_MAX 2.0f
```

Return to guided explanation, p. 283

ORIGINAL SOURCE / CHAPTER 12 / 2 OF 2

include/policy_config.h

Original lines 39-77 | Code and comments unchanged; long lines wrap visually

This is the active lamp/rule policy. Ordinary alarm rules are disabled; the special latency build changes one rule.

```
#define POLICY_LIGHT_LUX_PER_MV_MIN 0.01f
#define POLICY_LIGHT_LUX_PER_MV_MAX 100.0f
// cal_capture uses a reading at most this old (2 frames + slack).
#define POLICY_CAL_CAPTURE_MAX_AGE_MS ((uint32_t)5000)

// WP9 low-power mode (services/power_coordinator.h). 1 = losing the external
// 5 V moves sensing to the low-power worker (3.3 V sensors only) and switches
// Wi-Fi off. 0 = kill switch: report the sense level only, change nothing.
// Not hardware-tested yet (written before the circuit existed): if a power
// transition misbehaves on the bench, set 0 and report it.
#define POLICY_POWER_TRANSITIONS_ENABLED 1

// Retry interval for a driver whose init() failed (§18: bounded, no loop).
#define POLICY_INIT_RETRY_MS ((uint32_t)5000)

// Alarm rules. ruleId = index. All DISABLED: no trip/clear limit has been
// commissioned (§20). Enable a rule only with measured, agreed limits.
#define POLICY_RULE_COUNT 2
static const AlarmRule kPolicyRules[POLICY_RULE_COUNT] = {
#ifdef FW_ALARM_LATENCY_TEST
    // Test build only (env:alarm_latency): trips on every valid air sample,
    // non-latching; acquisition re-arms it after each TRIP (§19 latency test).
    {.ruleId = 0, .enabled = true, .metric = METRIC_AIR_TEMPERATURE,
     .comparison = COMPARISON_ABOVE, .tripThreshold = -100.0f, .clearThreshold = -200.0f,
     .persistMs = 0, .severity = SEVERITY_CRITICAL, .latching = false,
     .actuatorId = ACTUATOR_RELAY1, .action = ACTUATOR_ACTION_DEENERGIZE},
#else
    // Example from §12: air over-temperature, latched, relay de-energised.
    {.ruleId = 0, .enabled = false, .metric = METRIC_AIR_TEMPERATURE,
     .comparison = COMPARISON_ABOVE, .tripThreshold = 40.0f, .clearThreshold = 37.0f,
     .persistMs = 0, .severity = SEVERITY_CRITICAL, .latching = true,
     .actuatorId = ACTUATOR_RELAY1, .action = ACTUATOR_ACTION_DEENERGIZE},
#endif
    // Water temperature high, warning only.
    {.ruleId = 1, .enabled = false, .metric = METRIC_WATER_TEMPERATURE,
     .comparison = COMPARISON_ABOVE, .tripThreshold = 30.0f, .clearThreshold = 28.0f,
     .persistMs = 10000, .severity = SEVERITY_WARNING, .latching = false,
     .actuatorId = ACTUATOR_NONE, .action = ACTUATOR_ACTION_NONE},
};
```

Return to guided explanation, p. 283

ORIGINAL SOURCE / CHAPTER 12 / 1 OF 2

src/domain/alarm.h

Original lines 1-35 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // Alarm rules and events (§10). Pure C.
2  // Work package WP7 (Stage 4). Thresholds are NOT commissioned: the 40/50 degC
3  // values in the original request are examples only (§1).
4  #pragma once
5  #include <stdbool.h>
6  #include <stdint.h>
7  #include "domain/reading.h"
8
9  typedef uint8_t Severity;
10 enum { SEVERITY_INFO = 0, SEVERITY_WARNING, SEVERITY_CRITICAL };
11
12 typedef uint8_t Comparison;
13 enum { COMPARISON_ABOVE = 0, COMPARISON_BELOW };
14
15 typedef uint8_t ActuatorAction;
16 enum { ACTUATOR_ACTION_NONE = 0, ACTUATOR_ACTION_DEENERGIZE, ACTUATOR_ACTION_ENERGIZE };
17
18 // Actuator ids used by AlarmRule.actuatorId (board has one relay, §20).
19 enum { ACTUATOR_NONE = 0, ACTUATOR_RELAY1 = 1 };
20
21 typedef struct AlarmRule {
22   uint8_t ruleId;         // index of the rule in its table
23   bool enabled;           // false until the limit is commissioned
24   MetricId metric;
25   Comparison comparison;
26   float tripThreshold;
27   float clearThreshold;   // hysteresis
28   uint32_t persistMs;     // 0 = trip on one valid sample
29   Severity severity;
30   bool latching;          // requires explicit reset after clearing
31   uint8_t actuatorId;
32   ActuatorAction action;
33 } AlarmRule;
34
35 typedef uint8_t RuleDecision;
```

Return to guided explanation, p. 283

ORIGINAL SOURCE / CHAPTER 12 / 2 OF 2

src/domain/alarm.h

Original lines 36-71 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
36 enum {
37   RULE_DECISION_NOT_ELIGIBLE = 0,
38   RULE_DECISION_NO_CHANGE,
39   RULE_DECISION_TRIP,
40   RULE_DECISION_CLEAR
41 };
42
43 // Event submitted by acquisition to Safety immediately after a sample (§4).
44 typedef struct AlarmEvent {
45   uint8_t ruleId;
46   RuleDecision decision;
47   Reading sample;
48 } AlarmEvent;
49
50 // Per-rule evaluation state, owned by the caller (acquisition, one per rule).
51 typedef struct AlarmRuleState {
52   bool tripped;
53   bool pending;           // beyond the trip threshold, persistMs not yet elapsed
54   MonoMs pendingSinceMs;
55 } AlarmRuleState;
56
57 // Freshness limit for a metric (§10): BME280 metrics 1 s, all others 3 s.
58 // Used by acquisition when it evaluates rules and by Safety's stale checks.
59 uint32_t alarmStaleLimitMs(MetricId metric);
60
61 // Pure threshold evaluation for one sample. Only `state` is modified.
62 // A sample counts only if readingIsActionableAt(sample, now, staleLimitMs):
63 // a stale or non-OK reading is NOT_ELIGIBLE, neither trips nor clears, and
64 // restarts any persistence window (a gap in valid data is not "sustained").
65 //
66 // ABOVE: trip when value >= trip for persistMs (0 = this sample), clear when
67 // value <= clear. BELOW mirrors it. trip/clear must leave a hysteresis band
68 // (ABOVE: clear < trip). Persistence is timed on the samples' intervalEndMs.
69 // Latching is Safety's job: CLEAR is reported even for a latching rule.
70 RuleDecision evaluateRule(const AlarmRule* rule, const Reading* sample, MonoMs now,
71                           uint32_t staleLimitMs, AlarmRuleState* state);
```

Return to guided explanation, p. 283

GUIDED SOURCE READING / CHAPTER 12

Validate the observation before starting persistence

A threshold alone is not an alarm. evaluateRule combines accepted input, trip/clear hysteresis and any required persistence before emitting a transition.

Original excerpt: src/domain/alarm.c (p. 291) / lines 24-40. The complete unchanged file follows.

```
24  RuleDecision evaluateRule(const AlarmRule* rule, const Reading* sample, MonoMs now,
25                            uint32_t staleLimitMs, AlarmRuleState* state) {
26    if (!readingIsActionableAt(sample, now, staleLimitMs)) {
27      state->pending = false;
28      return RULE_DECISION_NOT_ELIGIBLE;
29    }
30    const float v = sample->value;
31    const MonoMs t = sample->intervalEndMs;
32
33    if (state->tripped) {
34      if (!pastClear(rule, v)) return RULE_DECISION_NO_CHANGE;
35      state->tripped = false;
36      return RULE_DECISION_CLEAR;
37    }
38
39    if (!beyondTrip(rule, v)) {
40      state->pending = false;
```

The first guard requires fresh OK data. Rejected input clears pending persistence and returns NOT_ELIGIBLE, so an unobserved interval cannot count as evidence that the condition continuously persisted.

v and t are then taken from the accepted Reading. t is the sample's interval end, not merely when the consumer happened to run. Persistence therefore follows observation time. A delayed repeat evaluation does not create a new measurement timestamp.

If already tripped, the clear threshold is checked before any new trip logic. A value inside the hysteresis band preserves the current alarm state. Once clear is reached, tripped is reset and CLEAR is emitted once.

For an untripped rule, the remaining code starts pendingSinceMs on the first beyond-trip sample and requires enough sample-time duration. Only then does it clear pending, set tripped and return TRIP. The caller submits that transition to Safety; this pure function neither writes GPIO nor owns a latch.

ORIGINAL SOURCE / CHAPTER 12 / 1 OF 1

src/domain/alarm.c

Original lines 1-51 | Code and comments unchanged; long lines wrap visually

alarmStaleLimitMs (L5-12): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.
beyondTrip (L14-17): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.

```
#include "domain/alarm.h"

#include "app_config.h"

uint32_t alarmStaleLimitMs(MetricId metric) {
  switch (metric) {
    case METRIC_AIR_TEMPERATURE:
    case METRIC_AIR_HUMIDITY:
    case METRIC_AIR_PRESSURE: return CFG_STALE_BME280_MS;
    default: return CFG_STALE_ORDINARY_MS;
  }
}

static bool beyondTrip(const AlarmRule* rule, float v) {
  return rule->comparison == COMPARISON_ABOVE ? v >= rule->tripThreshold
                                              : v <= rule->tripThreshold;
}

static bool pastClear(const AlarmRule* rule, float v) {
  return rule->comparison == COMPARISON_ABOVE ? v <= rule->clearThreshold
                                              : v >= rule->clearThreshold;
}

RuleDecision evaluateRule(const AlarmRule* rule, const Reading* sample, MonoMs now,
                          uint32_t staleLimitMs, AlarmRuleState* state) {
  if (!readingIsActionableAt(sample, now, staleLimitMs)) {
    state->pending = false;
    return RULE_DECISION_NOT_ELIGIBLE;
  }
  const float v = sample->value;
  const MonoMs t = sample->intervalEndMs;

  if (state->tripped) {
    if (!pastClear(rule, v)) return RULE_DECISION_NO_CHANGE;
    state->tripped = false;
    return RULE_DECISION_CLEAR;
  }

  if (!beyondTrip(rule, v)) {
    state->pending = false;
    return RULE_DECISION_NO_CHANGE;
  }
  if (!state->pending) {
    state->pending = true;
    state->pendingSinceMs = t;
  }
  if (t - state->pendingSinceMs < rule->persistMs) return RULE_DECISION_NO_CHANGE;
  state->pending = false;
  state->tripped = true;
  return RULE_DECISION_TRIP;
}
```

Return to guided explanation, p. 290

ORIGINAL SOURCE / CHAPTER 12 / 1 OF 1

src/domain/alarm_sticky.h

Original lines 1-38 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // Sticky per-rule fallback for the alarm detail queue (§11). Pure C, no
2  // locking: services/safety.c wraps every call in its portMUX and does the
3  // queue send between stickyBegin() and stickyEnd().
4  //
5  // Producer (acquisition), per event:
6  //   stickyBegin()   record the decision, mark the slot in flight
7  //   xQueueSend()    outside the critical section
8  //   stickyEnd()     mark the record lost if the send failed, end the flight
9  // Consumer (Safety), per pass: drain the queue, then stickyCollect().
10 //
11 // A slot in flight is left untouched by stickyCollect(): its send result is
12 // not known yet, so its flags must survive until stickyEnd(). The producer
13 // wakes Safety after stickyEnd(), so the next pass sees the outcome. Without
14 // this, a drain between a failed send and the "lost" mark erased the TRIP
15 // from both delivery paths.
16 #pragma once
17 #include <stdbool.h>
18 #include <stdint.h>
19
20 #include "domain/alarm.h"
21
22 typedef struct StickySlot {
23   bool tripSinceDrain;  // a TRIP was submitted since Safety last collected
24   bool lost;            // a detail record of this rule was dropped
25   uint8_t inFlight;     // submits between stickyBegin() and stickyEnd()
26   RuleDecision last;    // last decision submitted (TRIP or CLEAR)
27   AlarmEvent trip;      // worst TRIP since the last collect
28   AlarmEvent clear;     // the last CLEAR
29 } StickySlot;
30
31 void stickyBegin(StickySlot* slot, const AlarmRule* rule, const AlarmEvent* e);
32 void stickyEnd(StickySlot* slot, bool queued);
33
34 // After the queue was drained: for each slot (not in flight) whose detail
35 // record was dropped, append the TRIP if none came through the queue
36 // (sawTrip[rule]) and, if the rule's last decision was CLEAR, that CLEAR
37 // too. Writes at most 2 * count events at out[0..]; returns how many.
38 uint8_t stickyCollect(StickySlot* slots, uint8_t count, const bool* sawTrip, AlarmEvent*
 > out);
```

Return to guided explanation, p. 283

ORIGINAL SOURCE / CHAPTER 12 / 1 OF 1

src/domain/alarm_sticky.c

Original lines 1-36 | Code and comments unchanged; long lines wrap visually

moreExtreme (L3-5): In-flight retention prevents a queue-send outcome race from erasing a dropped alarm decision.
stickyBegin (L7-16): Retain the latest decision and worst trip while marking a producer send in flight.

```
#include "domain/alarm_sticky.h"

static bool moreExtreme(const AlarmRule* rule, float a, float b) {
  return rule->comparison == COMPARISON_ABOVE ? a > b : a < b;
}

void stickyBegin(StickySlot* slot, const AlarmRule* rule, const AlarmEvent* e) {
  slot->last = e->decision;
  if (e->decision == RULE_DECISION_TRIP) {
    if (!slot->tripSinceDrain || moreExtreme(rule, e->sample.value, slot->trip.sample.value))
slot->trip = *e;
    slot->tripSinceDrain = true;
  } else {
    slot->clear = *e;
  }
  if (slot->inFlight < UINT8_MAX) ++slot->inFlight;
}

void stickyEnd(StickySlot* slot, bool queued) {
  if (!queued) slot->lost = true;
  if (slot->inFlight > 0) --slot->inFlight;
}

uint8_t stickyCollect(StickySlot* slots, uint8_t count, const bool* sawTrip, AlarmEvent* out)
{
  uint8_t n = 0;
  for (uint8_t r = 0; r < count; ++r) {
    StickySlot* slot = &slots[r];
    if (slot->inFlight > 0) continue;  // outcome unknown: keep everything for the next pass
    if (slot->lost) {
      if (slot->tripSinceDrain && !sawTrip[r]) out[n++] = slot->trip;
      if (slot->last == RULE_DECISION_CLEAR) out[n++] = slot->clear;
    }
    slot->tripSinceDrain = false;
    slot->lost = false;
  }
  return n;
}
```

Return to guided explanation, p. 283

ORIGINAL SOURCE / CHAPTER 12 / 1 OF 1

src/domain/event_journal.h

Original lines 1-51 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // Alarm event journal with independent reader cursors (§11, RTOS §14.1).
2  // Pure C, no locking: services/safety.c wraps writes and reads in a portMUX.
3  // Work package WP8 (Stage 5).
4  //
5  // Safety appends; BLE and LAN each keep their own cursor, so one slow or
6  // disconnected link never holds events back from the other. When a reader
7  // falls more than EVENT_JOURNAL_LEN events behind, the oldest details are
8  // lost and the next read reports a gap (drop oldest, with gap indication).
9  #pragma once
10 #include <stdbool.h>
11 #include <stdint.h>
12
13 #include "domain/alarm.h"
14
15 #define EVENT_JOURNAL_LEN 64  // = CFG_ALARM_JOURNAL_LEN (checked in services/safety.c)
16
17 typedef uint8_t JournalKind;
18 enum { JOURNAL_KIND_TRIP = 0, JOURNAL_KIND_CLEAR, JOURNAL_KIND_RESET };
19
20 typedef struct JournalEvent {
21   uint32_t n;          // 1, 2, 3, ... assigned on append
22   uint8_t ruleId;
23   Severity severity;
24   JournalKind kind;
25   MonoMs atMs;         // sample end time (trip/clear) or reset time
26   float value;
27 } JournalEvent;
28
29 typedef struct EventJournal {
30   JournalEvent ring[EVENT_JOURNAL_LEN];
31   uint32_t nextN;      // n the next append gets
32 } EventJournal;
33
34 typedef struct JournalCursor {
35   uint32_t nextN;      // next event this reader wants
36 } JournalCursor;
37
38 void journalInit(EventJournal* j);
39
40 // Stores a copy of *e with a new n; returns that n.
41 uint32_t journalAppend(EventJournal* j, const JournalEvent* e);
42
43 // A new reader starts at the next event (it does not replay history).
44 void journalCursorInit(const EventJournal* j, JournalCursor* c);
45
46 // Next event for this reader, false if none. *gap is true when events were
47 // lost before this one (the cursor jumps to the oldest one still stored).
48 bool journalNext(const EventJournal* j, JournalCursor* c, JournalEvent* out, bool* gap);
49
50 const char* journalKindName(JournalKind k);
51 const char* severityName(Severity s);
```

Return to guided explanation, p. 283

GUIDED SOURCE READING / CHAPTER 12

Advance a reader only when its event is accounted for

After Safety processes alarms, a bounded journal lets a transport observe transitions without owning Safety state or blocking the relay task.

Original excerpt: src/domain/event_journal.c (p. 296) / lines 24-35. The complete unchanged file follows.

```
24  bool journalNext(const EventJournal* j, JournalCursor* c, JournalEvent* out, bool* gap) {
25    *gap = false;
26    if (c->nextN >= j->nextN) return false;
27    const uint32_t oldest = oldestStored(j);
28    if (c->nextN < oldest) {
29      *gap = true;
30      c->nextN = oldest;
31    }
32    *out = j->ring[c->nextN % EVENT_JOURNAL_LEN];
33    ++c->nextN;
34    return true;
35  }
```

gap is initialized false for this call. If the reader is at the producer's next event number, there is nothing new and the function returns. Otherwise oldestStored identifies the earliest record still in the 64-entry ring.

A cursor older than that point has missed overwritten entries. The function sets gap and moves it to the oldest retained number before indexing the ring. Reading the old slot first would return an unrelated newer event that reused the same modulo position.

The selected event is copied out, then the cursor advances. journalNext itself does not transmit anything. The BLE sender uses a tentative cursor, attempts the send and commits its real cursor only on success. That extra caller-level ordering keeps a send failure from silently consuming an event.

The ring's capacity is an explicit retention limit. A slow link cannot hold Safety's producer back, and a reboot clears this RAM history. These are channel semantics, not a reason to reinterpret a current-state Snapshot as an event log.

ORIGINAL SOURCE / CHAPTER 12 / 1 OF 2

src/domain/event_journal.c

Original lines 1-26 | Code and comments unchanged; long lines wrap visually

journalInit (L5-8): Clear the RAM ring and start event numbering at one.
journalAppend (L10-16): Write the next numbered event into its modulo-64 slot, replacing old history when needed.

```
1  #include "domain/event_journal.h"
2
3  #include <string.h>
4
5  void journalInit(EventJournal* j) {
6    memset(j, 0, sizeof(*j));
7    j->nextN = 1;
8  }
9
10 uint32_t journalAppend(EventJournal* j, const JournalEvent* e) {
11   const uint32_t n = j->nextN++;
12   JournalEvent* slot = &j->ring[n % EVENT_JOURNAL_LEN];
13   *slot = *e;
14   slot->n = n;
15   return n;
16 }
17
18 void journalCursorInit(const EventJournal* j, JournalCursor* c) { c->nextN = j->nextN; }
19
20 static uint32_t oldestStored(const EventJournal* j) {
21   return j->nextN > EVENT_JOURNAL_LEN ? j->nextN - EVENT_JOURNAL_LEN : 1;
22 }
23
24 bool journalNext(const EventJournal* j, JournalCursor* c, JournalEvent* out, bool* gap) {
25   *gap = false;
26   if (c->nextN >= j->nextN) return false;
```

Return to guided explanation, p. 295

ORIGINAL SOURCE / CHAPTER 12 / 2 OF 2

src/domain/event_journal.c

Original lines 27-53 | Code and comments unchanged; long lines wrap visually

journalNext (L24-35): Return one retained event and advance the reader, indicating a gap if earlier events were overwritten.
journalKindName (L37-44): Translate an identity/state into a human-readable label for diagnostics.

```
27    const uint32_t oldest = oldestStored(j);
28    if (c->nextN < oldest) {
29      *gap = true;
30      c->nextN = oldest;
31    }
32    *out = j->ring[c->nextN % EVENT_JOURNAL_LEN];
33    ++c->nextN;
34    return true;
35  }
36
37  const char* journalKindName(JournalKind k) {
38    switch (k) {
39      case JOURNAL_KIND_TRIP: return "trip";
40      case JOURNAL_KIND_CLEAR: return "clear";
41      case JOURNAL_KIND_RESET: return "reset";
42    }
43    return "?";
44  }
45
46  const char* severityName(Severity s) {
47    switch (s) {
48      case SEVERITY_INFO: return "INFO";
49      case SEVERITY_WARNING: return "WARNING";
50      case SEVERITY_CRITICAL: return "CRITICAL";
51    }
52    return "?";
53  }
```

Return to guided explanation, p. 295

ORIGINAL SOURCE / CHAPTER 12 / 1 OF 2

src/domain/lamp_control.h

Original lines 1-27 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // Grow-lamp decision from the DFR0026 light level (bring-up, ahead of WP7).
2  // Pure C, host-tested. Decides only: the caller drives the relay.
3  //
4  // Hysteresis: ON below onBelowMv, OFF above offAboveMv, unchanged in between.
5  // Dwell: after a change the state is held for minOnMs / minOffMs, which also
6  // covers boot (init counts as a change to OFF), so a reset loop cannot cycle
7  // the lamp. The sensor may see the lamp it controls: offAboveMv must sit above
8  // the "night + lamp ON" reading measured at the final sensor position.
9  //
10 // Readings: OK and UNCALIBRATED (the light driver publishes uncalibrated mV)
11 // are used; a saturated one counts as bright. Anything else holds the state,
12 // and faultsToOff such readings in a row force OFF at once (safe state: relay
13 // de-energised, lamp on NO contact).
14 //
15 // Acts on UNCALIBRATED mV by explicit decision (relative light level; no lux
16 // calibration, D13). It does not check reading age itself: in production
17 // domain/safety_core feeds it each new light result and calls
18 // lampControlForceOff() when the light data is stale (audit B1).
19 #pragma once
20 #include <stdbool.h>
21 #include <stdint.h>
22
23 #include "domain/reading.h"
24 #include "domain/types.h"
25
26 typedef struct LampControlConfig {
27   float onBelowMv;     // must be < offAboveMv
```

Return to guided explanation, p. 283

ORIGINAL SOURCE / CHAPTER 12 / 2 OF 2

src/domain/lamp_control.h

Original lines 28-55 | Code and comments unchanged; long lines wrap visually

lampControlHeldMs (L53-55): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.

```
28    float offAboveMv;
29    uint32_t minOnMs;
30    uint32_t minOffMs;
31    uint8_t faultsToOff;  // >= 1
32  } LampControlConfig;
33
34  typedef struct LampControl {
35    LampControlConfig config;
36    bool on;
37    bool waiting;  // the level asks for a change the dwell is still holding back
38    MonoMs lastChangeMs;
39    uint8_t faults;  // unusable readings in a row
40  } LampControl;
41
42  // Starts OFF, as if it had just switched OFF at nowMs.
43  void lampControlInit(LampControl* self, const LampControlConfig* config, MonoMs nowMs);
44
45  // Feed one light reading; returns the lamp state to drive.
46  bool lampControlUpdate(LampControl* self, const Reading* light, MonoMs nowMs);
47
48  // Switch OFF now, ignoring the dwell (stale light data, interlock: §12). The
49  // OFF dwell starts here, so the lamp cannot come straight back on.
50  void lampControlForceOff(LampControl* self, MonoMs nowMs);
51
52  // Time in the current state.
53  static inline uint32_t lampControlHeldMs(const LampControl* self, MonoMs nowMs) {
54    return (uint32_t)(nowMs - self->lastChangeMs);
55  }
```

Return to guided explanation, p. 283

GUIDED SOURCE READING / CHAPTER 12

Use the electrical scale before comparing lamp thresholds

Calibrating the light display changes value from mV to lux. The lamp must still compare the same physical pin-voltage thresholds, so the policy selects its comparison input first.

Original excerpt: src/domain/lamp_control.c (p. 301) / lines 32-48. The complete unchanged file follows.

```
32    const float levelMv = light->unit == UNIT_MILLIVOLT || !light->hasRaw ? light->value :
 >  (float)light->raw;
33    const bool bright = (light->qualityFlags & kQualitySaturated) != 0;
34    bool want = self->on;
35    if (bright || levelMv > self->config.offAboveMv) {
36      want = false;
37    } else if (levelMv < self->config.onBelowMv) {
38      want = true;
39    }
40    if (want == self->on) {
41      self->waiting = false;
42      return self->on;
43    }
44
45    const uint32_t dwellMs = self->on ? self->config.minOnMs : self->config.minOffMs;
46    if (lampControlHeldMs(self, nowMs) >= dwellMs) {
47      setState(self, want, nowMs);
48    } else {
```

The conditional expression chooses value when it is already mV, or raw when value uses another unit and raw is available. This line comes before threshold comparisons because choosing the scale afterwards would compare lux against millivolt thresholds.

bright represents clipping, which also requests OFF. want starts as the current state: inside the 40-120 mV band, neither strict comparison changes it. That is hysteresis. Replacing want's initial state with false would make the middle band request OFF every time and destroy that memory.

If want already equals the current state, no transition or dwell timer is needed. Otherwise dwellMs is chosen from the current state: an ON lamp must satisfy minimum ON time before switching OFF, and vice versa. The later timestamp update occurs only when a change is accepted.

This policy reduces ordinary switching. Safety's forced-OFF path handles stale data, power and interlocks separately and can bypass dwell. Separating normal preference from a required hold prevents a 30 s anti-chatter setting from delaying a protective OFF action.

ORIGINAL SOURCE / CHAPTER 12 / 1 OF 2

src/domain/lamp_control.c

Original lines 1-26 | Code and comments unchanged; long lines wrap visually

setState (L3-9): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.
lampControlInit (L11-17): Check dependencies, configure this module and return whether initialisation succeeded.

```
1  #include "domain/lamp_control.h"
2
3  static void setState(LampControl* self, bool on, MonoMs nowMs) {
4    if (on != self->on) {
5      self->on = on;
6      self->lastChangeMs = nowMs;
7    }
8    self->waiting = false;
9  }
10
11 void lampControlInit(LampControl* self, const LampControlConfig* config, MonoMs nowMs) {
12   self->config = *config;
13   self->on = false;
14   self->waiting = false;
15   self->lastChangeMs = nowMs;
16   self->faults = 0;
17 }
18
19 void lampControlForceOff(LampControl* self, MonoMs nowMs) { setState(self, false, nowMs); }
20
21 bool lampControlUpdate(LampControl* self, const Reading* light, MonoMs nowMs) {
22   const bool usable = light->health == HEALTH_OK || light->health == HEALTH_UNCALIBRATED;
23   if (!usable) {
24     if (self->faults < UINT8_MAX) ++self->faults;
25     if (self->faults >= self->config.faultsToOff) setState(self, false, nowMs);
26     return self->on;
```

Return to guided explanation, p. 300

ORIGINAL SOURCE / CHAPTER 12 / 2 OF 2

src/domain/lamp_control.c

Original lines 27-52 | Code and comments unchanged; long lines wrap visually

lampControlUpdate (L21-52): Use pin mV (raw when value is lux), then apply hysteresis, dwell, saturation or fault handling.

```
  }
  self->faults = 0;

  // Thresholds are pin mV. After an on-device lux fit the value is lux, but
  // raw still carries the filtered pin mV: calibration never moves the lamp.
  const float levelMv = light->unit == UNIT_MILLIVOLT || !light->hasRaw ? light->value :
(float)light->raw;
  const bool bright = (light->qualityFlags & kQualitySaturated) != 0;
  bool want = self->on;
  if (bright || levelMv > self->config.offAboveMv) {
    want = false;
  } else if (levelMv < self->config.onBelowMv) {
    want = true;
  }
  if (want == self->on) {
    self->waiting = false;
    return self->on;
  }

  const uint32_t dwellMs = self->on ? self->config.minOnMs : self->config.minOffMs;
  if (lampControlHeldMs(self, nowMs) >= dwellMs) {
    setState(self, want, nowMs);
  } else {
    self->waiting = true;
  }
  return self->on;
}
```

Return to guided explanation, p. 300

ORIGINAL SOURCE / CHAPTER 12 / 1 OF 3

src/domain/safety_core.h

Original lines 1-47 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // Safety decision logic, pure C (§12). Work package WP7 (Stage 4).
2  // services/safety.c only waits, copies inputs in, calls safetyCoreStep() and
3  // writes the relay; every decision is here so it is host-tested with a fake
4  // clock.
5  //
6  // One step = one Safety wake, in the §12 order:
7  //   1 power validity      (always valid until WP9)
8  //   2 alarm events        TRIP latches (latching rules) / sets active; CLEAR
9  //                         releases only non-latching rules
10 //   3 freshness           new light result -> lamp_control; light result
11 //                         older than lightStaleMs -> lamp forced OFF; stale
12 //                         inputs of enabled rules are reported
13 //   4 commands            WP8: latch resets, refused when expired or while
14 //                         the rule is still active (§12 step 4)
15 //   5 output              relay = auto enabled AND lamp ON AND no interlock
16 //                         AND every relay-interlock input available
17 // Interlock: any enabled rule with action DEENERGIZE on ACTUATOR_RELAY1 that
18 // is latched or active, or that tripped in this step's events (so a TRIP
19 // followed by a CLEAR in the same batch still de-energises once).
20 // Unavailable input (§12): an enabled relay-interlock rule whose input is in
21 // staleMask cannot watch the relay, so the relay is held OFF (reason
22 // INPUT_MISSING) until its data is fresh again. A rule whose sensor was
23 // switched off is not monitored (staleMask), unless it is still latched.
24 #pragma once
25 #include <stdbool.h>
26 #include <stdint.h>
27
28 #include "domain/alarm.h"
29 #include "domain/lamp_control.h"
30 #include "domain/reading.h"
31 #include "domain/sensor_set.h"
32
33 #define SAFETY_MAX_RULES 8
34 #define SAFETY_MAX_COMMANDS 4
35
36 typedef uint8_t LampReason;
37 enum {
38   LAMP_REASON_NO_DATA = 0,  // no light result yet: lamp stays OFF
39   LAMP_REASON_LEVEL,        // following the light level (ON or OFF)
40   LAMP_REASON_DWELL,        // level asks for a change, dwell still holding
41   LAMP_REASON_FAULT,        // unusable light results
42   LAMP_REASON_STALE,        // light result too old: forced OFF
43   LAMP_REASON_INTERLOCK,    // alarm interlock: relay de-energised
44   LAMP_REASON_DISABLED,     // POLICY_LAMP_AUTO_ENABLED = 0
45   LAMP_REASON_UNPOWERED,    // WP9: external 5 V down, the relay coil has no supply
46   LAMP_REASON_INPUT_MISSING // a relay-interlock rule has no fresh input: relay held OFF
47 };
```

Return to guided explanation, p. 283

ORIGINAL SOURCE / CHAPTER 12 / 2 OF 3

src/domain/safety_core.h

Original lines 48-94 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
48
49  typedef struct SafetyConfig {
50    bool lampAutoEnabled;
51    LampControlConfig lamp;
52    uint32_t lightStaleMs;
53  } SafetyConfig;
54
55  typedef struct SafetyRuleState {
56    bool active;        // last decision was TRIP and no CLEAR since
57    bool latched;       // latching rule tripped; only safetyCoreResetLatch() clears
58    uint32_t trips;     // TRIP events seen
59    float worst;        // most extreme tripping value
60    MonoMs firstTripMs; // sample time of the first TRIP (0 = never)
61  } SafetyRuleState;
62
63  typedef struct SafetyCore {
64    SafetyConfig config;
65    const AlarmRule* rules;
66    uint8_t ruleCount;
67    SafetyRuleState rule[SAFETY_MAX_RULES];
68    LampControl lamp;
69    bool haveLight;
70    MonoMs lastLightEndMs;  // intervalEndMs of the last light result fed to lamp
71  } SafetyCore;
72
73  typedef uint8_t SafetyCommandKind;
74  enum { SAFETY_CMD_RESET_LATCH = 0 };
75
76  // A bounded remote request (WP8). The transport sets the expiry; a command
77  // that waited too long is refused, never applied late (§11, §12).
78  typedef struct SafetyCommand {
79    SafetyCommandKind kind;
80    uint8_t ruleId;
81    MonoMs expiresMs;
82    uint32_t tag;               // caller's request handle, echoed in the result
83  } SafetyCommand;
84
85  typedef uint8_t SafetyCommandOutcome;
86  enum {
87    SAFETY_CMD_APPLIED = 0,
88    SAFETY_CMD_EXPIRED,
89    SAFETY_CMD_STILL_ACTIVE,    // condition persists: no valid recovery yet
90    SAFETY_CMD_BAD_RULE
91  };
92
93  typedef struct SafetyCommandResult {
94    uint32_t tag;
```

Return to guided explanation, p. 283

ORIGINAL SOURCE / CHAPTER 12 / 3 OF 3

src/domain/safety_core.h

Original lines 95-142 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
  SafetyCommandOutcome outcome;
} SafetyCommandResult;

typedef struct SafetyInputs {
  const Snapshot* snap;       // NULL until the first publish
  const AlarmEvent* events;   // drained this wake, in arrival order
  uint8_t eventCount;
  MonoMs now;
  // WP8. Zero-initialised inputs keep the WP7 behaviour.
  const SafetyCommand* commands;
  uint8_t commandCount;        // at most SAFETY_MAX_COMMANDS are handled
  SensorMask sensorDisabled;   // bit per SensorId switched off by Configuration
  // WP9. The external 5 V rail is not valid (rail_state): the relay coil has
  // no supply, so the relay output is driven OFF (unavailable-actuator
  // policy, §13) and the lamp restarts from OFF, with its dwell, when it returns.
  bool railDown;
} SafetyInputs;

typedef struct SafetyOutputs {
  bool relayOn;          // what relay 1 must be
  bool lampWantsOn;      // lamp policy decision before interlock / enable
  LampReason lampReason;
  bool interlock;
  uint16_t latchedMask;  // bit per rule index
  uint16_t activeMask;
  uint16_t staleMask;    // enabled rules whose input is not fresh and OK
                         // (a disabled sensor's rule only while latched)
  SafetyCommandResult results[SAFETY_MAX_COMMANDS];
  uint8_t resultCount;
} SafetyOutputs;

// rules may be NULL with ruleCount 0. At most SAFETY_MAX_RULES are used.
void safetyCoreInit(SafetyCore* self, const SafetyConfig* config, const AlarmRule* rules,
                    uint8_t ruleCount, MonoMs now);

void safetyCoreStep(SafetyCore* self, const SafetyInputs* in, SafetyOutputs* out);

// Explicit reset of a latched rule (WP8 command). Refused (false) while the
// rule is still active: clearing needs a valid recovery observation first.
bool safetyCoreResetLatch(SafetyCore* self, uint8_t ruleId);

// True if any enabled rule on one of this sensor's metrics is latched or
// active in the given masks. Configuration refuses to disable such a sensor:
// switching an input off must never silence an alarm (§12).
bool safetyRulesBlockSensor(const AlarmRule* rules, uint8_t ruleCount, uint16_t latchedMask,
                            uint16_t activeMask, SensorId sensor);

const char* lampReasonName(LampReason r);
```

Return to guided explanation, p. 283

GUIDED SOURCE READING / CHAPTER 12

Apply events before accepting a latch reset

SafetyCore receives one batch of events, commands and copied state. Its fixed order prevents a reset from erasing a trip that arrived in the same pass.

Original excerpt: src/domain/safety_core.c (p. 307) / lines 120-132. The complete unchanged file follows.

```
120    // 2 Alarm events.
121    const bool trippedNow = applyEvents(self, in);
122
123    // 3 Freshness.
124    LampReason reason = updateLamp(self, in);
125    out->staleMask = staleRules(self, in);
126
127    // 4 Commands.
128    applyCommands(self, in, out);
129
130    // 5 Output.
131    bool interlock = trippedNow;
132    bool inputMissing = false;
```

applyEvents runs first and records active/latched state. Freshness and the new light observation are evaluated next. Commands then see the state after this pass's alarm events rather than the state before them. A simultaneous new TRIP therefore prevents a reset from being accepted while active.

The output phase combines trippedNow with current active and latched rules. trippedNow preserves an OFF action even if a CLEAR followed the TRIP in the same batch. A latching rule retains its memory beyond that pass; a non-latching rule still gets the one-pass action.

Missing input contributes an output hold only for enabled relay-deenergise rules. railDown is another independent hold. The final boolean requires lamp auto enabled, no hold and a lamp ON state before energizing.

Reason selection happens after the hold decision so status explains why the output is OFF. lampWantsOn is assigned after force-off in this implementation, so it describes the resulting internal lamp state rather than retaining a pre-interlock request. Following the actual assignment order is how a reader learns a field's implemented meaning.

ORIGINAL SOURCE / CHAPTER 12 / 1 OF 4

src/domain/safety_core.c

Original lines 1-49 | Code and comments unchanged; long lines wrap visually

isRelayInterlock (L5-8): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.
moreExtreme (L10-12): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.

```
#include "domain/safety_core.h"

#include <string.h>

static bool isRelayInterlock(const AlarmRule* rule) {
  return rule->enabled && rule->actuatorId == ACTUATOR_RELAY1 &&
         rule->action == ACTUATOR_ACTION_DEENERGIZE;
}

static bool moreExtreme(const AlarmRule* rule, float candidate, float current) {
  return rule->comparison == COMPARISON_ABOVE ? candidate > current : candidate < current;
}

void safetyCoreInit(SafetyCore* self, const SafetyConfig* config, const AlarmRule* rules,
                    uint8_t ruleCount, MonoMs now) {
  memset(self, 0, sizeof(*self));
  self->config = *config;
  self->rules = rules;
  self->ruleCount = ruleCount > SAFETY_MAX_RULES ? SAFETY_MAX_RULES : ruleCount;
  if (self->rules == NULL) self->ruleCount = 0;
  lampControlInit(&self->lamp, &config->lamp, now);  // boot counts as a switch to OFF
}

// Step 2. Returns true if a relay-interlock rule tripped in this batch.
static bool applyEvents(SafetyCore* self, const SafetyInputs* in) {
  bool trippedNow = false;
  for (uint8_t i = 0; i < in->eventCount; ++i) {
    const AlarmEvent* e = &in->events[i];
    if (e->ruleId >= self->ruleCount) continue;
    const AlarmRule* rule = &self->rules[e->ruleId];
    SafetyRuleState* st = &self->rule[e->ruleId];
    if (e->decision == RULE_DECISION_TRIP) {
      st->active = true;
      if (rule->latching) st->latched = true;
      if (st->trips == 0 || moreExtreme(rule, e->sample.value, st->worst)) {
        st->worst = e->sample.value;
      }
      if (st->trips == 0) st->firstTripMs = e->sample.intervalEndMs;
      if (st->trips < UINT32_MAX) ++st->trips;
      if (isRelayInterlock(rule)) trippedNow = true;
    } else if (e->decision == RULE_DECISION_CLEAR) {
      st->active = false;  // a latched rule stays latched until reset
    }
  }
  return trippedNow;
}

// Step 3 (lamp). Feeds lamp_control once per new light result, forces OFF on
// stale data even when no new result arrives.
```

Return to guided explanation, p. 306

ORIGINAL SOURCE / CHAPTER 12 / 2 OF 4

src/domain/safety_core.c

Original lines 50-97 | Code and comments unchanged; long lines wrap visually

updateLamp (L50-74): Feed a new light result once and force OFF if its age is too large.
staleRules (L76-94): Identify missing enabled-rule inputs; deliberately disabled inputs are omitted unless the rule remains latched.

```
50  static LampReason updateLamp(SafetyCore* self, const SafetyInputs* in) {
51    if (in->snap == NULL) return LAMP_REASON_NO_DATA;
52    const Reading* light = &in->snap->readings[METRIC_LIGHT_LEVEL];
53    if (light->health == HEALTH_STARTING || light->intervalEndMs == 0) {
54      if (!self->haveLight) return LAMP_REASON_NO_DATA;  // lamp is OFF since boot
55      if (self->lamp.on) lampControlForceOff(&self->lamp, in->now);
56      return LAMP_REASON_STALE;
57    }
58
59    // Age from intervalEndMs: readingHealthAt() does not age UNCALIBRATED
60    // readings, and the light driver publishes UNCALIBRATED mV.
61    const MonoMs end = light->intervalEndMs;
62    if (in->now > end && in->now - end > self->config.lightStaleMs) {
63      if (self->lamp.on) lampControlForceOff(&self->lamp, in->now);
64      return LAMP_REASON_STALE;
65    }
66
67    if (!self->haveLight || end != self->lastLightEndMs) {
68      self->haveLight = true;
69      self->lastLightEndMs = end;
70      lampControlUpdate(&self->lamp, light, in->now);
71    }
72    if (self->lamp.faults > 0) return LAMP_REASON_FAULT;
73    return self->lamp.waiting ? LAMP_REASON_DWELL : LAMP_REASON_LEVEL;
74  }
75
76  static uint16_t staleRules(const SafetyCore* self, const SafetyInputs* in) {
77    uint16_t mask = 0;
78    for (uint8_t i = 0; i < self->ruleCount; ++i) {
79      const AlarmRule* rule = &self->rules[i];
80      if (!rule->enabled) continue;
81      // A switched-off input is not monitored, unless its rule is still latched:
82      // then the missing data stays visible until the sensor is back.
83      if (rule->metric < METRIC_COUNT && sensorMaskHas(in->sensorDisabled,
>   sensorOfMetric(rule->metric)) &&
84          !self->rule[i].latched) {
85        continue;
86      }
87      const uint32_t limit = alarmStaleLimitMs(rule->metric);
88      if (in->snap == NULL || rule->metric >= METRIC_COUNT ||
89          !readingIsActionableAt(&in->snap->readings[rule->metric], in->now, limit)) {
90        mask = (uint16_t)(mask | (1u << i));
91      }
92    }
93    return mask;
94  }
95
96  static void applyCommands(SafetyCore* self, const SafetyInputs* in, SafetyOutputs* out) {
97    if (in->commands == NULL) return;
```

Return to guided explanation, p. 306

ORIGINAL SOURCE / CHAPTER 12 / 3 OF 4

src/domain/safety_core.c

Original lines 98-143 | Code and comments unchanged; long lines wrap visually

applyCommands (L96-111): Check expiry, supported reset kind and rule ID; refuse to clear an active rule and return per-command outcomes.
safetyCoreStep (L113-157): Apply alarms, freshness and reset commands, then hold OFF for interlock, missing relay-rule input or invalid rail.

```
  const uint8_t n = in->commandCount > SAFETY_MAX_COMMANDS ? SAFETY_MAX_COMMANDS :
in->commandCount;
  for (uint8_t i = 0; i < n; ++i) {
    const SafetyCommand* c = &in->commands[i];
    SafetyCommandResult* r = &out->results[out->resultCount++];
    r->tag = c->tag;
    if (in->now > c->expiresMs) {
      r->outcome = SAFETY_CMD_EXPIRED;
    } else if (c->kind != SAFETY_CMD_RESET_LATCH || c->ruleId >= self->ruleCount) {
      r->outcome = SAFETY_CMD_BAD_RULE;
    } else {
      r->outcome = safetyCoreResetLatch(self, c->ruleId) ? SAFETY_CMD_APPLIED :
SAFETY_CMD_STILL_ACTIVE;
    }
  }
}

void safetyCoreStep(SafetyCore* self, const SafetyInputs* in, SafetyOutputs* out) {
  memset(out, 0, sizeof(*out));

  // 1 Power validity (WP9): 5 V readings are already gated on the rail epoch
  // by acquisition (published UNPOWERED, never actionable); here only the
  // actuator on that rail is handled (step 5).

  // 2 Alarm events.
  const bool trippedNow = applyEvents(self, in);

  // 3 Freshness.
  LampReason reason = updateLamp(self, in);
  out->staleMask = staleRules(self, in);

  // 4 Commands.
  applyCommands(self, in, out);

  // 5 Output.
  bool interlock = trippedNow;
  bool inputMissing = false;
  for (uint8_t i = 0; i < self->ruleCount; ++i) {
    const SafetyRuleState* st = &self->rule[i];
    if (st->latched) out->latchedMask = (uint16_t)(out->latchedMask | (1u << i));
    if (st->active) out->activeMask = (uint16_t)(out->activeMask | (1u << i));
    if (!isRelayInterlock(&self->rules[i])) continue;
    if (st->latched || st->active) interlock = true;
    if ((out->staleMask & (1u << i)) != 0) inputMissing = true;  // the rule cannot see a
fault now
  }
  const bool hold = interlock || inputMissing || in->railDown;
  if (hold && self->lamp.on) lampControlForceOff(&self->lamp, in->now);

```

Return to guided explanation, p. 306

ORIGINAL SOURCE / CHAPTER 12 / 4 OF 4

src/domain/safety_core.c

Original lines 144-191 | Code and comments unchanged; long lines wrap visually

safetyCoreStep (L113-157): Apply alarms, freshness and reset commands, then hold OFF for interlock, missing relay-rule input or invalid rail.
safetyCoreResetLatch (L159-165): Refuse to reset an alarm still active; otherwise clear the retained latch.

```
144   out->interlock = interlock;
145   out->lampWantsOn = self->lamp.on;
146   if (interlock) {
147     reason = LAMP_REASON_INTERLOCK;
148   } else if (inputMissing) {
149     reason = LAMP_REASON_INPUT_MISSING;
150   } else if (in->railDown) {
151     reason = LAMP_REASON_UNPOWERED;
152   } else if (!self->config.lampAutoEnabled) {
153     reason = LAMP_REASON_DISABLED;
154   }
155   out->lampReason = reason;
156   out->relayOn = self->config.lampAutoEnabled && !hold && self->lamp.on;
157 }
158
159 bool safetyCoreResetLatch(SafetyCore* self, uint8_t ruleId) {
160   if (ruleId >= self->ruleCount) return false;
161   SafetyRuleState* st = &self->rule[ruleId];
162   if (st->active) return false;
163   st->latched = false;
164   return true;
165 }
166
167 bool safetyRulesBlockSensor(const AlarmRule* rules, uint8_t ruleCount, uint16_t latchedMask,
168                             uint16_t activeMask, SensorId sensor) {
169   if (rules == NULL) return false;
170   const uint16_t held = (uint16_t)(latchedMask | activeMask);
171   for (uint8_t i = 0; i < ruleCount && i < 16; ++i) {
172     if (!rules[i].enabled || (held & (1u << i)) == 0) continue;
173     if (rules[i].metric < METRIC_COUNT && sensorOfMetric(rules[i].metric) == sensor) return
  >
174 true;
175   }
176   return false;
177 }
178
179 const char* lampReasonName(LampReason r) {
180   switch (r) {
181     case LAMP_REASON_NO_DATA: return "no data";
182     case LAMP_REASON_LEVEL: return "level";
183     case LAMP_REASON_DWELL: return "dwell";
184     case LAMP_REASON_FAULT: return "fault";
185     case LAMP_REASON_STALE: return "stale";
186     case LAMP_REASON_INTERLOCK: return "interlock";
187     case LAMP_REASON_DISABLED: return "disabled";
188     case LAMP_REASON_UNPOWERED: return "no 5 V";
189     case LAMP_REASON_INPUT_MISSING: return "input missing";
190   }
191   return "?";
192 }
```

Return to guided explanation, p. 306

ORIGINAL SOURCE / CHAPTER 12 / 1 OF 1

src/hal/gpio_safe.h

Original lines 1-5 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // Boot-time safe output defaults (§5 "thin main", §10 actuator policy).
2  // Called first in setup(), before any network or sensor start.
3  #pragma once
4
5  void gpioApplySafeDefaults(void);
```

Return to guided explanation, p. 283

ORIGINAL SOURCE / CHAPTER 12 / 1 OF 1

src/hal/gpio_safe.c

Original lines 1-25 | Code and comments unchanged; long lines wrap visually

driveInactive (L9-15): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.
gpioApplySafeDefaults (L17-25): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.

```
1  #include "hal/gpio_safe.h"
2
3  #include <Arduino.h>
4
5  #include "board_profile.h"
6
7  // Level first, then output: the pin never drives the active level, even for
8  // the instant between the two calls.
9  static void driveInactive(int8_t pin, bool activeLow) {
10   if (!boardPinAssigned(pin)) return;
11   const uint8_t level = activeLow ? HIGH : LOW;
12   digitalWrite(pin, level);
13   pinMode(pin, OUTPUT);
14   digitalWrite(pin, level);
15 }
16
17 void gpioApplySafeDefaults(void) {
18   // Relay 1 (GPIO15, active-high): de-energised = lamp OFF, lamp on the NO
19   // contact. The 10 kOhm gate pull-down holds it off until here; from now on
20   // only Safety changes it.
21   driveInactive(BOARD_PIN_RELAY1, BOARD_RELAY1_ACTIVE_LOW);
22   // HC-SR04 TRIG (GPIO17) would float until pulseCaptureBegin(). LOW never
23   // back-feeds the module when its 5 V rail is off (§14).
24   driveInactive(BOARD_PIN_HCSR04_TRIG, false);
25 }
```

Return to guided explanation, p. 283

ORIGINAL SOURCE / CHAPTER 12 / 1 OF 1

src/hal/relay.h

Original lines 1-12 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // Relay 1 output (GPIO15, IRLZ44N low side). HAL, Stage 4 (WP7).
2  // SOLE USER: services/safety.c (§11 invariant "Safety alone writes relay
3  // outputs"). Do not include this header anywhere else.
4  //
5  // gpioApplySafeDefaults() has already driven the pin to its de-energised
6  // level and made it an output; relayWrite() only changes the level.
7  #pragma once
8  #include <stdbool.h>
9
10 // on = coil energised. Polarity from BOARD_RELAY1_ACTIVE_LOW.
11 void relayWrite(bool on);
12 bool relayIsOn(void);
```

Return to guided explanation, p. 283

ORIGINAL SOURCE / CHAPTER 12 / 1 OF 1

src/hal/relay.c

Original lines 1-16 | Code and comments unchanged; long lines wrap visually

relayWrite (L9-14): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.
relayIsOn (L16-16): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.

```
1  #include "hal/relay.h"
2
3  #include <Arduino.h>
4
5  #include "board_profile.h"
6
7  static bool s_on = false;  // gpioApplySafeDefaults() leaves it de-energised
8
9  void relayWrite(bool on) {
10   if (!boardPinAssigned(BOARD_PIN_RELAY1)) return;
11   const bool high = BOARD_RELAY1_ACTIVE_LOW ? !on : on;
12   digitalWrite(BOARD_PIN_RELAY1, high ? HIGH : LOW);
13   s_on = on;
14 }
15
16 bool relayIsOn(void) { return s_on; }
```

Return to guided explanation, p. 283

ORIGINAL SOURCE / CHAPTER 12 / 1 OF 2

test/bringup/lamp_main.c

Original lines 1-49 | Code and comments unchanged; long lines wrap visually

relayWrite (L43-46): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.
afterLight (L48-70): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.

```
// Bring-up: DFR0026 light level -> relay module -> grow lamp.
//   pio run -e test_lamp -t upload -t monitor
// Light as in test_light (S -> 10 kOhm -> GPIO1, 1 uF to GND). Relay: a
// TONGLING JQC-3FF-S-Z 5 V "Low Level Trigger" module, VCC 5 V, GND common.
// IN is driven through an N-MOSFET or NPN like the PCB stage (GPIO15 -> 220 Ohm
// gate / 1 kOhm base, 10 kOhm pull-down, IN on drain/collector), so GPIO15
// HIGH = relay ON: active-high, as BOARD_RELAY1_ACTIVE_LOW false. Direct drive
// (IN on GPIO15) failed on hardware 24 Oct 2026: 3.3 V cannot release the
// module's 5 V-referenced input, the relay stayed on with GPIO15 HIGH, and an
// unpowered module back-fed ~2.5 V onto VCC through IN.
// Lamp live on COM -> NO, NC empty: relay off = lamp off.
//
// Not the production path (WP7: Safety owns the relay). The decision is
// domain/lamp_control; this file only feeds it readings and drives the pin.
// One line per reading, e.g.
//   [lamp] 32 mV (uncal) -> ON  held 41 s
//   [lamp] 150 mV (uncal) -> ON  held 12 s, OFF waiting (dwell 30 s)
// LAMP_* thresholds are bench placeholders; set them from the lamp-off /
// lamp-on readings at the final sensor position (README "Lamp relay bring-up").
#include <Arduino.h>
#include <stdio.h>

#include "app_config.h"
#include "board_profile.h"
#include "bringup_harness.h"
#include "domain/lamp_control.h"
#include "drivers/light.h"
#include "hal/adc_channel.h"
#include "hal/clock.h"

#define LAMP_RELAY_ACTIVE_LOW 0  // transistor stage; 1 only for a module that releases at
3.3 V
#define LAMP_ON_BELOW_MV 40.0f
#define LAMP_OFF_ABOVE_MV 120.0f
#define LAMP_MIN_ON_MS ((uint32_t)30000)   // bench; minutes in real use
#define LAMP_MIN_OFF_MS ((uint32_t)30000)
#define LAMP_FAULTS_TO_OFF 5

static AdcChannel s_adc = ADC_CHANNEL_INIT;
static LightConfig s_config = {&s_adc};
static LightDriver s_driver;
static LampControl s_lamp;

static void relayWrite(bool on) {
  const bool high = LAMP_RELAY_ACTIVE_LOW ? !on : on;
  digitalWrite(BOARD_PIN_RELAY1, high ? HIGH : LOW);
}

static void afterLight(SensorDriver* driver) {
  Reading r;
```

Return to guided explanation, p. 283

ORIGINAL SOURCE / CHAPTER 12 / 2 OF 2

test/bringup/lamp_main.c

Original lines 50-98 | Code and comments unchanged; long lines wrap visually

afterLight (L48-70): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.
setup (L74-96): Establish safe outputs, logger, gate, board check and queues; load configuration; start Safety, both acquisition workers, power, radios and supervision.

```
  sensorDriverReadResult(driver, &r, 1);
  const MonoMs now = monoNowMs();
  const bool wasOn = s_lamp.on;
  const bool on = lampControlUpdate(&s_lamp, &r, now);
  relayWrite(on);

  printf("[lamp] ");
  if (r.health == HEALTH_OK || r.health == HEALTH_UNCALIBRATED) {
    printf("%.0f mV%s", (double)r.value, (r.qualityFlags & kQualitySaturated) ? " SAT" : "");
  } else {
    printf("light=%s (%u in a row)", healthName(r.health), (unsigned)s_lamp.faults);
  }
  printf(" -> %s  held %lu s", on ? "ON " : "OFF",
         (unsigned long)(lampControlHeldMs(&s_lamp, now) / 1000));
  if (s_lamp.waiting) {
    printf(", %s waiting (dwell %lu s)", on ? "OFF" : "ON",
           (unsigned long)((on ? LAMP_MIN_ON_MS : LAMP_MIN_OFF_MS) / 1000));
  }
  if (on != wasOn) printf("   <-- relay %s", on ? "ON" : "OFF");
  printf("\n");
}

static BringupContext s_ctx = {&s_driver.base, CFG_FRAME_PERIOD_MS, afterLight};

void setup(void) {
  // Relay off before the pin becomes an output, so the lamp never blinks.
  relayWrite(false);
  pinMode(BOARD_PIN_RELAY1, OUTPUT);
  relayWrite(false);

  static const LampControlConfig lampConfig = {LAMP_ON_BELOW_MV, LAMP_OFF_ABOVE_MV,
                                               LAMP_MIN_ON_MS, LAMP_MIN_OFF_MS,
                                               LAMP_FAULTS_TO_OFF};
  lampControlInit(&s_lamp, &lampConfig, monoNowMs());
  lightDriverConstruct(&s_driver, &s_config);
  delay(200);
  printf("\n=== bring-up: lamp (light GPIO%d -> relay GPIO%d, active-%s) ===\n",
         (int)BOARD_PIN_LIGHT_ADC, (int)BOARD_PIN_RELAY1, LAMP_RELAY_ACTIVE_LOW ? "low" :
"high");
  printf("ON below %.0f mV, OFF above %.0f mV, min on/off %lu/%lu s, OFF after %d bad
readings\n",
         (double)LAMP_ON_BELOW_MV, (double)LAMP_OFF_ABOVE_MV,
         (unsigned long)(LAMP_MIN_ON_MS / 1000), (unsigned long)(LAMP_MIN_OFF_MS / 1000),
         LAMP_FAULTS_TO_OFF);
  if (!adcChannelBegin(&s_adc, BOARD_PIN_LIGHT_ADC)) {
    printf("GPIO%d is not an ADC1 pin\n", (int)BOARD_PIN_LIGHT_ADC);
  }
  bringupStart(&s_ctx);
}

void loop(void) { vTaskDelay(portMAX_DELAY); }
```

Return to guided explanation, p. 283

PART III / SYSTEM SERVICES

CHAPTER 13

Power Modes, Rail Validity and Ownership Handover

A measurement can cross a power failure even when supply has returned by its final sample. Rail epochs record that history. A generation-checked lease then transfers the shared measurement engine between normal and low-power workers without creating a second owner.

In this chapter

1. Follow immediate rail invalidation and qualified power modes.
2. Trace quiesce, release acknowledgement and the next grant.
3. Read timeout, return-stability and diagnostic paths.

Chapter contents

13.1 Rail epochs prevent accepting a measurement across a transition 318
13.2 Power mode is a state machine, not a long delay 319
13.3 The power ISR records the urgent fact and defers policy 320
13.4 Power-sense diagnostics are a separate application 321
13.5 Power-sense analysis helpers, step by step 322

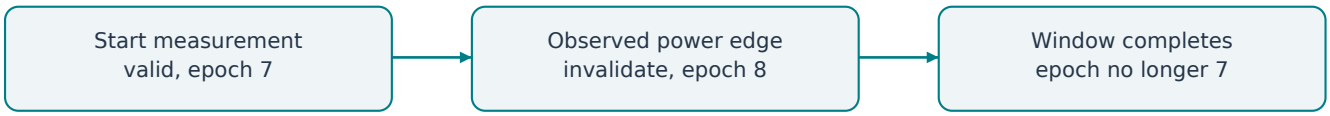
Guided source readings begin on page 327.

Complete original listings begin on page 323. Shared files are linked in the explanations; Appendix A contains the full filename index.

CHAPTER 13

13.1 Rail epochs prevent accepting a measurement across a transition

RailState is a locked pair: valid5v and epoch. Production starts invalid at epoch1. A sense-edge ISR invalidates the rail and advances the epoch. Restore succeeds only for the expected epoch, so a newer edge wins over an older restore decision.



A 5V result spanning the change becomes UNPOWERED.

Consumers also gate retained results against the live rail epoch.

Check before, during and after

tryStart refuses HC-SR04, pH and TDS while the rail is invalid and publishes UNPOWERED. When valid, it saves startEpoch. At completion, storeResults accepts power continuity only if the rail remains valid and its epoch matches startEpoch. A down-and-up event therefore invalidates a window even when power is back by the last tap.

The helper railSampleStillValid expresses this same rule, but acquisition performs the explicit validity/epoch comparison itself. A helper call is not required for the protection to exist.

A stored result can become invalid later

Power can fail just after acquisition publishes. Safety, transport telemetry and calibration capture each gate their own Snapshot copy against the current rail state. snapshotGateRail changes affected stored health to UNPOWERED without rewriting the producer's record. A display or capture must not revive a number from a previous power epoch.

Epoch is history, not voltage

An epoch counts invalidations, not volts, elapsed time or number of power cycles. The ISR mutes itself during chatter, so one storm can be represented by one invalidation. PowerCoordinator polls while muted and restarts its stable-return timer when history changes or edges are unobservable.

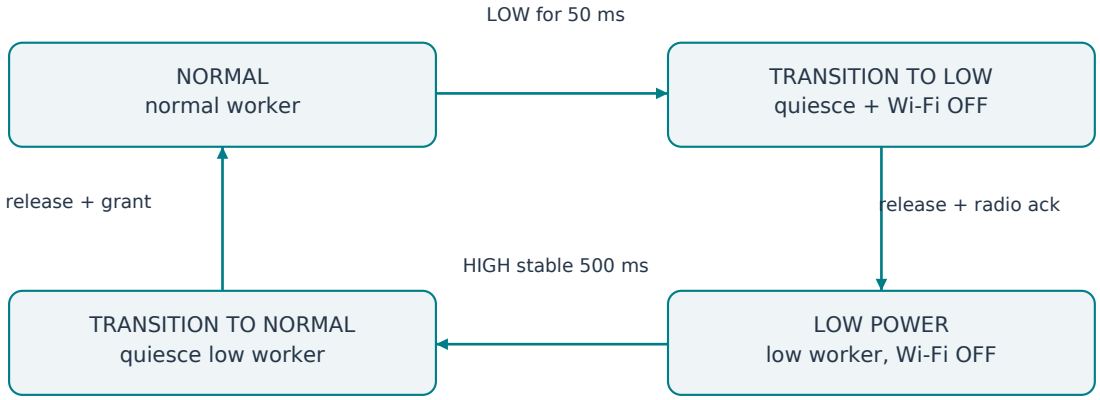
The ISR fallback is important: if arming fails and only polling remains, a pulse wholly between samples can be missed. This makes the distinction between observed power history and sampled level explicit: the ISR catches edges, while the task qualifies the level over time.

Original source in this PDF: src/services/rail_state.c (p. 324); src/services/acquisition.c (p. 96); src/domain/sensor_set.c (p. 55); src/services/power_coordinator.c (p. 334)

CHAPTER 13

13.2 Power mode is a state machine, not a long delay

powerModeStep takes an input record and returns actions. It never touches GPIO, tasks or Wi-Fi. PowerCoordinator executes those actions. This separation lets the same decisions run with fake time in native tests.



Missing acknowledgement -> DEGRADED, keep waiting safely.

A fresh loss aborts a pending return. Rail epoch checks remain active.

Loss and return use different timing

Loss requires 50 ms continuously LOW. Return requires 500 ms stable HIGH. Loss initiates quiescence and Wi-Fi shutdown. Once the current generation releases, the low-power worker is granted BME280, DS18B20, light and SGP30. Mode becomes LOW_POWER after Wi-Fi confirms radio OFF.

Return quiesces the low-power worker, checks the release generation, restores the expected rail epoch, grants normal ownership and restarts Wi-Fi. Another loss before the normal grant aborts the return path. At boot with power absent, the conservative path grants the low-power worker.

Two bounded acknowledgement waits

The worker deadline is 200 ms; Wi-Fi OFF has 2000 ms. A late acknowledgement can recover from DEGRADED. The function keeps waiting rather than abandoning the ownership requirement. Mode labels summarize the current transition; an owner field and phase hold the details.

In NORMAL, a short observed glitch invalidates the rail even when it does not reach loss debounce. The stable-return timer then restores validity only for the matching epoch. A new epoch or muted ISR restarts that timer, preventing an unseen edge from being accidentally acknowledged.

Original source in this PDF: src/domain/power_mode.h (p. 325); src/domain/power_mode.c (p. 328)

CHAPTER 13

13.3 The power ISR records the urgent fact and defers policy

The GPIO8 CHANGE handler invalidates the rail, disables its own interrupt, counts the edge and notifies PowerCoordinator. It does not debounce, switch workers, print or wait. The task samples every 10 ms while settling or transitioning and up to every 100 ms in steady operation.

Why mute an interrupt

The measured divider can chatter heavily during a loss. Handling every edge could monopolize CPU time. After the first edge, the task observes the level and rearms after 50 ms at the same sampled level. While muted, the power state machine treats edge history as unobservable and does not claim stable return.

Why HAL startup placement matters

AcqNormal installs the GPIO ISR service on core1 through the ultrasonic capture HAL. PowerCoordinator waits up to 2 s for HAL readiness before attaching its sense interrupt, avoiding installation on core0 next to radio activity. If readiness fails it falls back to polling, which cannot detect pulses entirely between samples.

The service is installed without the IRAM allocation flag. Marking an individual function IRAM does not make that entire interrupt service run through flash-cache-disabled windows. The flash gate is a separate measure against ultrasonic timestamp corruption during NVS writes.

Low power is a policy subset

The current mode does not reduce CPU frequency, enter light sleep or stop core1. Safety stays on core1. BLE remains the data link. Wi-Fi is stopped, and only retained-supply sensors are scheduled. Hardware validation must establish real current consumption and transition behaviour.

With POLICY_POWER_TRANSITIONS_ENABLED set to 0, there is no sense ISR, normal ownership allows all sensors and mode mainly reports power level. GPIO8 must have the specified divider; a floating input is not a trustworthy power detector.

Original source in this PDF: src/services/power_coordinator.c (p. 334); src/services/acquisition.c (p. 96); include/policy_config.h (p. 286)

CHAPTER 13

13.4 Power-sense diagnostics are a separate application

`test_power_sense` replaces production main. It samples GPIO8 two ways: CHANGE interrupts record edge microseconds, and an ADC task records voltage at a nominal 1 kHz. Its NORMAL/LOW-POWER banner reports the test's qualified input level; it does not run the production power coordinator.

The divider is 220 Ohm over 440 Ohm, giving 5V * 440/660 = 3.333V nominal. The current is about 7.58 mA; top-resistor dissipation is about 12.6 mW and total divider dissipation about 37.9 mW at 5V. The ADC clips around the code's 3050 mV threshold, so it cannot establish the actual HIGH ceiling.

The diagnostic data structures

Sample stores timestamp, millivolts and digital levels before/after conversion. Edge stores timestamp and level. Transition summarises direction, edge count, bounce, threshold times and outcome. An 8192-sample ring holds about 8.2 s at the nominal rate; a 1024-edge ring retains bursts; 48 transition summaries provide a short history.

`samplerTask()` runs on core 0 at priority 5. `onEdge()` only records to the edge ring. `analyzerTask()` runs on core 1 at priority 2, drains edges, groups transitions, scans ADC history, qualifies a level after 20 ms of edge quiet and prints results.

The low-level ADC1 API is used because repeated Arduino analogRead calls can alter pad configuration and disable the digital input path. Setup configures ADC first, then restores the digital input and performs a self-check. These diagnostics test that the two observation paths still agree.

Interpreting its output

A clipped HIGH is only a lower bound; a multimeter is required for its exact voltage. Threshold timing, bounce and temporary outages are distinct from a final stable level. The README records hundreds of edges during some falling transitions and a delayed first digital edge, which motivates qualification and epoch invalidation.

The diagnostic rings use volatile counters and specialised producer/consumer assumptions. They are not a ready-made, fully synchronised event channel for general production reuse.

Original source in this PDF: test/bringup/power_sense_main.c (p. 338); include/board_profile.h (p. 32); README.md

CHAPTER 13

13.5 Power-sense analysis helpers, step by step

The power diagnostic is larger than a sensor test because it correlates analog decay with digital edges. Its functions fall into acquisition, indexing, analysis and presentation.

Functions	Reasoning
`gpioLevel`, `onEdge`, `adcReadMv`, `samplerTask`	Capture fast evidence with minimal ISR work and periodic analog sampling.
`sampleAt`, `sampleIndexAt`, `spanMv`	Locate ring-buffer samples and compute voltage ranges/means over selected intervals.
`bandName`, `msSince`	Translate voltage into guaranteed LOW/HIGH bands and short relative times.
`reportTransition`	Reconstruct departure, threshold crossings, pulse bursts and final outcome.
`printSummary`, `printTable`	Explain one transition and retain a compact history.
`handleKeys`, `printHelp`	Offer details, history, ADC toggle and counter reset through nonblocking console input.

`reportTransition()` estimates pre/post voltage means, finds departure from the old level, and records times crossing digital bands. It separately recognises an analog dip that returns without edges and an outage that returns to the original band. Neither is equivalent to a clean one-way final transition.

Qualification versus settling

The mode banner uses 20 ms with no edge to qualify the current digital level. Detailed reporting waits for 300 ms edge quiet and an analog stable window of 200 ms with less than 30 mV span, or a 7 s timeout. These serve different purposes: prompt mode indication and a more complete diagnostic report.

The analyzer notices edge-ring overrun and skips to the retained range, but the rings are bounded history, not an unlimited recorder. Their 32-bit microsecond arithmetic assumes short intervals. A long-running timestamp absolute value cannot be treated as never wrapping.

Setup checks the ADC channel-to-pin mapping, characterises the ADC, restores digital input, attaches the ISR and enables nonblocking stdin. Keys `v`, `d`, `a`, `r`, `h` change only this diagnostic application.

Original source in this PDF: test/bringup/power_sense_main.c (p. 338)

ORIGINAL SOURCE / CHAPTER 13 / 1 OF 1

src/services/rail_state.h

Original lines 1-22 | Code and comments unchanged; long lines wrap visually

railSampleStillValid (L20-22): Require valid supply before and after with matching epoch; acquisition implements the equivalent explicit comparison.

```
1  // 5 V rail validity + epoch (§12). Work package WP9 (Stage 6).
2  // The ISR invalidates and bumps the epoch on every sense edge; only
3  // PowerCoordinator may restore validity. Copies are taken in a short
4  // portMUX critical section so both cores see a coherent {valid, epoch}.
5  #pragma once
6  #include <stdbool.h>
7  #include <stdint.h>
8
9  typedef struct RailState {
10   bool valid5v;
11   uint32_t epoch;
12 } RailState;
13
14 RailState railStateRead(void);
15 void railStateInvalidateFromIsr(void);          // ISR-safe
16 void railStateRestore(uint32_t expectedEpoch);  // PowerCoordinator only
17
18 // Accept a 5 V result only if the rail was valid before and after and the
19 // epoch did not change in between.
20 static inline bool railSampleStillValid(const RailState* before, const RailState* after) {
21   return before->valid5v && after->valid5v && before->epoch == after->epoch;
22 }
```

Return to guided explanation, p. 318

ORIGINAL SOURCE / CHAPTER 13 / 1 OF 1

src/services/rail_state.c

Original lines 1-33 | Code and comments unchanged; long lines wrap visually

railStateRead (L14-20): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.
railStateInvalidateFromIsr (L22-27): Invalidate the 5V rail and advance its history counter using an ISR-safe critical section.

```
1  #include "services/rail_state.h"
2
3  #include <freertos/FreeRTOS.h>
4
5  static portMUX_TYPE s_railMux = portMUX_INITIALIZER_UNLOCKED;
6  // Bench bring-up assumes the 5 V rail is present. Otherwise unknown power at
7  // boot is treated conservatively as unavailable (§11) until WP9 samples it.
8  #ifdef FW_BRINGUP
9  static RailState s_rail = {true, 1};
10 #else
11 static RailState s_rail = {false, 1};
12 #endif
13
14 RailState railStateRead(void) {
15   RailState copy;
16   portENTER_CRITICAL(&s_railMux);
17   copy = s_rail;
18   portEXIT_CRITICAL(&s_railMux);
19   return copy;
20 }
21
22 void railStateInvalidateFromIsr(void) {
23   portENTER_CRITICAL_ISR(&s_railMux);
24   s_rail.valid5v = false;
25   s_rail.epoch++;
26   portEXIT_CRITICAL_ISR(&s_railMux);
27 }
28
29 void railStateRestore(uint32_t expectedEpoch) {
30   portENTER_CRITICAL(&s_railMux);
31   if (s_rail.epoch == expectedEpoch) s_rail.valid5v = true;
32   portEXIT_CRITICAL(&s_railMux);
33 }
```

Return to guided explanation, p. 318

ORIGINAL SOURCE / CHAPTER 13 / 1 OF 2

src/domain/power_mode.h

Original lines 1-51 | Code and comments unchanged; long lines wrap visually

Mode, owner, generation, timestamps and returned actions make the ownership protocol explicit.

```
1  // Power-mode state machine and acquisition lease decisions (§13, §14).
2  // Pure C, host-tested. Work package WP9 (Stage 6).
3  //
4  // PowerCoordinator (services/power_coordinator.c) feeds it the external-power
5  // sense level, the rail validity and the acknowledgements, once per tick, and
6  // executes the actions it returns. It never touches hardware itself.
7  //
8  //   NORMAL --(power low for lossDebounceMs)--> TRANSITION_TO_LOW:
9  //       quiesce the normal worker, stop Wi-Fi;
10 //       on RELEASED(gen): grant the low-power worker (3.3 V sensors only);
11 //       when Wi-Fi reports its radio off: LOW_POWER.
12 //   LOW_POWER --(power high for returnStableMs)--> TRANSITION_TO_NORMAL:
13 //       quiesce the low-power worker;
14 //       on RELEASED(gen): restore the 5 V rail, grant the normal worker (all
15 //       sensors), start Wi-Fi: NORMAL.
16 //       Power lost again before the grant aborts the return (back to the
17 //       low-power path).
18 //   A missing RELEASED after ackTimeoutMs, or Wi-Fi not off after
19 //   networkAckTimeoutMs, reports DEGRADED. The machine keeps waiting: it never
20 //   grants a second owner while the first has not released (§14), and moves
21 //   on as soon as the late acknowledgement arrives.
22 //
23 // Rail validity: the ISR invalidates the rail on every sense edge (also a
24 // short glitch). In NORMAL the machine restores it after returnStableMs of
25 // stable power. Stability is measured from the newest rail epoch: an edge the
26 // ISR saw but this machine did not sample (a LOW-HIGH pulse between two ticks)
27 // still restarts the timer, and so does a muted ISR (edges unobservable). The
28 // restore names the epoch it was decided on, so a newer edge is never
29 // acknowledged by mistake. Boot: power present -> normal worker, rail restored once
30 // stable; power absent -> conservative low-power path.
31 //
32 // transitionsEnabled = false (POLICY_POWER_TRANSITIONS_ENABLED 0): the normal
33 // worker keeps every sensor, the rail is kept valid and the mode only reports
34 // the debounced sense level (NORMAL / LOW_POWER), nothing is switched.
35 #pragma once
36 #include <stdbool.h>
37 #include <stdint.h>
38
39 #include "domain/types.h"
40
41 typedef uint8_t PowerMode;
42 enum {
43   POWER_MODE_UNKNOWN = 0,
44   POWER_MODE_NORMAL,
45   POWER_MODE_TRANSITION_TO_LOW,
46   POWER_MODE_LOW_POWER,
47   POWER_MODE_TRANSITION_TO_NORMAL,
48   POWER_MODE_DEGRADED
49 };
50
51 typedef uint8_t PowerWorker;
```

Return to guided explanation, p. 318

ORIGINAL SOURCE / CHAPTER 13 / 2 OF 2

src/domain/power_mode.h

Original lines 52-103 | Code and comments unchanged; long lines wrap visually

Mode, owner, generation, timestamps and returned actions make the ownership protocol explicit.

```
52  enum { POWER_WORKER_NONE = 0, POWER_WORKER_NORMAL, POWER_WORKER_LOW };
53
54  typedef struct PowerModeConfig {
55    uint32_t lossDebounceMs;       // power low this long = lost (CFG_POWER_LOSS_DEBOUNCE_MS)
56    uint32_t returnStableMs;       // power high this long = back (CFG_POWER_RETURN_STABLE_MS)
57    uint32_t ackTimeoutMs;         // worker RELEASED deadline (CFG_TRANSITION_ACK_MS)
58    uint32_t networkAckTimeoutMs;  // Wi-Fi radio off deadline (CFG_NETWORK_SHUTDOWN_ACK_MS)
59    bool transitionsEnabled;
60  } PowerModeConfig;
61
62  typedef struct PowerModeInputs {
63    MonoMs now;
64    bool externalPower;     // sense level now (true = external 5 V present)
65    bool railValid;         // rail_state validity now
66    uint32_t releasedGen;   // newest generation the lease holder released (0 = none)
67    bool networkOff;        // Wi-Fi acknowledged: radio off
68    uint32_t railEpoch;     // rail_state epoch, read together with railValid
69    bool edgesMissed;       // the sense ISR is muted: edges are not counted now
70  } PowerModeInputs;
71
72  // What PowerCoordinator must do after a step, in this order: quiesce,
73  // restore the rail, grant, Wi-Fi.
74  typedef struct PowerModeActions {
75    PowerWorker quiesce;    // NONE = nothing
76    uint32_t quiesceGen;
77    bool restoreRail;
78    uint32_t restoreEpoch;  // restore only if the rail is still at this epoch
79    PowerWorker grant;      // NONE = nothing
80    uint32_t grantGen;
81    bool wifiStop;
82    bool wifiStart;
83  } PowerModeActions;
84
85  typedef struct PowerModeFsm {
86    PowerModeConfig cfg;
87    PowerMode mode;
88    uint8_t phase;          // private
89    PowerWorker owner;      // lease holder (NONE while handing over)
90    PowerWorker target;     // worker to grant once the owner has released
91    uint32_t gen;           // generation of the current / pending lease
92    bool netPending;        // waiting for Wi-Fi radio off
93    bool haveLow, haveHigh;
94    uint32_t lastEpoch;     // rail epoch seen on the previous step
95    MonoMs lowSinceMs, highSinceMs;
96    MonoMs ackDeadlineMs, netDeadlineMs;
97  } PowerModeFsm;
98
99  void powerModeInit(PowerModeFsm* f, const PowerModeConfig* cfg);
100 void powerModeStep(PowerModeFsm* f, const PowerModeInputs* in, PowerModeActions* out);
101
102 // Uppercase name ("NORMAL", ...). The contract sends it lowercased.
103 const char* powerModeName(PowerMode m);
```

Return to guided explanation, p. 318

GUIDED SOURCE READING / CHAPTER 13

An acknowledgement must name the lease being released

Power loss requests quiescence. stepQuiescing then waits for evidence that the existing owner has stopped before it grants a replacement.

Original excerpt: src/domain/power_mode.c (p. 328) / lines 138-152. The complete unchanged file follows.

```
138    if (in->releasedGen != f->gen) {
139      if (in->now > f->ackDeadlineMs) f->mode = POWER_MODE_DEGRADED;  // keep waiting: never
  >  two owners
140      return;
141    }
142    f->owner = POWER_WORKER_NONE;
143    if (f->target == POWER_WORKER_NORMAL) {
144      // A glitch during the hand-over restarted the timer: stepSteady restores
145      // the rail once power has been stable again.
146      if (powerStable(f, in->now)) restore(in, out);
147      grant(f, POWER_WORKER_NORMAL, out);
148      out->wifiStart = true;
149      f->netPending = false;
150      f->mode = POWER_MODE_NORMAL;
151      f->phase = PHASE_STEADY;
152      return;
```

The generation comparison comes before changing owner. A release from an earlier transition cannot prove that the present holder is finished. If the deadline expires, mode becomes DEGRADED and the function returns, leaving the ownership condition unsatisfied.

Only a matching acknowledgement clears owner. The target decides whether to return to normal or settle low power. Before a normal grant, stable power can request an epoch-specific rail restore. Grant then increments the generation and produces an action for the coordinator to execute.

The earlier target adjustment handles renewed loss during a return. It redirects the handover toward low power before the release branch chooses its grant. If that check followed the grant, an already-invalid power assumption could authorize the wrong worker for one pass.

This is why timeout is not permission to continue blindly. The state machine can report an incomplete handover while maintaining its most important resource invariant: no second owner before the first releases. The RTOS runs both tasks, but the application protocol controls their access to shared hardware.

ORIGINAL SOURCE / CHAPTER 13 / 1 OF 4

src/domain/power_mode.c

Original lines 1-48 | Code and comments unchanged; long lines wrap visually

powerModeInit (L13-18): Clear state, copy timing policy and select the boot phase; the first step decides the owner.
track (L20-37): Reset stable-return tracking for a new epoch or muted ISR, then track sampled HIGH/LOW durations.

```
#include "domain/power_mode.h"

#include <string.h>

enum {
  PHASE_BOOT = 0,
  PHASE_STEADY,     // one owner, nothing pending (Wi-Fi may still be pending in LOW: see
NET_WAIT)
  PHASE_QUIESCING,  // waiting for the owner's RELEASED
  PHASE_NET_WAIT,   // low-power worker granted, waiting for Wi-Fi radio off
  PHASE_REPORT      // transitions disabled: report the level only
};

void powerModeInit(PowerModeFsm* f, const PowerModeConfig* cfg) {
  memset(f, 0, sizeof(*f));
  f->cfg = *cfg;
  f->mode = POWER_MODE_UNKNOWN;
  f->phase = PHASE_BOOT;
}

static void track(PowerModeFsm* f, const PowerModeInputs* in) {
  // A new epoch is an edge, even one this machine never sampled LOW.
  if (in->railEpoch != f->lastEpoch || in->edgesMissed) f->haveHigh = false;
  f->lastEpoch = in->railEpoch;
  if (in->externalPower) {
    f->haveLow = false;
    if (!f->haveHigh && !in->edgesMissed) {  // stability counts from the first watched tick
      f->haveHigh = true;
      f->highSinceMs = in->now;
    }
  } else {
    f->haveHigh = false;
    if (!f->haveLow) {
      f->haveLow = true;
      f->lowSinceMs = in->now;
    }
  }
}

static bool powerLost(const PowerModeFsm* f, MonoMs now) {
  return f->haveLow && now - f->lowSinceMs >= f->cfg.lossDebounceMs;
}

static bool powerStable(const PowerModeFsm* f, MonoMs now) {
  return f->haveHigh && now - f->highSinceMs >= f->cfg.returnStableMs;
}

static void restore(const PowerModeInputs* in, PowerModeActions* out) {
  out->restoreRail = true;
```

Return to guided explanation, p. 327

ORIGINAL SOURCE / CHAPTER 13 / 2 OF 4

src/domain/power_mode.c

Original lines 49-95 | Code and comments unchanged; long lines wrap visually

restore (L47-50): Pure power-state decisions return ordered actions; no hardware, RTOS calls or long waits occur here.
grant (L52-57): Pure power-state decisions return ordered actions; no hardware, RTOS calls or long waits occur here.

```
  out->restoreEpoch = in->railEpoch;
}

static void grant(PowerModeFsm* f, PowerWorker w, PowerModeActions* out) {
  ++f->gen;
  f->owner = w;
  out->grant = w;
  out->grantGen = f->gen;
}

static void startQuiesce(PowerModeFsm* f, PowerWorker target, MonoMs now, PowerModeActions*
out) {
  out->quiesce = f->owner;
  out->quiesceGen = f->gen;
  f->target = target;
  f->ackDeadlineMs = now + f->cfg.ackTimeoutMs;
  f->phase = PHASE_QUIESCING;
}

static void enterLowPath(PowerModeFsm* f, MonoMs now, PowerModeActions* out) {
  out->wifiStop = true;
  f->netPending = true;
  f->netDeadlineMs = now + f->cfg.networkAckTimeoutMs;
  f->mode = POWER_MODE_TRANSITION_TO_LOW;
}

// The low-power worker holds the lease: LOW_POWER once Wi-Fi is off.
static void settleLow(PowerModeFsm* f, const PowerModeInputs* in) {
  if (f->netPending && in->networkOff) f->netPending = false;
  if (!f->netPending) {
    f->mode = POWER_MODE_LOW_POWER;
    f->phase = PHASE_STEADY;
  } else {
    f->phase = PHASE_NET_WAIT;
    if (in->now > f->netDeadlineMs) f->mode = POWER_MODE_DEGRADED;  // radio not confirmed
off
  }
}

static void stepBoot(PowerModeFsm* f, const PowerModeInputs* in, PowerModeActions* out) {
  if (!f->cfg.transitionsEnabled) {
    grant(f, POWER_WORKER_NORMAL, out);
    if (!in->railValid) restore(in, out);
    f->mode = in->externalPower ? POWER_MODE_NORMAL : POWER_MODE_LOW_POWER;
    f->phase = PHASE_REPORT;
    return;
  }
  if (in->externalPower) {
    grant(f, POWER_WORKER_NORMAL, out);  // the rail is restored once power is stable
```

Return to guided explanation, p. 327

ORIGINAL SOURCE / CHAPTER 13 / 3 OF 4

src/domain/power_mode.c

Original lines 96-142 | Code and comments unchanged; long lines wrap visually

stepBoot (L86-104): Grant normal ownership when power is present, otherwise select the conservative low-power path; respect the transition kill switch.
stepReport (L106-113): Kill-switch mode: keep rail-valid sensing and normal ownership while reporting the debounced sense level.

```
96      f->mode = POWER_MODE_NORMAL;
97      f->phase = PHASE_STEADY;
98      return;
99    }
100   // Unknown or absent power at boot: conservative low-power policy (§13).
101   grant(f, POWER_WORKER_LOW, out);
102   enterLowPath(f, in->now, out);
103   settleLow(f, in);
104 }
105
106 static void stepReport(PowerModeFsm* f, const PowerModeInputs* in, PowerModeActions* out) {
107   if (!in->railValid) restore(in, out);  // no ISR in this mode; keep 5 V sensing on
108   if (powerLost(f, in->now)) {
109     f->mode = POWER_MODE_LOW_POWER;
110   } else if (in->externalPower) {
111     f->mode = POWER_MODE_NORMAL;
112   }
113 }
114
115 static void stepSteady(PowerModeFsm* f, const PowerModeInputs* in, PowerModeActions* out) {
116   if (f->owner == POWER_WORKER_NORMAL) {
117     if (powerLost(f, in->now)) {
118       enterLowPath(f, in->now, out);
119       startQuiesce(f, POWER_WORKER_LOW, in->now, out);
120     } else if (!in->railValid && powerStable(f, in->now)) {
121       restore(in, out);  // a glitch the ISR saw, now stable again
122     }
123     return;
124   }
125   if (powerStable(f, in->now)) {  // owner LOW
126     f->mode = POWER_MODE_TRANSITION_TO_NORMAL;
127     startQuiesce(f, POWER_WORKER_NORMAL, in->now, out);
128   }
129 }
130
131 static void stepQuiescing(PowerModeFsm* f, const PowerModeInputs* in, PowerModeActions* out)
  > {
132   // A new loss before the normal worker is granted aborts the return (§13).
133   if (f->target == POWER_WORKER_NORMAL && !in->externalPower) {
134     f->target = POWER_WORKER_LOW;
135     f->mode = POWER_MODE_TRANSITION_TO_LOW;
136     // Wi-Fi was never restarted on this path: it is still off.
137   }
138   if (in->releasedGen != f->gen) {
139     if (in->now > f->ackDeadlineMs) f->mode = POWER_MODE_DEGRADED;  // keep waiting: never
  > two owners
140     return;
141   }
142   f->owner = POWER_WORKER_NONE;
```

Return to guided explanation, p. 327

ORIGINAL SOURCE / CHAPTER 13 / 4 OF 4

src/domain/power_mode.c

Original lines 143-191 | Code and comments unchanged; long lines wrap visually

stepQuiescing (L131-157): Require the current release generation before granting; report DEGRADED on timeout and abort a return after renewed loss.
stepNetWait (L159-167): Wait for Wi-Fi OFF to settle low power, or initiate a return if supply is stable again.

```
143   if (f->target == POWER_WORKER_NORMAL) {
144     // A glitch during the hand-over restarted the timer: stepSteady restores
145     // the rail once power has been stable again.
146     if (powerStable(f, in->now)) restore(in, out);
147     grant(f, POWER_WORKER_NORMAL, out);
148     out->wifiStart = true;
149     f->netPending = false;
150     f->mode = POWER_MODE_NORMAL;
151     f->phase = PHASE_STEADY;
152     return;
153   }
154   grant(f, POWER_WORKER_LOW, out);
155   f->mode = POWER_MODE_TRANSITION_TO_LOW;
156   settleLow(f, in);
157 }
158 
159 static void stepNetWait(PowerModeFsm* f, const PowerModeInputs* in, PowerModeActions* out) {
160   if (powerStable(f, in->now)) {  // power back before Wi-Fi confirmed: return anyway
161     f->netPending = false;
162     f->mode = POWER_MODE_TRANSITION_TO_NORMAL;
163     startQuiesce(f, POWER_WORKER_NORMAL, in->now, out);
164     return;
165   }
166   settleLow(f, in);
167 }
168 
169 void powerModeStep(PowerModeFsm* f, const PowerModeInputs* in, PowerModeActions* out) {
170   memset(out, 0, sizeof(*out));
171   track(f, in);
172   switch (f->phase) {
173     case PHASE_BOOT: stepBoot(f, in, out); break;
174     case PHASE_REPORT: stepReport(f, in, out); break;
175     case PHASE_STEADY: stepSteady(f, in, out); break;
176     case PHASE_QUIESCING: stepQuiescing(f, in, out); break;
177     case PHASE_NET_WAIT: stepNetWait(f, in, out); break;
178     default: break;
179   }
180 }
181 
182 const char* powerModeName(PowerMode m) {
183   switch (m) {
184     case POWER_MODE_NORMAL: return "NORMAL";
185     case POWER_MODE_TRANSITION_TO_LOW: return "TRANSITION_TO_LOW";
186     case POWER_MODE_LOW_POWER: return "LOW_POWER";
187     case POWER_MODE_TRANSITION_TO_NORMAL: return "TRANSITION_TO_NORMAL";
188     case POWER_MODE_DEGRADED: return "DEGRADED";
189     default: return "UNKNOWN";
190   }
191 }
```

Return to guided explanation, p. 327

ORIGINAL SOURCE / CHAPTER 13 / 1 OF 1

src/services/power_coordinator.h

Original lines 1-43 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // PowerCoordinator: power-mode state machine and acquisition lease owner.
2  // Work package WP9 (Stage 6). Architecture §13, §14, §15.
3  //
4  // Task "PowerCoord" (cfgTaskSpec: core 0, priority 5, 4 KiB). The decisions
5  // live in domain/power_mode (host-tested); this service only feeds it and
6  // carries out its actions:
7  //   sense    GPIO8 (external 5 V through 220 / 2 x 220 Ohm, measured 25 Oct
8  //            2026, BOARD_POWER_SENSE_VERIFIED). The edge ISR runs on core 1
9  //            (the GPIO ISR service is installed there by AcqNormal for the
10 //            HC-SR04 echo): it invalidates the rail (rail_state, new epoch),
11 //            disables its own interrupt (a loss chatters 479-928 edges) and
12 //            notifies this task. The task samples the level every
13 //            POWER_TICK_MS and re-arms the interrupt once the level has been
14 //            quiet for CFG_POWER_LOSS_DEBOUNCE_MS.
15 //   lease    acquisitionRequestQuiesce / acquisitionReleased / acquisitionGrant
16 //   network  wifiMqttRequestStop / wifiMqttRadioOff / wifiMqttRequestStart
17 //   rail     railStateRestore (only this task restores validity)
18 // Debounce: CFG_POWER_LOSS_DEBOUNCE_MS (50 ms, app_config.h) is the mode
19 // debounce; BOARD_POWER_SENSE_QUALIFY_MS (20 ms) was the bench qualification
20 // window and is not used here. The measured bounce is 5-14 ms.
21 //
22 // Safety stays on core 1 in every mode (D14, 28 Oct 2026): unlike §15, core 1
23 // is not idle in low power (Safety and SerialLogger keep running there; only
24 // AcqNormal is blocked). CPU frequency and light sleep are unchanged (§15:
25 // later decision, measure first).
26 //
27 // POLICY_POWER_TRANSITIONS_ENABLED 0 (policy_config.h) is the kill switch:
28 // no ISR, every sensor on the normal worker, the rail kept valid, and the
29 // mode only reports the debounced sense level.
30 #pragma once
31 #include <stdbool.h>
32 #include <stdint.h>
33
34 #include "domain/power_mode.h"
35
36 // Starts the task. On failure the normal worker is granted every sensor and
37 // the rail is restored, so sensing runs as before WP9.
38 bool powerCoordinatorInit(void);
39
40 PowerMode powerModeCurrent(void);  // zero wait, any task
41
42 // For Supervisor. NULL before init.
43 void* powerCoordinatorTaskHandle(void);
```

Return to guided explanation, p. 318

GUIDED SOURCE READING / CHAPTER 13

The interrupt records loss before waking the decision task

The pure power state machine needs observations. The GPIO8 ISR supplies the urgent power-history fact and leaves the longer transition policy to PowerCoordinator.

Original excerpt: src/services/power_coordinator.c (p. 334) / lines 39-47. The complete unchanged file follows.

```
39  static void onSenseEdge(void) {
40    railStateInvalidateFromIsr();
41    gpio_intr_disable((gpio_num_t)BOARD_PIN_EXTERNAL_POWER_SENSE);
42    s_isrArmed = false;
43    s_edges = s_edges + 1u;
44    BaseType_t woken = pdFALSE;
45    if (s_task != NULL) vTaskNotifyGiveFromISR(s_task, &woken);
46    portYIELD_FROM_ISR(woken);
47  }
```

railStateInvalidateFromIsr runs first. Any consumer checking supply validity after that update must treat the old epoch as invalid, even before the power task finishes its 50 ms level qualification.

The ISR disables its own pin interrupt and marks it unarmed, limiting chatter work. It increments a diagnostic count, initializes woken to false and uses the ISR-specific notification API. The API can set woken if unblocking a higher-priority task requires a scheduling decision.

portYIELD_FROM_ISR requests that scheduling decision at interrupt exit. It does not run powerTask as a normal function call inside the handler, and it does not guarantee a complete power transition before other tasks execute. The task still follows priority, core affinity and its acknowledgement protocol.

PowerCoordinator later polls the sampled level, rearms after qualification and executes returned actions outside the ISR. No flash write, sensor conversion or relay policy belongs in this handler because those operations would extend interrupt latency and mix incompatible ownership roles.

ORIGINAL SOURCE / CHAPTER 13 / 1 OF 4

src/services/power_coordinator.c

Original lines 1-46 | Code and comments unchanged; long lines wrap visually

senseHigh (L34-34): The power task converts GPIO/rail observations into pure decisions and executes quiesce, restore, grant and Wi-Fi actions.
onSenseEdge (L39-47): ISR: invalidate rail history, mute further chatter, increment diagnostics and wake PowerCoordinator.

```
#include "services/power_coordinator.h"

#include <Arduino.h>
#include <driver/gpio.h>
#include <esp_task_wdt.h>
#include <freertos/FreeRTOS.h>
#include <freertos/task.h>

#include "app_config.h"
#include "board_profile.h"
#include "diagnostics/logger.h"
#include "diagnostics/progress.h"
#include "hal/clock.h"
#include "policy_config.h"
#include "services/acquisition.h"
#include "services/rail_state.h"
#include "transports/wifi_mqtt.h"

#define POWER_TICK_MS 10u          // level sampling while anything is pending
#define POWER_IDLE_TICK_MS 100u    // steady and armed: the ISR wakes us
#define POWER_HAL_WAIT_MS 2000u    // AcqNormal installs the GPIO ISR service on core 1

static StaticTask_t s_tcb;
static StackType_t s_stack[4 * 1024];  // cfgTaskSpec(TASK_ID_POWER_COORDINATOR)->stackBytes
static TaskHandle_t s_task = NULL;

static portMUX_TYPE s_modeMux = portMUX_INITIALIZER_UNLOCKED;
static PowerMode s_mode = POWER_MODE_UNKNOWN;

static PowerModeFsm s_fsm;  // task-private
static volatile bool s_isrArmed;
static volatile uint32_t s_edges;  // ISR count since boot (diagnostics)

static inline bool senseHigh(void) { return digitalRead(BOARD_PIN_EXTERNAL_POWER_SENSE) ==
HIGH; }

// Core 1 (the GPIO ISR service lives there). The Arduino service is installed
// without ESP_INTR_FLAG_IRAM, so this never runs while the flash cache is off.
// It does the minimum: invalidate the 5 V epoch, mute itself, wake the task.
static void onSenseEdge(void) {
  railStateInvalidateFromIsr();
  gpio_intr_disable((gpio_num_t)BOARD_PIN_EXTERNAL_POWER_SENSE);
  s_isrArmed = false;
  s_edges = s_edges + 1u;
  BaseType_t woken = pdFALSE;
  if (s_task != NULL) vTaskNotifyGiveFromISR(s_task, &woken);
  portYIELD_FROM_ISR(woken);
```

Return to guided explanation, p. 333

ORIGINAL SOURCE / CHAPTER 13 / 2 OF 4

src/services/power_coordinator.c

Original lines 47-89 | Code and comments unchanged; long lines wrap visually

onSenseEdge (L39-47): ISR: invalidate rail history, mute further chatter, increment diagnostics and wake PowerCoordinator.
setMode (L49-55): The power task converts GPIO/rail observations into pure decisions and executes quiesce, restore, grant and Wi-Fi actions.

```
47  }
48
49  static void setMode(PowerMode m) {
50    portENTER_CRITICAL(&s_modeMux);
51    const PowerMode old = s_mode;
52    s_mode = m;
53    portEXIT_CRITICAL(&s_modeMux);
54    if (old != m) logPrintf(LOG_LEVEL_INFO, "power: %s -> %s", powerModeName(old),
 >  powerModeName(m));
55  }
56
57  static void execute(const PowerModeActions* a) {
58    if (a->quiesce != POWER_WORKER_NONE) {
59      acquisitionRequestQuiesce(a->quiesceGen);
60      logPrintf(LOG_LEVEL_INFO, "power: quiesce %s worker (gen %lu)", a->quiesce ==
 >  POWER_WORKER_NORMAL ? "normal" : "low",
61                (unsigned long)a->quiesceGen);
62    }
63    if (a->restoreRail) {
64      railStateRestore(a->restoreEpoch);  // a newer edge keeps the rail invalid
65      const RailState rail = railStateRead();
66      if (rail.valid5v) {
67        logPrintf(LOG_LEVEL_INFO, "power: 5 V rail valid (epoch %lu)", (unsigned
 >  long)rail.epoch);
68      } else {
69        logPrintf(LOG_LEVEL_INFO, "power: 5 V restore skipped, epoch %lu -> %lu", (unsigned
 >  long)a->restoreEpoch,
70                  (unsigned long)rail.epoch);
71      }
72    }
73    if (a->grant != POWER_WORKER_NONE) {
74      const bool normal = a->grant == POWER_WORKER_NORMAL;
75      const bool all = normal || !POLICY_POWER_TRANSITIONS_ENABLED;
76      acquisitionGrant(a->grant, a->grantGen, all ? SENSOR_MASK_ALL : ACQ_LOW_POWER_SENSORS);
77      logPrintf(LOG_LEVEL_INFO, "power: grant %s worker (gen %lu)", normal ? "normal" : "low",
78                (unsigned long)a->grantGen);
79    }
80    if (a->wifiStop) wifiMqttRequestStop();
81    if (a->wifiStart) wifiMqttRequestStart();
82  }
83
84  // After a mode change the edge storm is over once the level has held for the
85  // debounce time; only then may the interrupt fire again.
86  static void maybeRearm(bool high, bool* lastHigh, MonoMs* sinceMs, MonoMs now) {
87    if (high != *lastHigh) {
88      *lastHigh = high;
89      *sinceMs = now;
```

Return to guided explanation, p. 333

ORIGINAL SOURCE / CHAPTER 13 / 3 OF 4

src/services/power_coordinator.c

Original lines 90-132 | Code and comments unchanged; long lines wrap visually

maybeRearm (L86-95): Track stable sampled level and re-enable the sense interrupt after its qualification interval.
armIsr (L97-107): Wait a bounded time for core1 HAL setup, then attach to the existing GPIO interrupt service.

```
    return;
  }
  if (s_isrArmed || now - *sinceMs < CFG_POWER_LOSS_DEBOUNCE_MS) return;
  s_isrArmed = true;
  gpio_intr_enable((gpio_num_t)BOARD_PIN_EXTERNAL_POWER_SENSE);  // on the service's core (1)
}

static bool armIsr(void) {
  // The service must already exist on core 1, or attachInterrupt() would
  // install it on this core (0) and move the HC-SR04 echo ISR next to the radios.
  for (uint32_t waited = 0; !acquisitionHalsReady(); waited += POWER_TICK_MS) {
    if (waited >= POWER_HAL_WAIT_MS) return false;
    vTaskDelay(pdMS_TO_TICKS(POWER_TICK_MS));
  }
  attachInterrupt(digitalPinToInterrupt(BOARD_PIN_EXTERNAL_POWER_SENSE), onSenseEdge,
CHANGE);
  s_isrArmed = true;
  return true;
}

static void powerTask(void* arg) {
  (void)arg;
  if (esp_task_wdt_add(NULL) != ESP_OK) logPrintf(LOG_LEVEL_ERROR, "power: task watchdog add
failed");
  bool isr = false;
  if (POLICY_POWER_TRANSITIONS_ENABLED) {
    isr = armIsr();
    if (!isr) logPrintf(LOG_LEVEL_ERROR, "power: GPIO ISR service not ready - polling the
sense pin only");
  }
  bool lastHigh = senseHigh();
  MonoMs sinceMs = monoNowMs();
  for (;;) {
    const MonoMs now = monoNowMs();
    const RailState rail = railStateRead();  // one coherent {valid, epoch} per tick
    const bool high = senseHigh();
    const PowerModeInputs in = {now, high, rail.valid5v, acquisitionReleasedGeneration(),
wifiMqttRadioOff(),
                                rail.epoch, isr && !s_isrArmed};
    PowerModeActions a;
    powerModeStep(&s_fsm, &in, &a);
    execute(&a);
    setMode(s_fsm.mode);
    if (isr) maybeRearm(high, &lastHigh, &sinceMs, now);

    progressMark(TASK_ID_POWER_COORDINATOR);
    esp_task_wdt_reset();
```

Return to guided explanation, p. 333

ORIGINAL SOURCE / CHAPTER 13 / 4 OF 4

src/services/power_coordinator.c

Original lines 133-172 | Code and comments unchanged; long lines wrap visually

powerTask (L109-137): Sample coherent rail state and level, step the pure machine, execute actions, rearm the ISR and wait with a bounded timeout.
powerCoordinatorInit (L139-163): Configure the sense input and pure FSM, create the task, or grant fallback normal ownership if task creation fails.

```
133     const bool busy = s_fsm.mode != POWER_MODE_NORMAL && s_fsm.mode != POWER_MODE_LOW_POWER;
134     const bool settling = !high || !in.railValid || (isr && !s_isrArmed);
135     ulTaskNotifyTake(pdTRUE, pdMS_TO_TICKS(busy || settling ? POWER_TICK_MS :
 >  POWER_IDLE_TICK_MS));
136   }
137 }
138
139 bool powerCoordinatorInit(void) {
140   if (s_task != NULL) return true;
141   pinMode(BOARD_PIN_EXTERNAL_POWER_SENSE, INPUT);  // no pull: the 440 Ohm leg defines LOW
142   const PowerModeConfig cfg = {CFG_POWER_LOSS_DEBOUNCE_MS, CFG_POWER_RETURN_STABLE_MS,
 >  CFG_TRANSITION_ACK_MS,
143                                CFG_NETWORK_SHUTDOWN_ACK_MS, POLICY_POWER_TRANSITIONS_ENABLED
 >  != 0};
144   powerModeInit(&s_fsm, &cfg);
145
146   const TaskSpec* spec = cfgTaskSpec(TASK_ID_POWER_COORDINATOR);
147   if (spec->stackBytes != sizeof(s_stack)) {
148     logPrintf(LOG_LEVEL_WARN, "power: stack %u B, task table says %lu B",
 >  (unsigned)sizeof(s_stack),
149               (unsigned long)spec->stackBytes);
150   }
151   s_task = xTaskCreateStaticPinnedToCore(powerTask, spec->name, sizeof(s_stack), NULL,
 >  spec->priority, s_stack,
152                                          &s_tcb, spec->core);
153   if (s_task == NULL) {
154     // Never leave sensing without an owner: run as before WP9.
155     railStateRestore(railStateRead().epoch);
156     acquisitionGrant(POWER_WORKER_NORMAL, 1, SENSOR_MASK_ALL);
157     return false;
158   }
159   logPrintf(LOG_LEVEL_INFO, "power: started (core %u, prio %u), external power %s,
 >  transitions %s",
160             (unsigned)spec->core, (unsigned)spec->priority, senseHigh() ? "present" :
 >  "ABSENT",
161             POLICY_POWER_TRANSITIONS_ENABLED ? "on" : "OFF (kill switch)");
162   return true;
163 }
164
165 PowerMode powerModeCurrent(void) {
166   portENTER_CRITICAL(&s_modeMux);
167   const PowerMode m = s_mode;
168   portEXIT_CRITICAL(&s_modeMux);
169   return m;
170 }
171
172 void* powerCoordinatorTaskHandle(void) { return s_task; }
```

Return to guided explanation, p. 333

ORIGINAL SOURCE / CHAPTER 13 / 1 OF 13

test/bringup/power_sense_main.c

Original lines 1-49 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // Bring-up entry point for the external-power sense (WP9 gate, §13, §20).
2  //   pio run -e test_power_sense -t upload -t monitor
3  // Ext. 5 V + rail -> 220 Ohm -> GPIO8 (ADC1_CH7) -> 2 x 220 Ohm -> GND.
4  // Nominal 3.33 V at 5.00 V, 0 V on battery. No hysteresis on the divider.
5  // Replaces src/main.c in this env only. No production driver is involved: this
6  // is the commissioning tool that decides BOARD_POWER_SENSE_VERIFIED.
7  //
8  // GPIO8 is read two ways at the same time:
9  //  - digital: CHANGE interrupt, every edge timestamped in us (esp_timer);
10 //  - analog: ADC1_CH7 sampled every 1 ms into an 8 s ring buffer.
11 // The ADC is configured once through the IDF driver and read with
12 // adc1_get_raw(). Arduino's analogRead*() is NOT used: each call does
13 // pinMode(pin, ANALOG), which disables the digital input buffer and leaves the
14 // interrupt deaf. The status line checks that both paths still agree
15 // ("isr" and "mismatch" counters); if they do not, press 'a' for
16 // digital-only mode and repeat the steps.
17 //
18 // The ADC clips at ~3050 mV (11 dB), so it cannot measure the 3.33 V high
19 // level. It only shows ">= VIH (clipped)". The exact high level needs a
20 // multimeter at GPIO8.
21 //
22 // Output: a MODE banner (NORMAL / LOW-POWER) each time the level has been
23 // stable for BOARD_POWER_SENSE_QUALIFY_MS, as the WP9 power task will decide,
24 // then indented result / timing / bounce lines once the pin has settled.
25 // A heartbeat repeats the mode every 30 s; problems print as "WARNING".
26 //
27 // Procedure:
28 //  1. External 5 V plugged in: banner "MODE: NORMAL". With a multimeter,
29 //     GPIO8 must read >= 2.475 V and <= 3.6 V (VDD + 0.3 V).
30 //  2. Unplug the external 5 V: banner "MODE: LOW-POWER", then result PASS.
31 //  3. Plug it back in: banner "MODE: NORMAL", then result PASS.
32 //  4. Repeat 2-3 at least 5 times, once with the sensors working (pH, TDS,
33 //     HC-SR04 loaded) and once by pulling the cable slowly / wiggling it.
34 //  5. Press 'd' for the table of all transitions and paste it back.
35 // Keys: v = edge details on/off, d = transition table, a = ADC on/off,
36 // r = reset counters, h = help.
37 // The monitor opens with DTR/RTS released (platformio.ini), so reconnecting
38 // after an unplug does not reset the board and the RAM history survives.
39 #include <Arduino.h>
40 #include <driver/adc.h>
41 #include <esp_adc_cal.h>
42 #include <esp_timer.h>
43 #include <fcntl.h>
44 #include <soc/gpio_reg.h>
45 #include <stdio.h>
46 #include <string.h>
47 #include <unistd.h>
48
49 #include "board_profile.h"
```

Return to guided explanation, p. 318

ORIGINAL SOURCE / CHAPTER 13 / 2 OF 13

test/bringup/power_sense_main.c

Original lines 50-98 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
50  #include "hal/arduino_entry.h"
51
52  #define PS_PIN BOARD_PIN_EXTERNAL_POWER_SENSE
53  #define PS_ADC_CHANNEL ADC1_CHANNEL_7
54
55  // ESP32-S3 input thresholds at VDD = 3.3 V (architecture §5, [L14]).
56  #define PS_VIH_MV 2475u
57  #define PS_VIL_MV 825u
58  #define PS_VMAX_MV 3600u      // VDD + 0.3 V
59  #define PS_CLIP_MV 3050u      // same ceiling as hal/adc_channel.c
60  #define PS_ZERO_MV 50u        // "fully discharged"
61
62  #define PS_SAMPLE_PERIOD_MS 1
63  #define PS_SAMPLE_RING 8192u      // 8.2 s at 1 kHz, power of two
64  #define PS_EDGE_RING 1024u        // power of two
65  #define PS_PRE_TRIGGER_MS 300u    // ADC history kept before the trigger
66  #define PS_EDGE_QUIET_MS 300u     // no edge for this long = digital settled
67  #define PS_STABLE_WINDOW_MS 200u  // ADC span below PS_STABLE_SPAN_MV = settled
68  #define PS_STABLE_SPAN_MV 30u
69  #define PS_TRIGGER_DELTA_MV 200u  // ADC move that opens a transition
70  #define PS_TRANSITION_TIMEOUT_MS 7000u
71  #define PS_STATUS_PERIOD_MS 30000u  // heartbeat
72  #define PS_EDGES_LISTED 12u
73  #define PS_HISTORY 48u
74  #define PS_QUALIFY_US (BOARD_POWER_SENSE_QUALIFY_MS * 1000u)
75
76  typedef struct Sample {
77    uint32_t tUs;    // esp_timer low 32 bits (wrap-safe for intervals < 71 min)
78    uint16_t mv;     // 0 when the ADC is off
79    uint8_t before;  // GPIO level just before the conversion
80    uint8_t after;   // GPIO level just after the conversion
81  } Sample;
82
83  typedef struct Edge {
84    uint32_t tUs;
85    uint8_t level;  // level read at ISR entry
86  } Edge;
87
88  typedef struct Transition {
89    bool rising;
90    uint32_t tStartUs;      // first edge or ADC departure
91    uint32_t edges;         // ISR edges in the transition
92    uint32_t bounceUs;      // first edge -> last edge
93    uint32_t minGapUs;      // shortest interval between two edges
94    uint8_t endLevel;
95    bool adcValid;
96    uint16_t mvBefore, mvAfter;
97    int32_t tVihMs, tVilMs, tEndMs;  // from ADC departure; -1 = not reached
98    uint16_t flipMinMv, flipMaxMv;   // ADC value where the sampler saw GPIO flip
```

Return to guided explanation, p. 318

ORIGINAL SOURCE / CHAPTER 13 / 3 OF 13

test/bringup/power_sense_main.c

Original lines 99-147 | Code and comments unchanged; long lines wrap visually

gpioLevel (L125-127): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.
adcReadMv (L138-142): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.

```
99    bool timedOut;
100   bool dip;     // ADC excursion that recovered with no GPIO edge
101   bool outage;  // GPIO left its level and came back before the pin settled
102 } Transition;
103
104 static Sample s_samples[PS_SAMPLE_RING];
105 static volatile uint32_t s_sampleCount;
106 static volatile bool s_adcOn = true;
107
108 static Edge s_edges[PS_EDGE_RING];
109 static volatile uint32_t s_edgeHead;   // written by the ISR only
110 static volatile uint32_t s_edgeTotal;  // never wraps the ring
111 static volatile uint32_t s_lastEdgeUs;
112 static uint32_t s_edgeTail;
113 static bool s_verbose;
114
115 static esp_adc_cal_characteristics_t s_cal;
116
117 // Health of the shared-pin setup.
118 static volatile uint32_t s_samplerFlips;  // level changes seen by the sampler
119 static volatile uint32_t s_adcDisturbed;  // level differs before/after a conversion
120 static uint32_t s_mismatch;               // steady GPIO level disagrees with mV
121
122 static Transition s_history[PS_HISTORY];
123 static uint32_t s_historyCount;
124
125 static inline uint8_t gpioLevel(void) {
126   return (uint8_t)((REG_READ(GPIO_IN_REG) >> PS_PIN) & 1u);
127 }
128
129 static void IRAM_ATTR onEdge(void) {
130   const uint32_t head = s_edgeHead;
131   s_edges[head & (PS_EDGE_RING - 1u)].tUs = (uint32_t)esp_timer_get_time();
132   s_edges[head & (PS_EDGE_RING - 1u)].level = gpioLevel();
133   s_lastEdgeUs = s_edges[head & (PS_EDGE_RING - 1u)].tUs;
134   s_edgeHead = head + 1u;
135   s_edgeTotal++;
136 }
137
138 static uint16_t adcReadMv(void) {
139   const int raw = adc1_get_raw(PS_ADC_CHANNEL);
140   if (raw < 0) return 0;
141   return (uint16_t)esp_adc_cal_raw_to_voltage((uint32_t)raw, &s_cal);
142 }
143
144 // ---- sampler (core 0): 1 kHz ADC + GPIO level --------------------------------
145 static void samplerTask(void* arg) {
146   (void)arg;
147   TickType_t lastWake = xTaskGetTickCount();
```

Return to guided explanation, p. 318

ORIGINAL SOURCE / CHAPTER 13 / 4 OF 13

test/bringup/power_sense_main.c

Original lines 148-194 | Code and comments unchanged; long lines wrap visually

samplerTask (L145-162): Power diagnostic: sample ADC plus digital levels into a bounded history ring.
sampleAt (L165-167): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.

```
148   uint8_t prev = gpioLevel();
149   for (;;) {
150     vTaskDelayUntil(&lastWake, pdMS_TO_TICKS(PS_SAMPLE_PERIOD_MS));
151     Sample s;
152     s.tUs = (uint32_t)esp_timer_get_time();
153     s.before = gpioLevel();
154     s.mv = s_adcOn ? adcReadMv() : 0;
155     s.after = gpioLevel();
156     if (s.before != s.after) s_adcDisturbed++;
157     if (s.after != prev) s_samplerFlips++;
158     prev = s.after;
159     s_samples[s_sampleCount & (PS_SAMPLE_RING - 1u)] = s;
160     s_sampleCount = s_sampleCount + 1u;
161   }
162 }
163
164 // ---- analysis helpers (analyzer task only) -----------------------------------
165 static const Sample* sampleAt(uint32_t index) {
166   return &s_samples[index & (PS_SAMPLE_RING - 1u)];
167 }
168
169 // Index of the first sample taken at or after tUs, searching back from 'last'.
170 static uint32_t sampleIndexAt(uint32_t tUs, uint32_t last) {
171   uint32_t i = last;
172   const uint32_t oldest = (last >= PS_SAMPLE_RING - 16u) ? last - (PS_SAMPLE_RING - 16u) :
  >
0u;
173   while (i > oldest && (int32_t)(sampleAt(i - 1u)->tUs - tUs) >= 0) --i;
174   return i;
175 }
176
177 static void spanMv(uint32_t from, uint32_t to, uint16_t* lo, uint16_t* hi, uint32_t* mean) {
178   uint16_t mn = 0xFFFF, mx = 0;
179   uint64_t sum = 0;
180   for (uint32_t i = from; i < to; ++i) {
181     const uint16_t v = sampleAt(i)->mv;
182     if (v < mn) mn = v;
183     if (v > mx) mx = v;
184     sum += v;
185   }
186   const uint32_t n = to > from ? to - from : 1u;
187   if (lo) *lo = (to > from) ? mn : 0;
188   if (hi) *hi = mx;
189   if (mean) *mean = (uint32_t)(sum / n);
190 }
191
192 static const char* bandName(uint32_t mv) {
193   if (mv >= PS_VIH_MV) return "HIGH";
194   if (mv <= PS_VIL_MV) return "LOW";
```

Return to guided explanation, p. 318

ORIGINAL SOURCE / CHAPTER 13 / 5 OF 13

test/bringup/power_sense_main.c

Original lines 195-242 | Code and comments unchanged; long lines wrap visually

bandName (L192-196): Translate an identity/state into a human-readable label for diagnostics.
msSince (L198-200): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.

```
195   return "UNDEFINED";
196 }
197
198 static int32_t msSince(uint32_t fromUs, uint32_t toUs) {
199   return (int32_t)((toUs - fromUs) / 1000u);
200 }
201
202 // "mm:ss.s" since boot, for the banners (the monitor may not add timestamps).
203 static void printUptime(void) {
204   const uint32_t ds = (uint32_t)(esp_timer_get_time() / 100000);
205   printf("%02lu:%02lu.%lu", (unsigned long)(ds / 600u), (unsigned long)(ds / 10u % 60u),
206          (unsigned long)(ds % 10u));
207 }
208
209 static void printModeBanner(uint8_t level) {
210   printf("\n+-------------------------------------------------------+\n");
211   printf("|  MODE: %-10s %-27s ", level ? "NORMAL" : "LOW-POWER",
212          level ? "external 5 V present" : "external 5 V LOST");
213   printUptime();
214   printf("  |\n+-------------------------------------------------------+\n");
215 }
216
217 // "after N ms", or "not reached" for -1.
218 static void printAfter(int32_t ms) {
219   if (ms < 0) {
220     printf("not reached");
221   } else {
222     printf("after %ld ms", (long)ms);
223   }
224 }
225
226 static void printSummary(const Transition* t) {
227   bool pass = t->endLevel == (t->rising ? 1 : 0);
228   if (t->adcValid) pass = pass && (t->rising ? t->mvAfter >= PS_VIH_MV : t->mvAfter <=
  > PS_VIL_MV);
229
230   printf("  result  ");
231   if (!t->adcValid) {
232     printf("ADC off");
233   } else if (t->mvAfter >= PS_CLIP_MV) {
234     printf("pin %u -> >=%u mV (clipped)", t->mvBefore, PS_CLIP_MV);
235   } else if (t->mvBefore >= PS_CLIP_MV) {
236     printf("pin >=%u -> %u mV", PS_CLIP_MV, t->mvAfter);
237   } else {
238     printf("pin %u -> %u mV", t->mvBefore, t->mvAfter);
239   }
240   printf(", GPIO %u   %s\n", t->endLevel, pass ? "PASS" : "FAIL");
241   if (!pass) {
242     printf("  WARNING: expected GPIO %u and %s\n", t->rising ? 1 : 0,
```

Return to guided explanation, p. 318

ORIGINAL SOURCE / CHAPTER 13 / 6 OF 13

test/bringup/power_sense_main.c

Original lines 243-288 | Code and comments unchanged; long lines wrap visually

printSummary (L226-279): Present already-collected diagnostic information; this does not make the measurement more valid.
reportTransition (L282-391): Estimate departure, threshold crossings and bounce; distinguish dips and short outages from one-way transitions.

```
243             t->rising ? "pin >= 2475 mV" : "pin <= 825 mV (rail back-fed or not switched
  >   off?)");
244     }
245
246     if (t->adcValid && t->rising) {
247       printf("  timing  > %u mV ", PS_VIH_MV);
248       printAfter(t->tVihMs);
249       printf("\n");
250     } else if (t->adcValid) {
251       printf("  timing  < %u mV ", PS_VIH_MV);
252       printAfter(t->tVihMs);
253       printf(", < %u mV ", PS_VIL_MV);
254       printAfter(t->tVilMs);
255       if (t->tEndMs >= 0) {
256         printf(", ~0 V ");
257         printAfter(t->tEndMs);
258       } else {
259         printf(", settled at %u mV", t->mvAfter);
260       }
261       printf("\n");
262     }
263
264     printf("  bounce  %lu edge(s) in ", (unsigned long)t->edges);
265     if (t->bounceUs < 1000u) {
266       printf("%lu us", (unsigned long)t->bounceUs);
267     } else {
268       printf("%lu.%lu ms", (unsigned long)(t->bounceUs / 1000u),
269              (unsigned long)(t->bounceUs % 1000u / 100u));
270     }
271     if (t->bounceUs >= PS_QUALIFY_US) {
272       printf("\n  WARNING: longer than the %lu ms qualify window\n",
273              (unsigned long)BOARD_POWER_SENSE_QUALIFY_MS);
274     } else {
275       printf(" -> filtered by the %lu ms qualify window\n",
276              (unsigned long)BOARD_POWER_SENSE_QUALIFY_MS);
277     }
278     if (t->timedOut) printf("  WARNING: pin did not settle within 7 s\n");
279 }
280
281 // Analyse samples [first, last) and the edges collected for this transition.
282 static void reportTransition(Transition* t, const Edge* edges, uint32_t nListed, uint32_t
  >   first,
283                              uint32_t last) {
284     t->adcValid = s_adcOn;
285     t->tVihMs = t->tVilMs = t->tEndMs = -1;
286     t->flipMinMv = 0xFFFF;
287     t->flipMaxMv = 0;
288
```

Return to guided explanation, p. 318

ORIGINAL SOURCE / CHAPTER 13 / 7 OF 13

test/bringup/power_sense_main.c

Original lines 289-337 | Code and comments unchanged; long lines wrap visually

reportTransition (L282-391): Estimate departure, threshold crossings and bounce; distinguish dips and short outages from one-way transitions.

```
  uint32_t departUs = t->tStartUs;
  if (t->adcValid && last > first + 2u * PS_STABLE_WINDOW_MS) {
    uint32_t meanBefore, meanAfter;
    spanMv(first, first + PS_PRE_TRIGGER_MS / 2u, NULL, NULL, &meanBefore);
    spanMv(last - PS_STABLE_WINDOW_MS, last, NULL, NULL, &meanAfter);
    t->mvBefore = (uint16_t)meanBefore;
    t->mvAfter = (uint16_t)meanAfter;
    t->rising = meanAfter > meanBefore;

    // Departure: first sample more than 50 mV away from the level before.
    uint32_t depart = first;
    for (uint32_t i = first; i < last; ++i) {
      const int32_t d = (int32_t)sampleAt(i)->mv - (int32_t)meanBefore;
      if ((t->rising && d > 50) || (!t->rising && d < -50)) {
        depart = i;
        break;
      }
    }
    departUs = sampleAt(depart)->tUs;
    for (uint32_t i = depart; i < last; ++i) {
      const Sample* s = sampleAt(i);
      const int32_t ms = msSince(departUs, s->tUs);
      if (t->rising) {
        if (t->tVilMs < 0 && s->mv > PS_VIL_MV) t->tVilMs = ms;
        if (t->tVihMs < 0 && s->mv >= PS_VIH_MV) t->tVihMs = ms;
        if (t->tEndMs < 0 && s->mv >= PS_CLIP_MV) t->tEndMs = ms;
      } else {
        if (t->tVihMs < 0 && s->mv < PS_VIH_MV) t->tVihMs = ms;
        if (t->tVilMs < 0 && s->mv <= PS_VIL_MV) t->tVilMs = ms;
        if (t->tEndMs < 0 && s->mv <= PS_ZERO_MV) t->tEndMs = ms;
      }
      if (i > first && s->before != sampleAt(i - 1u)->after) {
        if (s->mv < t->flipMinMv) t->flipMinMv = s->mv;
        if (s->mv > t->flipMaxMv) t->flipMaxMv = s->mv;
      }
    }
  } else if (t->edges > 0) {
    t->rising = t->endLevel == 1;
  }

  // ADC moved >= PS_TRIGGER_DELTA_MV and came back without a GPIO edge, e.g.
  // contact bounce of the plug. Not a power transition: no verdict.
  if (t->adcValid && t->edges == 0 && !t->timedOut &&
      strcmp(bandName(t->mvBefore), bandName(t->mvAfter)) == 0) {
    uint16_t dipLo, dipHi;
    spanMv(first, last, &dipLo, &dipHi, NULL);
    printf("  note    pin dipped to %u mV and recovered, GPIO stayed %u -> ignored\n",
           t->endLevel ? dipLo : dipHi, t->endLevel);
    t->tVihMs = t->tVilMs = t->tEndMs = -1;
```

Return to guided explanation, p. 318

ORIGINAL SOURCE / CHAPTER 13 / 8 OF 13

test/bringup/power_sense_main.c

Original lines 338-384 | Code and comments unchanged; long lines wrap visually

reportTransition (L282-391): Estimate departure, threshold crossings and bounce; distinguish dips and short outages from one-way transitions.

```
338     t->dip = true;
339     s_history[s_historyCount % PS_HISTORY] = *t;
340     s_historyCount++;
341     return;
342   }
343
344   // GPIO edges but the pin ended in the band it started in: power went away
345   // and came back (or the reverse) before the pin settled. The mode banners
346   // already showed both changes; report it as one short outage, no verdict.
347   if (t->adcValid && t->edges > 0 && !t->timedOut &&
348       strcmp(bandName(t->mvBefore), bandName(t->mvAfter)) == 0) {
349     const bool wasHigh = t->mvBefore >= PS_VIH_MV;
350     uint32_t extreme = wasHigh ? 0xFFFFu : 0u;
351     int32_t leftMs = -1, backMs = -1;
352     for (uint32_t i = sampleIndexAt(departUs, last); i < last; ++i) {
353       const Sample* s = sampleAt(i);
354       const bool inBand = wasHigh ? s->mv >= PS_VIH_MV : s->mv <= PS_VIL_MV;
355       if (wasHigh ? s->mv < extreme : s->mv > extreme) extreme = s->mv;
356       if (leftMs < 0 && !inBand) leftMs = msSince(departUs, s->tUs);
357       if (leftMs >= 0 && backMs < 0 && inBand) backMs = msSince(departUs, s->tUs);
358     }
359     printf("  result  short %s: pin %s for %ld ms (down to %lu mV), GPIO back to %u   OK\n",
360            wasHigh ? "outage" : "power blip", wasHigh ? "left HIGH" : "left LOW",
361            (leftMs >= 0 && backMs >= 0) ? (long)(backMs - leftMs) : -1L,
362            (unsigned long)extreme, t->endLevel);
363     printf("  bounce  %lu edge(s) over both changes\n", (unsigned long)t->edges);
364     t->outage = true;
365     t->tEndMs = backMs;
366     s_history[s_historyCount % PS_HISTORY] = *t;
367     s_historyCount++;
368     return;
369   }
370
371   printSummary(t);
372   if (s_verbose) {
373     if (t->adcValid && t->flipMaxMv > 0) {
374       printf("  detail  GPIO flipped at %u-%u mV (1 ms sampler)\n", t->flipMinMv,
  > t->flipMaxMv);
375     }
376     for (uint32_t i = 0; i < nListed; ++i) {
377       const int32_t dt = (int32_t)(edges[i].tUs - departUs);
378       const uint32_t si = sampleIndexAt(edges[i].tUs, last);
379       printf("  detail  edge %2lu  %+9.3f ms  level %u", (unsigned long)(i + 1u),
380              (double)dt / 1000.0, edges[i].level);
381       if (t->adcValid && si < last) printf("  ~%u mV", sampleAt(si)->mv);
382       printf("\n");
383     }
384     if (t->edges > nListed) {
```

Return to guided explanation, p. 318

ORIGINAL SOURCE / CHAPTER 13 / 9 OF 13

test/bringup/power_sense_main.c

Original lines 385-430 | Code and comments unchanged; long lines wrap visually

reportTransition (L282-391): Estimate departure, threshold crossings and bounce; distinguish dips and short outages from one-way transitions.
printTable (L393-408): Present already-collected diagnostic information; this does not make the measurement more valid.

```
      printf("  detail  ... %lu more edge(s), shortest gap %lu us\n",
             (unsigned long)(t->edges - nListed), (unsigned long)t->minGapUs);
    }
  }
  s_history[s_historyCount % PS_HISTORY] = *t;
  s_historyCount++;
}

static void printTable(void) {
  printf("\n%lu transition(s); last %u shown\n", (unsigned long)s_historyCount,
         (unsigned)(s_historyCount < PS_HISTORY ? s_historyCount : PS_HISTORY));
  printf("  #  dir   edges  bounce_ms  mV_before  mV_after  t_VIH  t_VIL  t_end  gpio
flip_mV\n");
  const uint32_t from = s_historyCount > PS_HISTORY ? s_historyCount - PS_HISTORY : 0u;
  for (uint32_t k = from; k < s_historyCount; ++k) {
    const Transition* t = &s_history[k % PS_HISTORY];
    printf("%3lu  %s  %5lu  %9.3f  %9u  %8u  %5ld  %5ld  %5ld  %4u  %u-%u%s\n",
           (unsigned long)(k + 1u), t->dip ? "DIP " : t->outage ? "OUT " : t->rising ? "RISE"
: "FALL",
           (unsigned long)t->edges,
           (double)t->bounceUs / 1000.0, t->mvBefore, t->mvAfter, (long)t->tVihMs,
           (long)t->tVilMs, (long)t->tEndMs, t->endLevel,
           t->flipMaxMv ? t->flipMinMv : 0, t->flipMaxMv, t->timedOut ? "  TIMEOUT" : "");
  }
  printf("  (t_* in ms from ADC departure; -1 = not reached; mV = 0 when the ADC was
off)\n\n");
}

static void printHelp(void) {
  printf("keys: v = edge details (%s), d = transition table, a = ADC (%s), r = reset, "
         "h = help\n", s_verbose ? "on" : "off", s_adcOn ? "on" : "off");
}

static void handleKeys(void) {
  char c;
  while (read(STDIN_FILENO, &c, 1) == 1) {
    switch (c) {
      case 'd': printTable(); break;
      case 'v':
        s_verbose = !s_verbose;
        printf("edge details %s\n", s_verbose ? "on" : "off");
        break;
      case 'a':
        s_adcOn = !s_adcOn;
        printf("ADC %s\n", s_adcOn ? "on (combined mode)" : "off (digital-only mode)");
        break;
      case 'r':
        s_historyCount = 0;
        s_mismatch = 0;
```

Return to guided explanation, p. 318

ORIGINAL SOURCE / CHAPTER 13 / 10 OF 13

test/bringup/power_sense_main.c

Original lines 431-477 | Code and comments unchanged; long lines wrap visually

handleKeys (L415-440): Read nonblocking diagnostic console commands without stopping sampling.
analyzerTask (L443-572): Power diagnostic: group edges and analog history into qualified transitions and reports.

```
        s_adcDisturbed = 0;
        s_samplerFlips = 0;
        s_edgeTotal = 0;
        printf("counters reset\n");
        break;
      case 'h': printHelp(); break;
      default: break;
    }
  }
}

// ---- analyzer (core 1): groups edges + ADC into transitions -------------------
static void analyzerTask(void* arg) {
  (void)arg;
  bool active = false;
  Transition cur;
  Edge listed[PS_EDGES_LISTED];
  uint32_t nListed = 0;
  uint32_t firstSample = 0;
  uint32_t lastEdgeUs = 0;
  uint32_t settledMv = 0;
  bool settledKnown = false;
  uint32_t lastStatusUs = (uint32_t)esp_timer_get_time();
  uint32_t edgesAtStatus = 0, flipsAtStatus = 0;
  int mode = -1;  // qualified GPIO level: 1 = NORMAL, 0 = LOW-POWER, -1 = not yet

  for (;;) {
    vTaskDelay(pdMS_TO_TICKS(10));
    handleKeys();
    const uint32_t nowUs = (uint32_t)esp_timer_get_time();
    const uint32_t last = s_sampleCount;
    if (last < PS_PRE_TRIGGER_MS + PS_STABLE_WINDOW_MS) continue;

    // Mode decision as WP9 will make it: act on the level only once no edge
    // has arrived for BOARD_POWER_SENSE_QUALIFY_MS.
    const uint8_t pinLevel = gpioLevel();
    if ((s_edgeTotal == 0 || nowUs - s_lastEdgeUs >= PS_QUALIFY_US) && (int)pinLevel != mode)
{
      mode = pinLevel;
      printModeBanner(pinLevel);
      lastStatusUs = nowUs;
    }

    // Drain new edges.
    const uint32_t head = s_edgeHead;
    if (head - s_edgeTail > PS_EDGE_RING) s_edgeTail = head - PS_EDGE_RING;  // overrun
    while (s_edgeTail != head) {
      const Edge e = s_edges[s_edgeTail & (PS_EDGE_RING - 1u)];
```

Return to guided explanation, p. 318

ORIGINAL SOURCE / CHAPTER 13 / 11 OF 13

test/bringup/power_sense_main.c

Original lines 478-525 | Code and comments unchanged; long lines wrap visually

analyzerTask (L443-572): Power diagnostic: group edges and analog history into qualified transitions and reports.

```
      s_edgeTail++;
      if (!active) {
        memset(&cur, 0, sizeof(cur));
        cur.tStartUs = e.tUs;
        cur.minGapUs = UINT32_MAX;
        nListed = 0;
        firstSample = sampleIndexAt(e.tUs - PS_PRE_TRIGGER_MS * 1000u, last);
        active = true;
      } else if (cur.edges > 0 && e.tUs - lastEdgeUs < cur.minGapUs) {
        cur.minGapUs = e.tUs - lastEdgeUs;
      }
      if (cur.edges == 0) cur.tStartUs = e.tUs;
      if (nListed < PS_EDGES_LISTED) listed[nListed++] = e;
      cur.edges++;
      cur.bounceUs = e.tUs - cur.tStartUs;
      lastEdgeUs = e.tUs;
    }

    uint16_t lo, hi;
    uint32_t mean;
    spanMv(last - PS_STABLE_WINDOW_MS, last, &lo, &hi, &mean);
    const bool adcStable = !s_adcOn || (uint32_t)(hi - lo) < PS_STABLE_SPAN_MV;

    // ADC departure without any edge (e.g. rail drops but stays above VIH).
    if (!active && s_adcOn && settledKnown) {
      const uint32_t now = sampleAt(last - 1u)->mv;
      const uint32_t delta = now > settledMv ? now - settledMv : settledMv - now;
      if (delta >= PS_TRIGGER_DELTA_MV) {
        memset(&cur, 0, sizeof(cur));
        cur.tStartUs = sampleAt(last - 1u)->tUs;
        cur.minGapUs = UINT32_MAX;
        nListed = 0;
        firstSample = sampleIndexAt(cur.tStartUs - PS_PRE_TRIGGER_MS * 1000u, last);
        active = true;
      }
    }

    if (active) {
      const bool edgesQuiet = cur.edges == 0 || nowUs - lastEdgeUs >= PS_EDGE_QUIET_MS *
> 1000u;
      const bool longEnough = nowUs - cur.tStartUs >= PS_EDGE_QUIET_MS * 1000u;
      const bool timedOut = nowUs - cur.tStartUs >= PS_TRANSITION_TIMEOUT_MS * 1000u;
      if ((edgesQuiet && adcStable && longEnough) || timedOut) {
        cur.timedOut = timedOut && !(edgesQuiet && adcStable);
        cur.endLevel = gpioLevel();
        if (cur.minGapUs == UINT32_MAX) cur.minGapUs = 0;
        reportTransition(&cur, listed, nListed, firstSample, last);
        active = false;
        settledMv = mean;
```

Return to guided explanation, p. 318

ORIGINAL SOURCE / CHAPTER 13 / 12 OF 13

test/bringup/power_sense_main.c

Original lines 526-571 | Code and comments unchanged; long lines wrap visually

analyzerTask (L443-572): Power diagnostic: group edges and analog history into qualified transitions and reports.

```
        settledKnown = true;
        lastStatusUs = nowUs;
      }
      continue;
    }

    if (!settledKnown && adcStable) {
      settledMv = mean;
      settledKnown = true;
    }

    // Steady state: GPIO level must agree with the ADC band.
    const uint8_t level = gpioLevel();
    if (s_adcOn && ((level == 1 && mean <= PS_VIL_MV) || (level == 0 && mean >= PS_VIH_MV)))
{
      s_mismatch++;
    }

    if (nowUs - lastStatusUs >= PS_STATUS_PERIOD_MS * 1000u) {
      lastStatusUs = nowUs;
      uint16_t sLo, sHi;
      uint32_t sMean;
      spanMv(last - 1000u, last, &sLo, &sHi, &sMean);
      const uint32_t edges = s_edgeTotal, flips = s_samplerFlips;
      // The sampler saw the level change but the ISR did not: interrupt is deaf.
      const bool isrOk = !(flips != flipsAtStatus && edges == edgesAtStatus);
      edgesAtStatus = edges;
      flipsAtStatus = flips;
      printf("  ");
      printUptime();
      printf("  still %s | pin ", mode == 1 ? "NORMAL" : mode == 0 ? "LOW-POWER" : "?");
      if (!s_adcOn) {
        printf("ADC off");
      } else if (sHi >= PS_CLIP_MV) {
        printf(">=%u mV", PS_CLIP_MV);
      } else {
        printf("%lu mV", (unsigned long)sMean);
      }
      printf(" | %lu transition(s)\n", (unsigned long)s_historyCount);
      if (!isrOk) printf("  WARNING: GPIO changed but the interrupt did not fire (ISR
deaf)\n");
      if (s_mismatch > 0) {
        printf("  WARNING: GPIO level disagreed with the ADC for %lu check(s)\n",
               (unsigned long)s_mismatch);
        s_mismatch = 0;
      }
    }
  }
```

Return to guided explanation, p. 318

ORIGINAL SOURCE / CHAPTER 13 / 13 OF 13

test/bringup/power_sense_main.c

Original lines 572-618 | Code and comments unchanged; long lines wrap visually

analyzerTask (L443-572): Power diagnostic: group edges and analog history into qualified transitions and reports.
setup (L574-616): Establish safe outputs, logger, gate, board check and queues; load configuration; start Safety, both acquisition workers, power, radios and supervision.

```
572 }
573
574 void setup(void) {
575   delay(200);
576   printf("\n=== power-sense test: GPIO%d (ADC1_CH7), qualify %lu ms ===\n", (int)PS_PIN,
577          (unsigned long)BOARD_POWER_SENSE_QUALIFY_MS);
578
579   gpio_num_t io = GPIO_NUM_NC;
580   if (adc1_pad_get_io_num(PS_ADC_CHANNEL, &io) != ESP_OK || (int)io != PS_PIN) {
581     printf("ADC1_CH7 is GPIO%d, not GPIO%d: fix PS_ADC_CHANNEL\n", (int)io, (int)PS_PIN);
582     return;
583   }
584
585   // ADC first (this routes the pad to the RTC mux), then the digital input:
586   // gpio_config() hands the pad back to the digital mux. The ADC still samples
587   // the pad; the counters below confirm it on this board.
588   adc1_config_width(ADC_WIDTH_BIT_12);
589   adc1_config_channel_atten(PS_ADC_CHANNEL, ADC_ATTEN_DB_12);
590   const esp_adc_cal_value_t calType =
591       esp_adc_cal_characterize(ADC_UNIT_1, ADC_ATTEN_DB_12, ADC_WIDTH_BIT_12, 1100, &s_cal);
592   pinMode(PS_PIN, INPUT);  // no pull: the 440 Ohm bottom leg defines LOW
593
594   // Self-check: one conversion must not change the digital level.
595   const uint8_t l0 = gpioLevel();
596   const uint16_t mv = adcReadMv();
597   const uint8_t l1 = gpioLevel();
598   const bool agree = (l1 == 1 && mv >= PS_VIH_MV) || (l1 == 0 && mv <= PS_VIL_MV);
599   if (calType != ESP_ADC_CAL_VAL_EFUSE_TP_FIT && calType != ESP_ADC_CAL_VAL_EFUSE_TP) {
600     printf("WARNING: no eFuse ADC calibration, mV values are approximate\n");
601   }
602   if (l0 == l1 && agree) {
603     printf("self-check OK: ADC and GPIO agree (%u mV, GPIO %u)\n", mv, l1);
604   } else if (l0 != l1) {
605     printf("WARNING: self-check failed, an ADC conversion changes the GPIO level; press
  > 'a'\n");
606   } else {
607     printf("WARNING: self-check: GPIO %u but ADC %u mV (pin in the undefined band?)\n", l1,
  > mv);
608   }
609
610   attachInterrupt(digitalPinToInterrupt(PS_PIN), onEdge, CHANGE);
611   fcntl(STDIN_FILENO, F_SETFL, fcntl(STDIN_FILENO, F_GETFL, 0) | O_NONBLOCK);
612   printHelp();
613
614   xTaskCreatePinnedToCore(samplerTask, "ps_sample", 3072, NULL, 5, NULL, 0);
615   xTaskCreatePinnedToCore(analyzerTask, "ps_analyze", 6144, NULL, 2, NULL, 1);
616 }
617
618 void loop(void) { vTaskDelay(portMAX_DELAY); }
```

Return to guided explanation, p. 318

PART III / SYSTEM SERVICES

CHAPTER 14

Saved Configuration and Sensor Calibration

A saved setting is not applied merely because its bytes reached flash. Configuration validates and stores a revision; acquisition applies it at a frame boundary and acknowledges it. The same ownership rule allows calibration capture without handing the ADC to a command task.

In this chapter

1. Follow A/B records, CRCs, migration and revision selection.
2. Separate validation, storage, publication and application.
3. Derive calibration fits and the flash-versus-echo mutex protocol.

Chapter contents

14.1 Configuration: one owner for settings and flash 352
14.2 A/B slots, revisions and a checksum 353
14.3 Calibration capture does not take over the ADC 354
14.4 A mutex protects the ultrasonic echo from flash work 355

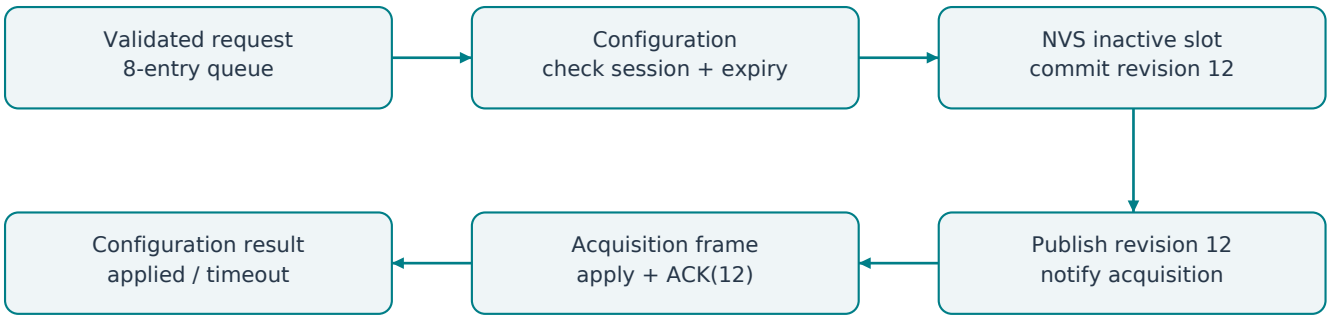
Guided source readings begin on page 359.

Complete original listings begin on page 356. Shared files are linked in the explanations; Appendix A contains the full filename index.

CHAPTER 14

14.1 Configuration: one owner for settings and flash

Configuration is the only application NVS writer. NVS means non-volatile storage: settings survive MCU resets. Other tasks submit requests and read copied published settings. This keeps slow storage work away from a radio callback, driver and relay decision.



Storage precedes application acknowledgement. Timeout is not rollback.

Four stages of a change

A command is validated, queued, stored and then applied. These are different milestones. A full eight-entry request queue returns busy. A stale session is discarded. An expired request is refused before flash work. A storage error does not report success.

For sensor selection or calibration, the task writes the inactive A/B slot, publishes a new revision and wakes acquisition. Acquisition applies at a frame boundary and acknowledges that revision. Configuration waits up to 3 s, polling every 50 ms and marking its own progress. The result reports whether that application acknowledgement arrived.

A timeout is not rollback

Storage and publication precede the wait. If the acknowledgement times out, the new revision remains stored and can take effect later. The published configuration mailbox is named applied in the API, but its publication is earlier than acquisition's acknowledgement. A Reading's configRevision is the local evidence for the settings stamped by acquisition.

Wi-Fi commands report stored rather than associated. Valid credentials do not imply an access point is reachable. The Wi-Fi owner follows its change counter and manages connection separately.

Original source in this PDF: src/services/configuration.c (p. 374); src/services/configuration.h (p. 370); src/services/acquisition.c (p. 96)

CHAPTER 14

14.2 A/B slots, revisions and a checksum

A ConfigBlob contains a magic value, schema, revision, sensor mask, lamp policy, pH/TDS/light fits, override bits and CRC. A magic value recognizes the record family; schema describes its layout; revision orders accepted versions; CRC detects changed bytes.

Preserve the last valid record

A change writes the slot that is not current. After a successful NVS commit it becomes current. Boot validates both and chooses the newest valid one, otherwise compiled defaults. The previous valid slot remains available while the new record is written. On restart, validation rejects an incomplete or corrupt candidate before selecting a current record.

Revisions use 16-bit serial-number arithmetic and skip zero. For example, revision 1 can follow 65535. Ordering is meaningful while versions are within the supported half-range; revision is not a timestamp.

Padding is part of this storage format

CRC covers every byte before the crc field, including compiler padding. Initialization zeroes the structure; copies use memcpy or reseal. Ordinary structure assignment may not preserve padding bytes. The record is tied to its compiled layout, which is why schema validation and migration matter.

Schema 1 migration preserves the sensor mask and revision while filling newer fields from current defaults. At load, compiled lamp values and non-overridden calibration defaults are refreshed. A valid user fit keeps its override bit so a firmware update does not silently replace it with the nominal conversion.

Separate keys, separate write rates

cfg_a/cfg_b hold the blob. wifi holds credentials. sgp_base holds a baseline pair; rst_hist holds eight reset reasons. Baselines are rate-limited to at most hourly and reset history is attempted after 30 s of uptime. Old LAN-token storage is erased; there is no current LAN-token interface.

Original source in this PDF: src/domain/config_blob.h (p. 356); src/domain/config_blob.c (p. 360); src/services/configuration.c (p. 374)

CHAPTER 14

14.3 Calibration capture does not take over the ADC

cal_capture peeks a Snapshot. Each analog driver already publishes filtered pin mV in raw, so a second ADC owner is unnecessary. The capture checks enabled state, current rail epoch, health and age. Accepted raw health is OK or UNCALIBRATED; older than 5 s is refused.

Sensor	Fit input x	Reference and fit
pH	Module mV recovered through the divider	Two buffers at least 1 pH apart; straight line.
TDS	Nominal ppm corrected to 25 degC, before gain	One reference ppm; K = reference / x.
Light	Filtered pin mV	One reference lux; gain through zero.

TDS also needs fresh actionable water temperature within 0-55 degC. Its capture recomputes from the raw signal and latest accepted temperature; it never starts a new measurement.

Validate before saving

The pH slope magnitude must be 0.002-0.02 pH/mV. TDS gain must be 0.5-2.0. Light gain must be 0.01-100 lux/mV. Non-finite, nearly coincident or implausible points are refused as bad_fit. A fitted sensor whose alarm rule is active or latched cannot be recalibrated to hide that condition.

An illustrative pH pair 2500 mV /6.86 and 3000 mV /4.01 yields slope -0.0057 and offset 21.11. An illustrative TDS raw result 500 ppm against a 600 ppm reference yields K = 1.2. A light reference 600 lux at 200 mV yields gain 3. These are calculations, not measured calibration data.

Apply and reset

Configuration stores the fit, marks its override bit and gives it the new revision. Acquisition copies it to the driver between frames. reset_calibration clears the override and restores compiled defaults: provisional pH, nominal TDS and light mV. BME280, DS18B20, SGP30 and HC-SR04 have no such calibration command in this implementation.

Original source in this PDF: src/domain/cal_capture.c (p. 366); src/domain/calibration.c (p. 158); src/services/configuration.c (p. 374)

CHAPTER 14

14.4 A mutex protects the ultrasonic echo from flash work

NVS writes can disable the flash cache and delay interrupt handling. A late HC-SR04 edge timestamp gives the wrong pulse width. The flash gate prevents an NVS commit from overlapping a tracked ping window.

A mutex has an owner

flashGateInit creates a static FreeRTOS mutex before Configuration and acquisition start. Acquisition tries with zero wait before triggering. If busy, it defers the ping and keeps servicing other work. After start it holds the gate through result, failure, guard timeout or cancellation, then gives it from the same worker task.

Configuration takes the same mutex around nvs_set_blob or erase plus nvs_commit. It waits 100 ms per attempt, at most five attempts. It marks progress between attempts. This is a bounded storage wait, not permission for acquisition to block behind flash indefinitely.

Why it is not a spinlock

A spinlock critical section must be very short and cannot surround a wait or NVS commit. A mutex allows its owner to block while other tasks run; the competing task can wait or try. Here the high-priority acquisition path chooses try-and-defer.

The mutex gate is not a global stop for all timing-sensitive work. It specifically coordinates application NVS commits with HC-SR04 pings. Framework and library storage may have other owners, and a cache pause can affect other work. Loaded-target testing measures those remaining timing interactions.

If the gate is absent, its API permits operations; that supports bring-up images with no Configuration writer. Production logs a failed gate initialization rather than pretending ping protection exists.

Original source in this PDF: src/services/flash_gate.h (p. 367); src/services/flash_gate.c (p. 369); src/services/configuration.c (p. 374); src/services/acquisition.c (p. 96)

ORIGINAL SOURCE / CHAPTER 14 / 1 OF 3

src/domain/config_blob.h

Original lines 1-39 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // Versioned configuration record persisted by services/configuration (§17).
2  // Pure C. Work package WP8 (Stage 5).
3  //
4  // Stored in two NVS slots (A/B). A change writes the inactive slot with the
5  // next revision; boot loads the newest slot whose magic, schema and CRC are
6  // valid, else the compiled defaults (policy_config.h). The revision is what
7  // every Reading carries as configRevision.
8  //
9  // Wi-Fi credentials and the SGP30 baseline are NOT in here: they have their
10 // own NVS keys and write rates.
11 //
12 // The sensor mask and the pH / TDS / light calibrations are set by commands.
13 // The lamp policy has no command, so it always comes from the compiled
14 // defaults (policy_config.h). configBlobRefreshDefaults() copies the compiled
15 // values on every load, so a re-measured value in the source is never shadowed
16 // by an old copy in flash - except for a calibration whose CAL_BIT_* is set in
17 // calOverridden: the user fitted it on the device and it is kept.
18 //
19 // Schema 2 (calibration commands) added `light` and `calOverridden`. A schema 1
20 // slot is migrated at boot (configBlobMigrateV1) so the sensor mask survives
21 // the firmware update.
22 #pragma once
23 #include <stdbool.h>
24 #include <stdint.h>
25
26 #include "domain/calibration.h"
27 #include "domain/lamp_control.h"
28 #include "domain/sensor_set.h"
29
30 #define CONFIG_BLOB_MAGIC 0x42445948u  // "HYDB" little-endian
31 #define CONFIG_BLOB_SCHEMA 2u
32
33 typedef struct ConfigBlob {
34   uint32_t magic;
35   uint16_t schema;
36   uint16_t revision;          // 1.., wraps past 0xFFFF to 1
37   SensorMask sensorEnabled;
38   bool lampAutoEnabled;
39   uint8_t calOverridden;      // CAL_BIT_* (calibration.h): fitted by the user
```

Return to guided explanation, p. 352

ORIGINAL SOURCE / CHAPTER 14 / 2 OF 3

src/domain/config_blob.h

Original lines 40-77 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
  uint8_t reserved;           // keep zero (CRC covers every byte)
  LampControlConfig lamp;
  LinearCalibration ph;
  LinearCalibration tds;
  LinearCalibration light;    // x = pin mV, y = lux
  uint32_t crc;               // CRC-32 (IEEE) of every byte before this field
} ConfigBlob;

// Schema 1 layout (firmware up to Stage 8.4), only for migration.
typedef struct ConfigBlobV1 {
  uint32_t magic;
  uint16_t schema;
  uint16_t revision;
  SensorMask sensorEnabled;
  bool lampAutoEnabled;
  uint8_t reserved[2];
  LampControlConfig lamp;
  LinearCalibration ph;
  LinearCalibration tds;
  uint32_t crc;
} ConfigBlobV1;

// Compiled defaults of every field except the sensor mask.
typedef struct ConfigDefaults {
  bool lampAutoEnabled;
  LampControlConfig lamp;
  LinearCalibration ph;
  LinearCalibration tds;
  LinearCalibration light;
} ConfigDefaults;
// The CRC covers padding bytes too: copy a sealed blob with memcpy (struct
// assignment may not preserve padding), or re-seal after assigning.

uint32_t crc32Ieee(const void* data, uint32_t len);

// Revision 1, all sensors enabled, nothing overridden, the given defaults.
void configBlobInit(ConfigBlob* b, const ConfigDefaults* d);

```

Return to guided explanation, p. 352

ORIGINAL SOURCE / CHAPTER 14 / 3 OF 3

src/domain/config_blob.h

Original lines 78-113 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
78  void configBlobSeal(ConfigBlob* b);
79
80  // Magic, schema, CRC and value ranges (lamp band, faultsToOff, sensor mask).
81  bool configBlobValid(const ConfigBlob* b);
82
83  // Newest valid of two slots (either may be NULL or invalid); NULL if none.
84  // Revisions compare with wrap-around (serial-number arithmetic).
85  const ConfigBlob* configBlobPickNewest(const ConfigBlob* a, const ConfigBlob* b);
86
87  // next = cur with the sensor switched, a new revision and a fresh CRC.
88  // Returns false (next untouched) if nothing would change.
89  bool configBlobWithSensor(const ConfigBlob* cur, SensorId sensor, bool enabled, ConfigBlob*
 >  next);
90
91  uint16_t configRevisionNext(uint16_t revision);
92
93  // Copies the compiled lamp policy and every calibration NOT marked in
94  // calOverridden into b. If anything differed, b gets a new revision and a
95  // fresh CRC and the result is true (the caller stores it); otherwise b is
96  // untouched.
97  bool configBlobRefreshDefaults(ConfigBlob* b, const ConfigDefaults* d);
98
99  // next = cur with the sensor's calibration replaced, a new revision and a
100 // fresh CRC. overridden = true for a user fit (its revision becomes the new
101 // config revision, reported as cal_rev), false for a reset to the compiled
102 // default. false (next untouched) for a sensor without a calibration.
103 bool configBlobWithCalibration(const ConfigBlob* cur, SensorId sensor, const
 >  LinearCalibration* cal,
104                                bool overridden, ConfigBlob* next);
105
106 // The stored calibration of a calibratable sensor (NULL otherwise).
107 const LinearCalibration* configBlobCalibration(const ConfigBlob* b, SensorId sensor);
108 uint8_t configCalibrationBit(SensorId sensor);  // CAL_BIT_*, 0 if none
109
110 // A valid schema 1 blob (len must be sizeof(ConfigBlobV1)) becomes a schema 2
111 // blob with the same revision and sensor mask, the defaults for everything
112 // else and nothing overridden. false if v1 is not valid.
113 bool configBlobMigrateV1(const void* v1, uint32_t len, const ConfigDefaults* d, ConfigBlob*
 >  out);
```

Return to guided explanation, p. 352

GUIDED SOURCE READING / CHAPTER 14

Seal a new version only after its fields are complete

Configuration persists a structure rather than an isolated setting. Changing one sensor bit therefore creates a complete next version that can be validated on boot.

Original excerpt: src/domain/config_blob.c (p. 360) / lines 68-79. The complete unchanged file follows.

```
68  bool configBlobWithSensor(const ConfigBlob* cur, SensorId sensor, bool enabled, ConfigBlob*
 >  next) {
69    if (sensor >= SENSOR_COUNT) return false;
70    const SensorMask bit = (SensorMask)(1u << sensor);
71    const SensorMask mask = enabled ? (SensorMask)(cur->sensorEnabled | bit)
72                                    : (SensorMask)(cur->sensorEnabled & (SensorMask)~bit);
73    if (mask == cur->sensorEnabled) return false;
74    memcpy(next, cur, sizeof(*next));
75    next->sensorEnabled = mask;
76    next->revision = configRevisionNext(cur->revision);
77    configBlobSeal(next);
78    return true;
79  }
```

The sensor-range guard comes before shifting to form its bit. The conditional mask then either sets that bit with OR or clears it with AND and an inverted bit. Other sensor choices remain unchanged.

The no-change return avoids a new revision and flash write when the requested state already exists. Only a real change copies the current blob, changes the mask and advances revision. configBlobSeal comes last because its checksum must describe the final bytes, not an earlier partial record.

memcpy preserves the stored byte representation, including padding covered by CRC. The initializer zeroes those bytes for the same reason. A raw structure stored in NVS has a compiler-layout contract; it is different from the explicit JSON wire fields used by transports.

The caller writes the inactive A/B slot before replacing current RAM state. At next boot, validation and revision ordering select a valid record. These small ordering choices are what make a setting update a recoverable versioned operation rather than an untracked mutation of the only copy.

ORIGINAL SOURCE / CHAPTER 14 / 1 OF 4

src/domain/config_blob.c

Original lines 1-40 | Code and comments unchanged; long lines wrap visually

crc32Ieee (L6-16): Calculate reflected IEEE CRC-32 over the requested byte range, including any serialized padding.
blobCrc (L18-20): Preserve a validated A/B configuration with CRC, revision ordering, defaults refresh and schema migration.

```
1  #include "domain/config_blob.h"
2
3  #include <stddef.h>
4  #include <string.h>
5
6  uint32_t crc32Ieee(const void* data, uint32_t len) {
7    const uint8_t* p = (const uint8_t*)data;
8    uint32_t crc = 0xFFFFFFFFu;
9    for (uint32_t i = 0; i < len; ++i) {
10     crc ^= p[i];
11     for (int bit = 0; bit < 8; ++bit) {
12       crc = (crc & 1u) ? (crc >> 1) ^ 0xEDB88320u : crc >> 1;
13     }
14   }
15   return ~crc;
16 }
17
18 static uint32_t blobCrc(const ConfigBlob* b) {
19   return crc32Ieee(b, (uint32_t)offsetof(ConfigBlob, crc));
20 }
21
22 void configBlobInit(ConfigBlob* b, const ConfigDefaults* d) {
23   memset(b, 0, sizeof(*b));  // padding bytes are part of the CRC
24   b->magic = CONFIG_BLOB_MAGIC;
25   b->schema = (uint16_t)CONFIG_BLOB_SCHEMA;
26   b->revision = 1;
27   b->sensorEnabled = SENSOR_MASK_ALL;
28   b->lampAutoEnabled = d->lampAutoEnabled;
29   memcpy(&b->lamp, &d->lamp, sizeof(b->lamp));
30   memcpy(&b->ph, &d->ph, sizeof(b->ph));
31   memcpy(&b->tds, &d->tds, sizeof(b->tds));
32   memcpy(&b->light, &d->light, sizeof(b->light));
33   configBlobSeal(b);
34 }
35
36 void configBlobSeal(ConfigBlob* b) { b->crc = blobCrc(b); }
37
38 bool configBlobValid(const ConfigBlob* b) {
39   if (b == NULL || b->magic != CONFIG_BLOB_MAGIC || b->schema != CONFIG_BLOB_SCHEMA) return
 > false;
40   if (b->crc != blobCrc(b)) return false;
```

Return to guided explanation, p. 359

ORIGINAL SOURCE / CHAPTER 14 / 2 OF 4

src/domain/config_blob.c

Original lines 41-80 | Code and comments unchanged; long lines wrap visually

configBlobValid (L38-49): Check identity, schema, CRC, nonzero revision, masks, lamp band and finite calibration values.
revisionNewer (L52-52): Preserve a validated A/B configuration with CRC, revision ordering, defaults refresh and schema migration.

```
  if (b->revision == 0) return false;
  if ((b->sensorEnabled & (SensorMask)~SENSOR_MASK_ALL) != 0) return false;
  if ((b->calOverridden & (uint8_t)~CAL_BITS_ALL) != 0) return false;
  if (!(b->lamp.onBelowMv < b->lamp.offAboveMv) || b->lamp.faultsToOff == 0) return false;
  if (!linearCalibrationFinite(&b->ph) || !linearCalibrationFinite(&b->tds) ||
!linearCalibrationFinite(&b->light)) {
    return false;
  }
  return true;
}

// true if a is newer than b (RFC 1982 style on 16 bits).
static bool revisionNewer(uint16_t a, uint16_t b) { return (int16_t)(uint16_t)(a - b) > 0; }

const ConfigBlob* configBlobPickNewest(const ConfigBlob* a, const ConfigBlob* b) {
  const bool va = configBlobValid(a);
  const bool vb = configBlobValid(b);
  if (va && vb) return revisionNewer(b->revision, a->revision) ? b : a;
  if (va) return a;
  if (vb) return b;
  return NULL;
}

uint16_t configRevisionNext(uint16_t revision) {
  const uint16_t next = (uint16_t)(revision + 1u);
  return next == 0 ? 1 : next;
}

bool configBlobWithSensor(const ConfigBlob* cur, SensorId sensor, bool enabled, ConfigBlob*
next) {
  if (sensor >= SENSOR_COUNT) return false;
  const SensorMask bit = (SensorMask)(1u << sensor);
  const SensorMask mask = enabled ? (SensorMask)(cur->sensorEnabled | bit)
                                  : (SensorMask)(cur->sensorEnabled & (SensorMask)~bit);
  if (mask == cur->sensorEnabled) return false;
  memcpy(next, cur, sizeof(*next));
  next->sensorEnabled = mask;
  next->revision = configRevisionNext(cur->revision);
  configBlobSeal(next);
  return true;
}

```

Return to guided explanation, p. 359

ORIGINAL SOURCE / CHAPTER 14 / 3 OF 4

src/domain/config_blob.c

Original lines 81-116 | Code and comments unchanged; long lines wrap visually

sameCal (L81-83): Preserve a validated A/B configuration with CRC, revision ordering, defaults refresh and schema migration.
copyCal (L85-85): Preserve a validated A/B configuration with CRC, revision ordering, defaults refresh and schema migration.

```
81  static bool sameCal(const LinearCalibration* a, const LinearCalibration* b) {
82    return a->slope == b->slope && a->offset == b->offset && a->revision == b->revision &&
 >  a->valid == b->valid;
83  }
84
85  static void copyCal(LinearCalibration* dst, const LinearCalibration* src) { memcpy(dst, src,
 >  sizeof(*dst)); }
86
87  bool configBlobRefreshDefaults(ConfigBlob* b, const ConfigDefaults* d) {
88    ConfigBlob fresh;
89    memcpy(&fresh, b, sizeof(fresh));
90    fresh.lampAutoEnabled = d->lampAutoEnabled;
91    memcpy(&fresh.lamp, &d->lamp, sizeof(fresh.lamp));
92    if ((b->calOverridden & CAL_BIT_PH) == 0) copyCal(&fresh.ph, &d->ph);
93    if ((b->calOverridden & CAL_BIT_TDS) == 0) copyCal(&fresh.tds, &d->tds);
94    if ((b->calOverridden & CAL_BIT_LIGHT) == 0) copyCal(&fresh.light, &d->light);
95    // Compare field by field: the inputs may carry padding garbage.
96    const bool same = fresh.lampAutoEnabled == b->lampAutoEnabled &&
97                      d->lamp.onBelowMv == b->lamp.onBelowMv && d->lamp.offAboveMv ==
 >  b->lamp.offAboveMv &&
98                      d->lamp.minOnMs == b->lamp.minOnMs && d->lamp.minOffMs ==
 >  b->lamp.minOffMs &&
99                      d->lamp.faultsToOff == b->lamp.faultsToOff && sameCal(&fresh.ph, &b->ph)
 >  &&
100                     sameCal(&fresh.tds, &b->tds) && sameCal(&fresh.light, &b->light);
101   if (same) return false;
102   fresh.revision = configRevisionNext(b->revision);
103   configBlobSeal(&fresh);
104   memcpy(b, &fresh, sizeof(*b));
105   return true;
106 }
107
108 uint8_t configCalibrationBit(SensorId sensor) {
109   switch (sensor) {
110     case SENSOR_PH: return CAL_BIT_PH;
111     case SENSOR_TDS: return CAL_BIT_TDS;
112     case SENSOR_LIGHT: return CAL_BIT_LIGHT;
113     default: return 0;
114   }
115 }
116
```

Return to guided explanation, p. 359

ORIGINAL SOURCE / CHAPTER 14 / 4 OF 4

src/domain/config_blob.c

Original lines 117-156 | Code and comments unchanged; long lines wrap visually

configBlobCalibration (L117-124): Preserve a validated A/B configuration with CRC, revision ordering, defaults refresh and schema migration.
configBlobWithCalibration (L126-142): Copy a fit into a new revision, set/clear its override bit and seal the updated record.

```
117 const LinearCalibration* configBlobCalibration(const ConfigBlob* b, SensorId sensor) {
118   switch (sensor) {
119     case SENSOR_PH: return &b->ph;
120     case SENSOR_TDS: return &b->tds;
121     case SENSOR_LIGHT: return &b->light;
122     default: return NULL;
123   }
124 }
125
126 bool configBlobWithCalibration(const ConfigBlob* cur, SensorId sensor, const
  > LinearCalibration* cal,
127                                bool overridden, ConfigBlob* next) {
128   const uint8_t bit = configCalibrationBit(sensor);
129   if (bit == 0) return false;
130   memcpy(next, cur, sizeof(*next));
131   next->revision = configRevisionNext(cur->revision);
132   LinearCalibration* dst = (LinearCalibration*)configBlobCalibration(next, sensor);
133   copyCal(dst, cal);
134   if (overridden) {
135     dst->revision = next->revision;
136     next->calOverridden = (uint8_t)(next->calOverridden | bit);
137   } else {
138     next->calOverridden = (uint8_t)(next->calOverridden & (uint8_t)~bit);
139   }
140   configBlobSeal(next);
141   return true;
142 }
143
144 bool configBlobMigrateV1(const void* v1, uint32_t len, const ConfigDefaults* d, ConfigBlob*
  > out) {
145   if (v1 == NULL || len != sizeof(ConfigBlobV1)) return false;
146   ConfigBlobV1 old;
147   memcpy(&old, v1, sizeof(old));
148   if (old.magic != CONFIG_BLOB_MAGIC || old.schema != 1u || old.revision == 0) return false;
149   if (old.crc != crc32Ieee(&old, (uint32_t)offsetof(ConfigBlobV1, crc))) return false;
150   if ((old.sensorEnabled & (SensorMask)~SENSOR_MASK_ALL) != 0) return false;
151   configBlobInit(out, d);
152   out->revision = old.revision;
153   out->sensorEnabled = old.sensorEnabled;
154   configBlobSeal(out);
155   return true;
156 }
```

Return to guided explanation, p. 359

ORIGINAL SOURCE / CHAPTER 14 / 1 OF 1

src/domain/cal_capture.h

Original lines 1-32 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // Raw calibration input from the latest snapshot (WP8, Calibration page).
2  // Pure C, host-tested.
3  //
4  // Acquisition stays the only ADC owner: a capture never touches a driver. It
5  // reads the raw level every calibratable driver already publishes in
6  // Reading.raw (filtered pin mV) and converts it to the x the stored fit uses:
7  //   pH     module mV   (pin mV through the 10k/15k divider)
8  //   TDS    ppm referred to 25 degC with the nominal datasheet curve, before K
9  //          (needs a fresh water temperature, as the driver does)
10 //   light  pin mV
11 #pragma once
12 #include <stdint.h>
13
14 #include "domain/reading.h"
15
16 typedef uint8_t CaptureStatus;
17 enum {
18   CAPTURE_OK = 0,
19   CAPTURE_STALE,           // no usable reading newer than maxAgeMs (also FAULT / UNPOWERED)
20   CAPTURE_NO_TEMPERATURE,  // TDS only: water temperature missing, old or out of range
21   CAPTURE_UNSUPPORTED      // not a calibratable sensor
22 };
23
24 typedef struct CaptureLimits {
25   uint32_t maxAgeMs;            // the sensor's reading must be at most this old
26   uint32_t temperatureMaxAgeMs; // TDS compensation input
27   uint32_t phDividerTopOhm;
28   uint32_t phDividerBottomOhm;
29 } CaptureLimits;
30
31 CaptureStatus calCaptureFromSnapshot(SensorId sensor, const Snapshot* snap, MonoMs now, const
 > CaptureLimits* limits,
32                                      float* xOut, uint32_t* ageMsOut);
```

Return to guided explanation, p. 352

GUIDED SOURCE READING / CHAPTER 14

Capture a stable observation instead of taking a competing sample

On-device calibration needs a raw reference input. The snapshot already contains filtered pin voltage, so capture can use that completed work without taking ADC ownership away from acquisition.

Original excerpt: src/domain/cal_capture.c (p. 366) / lines 20-32. The complete unchanged file follows.

```
20      case SENSOR_PH: {
21        const Reading* r = &snap->readings[METRIC_PH];
22        if (!usableRaw(r, now, limits->maxAgeMs, ageMsOut)) return CAPTURE_STALE;
23        *xOut = dividerRecoverMv((float)r->raw, limits->phDividerTopOhm,
 >  limits->phDividerBottomOhm);
24        return CAPTURE_OK;
25      }
26      case SENSOR_LIGHT: {
27        const Reading* r = &snap->readings[METRIC_LIGHT_LEVEL];
28        if (!usableRaw(r, now, limits->maxAgeMs, ageMsOut)) return CAPTURE_STALE;
29        *xOut = (float)r->raw;
30        return CAPTURE_OK;
31      }
32      case SENSOR_TDS: {
```

The pH branch selects the pH metric, then usableRaw requires an available raw value, acceptable health, a nonzero timestamp and age within the bound. Divider recovery occurs only after those checks. Converting old or failed voltage first would make it easier to mistake a computed number for a valid capture.

Light needs the same accepted raw voltage without divider recovery. TDS first requires fresh accepted water temperature, then recomputes nominal compensated ppm from its stored raw voltage. Its fit input is therefore different from both pin mV and already-calibrated ppm.

The output pointers are initialized at function entry, giving defined outputs on a rejected path. A success status then tells the caller when those values actually describe a capture. Merely seeing zero in xOut is not a success test.

The transport also checks enabled state and live rail history before calling this pure helper. Mathematical capture, power validity and command ownership are separate checks placed at the layers that possess the necessary information.

ORIGINAL SOURCE / CHAPTER 14 / 1 OF 1

src/domain/cal_capture.c

Original lines 1-46 | Code and comments unchanged; long lines wrap visually

usableRaw (L6-13): Accept only timestamped OK/UNCALIBRATED raw data within the capture age bound. **calCaptureFromSnapshot (L15-46)**: Validate age/health and derive fit input from stored raw pin mV; TDS also needs fresh water temperature.

```
#include "domain/cal_capture.h"

#include "domain/calibration.h"
#include "domain/tds_codec.h"

static bool usableRaw(const Reading* r, MonoMs now, uint32_t maxAgeMs, uint32_t* ageMs) {
  if (!r->hasRaw || r->intervalEndMs == 0) return false;
  if (r->health != HEALTH_OK && r->health != HEALTH_UNCALIBRATED) return false;
  const MonoMs age = now > r->intervalEndMs ? now - r->intervalEndMs : 0;
  if (age > maxAgeMs) return false;
  *ageMs = (uint32_t)age;
  return true;
}

CaptureStatus calCaptureFromSnapshot(SensorId sensor, const Snapshot* snap, MonoMs now, const
CaptureLimits* limits,
                                     float* xOut, uint32_t* ageMsOut) {
  *xOut = 0.0f;
  *ageMsOut = 0;
  switch (sensor) {
    case SENSOR_PH: {
      const Reading* r = &snap->readings[METRIC_PH];
      if (!usableRaw(r, now, limits->maxAgeMs, ageMsOut)) return CAPTURE_STALE;
      *xOut = dividerRecoverMv((float)r->raw, limits->phDividerTopOhm,
limits->phDividerBottomOhm);
      return CAPTURE_OK;
    }
    case SENSOR_LIGHT: {
      const Reading* r = &snap->readings[METRIC_LIGHT_LEVEL];
      if (!usableRaw(r, now, limits->maxAgeMs, ageMsOut)) return CAPTURE_STALE;
      *xOut = (float)r->raw;
      return CAPTURE_OK;
    }
    case SENSOR_TDS: {
      const Reading* t = &snap->readings[METRIC_WATER_TEMPERATURE];
      if (!readingIsActionableAt(t, now, limits->temperatureMaxAgeMs) || t->value <
TDS_MIN_WATER_TEMP_C ||
          t->value > TDS_MAX_WATER_TEMP_C) {
        return CAPTURE_NO_TEMPERATURE;
      }
      const Reading* r = &snap->readings[METRIC_TDS];
      // DEPENDENCY_INVALID (no temperature at the driver) is not usable either.
      if (!usableRaw(r, now, limits->maxAgeMs, ageMsOut)) return CAPTURE_STALE;
      *xOut = tdsPpmFromModuleMv((float)r->raw, tdsCompensationCoefficient(t->value));
      return CAPTURE_OK;
    }
    default: return CAPTURE_UNSUPPORTED;
  }
}
```

Return to guided explanation, p. 365

ORIGINAL SOURCE / CHAPTER 14 / 1 OF 1

src/services/flash_gate.h

Original lines 1-27 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // Flash-write gate: keeps NVS commits out of the HC-SR04 echo window.
2  // Work package WP8 (Stage 5). RTOS Architecture content_2 §227.
3  //
4  // A flash write or erase disables the cache on both cores, which delays the
5  // echo-edge interrupt on core 1 and corrupts the distance. Acquisition holds
6  // the gate for the length of one ping; Configuration holds it around each
7  // NVS commit.
8  //
9  // Rules:
10 // - Acquisition only ever TRIES (zero wait) and defers the ping if the gate
11 //   is busy. It never blocks on Configuration.
12 // - Configuration waits at most a bounded time, then retries later.
13 // - Created by flashGateInit() at the very start of boot, independent of
14 //   Configuration. Before that (bring-up builds never call it), every take
15 //   succeeds: there is no writer to keep out.
16 // Mutex owned by one task at a time; take and give from the same task.
17 #pragma once
18 #include <stdbool.h>
19 #include <stdint.h>
20
21 bool flashGateInit(void);
22
23 bool flashGateTryTakeForPing(void);       // zero wait (acquisition)
24 void flashGateGiveAfterPing(void);        // safe to call when not held
25
26 bool flashGateTakeForWrite(uint32_t waitMs);  // bounded wait (Configuration)
27 void flashGateGiveAfterWrite(void);
```

Return to guided explanation, p. 352

GUIDED SOURCE READING / CHAPTER 14

Choose a mutex where the owner may wait

NVS commits and ultrasonic pings share a timing resource: a flash-cache pause must not overlap the tracked echo window. A mutex lets the two task owners coordinate that interval.

Original excerpt: src/services/flash_gate.c (p. 369) / lines 15-28. The complete unchanged file follows.

```
15  bool flashGateTryTakeForPing(void) {
16    if (s_gate == NULL) return true;
17    if (s_pingHeld) return true;
18    s_pingHeld = xSemaphoreTake(s_gate, 0) == pdTRUE;
19    return s_pingHeld;
20  }
21
22  void flashGateGiveAfterPing(void) {
23    if (s_gate == NULL || !s_pingHeld) return;
24    s_pingHeld = false;
25    xSemaphoreGive(s_gate);
26  }
27
28  bool flashGateTakeForWrite(uint32_t waitMs) {
```

The absent-gate return supports bring-up images with no Configuration writer. In production, xSemaphoreTake with timeout 0 means acquisition tries immediately. A false result leaves the ping wanted so the service can defer it and work elsewhere.

s_pingHeld records a successful take and avoids taking the same non-recursive mutex twice during one ping. The matching give clears that flag and releases the mutex only when held. The same acquisition task must release what it acquired, including cancellation and guard-timeout paths.

Configuration uses a bounded wait through flashGateTakeForWrite, then holds the mutex across set/erase and commit. This is why a spinlock would be the wrong shape: storage can take time, and waiting tasks should block rather than keep a core spinning with a long critical section.

The gate is released after commit, not after merely queuing the write. For the ping it is released after capture is complete or cancelled, not immediately after the 10 us trigger. Matching the lock lifetime to the actual conflicting work is the purpose of every take/give position.

ORIGINAL SOURCE / CHAPTER 14 / 1 OF 1

src/services/flash_gate.c

Original lines 1-35 | Code and comments unchanged; long lines wrap visually

flashGateInit (L10-13): Check dependencies, configure this module and return whether initialisation succeeded. **flashGateTryTakeForPing (L15-20)**: Attempt the mutex without waiting; acquisition defers the ping while Configuration holds it.

```
1  #include "services/flash_gate.h"
2
3  #include <freertos/FreeRTOS.h>
4  #include <freertos/semphr.h>
5
6  static StaticSemaphore_t s_gateControl;
7  static SemaphoreHandle_t s_gate = NULL;
8  static bool s_pingHeld;  // touched by acquisition only
9
10 bool flashGateInit(void) {
11   if (s_gate == NULL) s_gate = xSemaphoreCreateMutexStatic(&s_gateControl);
12   return s_gate != NULL;
13 }
14
15 bool flashGateTryTakeForPing(void) {
16   if (s_gate == NULL) return true;
17   if (s_pingHeld) return true;
18   s_pingHeld = xSemaphoreTake(s_gate, 0) == pdTRUE;
19   return s_pingHeld;
20 }
21
22 void flashGateGiveAfterPing(void) {
23   if (s_gate == NULL || !s_pingHeld) return;
24   s_pingHeld = false;
25   xSemaphoreGive(s_gate);
26 }
27
28 bool flashGateTakeForWrite(uint32_t waitMs) {
29   if (s_gate == NULL) return true;
30   return xSemaphoreTake(s_gate, pdMS_TO_TICKS(waitMs)) == pdTRUE;
31 }
32
33 void flashGateGiveAfterWrite(void) {
34   if (s_gate != NULL) xSemaphoreGive(s_gate);
35 }
```

Return to guided explanation, p. 368

ORIGINAL SOURCE / CHAPTER 14 / 1 OF 2

src/services/configuration.h

Original lines 1-49 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
// Configuration: validated commands, versioned config, sole NVS writer.
// Work package WP8 (Stage 5). Architecture §17; RTOS §14.1.
//
// NVS namespace "hydro":
//   cfg_a / cfg_b  ConfigBlob A/B slots; boot loads the newest valid one
//                  (schema 1 slots are migrated: the sensor mask survives)
//   wifi           ConfigWifi (credentials; never logged, never sent back)
//   sgp_base       SGP30 baseline (co2eq, tvoc), written at most hourly
//   rst_hist       last 8 reset reasons, written once ~30 s after boot
//
// configurationInit() loads and PUBLISHES the applied configuration before it
// returns, so Safety and acquisition (started next) read it on their first
// pass. If NVS fails, the compiled defaults (all sensors on) are published.
//
// The LAN link and its "lan_tok" key are gone (the app link is Bluetooth
// only); a left-over lan_tok is erased once at boot.
//
// Calibration (Calibration page): set_calibration fits the points sent by the
// app (domain/calibration), refuses an implausible fit (bad_fit) or a sensor
// whose alarm rule is latched / active, stores the fit with its CAL_BIT_* set
// so the boot refresh keeps it, and waits until acquisition has applied it.
// reset_calibration restores the compiled default and clears the bit.
//
// Requests come through an 8-entry queue (CFG_COMMAND_QUEUE_LEN) from the
// transports; a full queue is reported as busy, never as success (§11). The
// Configuration task checks expiry and session before touching flash, writes
// the inactive slot, publishes, waits (bounded) until acquisition has applied
// the new revision, then replies with the actual outcome.
#pragma once
#include <stdbool.h>
#include <stdint.h>

#include "domain/config_blob.h"
#include "domain/protocol_codec.h"
#include "domain/types.h"

typedef uint8_t ConfigRequestKind;
enum {
  CONFIG_REQ_SET_SENSOR_ENABLED = 0,
  CONFIG_REQ_SET_WIFI,
  CONFIG_REQ_FORGET_WIFI,
  CONFIG_REQ_SET_CALIBRATION,
  CONFIG_REQ_RESET_CALIBRATION,
};

typedef struct ConfigResult {
  uint32_t tag;
  ProtocolError error;              // PROTOCOL_OK on success
  uint16_t cfgRev;                  // sensor / calibration requests: the applied revision
```

Return to guided explanation, p. 352

ORIGINAL SOURCE / CHAPTER 14 / 2 OF 2

src/services/configuration.h

Original lines 50-99 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
50  } ConfigResult;
51
52  // Called from the Configuration task: copy the result and return at once.
53  typedef void (*ConfigReplyFn)(const ConfigResult* result, void* ctx);
54
55  typedef struct ConfigRequest {
56    ConfigRequestKind kind;
57    uint32_t tag;            // caller's handle, echoed in the result
58    uint32_t session;        // transport session generation at receive time
59    uint32_t (*sessionNow)(void);  // current generation; NULL = no session check
60    MonoMs expiresMs;        // not processed after this
61    ConfigReplyFn reply;
62    void* replyCtx;
63    SensorId sensor;
64    bool enabled;
65    char ssid[PROTOCOL_SSID_MAX + 1];
66    char password[PROTOCOL_PASSWORD_MAX + 1];
67    int32_t calX1, calY1, calX2, calY2;  // SET_CALIBRATION, scaled as in protocol_codec.h
68  } ConfigRequest;
69
70  typedef struct ConfigWifi {
71    char ssid[PROTOCOL_SSID_MAX + 1];
72    char password[PROTOCOL_PASSWORD_MAX + 1];
73  } ConfigWifi;
74
75  bool configurationInit(void);
76
77  // Zero wait. false = queue full (reply "busy") or not initialised.
78  bool configurationSubmit(const ConfigRequest* request);
79
80  // Latest applied configuration (zero wait). Before init or on failure it
81  // fills the compiled defaults and returns false.
82  bool configurationPeekApplied(ConfigBlob* out);
83
84  // Stored SGP30 baseline from the last boot, for acquisition before init.
85  bool configurationStoredBaseline(uint16_t* co2eq, uint16_t* tvoc);
86
87  // Acquisition offers the current baseline (zero wait); written at most hourly
88  // and only when it changed.
89  void configurationOfferBaseline(uint16_t co2eq, uint16_t tvoc);
90
91  // Acquisition reports the revision it applied at a frame boundary.
92  void configurationAckApplied(uint16_t revision);
93
94  // Wi-Fi credentials for the Wi-Fi owner (Stage 8.4). changeCount increases on
95  // every set/forget, so the reader knows when to reconnect.
96  bool configurationGetWifi(ConfigWifi* out, uint32_t* changeCount);
97
98  // For Supervisor. NULL before init.
99  void* configurationTaskHandle(void);
```

Return to guided explanation, p. 352

GUIDED SOURCE READING / CHAPTER 14

Stored, published and acknowledged are different milestones

The next configuration record has been validated. Configuration now performs the slow storage step, then asks the measurement owner to apply it.

Original excerpt: src/services/configuration.c (p. 374) / lines 204-217. The complete unchanged file follows.

```
204    if (!storeConfig(&next)) {
205      reply(req, PROTOCOL_ERR_STORAGE, s_current.revision);
206      return;
207    }
208    publish();
209    logPrintf(LOG_LEVEL_INFO, "cfg: rev %u: %s %s", (unsigned)s_current.revision,
  >  sensorKey(req->sensor),
210                req->enabled ? "enabled" : "disabled");
211    const bool applied = waitApplied(s_current.revision);
212    if (!applied) logPrintf(LOG_LEVEL_WARN, "cfg: rev %u stored but not applied in time",
  >  (unsigned)s_current.revision);
213    reply(req, applied ? PROTOCOL_OK : PROTOCOL_ERR_TIMEOUT, s_current.revision);
214  }
215
216  static void handleWifi(const ConfigRequest* req, bool forget) {
217    ConfigWifi wifi;
```

The storage error branch returns before publish, so a failed commit does not announce a successful new current configuration. After storage succeeds, publish overwrites the length-one settings mailbox. Acquisition can now read the new revision at its frame boundary.

waitApplied follows publication and wakes the active worker. It waits for that exact revision's acknowledgement rather than merely for some task notification. A notification can be merged or stale; an acknowledgement carries the meaning needed by the request.

The response uses the actual wait result. If application times out, storage remains committed: this function has no rollback branch. On a later frame or reboot, the saved revision may still take effect. A client must inspect revision/state when resolving such an outcome.

This order also explains the API name configurationPeekApplied. Its mailbox publication precedes acquisition's application acknowledgement. The Reading's configRevision shows the revision acquisition actually stamped, while the command reply describes whether the bounded acknowledgement arrived. Reading each assignment in context prevents a misleading interpretation of a convenient function name.

GUIDED SOURCE READING / CHAPTER 14

A storage commit is the end of the protected interval

Configuration has acquired the flash gate. It must keep that gate until the operation that can affect flash timing is complete.

Original excerpt: src/services/configuration.c (p. 374) / lines 113-120. The complete unchanged file follows.

```
113     esp_err_t err = data != NULL ? nvs_set_blob(s_nvs, key, data, len) : nvs_erase_key(s_nvs,
  >  key);
114     if (err == ESP_ERR_NVS_NOT_FOUND && data == NULL) err = ESP_OK;  // already gone
115     if (err == ESP_OK) err = nvs_commit(s_nvs);
116     flashGateGiveAfterWrite();
117     if (err != ESP_OK) logPrintf(LOG_LEVEL_ERROR, "cfg: NVS write %s failed (%d)", key,
  >  (int)err);
118     return err == ESP_OK;
119   }
120   logPrintf(LOG_LEVEL_WARN, "cfg: flash gate busy, %s not written", key);
```

The data pointer selects set versus erase. An already-absent key is treated as a successful erase because the requested final state already exists. This is an example of defining success by the operation's meaning rather than by one raw API error code.

nvs_commit follows only a successful set/erase, making the requested change durable according to the storage API. The mutex is released after commit. Releasing it immediately after nvs_set_blob would allow a ping to begin before the commit's flash work ended.

Error reporting happens after releasing the gate. A log record does not need to prolong the protected hardware-timing interval. The return bool then controls whether storeConfig advances its current slot and publishes new settings.

This function owns the mechanics of one write. Request expiry and session validation happen earlier in the command handler, while the later application acknowledgement belongs to acquisition. Keeping these stages explicit is how the document's stored/published/applied distinction appears in executable statements.

ORIGINAL SOURCE / CHAPTER 14 / 1 OF 11

src/services/configuration.c

Original lines 1-47 | Code and comments unchanged; long lines wrap visually

The sole NVS owner checks request lifetime, persists changes and waits for the measurement owner to acknowledge application.

```
#include "services/configuration.h"

#include <esp_system.h>
#include <esp_task_wdt.h>
#include <freertos/FreeRTOS.h>
#include <freertos/queue.h>
#include <freertos/task.h>
#include <nvs.h>
#include <stdio.h>
#include <string.h>

#include "app_config.h"
#include "diagnostics/logger.h"
#include "diagnostics/progress.h"
#include "domain/safety_core.h"
#include "hal/clock.h"
#include "policy_config.h"
#include "services/acquisition.h"
#include "services/flash_gate.h"
#include "services/safety.h"

#define CONFIG_NAMESPACE "hydro"
#define CONFIG_WAKE_MS 500u                 // idle wake: housekeeping, progress
#define CONFIG_ACK_TIMEOUT_MS 3000u         // acquisition applies at a frame boundary (<= 1
> s)
#define CONFIG_ACK_POLL_MS 50u
#define CONFIG_GATE_WAIT_MS 100u            // one HC-SR04 ping is <= ~35 ms
#define CONFIG_GATE_TRIES 5
#define CONFIG_BASELINE_MIN_INTERVAL_MS (60u * 60u * 1000u)  // §16: rate-limited
#define CONFIG_RESET_HISTORY_AFTER_MS 30000u // not during a reset loop (unstable supply)
#define CONFIG_RESET_HISTORY_LEN 8

static const char* const kSlotKey[2] = {"cfg_a", "cfg_b"};

// ---- Mailboxes and queues (static) ------------------------------------------
static StaticQueue_t s_appliedControl;
static uint8_t s_appliedStorage[sizeof(ConfigBlob)];
static QueueHandle_t s_applied = NULL;

static StaticQueue_t s_requestControl;
static uint8_t s_requestStorage[CFG_COMMAND_QUEUE_LEN * sizeof(ConfigRequest)];
static QueueHandle_t s_requests = NULL;

static StaticTask_t s_tcb;
static StackType_t s_stack[4 * 1024];  // cfgTaskSpec(TASK_ID_CONFIGURATION)->stackBytes
static TaskHandle_t s_task = NULL;

// ---- State ------------------------------------------------------------------
```

Return to guided explanation, p. 373

ORIGINAL SOURCE / CHAPTER 14 / 2 OF 11

src/services/configuration.c

Original lines 48-94 | Code and comments unchanged; long lines wrap visually

compiledDefaults (L69-90): Build the lamp policy, provisional pH line, nominal TDS curve and uncalibrated light scale from compiled settings.
defaultBlob (L92-96): The sole NVS owner checks request lifetime, persists changes and waits for the measurement owner to acknowledge application.

```
48  static nvs_handle_t s_nvs;
49  static bool s_nvsOk;
50  static ConfigBlob s_current;   // task-owned after init
51  static uint8_t s_currentSlot;  // slot holding s_current (the other one is written next)
52  static bool s_haveSlot;        // s_current is stored somewhere
53
54  static portMUX_TYPE s_mux = portMUX_INITIALIZER_UNLOCKED;
55  static uint16_t s_ackRevision;
56  static ConfigWifi s_wifi;
57  static uint32_t s_wifiChanges;
58  static bool s_haveStoredBaseline;
59  static uint16_t s_storedCo2, s_storedTvoc;
60  static bool s_offerPending;
61  static uint16_t s_offerCo2, s_offerTvoc;
62
63  static MonoMs s_lastBaselineWriteMs;
64  static bool s_baselineWritten;
65  static bool s_historyWritten;
66  static ConfigRequest s_req;  // task-private: keeps the stack small
67
68  // ---- Compiled defaults ------------------------------------------------------
69  static void compiledDefaults(ConfigDefaults* d) {
70    const LampControlConfig l = {POLICY_LAMP_ON_BELOW_MV, POLICY_LAMP_OFF_ABOVE_MV,
 >  POLICY_LAMP_MIN_ON_MS,
71                                 POLICY_LAMP_MIN_OFF_MS, POLICY_LAMP_FAULTS_TO_OFF};
72    memset(d, 0, sizeof(*d));
73    d->lampAutoEnabled = POLICY_LAMP_AUTO_ENABLED != 0;
74    d->lamp = l;
75    // pH: nominal fit from policy_config (published UNCALIBRATED as provisional).
76    d->ph.slope = POLICY_PH_NOMINAL_PH_PER_MV;
77    d->ph.offset = 7.0f - POLICY_PH_NOMINAL_PH_PER_MV * POLICY_PH_NOMINAL_MV_AT_PH7;
78    d->ph.revision = 0;
79    d->ph.valid = true;
80    // TDS: nominal datasheet curve, K = 1 (D13), not calibrated.
81    d->tds.slope = 1.0f;
82    d->tds.offset = 0.0f;
83    d->tds.revision = 0;
84    d->tds.valid = false;
85    // Light: no lux mapping until a reference-meter fit; published as mV.
86    d->light.slope = 1.0f;
87    d->light.offset = 0.0f;
88    d->light.revision = 0;
89    d->light.valid = false;
90  }
91
92  static void defaultBlob(ConfigBlob* b) {
93    ConfigDefaults d;
94    compiledDefaults(&d);
```

Return to guided explanation, p. 373

ORIGINAL SOURCE / CHAPTER 14 / 3 OF 11

src/services/configuration.c

Original lines 95-139 | Code and comments unchanged; long lines wrap visually

defaultBlob (L92-96): The sole NVS owner checks request lifetime, persists changes and waits for the measurement owner to acknowledge application.
keepAlive (L99-104): The sole NVS owner checks request lifetime, persists changes and waits for the measurement owner to acknowledge application.

```
  configBlobInit(b, &d);
}

// ---- NVS helpers (every write goes through the flash gate) ---------------------
static void keepAlive(void) {
  if (s_task != NULL && xTaskGetCurrentTaskHandle() == s_task) {
    progressMark(TASK_ID_CONFIGURATION);
    esp_task_wdt_reset();
  }
}

static bool gatedSetBlob(const char* key, const void* data, size_t len) {
  if (!s_nvsOk) return false;
  for (int tries = 0; tries < CONFIG_GATE_TRIES; ++tries) {
    if (!flashGateTakeForWrite(CONFIG_GATE_WAIT_MS)) {
      keepAlive();
      continue;
    }
    esp_err_t err = data != NULL ? nvs_set_blob(s_nvs, key, data, len) : nvs_erase_key(s_nvs,
key);
    if (err == ESP_ERR_NVS_NOT_FOUND && data == NULL) err = ESP_OK;  // already gone
    if (err == ESP_OK) err = nvs_commit(s_nvs);
    flashGateGiveAfterWrite();
    if (err != ESP_OK) logPrintf(LOG_LEVEL_ERROR, "cfg: NVS write %s failed (%d)", key,
(int)err);
    return err == ESP_OK;
  }
  logPrintf(LOG_LEVEL_WARN, "cfg: flash gate busy, %s not written", key);
  return false;
}

static bool getBlob(const char* key, void* out, size_t len) {
  if (!s_nvsOk) return false;
  size_t size = len;
  return nvs_get_blob(s_nvs, key, out, &size) == ESP_OK && size == len;
}

// A schema 1 slot (firmware up to Stage 8.4) as a schema 2 blob, same revision.
static bool loadV1(const char* key, const ConfigDefaults* d, ConfigBlob* out) {
  ConfigBlobV1 old;
  if (!getBlob(key, &old, sizeof(old))) return false;
  if (!configBlobMigrateV1(&old, sizeof(old), d, out)) return false;
  logPrintf(LOG_LEVEL_INFO, "cfg: %s migrated from schema 1 (rev %u)", key,
(unsigned)out->revision);
  return true;
}

// Writes the slot not holding the current config; on success it becomes current.
```

Return to guided explanation, p. 373

ORIGINAL SOURCE / CHAPTER 14 / 4 OF 11

src/services/configuration.c

Original lines 140-187 | Code and comments unchanged; long lines wrap visually

storeConfig (L140-147): Write the inactive A/B slot; only after success copy it into current state and make that slot current.
publish (L149-149): The sole NVS owner checks request lifetime, persists changes and waits for the measurement owner to acknowledge application.

```
140 static bool storeConfig(const ConfigBlob* next) {
141   const uint8_t slot = s_haveSlot ? (uint8_t)(1u - s_currentSlot) : 0u;
142   if (!gatedSetBlob(kSlotKey[slot], next, sizeof(*next))) return false;
143   memcpy(&s_current, next, sizeof(s_current));
144   s_currentSlot = slot;
145   s_haveSlot = true;
146   return true;
147 }
148
149 static void publish(void) { xQueueOverwrite(s_applied, &s_current); }
150
151 // ---- Requests ---------------------------------------------------------------
152 static void reply(const ConfigRequest* req, ProtocolError error, uint16_t cfgRev) {
153   if (req->reply == NULL) return;
154   ConfigResult r;
155   memset(&r, 0, sizeof(r));
156   r.tag = req->tag;
157   r.error = error;
158   r.cfgRev = cfgRev;
159   req->reply(&r, req->replyCtx);
160 }
161
162 static uint16_t ackedRevision(void) {
163   portENTER_CRITICAL(&s_mux);
164   const uint16_t rev = s_ackRevision;
165   portEXIT_CRITICAL(&s_mux);
166   return rev;
167 }
168
169 static bool waitApplied(uint16_t revision) {
170   if (acquisitionTaskHandle() == NULL) return false;
171   xTaskNotifyGive(acquisitionTaskHandle());  // apply at the next frame boundary
172   for (uint32_t waited = 0; waited < CONFIG_ACK_TIMEOUT_MS; waited += CONFIG_ACK_POLL_MS) {
173     if (ackedRevision() == revision) return true;
174     vTaskDelay(pdMS_TO_TICKS(CONFIG_ACK_POLL_MS));
175     keepAlive();  // the wait is normal work, not a stall
176   }
177   return ackedRevision() == revision;
178 }
179
180 static void handleSetSensor(const ConfigRequest* req) {
181   if (req->sensor >= SENSOR_COUNT) {
182     reply(req, PROTOCOL_ERR_BAD_FIELD, s_current.revision);
183     return;
184   }
185   if (!req->enabled) {
186     // Switching an input off must never silence an alarm (§12). The status can
187     // be up to one Safety pass (100 ms) old; a TRIP in flight still latches,
```

Return to guided explanation, p. 373

ORIGINAL SOURCE / CHAPTER 14 / 5 OF 11

src/services/configuration.c

Original lines 188-231 | Code and comments unchanged; long lines wrap visually

handleSetSensor (L180-214): Validate disable permission, avoid no-op writes, store/publish the new mask, then report application acknowledgement or timeout.
handleWifi (L216-233): Store or erase credentials, update their protected change counter and report storage outcome without exposing the password.

```
188     // and a latched rule stays visible as stale (safety_core), so the race is
189     // fail-safe. Accepted gap: a rule that has not tripped yet cannot trip
190     // once its sensor is off.
191     SafetyStatus st;
192     memset(&st, 0, sizeof(st));
193     if (safetyStatusPeek(&st) &&
194         safetyRulesBlockSensor(kPolicyRules, POLICY_RULE_COUNT, st.latchedMask,
 >  st.activeMask, req->sensor)) {
195       reply(req, PROTOCOL_ERR_RULE_LATCHED, s_current.revision);
196       return;
197     }
198   }
199   ConfigBlob next;
200   if (!configBlobWithSensor(&s_current, req->sensor, req->enabled, &next)) {
201     reply(req, PROTOCOL_OK, s_current.revision);  // already so
202     return;
203   }
204   if (!storeConfig(&next)) {
205     reply(req, PROTOCOL_ERR_STORAGE, s_current.revision);
206     return;
207   }
208   publish();
209   logPrintf(LOG_LEVEL_INFO, "cfg: rev %u: %s %s", (unsigned)s_current.revision,
 >  sensorKey(req->sensor),
210             req->enabled ? "enabled" : "disabled");
211   const bool applied = waitApplied(s_current.revision);
212   if (!applied) logPrintf(LOG_LEVEL_WARN, "cfg: rev %u stored but not applied in time",
 >  (unsigned)s_current.revision);
213   reply(req, applied ? PROTOCOL_OK : PROTOCOL_ERR_TIMEOUT, s_current.revision);
214 }
215
216 static void handleWifi(const ConfigRequest* req, bool forget) {
217   ConfigWifi wifi;
218   memset(&wifi, 0, sizeof(wifi));
219   if (!forget) {
220     snprintf(wifi.ssid, sizeof(wifi.ssid), "%s", req->ssid);
221     snprintf(wifi.password, sizeof(wifi.password), "%s", req->password);
222   }
223   const bool ok = gatedSetBlob("wifi", forget ? NULL : &wifi, sizeof(wifi));
224   if (ok) {
225     portENTER_CRITICAL(&s_mux);
226     memcpy(&s_wifi, &wifi, sizeof(s_wifi));
227     ++s_wifiChanges;
228     portEXIT_CRITICAL(&s_mux);
229   }
230   memset(&wifi, 0, sizeof(wifi));  // credentials never linger on the stack
231   logPrintf(LOG_LEVEL_INFO, "cfg: wifi %s %s", forget ? "forget" : "set", ok ? "stored" :
 >  "FAILED");
```

Return to guided explanation, p. 372

ORIGINAL SOURCE / CHAPTER 14 / 6 OF 11

src/services/configuration.c

Original lines 232-275 | Code and comments unchanged; long lines wrap visually

handleWifi (L216-233): Store or erase credentials, update their protected change counter and report storage outcome without exposing the password.
calibrationBlocked (L237-248): Refuse a fit/reset while an enabled rule on that sensor remains latched or active.

```
232   reply(req, ok ? PROTOCOL_OK : PROTOCOL_ERR_STORAGE, s_current.revision);
233 }
234
235 // A calibration must never hide an alarm the sensor is part of (§12), as for
236 // switching it off: refused while one of its rules is latched or active.
237 static ProtocolError calibrationBlocked(SensorId sensor) {
238   SafetyStatus st;
239   memset(&st, 0, sizeof(st));
240   if (!safetyStatusPeek(&st)) return PROTOCOL_OK;
241   if (safetyRulesBlockSensor(kPolicyRules, POLICY_RULE_COUNT, st.latchedMask, 0, sensor)) {
242     return PROTOCOL_ERR_RULE_LATCHED;
243   }
244   if (safetyRulesBlockSensor(kPolicyRules, POLICY_RULE_COUNT, 0, st.activeMask, sensor)) {
245     return PROTOCOL_ERR_RULE_ACTIVE;
246   }
247   return PROTOCOL_OK;
248 }
249
250 static bool fitCalibration(const ConfigRequest* req, LinearCalibration* out) {
251   const float x1 = (float)req->calX1 / CAL_X_SCALE;
252   switch (req->sensor) {
253     case SENSOR_PH:
254       return phFitFromBuffers(x1, (float)req->calY1 / CAL_PH_Y_SCALE, (float)req->calX2 /
CAL_X_SCALE,
255                               (float)req->calY2 / CAL_PH_Y_SCALE, POLICY_PH_SLOPE_MIN_ABS,
POLICY_PH_SLOPE_MAX_ABS,
256                               0, out);
257     case SENSOR_TDS:
258       return linearGainFromPoint(x1, (float)req->calY1 / CAL_Y_SCALE, POLICY_TDS_K_MIN,
POLICY_TDS_K_MAX, 0, out);
259     case SENSOR_LIGHT:
260       return linearGainFromPoint(x1, (float)req->calY1 / CAL_Y_SCALE,
POLICY_LIGHT_LUX_PER_MV_MIN,
261                                  POLICY_LIGHT_LUX_PER_MV_MAX, 0, out);
262     default: return false;
263   }
264 }
265
266 static void handleCalibration(const ConfigRequest* req, bool reset) {
267   if (configCalibrationBit(req->sensor) == 0) {
268     reply(req, PROTOCOL_ERR_BAD_FIELD, s_current.revision);
269     return;
270   }
271   const ProtocolError blocked = calibrationBlocked(req->sensor);
272   if (blocked != PROTOCOL_OK) {
273     reply(req, blocked, s_current.revision);
274     return;
275   }
```

Return to guided explanation, p. 372

ORIGINAL SOURCE / CHAPTER 14 / 7 OF 11

src/services/configuration.c

Original lines 276-321 | Code and comments unchanged; long lines wrap visually

handleCalibration (L266-302): Choose a validated fit or compiled default, persist/publish its revision and wait for the acquisition acknowledgement.
handle (L304-319): Before dispatch, drop closed-session requests and reject expired ones; this is not a rollback after an already completed commit.

```
276   LinearCalibration cal;
277   if (reset) {
278     ConfigDefaults d;
279     compiledDefaults(&d);
280     cal = req->sensor == SENSOR_PH ? d.ph : req->sensor == SENSOR_TDS ? d.tds : d.light;
281   } else if (!fitCalibration(req, &cal)) {
282     reply(req, PROTOCOL_ERR_BAD_FIT, s_current.revision);
283     return;
284   }
285   ConfigBlob next;
286   configBlobWithCalibration(&s_current, req->sensor, &cal, !reset, &next);
287   if (!storeConfig(&next)) {
288     reply(req, PROTOCOL_ERR_STORAGE, s_current.revision);
289     return;
290   }
291   publish();
292   if (reset) {
293     logPrintf(LOG_LEVEL_INFO, "cfg: rev %u: %s calibration reset to default",
  > (unsigned)s_current.revision,
294               sensorKey(req->sensor));
295   } else {
296     logPrintf(LOG_LEVEL_INFO, "cfg: rev %u: %s calibrated, slope %.6f offset %.4f",
  > (unsigned)s_current.revision,
297               sensorKey(req->sensor), (double)cal.slope, (double)cal.offset);
298   }
299   const bool applied = waitApplied(s_current.revision);
300   if (!applied) logPrintf(LOG_LEVEL_WARN, "cfg: rev %u stored but not applied in time",
  > (unsigned)s_current.revision);
301   reply(req, applied ? PROTOCOL_OK : PROTOCOL_ERR_TIMEOUT, s_current.revision);
302 }
303
304 static void handle(const ConfigRequest* req) {
305   // Stale requests never change flash (§11): wrong session = no reply at all.
306   if (req->sessionNow != NULL && req->sessionNow() != req->session) return;
307   if (monoNowMs() > req->expiresMs) {
308     reply(req, PROTOCOL_ERR_EXPIRED, s_current.revision);
309     return;
310   }
311   switch (req->kind) {
312     case CONFIG_REQ_SET_SENSOR_ENABLED: handleSetSensor(req); break;
313     case CONFIG_REQ_SET_WIFI: handleWifi(req, false); break;
314     case CONFIG_REQ_FORGET_WIFI: handleWifi(req, true); break;
315     case CONFIG_REQ_SET_CALIBRATION: handleCalibration(req, false); break;
316     case CONFIG_REQ_RESET_CALIBRATION: handleCalibration(req, true); break;
317     default: reply(req, PROTOCOL_ERR_UNKNOWN_OP, s_current.revision); break;
318   }
319 }
320
321 // ---- Housekeeping ------------------------------------------------------------
```

Return to guided explanation, p. 372

ORIGINAL SOURCE / CHAPTER 14 / 8 OF 11

src/services/configuration.c

Original lines 322-364 | Code and comments unchanged; long lines wrap visually

maybeWriteBaseline (L322-340): Consume a changed baseline offer and commit only within the hourly write-rate limit.
maybeWriteResetHistory (L342-353): After 30 seconds of uptime, shift eight reset reasons and attempt one gated write.

```
322 static void maybeWriteBaseline(MonoMs now) {
323   portENTER_CRITICAL(&s_mux);
324   const bool pending = s_offerPending;
325   const uint16_t co2 = s_offerCo2, tvoc = s_offerTvoc;
326   s_offerPending = false;
327   portEXIT_CRITICAL(&s_mux);
328   if (!pending) return;
329   if (s_haveStoredBaseline && co2 == s_storedCo2 && tvoc == s_storedTvoc) return;
330   if (s_baselineWritten && now - s_lastBaselineWriteMs < CONFIG_BASELINE_MIN_INTERVAL_MS)
  > return;
331   const uint16_t pair[2] = {co2, tvoc};
332   if (gatedSetBlob("sgp_base", pair, sizeof(pair))) {
333     s_storedCo2 = co2;
334     s_storedTvoc = tvoc;
335     s_haveStoredBaseline = true;
336     s_baselineWritten = true;
337     s_lastBaselineWriteMs = now;
338     logPrintf(LOG_LEVEL_INFO, "cfg: SGP30 baseline stored (co2eq 0x%04x, tvoc 0x%04x)", co2,
  > tvoc);
339   }
340 }
341
342 static void maybeWriteResetHistory(MonoMs now) {
343   if (s_historyWritten || now < CONFIG_RESET_HISTORY_AFTER_MS) return;
344   s_historyWritten = true;
345   uint8_t hist[CONFIG_RESET_HISTORY_LEN];
346   if (!getBlob("rst_hist", hist, sizeof(hist))) memset(hist, 0, sizeof(hist));  // 0 =
  > ESP_RST_UNKNOWN
347   memmove(&hist[1], &hist[0], sizeof(hist) - 1);
348   hist[0] = (uint8_t)esp_reset_reason();
349   if (gatedSetBlob("rst_hist", hist, sizeof(hist))) {
350     logPrintf(LOG_LEVEL_INFO, "cfg: reset history (newest first, esp_reset_reason_t): %u %u
  > %u %u %u %u %u %u",
351               hist[0], hist[1], hist[2], hist[3], hist[4], hist[5], hist[6], hist[7]);
352   }
353 }
354
355 static void configTask(void* arg) {
356   (void)arg;
357   if (esp_task_wdt_add(NULL) != ESP_OK) logPrintf(LOG_LEVEL_ERROR, "cfg: task watchdog add
  > failed");
358   for (;;) {
359     if (xQueueReceive(s_requests, &s_req, pdMS_TO_TICKS(CONFIG_WAKE_MS)) == pdTRUE) {
360       handle(&s_req);
361       memset(s_req.password, 0, sizeof(s_req.password));
362     }
363     const MonoMs now = monoNowMs();
364     maybeWriteBaseline(now);
```

Return to guided explanation, p. 372

ORIGINAL SOURCE / CHAPTER 14 / 9 OF 11

src/services/configuration.c

Original lines 365-407 | Code and comments unchanged; long lines wrap visually

configTask (L355-369): Handle queued requests, erase the working password, perform bounded housekeeping and mark/feed this task.
configurationInit (L372-454): Create mailboxes, load/validate/migrate slots, refresh defaults and publish before consumers; load other keys and create the sole NVS task.

```
365     maybeWriteResetHistory(now);
366     progressMark(TASK_ID_CONFIGURATION);
367     esp_task_wdt_reset();
368   }
369 }
370
371 // ---- Public API -----------------------------------------------------------
372 bool configurationInit(void) {
373   if (s_task != NULL) return true;
374   s_applied = xQueueCreateStatic(1, sizeof(ConfigBlob), s_appliedStorage, &s_appliedControl);
375   s_requests = xQueueCreateStatic(CFG_COMMAND_QUEUE_LEN, sizeof(ConfigRequest),
  > s_requestStorage,
376                                   &s_requestControl);
377   if (s_applied == NULL || s_requests == NULL) return false;
378
379   // NVS itself was initialised by the Arduino core (initArduino()).
380   s_nvsOk = nvs_open(CONFIG_NAMESPACE, NVS_READWRITE, &s_nvs) == ESP_OK;
381   if (!s_nvsOk) logPrintf(LOG_LEVEL_ERROR, "cfg: NVS open failed - compiled defaults, nothing
  > stored");
382
383   ConfigDefaults defaults;
384   compiledDefaults(&defaults);
385   ConfigBlob slots[2];
386   bool have[2];
387   for (uint8_t i = 0; i < 2; ++i) {
388     have[i] = getBlob(kSlotKey[i], &slots[i], sizeof(ConfigBlob)) || loadV1(kSlotKey[i],
  > &defaults, &slots[i]);
389   }
390   const ConfigBlob* newest = configBlobPickNewest(have[0] ? &slots[0] : NULL, have[1] ?
  > &slots[1] : NULL);
391   bool store = false;
392   if (newest != NULL) {
393     memcpy(&s_current, newest, sizeof(s_current));
394     s_currentSlot = newest == &slots[0] ? 0 : 1;
395     s_haveSlot = true;
396   } else {
397     defaultBlob(&s_current);
398     store = s_nvsOk;  // first boot or both slots corrupt
399     if (have[0] || have[1]) logPrintf(LOG_LEVEL_WARN, "cfg: stored config invalid -
  > defaults");
400   }
401   if (configBlobRefreshDefaults(&s_current, &defaults)) store = s_nvsOk;
402   if (store) {
403     ConfigBlob next;
404     memcpy(&next, &s_current, sizeof(next));
405     if (!storeConfig(&next)) logPrintf(LOG_LEVEL_WARN, "cfg: could not store rev %u",
  > (unsigned)next.revision);
406   }
407   s_ackRevision = 0;
```

Return to guided explanation, p. 372

ORIGINAL SOURCE / CHAPTER 14 / 10 OF 11

src/services/configuration.c

Original lines 408-450 | Code and comments unchanged; long lines wrap visually

configurationInit (L372-454): Create mailboxes, load/validate/migrate slots, refresh defaults and publish before consumers; load other keys and create the sole NVS task.

```
  publish();  // before Safety and acquisition start (they peek it on their first pass)

  ConfigWifi wifi;
  if (getBlob("wifi", &wifi, sizeof(wifi))) {
    wifi.ssid[PROTOCOL_SSID_MAX] = '\0';
    wifi.password[PROTOCOL_PASSWORD_MAX] = '\0';
    memcpy(&s_wifi, &wifi, sizeof(s_wifi));
    memset(&wifi, 0, sizeof(wifi));
  }
  size_t tokenSize = 0;  // LAN link removed: drop its token once
  if (s_nvsOk && nvs_get_blob(s_nvs, "lan_tok", NULL, &tokenSize) == ESP_OK) {
    gatedSetBlob("lan_tok", NULL, 0);
    logPrintf(LOG_LEVEL_INFO, "cfg: old LAN token erased");
  }
  // SGP30 baseline (Sensirion integration guide §3.8): valid only if the
  // sensor was off for less than 7 days. There is no wall clock, so the
  // downtime is known only for resets where the chip kept its supply (software,
  // panic, watchdog, reset pin: seconds). After a power-on, brown-out or
  // unknown reset it may have been off any time: relearn, and erase the old
  // baseline so a later warm reset cannot bring it back.
  uint16_t pair[2];
  if (getBlob("sgp_base", pair, sizeof(pair))) {
    const esp_reset_reason_t why = esp_reset_reason();
    const bool supplyKept = why == ESP_RST_SW || why == ESP_RST_PANIC || why ==
ESP_RST_INT_WDT ||
                            why == ESP_RST_TASK_WDT || why == ESP_RST_WDT || why ==
ESP_RST_EXT;
    if (supplyKept) {
      s_storedCo2 = pair[0];
      s_storedTvoc = pair[1];
      s_haveStoredBaseline = true;
    } else {
      gatedSetBlob("sgp_base", NULL, 0);
      logPrintf(LOG_LEVEL_INFO, "cfg: SGP30 baseline dropped (reset %d: off-time unknown,
relearning)", (int)why);
    }
  }

  const TaskSpec* spec = cfgTaskSpec(TASK_ID_CONFIGURATION);
  if (spec->stackBytes != sizeof(s_stack)) {
    logPrintf(LOG_LEVEL_WARN, "cfg: stack %u B, task table says %lu B",
(unsigned)sizeof(s_stack),
              (unsigned long)spec->stackBytes);
  }
  s_task = xTaskCreateStaticPinnedToCore(configTask, spec->name, sizeof(s_stack), NULL,
spec->priority,
                                         s_stack, &s_tcb, spec->core);
  logPrintf(LOG_LEVEL_INFO, "cfg: rev %u, sensors 0x%02x, calibrated 0x%02x, wifi %s, SGP30
baseline %s",
```

Return to guided explanation, p. 372

ORIGINAL SOURCE / CHAPTER 14 / 11 OF 11

src/services/configuration.c

Original lines 451-496 | Code and comments unchanged; long lines wrap visually

configurationInit (L372-454): Create mailboxes, load/validate/migrate slots, refresh defaults and publish before consumers; load other keys and create the sole NVS task.
configurationSubmit (L456-459): The sole NVS owner checks request lifetime, persists changes and waits for the measurement owner to acknowledge application.

```
451             (unsigned)s_current.revision, (unsigned)s_current.sensorEnabled,
  > (unsigned)s_current.calOverridden,
452             s_wifi.ssid[0] != '\0' ? "set" : "none", s_haveStoredBaseline ? "stored" :
  > "none");
453   return s_task != NULL;
454 }
455
456 bool configurationSubmit(const ConfigRequest* request) {
457   if (s_requests == NULL || request == NULL) return false;
458   return xQueueSend(s_requests, request, 0) == pdTRUE;
459 }
460
461 bool configurationPeekApplied(ConfigBlob* out) {
462   if (s_applied != NULL && xQueuePeek(s_applied, out, 0) == pdTRUE) return true;
463   defaultBlob(out);
464   return false;
465 }
466
467 bool configurationStoredBaseline(uint16_t* co2eq, uint16_t* tvoc) {
468   if (!s_haveStoredBaseline) return false;
469   *co2eq = s_storedCo2;
470   *tvoc = s_storedTvoc;
471   return true;
472 }
473
474 void configurationOfferBaseline(uint16_t co2eq, uint16_t tvoc) {
475   portENTER_CRITICAL(&s_mux);
476   s_offerPending = true;
477   s_offerCo2 = co2eq;
478   s_offerTvoc = tvoc;
479   portEXIT_CRITICAL(&s_mux);
480 }
481
482 void configurationAckApplied(uint16_t revision) {
483   portENTER_CRITICAL(&s_mux);
484   s_ackRevision = revision;
485   portEXIT_CRITICAL(&s_mux);
486 }
487
488 bool configurationGetWifi(ConfigWifi* out, uint32_t* changeCount) {
489   portENTER_CRITICAL(&s_mux);
490   memcpy(out, &s_wifi, sizeof(*out));
491   if (changeCount != NULL) *changeCount = s_wifiChanges;
492   portEXIT_CRITICAL(&s_mux);
493   return out->ssid[0] != '\0';
494 }
495
496 void* configurationTaskHandle(void) { return s_task; }
```

Return to guided explanation, p. 372

PART III / SYSTEM SERVICES

CHAPTER 15

Device Communication: Protocol, BLE and Wi-Fi

A communication link transports requests and copied observations across the existing service boundaries. It does not become a sensor or relay owner. Bounded framing, session generations and queued replies preserve that separation as BLE packets fragment or connections end.

In this chapter

1. Reassemble length-prefixed JSON and inspect bounded parsing.
2. Follow a BLE command through its service owner and reply.
3. Read Wi-Fi association, retry and power-stop states.

Chapter contents

15.1 Device communication crosses service boundaries 386
15.2 Bytes, messages and the bounded JSON codec 387
15.3 State, telemetry, events and command outcomes 388
15.4 Reading the two C++ adapters 389
15.5 Device-side BLE: short callbacks, one transport owner 390
15.6 A command crosses tasks without moving ownership 391
15.7 Wi-Fi joins in steps and backs off after failure 392

Guided source readings begin on page 394.

Complete original listings begin on page 393. Shared files are linked in the explanations; Appendix A contains the full filename index.

CHAPTER 15

15.1 Device communication crosses service boundaries

Production starts BLE and Wi-Fi tasks after the measurement and power services exist. The link tasks read copied observations and send requests to the task that owns each change. Bring-up builds omit the radio macros, so a standalone sensor test can run without those communication services.

Keep names separate from behaviour

WifiMqtt manages station association. It has no MQTT publisher. wifi_lan.cpp is a Wi-Fi adapter and does not create a LAN data server, sockets or mDNS service. protocol.c contains shared protocol constants; domain/protocol_codec.c contains the actual bounded JSON writer and command parser.

Keep ownership boundaries

The BLE callback only copies bytes. BleTransport reassembles and validates commands. Configuration owns saved changes; acquisition applies driver settings; Safety alone computes the relay. No radio callback reads the ADC or writes the relay.

This book explains the controller's side of the link because it is part of firmware startup and service ownership. The desktop Bluetooth application, its UI, packaging, history storage and separate ESP demo are excluded. Understanding these firmware chapters does not require opening the app source.

Original source in this PDF: src/transports/ble_transport.c (p. 436); src/transports/wifi_mqtt.c (p. 453); src/services/configuration.c (p. 374); src/domain/protocol_codec.c (p. 402)

CHAPTER 15

15.2 Bytes, messages and the bounded JSON codec

The device sends contract-version 2 JSON inside a length-prefixed frame. Two little-endian bytes give payload size, followed by that many UTF-8 bytes. Valid payload size is 1-4096. Transport packets can split a header, split a message or contain several messages.

Reassemble before parsing

frameAssemblerFeed consumes bytes until one frame is complete, then returns READY and a pointer into its own buffer. The caller copies or processes that payload before another feed reuses it. consumed tells how much input remains. Invalid length resets the assembler and returns ERROR.

Example: a 300-byte payload has header bytes 0x2C and 0x01 because 300 =44 +256. If the first packet contains only 0x2C, NEED_MORE is correct. A packet boundary is not a message boundary.

Build JSON without a growing heap object

JsonWriter holds a caller buffer, capacity, used length and overflow flag. Every append reserves a NUL terminator. On overflow the final length is zero, so callers must not transmit truncated JSON. Strings are escaped, integer fields are formatted explicitly and non-finite floats become null.

The parser accepts one flat command with an optional data object. It validates version, printable ID, operation, types and ranges before dispatch. It uses bounded fields and skips unknown containers only to a limited depth. It is a purpose-built command parser, not a complete Unicode/JSON document library: escaped surrogate pairs are unsupported.

Unknown operations and invalid fields produce distinct errors. Numeric calibration fields are scaled integers, so the parser does not need to retain floating-point values. Configuration later converts them and checks whether the resulting fit is plausible.

Original source in this PDF: src/domain/frame_codec.c (p. 395); src/domain/protocol_codec.h (p. 397); src/domain/protocol_codec.c (p. 402)

CHAPTER 15

15.3 State, telemetry, events and command outcomes

State describes the controller: boot ID, uptime, power mode, live epoch, configuration revision, enabled sensors, Wi-Fi status and Safety masks. Telemetry describes metric observations. Result correlates a command ID with success, busy or error. Event reports a journal transition.

Do not turn an old number into new data

Telemetry includes value, unit, health, age_ms, seq, quality q and cal_rev. Disabled or unusable metrics send value null. Both OK and UNCALIBRATED observations are checked for age at encoding; old ones are sent STALE with null. This explicit check matters because the generic readingHealthAt ages only OK.

Acquisition assigns a new sequence for every stored observation, including failure. A repeated telemetry packet can therefore be recognized as the same observation by boot ID and sequence. A new boot ID distinguishes a reset from counter continuity. A transmitted value must be interpreted using its unit; light can change from mV to lux.

Commands have owners

get_sensors, get_calibration and cal_capture can be answered from copied state. set_sensor_enabled, set/forget_wifi and set/reset_calibration go to Configuration. reset_latch goes to Safety. No operation here edits alarm limits, directly writes relay GPIO or assigns pins.

Calibration x values use input times 10. pH reference y uses pH times 1000; TDS and light y use times 10. Thus module 2500 mV and pH 6.86 become x 25000 and y 6860. The actual command fields and limits are printed in protocol_codec.h.

Bounded event history

Safety appends processed events to a 64-record RAM ring. A reader cursor begins at the next new event, not at historical replay. Falling behind the oldest retained event reports gap. The BLE sender advances its cursor only after send success; an unsent event is retried. Reboot loses this history.

Original source in this PDF: src/transports/frames.c (p. 425); src/domain/protocol_codec.c (p. 402); src/domain/event_journal.c (p. 296); src/transports/ble_transport.c (p. 436)

CHAPTER 15

15.4 Reading the two C++ adapters

Most controller logic is C. ble_gatt.cpp and wifi_lan.cpp use C++ because NimBLE and Arduino WiFi expose C++ interfaces. Their public functions present a C-compatible boundary to the rest of the firmware.

Names, constants and references

`extern "C"` gives an exported function C language linkage. It does not convert its implementation into C or synchronize its data. The unnamed `namespace { ... }` confines adapter names to that translation unit. `constexpr` requires a constant-expression initializer for these constants, such as the preferred MTU.

`nullptr` is the C++ null-pointer value. In `const NimBLEConnInfo& info`, the ampersand declares a reference: info aliases the supplied connection record rather than copying it. The const qualifier prevents modification through that reference. By contrast, a pointer is written with a star and can represent no object.

Classes provide library callback implementations

ServerCallbacks derives from NimBLEServerCallbacks. Public inheritance makes it usable through the library's callback interface. `override` checks that a method matches a virtual method in the base class; `final` prevents further derivation from this callback class. NimBLE calls those methods when connection events occur. A class method defined without an explicit access label is private by default. NimBLE invokes these overrides through its public base callback interface; private visibility on the derived method does not block that virtual dispatch.

The `->` operator accesses a member through a pointer, and the dot accesses a member on an object or reference. `NimBLEDevice::startSecurity(...)` uses the scope operator to select a class's static member function. A C++ static member function has no per-instance this pointer; it is a different use of static from a file-local C function.

Small callables and atomic link facts

`stats([](BleGattStats& s) { ++s.connects; });` passes a lambda, a small callable object. Empty brackets capture no surrounding variables; `[reason]` captures a copy of reason. `template <typename F>` lets stats accept those callable types and run each update under its lock.

`std::atomic<T>` protects an individual link fact; load, store and fetch_add express its operations. The adapter's plain calls use the default sequentially consistent ordering: these atomic operations participate in one order consistent with each thread's program order. They still need the session and ownership protocols described next. [R16]

Original source in this PDF: src/transports/ble_gatt.cpp (p. 429); src/transports/wifi_lan.cpp (p. 449)

CHAPTER 15

15.5 Device-side BLE: short callbacks, one transport owner

ble_gatt.cpp is the adapter to NimBLE-Arduino. It owns characteristics, authentication and session generation. ble_transport.c owns framing, command dispatch and send order. The desktop app is not needed to understand either role and is excluded from this book.

The small C++ boundary

NimBLE is a C++ library, so the adapter uses classes, overridden callbacks, atomics and extern C exported functions. extern C exposes names callable from C; it does not make shared state safe. Atomic secure, connection, MTU and generation fields bridge host callbacks and the transport task.

The GATT service has State, Telemetry, Command, Result and an authenticated Auth read. The preferred MTU is 247, but the negotiated value determines chunks: MTU minus 3 bytes, with the default 20-byte payload before a larger negotiation. Never assume one notification holds a whole JSON message.

Authenticate before dispatch

The adapter requests bonding, MITM authentication and LE Secure Connections. A secure link must be encrypted, authenticated, bonded and use a 16-byte key. The first bond is treated as owner; an additional newly paired owner is rejected. The passkey is generated at radio startup and displayed on serial. Holding BOOT for 10 s during runtime clears bonds and disconnects.

The RX callback runs in the NimBLE host task. It copies bytes to a 2048-byte stream buffer with zero wait and notifies BleTransport. It does not parse, write flash or touch sensors. An overflow is counted; this finite receive buffer does not make delivery lossless.

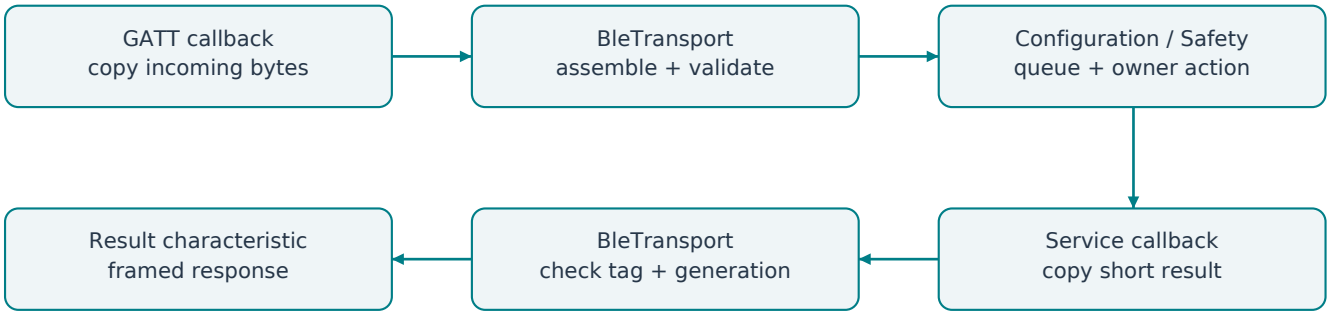
The next section follows command and reply lifetimes across a connection. Session generations prevent an old connection from authorising later work, while the send path preserves framing through congestion and disconnects.

Original source in this PDF: src/transports/ble_gatt.h (p. 427); src/transports/ble_gatt.cpp (p. 429); src/transports/ble_transport.c (p. 436)

CHAPTER 15

15.6 A command crosses tasks without moving ownership

BleTransport wakes at most every 50 ms or on notification. Its order is commands/results, alarm events, State, then Telemetry. State is refreshed every 2 s, telemetry every 3 s. Measurements continue at their acquisition cadence regardless of the slower transmission rate.



The link carries requests; it never takes sensor or relay ownership.

Keep a reply tied to its request

There are eight pending slots. Each retains command ID, operation, session generation and sequence. The internal tag combines slot and sequence, so a late reply cannot match a slot reused for another command. Configuration and Safety callbacks only copy a small result into a queue and wake the transport.

Session generation increments on connect and disconnect. Services discard a request from a closed session before handling it; transport also refuses to answer an old generation on a new link. Commands expire after 5 s; orphaned pending slots are freed on session change or after 10 s. This does not roll back a request already committed before disconnection.

A framed stream cannot skip its own middle

Before the first telemetry chunk is sent, a congested notification can drop the whole frame. Once transmission begins, the code retries each blocked chunk up to 40 tries with 5 ms waits. If the stream tears after a partial frame, it disconnects and marks that generation unusable to force resynchronization.

The wait bound is per chunk, not a universal 200 ms bound for a complete large frame. That distinction matters for watchdog and progress assumptions under prolonged congestion. sendFrame statistics record the observed maximum send time.

On session change the assembler and receive buffer reset. The journal cursor starts at new events on a secure connection; it does not replay all history accumulated while disconnected. Gap handling signals some retained-send loss, not guaranteed end-to-end event delivery.

Original source in this PDF: src/transports/ble_transport.c (p. 436); src/services/configuration.c (p. 374); src/services/safety.c (p. 115)

CHAPTER 15

15.7 Wi-Fi joins in steps and backs off after failure

WifiMqtt is the sole application owner of the station adapter. It wakes every 50 ms and progresses IDLE, ATTEMPT, WAIT or CONNECTED. There is currently no MQTT client or LAN data server despite the retained filenames.

Follow configuration separately from association

Configuration stores credentials and increments a change counter. Wi-Fi notices the counter, tears down the old association and begins with the new settings. With no SSID it stays idle. A connection requires both WL_CONNECTED and a nonzero local IP, so DHCP completion matters.

An attempt has a 15 s timeout. On failure the task schedules a retry instead of blocking for an access point. Backoff grows from 1 s toward 60 s, with 20 percent jitter. Without jitter, several devices recovering together could retry in lockstep. Success resets the backoff. A connected link loss returns to the retry path.

Why another C++ adapter

wifi_lan.cpp wraps Arduino WiFi calls in a small C interface. It disables library persistence and automatic reconnect so Configuration owns saved credentials and WifiMqtt owns retries. Modem sleep remains enabled for BLE coexistence. The adapter name does not imply a socket server exists.

Power requests have an acknowledgement

PowerCoordinator sets stopRequested and wakes the task. Wi-Fi disconnects, requests WIFI_OFF and records whether that succeeded. The power state machine waits for radioOff, separately from the acquisition release. Return requests station mode and reevaluates stored credentials.

Passwords are neither logged nor included in State. They are persisted by Configuration; this code does not establish encrypted-at-rest storage. A successful stored result means persistence succeeded, not that a network joined or external data was delivered.

Original source in this PDF: src/transports/wifi_mqtt.c (p. 453); src/transports/wifi_lan.cpp (p. 449); src/domain/backoff.c (p. 421)

ORIGINAL SOURCE / CHAPTER 15 / 1 OF 1

src/domain/frame_codec.h

Original lines 1-45 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // Length-prefixed frame reassembly (FirmwareContract.md v2, §17). Pure C.
2  // Work package WP8 (Stage 5). Shared by the BLE and LAN transports.
3  //
4  // Every frame is a 2-byte little-endian payload length followed by that many
5  // bytes of UTF-8 JSON. Transport packets may split or merge frames, so the
6  // receiver feeds whatever arrives and takes complete payloads out one by one.
7  // A length of 0 or above FRAME_MAX_PAYLOAD is a protocol error: the buffer is
8  // dropped (the caller decides whether to disconnect).
9  #pragma once
10 #include <stdbool.h>
11 #include <stddef.h>
12 #include <stdint.h>
13
14 #define FRAME_HEADER_BYTES ((size_t)2)
15 #define FRAME_MAX_PAYLOAD ((size_t)4096)
16
17 typedef uint8_t FrameStatus;
18 enum {
19   FRAME_NEED_MORE = 0,  // all input consumed, no complete frame yet
20   FRAME_READY,          // one payload available; feed the rest of the input again
21   FRAME_ERROR           // bad length; assembler reset
22 };
23
24 typedef struct FrameAssembler {
25   uint8_t buf[FRAME_HEADER_BYTES + FRAME_MAX_PAYLOAD];
26   size_t used;
27   bool ready;          // buf holds a complete frame returned by the last feed
28   uint32_t rejected;   // FRAME_ERROR count
29 } FrameAssembler;
30
31 void frameAssemblerInit(FrameAssembler* self);
32
33 // Drop any partial frame (e.g. on disconnect). Keeps the rejected counter.
34 void frameAssemblerReset(FrameAssembler* self);
35
36 // Consume bytes from data[0..len) until one frame completes or the input ends.
37 // *consumed says how many bytes were used; call again with the remainder.
38 // On FRAME_READY, *payload / *payloadLen point into the assembler and stay
39 // valid until the next feed.
40 FrameStatus frameAssemblerFeed(FrameAssembler* self, const uint8_t* data, size_t len,
41                                size_t* consumed, const uint8_t** payload, size_t*
>  payloadLen);
42
43 // Write the 2-byte header for a payload of payloadLen bytes. false if the
44 // length is 0 or above FRAME_MAX_PAYLOAD.
45 bool frameWriteHeader(uint8_t out[2], size_t payloadLen);
```

Return to guided explanation, p. 386

GUIDED SOURCE READING / CHAPTER 15

Read a fragmented message without assuming packet boundaries

BLE notifications carry chunks, while commands are complete length-prefixed messages. The assembler bridges those two sizes using persistent state.

Original excerpt: src/domain/frame_codec.c (p. 395) / lines 24-36. The complete unchanged file follows.

```
24      if (self->used < FRAME_HEADER_BYTES) {
25        self->buf[self->used++] = data[(*consumed)++];
26        if (self->used == FRAME_HEADER_BYTES) {
27          const size_t declared = declaredLength(self);
28          if (declared == 0 || declared > FRAME_MAX_PAYLOAD) {
29            frameAssemblerReset(self);
30            if (self->rejected < UINT32_MAX) ++self->rejected;
31            return FRAME_ERROR;
32          }
33        }
34        continue;
35      }
36      const size_t total = FRAME_HEADER_BYTES + declaredLength(self);
```

While fewer than two header bytes exist, each incoming byte is stored and used increases. Declared length is checked only after the whole header is available; checking after the first byte would mistake a partial length for the final value.

Zero or more than 4096 resets the partial frame and reports ERROR before copying a payload of that size. The rejected counter saturates rather than wrapping. Once length is valid, the later code copies at most the smaller of remaining payload and remaining input.

READY is returned at exactly one complete frame. consumed tells the caller which input bytes remain for another feed. The payload pointer refers into assembler storage and is valid only until reuse, so BleTransport copies the command before processing replies that use other shared buffers.

For a 300-byte payload, header 0x2C,0x01 may arrive as two separate packets. NEED_MORE after 0x2C is the correct outcome. The header-first ordering is what makes arbitrary fragmentation possible without inventing a message boundary at each callback.

ORIGINAL SOURCE / CHAPTER 15 / 1 OF 2

src/domain/frame_codec.c

Original lines 1-28 | Code and comments unchanged; long lines wrap visually

frameAssemblerInit (L5-5): Check dependencies, configure this module and return whether initialisation succeeded.
frameAssemblerReset (L7-10): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.

```
#include "domain/frame_codec.h"

#include <string.h>

void frameAssemblerInit(FrameAssembler* self) { memset(self, 0, sizeof(*self)); }

void frameAssemblerReset(FrameAssembler* self) {
  self->used = 0;
  self->ready = false;
}

static size_t declaredLength(const FrameAssembler* self) {
  return (size_t)self->buf[0] | ((size_t)self->buf[1] << 8);
}

FrameStatus frameAssemblerFeed(FrameAssembler* self, const uint8_t* data, size_t len,
                               size_t* consumed, const uint8_t** payload, size_t* payloadLen)
{
  *consumed = 0;
  *payload = NULL;
  *payloadLen = 0;
  if (self->ready) frameAssemblerReset(self);  // the previous payload is released

  while (*consumed < len) {
    if (self->used < FRAME_HEADER_BYTES) {
      self->buf[self->used++] = data[(*consumed)++];
      if (self->used == FRAME_HEADER_BYTES) {
        const size_t declared = declaredLength(self);
        if (declared == 0 || declared > FRAME_MAX_PAYLOAD) {
```

Return to guided explanation, p. 394

ORIGINAL SOURCE / CHAPTER 15 / 2 OF 2

src/domain/frame_codec.c

Original lines 29-57 | Code and comments unchanged; long lines wrap visually

frameAssemblerFeed (L16-50): Accumulate a two-byte length and payload; stop at one complete frame and report consumed bytes for the caller remainder.
frameWriteHeader (L52-57): Reject invalid lengths and encode the payload size low byte first.

```
          frameAssemblerReset(self);
          if (self->rejected < UINT32_MAX) ++self->rejected;
          return FRAME_ERROR;
        }
      }
      continue;
    }
    const size_t total = FRAME_HEADER_BYTES + declaredLength(self);
    size_t take = total - self->used;
    if (take > len - *consumed) take = len - *consumed;
    memcpy(&self->buf[self->used], &data[*consumed], take);
    self->used += take;
    *consumed += take;
    if (self->used == total) {
      self->ready = true;
      *payload = &self->buf[FRAME_HEADER_BYTES];
      *payloadLen = total - FRAME_HEADER_BYTES;
      return FRAME_READY;
    }
  }
  return FRAME_NEED_MORE;
}

bool frameWriteHeader(uint8_t out[2], size_t payloadLen) {
  if (payloadLen == 0 || payloadLen > FRAME_MAX_PAYLOAD) return false;
  out[0] = (uint8_t)(payloadLen & 0xFFu);
  out[1] = (uint8_t)(payloadLen >> 8);
  return true;
}
```

Return to guided explanation, p. 394

ORIGINAL SOURCE / CHAPTER 15 / 1 OF 4

src/domain/protocol_codec.h

Original lines 1-46 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // JSON encoders and command parser for contract v2 (FirmwareContract.md).
2  // Pure C, no heap, no cJSON (the native test env has no ESP SDK). WP8.
3  //
4  // Encoders write the JSON payload only (no length header) into a JsonWriter
5  // over a caller buffer. On overflow the writer stops and reports it; the
6  // caller must not send a truncated frame.
7  //
8  // The parser accepts one flat command object with an optional flat "data"
9  // object, validates every field the operation needs and ignores unknown keys.
10 // Anything malformed is rejected before it can reach Configuration or Safety.
11 #pragma once
12 #include <stdbool.h>
13 #include <stddef.h>
14 #include <stdint.h>
15
16 #include "domain/calibration.h"
17 #include "domain/reading.h"
18 #include "domain/sensor_set.h"
19
20 #define PROTOCOL_VERSION 2
21 #define PROTOCOL_ID_MAX 40
22 #define PROTOCOL_SSID_MAX 32        // bytes (802.11)
23 #define PROTOCOL_PASSWORD_MIN 8     // WPA2 passphrase, printable ASCII
24 #define PROTOCOL_PASSWORD_MAX 63
25 #define PROTOCOL_MAX_RULE_ID 15     // masks are 16 bits; Safety checks ruleCount
26
27 // ---------------------------------------------------------------- writer
28
29 typedef struct JsonWriter {
30   char* buf;
31   size_t cap;
32   size_t len;
33   bool overflow;
34 } JsonWriter;
35
36 void jsonWriterInit(JsonWriter* w, char* buf, size_t cap);
37 void jsonRaw(JsonWriter* w, const char* text);
38 void jsonString(JsonWriter* w, const char* s);  // quoted and escaped
39 void jsonKey(JsonWriter* w, const char* key);   // "key": (no comma handling)
40 void jsonU32(JsonWriter* w, uint32_t v);
41 void jsonI32(JsonWriter* w, int32_t v);
42 void jsonU64(JsonWriter* w, uint64_t v);
43 void jsonFloat(JsonWriter* w, float v);         // null for NaN / inf
44 void jsonBool(JsonWriter* w, bool v);
45 // Writes [i, j, ...] for the bits set in mask below `count`.
46 void jsonBitList(JsonWriter* w, uint16_t mask, uint8_t count);
```

Return to guided explanation, p. 386

ORIGINAL SOURCE / CHAPTER 15 / 2 OF 4

src/domain/protocol_codec.h

Original lines 47-90 | Code and comments unchanged; long lines wrap visually

jsonWriterLength (L49-49): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.

```
47
48  // Payload length, or 0 if the writer overflowed.
49  static inline size_t jsonWriterLength(const JsonWriter* w) { return w->overflow ? 0 : w->len;
 >  }
50
51  // ---------------------------------------------------------------- encoders
52
53  typedef struct StateView {
54    const char* node;
55    const char* fw;
56    uint32_t boot;
57    MonoMs uptimeMs;
58    const char* link;        // "ble" (the only data link)
59    const char* mode;        // lowercase power mode name
60    uint32_t powerEpoch;
61    uint16_t cfgRev;
62    const char* wifiState;   // unconfigured / connecting / connected / failed
63    const char* wifiSsid;    // NULL or "" when unconfigured
64    const char* wifiIp;      // only sent when connected
65    int32_t wifiRssi;
66    bool relayOn;
67    const char* lampReason;
68    bool interlock;
69    uint16_t latchedMask;
70    uint16_t activeMask;
71    uint16_t staleMask;
72    uint8_t ruleCount;
73    const char* sgp30Baseline;
74    SensorMask enabled;
75  } StateView;
76
77  size_t protocolEncodeState(const StateView* s, char* buf, size_t cap);
78
79  // All METRIC_COUNT entries keyed by metricName(). A value is sent only when
80  // the sensor is enabled, the stored health is OK or UNCALIBRATED and the
81  // sample is not older than alarmStaleLimitMs(); otherwise value is null and
82  // an aged OK/UNCALIBRATED reading is reported as STALE.
83  size_t protocolEncodeTelemetry(const Snapshot* snap, MonoMs now, uint32_t boot, uint16_t
 >  cfgRev,
84                                 SensorMask enabled, char* buf, size_t cap);
85
86  // status "ok" with dataJson (a complete JSON object, NULL means {}).
87  size_t protocolEncodeResultOk(const char* id, const char* dataJson, char* buf, size_t cap);
88  // status "error" with a short error code (see protocolErrorName()).
89  size_t protocolEncodeResultError(const char* id, const char* error, char* buf, size_t cap);
90  size_t protocolEncodeResultBusy(const char* id, char* buf, size_t cap);
```

Return to guided explanation, p. 386

ORIGINAL SOURCE / CHAPTER 15 / 3 OF 4

src/domain/protocol_codec.h

Original lines 91-135 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
91
92  typedef struct EventView {
93    uint32_t n;
94    uint8_t rule;
95    const char* level;  // INFO / WARNING / CRITICAL
96    const char* kind;   // trip / clear / reset
97    uint32_t ageMs;
98    bool gap;
99  } EventView;
100
101 size_t protocolEncodeEvent(const EventView* e, char* buf, size_t cap);
102
103 // {"sensors":[{"key","label","metrics":[...],"enabled","depends_note"},...]}
104 size_t protocolEncodeSensorList(SensorMask enabled, char* buf, size_t cap);
105
106 // Calibratable sensors (pH, TDS, light) and their stored fits.
107 typedef struct CalibrationView {
108   LinearCalibration ph;
109   LinearCalibration tds;
110   LinearCalibration light;
111   uint8_t overriddenMask;  // CAL_BIT_* : fitted on the device by the user
112 } CalibrationView;
113
114 // {"calibration":{"ph":{"overridden","rev","valid","slope","offset"},"tds":...,"light":...}}
115 size_t protocolEncodeCalibration(const CalibrationView* v, char* buf, size_t cap);
116
117 // {"sensor":"ph","x":<x scaled>,"age_ms":n}
118 size_t protocolEncodeCapture(SensorId sensor, int32_t xScaled, uint32_t ageMs, char* buf,
  > size_t cap);
119
120 // ------------------------------------------------------------------ commands
121
122 typedef uint8_t CommandOp;
123 enum {
124   CMD_OP_NONE = 0,
125   CMD_OP_GET_SENSORS,
126   CMD_OP_SET_SENSOR_ENABLED,
127   CMD_OP_RESET_LATCH,
128   CMD_OP_SET_WIFI,
129   CMD_OP_FORGET_WIFI,
130   CMD_OP_GET_CALIBRATION,
131   CMD_OP_CAL_CAPTURE,
132   CMD_OP_SET_CALIBRATION,
133   CMD_OP_RESET_CALIBRATION
134 };
135
```

Return to guided explanation, p. 386

ORIGINAL SOURCE / CHAPTER 15 / 4 OF 4

src/domain/protocol_codec.h

Original lines 136-180 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
136 // Calibration commands carry scaled integers (the parser keeps no floats):
137 //   ph     x = module mV x 10,           y = pH x 1000   (two points)
138 //   tds    x = uncalibrated ppm x 10,    y = ppm x 10    (one point, K)
139 //   light  x = pin mV x 10,              y = lux x 10    (one point, gain)
140 // cal_capture returns x in the same scale.
141 #define CAL_X_SCALE 10.0f
142 #define CAL_PH_Y_SCALE 1000.0f
143 #define CAL_Y_SCALE 10.0f
144
145 typedef uint8_t ProtocolError;
146 enum {
147   PROTOCOL_OK = 0,
148   PROTOCOL_ERR_BAD_REQUEST,   // not JSON / not an object / wrong version / no id
149   PROTOCOL_ERR_UNKNOWN_OP,
150   PROTOCOL_ERR_BAD_FIELD,     // missing, wrong type or out of range
151   PROTOCOL_ERR_RULE_LATCHED,
152   PROTOCOL_ERR_RULE_ACTIVE,
153   PROTOCOL_ERR_EXPIRED,
154   PROTOCOL_ERR_TIMEOUT,
155   PROTOCOL_ERR_STORAGE,
156   PROTOCOL_ERR_STALE,         // cal_capture: no fresh usable reading (or TDS without water
    > temperature)
157   PROTOCOL_ERR_BAD_FIT        // set_calibration: the points give an implausible fit
158 };
159
160 typedef struct Command {
161   CommandOp op;
162   char id[PROTOCOL_ID_MAX + 1];  // set as soon as it parses, even on later errors
163   SensorId sensor;
164   bool enabled;
165   uint8_t rule;
166   char ssid[PROTOCOL_SSID_MAX + 1];
167   char password[PROTOCOL_PASSWORD_MAX + 1];
168   int32_t calX1, calY1, calX2, calY2;  // set_calibration, scaled (above)
169   bool calTwoPoints;
170 } Command;
171
172 // Sensors that take a calibration command: SENSOR_PH, SENSOR_TDS, SENSOR_LIGHT.
173 bool protocolSensorCalibratable(SensorId s);
174
175 ProtocolError protocolParseCommand(const uint8_t* payload, size_t len, Command* out);
176
177 // Contract error string ("bad_request", "rule_latched", ...).
178 const char* protocolErrorName(ProtocolError e);
179
180 const char* commandOpName(CommandOp op);
```

Return to guided explanation, p. 386

GUIDED SOURCE READING / CHAPTER 15

Bounds checking belongs before the buffer copy

Once a service has state or a result to send, JsonWriter appends explicit fields to caller-owned storage. Its small put helper establishes the rule every encoder relies on.

Original excerpt: src/domain/protocol_codec.c (p. 402) / lines 19-28. The complete unchanged file follows.

```
19  static void put(JsonWriter* w, const char* text, size_t n) {
20    if (w->overflow) return;
21    if (w->len + n + 1 > w->cap) {  // keep room for the terminator
22      w->overflow = true;
23      return;
24    }
25    memcpy(&w->buf[w->len], text, n);
26    w->len += n;
27    w->buf[w->len] = '\0';
28  }
```

An existing overflow stops further writes. The capacity check includes n bytes plus one NUL terminator before memcpy. If the copy came first, discovering overflow afterwards could not undo overwritten adjacent memory.

The bytes are appended at the current length, then len advances, then the terminator is placed at the new end. That order makes the buffer a valid C string after each successful append. The wire payload length excludes that terminator.

jsonWriterLength returns zero on overflow. Callers use that result to avoid transmitting an incomplete JSON object. Partial bytes can remain in the buffer, but the explicit failure state says they are not a valid frame.

The parser implements the opposite boundary: it checks received syntax and required fields before dispatching a command. Neither encoder nor parser applies a calibration or writes the relay. They translate representations; the service owner makes the state-changing decision. This division keeps buffer correctness separate from command authority.

ORIGINAL SOURCE / CHAPTER 15 / 1 OF 18

src/domain/protocol_codec.c

Original lines 1-51 | Code and comments unchanged; long lines wrap visually

jsonWriterInit (L11-17): Check dependencies, configure this module and return whether initialisation succeeded. **put (L19-28)**: Append to the caller JSON buffer only if data plus a NUL terminator fits; overflow prevents a truncated send.

```c
#include "domain/protocol_codec.h"

#include <math.h>
#include <stdio.h>
#include <string.h>

#include "domain/alarm.h"

// ============================================================== writer

void jsonWriterInit(JsonWriter* w, char* buf, size_t cap) {
  w->buf = buf;
  w->cap = cap;
  w->len = 0;
  w->overflow = cap == 0;
  if (cap > 0) buf[0] = '\0';
}

static void put(JsonWriter* w, const char* text, size_t n) {
  if (w->overflow) return;
  if (w->len + n + 1 > w->cap) {  // keep room for the terminator
    w->overflow = true;
    return;
  }
  memcpy(&w->buf[w->len], text, n);
  w->len += n;
  w->buf[w->len] = '\0';
}

void jsonRaw(JsonWriter* w, const char* text) { put(w, text, strlen(text)); }

void jsonString(JsonWriter* w, const char* s) {
  put(w, "\"", 1);
  if (s != NULL) {
    for (const unsigned char* p = (const unsigned char*)s; *p != '\0'; ++p) {
      switch (*p) {
        case '"': put(w, "\\\"", 2); break;
        case '\\': put(w, "\\\\", 2); break;
        case '\n': put(w, "\\n", 2); break;
        case '\r': put(w, "\\r", 2); break;
        case '\t': put(w, "\\t", 2); break;
        default:
          if (*p < 0x20) {
            char esc[8];
            snprintf(esc, sizeof(esc), "\\u%04x", (unsigned)*p);
            put(w, esc, 6);
          } else {
            put(w, (const char*)p, 1);  // UTF-8 bytes pass through
          }
      }
    }
```

Return to guided explanation, p. 401

ORIGINAL SOURCE / CHAPTER 15 / 2 OF 18

src/domain/protocol_codec.c

Original lines 52-102 | Code and comments unchanged; long lines wrap visually

jsonString (L32-54): A bounded JSON writer and command parser encode explicit fields and reject malformed or unsupported requests.
jsonKey (L56-59): A bounded JSON writer and command parser encode explicit fields and reject malformed or unsupported requests.

```
  }
  put(w, "\"", 1);
}

void jsonKey(JsonWriter* w, const char* key) {
  jsonString(w, key);
  put(w, ":", 1);
}

void jsonU32(JsonWriter* w, uint32_t v) {
  char tmp[12];
  const int n = snprintf(tmp, sizeof(tmp), "%lu", (unsigned long)v);
  put(w, tmp, (size_t)n);
}

void jsonI32(JsonWriter* w, int32_t v) {
  char tmp[12];
  const int n = snprintf(tmp, sizeof(tmp), "%ld", (long)v);
  put(w, tmp, (size_t)n);
}

void jsonU64(JsonWriter* w, uint64_t v) {
  char tmp[24];
  const int n = snprintf(tmp, sizeof(tmp), "%llu", (unsigned long long)v);
  put(w, tmp, (size_t)n);
}

void jsonFloat(JsonWriter* w, float v) {
  if (isnan(v) || isinf(v)) {
    put(w, "null", 4);
    return;
  }
  char tmp[24];
  const int n = snprintf(tmp, sizeof(tmp), "%.7g", (double)v);  // float has ~7 digits
  put(w, tmp, (size_t)n);
}

void jsonBool(JsonWriter* w, bool v) {
  if (v) {
    put(w, "true", 4);
  } else {
    put(w, "false", 5);
  }
}

void jsonBitList(JsonWriter* w, uint16_t mask, uint8_t count) {
  put(w, "[", 1);
  bool first = true;
  for (uint8_t i = 0; i < count && i < 16; ++i) {
    if ((mask & (1u << i)) == 0) continue;
    if (!first) put(w, ",", 1);
```

Return to guided explanation, p. 401

ORIGINAL SOURCE / CHAPTER 15 / 3 OF 18

src/domain/protocol_codec.c

Original lines 103-153 | Code and comments unchanged; long lines wrap visually

jsonBitList (L97-107): A bounded JSON writer and command parser encode explicit fields and reject malformed or unsupported requests.
header (L111-114): A bounded JSON writer and command parser encode explicit fields and reject malformed or unsupported requests.

```
    jsonU32(w, i);
    first = false;
  }
  put(w, "]", 1);
}

// ================================================================ encoders

static void header(JsonWriter* w, const char* type) {
  jsonRaw(w, "{\"v\":2,\"type\":");
  jsonString(w, type);
}

size_t protocolEncodeState(const StateView* s, char* buf, size_t cap) {
  JsonWriter w;
  jsonWriterInit(&w, buf, cap);
  header(&w, "state");
  jsonRaw(&w, ",");
  jsonKey(&w, "node");
  jsonString(&w, s->node);
  jsonRaw(&w, ",");
  jsonKey(&w, "fw");
  jsonString(&w, s->fw);
  jsonRaw(&w, ",");
  jsonKey(&w, "boot");
  jsonU32(&w, s->boot);
  jsonRaw(&w, ",");
  jsonKey(&w, "uptime_ms");
  jsonU64(&w, s->uptimeMs);
  jsonRaw(&w, ",");
  jsonKey(&w, "link");
  jsonString(&w, s->link);
  jsonRaw(&w, ",");
  jsonKey(&w, "mode");
  jsonString(&w, s->mode);
  jsonRaw(&w, ",");
  jsonKey(&w, "power_epoch");
  jsonU32(&w, s->powerEpoch);
  jsonRaw(&w, ",");
  jsonKey(&w, "cfg_rev");
  jsonU32(&w, s->cfgRev);

  jsonRaw(&w, ",\"wifi\":{");
  jsonKey(&w, "state");
  jsonString(&w, s->wifiState);
  if (s->wifiSsid != NULL && s->wifiSsid[0] != '\0') {
    jsonRaw(&w, ",");
    jsonKey(&w, "ssid");
    jsonString(&w, s->wifiSsid);
  }
  if (s->wifiIp != NULL && s->wifiState != NULL && strcmp(s->wifiState, "connected") == 0) {
```

Return to guided explanation, p. 401

ORIGINAL SOURCE / CHAPTER 15 / 4 OF 18

src/domain/protocol_codec.c

Original lines 154-204 | Code and comments unchanged; long lines wrap visually

protocolEncodeState (L116-196): A bounded JSON writer and command parser encode explicit fields and reject malformed or unsupported requests.
encodeMetric (L198-238): Age OK/UNCALIBRATED data, emit null for unusable/disabled values and retain unit, health, sequence and quality metadata.

```
    jsonRaw(&w, ",");
    jsonKey(&w, "ip");
    jsonString(&w, s->wifiIp);
    jsonRaw(&w, ",");
    jsonKey(&w, "rssi");
    jsonI32(&w, s->wifiRssi);
  }
  jsonRaw(&w, "}");

  jsonRaw(&w, ",\"safety\":{");
  jsonKey(&w, "relay");
  jsonBool(&w, s->relayOn);
  jsonRaw(&w, ",");
  jsonKey(&w, "lamp_reason");
  jsonString(&w, s->lampReason);
  jsonRaw(&w, ",");
  jsonKey(&w, "interlock");
  jsonBool(&w, s->interlock);
  jsonRaw(&w, ",");
  jsonKey(&w, "latched");
  jsonBitList(&w, s->latchedMask, s->ruleCount);
  jsonRaw(&w, ",");
  jsonKey(&w, "active");
  jsonBitList(&w, s->activeMask, s->ruleCount);
  jsonRaw(&w, ",");
  jsonKey(&w, "stale");
  jsonBitList(&w, s->staleMask, s->ruleCount);
  jsonRaw(&w, "}");

  jsonRaw(&w, ",");
  jsonKey(&w, "sgp30_baseline");
  jsonString(&w, s->sgp30Baseline);
  jsonRaw(&w, ",\"enabled\":[");
  bool first = true;
  for (uint8_t i = 0; i < SENSOR_COUNT; ++i) {
    if (!sensorMaskHas(s->enabled, (SensorId)i)) continue;
    if (!first) jsonRaw(&w, ",");
    jsonString(&w, sensorKey((SensorId)i));
    first = false;
  }
  jsonRaw(&w, "]}");
  return jsonWriterLength(&w);
}

static void encodeMetric(JsonWriter* w, const Reading* r, MonoMs now, bool enabled) {
  const bool measured = r->intervalEndMs != 0 && r->health != HEALTH_STARTING;
  const MonoMs age = measured && now > r->intervalEndMs ? now - r->intervalEndMs : 0;
  Health health = r->health;
  bool usable = health == HEALTH_OK || health == HEALTH_UNCALIBRATED;
  if (usable && age > alarmStaleLimitMs(r->metric)) {
    health = HEALTH_STALE;  // never send a retained value as a new observation (§10)
```

Return to guided explanation, p. 401

ORIGINAL SOURCE / CHAPTER 15 / 5 OF 18

src/domain/protocol_codec.c

Original lines 205-254 | Code and comments unchanged; long lines wrap visually

encodeMetric (L198-238): Age OK/UNCALIBRATED data, emit null for unusable/disabled values and retain unit, health, sequence and quality metadata.
protocolEncodeTelemetry (L240-259): A bounded JSON writer and command parser encode explicit fields and reject malformed or unsupported requests.

```
205     usable = false;
206   }
207
208   jsonKey(w, metricName(r->metric));
209   jsonRaw(w, "{\"value\":");
210   if (usable && enabled) {
211     jsonFloat(w, r->value);
212   } else {
213     jsonRaw(w, "null");
214   }
215   jsonRaw(w, ",");
216   jsonKey(w, "unit");
217   jsonString(w, unitName(r->unit));
218   jsonRaw(w, ",");
219   jsonKey(w, "health");
220   jsonString(w, healthName(health));
221   jsonRaw(w, ",");
222   jsonKey(w, "age_ms");
223   if (measured) {
224     jsonU64(w, age);
225   } else {
226     jsonRaw(w, "null");
227   }
228   jsonRaw(w, ",");
229   jsonKey(w, "seq");
230   jsonU32(w, r->sequence);
231   jsonRaw(w, ",");
232   jsonKey(w, "q");
233   jsonU32(w, r->qualityFlags);
234   jsonRaw(w, ",");
235   jsonKey(w, "cal_rev");
236   jsonU32(w, r->calibrationRevision);
237   jsonRaw(w, "}");
238 }
239
240 size_t protocolEncodeTelemetry(const Snapshot* snap, MonoMs now, uint32_t boot, uint16_t
  > cfgRev,
241                                SensorMask enabled, char* buf, size_t cap) {
242   JsonWriter w;
243   jsonWriterInit(&w, buf, cap);
244   header(&w, "telemetry");
245   jsonRaw(&w, ",");
246   jsonKey(&w, "boot");
247   jsonU32(&w, boot);
248   jsonRaw(&w, ",");
249   jsonKey(&w, "cfg_rev");
250   jsonU32(&w, cfgRev);
251   jsonRaw(&w, ",\"m\":{");
252   for (uint8_t i = 0; i < METRIC_COUNT; ++i) {
253     if (i > 0) jsonRaw(&w, ",");
254     const Reading* r = &snap->readings[i];
```

Return to guided explanation, p. 401

ORIGINAL SOURCE / CHAPTER 15 / 6 OF 18

src/domain/protocol_codec.c

Original lines 255-305 | Code and comments unchanged; long lines wrap visually

protocolEncodeTelemetry (L240-259): A bounded JSON writer and command parser encode explicit fields and reject malformed or unsupported requests.
resultHead (L261-269): A bounded JSON writer and command parser encode explicit fields and reject malformed or unsupported requests.

```
    encodeMetric(&w, r, now, sensorMaskHas(enabled, sensorOfMetric(r->metric)));
  }
  jsonRaw(&w, "}}");
  return jsonWriterLength(&w);
}

static void resultHead(JsonWriter* w, const char* id, const char* status) {
  header(w, "result");
  jsonRaw(w, ",");
  jsonKey(w, "id");
  jsonString(w, id);
  jsonRaw(w, ",");
  jsonKey(w, "status");
  jsonString(w, status);
}

size_t protocolEncodeResultOk(const char* id, const char* dataJson, char* buf, size_t cap) {
  JsonWriter w;
  jsonWriterInit(&w, buf, cap);
  resultHead(&w, id, "ok");
  jsonRaw(&w, ",\"data\":");
  jsonRaw(&w, dataJson != NULL ? dataJson : "{}");
  jsonRaw(&w, "}");
  return jsonWriterLength(&w);
}

size_t protocolEncodeResultError(const char* id, const char* error, char* buf, size_t cap) {
  JsonWriter w;
  jsonWriterInit(&w, buf, cap);
  resultHead(&w, id, "error");
  jsonRaw(&w, ",");
  jsonKey(&w, "error");
  jsonString(&w, error);
  jsonRaw(&w, "}");
  return jsonWriterLength(&w);
}

size_t protocolEncodeResultBusy(const char* id, char* buf, size_t cap) {
  JsonWriter w;
  jsonWriterInit(&w, buf, cap);
  resultHead(&w, id, "busy");
  jsonRaw(&w, "}");
  return jsonWriterLength(&w);
}

size_t protocolEncodeEvent(const EventView* e, char* buf, size_t cap) {
  JsonWriter w;
  jsonWriterInit(&w, buf, cap);
  header(&w, "event");
  jsonRaw(&w, ",");
  jsonKey(&w, "n");
```

Return to guided explanation, p. 401

ORIGINAL SOURCE / CHAPTER 15 / 7 OF 18

src/domain/protocol_codec.c

Original lines 306-356 | Code and comments unchanged; long lines wrap visually

protocolEncodeEvent (L300-324): A bounded JSON writer and command parser encode explicit fields and reject malformed or unsupported requests.
protocolEncodeSensorList (L326-356): A bounded JSON writer and command parser encode explicit fields and reject malformed or unsupported requests.

```
306   jsonU32(&w, e->n);
307   jsonRaw(&w, ",");
308   jsonKey(&w, "rule");
309   jsonU32(&w, e->rule);
310   jsonRaw(&w, ",");
311   jsonKey(&w, "level");
312   jsonString(&w, e->level);
313   jsonRaw(&w, ",");
314   jsonKey(&w, "kind");
315   jsonString(&w, e->kind);
316   jsonRaw(&w, ",");
317   jsonKey(&w, "age_ms");
318   jsonU32(&w, e->ageMs);
319   jsonRaw(&w, ",");
320   jsonKey(&w, "gap");
321   jsonBool(&w, e->gap);
322   jsonRaw(&w, "}");
323   return jsonWriterLength(&w);
324 }
325
326 size_t protocolEncodeSensorList(SensorMask enabled, char* buf, size_t cap) {
327   JsonWriter w;
328   jsonWriterInit(&w, buf, cap);
329   jsonRaw(&w, "{\"sensors\":[");
330   for (uint8_t s = 0; s < SENSOR_COUNT; ++s) {
331     if (s > 0) jsonRaw(&w, ",");
332     jsonRaw(&w, "{");
333     jsonKey(&w, "key");
334     jsonString(&w, sensorKey((SensorId)s));
335     jsonRaw(&w, ",");
336     jsonKey(&w, "label");
337     jsonString(&w, sensorLabel((SensorId)s));
338     jsonRaw(&w, ",\"metrics\":[");
339     bool first = true;
340     for (uint8_t m = 0; m < METRIC_COUNT; ++m) {
341       if (sensorOfMetric((MetricId)m) != s) continue;
342       if (!first) jsonRaw(&w, ",");
343       jsonString(&w, metricName((MetricId)m));
344       first = false;
345     }
346     jsonRaw(&w, "],");
347     jsonKey(&w, "enabled");
348     jsonBool(&w, sensorMaskHas(enabled, (SensorId)s));
349     jsonRaw(&w, ",");
350     jsonKey(&w, "depends_note");
351     jsonString(&w, sensorDependsNote((SensorId)s));
352     jsonRaw(&w, "}");
353   }
354   jsonRaw(&w, "]}");
355   return jsonWriterLength(&w);
356 }
```

Return to guided explanation, p. 401

ORIGINAL SOURCE / CHAPTER 15 / 8 OF 18

src/domain/protocol_codec.c

Original lines 357-405 | Code and comments unchanged; long lines wrap visually

encodeFit (L358-376): A bounded JSON writer and command parser encode explicit fields and reject malformed or unsupported requests.
protocolEncodeCalibration (L378-389): A bounded JSON writer and command parser encode explicit fields and reject malformed or unsupported requests.

```
357
358 static void encodeFit(JsonWriter* w, const char* key, const LinearCalibration* c, bool
  > overridden) {
359   jsonKey(w, key);
360   jsonRaw(w, "{");
361   jsonKey(w, "overridden");
362   jsonBool(w, overridden);
363   jsonRaw(w, ",");
364   jsonKey(w, "rev");
365   jsonU32(w, c->revision);
366   jsonRaw(w, ",");
367   jsonKey(w, "valid");
368   jsonBool(w, c->valid);
369   jsonRaw(w, ",");
370   jsonKey(w, "slope");
371   jsonFloat(w, c->slope);
372   jsonRaw(w, ",");
373   jsonKey(w, "offset");
374   jsonFloat(w, c->offset);
375   jsonRaw(w, "}");
376 }
377
378 size_t protocolEncodeCalibration(const CalibrationView* v, char* buf, size_t cap) {
379   JsonWriter w;
380   jsonWriterInit(&w, buf, cap);
381   jsonRaw(&w, "{\"calibration\":{");
382   encodeFit(&w, "ph", &v->ph, (v->overriddenMask & CAL_BIT_PH) != 0);
383   jsonRaw(&w, ",");
384   encodeFit(&w, "tds", &v->tds, (v->overriddenMask & CAL_BIT_TDS) != 0);
385   jsonRaw(&w, ",");
386   encodeFit(&w, "light", &v->light, (v->overriddenMask & CAL_BIT_LIGHT) != 0);
387   jsonRaw(&w, "}}");
388   return jsonWriterLength(&w);
389 }
390
391 size_t protocolEncodeCapture(SensorId sensor, int32_t xScaled, uint32_t ageMs, char* buf,
  > size_t cap) {
392   JsonWriter w;
393   jsonWriterInit(&w, buf, cap);
394   jsonRaw(&w, "{");
395   jsonKey(&w, "sensor");
396   jsonString(&w, sensorKey(sensor));
397   jsonRaw(&w, ",");
398   jsonKey(&w, "x");
399   jsonI32(&w, xScaled);
400   jsonRaw(&w, ",");
401   jsonKey(&w, "age_ms");
402   jsonU32(&w, ageMs);
403   jsonRaw(&w, "}");
404   return jsonWriterLength(&w);
405 }
```

Return to guided explanation, p. 401

ORIGINAL SOURCE / CHAPTER 15 / 9 OF 18

src/domain/protocol_codec.c

Original lines 406-455 | Code and comments unchanged; long lines wrap visually

skipWs (L433-437): A bounded JSON writer and command parser encode explicit fields and reject malformed or unsupported requests.
expect (L439-444): A bounded JSON writer and command parser encode explicit fields and reject malformed or unsupported requests.

```
406
407 // ============================================================== parser
408
409 #define FIELD_KEY_MAX 15
410 #define FIELD_STR_MAX 64
411 #define DATA_FIELDS_MAX 8
412 #define SKIP_DEPTH_MAX 4
413
414 typedef uint8_t JsonType;
415 enum { JT_NONE = 0, JT_STRING, JT_NUMBER, JT_BOOL, JT_NULL, JT_OBJECT, JT_ARRAY };
416
417 typedef struct Field {
418   char key[FIELD_KEY_MAX + 1];
419   JsonType type;
420   char str[FIELD_STR_MAX + 1];  // decoded UTF-8 for JT_STRING
421   size_t strLen;
422   bool strTooLong;
423   bool isInteger;               // JT_NUMBER without fraction/exponent, in int32 range
424   int32_t integer;
425   bool boolean;
426 } Field;
427
428 typedef struct Parser {
429   const uint8_t* p;
430   const uint8_t* end;
431 } Parser;
432
433 static void skipWs(Parser* ps) {
434   while (ps->p < ps->end && (*ps->p == ' ' || *ps->p == '\t' || *ps->p == '\n' || *ps->p ==
  > '\r')) {
435     ++ps->p;
436   }
437 }
438
439 static bool expect(Parser* ps, char c) {
440   skipWs(ps);
441   if (ps->p >= ps->end || *ps->p != (uint8_t)c) return false;
442   ++ps->p;
443   return true;
444 }
445
446 static bool hexValue(uint8_t c, uint32_t* out) {
447   if (c >= '0' && c <= '9') *out = (uint32_t)(c - '0');
448   else if (c >= 'a' && c <= 'f') *out = (uint32_t)(c - 'a' + 10);
449   else if (c >= 'A' && c <= 'F') *out = (uint32_t)(c - 'A' + 10);
450   else return false;
451   return true;
452 }
453
454 static void appendByte(char* out, size_t cap, size_t* len, bool* tooLong, uint8_t b) {
455   if (*len < cap) {
```

Return to guided explanation, p. 401

ORIGINAL SOURCE / CHAPTER 15 / 10 OF 18

src/domain/protocol_codec.c

Original lines 456-506 | Code and comments unchanged; long lines wrap visually

appendByte (L454-461): A bounded JSON writer and command parser encode explicit fields and reject malformed or unsupported requests.
parseString (L466-518): Decode bounded string fields and escapes; reject unsupported surrogate escapes, NUL and malformed input.

```
    out[*len] = (char)b;
  } else {
    *tooLong = true;
  }
  ++*len;
}

// Parses a JSON string at ps->p (opening quote included) into out (cap bytes,
// always NUL-terminated within cap+1). Surrogate pairs are not supported (no
// field the firmware reads needs characters outside the BMP).
static bool parseString(Parser* ps, char* out, size_t cap, size_t* lenOut, bool* tooLong) {
  size_t len = 0;
  *tooLong = false;
  if (!expect(ps, '"')) return false;
  while (ps->p < ps->end) {
    const uint8_t c = *ps->p++;
    if (c == '"') {
      out[len < cap ? len : cap] = '\0';
      *lenOut = len;
      return true;
    }
    if (c < 0x20) return false;
    if (c != '\\') {
      appendByte(out, cap, &len, tooLong, c);
      continue;
    }
    if (ps->p >= ps->end) return false;
    const uint8_t e = *ps->p++;
    switch (e) {
      case '"': appendByte(out, cap, &len, tooLong, '"'); break;
      case '\\': appendByte(out, cap, &len, tooLong, '\\'); break;
      case '/': appendByte(out, cap, &len, tooLong, '/'); break;
      case 'b': appendByte(out, cap, &len, tooLong, '\b'); break;
      case 'f': appendByte(out, cap, &len, tooLong, '\f'); break;
      case 'n': appendByte(out, cap, &len, tooLong, '\n'); break;
      case 'r': appendByte(out, cap, &len, tooLong, '\r'); break;
      case 't': appendByte(out, cap, &len, tooLong, '\t'); break;
      case 'u': {
        if (ps->end - ps->p < 4) return false;
        uint32_t cp = 0;
        for (int i = 0; i < 4; ++i) {
          uint32_t nib;
          if (!hexValue(*ps->p++, &nib)) return false;
          cp = (cp << 4) | nib;
        }
        if (cp == 0 || (cp >= 0xD800 && cp <= 0xDFFF)) return false;
        if (cp < 0x80) {
          appendByte(out, cap, &len, tooLong, (uint8_t)cp);
        } else if (cp < 0x800) {
          appendByte(out, cap, &len, tooLong, (uint8_t)(0xC0 | (cp >> 6)));
          appendByte(out, cap, &len, tooLong, (uint8_t)(0x80 | (cp & 0x3F)));
```

Return to guided explanation, p. 401

ORIGINAL SOURCE / CHAPTER 15 / 11 OF 18

src/domain/protocol_codec.c

Original lines 507-557 | Code and comments unchanged; long lines wrap visually

parseString (L466-518): Decode bounded string fields and escapes; reject unsupported surrogate escapes, NUL and malformed input.
isDigit (L520-520): A bounded JSON writer and command parser encode explicit fields and reject malformed or unsupported requests.

```
        } else {
          appendByte(out, cap, &len, tooLong, (uint8_t)(0xE0 | (cp >> 12)));
          appendByte(out, cap, &len, tooLong, (uint8_t)(0x80 | ((cp >> 6) & 0x3F)));
          appendByte(out, cap, &len, tooLong, (uint8_t)(0x80 | (cp & 0x3F)));
        }
        break;
      }
      default: return false;
    }
  }
  return false;  // unterminated
}

static bool isDigit(uint8_t c) { return c >= '0' && c <= '9'; }

static bool parseNumber(Parser* ps, Field* f) {
  const uint8_t* p = ps->p;
  bool negative = false;
  if (p < ps->end && *p == '-') {
    negative = true;
    ++p;
  }
  if (p >= ps->end || !isDigit(*p)) return false;
  int64_t value = 0;
  bool overflow = false;
  if (*p == '0') {
    ++p;
  } else {
    while (p < ps->end && isDigit(*p)) {
      // Past the supported range: stop accumulating (no signed overflow on a
      // long number), keep validating the digits.
      if (!overflow) {
        value = value * 10 + (*p - '0');
        if (value > 2147483648LL) overflow = true;
      }
      ++p;
    }
  }
  bool integer = true;
  if (p < ps->end && *p == '.') {
    integer = false;
    ++p;
    if (p >= ps->end || !isDigit(*p)) return false;
    while (p < ps->end && isDigit(*p)) ++p;
  }
  if (p < ps->end && (*p == 'e' || *p == 'E')) {
    integer = false;
    ++p;
    if (p < ps->end && (*p == '+' || *p == '-')) ++p;
    if (p >= ps->end || !isDigit(*p)) return false;
    while (p < ps->end && isDigit(*p)) ++p;
```

Return to guided explanation, p. 401

ORIGINAL SOURCE / CHAPTER 15 / 12 OF 18

src/domain/protocol_codec.c

Original lines 558-608 | Code and comments unchanged; long lines wrap visually

parseNumber (L522-565): Validate JSON number syntax and accumulate safely; only exact int32 syntax qualifies for command integer fields.
matchLiteral (L567-572): A bounded JSON writer and command parser encode explicit fields and reject malformed or unsupported requests.

```
  }
  ps->p = p;
  f->type = JT_NUMBER;
  if (negative) value = -value;
  f->isInteger = integer && !overflow && value >= INT32_MIN && value <= INT32_MAX;
  f->integer = f->isInteger ? (int32_t)value : 0;
  return true;
}

static bool matchLiteral(Parser* ps, const char* lit) {
  const size_t n = strlen(lit);
  if ((size_t)(ps->end - ps->p) < n || memcmp(ps->p, lit, n) != 0) return false;
  ps->p += n;
  return true;
}

static bool skipValue(Parser* ps, int depth);

static bool skipContainer(Parser* ps, int depth, char close, bool object) {
  if (depth > SKIP_DEPTH_MAX) return false;
  ++ps->p;  // opening bracket
  skipWs(ps);
  if (ps->p < ps->end && *ps->p == (uint8_t)close) {
    ++ps->p;
    return true;
  }
  for (;;) {
    if (object) {
      char key[FIELD_KEY_MAX + 1];
      size_t keyLen;
      bool tooLong;
      skipWs(ps);
      if (!parseString(ps, key, FIELD_KEY_MAX, &keyLen, &tooLong)) return false;
      if (!expect(ps, ':')) return false;
    }
    if (!skipValue(ps, depth + 1)) return false;
    skipWs(ps);
    if (ps->p >= ps->end) return false;
    if (*ps->p == ',') {
      ++ps->p;
      continue;
    }
    if (*ps->p == (uint8_t)close) {
      ++ps->p;
      return true;
    }
    return false;
  }
}

// Parses any scalar into f, or skips a container (type JT_OBJECT/JT_ARRAY).
```

Return to guided explanation, p. 401

ORIGINAL SOURCE / CHAPTER 15 / 13 OF 18

src/domain/protocol_codec.c

Original lines 609-659 | Code and comments unchanged; long lines wrap visually

parseScalarOrSkip (L609-640): A bounded JSON writer and command parser encode explicit fields and reject malformed or unsupported requests.
skipValue (L642-646): A bounded JSON writer and command parser encode explicit fields and reject malformed or unsupported requests.

```
609 static bool parseScalarOrSkip(Parser* ps, Field* f, int depth) {
610   skipWs(ps);
611   if (ps->p >= ps->end) return false;
612   const uint8_t c = *ps->p;
613   if (c == '"') {
614     f->type = JT_STRING;
615     return parseString(ps, f->str, FIELD_STR_MAX, &f->strLen, &f->strTooLong);
616   }
617   if (c == '{') {
618     f->type = JT_OBJECT;
619     return skipContainer(ps, depth, '}', true);
620   }
621   if (c == '[') {
622     f->type = JT_ARRAY;
623     return skipContainer(ps, depth, ']', false);
624   }
625   if (c == 't' && matchLiteral(ps, "true")) {
626     f->type = JT_BOOL;
627     f->boolean = true;
628     return true;
629   }
630   if (c == 'f' && matchLiteral(ps, "false")) {
631     f->type = JT_BOOL;
632     f->boolean = false;
633     return true;
634   }
635   if (c == 'n' && matchLiteral(ps, "null")) {
636     f->type = JT_NULL;
637     return true;
638   }
639   return parseNumber(ps, f);
640 }
641
642 static bool skipValue(Parser* ps, int depth) {
643   Field scratch;
644   memset(&scratch, 0, sizeof(scratch));
645   return parseScalarOrSkip(ps, &scratch, depth);
646 }
647
648 // Parses the flat "data" object; nested values are skipped (type kept).
649 static bool parseData(Parser* ps, Field* fields, uint8_t* count) {
650   *count = 0;
651   if (!expect(ps, '{')) return false;
652   skipWs(ps);
653   if (ps->p < ps->end && *ps->p == '}') {
654     ++ps->p;
655     return true;
656   }
657   for (;;) {
658     Field f;
659     memset(&f, 0, sizeof(f));
```

Return to guided explanation, p. 401

ORIGINAL SOURCE / CHAPTER 15 / 14 OF 18

src/domain/protocol_codec.c

Original lines 660-711 | Code and comments unchanged; long lines wrap visually

parseData (L649-679): Read the flat command-data fields into bounded storage while checking every value syntax.
findField (L681-686): A bounded JSON writer and command parser encode explicit fields and reject malformed or unsupported requests.

```
    size_t keyLen;
    bool keyTooLong;
    skipWs(ps);
    if (!parseString(ps, f.key, FIELD_KEY_MAX, &keyLen, &keyTooLong)) return false;
    if (!expect(ps, ':')) return false;
    if (!parseScalarOrSkip(ps, &f, 2)) return false;
    if (!keyTooLong && *count < DATA_FIELDS_MAX) fields[(*count)++] = f;
    skipWs(ps);
    if (ps->p >= ps->end) return false;
    if (*ps->p == ',') {
      ++ps->p;
      continue;
    }
    if (*ps->p == '}') {
      ++ps->p;
      return true;
    }
    return false;
  }
}

static const Field* findField(const Field* fields, uint8_t count, const char* key) {
  for (uint8_t i = 0; i < count; ++i) {
    if (strcmp(fields[i].key, key) == 0) return &fields[i];
  }
  return NULL;
}

typedef struct OpName {
  const char* name;
  CommandOp op;
} OpName;

static const OpName kOps[] = {
    {"get_sensors", CMD_OP_GET_SENSORS},
    {"set_sensor_enabled", CMD_OP_SET_SENSOR_ENABLED},
    {"reset_latch", CMD_OP_RESET_LATCH},
    {"set_wifi", CMD_OP_SET_WIFI},
    {"forget_wifi", CMD_OP_FORGET_WIFI},
    {"get_calibration", CMD_OP_GET_CALIBRATION},
    {"cal_capture", CMD_OP_CAL_CAPTURE},
    {"set_calibration", CMD_OP_SET_CALIBRATION},
    {"reset_calibration", CMD_OP_RESET_CALIBRATION},
};

const char* commandOpName(CommandOp op) {
  for (size_t i = 0; i < sizeof(kOps) / sizeof(kOps[0]); ++i) {
    if (kOps[i].op == op) return kOps[i].name;
  }
  return "?";
}

```

Return to guided explanation, p. 401

ORIGINAL SOURCE / CHAPTER 15 / 15 OF 18

src/domain/protocol_codec.c

Original lines 712-755 | Code and comments unchanged; long lines wrap visually

printableAscii (L712-717): Present already-collected diagnostic information; this does not make the measurement more valid.
protocolSensorCalibratable (L719-721): A bounded JSON writer and command parser encode explicit fields and reject malformed or unsupported requests.

```
712 static bool printableAscii(const char* s, size_t len) {
713   for (size_t i = 0; i < len; ++i) {
714     if ((uint8_t)s[i] < 0x20 || (uint8_t)s[i] > 0x7E) return false;
715   }
716   return true;
717 }
718
719 bool protocolSensorCalibratable(SensorId s) {
720   return s == SENSOR_PH || s == SENSOR_TDS || s == SENSOR_LIGHT;
721 }
722
723 static bool sensorField(const Field* data, uint8_t count, Command* out) {
724   const Field* s = findField(data, count, "sensor");
725   if (s == NULL || s->type != JT_STRING || s->strTooLong) return false;
726   out->sensor = sensorFromKey(s->str, s->strLen);
727   return out->sensor < SENSOR_COUNT;
728 }
729
730 static bool intField(const Field* data, uint8_t count, const char* key, int32_t min, int32_t
  > max, int32_t* out) {
731   const Field* f = findField(data, count, key);
732   if (f == NULL || f->type != JT_NUMBER || !f->isInteger || f->integer < min || f->integer >
  > max) return false;
733   *out = f->integer;
734   return true;
735 }
736
737 // Ranges in the scaled units of protocol_codec.h (plausibility only: the fit
738 // itself is checked by Configuration).
739 static ProtocolError validateCalibration(const Field* data, uint8_t count, Command* out) {
740   if (!sensorField(data, count, out) || !protocolSensorCalibratable(out->sensor)) return
  > PROTOCOL_ERR_BAD_FIELD;
741   bool ok;
742   switch (out->sensor) {
743     case SENSOR_PH:  // module 0-5 V, pH 0-14, two buffers
744       out->calTwoPoints = true;
745       ok = intField(data, count, "x1", 0, 50000, &out->calX1) && intField(data, count, "y1",
  > 0, 14000, &out->calY1) &&
746            intField(data, count, "x2", 0, 50000, &out->calX2) && intField(data, count, "y2",
  > 0, 14000, &out->calY2);
747       break;
748     case SENSOR_TDS:  // 0.1-2000 ppm both ways
749       ok = intField(data, count, "x1", 1, 20000, &out->calX1) && intField(data, count, "y1",
  > 1, 20000, &out->calY1);
750       break;
751     default:  // light: pin 0.1-3300 mV, 0.1-100000 lux
752       ok = intField(data, count, "x1", 1, 33000, &out->calX1) && intField(data, count, "y1",
  > 1, 1000000, &out->calY1);
753       break;
754   }
755   return ok ? PROTOCOL_OK : PROTOCOL_ERR_BAD_FIELD;
```

Return to guided explanation, p. 401

ORIGINAL SOURCE / CHAPTER 15 / 16 OF 18

src/domain/protocol_codec.c

Original lines 756-803 | Code and comments unchanged; long lines wrap visually

validateCalibration (L739-756): Validate sensor-specific scaled integer fields; Configuration later checks the resulting mathematical fit.
validateData (L758-810): Check required fields/types for the chosen operation; parser success still requires the owner to accept and apply it.

```
}

static ProtocolError validateData(const Field* data, uint8_t count, bool haveData, Command*
out) {
  switch (out->op) {
    case CMD_OP_GET_SENSORS:
    case CMD_OP_FORGET_WIFI:
    case CMD_OP_GET_CALIBRATION:
      return PROTOCOL_OK;  // data ignored
    default: break;
  }
  if (!haveData) return PROTOCOL_ERR_BAD_FIELD;

  switch (out->op) {
    case CMD_OP_CAL_CAPTURE:
    case CMD_OP_RESET_CALIBRATION:
      return sensorField(data, count, out) && protocolSensorCalibratable(out->sensor) ?
PROTOCOL_OK
                                                                                      :
PROTOCOL_ERR_BAD_FIELD;
    case CMD_OP_SET_CALIBRATION: return validateCalibration(data, count, out);
    case CMD_OP_SET_SENSOR_ENABLED: {
      const Field* s = findField(data, count, "sensor");
      const Field* e = findField(data, count, "enabled");
      if (s == NULL || s->type != JT_STRING || s->strTooLong) return PROTOCOL_ERR_BAD_FIELD;
      if (e == NULL || e->type != JT_BOOL) return PROTOCOL_ERR_BAD_FIELD;
      out->sensor = sensorFromKey(s->str, s->strLen);
      if (out->sensor >= SENSOR_COUNT) return PROTOCOL_ERR_BAD_FIELD;
      out->enabled = e->boolean;
      return PROTOCOL_OK;
    }
    case CMD_OP_RESET_LATCH: {
      const Field* r = findField(data, count, "rule");
      if (r == NULL || r->type != JT_NUMBER || !r->isInteger || r->integer < 0 ||
          r->integer > PROTOCOL_MAX_RULE_ID) {
        return PROTOCOL_ERR_BAD_FIELD;
      }
      out->rule = (uint8_t)r->integer;
      return PROTOCOL_OK;
    }
    case CMD_OP_SET_WIFI: {
      const Field* s = findField(data, count, "ssid");
      const Field* p = findField(data, count, "password");
      if (s == NULL || s->type != JT_STRING || s->strTooLong || s->strLen == 0 ||
          s->strLen > PROTOCOL_SSID_MAX) {
        return PROTOCOL_ERR_BAD_FIELD;
      }
      if (p == NULL || p->type != JT_STRING || p->strTooLong || p->strLen <
PROTOCOL_PASSWORD_MIN ||
          p->strLen > PROTOCOL_PASSWORD_MAX || !printableAscii(p->str, p->strLen)) {
        return PROTOCOL_ERR_BAD_FIELD;
      }
```

Return to guided explanation, p. 401

ORIGINAL SOURCE / CHAPTER 15 / 17 OF 18

src/domain/protocol_codec.c

Original lines 804-852 | Code and comments unchanged; long lines wrap visually

validateData (L758-810): Check required fields/types for the chosen operation; parser success still requires the owner to accept and apply it.
protocolParseCommand (L812-887): Parse one bounded object, validate version/ID/operation and dispatch strict field checks without executing the command.

```
804       memcpy(out->ssid, s->str, s->strLen + 1);
805       memcpy(out->password, p->str, p->strLen + 1);
806       return PROTOCOL_OK;
807     }
808     default: return PROTOCOL_ERR_UNKNOWN_OP;
809   }
810 }
811
812 ProtocolError protocolParseCommand(const uint8_t* payload, size_t len, Command* out) {
813   memset(out, 0, sizeof(*out));
814   out->sensor = SENSOR_COUNT;
815   if (payload == NULL || len == 0) return PROTOCOL_ERR_BAD_REQUEST;
816
817   Parser ps = {payload, payload + len};
818   Field data[DATA_FIELDS_MAX];
819   uint8_t dataCount = 0;
820   bool haveData = false, haveVersion = false, haveId = false, idValid = false;
821   int32_t version = -1;
822   char op[24] = {0};
823   size_t opLen = 0;
824   bool opSeen = false, opTooLong = false;
825
826   if (!expect(&ps, '{')) return PROTOCOL_ERR_BAD_REQUEST;
827   skipWs(&ps);
828   if (ps.p < ps.end && *ps.p == '}') {
829     ++ps.p;
830   } else {
831     for (;;) {
832       char key[FIELD_KEY_MAX + 1];
833       size_t keyLen;
834       bool keyTooLong;
835       skipWs(&ps);
836       if (!parseString(&ps, key, FIELD_KEY_MAX, &keyLen, &keyTooLong)) return
  > PROTOCOL_ERR_BAD_REQUEST;
837       if (!expect(&ps, ':')) return PROTOCOL_ERR_BAD_REQUEST;
838       skipWs(&ps);
839
840       if (!keyTooLong && strcmp(key, "data") == 0 && ps.p < ps.end && *ps.p == '{') {
841         if (!parseData(&ps, data, &dataCount)) return PROTOCOL_ERR_BAD_REQUEST;
842         haveData = true;
843       } else if (!keyTooLong && strcmp(key, "op") == 0 && ps.p < ps.end && *ps.p == '"') {
844         if (!parseString(&ps, op, sizeof(op) - 1, &opLen, &opTooLong)) return
  > PROTOCOL_ERR_BAD_REQUEST;
845         opSeen = true;
846       } else {
847         Field f;
848         memset(&f, 0, sizeof(f));
849         if (!parseScalarOrSkip(&ps, &f, 1)) return PROTOCOL_ERR_BAD_REQUEST;
850         if (!keyTooLong && strcmp(key, "v") == 0) {
851           haveVersion = f.type == JT_NUMBER && f.isInteger;
852           version = f.integer;
```

Return to guided explanation, p. 401

ORIGINAL SOURCE / CHAPTER 15 / 18 OF 18

src/domain/protocol_codec.c

Original lines 853-904 | Code and comments unchanged; long lines wrap visually

protocolParseCommand (L812-887): Parse one bounded object, validate version/ID/operation and dispatch strict field checks without executing the command.
protocolErrorName (L889-904): Translate an identity/state into a human-readable label for diagnostics.

```
        } else if (!keyTooLong && strcmp(key, "id") == 0) {
          haveId = true;
          idValid = f.type == JT_STRING && !f.strTooLong && f.strLen > 0 &&
                    f.strLen <= PROTOCOL_ID_MAX && printableAscii(f.str, f.strLen);
          if (idValid) memcpy(out->id, f.str, f.strLen + 1);
        }
      }
      skipWs(&ps);
      if (ps.p >= ps.end) return PROTOCOL_ERR_BAD_REQUEST;
      if (*ps.p == ',') {
        ++ps.p;
        continue;
      }
      if (*ps.p == '}') {
        ++ps.p;
        break;
      }
      return PROTOCOL_ERR_BAD_REQUEST;
    }
  }
  skipWs(&ps);
  if (ps.p != ps.end) return PROTOCOL_ERR_BAD_REQUEST;  // trailing bytes
  if (!haveVersion || version != PROTOCOL_VERSION || !haveId || !idValid) {
    return PROTOCOL_ERR_BAD_REQUEST;
  }
  if (!opSeen || opTooLong) return PROTOCOL_ERR_UNKNOWN_OP;
  for (size_t i = 0; i < sizeof(kOps) / sizeof(kOps[0]); ++i) {
    if (strlen(kOps[i].name) == opLen && memcmp(kOps[i].name, op, opLen) == 0) {
      out->op = kOps[i].op;
      break;
    }
  }
  if (out->op == CMD_OP_NONE) return PROTOCOL_ERR_UNKNOWN_OP;
  return validateData(data, dataCount, haveData, out);
}

const char* protocolErrorName(ProtocolError e) {
  switch (e) {
    case PROTOCOL_OK: return "ok";
    case PROTOCOL_ERR_BAD_REQUEST: return "bad_request";
    case PROTOCOL_ERR_UNKNOWN_OP: return "unknown_op";
    case PROTOCOL_ERR_BAD_FIELD: return "bad_field";
    case PROTOCOL_ERR_RULE_LATCHED: return "rule_latched";
    case PROTOCOL_ERR_RULE_ACTIVE: return "rule_active";
    case PROTOCOL_ERR_EXPIRED: return "expired";
    case PROTOCOL_ERR_TIMEOUT: return "timeout";
    case PROTOCOL_ERR_STORAGE: return "storage";
    case PROTOCOL_ERR_STALE: return "stale";
    case PROTOCOL_ERR_BAD_FIT: return "bad_fit";
  }
  return "?";
}
```

Return to guided explanation, p. 401

ORIGINAL SOURCE / CHAPTER 15 / 1 OF 1

src/domain/backoff.h

Original lines 1-20 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // Exponential retry backoff with jitter (§17: 1, 2, 4, 8 s ... up to 60 s).
2  // Pure C; the caller supplies the random number. WP8 (Wi-Fi reconnect).
3  #pragma once
4  #include <stdint.h>
5
6  typedef struct Backoff {
7    uint32_t initialMs;
8    uint32_t maxMs;
9    uint32_t currentMs;   // base delay of the next attempt
10   uint8_t jitterPct;    // +/- percentage applied to each delay
11 } Backoff;
12
13 void backoffInit(Backoff* b, uint32_t initialMs, uint32_t maxMs, uint8_t jitterPct);
14
15 // Back to initialMs (after a successful connection).
16 void backoffReset(Backoff* b);
17
18 // Delay before the next attempt: currentMs +/- jitter, never above maxMs and
19 // never 0. Then the base doubles, capped at maxMs.
20 uint32_t backoffNext(Backoff* b, uint32_t random);
```

Return to guided explanation, p. 386

ORIGINAL SOURCE / CHAPTER 15 / 1 OF 1

src/domain/backoff.c

Original lines 1-22 | Code and comments unchanged; long lines wrap visually

backoffInit (L3-8): Check dependencies, configure this module and return whether initialisation succeeded. **backoffReset (L10-10)**: Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.

```
#include "domain/backoff.h"

void backoffInit(Backoff* b, uint32_t initialMs, uint32_t maxMs, uint8_t jitterPct) {
  b->initialMs = initialMs == 0 ? 1 : initialMs;
  b->maxMs = maxMs < b->initialMs ? b->initialMs : maxMs;
  b->jitterPct = jitterPct > 100 ? 100 : jitterPct;
  b->currentMs = b->initialMs;
}

void backoffReset(Backoff* b) { b->currentMs = b->initialMs; }

uint32_t backoffNext(Backoff* b, uint32_t random) {
  const uint32_t base = b->currentMs;
  const uint32_t span = (uint32_t)(((uint64_t)base * b->jitterPct) / 100u);
  uint64_t delay = (uint64_t)base - span + (span > 0 ? random % (2u * (uint64_t)span + 1u) :
0u);
  if (delay > b->maxMs) delay = b->maxMs;
  if (delay == 0) delay = 1;

  const uint64_t doubled = (uint64_t)base * 2u;
  b->currentMs = doubled > b->maxMs ? b->maxMs : (uint32_t)doubled;
  return (uint32_t)delay;
}
```

Return to guided explanation, p. 386

ORIGINAL SOURCE / CHAPTER 15 / 1 OF 1

src/transports/protocol.h

Original lines 1-14 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // Protocol adapters: snapshot -> JSON state/telemetry frames.
2  // Work package WP8 (Stage 5). Architecture §17.
3  //
4  // The JSON itself is built in domain/protocol_codec (host-tested) and the
5  // frames in transports/frames; never from acquisition or ISRs. Coordinate
6  // schema changes with app/Protocol.py, app/Catalog.py and app/FirmwareContract.md.
7  #pragma once
8  #include <stddef.h>
9
10 #define PROTOCOL_FRAME_HEADER_BYTES ((size_t)2)  // little-endian payload length
11 #define PROTOCOL_MAX_PAYLOAD_BYTES ((size_t)4096)
12
13 // Reported as "fw" in every State frame.
14 #define PROTOCOL_FIRMWARE_VERSION "0.9.0"
```

Return to guided explanation, p. 386

ORIGINAL SOURCE / CHAPTER 15 / 1 OF 1

src/transports/protocol.c

Original lines 1-4 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  #include "transports/protocol.h"
2
3  // Nothing to implement here: the contract v2 encoders live in
4  // domain/protocol_codec.c and the shared frame builders in transports/frames.c.
```

Return to guided explanation, p. 386

ORIGINAL SOURCE / CHAPTER 15 / 1 OF 1

src/transports/frames.h

Original lines 1-35 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // State / Telemetry / event frames for the BLE link (WP8).
2  // FirmwareContract.md v2. The app keeps one row per (node, boot, metric, seq).
3  //
4  // Each calling task passes its own scratch (no shared buffers, no locks);
5  // every input is a zero-wait peek (snapshot, SafetyStatus, applied config,
6  // Wi-Fi status). Encoders write the JSON payload only (no length header).
7  #pragma once
8  #include <stdbool.h>
9  #include <stddef.h>
10 #include <stdint.h>
11
12 #include "domain/config_blob.h"
13 #include "domain/event_journal.h"
14 #include "domain/reading.h"
15 #include "services/configuration.h"
16
17 #define TRANSPORT_NODE_NAME "Hydro-ESP1"
18
19 typedef struct TransportScratch {
20   Snapshot snap;
21   ConfigBlob cfg;
22   char ssid[PROTOCOL_SSID_MAX + 1];
23   char ip[16];
24 } TransportScratch;
25
26 // Random per boot, drawn on first use (after the radio starts: true RNG).
27 uint32_t transportBootId(void);
28
29 // link: "ble" (the LAN link was removed after Stage 8.4).
30 size_t transportEncodeState(TransportScratch* s, const char* link, char* buf, size_t cap);
31
32 // 0 if no snapshot has been published yet.
33 size_t transportEncodeTelemetry(TransportScratch* s, char* buf, size_t cap);
34
35 size_t transportEncodeEvent(const JournalEvent* e, bool gap, char* buf, size_t cap);
```

Return to guided explanation, p. 386

ORIGINAL SOURCE / CHAPTER 15 / 1 OF 2

src/transports/frames.c

Original lines 1-42 | Code and comments unchanged; long lines wrap visually

transportBootId (L24-34): Fix one random boot identifier under a lock so frames from this boot can be grouped. **transportEncodeState (L36-64)**: Collect copied service status and format mode, configuration, Safety and Wi-Fi metadata for the link.

```
#include "transports/frames.h"

#include <ctype.h>
#include <esp_system.h>
#include <freertos/FreeRTOS.h>
#include <string.h>

#include "domain/protocol_codec.h"
#include "domain/sensor_set.h"
#include "hal/clock.h"
#include "policy_config.h"
#include "services/acquisition.h"
#include "services/power_coordinator.h"
#include "services/rail_state.h"
#include "services/safety.h"
#include "services/snapshot_store.h"
#include "transports/protocol.h"
#include "transports/wifi_mqtt.h"

static portMUX_TYPE s_bootMux = portMUX_INITIALIZER_UNLOCKED;
static uint32_t s_boot;
static bool s_bootSet;

uint32_t transportBootId(void) {
  const uint32_t candidate = esp_random();  // drawn outside the critical section
  portENTER_CRITICAL(&s_bootMux);
  if (!s_bootSet) {
    s_boot = candidate;
    s_bootSet = true;
  }
  const uint32_t boot = s_boot;
  portEXIT_CRITICAL(&s_bootMux);
  return boot;
}

size_t transportEncodeState(TransportScratch* s, const char* link, char* buf, size_t cap) {
  char mode[24];
  const char* name = powerModeName(powerModeCurrent());
  size_t i = 0;
  for (; name[i] != '\0' && i < sizeof(mode) - 1; ++i) mode[i] = (char)tolower((unsigned
char)name[i]);
  mode[i] = '\0';

```

Return to guided explanation, p. 386

ORIGINAL SOURCE / CHAPTER 15 / 2 OF 2

src/transports/frames.c

Original lines 43-80 | Code and comments unchanged; long lines wrap visually

transportEncodeState (L36-64): Collect copied service status and format mode, configuration, Safety and Wi-Fi metadata for the link.
transportEncodeTelemetry (L66-73): Peek a copied snapshot, apply live rail gating and encode fields with the current enabled mask.

```
  SafetyStatus st;
  memset(&st, 0, sizeof(st));
  const bool haveSafety = safetyStatusPeek(&st);
  configurationPeekApplied(&s->cfg);
  WifiStatus wifi;
  wifiMqttGetStatus(&wifi);
  memcpy(s->ssid, wifi.ssid, sizeof(s->ssid));
  memcpy(s->ip, wifi.ip, sizeof(s->ip));
  const bool connected = wifi.state == WIFI_STATE_CONNECTED;
  const StateView v = {
      .node = TRANSPORT_NODE_NAME, .fw = PROTOCOL_FIRMWARE_VERSION, .boot =
transportBootId(),
      .uptimeMs = monoNowMs(), .link = link, .mode = mode,
      .powerEpoch = railStateRead().epoch, .cfgRev = s->cfg.revision,  // live 5 V epoch
(WP9)
      .wifiState = wifiStateName(wifi.state), .wifiSsid = s->ssid[0] != '\0' ? s->ssid :
NULL,
      .wifiIp = connected ? s->ip : NULL, .wifiRssi = wifi.rssi,
      .relayOn = st.relayOn, .lampReason = haveSafety ? lampReasonName(st.lampReason) : "no
data",
      .interlock = st.interlock, .latchedMask = st.latchedMask, .activeMask = st.activeMask,
      .staleMask = st.staleMask, .ruleCount = POLICY_RULE_COUNT,
      .sgp30Baseline = acquisitionSgp30BaselineName(), .enabled = s->cfg.sensorEnabled,
  };
  return protocolEncodeState(&v, buf, cap);
}

size_t transportEncodeTelemetry(TransportScratch* s, char* buf, size_t cap) {
  if (!snapshotPeek(&s->snap)) return 0;
  const RailState rail = railStateRead();
  snapshotGateRail(&s->snap, rail.valid5v, rail.epoch);  // the rail may have failed since
acquisition
  configurationPeekApplied(&s->cfg);
  return protocolEncodeTelemetry(&s->snap, monoNowMs(), transportBootId(), s->cfg.revision,
s->cfg.sensorEnabled,
                                 buf, cap);
}

size_t transportEncodeEvent(const JournalEvent* e, bool gap, char* buf, size_t cap) {
  const MonoMs now = monoNowMs();
  const EventView v = {e->n, e->ruleId, severityName(e->severity), journalKindName(e->kind),
                       now > e->atMs ? (uint32_t)(now - e->atMs) : 0u, gap};
  return protocolEncodeEvent(&v, buf, cap);
}
```

Return to guided explanation, p. 386

ORIGINAL SOURCE / CHAPTER 15 / 1 OF 2

src/transports/ble_gatt.h

Original lines 1-40 | Code and comments unchanged; long lines wrap visually

The C interface hides NimBLE classes while defining secure-link, chunk, callback and session-generation contracts.

```
// GATT server shim over NimBLE-Arduino 2.5.1 (C++), callable from C.
// Work package WP8 (Stage 5). Architecture §17; FirmwareContract.md v2.
//
// The shim is the only code that touches NimBLE. It owns the service map,
// LE Secure Connections pairing with a displayed passkey (MITM), the
// single-owner rule and the link "session generation". It does no protocol
// work: received Command bytes go to one callback, and the caller notifies
// already-framed bytes one chunk at a time.
//
// Threading: the RX callback runs in the NimBLE host task (core 0). It must
// only copy the bytes and return (e.g. xStreamBufferSend with zero wait).
// All other functions may be called from one application task
// (BleTransport). Notifications are sent only on a secure link.
#pragma once
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>

#ifdef __cplusplus
extern "C" {
#endif

typedef uint8_t BleChannel;
enum { BLE_CHANNEL_STATE = 0, BLE_CHANNEL_TELEMETRY, BLE_CHANNEL_RESULT, BLE_CHANNEL_COUNT };

// Command characteristic write, raw bytes as received (any chunking).
typedef void (*BleRxFn)(const uint8_t* data, size_t len, uint32_t generation, void* ctx);

typedef struct BleGattStats {
  uint32_t connects;
  uint32_t disconnects;
  uint32_t pairingsAccepted;
  uint32_t pairingsRejected;   // not authenticated/bonded, or a second owner
  uint32_t notifyFailures;     // chunks NimBLE could not queue
  uint32_t rxDropped;          // writes received while the link was not secure
  int lastDisconnectReason;
} BleGattStats;

// Starts NimBLE, the GATT service and advertising. The six-digit passkey is
// drawn after NimBLE has started the radio (esp_random() is then a true RNG)
```

Return to guided explanation, p. 386

ORIGINAL SOURCE / CHAPTER 15 / 2 OF 2

src/transports/ble_gatt.h

Original lines 41-81 | Code and comments unchanged; long lines wrap visually

The C interface hides NimBLE classes while defining secure-link, chunk, callback and session-generation contracts.

```
41  // and before advertising; the caller prints bleGattPasskey() on serial.
42  // Returns false if NimBLE failed to start.
43  bool bleGattStart(const char* deviceName, BleRxFn rx, void* rxCtx);
44
45  uint32_t bleGattPasskey(void);
46
47  // True while one client is connected, encrypted, authenticated (MITM) and
48  // bonded, and it is the bonded owner.
49  bool bleGattSecure(void);
50
51  // Incremented on every connect and disconnect: requests from an earlier
52  // session must not be answered on a newer one.
53  uint32_t bleGattGeneration(void);
54
55  // Largest notification payload on the current link (ATT MTU - 3), >= 20.
56  size_t bleGattChunkSize(void);
57
58  // Sends one notification of at most bleGattChunkSize() bytes. false if the
59  // link is not secure, the chunk is too big, or NimBLE is out of buffers
60  // (the caller may retry shortly or drop the frame).
61  bool bleGattNotify(BleChannel channel, const uint8_t* data, size_t len);
62
63  // Value returned by a read of the State characteristic (one whole frame,
64  // header included; long reads are supported up to BLE_GATT_STATE_MAX).
65  #define BLE_GATT_STATE_MAX 1024
66  bool bleGattSetStateValue(const uint8_t* frame, size_t len);
67
68  // Number of stored bonds, and the recovery path (BOOT held 10 s at runtime):
69  // removes every bond and drops the current client.
70  int bleGattBondCount(void);
71  void bleGattClearBonds(void);
72
73  // Drops the current client (bonds are kept). Used when a notification frame
74  // was cut after its first chunk: the client's stream is out of step.
75  void bleGattDisconnect(void);
76
77  void bleGattGetStats(BleGattStats* out);
78
79  #ifdef __cplusplus
80  }
81  #endif
```

Return to guided explanation, p. 386

ORIGINAL SOURCE / CHAPTER 15 / 1 OF 5

src/transports/ble_gatt.cpp

Original lines 1-45 | Code and comments unchanged; long lines wrap visually

stats (L45-49): C++ adapts NimBLE callbacks to a C interface; atomics protect link facts and authenticated checks gate access.

```
1  // NimBLE-Arduino implementation of ble_gatt.h. The only C++ file in the
2  // project: NimBLE-Arduino is a C++ library, and ESP-Demo already proved this
3  // version on espressif32@6.12.0 / ESP32-S3. Everything it exports is C.
4  //
5  // Compiled only with -D FW_ENABLE_BLE (the production envs), so the bring-up
6  // images do not link NimBLE.
7  #ifdef FW_ENABLE_BLE
8  #include "transports/ble_gatt.h"
9
10 #include <NimBLEDevice.h>
11
12 #include <esp_system.h>
13
14 #include <atomic>
15 #include <string.h>
16
17 namespace {
18
19 constexpr const char* kServiceUuid = "98a90f00-346e-4e65-9cbd-7687685bdc01";
20 constexpr const char* kStateUuid = "98a90f01-346e-4e65-9cbd-7687685bdc01";
21 constexpr const char* kTelemetryUuid = "98a90f02-346e-4e65-9cbd-7687685bdc01";
22 constexpr const char* kCommandUuid = "98a90f03-346e-4e65-9cbd-7687685bdc01";
23 constexpr const char* kResultUuid = "98a90f04-346e-4e65-9cbd-7687685bdc01";
24 constexpr const char* kAuthUuid = "98a90f05-346e-4e65-9cbd-7687685bdc01";
25 constexpr const char* kAuthValue = "HYDRO_SECURE_V1";
26 constexpr uint16_t kPreferredMtu = 247;
27 constexpr size_t kMinChunk = 20;
28
29 NimBLEServer* s_server = nullptr;
30 NimBLECharacteristic* s_channel[BLE_CHANNEL_COUNT] = {};
31 NimBLECharacteristic* s_command = nullptr;
32 BleRxFn s_rx = nullptr;
33 void* s_rxCtx = nullptr;
34 uint32_t s_passkey = 0;
35
36 std::atomic<bool> s_secure{false};
37 std::atomic<uint32_t> s_generation{0};
38 std::atomic<uint16_t> s_conn{BLE_HS_CONN_HANDLE_NONE};
39 std::atomic<uint16_t> s_mtu{23};
40
41 portMUX_TYPE s_statsMux = portMUX_INITIALIZER_UNLOCKED;
42 BleGattStats s_stats = {};
43
44 template <typename F>
45 void stats(F update) {
```

Return to guided explanation, p. 386

ORIGINAL SOURCE / CHAPTER 15 / 2 OF 5

src/transports/ble_gatt.cpp

Original lines 46-88 | Code and comments unchanged; long lines wrap visually

stats (L45-49): C++ adapts NimBLE callbacks to a C interface; atomics protect link facts and authenticated checks gate access.
linkIsSecure (L51-53): C++ adapts NimBLE callbacks to a C interface; atomics protect link facts and authenticated checks gate access.

```
  portENTER_CRITICAL(&s_statsMux);
  update(s_stats);
  portEXIT_CRITICAL(&s_statsMux);
}

bool linkIsSecure(const NimBLEConnInfo& info) {
  return info.isEncrypted() && info.isAuthenticated() && info.isBonded() &&
> info.getSecKeySize() >= 16;
}

class ServerCallbacks final : public NimBLEServerCallbacks {
  void onConnect(NimBLEServer* server, NimBLEConnInfo& info) override {
    if (server->getConnectedCount() > 1) {  // one client at a time
      server->disconnect(info.getConnHandle());
      return;
    }
    s_secure.store(false);
    s_conn.store(info.getConnHandle());
    s_mtu.store(23);
    s_generation.fetch_add(1);
    stats([](BleGattStats& s) { ++s.connects; });
    // Ask for security at once: Windows and BlueZ then pair before discovery.
    NimBLEDevice::startSecurity(info.getConnHandle());
  }

  void onDisconnect(NimBLEServer*, NimBLEConnInfo& info, int reason) override {
    if (info.getConnHandle() == s_conn.load()) {
      s_secure.store(false);
      s_conn.store(BLE_HS_CONN_HANDLE_NONE);
      s_generation.fetch_add(1);
    }
    stats([reason](BleGattStats& s) {
      ++s.disconnects;
      s.lastDisconnectReason = reason;
    });
    NimBLEDevice::startAdvertising();
  }

  void onMTUChange(uint16_t mtu, NimBLEConnInfo&) override { s_mtu.store(mtu); }

  uint32_t onPassKeyDisplay() override { return s_passkey; }

  void onAuthenticationComplete(NimBLEConnInfo& info) override {
    bool secure = linkIsSecure(info);
```

Return to guided explanation, p. 386

ORIGINAL SOURCE / CHAPTER 15 / 3 OF 5

src/transports/ble_gatt.cpp

Original lines 89-132 | Code and comments unchanged; long lines wrap visually

C++ adapts NimBLE callbacks to a C interface; atomics protect link facts and authenticated checks gate access.

```
    // Single owner: the first bond is the owner. A second bond means another
    // computer just paired: remove it and drop that client.
    if (secure && NimBLEDevice::getNumBonds() > 1) {
      NimBLEDevice::deleteBond(info.getIdAddress());
      secure = false;
    }
    s_secure.store(secure);
    if (secure) {
      s_mtu.store(info.getMTU());
      stats([](BleGattStats& s) { ++s.pairingsAccepted; });
    } else {
      stats([](BleGattStats& s) { ++s.pairingsRejected; });
      s_server->disconnect(info.getConnHandle());
    }
  }
};

class CommandCallbacks final : public NimBLECharacteristicCallbacks {
  void onWrite(NimBLECharacteristic* characteristic, NimBLEConnInfo& info) override {
    if (!s_secure.load() || !linkIsSecure(info)) {
      stats([](BleGattStats& s) { ++s.rxDropped; });
      s_server->disconnect(info.getConnHandle());
      return;
    }
    const NimBLEAttValue value = characteristic->getValue();
    if (s_rx != nullptr && value.length() > 0) s_rx(value.data(), value.length(),
s_generation.load(), s_rxCtx);
  }
};

ServerCallbacks s_serverCallbacks;
CommandCallbacks s_commandCallbacks;

}  // namespace

extern "C" bool bleGattStart(const char* deviceName, BleRxFn rx, void* rxCtx) {
  if (s_server != nullptr) return true;
  s_rx = rx;
  s_rxCtx = rxCtx;
  if (!NimBLEDevice::init(deviceName)) return false;
  // The controller is running now, so the hardware RNG is seeded from RF noise.
  s_passkey = 100000u + esp_random() % 900000u;
  NimBLEDevice::setSecurityAuth(true, true, true);  // bonding, MITM, LE Secure Connections
  NimBLEDevice::setSecurityIOCap(BLE_HS_IO_DISPLAY_ONLY);
  NimBLEDevice::setSecurityPasskey(s_passkey);
```

Return to guided explanation, p. 386

ORIGINAL SOURCE / CHAPTER 15 / 4 OF 5

src/transports/ble_gatt.cpp

Original lines 133-171 | Code and comments unchanged; long lines wrap visually

C++ adapts NimBLE callbacks to a C interface; atomics protect link facts and authenticated checks gate access.

```
133   NimBLEDevice::setMTU(kPreferredMtu);
134
135   s_server = NimBLEDevice::createServer();
136   s_server->setCallbacks(&s_serverCallbacks);
137   NimBLEService* service = s_server->createService(kServiceUuid);
138   // *_AUTHEN: the stack itself refuses access without an authenticated (MITM) link.
139   s_channel[BLE_CHANNEL_STATE] = service->createCharacteristic(
140       kStateUuid, NIMBLE_PROPERTY::READ | NIMBLE_PROPERTY::READ_AUTHEN |
 >  NIMBLE_PROPERTY::NOTIFY,
141       BLE_GATT_STATE_MAX);
142   s_channel[BLE_CHANNEL_TELEMETRY] = service->createCharacteristic(
143       kTelemetryUuid, NIMBLE_PROPERTY::NOTIFY | NIMBLE_PROPERTY::READ |
 >  NIMBLE_PROPERTY::READ_AUTHEN, 20);
144   s_command = service->createCharacteristic(kCommandUuid,
145                                             NIMBLE_PROPERTY::WRITE |
 >  NIMBLE_PROPERTY::WRITE_AUTHEN, 512);
146   s_command->setCallbacks(&s_commandCallbacks);
147   s_channel[BLE_CHANNEL_RESULT] = service->createCharacteristic(
148       kResultUuid, NIMBLE_PROPERTY::NOTIFY | NIMBLE_PROPERTY::READ |
 >  NIMBLE_PROPERTY::READ_AUTHEN, 20);
149   NimBLECharacteristic* auth =
150       service->createCharacteristic(kAuthUuid, NIMBLE_PROPERTY::READ |
 >  NIMBLE_PROPERTY::READ_AUTHEN, 32);
151   auth->setValue(kAuthValue);
152   if (!s_server->start()) return false;
153
154   // 31-byte advertising packet: flags + 128-bit service UUID (21 B) leave no room
155   // for the name, so the name goes in the scan response. Order matters:
156   // setName() only targets the scan response once it is enabled.
157   NimBLEAdvertising* advertising = NimBLEDevice::getAdvertising();
158   advertising->enableScanResponse(true);
159   advertising->setName(deviceName);
160   advertising->addServiceUUID(kServiceUuid);
161   return advertising->start();
162 }
163
164 extern "C" uint32_t bleGattPasskey(void) { return s_passkey; }
165
166 extern "C" bool bleGattSecure(void) { return s_secure.load(); }
167
168 extern "C" uint32_t bleGattGeneration(void) { return s_generation.load(); }
169
170 extern "C" size_t bleGattChunkSize(void) {
171   const size_t mtu = s_mtu.load();
```

Return to guided explanation, p. 386

ORIGINAL SOURCE / CHAPTER 15 / 5 OF 5

src/transports/ble_gatt.cpp

Original lines 172-215 | Code and comments unchanged; long lines wrap visually

C++ adapts NimBLE callbacks to a C interface; atomics protect link facts and authenticated checks gate access.

```
172   return mtu > kMinChunk + 3 ? mtu - 3 : kMinChunk;
173 }
174
175 extern "C" bool bleGattNotify(BleChannel channel, const uint8_t* data, size_t len) {
176   const uint16_t conn = s_conn.load();
177   if (channel >= BLE_CHANNEL_COUNT || !s_secure.load() || conn == BLE_HS_CONN_HANDLE_NONE ||
178       len == 0 || len > bleGattChunkSize()) {
179     return false;
180   }
181   if (!s_channel[channel]->notify(data, len, conn)) {
182     stats([](BleGattStats& s) { ++s.notifyFailures; });
183     return false;
184   }
185   return true;
186 }
187
188 extern "C" bool bleGattSetStateValue(const uint8_t* frame, size_t len) {
189   if (s_channel[BLE_CHANNEL_STATE] == nullptr || len > BLE_GATT_STATE_MAX) return false;
190   s_channel[BLE_CHANNEL_STATE]->setValue(frame, len);
191   return true;
192 }
193
194 extern "C" int bleGattBondCount(void) { return s_server != nullptr ?
  > NimBLEDevice::getNumBonds() : 0; }
195
196 extern "C" void bleGattClearBonds(void) {
197   if (s_server == nullptr) return;
198   const uint16_t conn = s_conn.load();
199   if (conn != BLE_HS_CONN_HANDLE_NONE) s_server->disconnect(conn);
200   NimBLEDevice::deleteAllBonds();
201 }
202
203 extern "C" void bleGattDisconnect(void) {
204   if (s_server == nullptr) return;
205   const uint16_t conn = s_conn.load();
206   if (conn != BLE_HS_CONN_HANDLE_NONE) s_server->disconnect(conn);
207 }
208
209 extern "C" void bleGattGetStats(BleGattStats* out) {
210   portENTER_CRITICAL(&s_statsMux);
211   *out = s_stats;
212   portEXIT_CRITICAL(&s_statsMux);
213 }
214
215 #endif  // FW_ENABLE_BLE
```

Return to guided explanation, p. 386

ORIGINAL SOURCE / CHAPTER 15 / 1 OF 1

src/transports/ble_transport.h

Original lines 1-43 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
// BleTransport: the one application GATT sender (Architecture §17).
// Work package WP8 (Stage 8.3). FirmwareContract.md v2.
//
// Task "BleTransport" (cfgTaskSpec: core 0, priority 2, 6 KB). It owns the
// GATT shim (ble_gatt.h), reassembles command frames, routes them (get_sensors
// here, and get_calibration / cal_capture from zero-wait peeks;
// set_sensor_enabled / set_wifi / forget_wifi / set_calibration /
// reset_calibration to Configuration; reset_latch to Safety) and sends:
//   results and alarm events first, then State (every 2 s and on link-up),
//   then Telemetry (every 3 s). A telemetry frame that cannot be queued is
//   dropped and superseded by the next one; results are retried.
// It peeks the snapshot, SafetyStatus and the applied config with zero wait
// and never calls a driver. Requests carry the link's session generation and
// an expiry: a reply is never sent on a newer link, a command never applied
// late.
//
// Pairing: LE Secure Connections + bonding + MITM, passkey drawn after the
// radio starts and printed once on serial; single bonded owner. Holding BOOT
// (GPIO0) for 10 s while running clears all bonds.
//
// Built only with -D FW_ENABLE_BLE (production envs). Without it
// bleTransportInit() returns false and nothing links NimBLE.
#pragma once
#include <stdbool.h>
#include <stdint.h>

bool bleTransportInit(void);

typedef struct BleTransportStats {
  uint32_t framesSent;
  uint32_t telemetryDropped;   // superseded: link congested
  uint32_t resultsDropped;     // link closed or buffers stayed full
  uint32_t commands;
  uint32_t commandErrors;      // malformed frames / fields
  uint32_t busy;               // replies "busy" (queues full)
  uint32_t rxOverflow;         // command bytes lost before parsing
  uint32_t maxSendMs;          // longest frame send (all chunks)
} BleTransportStats;

void bleTransportGetStats(BleTransportStats* out);

// For Supervisor. NULL before init or without FW_ENABLE_BLE.
void* bleTransportTaskHandle(void);
```

Return to guided explanation, p. 386

GUIDED SOURCE READING / CHAPTER 15

A callback copies bytes; a task interprets them

The NimBLE host calls onRx in its own task. Keeping that callback small prevents incoming bytes from turning a radio-library execution path into a flash writer or sensor owner.

Original excerpt: src/transports/ble_transport.c (p. 436) / lines 112-117. The complete unchanged file follows.

```
112  static void onRx(const uint8_t* data, size_t len, uint32_t generation, void* ctx) {
113    (void)generation;
114    (void)ctx;
115    if (xStreamBufferSend(s_rx, data, len, 0) != len) STATS_ADD(rxOverflow, 1);
116    if (s_task != NULL) xTaskNotifyGive(s_task);
117  }
```

The callback explicitly ignores its unused parameters, then attempts a zero-wait stream-buffer copy. If not every byte fits, it increments rxOverflow. That counter describes finite buffering; it does not claim that all received commands were preserved.

Only after the copy does it notify BleTransport. The awakened task can therefore inspect bytes already stored. Parsing before copying would keep the NimBLE callback occupied with variable command work and make its timing depend on JSON complexity.

drainRx later feeds the frame assembler, copies a complete payload and calls handleCommand. Read-only requests can use copied state. Mutations claim a pending slot and enter Configuration or Safety's queue with expiry, session and reply tag.

The receiving service does the operation and returns a short result through a callback. That callback queues the outcome; BleTransport matches its tag and sends it. Each handoff transfers a record or a wake signal while leaving sensor, flash and relay ownership with their designated tasks.

ORIGINAL SOURCE / CHAPTER 15 / 1 OF 12

src/transports/ble_transport.c

Original lines 1-48 | Code and comments unchanged; long lines wrap visually

bleTransportInit (L5-5): Create finite buffers/queues, start the GATT adapter, populate State and create the transport task.
bleTransportGetStats (L6-6): Framing, pending request tags and session checks belong to this task; callbacks only copy and notify.

```
#include "transports/ble_transport.h"

#ifndef FW_ENABLE_BLE

bool bleTransportInit(void) { return false; }
void bleTransportGetStats(BleTransportStats* out) { *out = (BleTransportStats){0}; }
void* bleTransportTaskHandle(void) { return 0; }

#else

#include <Arduino.h>
#include <ctype.h>
#include <esp_system.h>
#include <esp_task_wdt.h>
#include <freertos/FreeRTOS.h>
#include <freertos/queue.h>
#include <freertos/stream_buffer.h>
#include <freertos/task.h>
#include <stdio.h>
#include <string.h>

#include "app_config.h"
#include "board_profile.h"
#include "diagnostics/logger.h"
#include "diagnostics/progress.h"
#include "domain/cal_capture.h"
#include "domain/frame_codec.h"
#include "domain/protocol_codec.h"
#include "domain/sensor_set.h"
#include "hal/clock.h"
#include "policy_config.h"
#include "services/acquisition.h"
#include "services/configuration.h"
#include "services/rail_state.h"
#include "services/safety.h"
#include "services/snapshot_store.h"
#include "transports/ble_gatt.h"
#include "transports/frames.h"
#include "transports/protocol.h"

#define BLE_NODE_NAME TRANSPORT_NODE_NAME
#define BLE_BOOT_PIN 0
#define BLE_BOND_CLEAR_MS 10000u
#define BLE_WAKE_MS 50u
#define BLE_RX_BYTES 2048
#define BLE_NOTIFY_RETRIES 40     // x 5 ms: NimBLE buffers free up as packets leave
#define BLE_COMMAND_TTL_MS 5000u  // not handled within this: refused, never applied late
#define BLE_PENDING 8             // = CFG_COMMAND_QUEUE_LEN: one slot per queued command
```

Return to guided explanation, p. 435

ORIGINAL SOURCE / CHAPTER 15 / 2 OF 12

src/transports/ble_transport.c

Original lines 49-95 | Code and comments unchanged; long lines wrap visually

Framing, pending request tags and session checks belong to this task; callbacks only copy and notify.

```
#define BLE_PENDING_LIMIT_MS (BLE_COMMAND_TTL_MS + 5000u)  // a reply that never came frees
> its slot
#define BLE_STATS_LOG_MS 60000u

// ---- RX: filled by the NimBLE host callback, drained by this task ----------------
static StaticStreamBuffer_t s_rxStruct;
static uint8_t s_rxStorage[BLE_RX_BYTES + 1];
static StreamBufferHandle_t s_rx;

// ---- Task and buffers (static: never on the 6 KB stack) --------------------------
static StaticTask_t s_tcb;
static StackType_t s_stack[6 * 1024];  // cfgTaskSpec(TASK_ID_BLE_TRANSPORT)->stackBytes
static TaskHandle_t s_task = NULL;

static FrameAssembler s_assembler;
static uint8_t s_command[FRAME_MAX_PAYLOAD];
static uint8_t s_frame[FRAME_HEADER_BYTES + FRAME_MAX_PAYLOAD];
static char s_data[1536];
static TransportScratch s_scratch;
static ConfigBlob s_cfg;
static ConfigRequest s_req;  // static: a Wi-Fi password never sits on the stack

static portMUX_TYPE s_statsMux = portMUX_INITIALIZER_UNLOCKED;
static BleTransportStats s_stats;
#define STATS_ADD(field, n)          \
  do {                               \
    portENTER_CRITICAL(&s_statsMux); \
    s_stats.field += (n);            \
    portEXIT_CRITICAL(&s_statsMux);  \
  } while (0)

// ---- Commands in flight ----------------------------------------------------------
// Configuration and Safety answer from their own tasks; their callbacks only
// copy the outcome into s_results. The tag is slot | sequence << 8, so a late
// reply can never be matched to a slot that was freed and reused.
typedef struct Pending {
  bool used;
  CommandOp op;
  uint32_t generation;
  uint32_t seq;
  MonoMs sinceMs;
  char id[PROTOCOL_ID_MAX + 1];
} Pending;

typedef struct BleResult {
  uint32_t tag;
  ProtocolError error;
  uint16_t cfgRev;
```

Return to guided explanation, p. 435

ORIGINAL SOURCE / CHAPTER 15 / 3 OF 12

src/transports/ble_transport.c

Original lines 96-143 | Code and comments unchanged; long lines wrap visually

onRx (L112-117): NimBLE callback: copy bytes to the finite stream buffer without waiting, count overflow and notify the owner task.
postResult (L119-122): Framing, pending request tags and session checks belong to this task; callbacks only copy and notify.

```
96  } BleResult;
97
98  static Pending s_pending[BLE_PENDING];
99  static uint32_t s_seq;
100 static StaticQueue_t s_resultControl;
101 static uint8_t s_resultStorage[BLE_PENDING * sizeof(BleResult)];
102 static QueueHandle_t s_results;
103 static JournalCursor s_journalCursor;
104 static bool s_eventPending;    // an event failed to send and waits at the cursor
105 static bool s_eventLost;       // ... and its link closed: the next event reports gap:true
106 static uint32_t s_tornGeneration = UINT32_MAX;  // link whose stream was cut mid-frame
107 #ifdef FW_INJECT_BLE_TEAR
108 static uint32_t s_tearCount;  // env:ble_tear: telemetry frames sent
109 #endif
110
111 // NimBLE host task: copy and return (ble_gatt.h threading rule).
112 static void onRx(const uint8_t* data, size_t len, uint32_t generation, void* ctx) {
113   (void)generation;
114   (void)ctx;
115   if (xStreamBufferSend(s_rx, data, len, 0) != len) STATS_ADD(rxOverflow, 1);
116   if (s_task != NULL) xTaskNotifyGive(s_task);
117 }
118
119 static void postResult(const BleResult* r) {
120   xQueueSend(s_results, r, 0);  // never full: one entry per pending slot
121   if (s_task != NULL) xTaskNotifyGive(s_task);
122 }
123
124 static void onConfigResult(const ConfigResult* r, void* ctx) {
125   (void)ctx;
126   BleResult out;
127   memset(&out, 0, sizeof(out));
128   out.tag = r->tag;
129   out.error = r->error;
130   out.cfgRev = r->cfgRev;
131   postResult(&out);
132 }
133
134 static void onSafetyResult(uint32_t tag, SafetyCommandOutcome outcome, void* ctx) {
135   (void)ctx;
136   BleResult out;
137   memset(&out, 0, sizeof(out));
138   out.tag = tag;
139   out.error = outcome == SAFETY_CMD_APPLIED         ? PROTOCOL_OK
140               : outcome == SAFETY_CMD_EXPIRED       ? PROTOCOL_ERR_EXPIRED
141               : outcome == SAFETY_CMD_STILL_ACTIVE  ? PROTOCOL_ERR_RULE_ACTIVE
142                                                     : PROTOCOL_ERR_BAD_FIELD;
143   postResult(&out);
```

Return to guided explanation, p. 435

ORIGINAL SOURCE / CHAPTER 15 / 4 OF 12

src/transports/ble_transport.c

Original lines 144-188 | Code and comments unchanged; long lines wrap visually

onSafetyResult (L134-144): Framing, pending request tags and session checks belong to this task; callbacks only copy and notify.
sessionNow (L146-146): Framing, pending request tags and session checks belong to this task; callbacks only copy and notify.

```
144 }
145
146 static uint32_t sessionNow(void) { return bleGattGeneration(); }
147
148 // ---- Sending ----------------------------------------------------------------------
149 // Header + payload already in s_frame. Sent in MTU-sized chunks. A frame not
150 // started yet may be dropped (telemetry does not wait: the next frame
151 // replaces it). Once its first chunk is out, the frame must finish: the
152 // client reads each characteristic as one length-prefixed stream, so a cut
153 // frame would turn the next one into its continuation. If NimBLE buffers stay
154 // full mid-frame, the link is dropped instead (the app clears its decoders on
155 // reconnect) and nothing more is sent on it.
156 static bool sendFrame(BleChannel channel, size_t payloadLen, uint32_t generation) {
157   if (payloadLen == 0 || !frameWriteHeader(s_frame, payloadLen)) return false;
158   if (generation == s_tornGeneration) return false;  // being dropped
159   const MonoMs start = monoNowMs();
160   const size_t total = FRAME_HEADER_BYTES + payloadLen;
161   bool tear = false;  // env:ble_tear: buffers "stay full" after the first chunk
162 #ifdef FW_INJECT_BLE_TEAR
163   if (channel == BLE_CHANNEL_TELEMETRY && ++s_tearCount % (FW_INJECT_BLE_TEAR) == 0) tear =
  > true;
164 #endif
165   for (size_t offset = 0; offset < total;) {
166     if (!bleGattSecure() || bleGattGeneration() != generation) return false;  // link gone:
  > nothing to resync
167     size_t chunk = bleGattChunkSize();
168     if (chunk > total - offset) chunk = total - offset;
169     int tries = 0;
170     while ((tear && offset > 0) || !bleGattNotify(channel, &s_frame[offset], chunk)) {
171       if (offset == 0 && channel == BLE_CHANNEL_TELEMETRY) return false;
172       if (!bleGattSecure() || bleGattGeneration() != generation) return false;
173       if (++tries >= BLE_NOTIFY_RETRIES) {
174         if (offset > 0) {
175           s_tornGeneration = generation;
176           bleGattDisconnect();
177           logPrintf(LOG_LEVEL_WARN, "ble: frame cut after %u of %u bytes - link dropped to
  > resync",
178                     (unsigned)offset, (unsigned)total);
179         }
180         return false;
181       }
182       vTaskDelay(pdMS_TO_TICKS(5));
183     }
184     offset += chunk;
185   }
186   const uint32_t took = (uint32_t)(monoNowMs() - start);
187   portENTER_CRITICAL(&s_statsMux);
188   ++s_stats.framesSent;
```

Return to guided explanation, p. 435

ORIGINAL SOURCE / CHAPTER 15 / 5 OF 12

src/transports/ble_transport.c

Original lines 189-235 | Code and comments unchanged; long lines wrap visually

sendFrame (L156-192): Chunk a length-prefixed message to negotiated MTU, retry congestion per chunk and drop the link after a partial-frame tear.
payload (L194-194): Framing, pending request tags and session checks belong to this task; callbacks only copy and notify.

```
189   if (took > s_stats.maxSendMs) s_stats.maxSendMs = took;
190   portEXIT_CRITICAL(&s_statsMux);
191   return true;
192 }
193
194 static char* payload(void) { return (char*)&s_frame[FRAME_HEADER_BYTES]; }
195
196 static void sendResultFrame(size_t len, uint32_t generation) {
197   if (!sendFrame(BLE_CHANNEL_RESULT, len, generation)) STATS_ADD(resultsDropped, 1);
198 }
199
200 static void sendError(const char* id, ProtocolError error, uint32_t generation) {
201   sendResultFrame(protocolEncodeResultError(id, protocolErrorName(error), payload(),
  > FRAME_MAX_PAYLOAD), generation);
202 }
203
204 static void sendBusy(const char* id, uint32_t generation) {
205   STATS_ADD(busy, 1);
206   sendResultFrame(protocolEncodeResultBusy(id, payload(), FRAME_MAX_PAYLOAD), generation);
207 }
208
209 // ---- State and telemetry -----------------------------------------------------
210 static void publishState(bool notify, uint32_t generation) {
211   const size_t len = transportEncodeState(&s_scratch, "ble", payload(), FRAME_MAX_PAYLOAD);
212   if (len == 0 || !frameWriteHeader(s_frame, len)) return;
213   bleGattSetStateValue(s_frame, FRAME_HEADER_BYTES + len);  // value for a plain read
214   if (notify) sendFrame(BLE_CHANNEL_STATE, len, generation);
215 }
216
217 static void publishTelemetry(uint32_t generation) {
218   const size_t len = transportEncodeTelemetry(&s_scratch, payload(), FRAME_MAX_PAYLOAD);
219   if (len == 0) return;
220   if (!sendFrame(BLE_CHANNEL_TELEMETRY, len, generation)) STATS_ADD(telemetryDropped, 1);
221 }
222
223 // ---- Commands ----------------------------------------------------------------
224 static int claimSlot(const Command* cmd, uint32_t generation) {
225   for (int i = 0; i < BLE_PENDING; ++i) {
226     if (s_pending[i].used) continue;
227     Pending* p = &s_pending[i];
228     p->used = true;
229     p->op = cmd->op;
230     p->generation = generation;
231     p->seq = ++s_seq & 0x00FFFFFFu;
232     p->sinceMs = monoNowMs();
233     memcpy(p->id, cmd->id, sizeof(p->id));
234     return i;
235   }
```

Return to guided explanation, p. 435

ORIGINAL SOURCE / CHAPTER 15 / 6 OF 12

src/transports/ble_transport.c

Original lines 236-278 | Code and comments unchanged; long lines wrap visually

claimSlot (L224-237): Reserve a finite pending slot and retain request ID, operation, generation and reuse sequence.
tagOf (L239-239): Framing, pending request tags and session checks belong to this task; callbacks only copy and notify.

```
236   return -1;
237 }
238
239 static uint32_t tagOf(int slot) { return (uint32_t)slot | (s_pending[slot].seq << 8); }
240
241 // Answered here, like get_sensors: a capture only peeks the snapshot (the raw
242 // level every calibratable driver publishes), it never touches a driver.
243 static void handleCapture(const Command* cmd, uint32_t generation) {
244   configurationPeekApplied(&s_cfg);
245   if (!sensorMaskHas(s_cfg.sensorEnabled, cmd->sensor) || !snapshotPeek(&s_scratch.snap)) {
246     sendError(cmd->id, PROTOCOL_ERR_STALE, generation);
247     return;
248   }
249   const RailState rail = railStateRead();
250   snapshotGateRail(&s_scratch.snap, rail.valid5v, rail.epoch);  // never capture from a
 >  failed rail
251   const CaptureLimits limits = {POLICY_CAL_CAPTURE_MAX_AGE_MS, CFG_STALE_ORDINARY_MS,
 >  BOARD_PH_DIVIDER_TOP_OHM,
252                                 BOARD_PH_DIVIDER_BOTTOM_OHM};
253   float x;
254   uint32_t ageMs;
255   const CaptureStatus st = calCaptureFromSnapshot(cmd->sensor, &s_scratch.snap, monoNowMs(),
 >  &limits, &x, &ageMs);
256   if (st != CAPTURE_OK) {
257     sendError(cmd->id, st == CAPTURE_UNSUPPORTED ? PROTOCOL_ERR_BAD_FIELD :
 >  PROTOCOL_ERR_STALE, generation);
258     return;
259   }
260   const int32_t scaled = (int32_t)(x * CAL_X_SCALE + (x >= 0.0f ? 0.5f : -0.5f));
261   if (protocolEncodeCapture(cmd->sensor, scaled, ageMs, s_data, sizeof(s_data)) > 0) {
262     sendResultFrame(protocolEncodeResultOk(cmd->id, s_data, payload(), FRAME_MAX_PAYLOAD),
 >  generation);
263   }
264 }
265
266 static void handleCommand(size_t len, uint32_t generation) {
267   Command cmd;
268   const ProtocolError error = protocolParseCommand(s_command, len, &cmd);
269   STATS_ADD(commands, 1);
270   if (error != PROTOCOL_OK) {
271     STATS_ADD(commandErrors, 1);
272     logPrintf(LOG_LEVEL_INFO, "ble: command ? -> %s", protocolErrorName(error));
273     sendError(cmd.id, error, generation);
274     return;
275   }
276   logPrintf(LOG_LEVEL_INFO, "ble: command %s", commandOpName(cmd.op));  // never the fields
 >  (set_wifi)
277   if (cmd.op == CMD_OP_GET_SENSORS) {
278     configurationPeekApplied(&s_cfg);
```

Return to guided explanation, p. 435

ORIGINAL SOURCE / CHAPTER 15 / 7 OF 12

src/transports/ble_transport.c

Original lines 279-323 | Code and comments unchanged; long lines wrap visually

handleCommand (L266-346): Validate and answer read-only operations from copies; queue Configuration or Safety mutations with a session-aware pending tag.

```
279     if (protocolEncodeSensorList(s_cfg.sensorEnabled, s_data, sizeof(s_data)) > 0) {
280       sendResultFrame(protocolEncodeResultOk(cmd.id, s_data, payload(), FRAME_MAX_PAYLOAD),
  >   generation);
281     }
282     return;
283   }
284   if (cmd.op == CMD_OP_GET_CALIBRATION) {
285     configurationPeekApplied(&s_cfg);
286     const CalibrationView v = {s_cfg.ph, s_cfg.tds, s_cfg.light, s_cfg.calOverridden};
287     if (protocolEncodeCalibration(&v, s_data, sizeof(s_data)) > 0) {
288       sendResultFrame(protocolEncodeResultOk(cmd.id, s_data, payload(), FRAME_MAX_PAYLOAD),
  >   generation);
289     }
290     return;
291   }
292   if (cmd.op == CMD_OP_CAL_CAPTURE) {
293     handleCapture(&cmd, generation);
294     return;
295   }
296   const int slot = claimSlot(&cmd, generation);
297   if (slot < 0) {
298     sendBusy(cmd.id, generation);
299     return;
300   }
301   const MonoMs expires = monoNowMs() + BLE_COMMAND_TTL_MS;
302   bool queued;
303   if (cmd.op == CMD_OP_RESET_LATCH) {
304     const SafetyCommand sc = {SAFETY_CMD_RESET_LATCH, cmd.rule, expires, tagOf(slot)};
305     queued = safetySubmitCommand(&sc, generation, sessionNow, onSafetyResult, NULL);  //
  >   dropped if the link closes
306   } else {
307     memset(&s_req, 0, sizeof(s_req));
308     s_req.tag = tagOf(slot);
309     s_req.session = generation;
310     s_req.sessionNow = sessionNow;
311     s_req.expiresMs = expires;
312     s_req.reply = onConfigResult;
313     switch (cmd.op) {
314       case CMD_OP_SET_SENSOR_ENABLED:
315         s_req.kind = CONFIG_REQ_SET_SENSOR_ENABLED;
316         s_req.sensor = cmd.sensor;
317         s_req.enabled = cmd.enabled;
318         break;
319       case CMD_OP_SET_WIFI:
320         s_req.kind = CONFIG_REQ_SET_WIFI;
321         memcpy(s_req.ssid, cmd.ssid, sizeof(s_req.ssid));
322         memcpy(s_req.password, cmd.password, sizeof(s_req.password));
323         break;
```

Return to guided explanation, p. 435

ORIGINAL SOURCE / CHAPTER 15 / 8 OF 12

src/transports/ble_transport.c

Original lines 324-371 | Code and comments unchanged; long lines wrap visually

handleCommand (L266-346): Validate and answer read-only operations from copies; queue Configuration or Safety mutations with a session-aware pending tag.
drainRx (L348-369): Feed arriving stream bytes to the assembler and parse only complete command payloads.

```
      case CMD_OP_FORGET_WIFI: s_req.kind = CONFIG_REQ_FORGET_WIFI; break;
      case CMD_OP_SET_CALIBRATION:
        s_req.kind = CONFIG_REQ_SET_CALIBRATION;
        s_req.sensor = cmd.sensor;
        s_req.calX1 = cmd.calX1;
        s_req.calY1 = cmd.calY1;
        s_req.calX2 = cmd.calX2;
        s_req.calY2 = cmd.calY2;
        break;
      default:
        s_req.kind = CONFIG_REQ_RESET_CALIBRATION;
        s_req.sensor = cmd.sensor;
        break;
    }
    queued = configurationSubmit(&s_req);
    memset(s_req.password, 0, sizeof(s_req.password));
  }
  memset(cmd.password, 0, sizeof(cmd.password));
  if (!queued) {
    s_pending[slot].used = false;
    sendBusy(cmd.id, generation);
  }
}

static void drainRx(uint32_t generation) {
  uint8_t chunk[128];
  size_t got;
  while ((got = xStreamBufferReceive(s_rx, chunk, sizeof(chunk), 0)) > 0) {
    size_t offset = 0;
    while (offset < got) {
      size_t used;
      const uint8_t* framePayload;
      size_t payloadLen;
      const FrameStatus status =
          frameAssemblerFeed(&s_assembler, &chunk[offset], got - offset, &used,
&framePayload, &payloadLen);
      offset += used;
      if (status == FRAME_READY) {
        memcpy(s_command, framePayload, payloadLen);  // replies reuse s_frame
        handleCommand(payloadLen, generation);
      } else if (status == FRAME_ERROR) {
        STATS_ADD(commandErrors, 1);
        logPrintf(LOG_LEVEL_WARN, "ble: bad frame length from client, buffer dropped");
      }
    }
  }
}

static void drainResults(uint32_t generation) {
```

Return to guided explanation, p. 435

ORIGINAL SOURCE / CHAPTER 15 / 9 OF 12

src/transports/ble_transport.c

Original lines 372-414 | Code and comments unchanged; long lines wrap visually

drainResults (L371-396): Match slot and sequence, discard old-session replies and send the actual stored/applied/error outcome.
expirePending (L401-406): Free orphaned or excessively old pending slots; service session checks separately prevent stale queued mutations.

```
372   BleResult r;
373   while (xQueueReceive(s_results, &r, 0) == pdTRUE) {
374     const uint32_t slot = r.tag & 0xFFu;
375     if (slot >= BLE_PENDING) continue;
376     Pending* p = &s_pending[slot];
377     if (!p->used || p->seq != (r.tag >> 8)) continue;  // freed already: a late reply
378     p->used = false;
379     if (p->generation != generation) continue;  // asked on an earlier link: never answered
  >  on this one
380     if (r.error != PROTOCOL_OK) {
381       sendError(p->id, r.error, generation);
382       logPrintf(LOG_LEVEL_INFO, "ble: %s -> %s", commandOpName(p->op),
  >  protocolErrorName(r.error));
383       continue;
384     }
385     if (p->op == CMD_OP_SET_SENSOR_ENABLED || p->op == CMD_OP_SET_CALIBRATION ||
386         p->op == CMD_OP_RESET_CALIBRATION) {
387       snprintf(s_data, sizeof(s_data), "{\"applied\":true,\"cfg_rev\":%u}",
  >  (unsigned)r.cfgRev);
388     } else if (p->op == CMD_OP_RESET_LATCH) {
389       snprintf(s_data, sizeof(s_data), "{\"applied\":true}");
390     } else {
391       snprintf(s_data, sizeof(s_data), "{\"stored\":true}");
392     }
393     logPrintf(LOG_LEVEL_INFO, "ble: %s -> ok", commandOpName(p->op));
394     sendResultFrame(protocolEncodeResultOk(p->id, s_data, payload(), FRAME_MAX_PAYLOAD),
  >  generation);
395   }
396 }
397
398 // A link change orphans every request of the old link (Configuration drops
399 // stale-session requests without replying); a reply that never comes frees
400 // its slot after BLE_PENDING_LIMIT_MS.
401 static void expirePending(uint32_t generation, MonoMs now) {
402   for (int i = 0; i < BLE_PENDING; ++i) {
403     Pending* p = &s_pending[i];
404     if (p->used && (p->generation != generation || now - p->sinceMs > BLE_PENDING_LIMIT_MS))
  >  p->used = false;
405   }
406 }
407
408 // Peek, send, then commit: an event that could not be sent stays at the
409 // cursor and is retried on the next wake (the journal reports a gap if it
410 // overruns meanwhile). If the link closes first, the next session starts at
411 // the journal head, so its first event reports gap:true.
412 static void sendEvents(uint32_t generation) {
413   JournalEvent e;
414   bool gap;
```

Return to guided explanation, p. 435

ORIGINAL SOURCE / CHAPTER 15 / 10 OF 12

src/transports/ble_transport.c

Original lines 415-455 | Code and comments unchanged; long lines wrap visually

sendEvents (L412-427): Peek an event through a tentative cursor, send it and commit cursor progress only on success.
logStats (L429-440): Framing, pending request tags and session checks belong to this task; callbacks only copy and notify.

```
  JournalCursor next = s_journalCursor;
  while (safetyJournalNext(&next, &e, &gap)) {
    const size_t len = transportEncodeEvent(&e, gap || s_eventLost, payload(),
FRAME_MAX_PAYLOAD);
    if (!sendFrame(BLE_CHANNEL_RESULT, len, generation)) {
      STATS_ADD(resultsDropped, 1);
      s_eventPending = true;
      break;
    }
    s_journalCursor = next;
    s_eventPending = false;
    s_eventLost = false;
  }
}

static void logStats(void) {
  BleGattStats g;
  BleTransportStats t;
  bleGattGetStats(&g);
  bleTransportGetStats(&t);
  logPrintf(LOG_LEVEL_INFO, "ble: secure=%d bonds=%d conn=%lu disc=%lu pair ok=%lu rej=%lu |
frames %lu tlmDrop %lu "
            "resDrop %lu busy %lu maxSend %lums notifyFail %lu",
            bleGattSecure(), bleGattBondCount(), (unsigned long)g.connects, (unsigned
long)g.disconnects,
            (unsigned long)g.pairingsAccepted, (unsigned long)g.pairingsRejected, (unsigned
long)t.framesSent,
            (unsigned long)t.telemetryDropped, (unsigned long)t.resultsDropped, (unsigned
long)t.busy,
            (unsigned long)t.maxSendMs, (unsigned long)g.notifyFailures);
}

// ---- Task -------------------------------------------------------------------
static void bleTask(void* arg) {
  (void)arg;
  if (esp_task_wdt_add(NULL) != ESP_OK) logPrintf(LOG_LEVEL_ERROR, "ble: task watchdog add
failed");
  logPrintf(LOG_LEVEL_WARN, "ble: PAIRING PASSKEY %06lu (bonded owners: %d)", (unsigned
long)bleGattPasskey(),
            bleGattBondCount());

  uint32_t generation = bleGattGeneration();
  MonoMs nextState = 0, nextTelemetry = 0, nextStats = monoNowMs() + BLE_STATS_LOG_MS,
bootPressedAt = 0;
  bool wasSecure = false, bondsCleared = false;
  for (;;) {
    ulTaskNotifyTake(pdTRUE, pdMS_TO_TICKS(BLE_WAKE_MS));
    const MonoMs now = monoNowMs();

```

Return to guided explanation, p. 435

ORIGINAL SOURCE / CHAPTER 15 / 11 OF 12

src/transports/ble_transport.c

Original lines 456-501 | Code and comments unchanged; long lines wrap visually

bleTask (L443-507): Reset partial state on session changes, prioritize commands/results/events, publish periodic state/telemetry and handle bond recovery.

```
456     if (bleGattGeneration() != generation) {  // new session: drop half-received commands
457       generation = bleGattGeneration();
458       frameAssemblerReset(&s_assembler);
459       xStreamBufferReset(s_rx);
460       if (s_eventPending) s_eventLost = true;  // never delivered: the next link is told
461       s_eventPending = false;
462     }
463     expirePending(generation, now);
464     const bool secure = bleGattSecure();
465     if (secure != wasSecure) {
466       logPrintf(LOG_LEVEL_INFO, "ble: %s", secure ? "owner connected (secure)" : "link
  > closed");
467       wasSecure = secure;
468       if (secure) {
469         nextState = now;  // State at once
470         safetyJournalCursorInit(&s_journalCursor);  // events from now on, not history
471       }
472     }
473
474     // Priority: commands and their results, alarm events, State, Telemetry.
475     if (secure) drainRx(generation);
476     drainResults(generation);
477     if (secure) sendEvents(generation);
478     if (now >= nextState) {
479       publishState(secure, generation);
480       nextState = now + CFG_BLE_STATE_PERIOD_MS;
481     }
482     if (secure && now >= nextTelemetry) {
483       publishTelemetry(generation);
484       nextTelemetry = now + CFG_BLE_TELEMETRY_PERIOD_MS;
485     }
486
487     // Owner recovery: BOOT held 10 s while running (not at reset: strap pin).
488     if (digitalRead(BLE_BOOT_PIN) == LOW) {
489       if (bootPressedAt == 0) bootPressedAt = now;
490       if (!bondsCleared && now - bootPressedAt >= BLE_BOND_CLEAR_MS) {
491         bleGattClearBonds();
492         logPrintf(LOG_LEVEL_WARN, "ble: BOOT held 10 s - all bonds cleared, pair again with
  > the passkey");
493         bondsCleared = true;
494       }
495     } else {
496       bootPressedAt = 0;
497       bondsCleared = false;
498     }
499
500     if (now >= nextStats) {
501       logStats();
```

Return to guided explanation, p. 435

ORIGINAL SOURCE / CHAPTER 15 / 12 OF 12

src/transports/ble_transport.c

Original lines 502-545 | Code and comments unchanged; long lines wrap visually

bleTask (L443-507): Reset partial state on session changes, prioritize commands/results/events, publish periodic state/telemetry and handle bond recovery.
bleTransportInit (L509-535): Create finite buffers/queues, start the GATT adapter, populate State and create the transport task.

```
502       nextStats = now + BLE_STATS_LOG_MS;
503     }
504     progressMark(TASK_ID_BLE_TRANSPORT);
505     esp_task_wdt_reset();
506   }
507 }
508
509 bool bleTransportInit(void) {
510   if (s_task != NULL) return true;
511   s_rx = xStreamBufferCreateStatic(BLE_RX_BYTES, 1, s_rxStorage, &s_rxStruct);
512   s_results = xQueueCreateStatic(BLE_PENDING, sizeof(BleResult), s_resultStorage,
 >  &s_resultControl);
513   if (s_rx == NULL || s_results == NULL) return false;
514   frameAssemblerInit(&s_assembler);
515   pinMode(BLE_BOOT_PIN, INPUT_PULLUP);
516   if (!bleGattStart(BLE_NODE_NAME, onRx, NULL)) {
517     logPrintf(LOG_LEVEL_ERROR, "ble: NimBLE failed to start");
518     return false;
519   }
520   publishState(false, bleGattGeneration());  // also fixes the shared boot id (radio running)
 >   // readable State before the first client
521
522   const TaskSpec* spec = cfgTaskSpec(TASK_ID_BLE_TRANSPORT);
523   if (spec->stackBytes != sizeof(s_stack)) {
524     logPrintf(LOG_LEVEL_WARN, "ble: stack %u B, task table says %lu B",
 >  (unsigned)sizeof(s_stack),
525                 (unsigned long)spec->stackBytes);
526   }
527   s_task = xTaskCreateStaticPinnedToCore(bleTask, spec->name, sizeof(s_stack), NULL,
 >  spec->priority, s_stack,
528                                          &s_tcb, spec->core);
529   if (s_task == NULL) return false;
530   TaskHandle_t host = xTaskGetHandle("nimble_host");
531   logPrintf(LOG_LEVEL_INFO, "ble: started (core %u, prio %u), NimBLE host core %d prio %u",
532             (unsigned)spec->core, (unsigned)spec->priority, host != NULL ?
 >  (int)xTaskGetAffinity(host) : -1,
533             host != NULL ? (unsigned)uxTaskPriorityGet(host) : 0u);
534   return true;
535 }
536
537 void bleTransportGetStats(BleTransportStats* out) {
538   portENTER_CRITICAL(&s_statsMux);
539   *out = s_stats;
540   portEXIT_CRITICAL(&s_statsMux);
541 }
542
543 void* bleTransportTaskHandle(void) { return s_task; }
544
545 #endif  // FW_ENABLE_BLE
```

Return to guided explanation, p. 435

ORIGINAL SOURCE / CHAPTER 15 / 1 OF 1

src/transports/wifi_lan.h

Original lines 1-32 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // Wi-Fi station shim over the Arduino WiFi library (C++),
2  // callable from C. Work package WP8 (Stage 8.4). Used only by the WifiMqtt
3  // task (transports/wifi_mqtt.c), which is the sole Wi-Fi owner.
4  //
5  // The Arduino libraries keep netif and the default event loop initialised
6  // exactly once (initialising them again from IDF calls would fail under
7  // arduino-esp32 2.0.x). Credentials stay in RAM: WiFi.persistent(false) sets
8  // WIFI_STORAGE_RAM, so the Wi-Fi driver never writes them to NVS.
9  // Compiled only with -D FW_ENABLE_WIFI.
10 #pragma once
11 #include <stdbool.h>
12 #include <stdint.h>
13
14 #ifdef __cplusplus
15 extern "C" {
16 #endif
17
18 bool wifiLanBegin(const char* hostname);  // station mode, RAM storage, no auto-reconnect
19 void wifiLanConnect(const char* ssid, const char* password);  // non-blocking
20 void wifiLanDisconnect(void);
21 bool wifiLanConnected(void);              // associated and has an IPv4 address
22 bool wifiLanIp(char out[16]);
23 int32_t wifiLanRssi(void);
24
25 // WP9 low power: WIFI_OFF (radio and its RF calibration stopped), and back to
26 // station mode. Both return false if the driver refused.
27 bool wifiLanRadioOff(void);
28 bool wifiLanRadioOn(void);
29
30 #ifdef __cplusplus
31 }
32 #endif
```

Return to guided explanation, p. 386

ORIGINAL SOURCE / CHAPTER 15 / 1 OF 1

src/transports/wifi_lan.cpp

Original lines 1-48 | Code and comments unchanged; long lines wrap visually

Small extern C wrappers call Arduino WiFi; automatic reconnect/persistence are disabled in favor of explicit owners.

```
1  // Arduino WiFi implementation of wifi_lan.h. C++ only because the
2  // Arduino libraries are; everything it exports is C.
3  #ifdef FW_ENABLE_WIFI
4  #include "transports/wifi_lan.h"
5
6  #include <WiFi.h>
7  #include <stdio.h>
8
9  extern "C" bool wifiLanBegin(const char* hostname) {
10   WiFi.persistent(false);         // WIFI_STORAGE_RAM: credentials never reach NVS from here
11   WiFi.setAutoReconnect(false);   // retries are WifiMqtt's backoff, not the driver's
12   if (!WiFi.mode(WIFI_STA)) return false;
13   WiFi.setHostname(hostname);
14   // Modem sleep stays on (the default): required while BLE shares the radio.
15   return true;
16 }
17
18 extern "C" void wifiLanConnect(const char* ssid, const char* password) {
19   WiFi.disconnect(false, false);
20   WiFi.begin(ssid, password);
21 }
22
23 extern "C" void wifiLanDisconnect(void) { WiFi.disconnect(false, false); }
24
25 extern "C" bool wifiLanConnected(void) {
26   return WiFi.status() == WL_CONNECTED && static_cast<uint32_t>(WiFi.localIP()) != 0;
27 }
28
29 extern "C" bool wifiLanIp(char out[16]) {
30   if (!wifiLanConnected()) {
31     out[0] = '\0';
32     return false;
33   }
34   const IPAddress ip = WiFi.localIP();
35   snprintf(out, 16, "%u.%u.%u.%u", ip[0], ip[1], ip[2], ip[3]);
36   return true;
37 }
38
39 extern "C" int32_t wifiLanRssi(void) { return wifiLanConnected() ? WiFi.RSSI() : 0; }
40
41 extern "C" bool wifiLanRadioOff(void) {
42   WiFi.disconnect(true, false);  // wifioff = true
43   return WiFi.mode(WIFI_OFF);
44 }
45
46 extern "C" bool wifiLanRadioOn(void) { return WiFi.mode(WIFI_STA); }
47
48 #endif  // FW_ENABLE_WIFI
```

Return to guided explanation, p. 386

ORIGINAL SOURCE / CHAPTER 15 / 1 OF 2

src/transports/wifi_mqtt.h

Original lines 1-27 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
// WifiMqtt: sole Wi-Fi owner (station), never restarts sensing. Work package
// WP8 (Stage 8.4). Architecture §17; FirmwareContract.md v2.
//
// Task "WifiMqtt" (cfgTaskSpec: core 0, priority 2, 6 KB):
//   station   credentials from Configuration; connect, and on failure retry
//             with exponential backoff 1, 2, 4 ... 60 s +/- 20 % jitter
//             (domain/backoff). A credential change or forget restarts it at
//             once. The Wi-Fi driver keeps nothing in NVS (WIFI_STORAGE_RAM).
//   status    state / SSID / IP / RSSI for the State frame (the app shows it
//             on its Wi-Fi page)
//   WP9       wifiMqttRequestStop() disconnects and puts the radio in
//             WIFI_OFF (low power); wifiMqttRadioOff() is the acknowledgement
//             PowerCoordinator waits for. RequestStart() turns it back on and
//             rejoins.
// The app talks to the controller over Bluetooth only: the read-only LAN link
// (TCP 47800 + mDNS) was removed after Stage 8.4. MQTT (server publishing) is
// not in scope yet; the name is kept from the architecture's task table.
//
// Built only with -D FW_ENABLE_WIFI (production envs). Without it the task
// does not exist and the status is always "unconfigured".
#pragma once
#include <stdbool.h>
#include <stdint.h>

#include "domain/protocol_codec.h"  // PROTOCOL_SSID_MAX

typedef uint8_t WifiState;
```

Return to guided explanation, p. 386

ORIGINAL SOURCE / CHAPTER 15 / 2 OF 2

src/transports/wifi_mqtt.h

Original lines 28-54 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
28  enum { WIFI_STATE_UNCONFIGURED = 0, WIFI_STATE_CONNECTING, WIFI_STATE_CONNECTED,
 >  WIFI_STATE_FAILED };
29
30  typedef struct WifiStatus {
31    WifiState state;
32    char ssid[PROTOCOL_SSID_MAX + 1];
33    char ip[16];              // dotted IPv4 when connected, else ""
34    int32_t rssi;             // dBm when connected
35  } WifiStatus;
36
37  bool wifiMqttInit(void);          // starts the task; never waits for an access point
38  void wifiMqttRequestStop(void);   // WP9 power transition: drop Wi-Fi, radio off, preempt
 >  retry waits
39  void wifiMqttRequestStart(void);
40  bool wifiMqttRadioOff(void);      // true once a stop request has the radio in WIFI_OFF
41
42  void wifiMqttGetStatus(WifiStatus* out);  // zero wait, any task
43  const char* wifiStateName(WifiState s);   // contract spelling
44
45  typedef struct WifiMqttStats {
46    uint32_t connects;          // station connections established
47    uint32_t connectFailures;   // attempts that timed out or were refused
48    uint32_t lastBackoffMs;
49  } WifiMqttStats;
50
51  void wifiMqttGetStats(WifiMqttStats* out);
52
53  // For Supervisor. NULL before init or without FW_ENABLE_WIFI.
54  void* wifiMqttTaskHandle(void);
```

Return to guided explanation, p. 386

GUIDED SOURCE READING / CHAPTER 15

A connection timeout should not be a 15-second task sleep

The Wi-Fi adapter begins association and DHCP, but the task must continue answering power-stop requests and reporting progress while those processes run.

Original excerpt: src/transports/wifi_mqtt.c (p. 453) / lines 180-193. The complete unchanged file follows.

```
180        case PHASE_ATTEMPT:
181          if (wifiLanConnected()) {
182            onConnected();
183          } else if (now >= s_deadlineMs) {
184            wifiLanDisconnect();
185            STATS_ADD(connectFailures, 1);
186            scheduleRetry(now);
187            setStatus(WIFI_STATE_FAILED);
188            logPrintf(LOG_LEVEL_WARN, "wifi: join failed, retry in %lu ms", (unsigned
  >  long)s_stats.lastBackoffMs);
189          }
190          break;
191        case PHASE_WAIT:
192          if (now >= s_deadlineMs) {
193            startAttempt(now);
```

Each wake checks connection status first. Success changes phase, resets backoff and publishes connected status. Otherwise the deadline test decides whether the attempt has expired. There is no loop waiting here for an access point to answer.

Failure disconnects, increments its counter, calculates a later retry time and enters WAIT. On a future wake, WAIT compares now with that deadline and starts the next attempt only when due. Placing vTaskDelay(15000 ms) inside startAttempt would prevent this task from handling a stop request promptly during that sleep.

Power-stop and credential-change handling precede the phase switch. A newly requested stop can therefore tear down association before the regular attempt logic advances. Credential changes similarly restart from the authoritative Configuration record.

Despite the retained WifiMqtt name, this task currently manages a station connection only. Its structure teaches a reusable RTOS pattern: initiate external work, retain a phase/deadline, block for a short useful wait, then reevaluate. A long external response time does not require long CPU occupation or an unresponsive service.

ORIGINAL SOURCE / CHAPTER 15 / 1 OF 6

src/transports/wifi_mqtt.c

Original lines 1-45 | Code and comments unchanged; long lines wrap visually

wifiStateName (L9-16): Translate an identity/state into a human-readable label for diagnostics.
wifiMqttGetStatus (L18-22): Despite its retained name, this is a Wi-Fi station state machine; it currently has no MQTT publisher or data server.

```
#include "transports/wifi_mqtt.h"

#include <freertos/FreeRTOS.h>
#include <string.h>

static portMUX_TYPE s_statusMux = portMUX_INITIALIZER_UNLOCKED;
static WifiStatus s_status;  // all zero = unconfigured

const char* wifiStateName(WifiState s) {
  switch (s) {
    case WIFI_STATE_CONNECTING: return "connecting";
    case WIFI_STATE_CONNECTED: return "connected";
    case WIFI_STATE_FAILED: return "failed";
    default: return "unconfigured";
  }
}

void wifiMqttGetStatus(WifiStatus* out) {
  portENTER_CRITICAL(&s_statusMux);
  *out = s_status;
  portEXIT_CRITICAL(&s_statusMux);
}

#ifndef FW_ENABLE_WIFI

bool wifiMqttInit(void) { return false; }
void wifiMqttRequestStop(void) {}
void wifiMqttRequestStart(void) {}
bool wifiMqttRadioOff(void) { return true; }  // no Wi-Fi in this build: nothing to switch
off
void wifiMqttGetStats(WifiMqttStats* out) { memset(out, 0, sizeof(*out)); }
void* wifiMqttTaskHandle(void) { return NULL; }

#else

#include <esp_system.h>
#include <esp_task_wdt.h>
#include <freertos/task.h>
#include <stdio.h>

#include "app_config.h"
#include "diagnostics/logger.h"
#include "diagnostics/progress.h"
#include "domain/backoff.h"
#include "hal/clock.h"
#include "services/configuration.h"
```

Return to guided explanation, p. 452

ORIGINAL SOURCE / CHAPTER 15 / 2 OF 6

src/transports/wifi_mqtt.c

Original lines 46-91 | Code and comments unchanged; long lines wrap visually

setStatus (L80-92): Despite its retained name, this is a Wi-Fi station state machine; it currently has no MQTT publisher or data server.

```
46  #include "transports/wifi_lan.h"
47
48  #define WIFI_HOSTNAME "hydro-esp1"
49  #define WIFI_WAKE_MS 50u
50  #define WIFI_ATTEMPT_TIMEOUT_MS 15000u  // one association + DHCP attempt
51  #define WIFI_JITTER_PCT 20u
52  #define WIFI_STATUS_REFRESH_MS 2000u    // RSSI / IP copy for State
53  #define WIFI_STATS_LOG_MS 60000u
54
55  typedef uint8_t Phase;
56  enum { PHASE_IDLE = 0, PHASE_ATTEMPT, PHASE_WAIT, PHASE_CONNECTED };
57
58  static StaticTask_t s_tcb;
59  static StackType_t s_stack[6 * 1024];  // cfgTaskSpec(TASK_ID_WIFI_MQTT)->stackBytes
60  static TaskHandle_t s_task = NULL;
61
62  static volatile bool s_stopRequested;
63  static volatile bool s_radioOff;  // WP9 acknowledgement: the radio is in WIFI_OFF
64  static portMUX_TYPE s_statsMux = portMUX_INITIALIZER_UNLOCKED;
65  static WifiMqttStats s_stats;
66  #define STATS_ADD(field, n)          \
67    do {                               \
68      portENTER_CRITICAL(&s_statsMux); \
69      s_stats.field += (n);            \
70      portEXIT_CRITICAL(&s_statsMux);  \
71    } while (0)
72
73  // Task-private (static: not on the 6 KB stack).
74  static ConfigWifi s_creds;
75  static uint32_t s_credsChanges = UINT32_MAX;
76  static Backoff s_backoff;
77  static Phase s_phase;
78  static MonoMs s_deadlineMs;
79
80  static void setStatus(WifiState state) {
81    WifiStatus st;
82    memset(&st, 0, sizeof(st));
83    st.state = state;
84    memcpy(st.ssid, s_creds.ssid, sizeof(st.ssid));
85    if (state == WIFI_STATE_CONNECTED) {
86      wifiLanIp(st.ip);
87      st.rssi = wifiLanRssi();
88    }
89    portENTER_CRITICAL(&s_statusMux);
90    s_status = st;
91    portEXIT_CRITICAL(&s_statusMux);
```

Return to guided explanation, p. 452

ORIGINAL SOURCE / CHAPTER 15 / 3 OF 6

src/transports/wifi_mqtt.c

Original lines 92-134 | Code and comments unchanged; long lines wrap visually

setStatus (L80-92): Despite its retained name, this is a Wi-Fi station state machine; it currently has no MQTT publisher or data server.
currentState (L94-99): Despite its retained name, this is a Wi-Fi station state machine; it currently has no MQTT publisher or data server.

```
92  }
93
94  static WifiState currentState(void) {
95    portENTER_CRITICAL(&s_statusMux);
96    const WifiState state = s_status.state;
97    portEXIT_CRITICAL(&s_statusMux);
98    return state;
99  }
100
101 // ---- Station ---------------------------------------------------------------------------
  >
-
102 static void teardown(void) { wifiLanDisconnect(); }
103
104 static void startAttempt(MonoMs now) {
105   wifiLanConnect(s_creds.ssid, s_creds.password);
106   s_deadlineMs = now + WIFI_ATTEMPT_TIMEOUT_MS;
107   s_phase = PHASE_ATTEMPT;
108 }
109
110 static void scheduleRetry(MonoMs now) {
111   const uint32_t delay = backoffNext(&s_backoff, esp_random());
112   portENTER_CRITICAL(&s_statsMux);
113   s_stats.lastBackoffMs = delay;
114   portEXIT_CRITICAL(&s_statsMux);
115   s_deadlineMs = now + delay;
116   s_phase = PHASE_WAIT;
117 }
118
119 static void onConnected(void) {
120   s_phase = PHASE_CONNECTED;
121   backoffReset(&s_backoff);
122   STATS_ADD(connects, 1);
123   setStatus(WIFI_STATE_CONNECTED);
124   logPrintf(LOG_LEVEL_INFO, "wifi: connected, %s (RSSI %ld dBm)", s_status.ip,
  >
(long)s_status.rssi);
125 }
126
127 // Credentials come from Configuration; a change takes effect at once.
128 static void followConfiguration(MonoMs now) {
129   uint32_t changes;
130   configurationGetWifi(&s_creds, &changes);
131   if (changes != s_credsChanges) {
132     s_credsChanges = changes;
133     teardown();
134     backoffReset(&s_backoff);
```

Return to guided explanation, p. 452

ORIGINAL SOURCE / CHAPTER 15 / 4 OF 6

src/transports/wifi_mqtt.c

Original lines 135-175 | Code and comments unchanged; long lines wrap visually

followConfiguration (L128-144): Notice changed credentials, tear down old association and start or idle according to configuration and power stop state.
logStats (L146-153): Despite its retained name, this is a Wi-Fi station state machine; it currently has no MQTT publisher or data server.

```
135     if (s_creds.ssid[0] != '\0' && !s_stopRequested) {
136       logPrintf(LOG_LEVEL_INFO, "wifi: joining \"%s\"", s_creds.ssid);
137       startAttempt(now);
138       setStatus(WIFI_STATE_CONNECTING);
139     } else {
140       s_phase = PHASE_IDLE;
141       setStatus(WIFI_STATE_UNCONFIGURED);
142     }
143   }
144 }
145
146 static void logStats(void) {
147   WifiMqttStats t;
148   wifiMqttGetStats(&t);
149   WifiStatus st;
150   wifiMqttGetStatus(&st);
151   logPrintf(LOG_LEVEL_INFO, "wifi: %s %s rssi %ld | joins %lu fails %lu backoff %lums",
 > wifiStateName(st.state), st.ip,
152             (long)st.rssi, (unsigned long)t.connects, (unsigned long)t.connectFailures,
 > (unsigned long)t.lastBackoffMs);
153 }
154
155 static void wifiTask(void* arg) {
156   (void)arg;
157   if (esp_task_wdt_add(NULL) != ESP_OK) logPrintf(LOG_LEVEL_ERROR, "wifi: task watchdog add
 > failed");
158   MonoMs nextStatus = 0, nextStats = monoNowMs() + WIFI_STATS_LOG_MS;
159   bool stopped = false;
160   for (;;) {
161     ulTaskNotifyTake(pdTRUE, pdMS_TO_TICKS(WIFI_WAKE_MS));
162     const MonoMs now = monoNowMs();
163
164     if (s_stopRequested != stopped) {  // WP9 power transitions
165       stopped = s_stopRequested;
166       s_credsChanges = UINT32_MAX;     // re-evaluated below: idle when stopped, rejoin when
 > started
167       if (stopped) {
168         teardown();
169         s_radioOff = wifiLanRadioOff();  // the PowerCoord acknowledgement
170         logPrintf(LOG_LEVEL_INFO, "wifi: stopped on request, radio %s", s_radioOff ? "off" :
 > "OFF FAILED");
171       } else {
172         s_radioOff = false;
173         if (!wifiLanRadioOn()) logPrintf(LOG_LEVEL_ERROR, "wifi: radio restart failed");
174         logPrintf(LOG_LEVEL_INFO, "wifi: started on request");
175       }
```

Return to guided explanation, p. 452

ORIGINAL SOURCE / CHAPTER 15 / 5 OF 6

src/transports/wifi_mqtt.c

Original lines 176-219 | Code and comments unchanged; long lines wrap visually

wifiTask (L155-220): Follow power and credential changes, progress association/retry/connected states and refresh copied status without waiting for an AP.

```
    }
    followConfiguration(now);

    switch (s_phase) {
      case PHASE_ATTEMPT:
        if (wifiLanConnected()) {
          onConnected();
        } else if (now >= s_deadlineMs) {
          wifiLanDisconnect();
          STATS_ADD(connectFailures, 1);
          scheduleRetry(now);
          setStatus(WIFI_STATE_FAILED);
          logPrintf(LOG_LEVEL_WARN, "wifi: join failed, retry in %lu ms", (unsigned
> long)s_stats.lastBackoffMs);
        }
        break;
      case PHASE_WAIT:
        if (now >= s_deadlineMs) {
          startAttempt(now);
          setStatus(WIFI_STATE_CONNECTING);
        }
        break;
      case PHASE_CONNECTED:
        if (!wifiLanConnected()) {
          logPrintf(LOG_LEVEL_WARN, "wifi: connection lost");
          teardown();
          scheduleRetry(now);  // from 1 s: the backoff was reset on connect
          setStatus(WIFI_STATE_CONNECTING);
          break;
        }
        if (now >= nextStatus) {  // RSSI, IP for State
          setStatus(WIFI_STATE_CONNECTED);
          nextStatus = now + WIFI_STATUS_REFRESH_MS;
        }
        break;
      default: break;
    }

    if (now >= nextStats && currentState() != WIFI_STATE_UNCONFIGURED) {
      logStats();
      nextStats = now + WIFI_STATS_LOG_MS;
    }
    progressMark(TASK_ID_WIFI_MQTT);
    esp_task_wdt_reset();
  }
```

Return to guided explanation, p. 452

ORIGINAL SOURCE / CHAPTER 15 / 6 OF 6

src/transports/wifi_mqtt.c

Original lines 220-261 | Code and comments unchanged; long lines wrap visually

wifiTask (L155-220): Follow power and credential changes, progress association/retry/connected states and refresh copied status without waiting for an AP.
wifiMqttInit (L222-239): Check dependencies, configure this module and return whether initialisation succeeded.

```
220 }
221
222 bool wifiMqttInit(void) {
223   if (s_task != NULL) return true;
224   backoffInit(&s_backoff, CFG_MQTT_BACKOFF_INITIAL_MS, CFG_MQTT_BACKOFF_MAX_MS,
  > WIFI_JITTER_PCT);
225   if (!wifiLanBegin(WIFI_HOSTNAME)) {
226     logPrintf(LOG_LEVEL_ERROR, "wifi: station mode failed");
227     return false;
228   }
229   const TaskSpec* spec = cfgTaskSpec(TASK_ID_WIFI_MQTT);
230   if (spec->stackBytes != sizeof(s_stack)) {
231     logPrintf(LOG_LEVEL_WARN, "wifi: stack %u B, task table says %lu B",
  > (unsigned)sizeof(s_stack),
232               (unsigned long)spec->stackBytes);
233   }
234   s_task = xTaskCreateStaticPinnedToCore(wifiTask, spec->name, sizeof(s_stack), NULL,
  > spec->priority, s_stack, &s_tcb,
235                                          spec->core);
236   if (s_task == NULL) return false;
237   logPrintf(LOG_LEVEL_INFO, "wifi: started (core %u, prio %u)", (unsigned)spec->core,
  > (unsigned)spec->priority);
238   return true;
239 }
240
241 void wifiMqttRequestStop(void) {
242   s_stopRequested = true;
243   if (s_task != NULL) xTaskNotifyGive(s_task);
244 }
245
246 void wifiMqttRequestStart(void) {
247   s_stopRequested = false;
248   if (s_task != NULL) xTaskNotifyGive(s_task);
249 }
250
251 bool wifiMqttRadioOff(void) { return s_task == NULL || (s_stopRequested && s_radioOff); }
252
253 void wifiMqttGetStats(WifiMqttStats* out) {
254   portENTER_CRITICAL(&s_statsMux);
255   *out = s_stats;
256   portEXIT_CRITICAL(&s_statsMux);
257 }
258
259 void* wifiMqttTaskHandle(void) { return s_task; }
260
261 #endif  // FW_ENABLE_WIFI
```

Return to guided explanation, p. 452

PART IV / INTEGRATION AND VERIFICATION

CHAPTER 16

Building, Bring-up and Native Tests

Production and bring-up builds select different entry points. The original native tests isolate decisions and codecs from hardware so their inputs and expected outcomes can be inspected directly. Together, these tools connect the architectural explanation to executable examples.

In this chapter

1. Select a production or one-sensor build.
2. Read native fixtures, fake time and meaningful assertions.
3. Distinguish host evidence, repository bench records and target measurements.

Chapter contents

16.1 Production and bring-up build different entry points 460
16.2 What each sensor bring-up program adds 461
16.3 Combined climate and water tests teach dependencies 462
16.4 Native tests as executable examples 463
16.5 The serial capture helper is a host-side tool 464

Guided source readings begin on page 496.

Complete original listings begin on page 465. Shared files are linked in the explanations; Appendix A contains the full filename index.

CHAPTER 16

16.1 Production and bring-up build different entry points

The production environment includes src/main.c. Each sensor-test environment includes the production modules but replaces that entry point with a test/bringup main. The `native` environment includes only domain code and the host tests.

```
# Build the production image
pio run -e esp32dev

# Build, upload and monitor a single-sensor image
pio run -e test_bme280 -t upload -t monitor

# Run pure-domain tests on the host
pio test -e native
```

These commands describe the repository workflow. Upload writes the connected target. A production image has automatic lamp control enabled; a bench test image has the behaviour defined by its own entry point.

The shared bringup_harness.h

BringupContext holds a driver pointer, period and optional afterResult callback. `bringupStart()` creates a core-1 priority-4 task. `bringupTask()` waits on an absolute period, initialises as needed, starts a measurement and polls with timed delays until READY/error or a period-length guard.

`bringupPrintResult()` prints accepted values with units, marks uncalibrated values, and otherwise prints health plus raw diagnostics when present. The callback runs in that same task after READY, preserving single-owner driver access.

Why a single-sensor test is easier to reason about

The harness can wait inside a measurement's overall progression because it has only one sensor to serve. Production instead interleaves seven drivers and protects fast climate and echo timing. Passing standalone bring-up proves useful wiring and driver behaviour, but it does not prove integrated scheduling.

Bring-up frequently uses direct printf. Production routes logs through SerialLogger. Their timing and stack demands differ. A driver time measured in one test environment is useful evidence but should be checked under production load rather than assumed identical.

Original source in this PDF: platformio.ini (p. 28); test/bringup/bringup_harness.h (p. 465); src/main.c (p. 82)

CHAPTER 16

16.2 What each sensor bring-up program adds

Program	Work beyond the shared harness
`bme280_main.c`	Probes both addresses and prints chip identity before constructing the driver.
`ds18b20_main.c`	Prints the selected ROM only when it is first discovered or changes.
`light_main.c`	Starts ADC1 on the light pin and reports filtered uncalibrated mV.
`hcsr04_main.c`	Begins capture and uses the configured echo timeout.
`ph_main.c`	Chooses two-buffer or provisional calibration and prints the fitted line.
`tds_main.c`	Optionally supplies a synthetic manual-temperature Reading.
`sgp30_main.c`	Reports feature/baseline state and accepted baseline words.
`lamp_main.c`	Feeds light into LampControl and drives a separate test relay path.

All define setup/loop using the C-to-Arduino linker bridge. Their setup functions construct objects and begin the necessary HAL. Their loop functions park the Arduino loop task.

Small callbacks have an ownership purpose

DS18B20's printRomIfNew reads identity from the driver's owner task. SGP30's printBaseline reads RAM baseline information after results. TDS's feedManualTemperature refreshes its synthetic dependency after a result. These callbacks avoid an extra task reaching into live driver state.

Test lamp is not the full Safety service

`afterLight()` calls LampControl and a local relay helper, then prints held time and dwell information. It does not provide production stale-snapshot checks or alarm interlocks. Its comments describe why directly driving a particular 5V-referenced relay module from 3.3V failed, motivating the transistor interface.

The calibration and lamp constants in test files are independent copies. Changing a test threshold or fit does not update production policy. Read the active PlatformIO environment before deciding which constants explain an observed output.

Original source in this PDF: test/bringup/bme280_main.c (p. 189); test/bringup/ds18b20_main.c (p. 210); test/bringup/sgp30_main.c (p. 262); test/bringup/lamp_main.c (p. 315)

CHAPTER 16

16.3 Combined climate and water tests teach dependencies

`test_climate` uses one task for both I2C sensors. Each 250 ms tick runs one BME280 operation. Every fourth tick it passes current accepted temperature/humidity into SGP30 and runs a gas measurement. It prints absolute humidity, compensation state, baseline state and timing.

Its `runOnce()` is a small blocking-at-the-test-level init/start/poll loop with a guard. `checkTiming()` reports first timing and later overruns. `printCell()`, `sgp30State()` and `printRow()` provide human-readable units and status. This is an integration probe, not production's fully interleaved acquisition loop.

test_water runs two pending operations together

The water test copies the previous DS18B20 Reading into TDS, begins both drivers and alternates due polls. `beginRun()` starts a driver without waiting; `stepRun()` advances one due poll; `cancelRun()` enforces the frame boundary. It sleeps until the earlier next poll.

The first frame can have no valid water temperature. Later TDS windows can use the preceding frame's temperature. The program prints both uncompensated and 25 degC-referred ppm so temperature effects are visible rather than hidden by one final number.

The trend column is a diagnostic heuristic

`trendUpdate()` keeps ten displayed ppm values, restarting when validity or compensation mode changes. It compares the mean of the older five against the newer five. Allowed drift is 0.4% of the combined mean, with a minimum tolerance of 2 ppm. It also requires each value within 2% of the mean, again with at least 2 ppm tolerance.

A stable label therefore means this recent window meets a drift/spread rule. It does not mean reference calibration passed, the probe has no bias, or production accepts the reading for alarm use. The test can display and trend values that are diagnostic only.

`waterCell()` and `tdsStateName()` translate failed, missing-temperature and overrange cases into table labels. `printRow()` selects compensated or uncompensated values consistently before updating the trend state.

Original source in this PDF: test/bringup/climate_main.c (p. 468); test/bringup/water_main.c (p. 473)

CHAPTER 16

16.4 Native tests as executable examples

PlatformIO native selects domain source only. The three original Unity suites can run on a PC without Arduino, FreeRTOS or a sensor. Each supplies deterministic records, times or byte arrays and checks results.

Original suite	Passing cases	Main coverage
test_domain	48	Readings, schedules, alarms, lamp/Safety, CRCs and sensor helpers.
test_power	11	Boot modes, lease actions, timeouts, return stability and epoch changes.
test_protocol	27	Framing, JSON, commands, configuration, journal, capture and fits.

All 86 existing tests passed with GCC in GNU11 mode and upstream Unity, with zero failures and zero ignored cases. The full test files are printed in this chapter. No firmware source was changed to obtain that result.

Read a test as an example with an answer

A typical test builds a known input, calls a pure function and asserts an expected outcome. Fake monotonic time makes a 500 ms stability check instantaneous on the PC. A split-frame test feeds fragments in several calls and verifies that READY appears only when the declared payload is complete.

The sticky regression test covers an in-flight queue-send interleaving. Power tests verify that a late release never grants two owners. Calibration tests verify mathematical and range checks. These cases check algorithm behaviour with deterministic inputs and outcomes.

Host verification and target verification

The verification record contains no ESP32-S3 execution or PlatformIO firmware build. Host tests do not execute the GPIO ISR, NimBLE callbacks, Wi-Fi driver, flash cache pauses, actual NVS commits or FreeRTOS cross-core scheduling. The repository identifies the newest power/calibration integration as written and still awaiting hardware validation.

Original source in this PDF: platformio.ini (p. 28); test/native/test_domain/test_main.c (p. 480); test/native/test_power/test_main.c (p. 497); test/native/test_protocol/test_main.c (p. 503)

CHAPTER 16

16.5 The serial capture helper is a host-side tool

tools/serial_log.py is included because it captures firmware evidence without adding monitor-induced resets. It is not compiled into the ESP32 image and does not participate in RTOS scheduling.

Read the small script from top to bottom

It reads a port, output path and duration from command-line arguments. It creates a pyserial object at 115200 baud, sets DTR and RTS before opening, then reads bytes until the duration ends. Newline-delimited records are timestamped with the PC monotonic clock and flushed to disk.

The optional --reset flag creates one deliberate RTS pulse so the log has a known boot. A serial adapter's control-line wiring determines actual pin behaviour; the comments describe the project's adapter convention. Do not confuse host elapsed seconds with the MCU's MonoMs timestamps.

Minimal dependency and limits

The helper needs Python and pyserial. It is a short bench utility with direct positional arguments, not a full argument-validation interface. A final unterminated line can remain in its buffer at timeout. For interpreting firmware behaviour, distinguish lost logging, monitor resets and genuine task-watchdog resets.

The repository also has PC BLE bench scripts importing the desktop application's BleLink and Protocol modules. They are excluded with the app. Firmware radio failure paths are explained here through their actual C/C++ code and native protocol tests.

Original source in this PDF: tools/serial_log.py (p. 519)

ORIGINAL SOURCE / CHAPTER 16 / 1 OF 3

test/bringup/bringup_harness.h

Original lines 1-37 | Code and comments unchanged; long lines wrap visually

bringupPrintResult (L32-47): Print values only in their declared usable/diagnostic categories, with health and raw detail otherwise.

```
1  // Minimal RTOS bring-up harness shared by every test/bringup/<sensor>_main.c.
2  // Architecture §17 stage 1-3, requirement R9.
3  //
4  // Runs ONE production driver from a task pinned like AcquisitionNormal
5  // (core 1, priority 4), on an absolute period (vTaskDelayUntil). Each period:
6  // init (retried until OK) -> start -> poll until READY/error -> readResult.
7  // Prints every reading with its health and the start->ready duration so the
8  // conversion budget in app_config.h can be checked against reality.
9  #pragma once
10 #include <Arduino.h>
11 #include <stdio.h>
12
13 #include "app_config.h"
14 #include "domain/reading.h"
15 #include "drivers/sensor_driver.h"
16 #include "hal/arduino_entry.h"
17 #include "hal/clock.h"
18
19 typedef struct BringupContext {
20   SensorDriver* driver;
21   uint32_t periodMs;
22   // Optional, called from the harness task after every READY result (so it
23   // may use driver getters without breaking single bus ownership). NULL = none.
24   void (*afterResult)(SensorDriver* driver);
25 } BringupContext;
26
27 // One compact line per measurement, e.g.
28 //   [bme280] 27.98 degC | 54.20 %RH | 101602.04 Pa   (11 ms)
29 // An UNCALIBRATED reading shows its raw-unit value with "(uncal)", e.g.
30 //   [ph4502c] 2498.31 mV (uncal)   (271 ms)
31 // Any other non-OK reading shows its metric and health instead of a value.
32 static inline void bringupPrintResult(const char* name, const Reading* r, uint8_t n,
33                                       uint32_t elapsedMs) {
34   printf("[%s] ", name);
35   for (uint8_t i = 0; i < n; ++i) {
36     if (i > 0) printf(" | ");
37     if (r[i].health == HEALTH_OK) {
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 2 OF 3

test/bringup/bringup_harness.h

Original lines 38-73 | Code and comments unchanged; long lines wrap visually

bringupPrintResult (L32-47): Print values only in their declared usable/diagnostic categories, with health and raw detail otherwise.
bringupTask (L49-103): Run one driver per period with timed polling, a frame guard and an optional result callback.

```
      printf("%.2f %s", (double)r[i].value, unitName(r[i].unit));
    } else if (r[i].health == HEALTH_UNCALIBRATED) {
      printf("%.2f %s (uncal)", (double)r[i].value, unitName(r[i].unit));
    } else {
      printf("%s=%s", metricName(r[i].metric), healthName(r[i].health));
      if (r[i].hasRaw) printf(" (raw %ld)", (long)r[i].raw);
    }
  }
  printf("   (%lu ms)\n", (unsigned long)elapsedMs);
}

static inline void bringupTask(void* arg) {
  BringupContext* ctx = (BringupContext*)arg;
  SensorDriver* drv = ctx->driver;
  bool initialised = false;
  TickType_t lastWake = xTaskGetTickCount();

  for (;;) {
    vTaskDelayUntil(&lastWake, pdMS_TO_TICKS(ctx->periodMs));

    if (!initialised) {
      DriverStatus st = sensorDriverInit(drv);
      printf("[%s] init -> %s\n", sensorDriverName(drv), driverStatusName(st));
      if (st != DRIVER_OK) continue;
      initialised = true;
    }

    const MonoMs t0 = monoNowMs();
    DriverStatus st = sensorDriverStart(drv, t0);
    if (st != DRIVER_OK) {
      printf("[%s] start -> %s\n", sensorDriverName(drv), driverStatusName(st));
      if (st != DRIVER_PENDING) initialised = false;
      continue;
    }

    PollResult pr = sensorDriverPoll(drv, monoNowMs());
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 3 OF 3

test/bringup/bringup_harness.h

Original lines 74-110 | Code and comments unchanged; long lines wrap visually

bringupTask (L49-103): Run one driver per period with timed polling, a frame guard and an optional result callback.
bringupStart (L106-110): Begin one operation and record its state; completion is handled later by poll.

```
    while (pr.status == DRIVER_PENDING) {
      const MonoMs now = monoNowMs();
      if (now - t0 >= ctx->periodMs) {  // harness-level guard, not the driver budget
        sensorDriverCancel(drv);
        pr.status = DRIVER_TIMEOUT;
        break;
      }
      uint32_t waitMs = 1;
      if (pr.nextPollMs > now) waitMs = (uint32_t)(pr.nextPollMs - now);
      TickType_t ticks = pdMS_TO_TICKS(waitMs);
      vTaskDelay(ticks > 0 ? ticks : 1);
      pr = sensorDriverPoll(drv, monoNowMs());
    }

    const uint32_t elapsedMs = (uint32_t)(monoNowMs() - t0);
    if (pr.status != DRIVER_READY) {
      printf("[%s] poll -> %s after %lu ms\n", sensorDriverName(drv),
             driverStatusName(pr.status), (unsigned long)elapsedMs);
      if (pr.status == DRIVER_NOT_IMPLEMENTED || pr.status == DRIVER_NOT_PRESENT) {
        initialised = false;
      }
      continue;
    }

    Reading out[4];
    const uint8_t n = sensorDriverReadResult(drv, out, 4);
    bringupPrintResult(sensorDriverName(drv), out, n, elapsedMs);
    if (ctx->afterResult != NULL) ctx->afterResult(drv);
  }
}

// Start the harness task with the AcquisitionNormal placement from §6.
static inline void bringupStart(BringupContext* ctx) {
  const TaskSpec* spec = cfgTaskSpec(TASK_ID_ACQUISITION_NORMAL);
  xTaskCreatePinnedToCore(bringupTask, "bringup", spec->stackBytes, ctx,
                          spec->priority, NULL, spec->core);
}
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 1 OF 5

test/bringup/climate_main.c

Original lines 1-48 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
1  // Combined bring-up for the shared I2C bus: BME280 (WP1) + SGP30 (WP6).
2  //   pio run -e test_climate -t upload -t monitor
3  // Both on the retained 3.3 V supply, SDA 21 / SCL 47, addresses 0x76/0x77 and
4  // 0x58. Not the production scheduler (WP7): one task owns the bus and runs a
5  // 250 ms tick. Each tick runs one BME280 measurement; every 4th tick (1 s) it
6  // passes the fresh BME280 T/RH to the SGP30 as humidity compensation, then
7  // runs one SGP30 measurement. One labelled line per second:
8  //   t=   17s  BME280  T  25.62 C  RH  58.28 %  P 1022.96 hPa  AH 13.87 g/m3  |
9  //     SGP30  TVOC     5 ppb  CO2eq   413 ppm  LEARNING          (one line)
10 // A value that is not usable shows its health instead (FAULT, warm, ...).
11 // state = SGP30 health: WARMING, LEARNING (baseline not yet learned, D13),
12 // OK, DEP_INVALID (humidity input stale), FAULT; "nocomp" = no humidity
13 // compensation. Events (init, faults, baseline and compensation changes,
14 // a driver over its time budget) print as their own "# ..." lines.
15 // Replaces src/main.c in this env only; links the production drivers.
16 #include <Arduino.h>
17 #include <stdio.h>
18 #include <string.h>
19
20 #include "app_config.h"
21 #include "board_profile.h"
22 #include "bringup_harness.h"
23 #include "domain/sgp30_codec.h"
24 #include "drivers/bme280.h"
25 #include "drivers/sgp30.h"
26 #include "hal/clock.h"
27 #include "hal/i2c_bus.h"
28
29 #define CLIMATE_SGP30_EVERY_TICKS 4  // 4 x 250 ms = the SGP30's 1 s cadence
30 #define CLIMATE_SGP30_BUDGET_MS 15   // architecture §9 host budget
31
32 static I2cBus s_i2c = I2C_BUS_INIT;
33 static Bme280Config s_bmeConfig;  // address chosen at runtime in setup()
34 static Bme280Driver s_bme;
35 static Sgp30Config s_sgpConfig = {&s_i2c, BOARD_SGP30_I2C_ADDRESS};
36 static Sgp30Driver s_sgp;
37
38 typedef struct DriverRun {
39   SensorDriver* driver;
40   bool initialised;
41   uint32_t guardMs;  // harness-level guard, not the driver budget
42   Reading out[4];
43   uint8_t n;
44   uint32_t elapsedMs;
45 } DriverRun;
46
47 // One init (if needed) -> start -> poll until READY/error -> readResult.
48 // Same logic as bringupTask(), for one measurement. true if readings arrived.
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 2 OF 5

test/bringup/climate_main.c

Original lines 49-96 | Code and comments unchanged; long lines wrap visually

runOnce (L49-91): Combined-climate test: initialise/start/poll one driver under a guard and retain its results. **printSgp30Baseline (L93-111)**: Present already-collected diagnostic information; this does not make the measurement more valid.

```
49  static bool runOnce(DriverRun* run) {
50    SensorDriver* drv = run->driver;
51    run->n = 0;
52    if (!run->initialised) {
53      const DriverStatus st = sensorDriverInit(drv);
54      printf("# %s init -> %s\n", sensorDriverName(drv), driverStatusName(st));
55      if (st != DRIVER_OK) return false;
56      run->initialised = true;
57    }
58
59    const MonoMs t0 = monoNowMs();
60    const DriverStatus st = sensorDriverStart(drv, t0);
61    if (st != DRIVER_OK) {
62      printf("# %s start -> %s\n", sensorDriverName(drv), driverStatusName(st));
63      if (st != DRIVER_PENDING) run->initialised = false;
64      return false;
65    }
66
67    PollResult pr = sensorDriverPoll(drv, monoNowMs());
68    while (pr.status == DRIVER_PENDING) {
69      const MonoMs now = monoNowMs();
70      if (now - t0 >= run->guardMs) {
71        sensorDriverCancel(drv);
72        pr.status = DRIVER_TIMEOUT;
73        break;
74      }
75      uint32_t waitMs = 1;
76      if (pr.nextPollMs > now) waitMs = (uint32_t)(pr.nextPollMs - now);
77      const TickType_t ticks = pdMS_TO_TICKS(waitMs);
78      vTaskDelay(ticks > 0 ? ticks : 1);
79      pr = sensorDriverPoll(drv, monoNowMs());
80    }
81
82    run->elapsedMs = (uint32_t)(monoNowMs() - t0);
83    if (pr.status != DRIVER_READY) {
84      printf("# %s poll -> %s after %lu ms\n", sensorDriverName(drv),
85             driverStatusName(pr.status), (unsigned long)run->elapsedMs);
86      if (pr.status == DRIVER_NOT_PRESENT) run->initialised = false;
87      return false;
88    }
89    run->n = sensorDriverReadResult(drv, run->out, 4);
90    return true;
91  }
92
93  static void printSgp30Baseline(void) {
94    static Sgp30BaselineStatus lastStatus = SGP30_BASELINE_NONE;
95    static uint16_t lastCo2eq = 0;
96    static uint16_t lastTvoc = 0;
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 3 OF 5

test/bringup/climate_main.c

Original lines 97-142 | Code and comments unchanged; long lines wrap visually

printSgp30Baseline (L93-111): Present already-collected diagnostic information; this does not make the measurement more valid.
checkTiming (L114-124): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.

```
97    const Sgp30BaselineStatus st = sgp30DriverBaselineStatus(&s_sgp);
98    if (st != lastStatus) {
99      printf("# sgp30 baseline %s (feature set 0x%04X)\n", sgp30BaselineStatusName(st),
100           (unsigned)sgp30DriverFeatureSet(&s_sgp));
101     lastStatus = st;
102   }
103   uint16_t co2eq = 0;
104   uint16_t tvoc = 0;
105   if (sgp30DriverGetBaseline(&s_sgp, &co2eq, &tvoc) && (co2eq != lastCo2eq || tvoc !=
  > lastTvoc)) {
106     printf("# sgp30 baseline co2eq 0x%04X tvoc 0x%04X (store these)\n", (unsigned)co2eq,
107           (unsigned)tvoc);
108     lastCo2eq = co2eq;
109     lastTvoc = tvoc;
110   }
111 }
112
113 // First timing once, then only a run over its budget.
114 static void checkTiming(const char* name, const DriverRun* run, uint32_t budgetMs,
115                         bool* reported) {
116   if (!*reported) {
117     printf("# %s took %lu ms (budget %lu ms)\n", name, (unsigned long)run->elapsedMs,
118           (unsigned long)budgetMs);
119     *reported = true;
120   } else if (run->elapsedMs > budgetMs) {
121     printf("# %s took %lu ms, over its %lu ms budget\n", name, (unsigned long)run->elapsedMs,
122           (unsigned long)budgetMs);
123   }
124 }
125
126 // One labelled value, e.g. "T 25.65 C". Not usable: its health, e.g. "T FAULT".
127 static void printCell(const char* label, const Reading* r, int width, int decimals,
128                       float scale, const char* unit) {
129   const Health h = r->health;
130   if (h == HEALTH_OK || h == HEALTH_UNCALIBRATED || h == HEALTH_DEPENDENCY_INVALID) {
131     printf("  %s %*.*f %s", label, width, decimals, (double)(r->value * scale), unit);
132     return;
133   }
134   const char* why = h == HEALTH_WARMING ? "warm" : h == HEALTH_STARTING ? "-" :
  > healthName(h);
135   printf("  %s %*s %*s", label, width, why, (int)strlen(unit), "");
136 }
137
138 static const char* sgp30State(const Reading* r) {
139   switch (r->health) {
140     case HEALTH_OK: return "OK";
141     case HEALTH_UNCALIBRATED: return "LEARNING";
142     case HEALTH_DEPENDENCY_INVALID: return "DEP_INVALID";
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 4 OF 5

test/bringup/climate_main.c

Original lines 143-190 | Code and comments unchanged; long lines wrap visually

sgp30State (L138-146): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.
printRow (L150-166): Present already-collected diagnostic information; this does not make the measurement more valid.

```
143     case HEALTH_STARTING: return "-";
144     default: return healthName(r->health);
145   }
146 }
147 
148 // e.g.  t=64s  BME280  T 25.65 C  RH 57.70 %  P 1022.89 hPa  AH 13.74 g/m3  |
149 //       SGP30  TVOC   22 ppb  CO2eq  413 ppm  LEARNING      (one line)
150 static void printRow(MonoMs now, const Reading* bme, bool ahValid, float ah,
151                      const Reading* sgp) {
152   printf("t=%5lus  BME280", (unsigned long)(now / 1000));
153   printCell("T", &bme[0], 6, 2, 1.0f, "C");
154   printCell("RH", &bme[1], 6, 2, 1.0f, "%");
155   printCell("P", &bme[2], 7, 2, 0.01f, "hPa");  // Pa -> hPa
156   if (ahValid) {
157     printf("  AH %5.2f g/m3", (double)ah);
158   } else {
159     printf("  AH %5s     ", "-");
160   }
161   printf("  |  SGP30");
162   printCell("TVOC", &sgp[0], 5, 0, 1.0f, "ppb");
163   printCell("CO2eq", &sgp[1], 5, 0, 1.0f, "ppm");
164   printf("  %s%s\n", sgp30State(&sgp[0]),
165          (sgp[0].qualityFlags & kQualityUncompensated) ? " nocomp" : "");
166 }
167 
168 static void climateTask(void* arg) {
169   (void)arg;
170   DriverRun bme = {&s_bme.base, false, CFG_BME280_PERIOD_MS, {{0}}, 0, 0};
171   DriverRun sgp = {&s_sgp.base, false, CFG_BME280_PERIOD_MS, {{0}}, 0, 0};
172   bool bmeTimed = false;
173   bool sgpTimed = false;
174   int lastCompensated = -1;  // unknown
175   uint32_t tick = 0;
176   TickType_t lastWake = xTaskGetTickCount();
177 
178   for (;;) {
179     vTaskDelayUntil(&lastWake, pdMS_TO_TICKS(CFG_BME280_PERIOD_MS));
180     const bool secondTick = (tick++ % CLIMATE_SGP30_EVERY_TICKS) == 0;
181 
182     if (runOnce(&bme)) checkTiming("bme280", &bme, CFG_BME280_BUDGET_MS, &bmeTimed);
183     if (!secondTick) continue;
184 
185     // Readings 0/1 are air temperature and humidity (bme280.h). A failed
186     // BME280 run leaves the last readings, which then age out as stale.
187     const MonoMs now = monoNowMs();
188     Reading latest[BME280_METRICS];
189     sensorDriverReadResult(&s_bme.base, latest, BME280_METRICS);
190     sgp30DriverSetHumidity(&s_sgp, &latest[0], &latest[1], now);
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 5 OF 5

test/bringup/climate_main.c

Original lines 191-237 | Code and comments unchanged; long lines wrap visually

climateTask (L168-206): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.
setup (L208-235): Establish safe outputs, logger, gate, board check and queues; load configuration; start Safety, both acquisition workers, power, radios and supervision.

```
191     const bool fresh = readingIsActionableAt(&latest[0], now, CFG_STALE_BME280_MS) &&
192                        readingIsActionableAt(&latest[1], now, CFG_STALE_BME280_MS);
193     const float ah = fresh ? sgp30AbsoluteHumidity(latest[0].value, latest[1].value) : 0.0f;
194     if ((int)fresh != lastCompensated) {
195       printf(fresh ? "# sgp30 humidity compensation ON (from BME280)\n"
196                    : "# BME280 T/RH not fresh -> sgp30 humidity compensation OFF\n");
197       lastCompensated = fresh;
198     }
199 
200     if (runOnce(&sgp)) checkTiming("sgp30", &sgp, CLIMATE_SGP30_BUDGET_MS, &sgpTimed);
201     Reading sgpOut[SGP30_METRICS];
202     sensorDriverReadResult(&s_sgp.base, sgpOut, SGP30_METRICS);
203     printRow(now, latest, fresh, ah, sgpOut);
204     printSgp30Baseline();
205   }
206 }
207 
208 void setup(void) {
209   delay(200);
210   printf("\n=== bring-up: climate (bme280 + sgp30, SDA %d, SCL %d, %lu Hz) ===\n",
211          BOARD_PIN_I2C_SDA, BOARD_PIN_I2C_SCL, (unsigned long)BOARD_I2C_CLOCK_HZ);
212 
213   const bool busOk = i2cBusBegin(&s_i2c, BOARD_PIN_I2C_SDA, BOARD_PIN_I2C_SCL,
214                                  BOARD_I2C_CLOCK_HZ);
215   printf("i2c begin -> %s\n", busOk ? "OK" : "FAILED");
216 
217   // BME280: whichever address ACKs (SDO strap); default if neither does.
218   uint8_t bmeAddress = BOARD_BME280_I2C_ADDRESS;
219   const bool ack76 = i2cBusProbe(&s_i2c, BOARD_BME280_I2C_ADDRESS);
220   const bool ack77 = i2cBusProbe(&s_i2c, BOARD_BME280_I2C_ADDRESS_ALT);
221   const bool ack58 = i2cBusProbe(&s_i2c, BOARD_SGP30_I2C_ADDRESS);
222   printf("probe bme280 0x%02X -> %s, 0x%02X -> %s | sgp30 0x%02X -> %s\n",
223          BOARD_BME280_I2C_ADDRESS, ack76 ? "ACK" : "-", BOARD_BME280_I2C_ADDRESS_ALT,
224          ack77 ? "ACK" : "-", BOARD_SGP30_I2C_ADDRESS, ack58 ? "ACK" : "-");
225   if (!ack76 && ack77) bmeAddress = BOARD_BME280_I2C_ADDRESS_ALT;
226 
227   s_bmeConfig.bus = &s_i2c;
228   s_bmeConfig.address = bmeAddress;
229   bme280DriverConstruct(&s_bme, &s_bmeConfig);
230   sgp30DriverConstruct(&s_sgp, &s_sgpConfig);
231 
232   const TaskSpec* spec = cfgTaskSpec(TASK_ID_ACQUISITION_NORMAL);
233   xTaskCreatePinnedToCore(climateTask, "climate", spec->stackBytes, NULL, spec->priority,
  > NULL,
234                           spec->core);
235 }
236 
237 void loop(void) { vTaskDelay(portMAX_DELAY); }
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 1 OF 7

test/bringup/water_main.c

Original lines 1-49 | Code and comments unchanged; long lines wrap visually

Read these declarations and comments as the contract for this module. The chapter walkthrough explains the fields, lifecycle and result meaning.

```
// Combined bring-up for the water probes: DS18B20 (WP2) + SEN0244 TDS (WP5).
//   pio run -e test_water -t upload -t monitor
// DS18B20 on GPIO14 (1-Wire, retained 3.3 V); TDS A on GPIO6 (ADC1_CH5,
// switched 5 V rail must be ON). Not the production scheduler (WP7): one task
// owns both buses and runs a 1 s frame. Each frame it passes the latest
// DS18B20 reading to the TDS driver (tdsDriverSetWaterTemperature), then runs
// both measurements interleaved: the DS18B20 converts for 750 ms while the
// TDS window (~560 ms) is sampled, so both fit one frame. TDS is therefore
// compensated with the previous frame's water temperature (~1 s old, inside
// the 3 s staleness limit); the first frame has none (NO TEMP).
//
// One table row per frame (header repeated every WATER_HEADER_EVERY rows):
//         | DS18B20 |  SEN0244 TDS                                  |
//    time | water T |    raw | uncomp |   @25C | state   | trend    |
//      12 |   24.81 |   1245 |    470 |    466 | UNCAL   | stable   |
//   water T  DS18B20 degC, or why not: FAULT, NOPRES (no presence pulse),
//            TIMEOUT (frame guard), "-" (no result yet)
//   raw      TDS pin mV (filtered window)
//   uncomp   ppm from raw with NO temperature compensation
//   @25C     ppm compensated to 25 degC (what production uses); "-" without
//            a fresh water temperature
//   state    OK, UNCAL (nominal curve, no K-factor, D13), NO TEMP (no fresh
//            water temperature), OVER (above 1000 ppm or ADC clipped), FAULT
//   trend    stable once the ppm shown (@25C, or uncomp when there is none)
//            has stopped drifting: over the last WATER_TREND_ROWS rows the mean
//            of the newer half is within 0.4 % (min 2 ppm) of the older half,
//            and no row is more than 2 % off. Otherwise settling. Judged on
//            @25C, comparing half-means, because the DS18B20 creeps towards
//            the water temperature for a minute or more (~0.15 %/s in ppm),
//            slower than the row-to-row noise.
// Events (init, faults, ROM code, compensation on/off, first timings) print
// as their own "# ..." lines between rows.
// Replaces src/main.c in this env only; links the production drivers.
#include <Arduino.h>
#include <stdio.h>
#include <string.h>

#include "app_config.h"
#include "board_profile.h"
#include "bringup_harness.h"
#include "domain/tds_codec.h"
#include "drivers/ds18b20.h"
#include "drivers/tds.h"
#include "hal/adc_channel.h"
#include "hal/clock.h"
#include "hal/onewire_bus.h"

#define WATER_HEADER_EVERY 20     // rows between repeated table headers
#define WATER_TREND_ROWS 10           // rows in the window (even: two halves)
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 2 OF 7

test/bringup/water_main.c

Original lines 50-96 | Code and comments unchanged; long lines wrap visually

beginRun (L81-105): Water test: start one driver without waiting for completion.

```
#define WATER_TREND_DRIFT_PCT 0.4f    // newer-half vs older-half mean, % ...
#define WATER_TREND_MIN_PPM 2.0f      // ... but never tighter than this
#define WATER_TREND_SPREAD_PCT 2.0f   // any single row vs the mean

static OneWireBus s_oneWire = ONE_WIRE_BUS_INIT;
static Ds18b20Config s_dsConfig = {&s_oneWire};
static Ds18b20Driver s_ds;
static AdcChannel s_adc = ADC_CHANNEL_INIT;
static TdsConfig s_tdsConfig = {&s_adc, {1.0f, 0.0f, 0, false}};  // nominal curve, no
K-factor
static TdsDriver s_tds;

typedef struct DriverRun {
  SensorDriver* driver;
  bool initialised;
  bool pending;    // started, not yet READY or failed
  bool ready;      // this frame produced a result
  DriverStatus failure;  // why not ready (DRIVER_OK = not started this frame)
  MonoMs t0;
  MonoMs nextPollMs;
  uint32_t elapsedMs;
} DriverRun;

// Last rows, for the trend column.
typedef struct Trend {
  uint8_t count;  // valid rows, up to WATER_TREND_ROWS
  uint8_t next;
  bool compensated;  // which column the window holds (@25C or uncomp)
  float ppm[WATER_TREND_ROWS];
} Trend;

// init (if needed) -> start. Same logic as bringupTask(), without waiting.
static void beginRun(DriverRun* run, MonoMs now) {
  SensorDriver* drv = run->driver;
  run->pending = false;
  run->ready = false;
  run->failure = DRIVER_OK;
  if (!run->initialised) {
    const DriverStatus st = sensorDriverInit(drv);
    printf("# %s init -> %s\n", sensorDriverName(drv), driverStatusName(st));
    if (st != DRIVER_OK) {
      run->failure = st;
      return;
    }
    run->initialised = true;
  }
  run->t0 = now;
  const DriverStatus st = sensorDriverStart(drv, now);
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 3 OF 7

test/bringup/water_main.c

Original lines 97-145 | Code and comments unchanged; long lines wrap visually

beginRun (L81-105): Water test: start one driver without waiting for completion.
stepRun (L108-126): Water test: advance one due poll, saving pending, ready or failure state.

```
97    if (st != DRIVER_OK) {
98      printf("# %s start -> %s\n", sensorDriverName(drv), driverStatusName(st));
99      if (st != DRIVER_PENDING) run->initialised = false;
100     run->failure = st;
101     return;
102   }
103   run->pending = true;
104   run->nextPollMs = now;
105 }
106
107 // One poll if it is due. Clears `pending` once READY or failed.
108 static void stepRun(DriverRun* run, MonoMs now) {
109   if (!run->pending || now < run->nextPollMs) return;
110   SensorDriver* drv = run->driver;
111   const PollResult pr = sensorDriverPoll(drv, now);
112   if (pr.status == DRIVER_PENDING) {
113     run->nextPollMs = pr.nextPollMs > now ? pr.nextPollMs : now + 1;
114     return;
115   }
116   run->pending = false;
117   run->elapsedMs = (uint32_t)(monoNowMs() - run->t0);
118   if (pr.status != DRIVER_READY) {
119     printf("# %s poll -> %s after %lu ms\n", sensorDriverName(drv),
120            driverStatusName(pr.status), (unsigned long)run->elapsedMs);
121     if (pr.status == DRIVER_NOT_PRESENT) run->initialised = false;
122     run->failure = pr.status;
123     return;
124   }
125   run->ready = true;
126 }
127
128 // Harness-level guard, not a driver budget: abandon what is left of the frame.
129 static void cancelRun(DriverRun* run) {
130   if (!run->pending) return;
131   sensorDriverCancel(run->driver);
132   run->pending = false;
133   run->failure = DRIVER_TIMEOUT;
134   printf("# %s TIMEOUT (frame guard)\n", sensorDriverName(run->driver));
135 }
136
137 static void printTimingOnce(const DriverRun* run, bool* reported) {
138   if (*reported || !run->ready) return;
139   printf("# %s took %lu ms\n", sensorDriverName(run->driver), (unsigned long)run->elapsedMs);
140   *reported = true;
141 }
142
143 static void printRomIfNew(void) {
144   static uint8_t last[8];
145   static bool printed = false;
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 4 OF 7

test/bringup/water_main.c

Original lines 146-193 | Code and comments unchanged; long lines wrap visually

printRomIfNew (L143-153): Present already-collected diagnostic information; this does not make the measurement more valid.
printHeader (L155-161): Present already-collected diagnostic information; this does not make the measurement more valid.

```
  uint8_t rom[8];
  if (!ds18b20DriverGetRom(&s_ds, rom)) return;
  if (printed && memcmp(rom, last, sizeof(rom)) == 0) return;
  memcpy(last, rom, sizeof(rom));
  printed = true;
  printf("# ds18b20 ROM %02X-%02X%02X%02X%02X%02X%02X crc %02X\n", rom[0], rom[6], rom[5],
         rom[4], rom[3], rom[2], rom[1], rom[7]);
}

static void printHeader(void) {
  printf("\n");
  printf("        | DS18B20 |  SEN0244 TDS                                  |\n");
  printf("   time | water T |    raw | uncomp |   @25C | state   | trend    |\n");
  printf("      s |    degC |     mV |    ppm |    ppm |         |          |\n");
  printf("--------+---------+--------+--------+--------+---------+----------+\n");
}

// Water temperature cell: the value, or why there is none this frame.
static void waterCell(const DriverRun* run, const Reading* t, char* cell, size_t len) {
  if (!run->ready) {
    const char* why = run->failure == DRIVER_NOT_PRESENT ? "NOPRES"
                    : run->failure == DRIVER_TIMEOUT   ? "TIMEOUT"
                    : run->failure == DRIVER_OK        ? "-"
                                                       : "FAULT";
    snprintf(cell, len, "%s", why);
  } else if (t->health == HEALTH_OK) {
    snprintf(cell, len, "%.2f", (double)t->value);
  } else {
    snprintf(cell, len, "%s", t->health == HEALTH_STARTING ? "-" : healthName(t->health));
  }
}

static const char* tdsStateName(const Reading* r) {
  switch (r->health) {
    case HEALTH_OK: return "OK";
    case HEALTH_UNCALIBRATED: return "UNCAL";
    case HEALTH_DEPENDENCY_INVALID: return "NO TEMP";
    case HEALTH_STARTING: return "-";
    case HEALTH_FAULT: return (r->qualityFlags & kQualitySaturated) ? "OVER" : "FAULT";
    default: return healthName(r->health);
  }
}

// Add this row and return "stable" / "settling" / "-". `ppm` is the value
// shown: @25C when compensated, else uncomp. A gap or a switch between the
// two restarts the window.
static const char* trendUpdate(Trend* tr, bool tdsOk, bool compensated, float ppm) {
  if (!tdsOk) {
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 5 OF 7

test/bringup/water_main.c

Original lines 194-241 | Code and comments unchanged; long lines wrap visually

trendUpdate (L192-225): Test-only heuristic: compare recent half-window means and spread to label a displayed trend stable/settling.
printRow (L227-253): Present already-collected diagnostic information; this does not make the measurement more valid.

```
194     tr->count = 0;
195     return "-";
196   }
197   if (tr->count > 0 && compensated != tr->compensated) tr->count = 0;
198   if (tr->count == 0) tr->next = 0;
199   tr->compensated = compensated;
200   tr->ppm[tr->next] = ppm;
201   tr->next = (uint8_t)((tr->next + 1) % WATER_TREND_ROWS);
202   if (tr->count < WATER_TREND_ROWS) ++tr->count;
203   if (tr->count < WATER_TREND_ROWS) return "settling";
204
205   // Oldest row is at `next` (the window is full).
206   const uint8_t half = WATER_TREND_ROWS / 2;
207   float older = 0.0f, newer = 0.0f;
208   for (uint8_t i = 0; i < WATER_TREND_ROWS; ++i) {
209     const float v = tr->ppm[(tr->next + i) % WATER_TREND_ROWS];
210     if (i < half) older += v; else newer += v;
211   }
212   older /= half;
213   newer /= half;
214   const float mean = (older + newer) / 2.0f;
215   float drift = mean * WATER_TREND_DRIFT_PCT / 100.0f;
216   if (drift < WATER_TREND_MIN_PPM) drift = WATER_TREND_MIN_PPM;
217   float spread = mean * WATER_TREND_SPREAD_PCT / 100.0f;
218   if (spread < WATER_TREND_MIN_PPM) spread = WATER_TREND_MIN_PPM;
219   if (newer - older > drift || older - newer > drift) return "settling";
220   for (uint8_t i = 0; i < WATER_TREND_ROWS; ++i) {
221     const float d = tr->ppm[i] - mean;
222     if (d > spread || -d > spread) return "settling";
223   }
224   return "stable";
225 }
226
227 static void printRow(MonoMs now, const DriverRun* ds, const Reading* t, const DriverRun* tds,
228                      const Reading* r, Trend* tr) {
229   char water[12], raw[12], uncomp[12], comp[12];
230   waterCell(ds, t, water, sizeof(water));
231
232   const bool tdsOk = tds->ready && r->hasRaw;
233   const char* state = tds->ready ? tdsStateName(r) : "-";
234   if (tdsOk) {
235     snprintf(raw, sizeof(raw), "%ld", (long)r->raw);
236     snprintf(uncomp, sizeof(uncomp), "%.0f", (double)tdsPpmFromModuleMv((float)r->raw,
  > 1.0f));
237   } else {
238     snprintf(raw, sizeof(raw), "-");
239     snprintf(uncomp, sizeof(uncomp), "-");
240   }
241   // value is compensated unless the driver flagged it uncompensated.
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 6 OF 7

test/bringup/water_main.c

Original lines 242-288 | Code and comments unchanged; long lines wrap visually

printRow (L227-253): Present already-collected diagnostic information; this does not make the measurement more valid.
waterTask (L255-316): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.

```
242   const bool compensated = tdsOk && !(r->qualityFlags & kQualityUncompensated);
243   if (compensated) {
244     snprintf(comp, sizeof(comp), "%.0f", (double)r->value);
245   } else {
246     snprintf(comp, sizeof(comp), "-");
247   }
248   const float shown = compensated ? r->value : tdsPpmFromModuleMv((float)r->raw, 1.0f);
249   const char* trend = trendUpdate(tr, tdsOk, compensated, shown);
250
251   printf("%7lu | %7s | %6s | %6s | %6s | %-7s | %-8s |\n", (unsigned long)(now / 1000),
  > water,
252          raw, uncomp, comp, state, trend);
253 }
254
255 static void waterTask(void* arg) {
256   (void)arg;
257   DriverRun ds = {&s_ds.base, false, false, false, DRIVER_OK, 0, 0, 0};
258   DriverRun tds = {&s_tds.base, false, false, false, DRIVER_OK, 0, 0, 0};
259   Trend trend;
260   memset(&trend, 0, sizeof(trend));
261   bool dsTimed = false;
262   bool tdsTimed = false;
263   int lastCompensated = -1;  // unknown
264   uint32_t rows = 0;
265   TickType_t lastWake = xTaskGetTickCount();
266
267   for (;;) {
268     vTaskDelayUntil(&lastWake, pdMS_TO_TICKS(CFG_FRAME_PERIOD_MS));
269     const MonoMs frameStart = monoNowMs();
270
271     // Latest DS18B20 result (previous frame) -> TDS compensation input. A
272     // failed or missing reading is passed as is: the TDS driver rejects it.
273     Reading water;
274     sensorDriverReadResult(&s_ds.base, &water, 1);
275     tdsDriverSetWaterTemperature(&s_tds, &water);
276     const bool fresh = readingIsActionableAt(&water, frameStart, CFG_STALE_ORDINARY_MS);
277     if ((int)fresh != lastCompensated) {
278       printf(fresh ? "# tds compensation ON (from DS18B20)\n"
279                    : "# DS18B20 not fresh -> tds uncompensated (NO TEMP)\n");
280       lastCompensated = fresh;
281     }
282
283     beginRun(&ds, monoNowMs());
284     beginRun(&tds, monoNowMs());
285     while (ds.pending || tds.pending) {
286       MonoMs now = monoNowMs();
287       if (now - frameStart >= CFG_FRAME_PERIOD_MS) {
288         cancelRun(&ds);
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 7 OF 7

test/bringup/water_main.c

Original lines 289-336 | Code and comments unchanged; long lines wrap visually

waterTask (L255-316): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.
setup (L318-334): Establish safe outputs, logger, gate, board check and queues; load configuration; start Safety, both acquisition workers, power, radios and supervision.

```
289         cancelRun(&tds);
290         break;
291       }
292       stepRun(&ds, now);
293       stepRun(&tds, monoNowMs());
294
295       // Sleep until the earlier of the two next polls.
296       now = monoNowMs();
297       MonoMs wake = now + 1;
298       bool any = false;
299       if (ds.pending) { wake = ds.nextPollMs; any = true; }
300       if (tds.pending && (!any || tds.nextPollMs < wake)) wake = tds.nextPollMs;
301       const uint32_t waitMs = wake > now ? (uint32_t)(wake - now) : 1;
302       const TickType_t ticks = pdMS_TO_TICKS(waitMs);
303       vTaskDelay(ticks > 0 ? ticks : 1);
304     }
305
306     printRomIfNew();
307     printTimingOnce(&ds, &dsTimed);
308     printTimingOnce(&tds, &tdsTimed);
309     Reading t;
310     Reading ppm;
311     sensorDriverReadResult(&s_ds.base, &t, 1);
312     sensorDriverReadResult(&s_tds.base, &ppm, 1);
313     if (rows++ % WATER_HEADER_EVERY == 0) printHeader();
314     printRow(monoNowMs(), &ds, &t, &tds, &ppm, &trend);
315   }
316 }
317
318 void setup(void) {
319   ds18b20DriverConstruct(&s_ds, &s_dsConfig);
320   tdsDriverConstruct(&s_tds, &s_tdsConfig);
321   delay(200);
322   printf("\n# water bring-up: DS18B20 GPIO%d + TDS GPIO%d, %lu ms frame\n",
323          (int)BOARD_PIN_ONE_WIRE, (int)BOARD_PIN_TDS_ADC, (unsigned
  >
long)CFG_FRAME_PERIOD_MS);
324   if (!oneWireBusBegin(&s_oneWire, BOARD_PIN_ONE_WIRE)) {
325     printf("# GPIO%d cannot be used for 1-Wire\n", (int)BOARD_PIN_ONE_WIRE);
326   }
327   if (!adcChannelBegin(&s_adc, BOARD_PIN_TDS_ADC)) {
328     printf("# GPIO%d is not an ADC1 pin\n", (int)BOARD_PIN_TDS_ADC);
329   }
330
331   const TaskSpec* spec = cfgTaskSpec(TASK_ID_ACQUISITION_NORMAL);
332   xTaskCreatePinnedToCore(waterTask, "water", spec->stackBytes, NULL, spec->priority, NULL,
333                           spec->core);
334 }
335
336 void loop(void) { vTaskDelay(portMAX_DELAY); }
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 1 OF 16

test/native/test_domain/test_main.c

Original lines 1-49 | Code and comments unchanged; long lines wrap visually

setUp (L22-22): Unity test hook; the original body is empty.
tearDown (L23-23): Unity test hook; the original body is empty.

```
1  // Host unit tests for src/domain (pio test -e native). Architecture §17 stage 4.
2  // Domain code must stay free of Arduino/FreeRTOS so these run on a PC.
3  #include <unity.h>
4
5  #include "app_config.h"
6  #include "board_profile.h"
7  #include <math.h>
8
9  #include "domain/alarm.h"
10 #include "domain/alarm_sticky.h"
11 #include "domain/calibration.h"
12 #include "domain/ds18b20_codec.h"
13 #include "domain/lamp_control.h"
14 #include "domain/reading.h"
15 #include "domain/safety_core.h"
16 #include "domain/schedule.h"
17 #include "domain/sensor_set.h"
18 #include "domain/sgp30_codec.h"
19 #include "domain/tds_codec.h"
20 #include "domain/ultrasonic.h"
21
22 void setUp(void) {}
23 void tearDown(void) {}
24
25 static void test_reset_reading_is_starting_and_not_actionable(void) {
26   Reading r;
27   readingReset(&r, SENSOR_BME280, METRIC_AIR_TEMPERATURE, UNIT_DEG_C);
28   TEST_ASSERT_TRUE(r.health == HEALTH_STARTING);
29   TEST_ASSERT_TRUE(r.sensor == SENSOR_BME280);
30   TEST_ASSERT_FALSE(readingIsActionable(&r));
31 }
32
33 static void test_ok_reading_becomes_stale_after_limit(void) {
34   Reading r;
35   readingReset(&r, SENSOR_DS18B20, METRIC_WATER_TEMPERATURE, UNIT_DEG_C);
36   r.health = HEALTH_OK;
37   r.intervalEndMs = 1000;
38   TEST_ASSERT_TRUE(readingHealthAt(&r, 2000, 1000) == HEALTH_OK);
39   TEST_ASSERT_TRUE(readingHealthAt(&r, 2001, 1000) == HEALTH_STALE);
40 }
41
42 static void test_stale_reading_is_not_actionable(void) {
43   Reading r;
44   readingReset(&r, SENSOR_PH, METRIC_PH, UNIT_PH);
45   r.health = HEALTH_OK;
46   r.intervalEndMs = 1000;
47   TEST_ASSERT_TRUE(readingIsActionableAt(&r, 4000, 3000));   // exactly at the limit
48   TEST_ASSERT_FALSE(readingIsActionableAt(&r, 4001, 3000));  // stale
49   TEST_ASSERT_TRUE(readingIsActionable(&r));                 // health-only check is
 > unchanged
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 2 OF 16

test/native/test_domain/test_main.c

Original lines 50-98 | Code and comments unchanged; long lines wrap visually

test_stale_reading_is_not_actionable (L42-52): Native test: construct a fixed case and assert the named expected behaviour.
test_fault_is_not_masked_by_staleness (L54-59): Native test: construct a fixed case and assert the named expected behaviour.

```
  r.health = HEALTH_UNCALIBRATED;
  TEST_ASSERT_FALSE(readingIsActionableAt(&r, 1000, 3000));  // fresh but not OK
}

static void test_fault_is_not_masked_by_staleness(void) {
  Reading r;
  readingReset(&r, SENSOR_PH, METRIC_PH, UNIT_PH);
  r.health = HEALTH_UNPOWERED;
  TEST_ASSERT_TRUE(readingHealthAt(&r, 100000, 1000) == HEALTH_UNPOWERED);
}

static void test_divider_recovery_uses_nominal_ratios(void) {
  // Power sense: 220/440 => x660/440. 3333 mV at GPIO8 -> 5000 mV external feed.
  TEST_ASSERT_FLOAT_WITHIN(0.5f, 4999.5f,
                           dividerRecoverMv(3333.0f, BOARD_POWER_SENSE_DIVIDER_TOP_OHM,
                                            BOARD_POWER_SENSE_DIVIDER_BOTTOM_OHM));
  // pH: 10k/15k => x25/15. 3000 mV at the pin -> 5000 mV at the module.
  TEST_ASSERT_FLOAT_WITHIN(0.5f, 5000.0f,
                           dividerRecoverMv(3000.0f, BOARD_PH_DIVIDER_TOP_OHM,
                                            BOARD_PH_DIVIDER_BOTTOM_OHM));
}

static void test_two_point_fit_maps_buffers(void) {
  // Module mV in pH 6.86 and 4.01 buffers, about -5.7 pH/V (tutorial).
  LinearCalibration c;
  TEST_ASSERT_TRUE(linearFromTwoPoints(2500.0f, 6.86f, 3000.0f, 4.01f, 1, &c));
  TEST_ASSERT_TRUE(c.valid);
  TEST_ASSERT_FLOAT_WITHIN(1e-6f, -0.0057f, c.slope);
  TEST_ASSERT_FLOAT_WITHIN(0.001f, 6.86f, linearApply(&c, 2500.0f));
  TEST_ASSERT_FLOAT_WITHIN(0.001f, 4.01f, linearApply(&c, 3000.0f));
  TEST_ASSERT_FALSE(linearFromTwoPoints(2500.0f, 6.86f, 2500.0f, 4.01f, 2, &c));
  TEST_ASSERT_FALSE(c.valid);
}

static void test_trimmed_mean_rejects_spikes(void) {
  uint32_t s[10] = {1500, 4000, 1502, 1498, 0, 1501, 1499, 1500, 3100, 1500};
  TEST_ASSERT_FLOAT_WITHIN(0.01f, 1500.333f, trimmedMeanU32(s, 10, 2));  // 1499..1502
  TEST_ASSERT_TRUE(s[0] == 0 && s[9] == 4000);  // sorted in place
  uint32_t few[4] = {1, 2, 3, 4};
  TEST_ASSERT_FLOAT_WITHIN(1e-6f, 0.0f, trimmedMeanU32(few, 4, 2));
}

static void test_snapshot_reset_indexes_by_metric(void) {
  Snapshot s;
  snapshotReset(&s);
  for (uint8_t i = 0; i < METRIC_COUNT; ++i) {
    TEST_ASSERT_TRUE(s.readings[i].metric == (MetricId)i);
    TEST_ASSERT_TRUE(s.readings[i].health == HEALTH_STARTING);
  }
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 3 OF 16

test/native/test_domain/test_main.c

Original lines 99-145 | Code and comments unchanged; long lines wrap visually

test_snapshot_reset_indexes_by_metric (L92-99): Native test: construct a fixed case and assert the named expected behaviour.
test_production_ready_requires_power_sense (L101-104): Native test: construct a fixed case and assert the named expected behaviour.

```
99  }
100
101 static void test_production_ready_requires_power_sense(void) {
102   BoardValidation v = boardProfileValidate();
103   TEST_ASSERT_EQUAL(v.powerSenseAssigned && BOARD_POWER_SENSE_VERIFIED, v.productionReady);
104 }
105
106 static void test_snapshot_reset_sets_identity(void) {
107   Snapshot s;
108   snapshotReset(&s);
109   TEST_ASSERT_EQUAL_UINT8(SENSOR_BME280, s.readings[METRIC_AIR_HUMIDITY].sensor);
110   TEST_ASSERT_EQUAL_UINT8(UNIT_PERCENT_RH, s.readings[METRIC_AIR_HUMIDITY].unit);
111   TEST_ASSERT_EQUAL_UINT8(SENSOR_DS18B20, s.readings[METRIC_WATER_TEMPERATURE].sensor);
112   TEST_ASSERT_EQUAL_UINT8(SENSOR_LIGHT, s.readings[METRIC_LIGHT_LEVEL].sensor);
113   TEST_ASSERT_EQUAL_UINT8(SENSOR_SGP30, s.readings[METRIC_CO2EQ].sensor);
114   TEST_ASSERT_EQUAL_UINT8(UNIT_PPB, s.readings[METRIC_TVOC].unit);
115   for (uint8_t i = 0; i < METRIC_COUNT; ++i) {
116     TEST_ASSERT_TRUE(s.readings[i].sensor < SENSOR_COUNT);
117   }
118 }
119
120 static void test_alarm_rule_ignores_stale_sample(void) {
121   AlarmRule rule = {0};
122   rule.metric = METRIC_AIR_TEMPERATURE;
123   AlarmRuleState state = {0};
124   Reading r;
125   readingReset(&r, SENSOR_BME280, METRIC_AIR_TEMPERATURE, UNIT_DEG_C);
126   r.health = HEALTH_OK;
127   r.value = 50.0f;
128   r.intervalEndMs = 1000;
129   TEST_ASSERT_EQUAL_UINT8(RULE_DECISION_NOT_ELIGIBLE, evaluateRule(&rule, &r, 2001, 1000,
>  &state));
130   TEST_ASSERT_TRUE(evaluateRule(&rule, &r, 1500, 1000, &state) !=
>  RULE_DECISION_NOT_ELIGIBLE);
131 }
132
133 static void test_sgp30_crc_matches_datasheet_vector(void) {
134   const uint8_t beef[2] = {0xBE, 0xEF};
135   TEST_ASSERT_EQUAL_HEX8(0x92, sgp30Crc8(beef, 2));
136 }
137
138 static void test_sgp30_words_round_trip_and_reject_bad_crc(void) {
139   uint8_t buf[6];
140   sgp30PackWord(0x01A3, &buf[0]);  // CO2eq 419 ppm
141   sgp30PackWord(0x0007, &buf[3]);  // TVOC 7 ppb
142   uint16_t w[2] = {0, 0};
143   TEST_ASSERT_TRUE(sgp30UnpackWords(buf, 2, w));
144   TEST_ASSERT_EQUAL_HEX16(0x01A3, w[0]);
145   TEST_ASSERT_EQUAL_HEX16(0x0007, w[1]);
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 4 OF 16

test/native/test_domain/test_main.c

Original lines 146-193 | Code and comments unchanged; long lines wrap visually

test_sgp30_words_round_trip_and_reject_bad_crc (L138-150): Native test: construct a fixed case and assert the named expected behaviour.
test_sgp30_humidity_fixed_point (L152-161): Native test: construct a fixed case and assert the named expected behaviour.

```
146   buf[4] ^= 0x01;  // one flipped bit in the second word
147   TEST_ASSERT_FALSE(sgp30UnpackWords(buf, 2, w));
148   const uint8_t floating[3] = {0xFF, 0xFF, 0xFF};  // released bus
149   TEST_ASSERT_FALSE(sgp30UnpackWords(floating, 1, w));
150 }
151
152 static void test_sgp30_humidity_fixed_point(void) {
153   TEST_ASSERT_EQUAL_HEX16(0x0F80, sgp30GramsToFixed8p8(15.5f));  // datasheet example
154   TEST_ASSERT_EQUAL_HEX16(0x0000, sgp30GramsToFixed8p8(0.0f));
155   TEST_ASSERT_EQUAL_HEX16(0x0001, sgp30GramsToFixed8p8(0.001f));
156   TEST_ASSERT_EQUAL_HEX16(0xFFFF, sgp30GramsToFixed8p8(300.0f));
157   // The datasheet formula gives 11.48 g/m3 at 25 degC / 50 %RH (its text rounds
158   // this example to 11.8).
159   TEST_ASSERT_FLOAT_WITHIN(0.02f, 11.48f, sgp30AbsoluteHumidity(25.0f, 50.0f));
160   TEST_ASSERT_TRUE(sgp30AbsoluteHumidity(25.0f, 120.0f) < 0.0f);
161 }
162
163 static void test_ultrasonic_echo_width_to_mm(void) {
164   // 58.3 us per cm at 343 m/s (HC-SR04 guide, equation 3).
165   TEST_ASSERT_FLOAT_WITHIN(0.1f, 1000.0f,
166                            ultrasonicEchoUsToMm(5831, ULTRASONIC_NOMINAL_SPEED_MPS));
167   TEST_ASSERT_FLOAT_WITHIN(0.1f, 20.0f,
168                            ultrasonicEchoUsToMm(117, ULTRASONIC_NOMINAL_SPEED_MPS));
169   TEST_ASSERT_FLOAT_WITHIN(1e-6f, 0.0f, ultrasonicEchoUsToMm(0,
  > ULTRASONIC_NOMINAL_SPEED_MPS));
170 }
171
172 static void test_ultrasonic_range_is_2_to_400_cm(void) {
173   TEST_ASSERT_TRUE(ultrasonicInRangeMm(20.0f));
174   TEST_ASSERT_TRUE(ultrasonicInRangeMm(4000.0f));
175   TEST_ASSERT_FALSE(ultrasonicInRangeMm(19.9f));
176   TEST_ASSERT_FALSE(ultrasonicInRangeMm(4000.1f));
177   TEST_ASSERT_FALSE(ultrasonicInRangeMm(0.0f));  // never a valid distance
178 }
179
180 static void test_tds_datasheet_curve(void) {
181   // (133.42 - 255.86 + 857.39) * 0.5 at 1.0 V, 25 degC (SEN0244 sample code).
182   TEST_ASSERT_FLOAT_WITHIN(1e-6f, 1.0f, tdsCompensationCoefficient(25.0f));
183   TEST_ASSERT_FLOAT_WITHIN(0.01f, 367.475f, tdsPpmFromModuleMv(1000.0f, 1.0f));
184   TEST_ASSERT_FLOAT_WITHIN(1e-6f, 0.0f, tdsPpmFromModuleMv(0.0f, 1.0f));
185   TEST_ASSERT_FLOAT_WITHIN(1e-6f, 0.0f, tdsPpmFromModuleMv(1000.0f, 0.0f));
186   // 2.3 V (top of the output range) is above the rated 1000 ppm.
187   TEST_ASSERT_FLOAT_WITHIN(0.5f, 1120.9f, tdsPpmFromModuleMv(2300.0f, 1.0f));
188   TEST_ASSERT_TRUE(tdsPpmFromModuleMv(2300.0f, 1.0f) > TDS_FULL_SCALE_PPM);
189 }
190
191 static void test_tds_temperature_compensation(void) {
192   // The conductivity (ppm) is compensated, not the voltage (GravityTDS).
193   const float coefficient = tdsCompensationCoefficient(35.0f);
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 5 OF 16

test/native/test_domain/test_main.c

Original lines 194-242 | Code and comments unchanged; long lines wrap visually

test_tds_temperature_compensation (L191-200): Native test: construct a fixed case and assert the named expected behaviour.
makeScratchpad (L203-208): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.

```
194   TEST_ASSERT_FLOAT_WITHIN(1e-6f, 1.2f, coefficient);
195   TEST_ASSERT_FLOAT_WITHIN(0.01f, 367.475f / 1.2f, tdsPpmFromModuleMv(1000.0f, coefficient));
196   // Measured 24 Oct 2026, same tap water cold and warm: within ~2 % at 25 degC.
197   const float cold = tdsPpmFromModuleMv(1234.0f, tdsCompensationCoefficient(11.44f));
198   const float warm = tdsPpmFromModuleMv(1768.0f, tdsCompensationCoefficient(33.62f));
199   TEST_ASSERT_FLOAT_WITHIN(0.03f * cold, cold, warm);
200 }
201
202 // Scratchpad with the given temperature register and 12-bit config, valid CRC.
203 static void makeScratchpad(int16_t raw, uint8_t sp[DS18B20_SCRATCHPAD_LEN]) {
204   const uint8_t base[8] = {(uint8_t)(raw & 0xFF), (uint8_t)((uint16_t)raw >> 8), 0x4B, 0x46,
205                            DS18B20_CONFIG_12BIT, 0xFF, 0x0C, 0x10};
206   for (uint8_t i = 0; i < 8; ++i) sp[i] = base[i];
207   sp[8] = oneWireCrc8(sp, 8);
208 }
209
210 static void test_onewire_crc_matches_maxim_vector(void) {
211   // Maxim application note 27: ROM 02 1C B8 01 00 00 00 has CRC A2.
212   const uint8_t rom[8] = {0x02, 0x1C, 0xB8, 0x01, 0x00, 0x00, 0x00, 0xA2};
213   TEST_ASSERT_EQUAL_HEX8(0xA2, oneWireCrc8(rom, 7));
214   TEST_ASSERT_EQUAL_HEX8(0x00, oneWireCrc8(rom, 8));  // data + CRC -> 0
215   TEST_ASSERT_FALSE(ds18b20RomValid(rom));  // family 02h, not a DS18B20
216   uint8_t ds[8] = {0x28, 0xFF, 0x64, 0x1E, 0x0F, 0x4A, 0x3C, 0x00};
217   ds[7] = oneWireCrc8(ds, 7);
218   TEST_ASSERT_TRUE(ds18b20RomValid(ds));
219   ds[3] ^= 0x01;
220   TEST_ASSERT_FALSE(ds18b20RomValid(ds));
221   const uint8_t zero[8] = {0};  // bus stuck low: CRC 0 matches, family does not
222   TEST_ASSERT_FALSE(ds18b20RomValid(zero));
223 }
224
225 static void test_ds18b20_temperature_table(void) {
226   // Datasheet Table 1.
227   uint8_t sp[DS18B20_SCRATCHPAD_LEN];
228   makeScratchpad(0x0191, sp);
229   TEST_ASSERT_TRUE(ds18b20ScratchpadValid(sp));
230   TEST_ASSERT_FLOAT_WITHIN(1e-6f, 25.0625f, ds18b20RawToDegC(ds18b20RawTemperature(sp)));
231   makeScratchpad((int16_t)0xFF5E, sp);
232   TEST_ASSERT_FLOAT_WITHIN(1e-6f, -10.125f, ds18b20RawToDegC(ds18b20RawTemperature(sp)));
233   makeScratchpad((int16_t)0xFC90, sp);
234   TEST_ASSERT_FLOAT_WITHIN(1e-6f, -55.0f, ds18b20RawToDegC(ds18b20RawTemperature(sp)));
235   makeScratchpad(0x07D0, sp);
236   TEST_ASSERT_FLOAT_WITHIN(1e-6f, 125.0f, ds18b20RawToDegC(ds18b20RawTemperature(sp)));
237   makeScratchpad(DS18B20_POWER_ON_RAW, sp);
238   TEST_ASSERT_FLOAT_WITHIN(1e-6f, 85.0f, ds18b20RawToDegC(ds18b20RawTemperature(sp)));
239 }
240
241 static void test_ds18b20_scratchpad_checks(void) {
242   uint8_t sp[DS18B20_SCRATCHPAD_LEN];
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 6 OF 16

test/native/test_domain/test_main.c

Original lines 243-291 | Code and comments unchanged; long lines wrap visually

test_ds18b20_scratchpad_checks (L241-254): Native test: construct a fixed case and assert the named expected behaviour.
lampLight (L258-264): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.

```
243   makeScratchpad(0x0191, sp);
244   sp[0] ^= 0x01;  // one bit flipped on the wire
245   TEST_ASSERT_FALSE(ds18b20ScratchpadValid(sp));
246   const uint8_t zero[DS18B20_SCRATCHPAD_LEN] = {0};  // CRC 0 matches, config does not
247   TEST_ASSERT_FALSE(ds18b20ScratchpadValid(zero));
248   uint8_t ff[DS18B20_SCRATCHPAD_LEN];  // floating / no device
249   for (uint8_t i = 0; i < DS18B20_SCRATCHPAD_LEN; ++i) ff[i] = 0xFF;
250   TEST_ASSERT_FALSE(ds18b20ScratchpadValid(ff));
251   TEST_ASSERT_TRUE(ds18b20Is12Bit(0x7F));
252   TEST_ASSERT_FALSE(ds18b20Is12Bit(0x5F));  // 11-bit
253   TEST_ASSERT_FALSE(ds18b20Is12Bit(0x1F));  // 9-bit
254 }
255
256 static const LampControlConfig kLampTest = {40.0f, 120.0f, 30000, 30000, 3};
257
258 static Reading lampLight(float mv, Health health) {
259   Reading r;
260   readingReset(&r, SENSOR_LIGHT, METRIC_LIGHT_LEVEL, UNIT_MILLIVOLT);
261   r.health = health;
262   r.value = mv;
263   return r;
264 }
265
266 static void test_lamp_hysteresis_band(void) {
267   LampControl c;
268   lampControlInit(&c, &kLampTest, 0);
269   Reading r = lampLight(80.0f, HEALTH_UNCALIBRATED);  // inside the band: stays OFF
270   TEST_ASSERT_FALSE(lampControlUpdate(&c, &r, 60000));
271   r.value = 39.0f;
272   TEST_ASSERT_TRUE(lampControlUpdate(&c, &r, 61000));
273   r.value = 119.0f;  // inside the band: stays ON
274   TEST_ASSERT_TRUE(lampControlUpdate(&c, &r, 200000));
275   r.value = 121.0f;
276   TEST_ASSERT_FALSE(lampControlUpdate(&c, &r, 201000));
277 }
278
279 // After an on-device lux fit the value is lux; the thresholds stay pin mV (raw).
280 static void test_lamp_uses_pin_mv_after_lux_fit(void) {
281   LampControl c;
282   lampControlInit(&c, &kLampTest, 0);
283   Reading r = lampLight(10.0f, HEALTH_OK);  // 10 mV at the pin: dark
284   r.unit = UNIT_LUX;
285   r.value = 500.0f;  // would be "bright" if read as mV
286   r.hasRaw = true;
287   r.raw = 10;
288   TEST_ASSERT_TRUE(lampControlUpdate(&c, &r, 60000));
289 }
290
291 static void test_lamp_dwell_holds_changes(void) {
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 7 OF 16

test/native/test_domain/test_main.c

Original lines 292-339 | Code and comments unchanged; long lines wrap visually

test_lamp_dwell_holds_changes (L291-302): Native test: construct a fixed case and assert the named expected behaviour.
test_lamp_faults_hold_then_force_off (L304-316): Native test: construct a fixed case and assert the named expected behaviour.

```
292   LampControl c;
293   lampControlInit(&c, &kLampTest, 0);
294   Reading r = lampLight(10.0f, HEALTH_OK);
295   TEST_ASSERT_FALSE(lampControlUpdate(&c, &r, 29999));  // boot counts as OFF
296   TEST_ASSERT_TRUE(c.waiting);
297   TEST_ASSERT_TRUE(lampControlUpdate(&c, &r, 30000));
298   TEST_ASSERT_FALSE(c.waiting);
299   r.value = 500.0f;
300   TEST_ASSERT_TRUE(lampControlUpdate(&c, &r, 59999));
301   TEST_ASSERT_FALSE(lampControlUpdate(&c, &r, 60000));
302 }
303
304 static void test_lamp_faults_hold_then_force_off(void) {
305   LampControl c;
306   lampControlInit(&c, &kLampTest, 0);
307   Reading r = lampLight(10.0f, HEALTH_OK);
308   TEST_ASSERT_TRUE(lampControlUpdate(&c, &r, 30000));
309   Reading bad = lampLight(0.0f, HEALTH_FAULT);
310   TEST_ASSERT_TRUE(lampControlUpdate(&c, &bad, 31000));
311   TEST_ASSERT_TRUE(lampControlUpdate(&c, &bad, 32000));
312   TEST_ASSERT_TRUE(lampControlUpdate(&c, &r, 33000));  // a good reading resets the count
313   TEST_ASSERT_TRUE(lampControlUpdate(&c, &bad, 34000));
314   TEST_ASSERT_TRUE(lampControlUpdate(&c, &bad, 35000));
315   TEST_ASSERT_FALSE(lampControlUpdate(&c, &bad, 36000));  // 3rd in a row, no dwell
316 }
317
318 static void test_lamp_saturated_counts_as_bright(void) {
319   LampControl c;
320   lampControlInit(&c, &kLampTest, 0);
321   Reading r = lampLight(10.0f, HEALTH_OK);
322   TEST_ASSERT_TRUE(lampControlUpdate(&c, &r, 30000));
323   r.value = 20.0f;  // a clipped value is only a lower bound
324   r.qualityFlags = kQualitySaturated;
325   TEST_ASSERT_FALSE(lampControlUpdate(&c, &r, 60000));
326 }
327
328 // ---- alarm rules (§12) ------------------------------------------------------
329
330 static const AlarmRule kHotRule = {.ruleId = 0, .enabled = true, .metric =
  > METRIC_AIR_TEMPERATURE,
331                                    .comparison = COMPARISON_ABOVE, .tripThreshold = 40.0f,
332                                    .clearThreshold = 37.0f, .persistMs = 0,
333                                    .severity = SEVERITY_CRITICAL, .latching = true,
334                                    .actuatorId = ACTUATOR_RELAY1,
335                                    .action = ACTUATOR_ACTION_DEENERGIZE};
336
337 static Reading airTemp(float degC, MonoMs endMs) {
338   Reading r;
339   readingReset(&r, SENSOR_BME280, METRIC_AIR_TEMPERATURE, UNIT_DEG_C);
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 8 OF 16

test/native/test_domain/test_main.c

Original lines 340-385 | Code and comments unchanged; long lines wrap visually

airTemp (L337-344): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.
evalAt (L346-349): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.

```
  r.health = HEALTH_OK;
  r.value = degC;
  r.intervalEndMs = endMs;
  return r;
}

static RuleDecision evalAt(const AlarmRule* rule, float v, MonoMs t, AlarmRuleState* st) {
  Reading r = airTemp(v, t);
  return evaluateRule(rule, &r, t, 1000, st);
}

static void test_alarm_trips_on_one_sample_and_clears_with_hysteresis(void) {
  AlarmRuleState st = {0};
  TEST_ASSERT_EQUAL_UINT8(RULE_DECISION_NO_CHANGE, evalAt(&kHotRule, 39.9f, 250, &st));
  TEST_ASSERT_EQUAL_UINT8(RULE_DECISION_TRIP, evalAt(&kHotRule, 40.0f, 500, &st));
  TEST_ASSERT_EQUAL_UINT8(RULE_DECISION_NO_CHANGE, evalAt(&kHotRule, 45.0f, 750, &st));
  TEST_ASSERT_EQUAL_UINT8(RULE_DECISION_NO_CHANGE, evalAt(&kHotRule, 38.0f, 1000, &st));  //
> in band
  TEST_ASSERT_EQUAL_UINT8(RULE_DECISION_CLEAR, evalAt(&kHotRule, 37.0f, 1250, &st));
  TEST_ASSERT_EQUAL_UINT8(RULE_DECISION_NO_CHANGE, evalAt(&kHotRule, 39.0f, 1500, &st));  //
> no chatter
}

static void test_alarm_persistence_needs_sustained_valid_samples(void) {
  AlarmRule rule = kHotRule;
  rule.persistMs = 1000;
  AlarmRuleState st = {0};
  TEST_ASSERT_EQUAL_UINT8(RULE_DECISION_NO_CHANGE, evalAt(&rule, 41.0f, 1000, &st));
  TEST_ASSERT_EQUAL_UINT8(RULE_DECISION_NO_CHANGE, evalAt(&rule, 41.0f, 1999, &st));
  TEST_ASSERT_EQUAL_UINT8(RULE_DECISION_TRIP, evalAt(&rule, 41.0f, 2000, &st));

  AlarmRuleState st2 = {0};
  TEST_ASSERT_EQUAL_UINT8(RULE_DECISION_NO_CHANGE, evalAt(&rule, 41.0f, 1000, &st2));
  Reading bad = airTemp(41.0f, 1500);
  bad.health = HEALTH_FAULT;  // a gap restarts the window
  TEST_ASSERT_EQUAL_UINT8(RULE_DECISION_NOT_ELIGIBLE, evaluateRule(&rule, &bad, 1500, 1000,
> &st2));
  TEST_ASSERT_EQUAL_UINT8(RULE_DECISION_NO_CHANGE, evalAt(&rule, 41.0f, 2000, &st2));
  TEST_ASSERT_EQUAL_UINT8(RULE_DECISION_TRIP, evalAt(&rule, 41.0f, 3000, &st2));
}

static void test_alarm_below_rule_mirrors_above(void) {
  AlarmRule rule = kHotRule;
  rule.comparison = COMPARISON_BELOW;
  rule.tripThreshold = 5.0f;
  rule.clearThreshold = 8.0f;
  AlarmRuleState st = {0};
  TEST_ASSERT_EQUAL_UINT8(RULE_DECISION_TRIP, evalAt(&rule, 4.0f, 100, &st));
  TEST_ASSERT_EQUAL_UINT8(RULE_DECISION_NO_CHANGE, evalAt(&rule, 7.0f, 200, &st));
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 9 OF 16

test/native/test_domain/test_main.c

Original lines 386-434 | Code and comments unchanged; long lines wrap visually

test_alarm_below_rule_mirrors_above (L378-387): Native test: construct a fixed case and assert the named expected behaviour.
test_alarm_stale_limits_per_metric (L389-393): Native test: construct a fixed case and assert the named expected behaviour.

```
386   TEST_ASSERT_EQUAL_UINT8(RULE_DECISION_CLEAR, evalAt(&rule, 8.5f, 300, &st));
387 }
388
389 static void test_alarm_stale_limits_per_metric(void) {
390   TEST_ASSERT_EQUAL_UINT32(CFG_STALE_BME280_MS, alarmStaleLimitMs(METRIC_AIR_HUMIDITY));
391   TEST_ASSERT_EQUAL_UINT32(CFG_STALE_ORDINARY_MS, alarmStaleLimitMs(METRIC_LIGHT_LEVEL));
392   TEST_ASSERT_EQUAL_UINT32(CFG_STALE_ORDINARY_MS, alarmStaleLimitMs(METRIC_TVOC));
393 }
394
395 // ---- schedule (§9) ------------------------------------------------------
396
397 static void test_schedule_bme280_runs_four_times_per_frame(void) {
398   ScheduleJob jobs[2];
399   scheduleJobInit(&jobs[0], CFG_FRAME_PERIOD_MS, 20, 0);
400   scheduleJobInit(&jobs[1], CFG_BME280_PERIOD_MS, CFG_BME280_BUDGET_MS, 0);
401   uint32_t frameRuns = 0, bmeRuns = 0;
402   MonoMs now = 0;
403   while (now < 1000) {
404     if (scheduleJobDue(&jobs[0], now)) { scheduleJobMarkRun(&jobs[0], now); ++frameRuns; }
405     if (scheduleJobDue(&jobs[1], now)) { scheduleJobMarkRun(&jobs[1], now); ++bmeRuns; }
406     now = scheduleEarliestDue(jobs, 2);
407   }
408   TEST_ASSERT_EQUAL_UINT32(1, frameRuns);
409   TEST_ASSERT_EQUAL_UINT32(4, bmeRuns);
410   TEST_ASSERT_EQUAL_UINT32(0, jobs[1].late);
411   TEST_ASSERT_TRUE(jobs[1].nextDueMs == 1000);
412 }
413
414 static void test_schedule_skips_missed_slots_without_burst(void) {
415   ScheduleJob j;
416   scheduleJobInit(&j, 250, 12, 500);
417   scheduleJobMarkRun(&j, 530);  // 30 ms late: keeps the 250 ms grid
418   TEST_ASSERT_TRUE(j.nextDueMs == 750);
419   TEST_ASSERT_EQUAL_UINT32(1, j.late);
420   TEST_ASSERT_EQUAL_UINT32(0, j.skipped);
421   TEST_ASSERT_EQUAL_UINT32(30, j.maxLateMs);
422
423   scheduleJobMarkRun(&j, 3760);  // stalled ~3 s: one run, the rest skipped
424   TEST_ASSERT_TRUE(j.nextDueMs == 4000);
425   TEST_ASSERT_EQUAL_UINT32(12, j.skipped);
426   TEST_ASSERT_FALSE(scheduleJobDue(&j, 3761));
427   TEST_ASSERT_EQUAL_UINT32(2, j.runs);
428 }
429
430 static void test_schedule_jitter_within_tolerance_is_not_late(void) {
431   ScheduleJob j;
432   scheduleJobInit(&j, 1000, 10, 1000);
433   scheduleJobMarkRun(&j, 1000 + SCHEDULE_LATE_TOLERANCE_MS);
434   TEST_ASSERT_EQUAL_UINT32(0, j.late);
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 10 OF 16

test/native/test_domain/test_main.c

Original lines 435-484 | Code and comments unchanged; long lines wrap visually

test_schedule_jitter_within_tolerance_is_not_late (L430-437): Native test: construct a fixed case and assert the named expected behaviour.
test_schedule_defers_step_crossing_bme280_slot (L439-443): Native test: construct a fixed case and assert the named expected behaviour.

```
435   TEST_ASSERT_EQUAL_UINT32(SCHEDULE_LATE_TOLERANCE_MS, j.lastLateMs);
436   TEST_ASSERT_TRUE(j.nextDueMs == 2000);
437 }
438
439 static void test_schedule_defers_step_crossing_bme280_slot(void) {
440   TEST_ASSERT_FALSE(scheduleFitsBefore(230, 50, 250));  // DS18B20 init ~50 ms
441   TEST_ASSERT_TRUE(scheduleFitsBefore(230, 19, 250));
442   TEST_ASSERT_TRUE(scheduleFitsBefore(262, 50, 500));
443 }
444
445 // ---- safety core (§12) ------------------------------------------------------
446
447 static const SafetyConfig kSafetyTest = {true, {40.0f, 120.0f, 30000, 30000, 3}, 3000};
448
449 static void setLight(Snapshot* s, float mv, Health h, MonoMs endMs) {
450   Reading* r = &s->readings[METRIC_LIGHT_LEVEL];
451   r->health = h;
452   r->value = mv;
453   r->intervalEndMs = endMs;
454   ++r->sequence;
455 }
456
457 static SafetyOutputs step(SafetyCore* c, const Snapshot* s, MonoMs now, const AlarmEvent* ev,
458                           uint8_t n) {
459   SafetyInputs in = {s, ev, n, now};
460   SafetyOutputs out;
461   safetyCoreStep(c, &in, &out);
462   return out;
463 }
464
465 // Dark from t=1 s, one light result and one 25 degC air temperature per
466 // second (the rules' input stays fresh); returns the core at t=31 s with the
467 // lamp ON (boot dwell 30 s).
468 static void lampOnAt31s(SafetyCore* c, Snapshot* s, const AlarmRule* rules, uint8_t n) {
469   safetyCoreInit(c, &kSafetyTest, rules, n, 0);
470   snapshotReset(s);
471   for (MonoMs t = 1000; t <= 31000; t += 1000) {
472     setLight(s, 10.0f, HEALTH_UNCALIBRATED, t);
473     s->readings[METRIC_AIR_TEMPERATURE] = airTemp(25.0f, t);
474     step(c, s, t, NULL, 0);
475   }
476 }
477
478 static void test_safety_boot_off_until_data_and_dwell(void) {
479   SafetyCore c;
480   safetyCoreInit(&c, &kSafetyTest, NULL, 0, 0);
481   SafetyOutputs o = step(&c, NULL, 100, NULL, 0);
482   TEST_ASSERT_FALSE(o.relayOn);
483   TEST_ASSERT_EQUAL_UINT8(LAMP_REASON_NO_DATA, o.lampReason);
484
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 11 OF 16

test/native/test_domain/test_main.c

Original lines 485-533 | Code and comments unchanged; long lines wrap visually

test_safety_boot_off_until_data_and_dwell (L478-495): Native test: construct a fixed case and assert the named expected behaviour.
test_safety_rail_down_forces_relay_off (L499-513): Native test: construct a fixed case and assert the named expected behaviour.

```
485   Snapshot s;
486   snapshotReset(&s);
487   o = step(&c, &s, 200, NULL, 0);  // published, but no light result yet
488   TEST_ASSERT_FALSE(o.relayOn);
489   TEST_ASSERT_EQUAL_UINT8(LAMP_REASON_NO_DATA, o.lampReason);
490
491   setLight(&s, 10.0f, HEALTH_UNCALIBRATED, 1000);  // dark, boot dwell running
492   o = step(&c, &s, 1000, NULL, 0);
493   TEST_ASSERT_FALSE(o.relayOn);
494   TEST_ASSERT_EQUAL_UINT8(LAMP_REASON_DWELL, o.lampReason);
495 }
496
497 // WP9: the relay coil is on the external 5 V. Rail down = relay OFF, and the
498 // lamp restarts from OFF (with its dwell) once the rail is back.
499 static void test_safety_rail_down_forces_relay_off(void) {
500   SafetyCore c;
501   Snapshot s;
502   lampOnAt31s(&c, &s, NULL, 0);
503   setLight(&s, 10.0f, HEALTH_UNCALIBRATED, 32000);
504   SafetyInputs in = {&s, NULL, 0, 32000, NULL, 0, 0, true};
505   SafetyOutputs o;
506   safetyCoreStep(&c, &in, &o);
507   TEST_ASSERT_FALSE(o.relayOn);
508   TEST_ASSERT_EQUAL(LAMP_REASON_UNPOWERED, o.lampReason);
509   TEST_ASSERT_EQUAL_STRING("no 5 V", lampReasonName(o.lampReason));
510   setLight(&s, 10.0f, HEALTH_UNCALIBRATED, 33000);
511   o = step(&c, &s, 33000, NULL, 0);  // rail back: still dark, but the OFF dwell holds
512   TEST_ASSERT_FALSE(o.relayOn);
513 }
514
515 static void test_safety_dark_turns_lamp_on_after_dwell(void) {
516   SafetyCore c;
517   Snapshot s;
518   lampOnAt31s(&c, &s, NULL, 0);
519   SafetyOutputs o = step(&c, &s, 31100, NULL, 0);
520   TEST_ASSERT_TRUE(o.relayOn);
521   TEST_ASSERT_EQUAL_UINT8(LAMP_REASON_LEVEL, o.lampReason);
522 }
523
524 static void test_safety_stale_light_forces_off_without_new_sample(void) {  // audit B1
525   SafetyCore c;
526   Snapshot s;
527   lampOnAt31s(&c, &s, NULL, 0);
528   // Acquisition stalls: the snapshot keeps the reading that ended at 31 s.
529   TEST_ASSERT_TRUE(step(&c, &s, 33900, NULL, 0).relayOn);
530   TEST_ASSERT_TRUE(step(&c, &s, 34000, NULL, 0).relayOn);  // exactly 3 s: still fresh
531   SafetyOutputs o = step(&c, &s, 34100, NULL, 0);
532   TEST_ASSERT_FALSE(o.relayOn);
533   TEST_ASSERT_EQUAL_UINT8(LAMP_REASON_STALE, o.lampReason);
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 12 OF 16

test/native/test_domain/test_main.c

Original lines 534-583 | Code and comments unchanged; long lines wrap visually

test_safety_stale_light_forces_off_without_new_sample (L524-539): Native test: construct a fixed case and assert the named expected behaviour.
test_safety_faults_count_per_result_not_per_wake (L541-554): Native test: construct a fixed case and assert the named expected behaviour.

```
534   // Data resumes, still dark: the OFF dwell started at the forced OFF.
535   setLight(&s, 10.0f, HEALTH_UNCALIBRATED, 35000);
536   TEST_ASSERT_FALSE(step(&c, &s, 35000, NULL, 0).relayOn);
537   setLight(&s, 10.0f, HEALTH_UNCALIBRATED, 64100);
538   TEST_ASSERT_TRUE(step(&c, &s, 64100, NULL, 0).relayOn);
539 }
540
541 static void test_safety_faults_count_per_result_not_per_wake(void) {
542   SafetyCore c;
543   Snapshot s;
544   lampOnAt31s(&c, &s, NULL, 0);
545   setLight(&s, 0.0f, HEALTH_FAULT, 32000);
546   for (MonoMs t = 32000; t < 33000; t += 100) {  // ten wakes, one FAULT result
547     TEST_ASSERT_TRUE(step(&c, &s, t, NULL, 0).relayOn);
548   }
549   TEST_ASSERT_EQUAL_UINT8(LAMP_REASON_FAULT, step(&c, &s, 33000, NULL, 0).lampReason);
550   setLight(&s, 0.0f, HEALTH_FAULT, 33000);
551   TEST_ASSERT_TRUE(step(&c, &s, 33000, NULL, 0).relayOn);
552   setLight(&s, 0.0f, HEALTH_FAULT, 34000);  // third in a row (faultsToOff = 3)
553   TEST_ASSERT_FALSE(step(&c, &s, 34000, NULL, 0).relayOn);
554 }
555
556 static AlarmEvent hotEvent(RuleDecision d, float v, MonoMs t) {
557   AlarmEvent e;
558   e.ruleId = 0;
559   e.decision = d;
560   e.sample = airTemp(v, t);
561   return e;
562 }
563
564 static void test_safety_latched_interlock_beats_lamp_until_reset(void) {
565   SafetyCore c;
566   Snapshot s;
567   lampOnAt31s(&c, &s, &kHotRule, 1);
568   const AlarmEvent trip = hotEvent(RULE_DECISION_TRIP, 41.0f, 31200);
569   SafetyOutputs o = step(&c, &s, 31200, &trip, 1);
570   TEST_ASSERT_FALSE(o.relayOn);
571   TEST_ASSERT_TRUE(o.interlock);
572   TEST_ASSERT_EQUAL_UINT8(LAMP_REASON_INTERLOCK, o.lampReason);
573   TEST_ASSERT_EQUAL_HEX16(0x0001, o.latchedMask);
574
575   TEST_ASSERT_FALSE(safetyCoreResetLatch(&c, 0));  // still active: refused
576   const AlarmEvent clear = hotEvent(RULE_DECISION_CLEAR, 36.0f, 40000);
577   setLight(&s, 10.0f, HEALTH_UNCALIBRATED, 40000);
578   o = step(&c, &s, 40000, &clear, 1);
579   TEST_ASSERT_FALSE(o.relayOn);  // cleared but latched
580   TEST_ASSERT_EQUAL_HEX16(0x0001, o.latchedMask);
581   TEST_ASSERT_EQUAL_HEX16(0x0000, o.activeMask);
582
583   TEST_ASSERT_TRUE(safetyCoreResetLatch(&c, 0));
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 13 OF 16

test/native/test_domain/test_main.c

Original lines 584-632 | Code and comments unchanged; long lines wrap visually

test_safety_latched_interlock_beats_lamp_until_reset (L564-590): Native test: construct a fixed case and assert the named expected behaviour.
test_safety_missing_interlock_input_holds_relay_off (L594-607): Native test: construct a fixed case and assert the named expected behaviour.

```
584   setLight(&s, 10.0f, HEALTH_UNCALIBRATED, 41000);
585   s.readings[METRIC_AIR_TEMPERATURE] = airTemp(36.0f, 41000);
586   TEST_ASSERT_FALSE(step(&c, &s, 41000, NULL, 0).relayOn);  // OFF dwell from the interlock
587   setLight(&s, 10.0f, HEALTH_UNCALIBRATED, 61300);
588   s.readings[METRIC_AIR_TEMPERATURE] = airTemp(36.0f, 61300);
589   TEST_ASSERT_TRUE(step(&c, &s, 61300, NULL, 0).relayOn);
590 }
591
592 // Audit P2: an enabled relay-interlock rule without fresh input cannot see a
593 // fault, so the relay is held OFF (not an interlock: nothing tripped).
594 static void test_safety_missing_interlock_input_holds_relay_off(void) {
595   SafetyCore c;
596   Snapshot s;
597   lampOnAt31s(&c, &s, &kHotRule, 1);
598   SafetyOutputs o = step(&c, &s, 31000, NULL, 0);
599   TEST_ASSERT_TRUE(o.relayOn);
600   setLight(&s, 10.0f, HEALTH_UNCALIBRATED, 32000);
601   o = step(&c, &s, 32100, NULL, 0);  // temperature 1.1 s old, light fresh and dark
602   TEST_ASSERT_EQUAL_HEX16(0x0001, o.staleMask);
603   TEST_ASSERT_FALSE(o.interlock);
604   TEST_ASSERT_FALSE(o.relayOn);
605   TEST_ASSERT_EQUAL_UINT8(LAMP_REASON_INPUT_MISSING, o.lampReason);
606   TEST_ASSERT_EQUAL_STRING("input missing", lampReasonName(o.lampReason));
607 }
608
609 static AlarmEvent stickyTrip(float v) {
610   AlarmEvent e;
611   memset(&e, 0, sizeof(e));
612   e.ruleId = 0;
613   e.decision = RULE_DECISION_TRIP;
614   e.sample = airTemp(v, 1000);
615   return e;
616 }
617
618 // Audit P1: the send fails, Safety drains before the "lost" mark: the TRIP
619 // must still come out of the sticky slot on the next pass.
620 static void test_sticky_trip_survives_drain_during_failed_send(void) {
621   StickySlot slot;
622   memset(&slot, 0, sizeof(slot));
623   const bool sawTrip[1] = {false};
624   AlarmEvent out[2];
625   const AlarmEvent trip = stickyTrip(45.0f);
626   stickyBegin(&slot, &kHotRule, &trip);
627   TEST_ASSERT_EQUAL_UINT8(0, stickyCollect(&slot, 1, sawTrip, out));  // in flight: untouched
628   stickyEnd(&slot, false);
629   TEST_ASSERT_EQUAL_UINT8(1, stickyCollect(&slot, 1, sawTrip, out));
630   TEST_ASSERT_EQUAL_UINT8(RULE_DECISION_TRIP, out[0].decision);
631   TEST_ASSERT_FLOAT_WITHIN(1e-6f, 45.0f, out[0].sample.value);
632   TEST_ASSERT_EQUAL_UINT8(0, stickyCollect(&slot, 1, sawTrip, out));  // delivered once
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 14 OF 16

test/native/test_domain/test_main.c

Original lines 633-682 | Code and comments unchanged; long lines wrap visually

test_sticky_trip_survives_drain_during_failed_send (L620-633): Native test: construct a fixed case and assert the named expected behaviour.
test_sticky_queued_record_is_not_duplicated (L635-645): Native test: construct a fixed case and assert the named expected behaviour.

```
633 }
634
635 static void test_sticky_queued_record_is_not_duplicated(void) {
636   StickySlot slot;
637   memset(&slot, 0, sizeof(slot));
638   AlarmEvent out[2];
639   const AlarmEvent trip = stickyTrip(41.0f);
640   stickyBegin(&slot, &kHotRule, &trip);
641   stickyEnd(&slot, true);
642   const bool sawTrip[1] = {true};
643   TEST_ASSERT_EQUAL_UINT8(0, stickyCollect(&slot, 1, sawTrip, out));
644   TEST_ASSERT_FALSE(slot.tripSinceDrain);
645 }
646
647 // Audit P2: consumers re-check 5 V readings against the live rail epoch.
648 static void test_snapshot_gate_rail_marks_old_epoch_unpowered(void) {
649   Snapshot s;
650   snapshotReset(&s);
651   s.readings[METRIC_PH].health = HEALTH_UNCALIBRATED;
652   s.readings[METRIC_PH].powerEpoch = 4;
653   s.readings[METRIC_TDS].health = HEALTH_OK;
654   s.readings[METRIC_TDS].powerEpoch = 5;
655   s.readings[METRIC_AIR_TEMPERATURE].health = HEALTH_OK;  // 3.3 V: never gated
656   s.readings[METRIC_AIR_TEMPERATURE].powerEpoch = 1;
657   snapshotGateRail(&s, true, 5);
658   TEST_ASSERT_EQUAL_UINT8(HEALTH_UNPOWERED, s.readings[METRIC_PH].health);
659   TEST_ASSERT_EQUAL_UINT8(HEALTH_OK, s.readings[METRIC_TDS].health);
660   TEST_ASSERT_EQUAL_UINT8(HEALTH_OK, s.readings[METRIC_AIR_TEMPERATURE].health);
661   TEST_ASSERT_EQUAL_UINT8(HEALTH_STARTING, s.readings[METRIC_WATER_DISTANCE].health);
662   snapshotGateRail(&s, false, 5);  // same epoch, rail invalid now
663   TEST_ASSERT_EQUAL_UINT8(HEALTH_UNPOWERED, s.readings[METRIC_TDS].health);
664 }
665
666 static void test_fits_reject_non_finite_values(void) {
667   LinearCalibration c;
668   TEST_ASSERT_FALSE(linearFromTwoPoints(NAN, 7.0f, 2000.0f, 4.0f, 1, &c));
669   TEST_ASSERT_FALSE(c.valid);
670   TEST_ASSERT_FALSE(linearFromTwoPoints(1000.0f, INFINITY, 2000.0f, 4.0f, 1, &c));
671   TEST_ASSERT_FALSE(linearGainFromPoint(INFINITY, 1.0f, 0.0f, 1e30f, 1, &c));
672   TEST_ASSERT_FALSE(phFitFromBuffers(NAN, 6.86f, 2000.0f, 4.01f, 0.001f, 0.02f, 1, &c));
673   c.slope = NAN;
674   c.offset = 0.0f;
675   TEST_ASSERT_FALSE(linearCalibrationFinite(&c));
676 }
677
678 static void test_safety_trip_and_clear_in_one_batch_still_deenergises(void) {
679   AlarmRule rule = kHotRule;
680   rule.latching = false;
681   SafetyCore c;
682   Snapshot s;
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 15 OF 16

test/native/test_domain/test_main.c

Original lines 683-731 | Code and comments unchanged; long lines wrap visually

test_safety_trip_and_clear_in_one_batch_still_deenergises (L678-692): Native test: construct a fixed case and assert the named expected behaviour.
test_safety_disabled_rule_and_auto_switch (L694-715): Native test: construct a fixed case and assert the named expected behaviour.

```
683   lampOnAt31s(&c, &s, &rule, 1);
684   const AlarmEvent batch[2] = {hotEvent(RULE_DECISION_TRIP, 50.0f, 31100),
685                                hotEvent(RULE_DECISION_CLEAR, 30.0f, 31150)};
686   SafetyOutputs o = step(&c, &s, 31200, batch, 2);
687   TEST_ASSERT_FALSE(o.relayOn);  // the unhandled TRIP is not erased by the CLEAR
688   TEST_ASSERT_TRUE(o.interlock);
689   TEST_ASSERT_EQUAL_HEX16(0x0000, o.activeMask);
690   TEST_ASSERT_EQUAL_UINT32(1, c.rule[0].trips);
691   TEST_ASSERT_FLOAT_WITHIN(1e-6f, 50.0f, c.rule[0].worst);
692 }
693
694 static void test_safety_disabled_rule_and_auto_switch(void) {
695   AlarmRule rule = kHotRule;
696   rule.enabled = false;
697   SafetyCore c;
698   Snapshot s;
699   lampOnAt31s(&c, &s, &rule, 1);
700   const AlarmEvent trip = hotEvent(RULE_DECISION_TRIP, 41.0f, 31200);
701   TEST_ASSERT_TRUE(step(&c, &s, 31200, &trip, 1).relayOn);  // disabled: no interlock
702
703   SafetyConfig off = kSafetyTest;
704   off.lampAutoEnabled = false;
705   safetyCoreInit(&c, &off, NULL, 0, 0);
706   snapshotReset(&s);
707   SafetyOutputs o = {0};
708   for (MonoMs t = 1000; t <= 31000; t += 1000) {
709     setLight(&s, 10.0f, HEALTH_UNCALIBRATED, t);
710     o = step(&c, &s, t, NULL, 0);
711   }
712   TEST_ASSERT_TRUE(o.lampWantsOn);
713   TEST_ASSERT_FALSE(o.relayOn);
714   TEST_ASSERT_EQUAL_UINT8(LAMP_REASON_DISABLED, o.lampReason);
715 }
716
717 static void test_safety_reports_stale_rule_inputs(void) {
718   SafetyCore c;
719   Snapshot s;
720   safetyCoreInit(&c, &kSafetyTest, &kHotRule, 1, 0);
721   snapshotReset(&s);
722   TEST_ASSERT_EQUAL_HEX16(0x0001, step(&c, &s, 100, NULL, 0).staleMask);  // never measured
723   s.readings[METRIC_AIR_TEMPERATURE] = airTemp(25.0f, 1000);
724   TEST_ASSERT_EQUAL_HEX16(0x0000, step(&c, &s, 1500, NULL, 0).staleMask);
725   TEST_ASSERT_EQUAL_HEX16(0x0001, step(&c, &s, 2100, NULL, 0).staleMask);  // > 1 s
726 }
727
728 int main(int argc, char** argv) {
729   (void)argc;
730   (void)argv;
731   UNITY_BEGIN();
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 16 OF 16

test/native/test_domain/test_main.c

Original lines 732-781 | Code and comments unchanged; long lines wrap visually

main (L728-781): Run the existing Unity cases and return the runner result.

```
732     RUN_TEST(test_reset_reading_is_starting_and_not_actionable);
733     RUN_TEST(test_ok_reading_becomes_stale_after_limit);
734     RUN_TEST(test_stale_reading_is_not_actionable);
735     RUN_TEST(test_fault_is_not_masked_by_staleness);
736     RUN_TEST(test_divider_recovery_uses_nominal_ratios);
737     RUN_TEST(test_two_point_fit_maps_buffers);
738     RUN_TEST(test_trimmed_mean_rejects_spikes);
739     RUN_TEST(test_snapshot_reset_indexes_by_metric);
740     RUN_TEST(test_production_ready_requires_power_sense);
741     RUN_TEST(test_snapshot_reset_sets_identity);
742     RUN_TEST(test_alarm_rule_ignores_stale_sample);
743     RUN_TEST(test_sgp30_crc_matches_datasheet_vector);
744     RUN_TEST(test_sgp30_words_round_trip_and_reject_bad_crc);
745     RUN_TEST(test_sgp30_humidity_fixed_point);
746     RUN_TEST(test_ultrasonic_echo_width_to_mm);
747     RUN_TEST(test_ultrasonic_range_is_2_to_400_cm);
748     RUN_TEST(test_tds_datasheet_curve);
749     RUN_TEST(test_tds_temperature_compensation);
750     RUN_TEST(test_onewire_crc_matches_maxim_vector);
751     RUN_TEST(test_ds18b20_temperature_table);
752     RUN_TEST(test_ds18b20_scratchpad_checks);
753     RUN_TEST(test_lamp_hysteresis_band);
754     RUN_TEST(test_lamp_dwell_holds_changes);
755     RUN_TEST(test_lamp_uses_pin_mv_after_lux_fit);
756     RUN_TEST(test_lamp_faults_hold_then_force_off);
757     RUN_TEST(test_lamp_saturated_counts_as_bright);
758     RUN_TEST(test_alarm_trips_on_one_sample_and_clears_with_hysteresis);
759     RUN_TEST(test_alarm_persistence_needs_sustained_valid_samples);
760     RUN_TEST(test_alarm_below_rule_mirrors_above);
761     RUN_TEST(test_alarm_stale_limits_per_metric);
762     RUN_TEST(test_schedule_bme280_runs_four_times_per_frame);
763     RUN_TEST(test_schedule_skips_missed_slots_without_burst);
764     RUN_TEST(test_schedule_jitter_within_tolerance_is_not_late);
765     RUN_TEST(test_schedule_defers_step_crossing_bme280_slot);
766     RUN_TEST(test_safety_boot_off_until_data_and_dwell);
767     RUN_TEST(test_safety_dark_turns_lamp_on_after_dwell);
768     RUN_TEST(test_safety_rail_down_forces_relay_off);
769     RUN_TEST(test_safety_stale_light_forces_off_without_new_sample);
770     RUN_TEST(test_safety_faults_count_per_result_not_per_wake);
771     RUN_TEST(test_safety_latched_interlock_beats_lamp_until_reset);
772     RUN_TEST(test_safety_trip_and_clear_in_one_batch_still_deenergises);
773     RUN_TEST(test_safety_disabled_rule_and_auto_switch);
774     RUN_TEST(test_safety_reports_stale_rule_inputs);
775     RUN_TEST(test_safety_missing_interlock_input_holds_relay_off);
776     RUN_TEST(test_sticky_trip_survives_drain_during_failed_send);
777     RUN_TEST(test_sticky_queued_record_is_not_duplicated);
778     RUN_TEST(test_snapshot_gate_rail_marks_old_epoch_unpowered);
779     RUN_TEST(test_fits_reject_non_finite_values);
780     return UNITY_END();
781 }
```

Return to guided explanation, p. 460

GUIDED SOURCE READING / CHAPTER 16

A test gives an executed explanation of an ordering rule

The power state machine is pure C, so its ownership protocol can be demonstrated with invented timestamps rather than a physical unplug. Read the test as a sequence of inputs and asserted answers.

Original excerpt: test/native/test_power/test_main.c (p. 497) / lines 120-134. The complete unchanged file follows.

```
120  static void test_missing_release_reports_degraded_and_never_grants_early(void) {
121    begin(true, &kCfg);
122    s_in.externalPower = false;
123    at(2000);
124    at(2050);  // quiesce sent
125    at(2251);  // 200 ms later, still not released
126    TEST_ASSERT_EQUAL(POWER_MODE_DEGRADED, s_f.mode);
127    TEST_ASSERT_EQUAL(POWER_WORKER_NONE, s_a.grant);
128    at(4000);
129    TEST_ASSERT_EQUAL(POWER_WORKER_NONE, s_a.grant);
130    s_in.releasedGen = 1;  // late, but it came
131    s_in.networkOff = true;
132    at(4010);
133    TEST_ASSERT_EQUAL(POWER_WORKER_LOW, s_a.grant);
134    TEST_ASSERT_EQUAL(POWER_MODE_LOW_POWER, s_f.mode);
```

The fixture initializes known timing policy and advances the machine through boot and loss. Assigning now changes the input time instantly; it does not make the PC wait. Actions are outputs to inspect, not hardware commands executed by the test.

The test deliberately withholds the expected release generation. After the acknowledgement deadline, it checks DEGRADED and the absence of a rival grant. A later matching release lets the machine continue. Those assertions encode the reason stepQuiescing returns early on a missing acknowledgement.

This is stronger than checking that a function was called once: the test demonstrates a meaningful property across several steps. It also shows the scope of the evidence. FreeRTOS notification timing and the GPIO divider are not executed by this pure function, so target testing addresses those other parts of the path.

The complete three native suites follow in their original form. Their named cases provide further worked examples for CRC errors, stale inputs, lamp dwell, frame fragmentation and configuration validation. Reading inputs and expected outputs beside the implementation is another way to understand why its guards are placed before state changes.

ORIGINAL SOURCE / CHAPTER 16 / 1 OF 6

test/native/test_power/test_main.c

Original lines 1-44 | Code and comments unchanged; long lines wrap visually

setUp (L10-10): Unity test hook; the original body is empty.
tearDown (L11-11): Unity test hook; the original body is empty.

```
1  // Host unit tests for the WP9 power-mode state machine (pio test -e native):
2  // boot paths, debounce, the quiesce -> release -> grant lease order, Wi-Fi
3  // acknowledgement, DEGRADED on missing acknowledgements, abort on a new loss
4  // during the return, and the report-only mode of the kill switch.
5  #include <string.h>
6  #include <unity.h>
7
8  #include "domain/power_mode.h"
9
10 void setUp(void) {}
11 void tearDown(void) {}
12
13 static const PowerModeConfig kCfg = {50, 500, 200, 2000, true};
14
15 static PowerModeFsm s_f;
16 static PowerModeActions s_a;
17 static PowerModeInputs s_in;
18
19 static void begin(bool power, const PowerModeConfig* cfg) {
20   powerModeInit(&s_f, cfg);
21   memset(&s_in, 0, sizeof(s_in));
22   s_in.externalPower = power;
23   s_in.now = 1000;
24   powerModeStep(&s_f, &s_in, &s_a);
25 }
26
27 static void at(MonoMs now) {
28   s_in.now = now;
29   powerModeStep(&s_f, &s_in, &s_a);
30 }
31
32 static void test_boot_with_power_grants_normal_and_restores_rail_when_stable(void) {
33   begin(true, &kCfg);
34   TEST_ASSERT_EQUAL(POWER_MODE_NORMAL, s_f.mode);
35   TEST_ASSERT_EQUAL(POWER_WORKER_NORMAL, s_a.grant);
36   TEST_ASSERT_EQUAL_UINT32(1, s_a.grantGen);
37   TEST_ASSERT_FALSE(s_a.restoreRail);  // not stable yet
38   at(1499);
39   TEST_ASSERT_FALSE(s_a.restoreRail);
40   at(1500);
41   TEST_ASSERT_TRUE(s_a.restoreRail);
42   s_in.railValid = true;
43   at(1600);
44   TEST_ASSERT_FALSE(s_a.restoreRail);
```

Return to guided explanation, p. 496

ORIGINAL SOURCE / CHAPTER 16 / 2 OF 6

test/native/test_power/test_main.c

Original lines 45-88 | Code and comments unchanged; long lines wrap visually

test_boot_with_power_grants_normal_and_restores_rail_when_stable (L32-45): Native test: construct a fixed case and assert the named expected behaviour.
test_boot_without_power_takes_the_low_power_path (L47-55): Native test: construct a fixed case and assert the named expected behaviour.

```
45  }
46
47  static void test_boot_without_power_takes_the_low_power_path(void) {
48    begin(false, &kCfg);
49    TEST_ASSERT_EQUAL(POWER_WORKER_LOW, s_a.grant);
50    TEST_ASSERT_TRUE(s_a.wifiStop);
51    TEST_ASSERT_EQUAL(POWER_MODE_TRANSITION_TO_LOW, s_f.mode);
52    s_in.networkOff = true;
53    at(1010);
54    TEST_ASSERT_EQUAL(POWER_MODE_LOW_POWER, s_f.mode);
55  }
56
57  // Full loss and return, every acknowledgement on time.
58  static void test_loss_and_return_hand_the_lease_over_in_order(void) {
59    begin(true, &kCfg);
60    s_in.railValid = true;
61    at(2000);
62    // The ISR invalidates the rail on the first edge; the level is low.
63    s_in.externalPower = false;
64    s_in.railValid = false;
65    at(2010);
66    TEST_ASSERT_EQUAL(POWER_MODE_NORMAL, s_f.mode);  // debouncing
67    TEST_ASSERT_EQUAL(POWER_WORKER_NONE, s_a.quiesce);
68    at(2060);
69    TEST_ASSERT_EQUAL(POWER_MODE_TRANSITION_TO_LOW, s_f.mode);
70    TEST_ASSERT_EQUAL(POWER_WORKER_NORMAL, s_a.quiesce);
71    TEST_ASSERT_EQUAL_UINT32(1, s_a.quiesceGen);
72    TEST_ASSERT_TRUE(s_a.wifiStop);
73    at(2100);  // not released yet: no grant
74    TEST_ASSERT_EQUAL(POWER_WORKER_NONE, s_a.grant);
75    s_in.releasedGen = 1;
76    at(2120);
77    TEST_ASSERT_EQUAL(POWER_WORKER_LOW, s_a.grant);
78    TEST_ASSERT_EQUAL_UINT32(2, s_a.grantGen);
79    TEST_ASSERT_EQUAL(POWER_MODE_TRANSITION_TO_LOW, s_f.mode);  // Wi-Fi not off yet
80    s_in.networkOff = true;
81    at(2200);
82    TEST_ASSERT_EQUAL(POWER_MODE_LOW_POWER, s_f.mode);
83
84    // Power back: 500 ms stable before anything moves.
85    s_in.externalPower = true;
86    s_in.networkOff = false;
87    at(5000);
88    at(5499);
```

Return to guided explanation, p. 496

ORIGINAL SOURCE / CHAPTER 16 / 3 OF 6

test/native/test_power/test_main.c

Original lines 89-132 | Code and comments unchanged; long lines wrap visually

test_loss_and_return_hand_the_lease_over_in_order (L58-101): Native test: construct a fixed case and assert the named expected behaviour.
test_short_glitch_does_not_change_mode_but_rail_waits_for_stability (L103-118): Native test: construct a fixed case and assert the named expected behaviour.

```
  TEST_ASSERT_EQUAL(POWER_MODE_LOW_POWER, s_f.mode);
  at(5500);
  TEST_ASSERT_EQUAL(POWER_MODE_TRANSITION_TO_NORMAL, s_f.mode);
  TEST_ASSERT_EQUAL(POWER_WORKER_LOW, s_a.quiesce);
  TEST_ASSERT_EQUAL_UINT32(2, s_a.quiesceGen);
  s_in.releasedGen = 2;
  at(5520);
  TEST_ASSERT_TRUE(s_a.restoreRail);
  TEST_ASSERT_EQUAL(POWER_WORKER_NORMAL, s_a.grant);
  TEST_ASSERT_EQUAL_UINT32(3, s_a.grantGen);
  TEST_ASSERT_TRUE(s_a.wifiStart);
  TEST_ASSERT_EQUAL(POWER_MODE_NORMAL, s_f.mode);
}

static void test_short_glitch_does_not_change_mode_but_rail_waits_for_stability(void) {
  begin(true, &kCfg);
  s_in.railValid = true;
  at(2000);
  s_in.externalPower = false;
  s_in.railValid = false;  // the ISR saw it
  at(2010);
  s_in.externalPower = true;
  at(2030);  // 20 ms low: below the 50 ms debounce
  TEST_ASSERT_EQUAL(POWER_MODE_NORMAL, s_f.mode);
  TEST_ASSERT_EQUAL(POWER_WORKER_NONE, s_a.quiesce);
  at(2529);
  TEST_ASSERT_FALSE(s_a.restoreRail);
  at(2530);
  TEST_ASSERT_TRUE(s_a.restoreRail);
}

static void test_missing_release_reports_degraded_and_never_grants_early(void) {
  begin(true, &kCfg);
  s_in.externalPower = false;
  at(2000);
  at(2050);  // quiesce sent
  at(2251);  // 200 ms later, still not released
  TEST_ASSERT_EQUAL(POWER_MODE_DEGRADED, s_f.mode);
  TEST_ASSERT_EQUAL(POWER_WORKER_NONE, s_a.grant);
  at(4000);
  TEST_ASSERT_EQUAL(POWER_WORKER_NONE, s_a.grant);
  s_in.releasedGen = 1;  // late, but it came
  s_in.networkOff = true;
  at(4010);
```

Return to guided explanation, p. 496

ORIGINAL SOURCE / CHAPTER 16 / 4 OF 6

test/native/test_power/test_main.c

Original lines 133-175 | Code and comments unchanged; long lines wrap visually

test_missing_release_reports_degraded_and_never_grants_early (L120-135): Native test: construct a fixed case and assert the named expected behaviour.
test_wifi_not_off_in_time_reports_degraded (L137-150): Native test: construct a fixed case and assert the named expected behaviour.

```
133   TEST_ASSERT_EQUAL(POWER_WORKER_LOW, s_a.grant);
134   TEST_ASSERT_EQUAL(POWER_MODE_LOW_POWER, s_f.mode);
135 }
136
137 static void test_wifi_not_off_in_time_reports_degraded(void) {
138   begin(true, &kCfg);
139   s_in.externalPower = false;
140   at(2000);
141   at(2050);
142   s_in.releasedGen = 1;
143   at(2060);
144   TEST_ASSERT_EQUAL(POWER_MODE_TRANSITION_TO_LOW, s_f.mode);
145   at(4051);  // 2 s after the stop request
146   TEST_ASSERT_EQUAL(POWER_MODE_DEGRADED, s_f.mode);
147   s_in.networkOff = true;
148   at(4100);
149   TEST_ASSERT_EQUAL(POWER_MODE_LOW_POWER, s_f.mode);
150 }
151
152 static void test_new_loss_during_return_aborts_it(void) {
153   begin(false, &kCfg);
154   s_in.networkOff = true;
155   at(1010);
156   TEST_ASSERT_EQUAL(POWER_MODE_LOW_POWER, s_f.mode);
157   s_in.externalPower = true;
158   at(2000);
159   at(2500);
160   TEST_ASSERT_EQUAL(POWER_MODE_TRANSITION_TO_NORMAL, s_f.mode);
161   TEST_ASSERT_EQUAL_UINT32(1, s_a.quiesceGen);
162   s_in.externalPower = false;  // lost again before the release
163   at(2510);
164   TEST_ASSERT_EQUAL(POWER_MODE_TRANSITION_TO_LOW, s_f.mode);
165   s_in.releasedGen = 1;
166   at(2520);
167   TEST_ASSERT_EQUAL(POWER_WORKER_LOW, s_a.grant);  // the low-power worker again, new
  >
generation
168   TEST_ASSERT_EQUAL_UINT32(2, s_a.grantGen);
169   TEST_ASSERT_FALSE(s_a.restoreRail);
170   TEST_ASSERT_FALSE(s_a.wifiStart);
171   TEST_ASSERT_EQUAL(POWER_MODE_LOW_POWER, s_f.mode);
172 }
173
174 static void test_kill_switch_only_reports_the_level(void) {
175   const PowerModeConfig off = {50, 500, 200, 2000, false};
```

Return to guided explanation, p. 496

ORIGINAL SOURCE / CHAPTER 16 / 5 OF 6

test/native/test_power/test_main.c

Original lines 176-218 | Code and comments unchanged; long lines wrap visually

test_kill_switch_only_reports_the_level (L174-191): Native test: construct a fixed case and assert the named expected behaviour.
test_unsampled_glitch_restarts_stability_from_the_new_epoch (L195-209): Native test: construct a fixed case and assert the named expected behaviour.

```
176   begin(true, &off);
177   TEST_ASSERT_EQUAL(POWER_WORKER_NORMAL, s_a.grant);
178   TEST_ASSERT_TRUE(s_a.restoreRail);
179   TEST_ASSERT_EQUAL(POWER_MODE_NORMAL, s_f.mode);
180   s_in.railValid = true;
181   s_in.externalPower = false;
182   at(2000);
183   at(2050);
184   TEST_ASSERT_EQUAL(POWER_MODE_LOW_POWER, s_f.mode);
185   TEST_ASSERT_EQUAL(POWER_WORKER_NONE, s_a.quiesce);
186   TEST_ASSERT_FALSE(s_a.wifiStop);
187   s_in.externalPower = true;
188   at(2060);
189   TEST_ASSERT_EQUAL(POWER_MODE_NORMAL, s_f.mode);
190   TEST_ASSERT_EQUAL_STRING("TRANSITION_TO_NORMAL",
  >  powerModeName(POWER_MODE_TRANSITION_TO_NORMAL));
191 }
192
193 // Audit P1: a LOW-HIGH pulse between two ticks is only visible as a new rail
194 // epoch. It must restart the stability timer, and the restore names the epoch.
195 static void test_unsampled_glitch_restarts_stability_from_the_new_epoch(void) {
196   begin(true, &kCfg);
197   s_in.railValid = true;
198   s_in.railEpoch = 1;
199   at(2000);
200   s_in.railValid = false;  // the ISR saw both edges; the level is HIGH again
201   s_in.railEpoch = 3;
202   at(2010);
203   TEST_ASSERT_FALSE(s_a.restoreRail);  // the old HIGH timer no longer counts
204   at(2509);
205   TEST_ASSERT_FALSE(s_a.restoreRail);
206   at(2510);
207   TEST_ASSERT_TRUE(s_a.restoreRail);
208   TEST_ASSERT_EQUAL_UINT32(3, s_a.restoreEpoch);
209 }
210
211 static void test_muted_isr_holds_the_rail_invalid(void) {
212   begin(true, &kCfg);
213   s_in.railValid = false;
214   s_in.edgesMissed = true;  // muted after the first edge: later edges are not counted
215   at(3000);
216   TEST_ASSERT_FALSE(s_a.restoreRail);
217   s_in.edgesMissed = false;  // re-armed
218   at(3010);
```

Return to guided explanation, p. 496

ORIGINAL SOURCE / CHAPTER 16 / 6 OF 6

test/native/test_power/test_main.c

Original lines 219-262 | Code and comments unchanged; long lines wrap visually

test_muted_isr_holds_the_rail_invalid (L211-223): Return this module's sensor identity; this does not access hardware.
test_glitch_during_return_defers_the_rail_restore (L225-244): Native test: construct a fixed case and assert the named expected behaviour.

```
  at(3509);
  TEST_ASSERT_FALSE(s_a.restoreRail);
  at(3510);
  TEST_ASSERT_TRUE(s_a.restoreRail);
}

static void test_glitch_during_return_defers_the_rail_restore(void) {
  begin(false, &kCfg);
  s_in.networkOff = true;
  at(1010);
  s_in.externalPower = true;
  at(2000);
  at(2500);
  TEST_ASSERT_EQUAL(POWER_MODE_TRANSITION_TO_NORMAL, s_f.mode);
  s_in.railEpoch = 2;  // an unsampled glitch while quiescing
  at(2505);
  s_in.releasedGen = 1;
  at(2510);
  TEST_ASSERT_EQUAL(POWER_WORKER_NORMAL, s_a.grant);
  TEST_ASSERT_FALSE(s_a.restoreRail);  // 5 ms of stability is not enough
  at(3004);
  TEST_ASSERT_FALSE(s_a.restoreRail);
  at(3005);
  TEST_ASSERT_TRUE(s_a.restoreRail);
  TEST_ASSERT_EQUAL_UINT32(2, s_a.restoreEpoch);
}

int main(int argc, char** argv) {
  (void)argc;
  (void)argv;
  UNITY_BEGIN();
  RUN_TEST(test_boot_with_power_grants_normal_and_restores_rail_when_stable);
  RUN_TEST(test_boot_without_power_takes_the_low_power_path);
  RUN_TEST(test_loss_and_return_hand_the_lease_over_in_order);
  RUN_TEST(test_short_glitch_does_not_change_mode_but_rail_waits_for_stability);
  RUN_TEST(test_missing_release_reports_degraded_and_never_grants_early);
  RUN_TEST(test_wifi_not_off_in_time_reports_degraded);
  RUN_TEST(test_new_loss_during_return_aborts_it);
  RUN_TEST(test_kill_switch_only_reports_the_level);
  RUN_TEST(test_unsampled_glitch_restarts_stability_from_the_new_epoch);
  RUN_TEST(test_muted_isr_holds_the_rail_invalid);
  RUN_TEST(test_glitch_during_return_defers_the_rail_restore);
  return UNITY_END();
}
```

Return to guided explanation, p. 496

ORIGINAL SOURCE / CHAPTER 16 / 1 OF 16

test/native/test_protocol/test_main.c

Original lines 1-49 | Code and comments unchanged; long lines wrap visually

setUp (L22-22): Unity test hook; the original body is empty.
tearDown (L23-23): Unity test hook; the original body is empty.

```
1  // Host unit tests for the WP8 domain modules (pio test -e native):
2  // frame reassembly, contract v2 encoders and command parser, configuration
3  // blob (schema 2, calibration, v1 migration), calibration fits and capture,
4  // event journal, backoff, and the Safety command / disabled-sensor rules.
5  #include <math.h>
6  #include <stdio.h>
7  #include <string.h>
8  #include <unity.h>
9
10 #include <stddef.h>
11
12 #include "domain/backoff.h"
13 #include "domain/cal_capture.h"
14 #include "domain/calibration.h"
15 #include "domain/config_blob.h"
16 #include "domain/event_journal.h"
17 #include "domain/frame_codec.h"
18 #include "domain/protocol_codec.h"
19 #include "domain/safety_core.h"
20 #include "domain/sensor_set.h"
21
22 void setUp(void) {}
23 void tearDown(void) {}
24
25 static FrameAssembler s_fa;
26 static char s_out[4096];
27
28 static ProtocolError parse(const char* json, Command* cmd) {
29   return protocolParseCommand((const uint8_t*)json, strlen(json), cmd);
30 }
31
32 // ------------------------------------------------------------ frames
33
34 static void test_frame_split_across_packets(void) {
35   const uint8_t frame[] = {5, 0, 'h', 'e', 'l', 'l', 'o'};
36   frameAssemblerInit(&s_fa);
37   size_t used;
38   const uint8_t* payload;
39   size_t n;
40   for (size_t i = 0; i < sizeof(frame) - 1; ++i) {
41     TEST_ASSERT_EQUAL(FRAME_NEED_MORE, frameAssemblerFeed(&s_fa, &frame[i], 1, &used,
 >  &payload, &n));
42     TEST_ASSERT_EQUAL(1, used);
43   }
44   TEST_ASSERT_EQUAL(FRAME_READY,
45                     frameAssemblerFeed(&s_fa, &frame[sizeof(frame) - 1], 1, &used, &payload,
 >  &n));
46   TEST_ASSERT_EQUAL(5, n);
47   TEST_ASSERT_EQUAL_MEMORY("hello", payload, 5);
48 }
49
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 2 OF 16

test/native/test_protocol/test_main.c

Original lines 50-98 | Code and comments unchanged; long lines wrap visually

test_frame_two_in_one_packet (L50-63): Native test: construct a fixed case and assert the named expected behaviour.
test_frame_rejects_zero_and_oversize (L65-81): Native test: construct a fixed case and assert the named expected behaviour.

```
50  static void test_frame_two_in_one_packet(void) {
51    const uint8_t data[] = {2, 0, 'a', 'b', 3, 0, 'c', 'd', 'e'};
52    frameAssemblerInit(&s_fa);
53    size_t used;
54    const uint8_t* payload;
55    size_t n;
56    TEST_ASSERT_EQUAL(FRAME_READY, frameAssemblerFeed(&s_fa, data, sizeof(data), &used,
 >  &payload, &n));
57    TEST_ASSERT_EQUAL(4, used);
58    TEST_ASSERT_EQUAL_MEMORY("ab", payload, 2);
59    TEST_ASSERT_EQUAL(FRAME_READY,
60                      frameAssemblerFeed(&s_fa, data + used, sizeof(data) - used, &used,
 >  &payload, &n));
61    TEST_ASSERT_EQUAL(5, used);
62    TEST_ASSERT_EQUAL_MEMORY("cde", payload, 3);
63  }
64
65  static void test_frame_rejects_zero_and_oversize(void) {
66    size_t used;
67    const uint8_t* payload;
68    size_t n;
69    const uint8_t zero[] = {0, 0};
70    const uint8_t big[] = {0x01, 0x10};  // 4097
71    frameAssemblerInit(&s_fa);
72    TEST_ASSERT_EQUAL(FRAME_ERROR, frameAssemblerFeed(&s_fa, zero, 2, &used, &payload, &n));
73    TEST_ASSERT_EQUAL(FRAME_ERROR, frameAssemblerFeed(&s_fa, big, 2, &used, &payload, &n));
74    TEST_ASSERT_EQUAL_UINT32(2, s_fa.rejected);
75    uint8_t hdr[2];
76    TEST_ASSERT_TRUE(frameWriteHeader(hdr, 4096));
77    TEST_ASSERT_EQUAL_HEX8(0x00, hdr[0]);
78    TEST_ASSERT_EQUAL_HEX8(0x10, hdr[1]);
79    TEST_ASSERT_FALSE(frameWriteHeader(hdr, 4097));
80    TEST_ASSERT_FALSE(frameWriteHeader(hdr, 0));
81  }
82
83  // ------------------------------------------------------------- encoders
84
85  static Snapshot s_snap;
86
87  static void test_telemetry_value_rules(void) {
88    snapshotReset(&s_snap);
89    Reading* air = &s_snap.readings[METRIC_AIR_TEMPERATURE];
90    air->health = HEALTH_OK;
91    air->value = 23.5f;
92    air->intervalEndMs = 9900;
93    air->sequence = 7;
94    Reading* ph = &s_snap.readings[METRIC_PH];
95    ph->health = HEALTH_UNCALIBRATED;
96    ph->unit = UNIT_MILLIVOLT;
97    ph->value = 1532.0f;
98    ph->intervalEndMs = 5000;  // 5 s old: beyond the 3 s ordinary limit
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 3 OF 16

test/native/test_protocol/test_main.c

Original lines 99-144 | Code and comments unchanged; long lines wrap visually

test_telemetry_value_rules (L87-120): Native test: construct a fixed case and assert the named expected behaviour.
test_telemetry_overflow_returns_zero (L122-126): Native test: construct a fixed case and assert the named expected behaviour.

```
99    Reading* tds = &s_snap.readings[METRIC_TDS];
100   tds->health = HEALTH_UNPOWERED;
101   tds->value = 999.0f;  // retained, must not be sent
102   tds->intervalEndMs = 9800;
103   Reading* light = &s_snap.readings[METRIC_LIGHT_LEVEL];
104   light->health = HEALTH_OK;
105   light->value = 80.0f;
106   light->intervalEndMs = 9900;
107
108   const SensorMask enabled = (SensorMask)(SENSOR_MASK_ALL & ~(1u << SENSOR_LIGHT));
109   const size_t n = protocolEncodeTelemetry(&s_snap, 10000, 42, 3, enabled, s_out,
 >  sizeof(s_out));
110   TEST_ASSERT_TRUE(n > 0);
111   TEST_ASSERT_NOT_NULL(strstr(s_out, "\"air_temperature\":{\"value\":23.5,\"unit\":\"degC\","
112                                      "\"health\":\"OK\",\"age_ms\":100,\"seq\":7"));
113   TEST_ASSERT_NOT_NULL(strstr(s_out, "\"ph\":{\"value\":null,\"unit\":\"mV\",\"health\":\"STA
 >  LE\""));
114   TEST_ASSERT_NOT_NULL(strstr(s_out, "\"tds\":{\"value\":null,\"unit\":\"ppm\",\"health\":\"U
 >  NPOWERED\""));
115   TEST_ASSERT_NOT_NULL(strstr(s_out, "\"light_level\":{\"value\":null"));  // disabled
116   TEST_ASSERT_NOT_NULL(strstr(s_out, "\"tvoc\":{\"value\":null,\"unit\":\"ppb\",\"health\":\"
 >  STARTING\","
117                                      "\"age_ms\":null"));
118   TEST_ASSERT_NULL(strstr(s_out, "999"));
119   TEST_ASSERT_TRUE(n < 1600);  // well inside one 4096-byte frame
120 }
121
122 static void test_telemetry_overflow_returns_zero(void) {
123   snapshotReset(&s_snap);
124   char small[64];
125   TEST_ASSERT_EQUAL(0, protocolEncodeTelemetry(&s_snap, 0, 1, 1, SENSOR_MASK_ALL, small,
 >  sizeof(small)));
126 }
127
128 static void test_state_escapes_and_hides_ip_unless_connected(void) {
129   StateView v;
130   memset(&v, 0, sizeof(v));
131   v.node = "Hydro \"ESP1\"";
132   v.fw = "0.8.0";
133   v.link = "ble";
134   v.mode = "unknown";
135   v.wifiState = "connecting";
136   v.wifiSsid = "Green\\house";
137   v.wifiIp = "192.168.1.42";
138   v.lampReason = "level";
139   v.latchedMask = 0x0002;
140   v.ruleCount = 2;
141   v.sgp30Baseline = "learning";
142   v.enabled = (SensorMask)((1u << SENSOR_PH) | (1u << SENSOR_TDS));
143   TEST_ASSERT_TRUE(protocolEncodeState(&v, s_out, sizeof(s_out)) > 0);
144   TEST_ASSERT_NOT_NULL(strstr(s_out, "\"node\":\"Hydro \\\"ESP1\\\"\""));
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 4 OF 16

test/native/test_protocol/test_main.c

Original lines 145-190 | Code and comments unchanged; long lines wrap visually

test_state_escapes_and_hides_ip_unless_connected (L128-153): Native test: construct a fixed case and assert the named expected behaviour.
test_results_and_sensor_list (L155-165): Native test: construct a fixed case and assert the named expected behaviour.

```
  TEST_ASSERT_NOT_NULL(strstr(s_out, "\"ssid\":\"Green\\\\house\""));
  TEST_ASSERT_NULL(strstr(s_out, "192.168"));
  TEST_ASSERT_NOT_NULL(strstr(s_out, "\"latched\":[1]"));
  TEST_ASSERT_NOT_NULL(strstr(s_out, "\"enabled\":[\"ph\",\"tds\"]"));
  v.wifiState = "connected";
  v.wifiRssi = -61;
  TEST_ASSERT_TRUE(protocolEncodeState(&v, s_out, sizeof(s_out)) > 0);
  TEST_ASSERT_NOT_NULL(strstr(s_out, "\"ip\":\"192.168.1.42\",\"rssi\":-61"));
}

static void test_results_and_sensor_list(void) {
  TEST_ASSERT_TRUE(protocolEncodeResultError("a1", "rule_latched", s_out, sizeof(s_out)) >
0);
  TEST_ASSERT_EQUAL_STRING(
      "{\"v\":2,\"type\":\"result\",\"id\":\"a1\",\"status\":\"error\",\"error\":\"rule_latch
ed\"}", s_out);
  TEST_ASSERT_TRUE(protocolEncodeResultBusy("a2", s_out, sizeof(s_out)) > 0);
  TEST_ASSERT_EQUAL_STRING("{\"v\":2,\"type\":\"result\",\"id\":\"a2\",\"status\":\"busy\"}",
s_out);
  TEST_ASSERT_TRUE(protocolEncodeSensorList(SENSOR_MASK_ALL, s_out, sizeof(s_out)) > 0);
  TEST_ASSERT_NOT_NULL(strstr(s_out, "\"key\":\"bme280\""));
  TEST_ASSERT_NOT_NULL(strstr(s_out, "\"metrics\":[\"air_temperature\",\"air_humidity\",\"air
_pressure\"]"));
  TEST_ASSERT_NOT_NULL(strstr(s_out, "\"metrics\":[\"tvoc\",\"co2eq\"]"));
}

// ------------------------------------------------------------- parser

typedef struct BadCase {
  const char* json;
  ProtocolError expected;
} BadCase;

static void test_parse_valid_commands(void) {
  Command c;
  TEST_ASSERT_EQUAL(PROTOCOL_OK,
                    parse("{\"v\":2,\"id\":\"x1\",\"op\":\"set_sensor_enabled\","
                          "\"data\":{\"sensor\":\"tds\",\"enabled\":false}}",
                          &c));
  TEST_ASSERT_EQUAL(CMD_OP_SET_SENSOR_ENABLED, c.op);
  TEST_ASSERT_EQUAL(SENSOR_TDS, c.sensor);
  TEST_ASSERT_FALSE(c.enabled);
  TEST_ASSERT_EQUAL_STRING("x1", c.id);

  // Key order is free; unknown keys and nested values are ignored.
  TEST_ASSERT_EQUAL(PROTOCOL_OK,
                    parse(" { \"data\" : {\"rule\": 1, \"extra\": [1,{\"a\":2}]},
\"op\":\"reset_latch\","
                          " \"x\":null, \"id\":\"r\", \"v\":2 } ",
                          &c));
  TEST_ASSERT_EQUAL(CMD_OP_RESET_LATCH, c.op);
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 5 OF 16

test/native/test_protocol/test_main.c

Original lines 191-233 | Code and comments unchanged; long lines wrap visually

test_parse_valid_commands (L174-203): Native test: construct a fixed case and assert the named expected behaviour.
test_parse_calibration_commands (L205-262): Native test: construct a fixed case and assert the named expected behaviour.

```
  TEST_ASSERT_EQUAL(1, c.rule);

  TEST_ASSERT_EQUAL(PROTOCOL_OK,
                    parse("{\"v\":2,\"id\":\"w\",\"op\":\"set_wifi\","
                          "\"data\":{\"ssid\":\"Caf\\u00e9\",\"password\":\"pa\\\"ss
word\"}}",
                          &c));
  TEST_ASSERT_EQUAL_STRING("Caf\xC3\xA9", c.ssid);
  TEST_ASSERT_EQUAL_STRING("pa\"ss word", c.password);

  TEST_ASSERT_EQUAL(PROTOCOL_OK, parse("{\"v\":2,\"id\":\"g\",\"op\":\"get_sensors\"}", &c));
  TEST_ASSERT_EQUAL(PROTOCOL_OK, parse("{\"v\":2,\"id\":\"g\",\"op\":\"get_calibration\"}",
&c));
  TEST_ASSERT_EQUAL(CMD_OP_GET_CALIBRATION, c.op);
}

static void test_parse_calibration_commands(void) {
  Command c;
  TEST_ASSERT_EQUAL(PROTOCOL_OK,
                    parse("{\"v\":2,\"id\":\"c\",\"op\":\"set_calibration\","
                          "\"data\":{\"sensor\":\"ph\",\"x1\":25000,\"y1\":7000,\"x2\":30130,
\"y2\":4010}}",
                          &c));
  TEST_ASSERT_EQUAL(CMD_OP_SET_CALIBRATION, c.op);
  TEST_ASSERT_EQUAL(SENSOR_PH, c.sensor);
  TEST_ASSERT_TRUE(c.calTwoPoints);
  TEST_ASSERT_EQUAL_INT32(25000, c.calX1);
  TEST_ASSERT_EQUAL_INT32(4010, c.calY2);
  TEST_ASSERT_EQUAL(PROTOCOL_OK, parse("{\"v\":2,\"id\":\"c\",\"op\":\"set_calibration\","
                                       "\"data\":{\"sensor\":\"tds\",\"x1\":6500,\"y1\":7070}
}",
                                       &c));
  TEST_ASSERT_EQUAL(SENSOR_TDS, c.sensor);
  TEST_ASSERT_FALSE(c.calTwoPoints);
  TEST_ASSERT_EQUAL(PROTOCOL_OK, parse("{\"v\":2,\"id\":\"c\",\"op\":\"set_calibration\","
                                       "\"data\":{\"sensor\":\"light\",\"x1\":12000,\"y1\":30
000}}",
                                       &c));
  TEST_ASSERT_EQUAL(SENSOR_LIGHT, c.sensor);
  TEST_ASSERT_EQUAL(PROTOCOL_OK,
                    parse("{\"v\":2,\"id\":\"c\",\"op\":\"cal_capture\",\"data\":{\"sensor\":
\"light\"}}", &c));
  TEST_ASSERT_EQUAL(CMD_OP_CAL_CAPTURE, c.op);
  TEST_ASSERT_EQUAL(PROTOCOL_OK,
                    parse("{\"v\":2,\"id\":\"c\",\"op\":\"reset_calibration\",\"data\":{\"sen
sor\":\"tds\"}}", &c));
  TEST_ASSERT_EQUAL(CMD_OP_RESET_CALIBRATION, c.op);

  static const BadCase kBad[] = {
      // pH needs both buffers
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 6 OF 16

test/native/test_protocol/test_main.c

Original lines 234-273 | Code and comments unchanged; long lines wrap visually

test_parse_calibration_commands (L205-262): Native test: construct a fixed case and assert the named expected behaviour.
test_encode_calibration_and_capture (L264-272): Native test: construct a fixed case and assert the named expected behaviour.

```
      {"{\"v\":2,\"id\":\"a\",\"op\":\"set_calibration\",\"data\":{\"sensor\":\"ph\",\"x1\":25000,\"y1\":7000}}",
       PROTOCOL_ERR_BAD_FIELD},
      // pH above 14
      {"{\"v\":2,\"id\":\"a\",\"op\":\"set_calibration\",\"data\":{\"sensor\":\"ph\",\"x1\":1,\"y1\":15000,"
       "\"x2\":2,\"y2\":4000}}",
       PROTOCOL_ERR_BAD_FIELD},
      // fractions are not accepted: values are scaled integers
      {"{\"v\":2,\"id\":\"a\",\"op\":\"set_calibration\",\"data\":{\"sensor\":\"tds\",\"x1\":650.5,\"y1\":707}}",
       PROTOCOL_ERR_BAD_FIELD},
      {"{\"v\":2,\"id\":\"a\",\"op\":\"set_calibration\",\"data\":{\"sensor\":\"tds\",\"x1\":0,\"y1\":707}}",
       PROTOCOL_ERR_BAD_FIELD},
      // sensors without a calibration
      {"{\"v\":2,\"id\":\"a\",\"op\":\"set_calibration\",\"data\":{\"sensor\":\"bme280\",\"x1\":1,\"y1\":1}}",
       PROTOCOL_ERR_BAD_FIELD},
      {"{\"v\":2,\"id\":\"a\",\"op\":\"cal_capture\",\"data\":{\"sensor\":\"hcsr04\"}}", PROTOCOL_ERR_BAD_FIELD},
      {"{\"v\":2,\"id\":\"a\",\"op\":\"reset_calibration\"}", PROTOCOL_ERR_BAD_FIELD},
      // the LAN ops are gone
      {"{\"v\":2,\"id\":\"a\",\"op\":\"get_lan_token\"}", PROTOCOL_ERR_UNKNOWN_OP},
      {"{\"v\":2,\"id\":\"a\",\"op\":\"hello\",\"data\":{\"token\":\"0123456789abcdef0123456789abcdef\"}}",
       PROTOCOL_ERR_UNKNOWN_OP},
  };
  for (size_t i = 0; i < sizeof(kBad) / sizeof(kBad[0]); ++i) {
    char msg[16];
    snprintf(msg, sizeof(msg), "case %u", (unsigned)i);
    TEST_ASSERT_EQUAL_MESSAGE(kBad[i].expected, parse(kBad[i].json, &c), msg);
  }
  TEST_ASSERT_EQUAL_STRING("stale", protocolErrorName(PROTOCOL_ERR_STALE));
  TEST_ASSERT_EQUAL_STRING("bad_fit", protocolErrorName(PROTOCOL_ERR_BAD_FIT));
}

static void test_encode_calibration_and_capture(void) {
  const CalibrationView v = {{-0.0057f, 22.7f, 0, true}, {1.05f, 0.0f, 9, true}, {1.0f, 0.0f, 0, false}, CAL_BIT_TDS};
  TEST_ASSERT_TRUE(protocolEncodeCalibration(&v, s_out, sizeof(s_out)) > 0);
  TEST_ASSERT_NOT_NULL(strstr(s_out, "\"ph\":{\"overridden\":false,\"rev\":0,\"valid\":true"));
  TEST_ASSERT_NOT_NULL(strstr(s_out, "\"tds\":{\"overridden\":true,\"rev\":9,\"valid\":true,\"slope\":1.05"));
  TEST_ASSERT_NOT_NULL(strstr(s_out, "\"light\":{\"overridden\":false,\"rev\":0,\"valid\":false"));
  TEST_ASSERT_TRUE(protocolEncodeCapture(SENSOR_PH, 25012, 412, s_out, sizeof(s_out)) > 0);
  TEST_ASSERT_EQUAL_STRING("{\"sensor\":\"ph\",\"x\":25012,\"age_ms\":412}", s_out);
}

```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 7 OF 16

test/native/test_protocol/test_main.c

Original lines 274-309 | Code and comments unchanged; long lines wrap visually

test_parse_rejects_malformed_and_out_of_range (L274-312): Native test: construct a fixed case and assert the named expected behaviour.

```
274 static void test_parse_rejects_malformed_and_out_of_range(void) {
275   static const BadCase kCases[] = {
276       {"", PROTOCOL_ERR_BAD_REQUEST},
277       {"[]", PROTOCOL_ERR_BAD_REQUEST},
278       {"{\"v\":2,\"id\":\"a\",\"op\":\"get_sensors\"", PROTOCOL_ERR_BAD_REQUEST},  //
 > truncated
279       {"{\"v\":2,\"id\":\"a\",\"op\":\"get_sensors\"}x", PROTOCOL_ERR_BAD_REQUEST},  //
 > trailing
280       {"{\"v\":1,\"id\":\"a\",\"op\":\"get_sensors\"}", PROTOCOL_ERR_BAD_REQUEST},  // old
 > version
281       {"{\"v\":2,\"op\":\"get_sensors\"}", PROTOCOL_ERR_BAD_REQUEST},              // no id
282       {"{\"v\":2,\"id\":\"a\"}", PROTOCOL_ERR_UNKNOWN_OP},
283       {"{\"v\":2,\"id\":\"a\",\"op\":\"add_device\"}", PROTOCOL_ERR_UNKNOWN_OP},  // v1 op
284       {"{\"v\":2,\"id\":\"a\",\"op\":\"set_sensor_enabled\",\"data\":{\"sensor\":\"mq7\",\"en
 > abled\":true}}",
285        PROTOCOL_ERR_BAD_FIELD},
286       {"{\"v\":2,\"id\":\"a\",\"op\":\"set_sensor_enabled\",\"data\":{\"sensor\":\"ph\",\"ena
 > bled\":1}}",
287        PROTOCOL_ERR_BAD_FIELD},
288       {"{\"v\":2,\"id\":\"a\",\"op\":\"set_sensor_enabled\"}", PROTOCOL_ERR_BAD_FIELD},
289       {"{\"v\":2,\"id\":\"a\",\"op\":\"reset_latch\",\"data\":{\"rule\":1.5}}",
 > PROTOCOL_ERR_BAD_FIELD},
290       {"{\"v\":2,\"id\":\"a\",\"op\":\"reset_latch\",\"data\":{\"rule\":-1}}",
 > PROTOCOL_ERR_BAD_FIELD},
291       {"{\"v\":2,\"id\":\"a\",\"op\":\"reset_latch\",\"data\":{\"rule\":16}}",
 > PROTOCOL_ERR_BAD_FIELD},
292       {"{\"v\":2,\"id\":\"a\",\"op\":\"set_wifi\",\"data\":{\"ssid\":\"x\",\"password\":\"sho
 > rt\"}}",
293        PROTOCOL_ERR_BAD_FIELD},
294       {"{\"v\":2,\"id\":\"a\",\"op\":\"set_wifi\",\"data\":{\"ssid\":\"\",\"password\":\"long
 > enough\"}}",
295        PROTOCOL_ERR_BAD_FIELD},
296       {"{\"v\":2,\"id\":\"a\",\"op\":\"set_wifi\",\"data\":{\"ssid\":\"123456789012345678901
 > 34567890123\","
297        "\"password\":\"longenough\"}}",
298        PROTOCOL_ERR_BAD_FIELD},  // 33-byte SSID
299       {"{\"v\":2,\"id\":\"a\",\"op\":\"set_wifi\",\"data\":{\"ssid\":\"x\",\"password\":\"tab
 > \\there!\"}}",
300        PROTOCOL_ERR_BAD_FIELD},  // control character
301       {"{\"v\":2,\"id\":\"a\\u0000\",\"op\":\"get_sensors\"}", PROTOCOL_ERR_BAD_REQUEST},
302       {"{\"v\":2,\"id\":\"12345678901234567890123456789012345678901\",\"op\":\"get_sensors\"}
 > ",
303        PROTOCOL_ERR_BAD_REQUEST},  // 41-char id
304       {"{\"v\":2,\"id\":\"a\",\"op\":\"get_sensors\",\"d\":[[[[[[1]]]]]]}",
 > PROTOCOL_ERR_BAD_REQUEST},
305   };
306   for (size_t i = 0; i < sizeof(kCases) / sizeof(kCases[0]); ++i) {
307     Command c;
308     char msg[16];
309     snprintf(msg, sizeof(msg), "case %u", (unsigned)i);
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 8 OF 16

test/native/test_protocol/test_main.c

Original lines 310-358 | Code and comments unchanged; long lines wrap visually

test_parse_rejects_malformed_and_out_of_range (L274-312): Native test: construct a fixed case and assert the named expected behaviour.
test_parse_keeps_id_for_error_reply (L314-320): Native test: construct a fixed case and assert the named expected behaviour.

```
    TEST_ASSERT_EQUAL_MESSAGE(kCases[i].expected, parse(kCases[i].json, &c), msg);
  }
}

static void test_parse_keeps_id_for_error_reply(void) {
  Command c;
  TEST_ASSERT_EQUAL(PROTOCOL_ERR_BAD_FIELD,
                    parse("{\"v\":2,\"id\":\"keep-me\",\"op\":\"reset_latch\",\"data\":{}}",
&c));
  TEST_ASSERT_EQUAL_STRING("keep-me", c.id);
  TEST_ASSERT_EQUAL_STRING("rule_latched", protocolErrorName(PROTOCOL_ERR_RULE_LATCHED));
}

// ------------------------------------------------------------- config blob

static const LampControlConfig kLamp = {40.0f, 120.0f, 30000, 30000, 5};
static const LinearCalibration kPh = {-0.0057f, 22.7f, 0, false};
static const LinearCalibration kTds = {1.0f, 0.0f, 1, true};
static const LinearCalibration kLight = {1.0f, 0.0f, 0, false};

static ConfigDefaults defaults(void) {
  ConfigDefaults d;
  memset(&d, 0, sizeof(d));
  d.lampAutoEnabled = true;
  d.lamp = kLamp;
  d.ph = kPh;
  d.tds = kTds;
  d.light = kLight;
  return d;
}

// Audit P2: a long integer must not overflow the accumulator (signed
// overflow is undefined), in a known field or in an ignored one.
static void test_parse_long_integers_without_overflow(void) {
  Command c;
  TEST_ASSERT_EQUAL(PROTOCOL_ERR_BAD_FIELD,
                    parse("{\"v\":2,\"id\":\"a\",\"op\":\"reset_latch\","
                          "\"data\":{\"rule\":12345678901234567890123456789012345678901234567890}}", &c));
  TEST_ASSERT_EQUAL(PROTOCOL_OK,
                    parse("{\"v\":2,\"id\":\"a\",\"op\":\"reset_latch\",\"pad\":-999999999999
999999999999999999,"
                          "\"data\":{\"rule\":1}}", &c));
  TEST_ASSERT_EQUAL_UINT8(1, c.rule);
}

// A stored fit that is not a finite number is a corrupt blob.
static void test_blob_rejects_non_finite_fit(void) {
  const ConfigDefaults d = defaults();
  ConfigBlob b;
  configBlobInit(&b, &d);
  b.ph.slope = NAN;
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 9 OF 16

test/native/test_protocol/test_main.c

Original lines 359-408 | Code and comments unchanged; long lines wrap visually

test_blob_rejects_non_finite_fit (L354-361): Native test: construct a fixed case and assert the named expected behaviour.
test_blob_defaults_seal_and_corruption (L363-377): Native test: construct a fixed case and assert the named expected behaviour.

```
359   configBlobSeal(&b);
360   TEST_ASSERT_FALSE(configBlobValid(&b));
361 }
362
363 static void test_blob_defaults_seal_and_corruption(void) {
364   const ConfigDefaults d = defaults();
365   ConfigBlob b;
366   configBlobInit(&b, &d);
367   TEST_ASSERT_TRUE(configBlobValid(&b));
368   TEST_ASSERT_EQUAL_UINT16(1, b.revision);
369   TEST_ASSERT_EQUAL_UINT16(2, b.schema);
370   TEST_ASSERT_EQUAL_HEX8(SENSOR_MASK_ALL, b.sensorEnabled);
371   TEST_ASSERT_EQUAL_HEX8(0, b.calOverridden);
372   ConfigBlob bad;
373   memcpy(&bad, &b, sizeof(bad));
374   ((uint8_t*)&bad)[12] ^= 0x01;  // one flipped bit in the lamp policy
375   TEST_ASSERT_FALSE(configBlobValid(&bad));
376   TEST_ASSERT_EQUAL_HEX32(0xCBF43926u, crc32Ieee("123456789", 9));  // standard check value
377 }
378
379 static void test_blob_sensor_change_and_slot_pick(void) {
380   const ConfigDefaults d = defaults();
381   ConfigBlob a, b;
382   configBlobInit(&a, &d);
383   TEST_ASSERT_FALSE(configBlobWithSensor(&a, SENSOR_TDS, true, &b));  // no change
384   TEST_ASSERT_TRUE(configBlobWithSensor(&a, SENSOR_TDS, false, &b));
385   TEST_ASSERT_TRUE(configBlobValid(&b));
386   TEST_ASSERT_EQUAL_UINT16(2, b.revision);
387   TEST_ASSERT_FALSE(sensorMaskHas(b.sensorEnabled, SENSOR_TDS));
388   TEST_ASSERT_TRUE(configBlobPickNewest(&a, &b) == &b);
389   TEST_ASSERT_TRUE(configBlobPickNewest(&b, &a) == &b);
390   b.crc ^= 1u;  // newest slot torn: fall back to the older one
391   TEST_ASSERT_TRUE(configBlobPickNewest(&a, &b) == &a);
392   TEST_ASSERT_NULL(configBlobPickNewest(NULL, &b));
393   // Revision wrap: 0xFFFF -> 1 is newer.
394   a.revision = 0xFFFF;
395   configBlobSeal(&a);
396   TEST_ASSERT_TRUE(configBlobWithSensor(&a, SENSOR_PH, false, &b));
397   TEST_ASSERT_EQUAL_UINT16(1, b.revision);
398   TEST_ASSERT_TRUE(configBlobPickNewest(&a, &b) == &b);
399 }
400
401 static void test_blob_refresh_defaults_follows_compiled_values(void) {
402   const ConfigDefaults d = defaults();
403   ConfigBlob b;
404   configBlobInit(&b, &d);
405   ConfigBlob tdsOff;
406   TEST_ASSERT_TRUE(configBlobWithSensor(&b, SENSOR_TDS, false, &tdsOff));
407   // Same compiled values: untouched.
408   TEST_ASSERT_FALSE(configBlobRefreshDefaults(&tdsOff, &d));
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 10 OF 16

test/native/test_protocol/test_main.c

Original lines 409-458 | Code and comments unchanged; long lines wrap visually

test_blob_refresh_defaults_follows_compiled_values (L401-419): Native test: construct a fixed case and assert the named expected behaviour.
test_blob_user_calibration_survives_refresh_and_reset (L421-446): Native test: construct a fixed case and assert the named expected behaviour.

```
409   TEST_ASSERT_EQUAL_UINT16(2, tdsOff.revision);
410   // A re-measured pH offset in the source wins over the stored copy, keeps the
411   // sensor mask, and gets a new revision.
412   ConfigDefaults d2 = d;
413   d2.ph.offset = 23.1f;
414   TEST_ASSERT_TRUE(configBlobRefreshDefaults(&tdsOff, &d2));
415   TEST_ASSERT_EQUAL_UINT16(3, tdsOff.revision);
416   TEST_ASSERT_EQUAL_FLOAT(23.1f, tdsOff.ph.offset);
417   TEST_ASSERT_FALSE(sensorMaskHas(tdsOff.sensorEnabled, SENSOR_TDS));
418   TEST_ASSERT_TRUE(configBlobValid(&tdsOff));
419 }
420
421 static void test_blob_user_calibration_survives_refresh_and_reset(void) {
422   const ConfigDefaults d = defaults();
423   ConfigBlob b, cal, reset;
424   configBlobInit(&b, &d);
425   const LinearCalibration fit = {-0.0061f, 23.9f, 0, true};
426   TEST_ASSERT_TRUE(configBlobWithCalibration(&b, SENSOR_PH, &fit, true, &cal));
427   TEST_ASSERT_TRUE(configBlobValid(&cal));
428   TEST_ASSERT_EQUAL_UINT16(2, cal.revision);
429   TEST_ASSERT_EQUAL_UINT16(2, cal.ph.revision);  // cal_rev = config revision of the fit
430   TEST_ASSERT_EQUAL_HEX8(CAL_BIT_PH, cal.calOverridden);
431   // A changed compiled pH default does NOT replace the user fit...
432   ConfigDefaults d2 = d;
433   d2.ph.offset = 99.0f;
434   d2.light.slope = 2.0f;
435   TEST_ASSERT_TRUE(configBlobRefreshDefaults(&cal, &d2));  // ...but the light default does
  >
change
436   TEST_ASSERT_EQUAL_FLOAT(23.9f, cal.ph.offset);
437   TEST_ASSERT_EQUAL_FLOAT(2.0f, cal.light.slope);
438   // Reset: compiled default back, bit cleared, refreshed from then on.
439   TEST_ASSERT_TRUE(configBlobWithCalibration(&cal, SENSOR_PH, &d2.ph, false, &reset));
440   TEST_ASSERT_EQUAL_HEX8(0, reset.calOverridden);
441   TEST_ASSERT_EQUAL_FLOAT(99.0f, reset.ph.offset);
442   TEST_ASSERT_EQUAL_UINT16(0, reset.ph.revision);
443   TEST_ASSERT_FALSE(configBlobWithCalibration(&cal, SENSOR_BME280, &fit, true, &reset));
444   TEST_ASSERT_TRUE(configBlobCalibration(&cal, SENSOR_LIGHT) == &cal.light);
445   TEST_ASSERT_NULL(configBlobCalibration(&cal, SENSOR_SGP30));
446 }
447
448 static void test_blob_migrates_schema_1(void) {
449   ConfigBlobV1 old;
450   memset(&old, 0, sizeof(old));
451   old.magic = CONFIG_BLOB_MAGIC;
452   old.schema = 1;
453   old.revision = 7;
454   old.sensorEnabled = (SensorMask)(SENSOR_MASK_ALL & ~(1u << SENSOR_TDS));
455   old.lampAutoEnabled = true;
456   old.lamp = kLamp;
457   old.ph = kPh;
458   old.tds = kTds;
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 11 OF 16

test/native/test_protocol/test_main.c

Original lines 459-502 | Code and comments unchanged; long lines wrap visually

test_blob_migrates_schema_1 (L448-472): Native test: construct a fixed case and assert the named expected behaviour.
test_calibration_fits (L476-496): Native test: construct a fixed case and assert the named expected behaviour.

```
  old.crc = crc32Ieee(&old, (uint32_t)offsetof(ConfigBlobV1, crc));
  const ConfigDefaults d = defaults();
  ConfigBlob b;
  TEST_ASSERT_TRUE(configBlobMigrateV1(&old, sizeof(old), &d, &b));
  TEST_ASSERT_TRUE(configBlobValid(&b));
  TEST_ASSERT_EQUAL_UINT16(7, b.revision);
  TEST_ASSERT_EQUAL_HEX8(old.sensorEnabled, b.sensorEnabled);  // the user's choice survives
> the update
  TEST_ASSERT_EQUAL_HEX8(0, b.calOverridden);
  TEST_ASSERT_FALSE(b.light.valid);
  old.crc ^= 1u;
  TEST_ASSERT_FALSE(configBlobMigrateV1(&old, sizeof(old), &d, &b));
  TEST_ASSERT_FALSE(configBlobMigrateV1(&old, sizeof(old) - 1, &d, &b));
  TEST_ASSERT_TRUE(sizeof(ConfigBlob) != sizeof(ConfigBlobV1));  // so getBlob() can tell
> them apart
}

// ------------------------------------------------------------ calibration fits / capture

static void test_calibration_fits(void) {
  LinearCalibration c;
  // Buffers 7.00 at 2500 mV and 4.00 at 3030 mV: slope -0.00566 pH/mV.
  TEST_ASSERT_TRUE(phFitFromBuffers(2500.0f, 7.0f, 3030.0f, 4.0f, 0.002f, 0.02f, 5, &c));
  TEST_ASSERT_TRUE(c.valid);
  TEST_ASSERT_FLOAT_WITHIN(1e-3f, 7.0f, linearApply(&c, 2500.0f));
  TEST_ASSERT_FLOAT_WITHIN(1e-3f, 4.0f, linearApply(&c, 3030.0f));
  TEST_ASSERT_FALSE(phFitFromBuffers(2500.0f, 7.0f, 2510.0f, 4.0f, 0.002f, 0.02f, 5, &c));
> // too steep
  TEST_ASSERT_FALSE(c.valid);
  TEST_ASSERT_FALSE(phFitFromBuffers(2500.0f, 7.0f, 2600.0f, 6.5f, 0.002f, 0.02f, 5, &c));
> // < 1 pH apart
  TEST_ASSERT_FALSE(phFitFromBuffers(2500.0f, 7.0f, 2500.0f, 4.0f, 0.002f, 0.02f, 5, &c));
> // same mV
  // TDS K: 650 ppm nominal in a 707 ppm solution.
  TEST_ASSERT_TRUE(linearGainFromPoint(650.0f, 707.0f, 0.5f, 2.0f, 0, &c));
  TEST_ASSERT_FLOAT_WITHIN(1e-4f, 707.0f / 650.0f, c.slope);
  TEST_ASSERT_EQUAL_FLOAT(0.0f, c.offset);
  TEST_ASSERT_FALSE(linearGainFromPoint(100.0f, 707.0f, 0.5f, 2.0f, 0, &c));  // K = 7
  TEST_ASSERT_FALSE(linearGainFromPoint(0.0f, 707.0f, 0.5f, 2.0f, 0, &c));
  // Light: 1200 mV under a 3000 lux reference.
  TEST_ASSERT_TRUE(linearGainFromPoint(1200.0f, 3000.0f, 0.01f, 100.0f, 0, &c));
  TEST_ASSERT_FLOAT_WITHIN(1e-3f, 3000.0f, linearApply(&c, 1200.0f));
}

static Reading reading(SensorId s, MetricId m, Unit u, Health h, float value, int32_t raw,
> MonoMs end) {
  Reading r;
  readingReset(&r, s, m, u);
  r.health = h;
  r.value = value;
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 12 OF 16

test/native/test_protocol/test_main.c

Original lines 503-541 | Code and comments unchanged; long lines wrap visually

reading (L498-507): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.
test_capture_from_snapshot (L509-538): Native test: construct a fixed case and assert the named expected behaviour.

```
503   r.hasRaw = true;
504   r.raw = raw;
505   r.intervalEndMs = end;
506   return r;
507 }
508
509 static void test_capture_from_snapshot(void) {
510   Snapshot snap;
511   snapshotReset(&snap);
512   const CaptureLimits lim = {5000, 30000, 10000, 15000};
513   float x;
514   uint32_t age;
515   // Nothing measured yet.
516   TEST_ASSERT_EQUAL(CAPTURE_STALE, calCaptureFromSnapshot(SENSOR_PH, &snap, 10000, &lim, &x,
  > &age));
517   // pH: pin 1500 mV through 10k/15k -> module 2500 mV.
518   snap.readings[METRIC_PH] = reading(SENSOR_PH, METRIC_PH, UNIT_PH, HEALTH_UNCALIBRATED,
  > 7.0f, 1500, 9000);
519   TEST_ASSERT_EQUAL(CAPTURE_OK, calCaptureFromSnapshot(SENSOR_PH, &snap, 10000, &lim, &x,
  > &age));
520   TEST_ASSERT_FLOAT_WITHIN(0.01f, 2500.0f, x);
521   TEST_ASSERT_EQUAL_UINT32(1000, age);
522   TEST_ASSERT_EQUAL(CAPTURE_STALE, calCaptureFromSnapshot(SENSOR_PH, &snap, 20000, &lim, &x,
  > &age));  // too old
523   snap.readings[METRIC_PH].health = HEALTH_UNPOWERED;
524   TEST_ASSERT_EQUAL(CAPTURE_STALE, calCaptureFromSnapshot(SENSOR_PH, &snap, 10000, &lim, &x,
  > &age));
525   // Light: raw pin mV even after a lux fit.
526   snap.readings[METRIC_LIGHT_LEVEL] =
527       reading(SENSOR_LIGHT, METRIC_LIGHT_LEVEL, UNIT_LUX, HEALTH_OK, 3000.0f, 1200, 9500);
528   TEST_ASSERT_EQUAL(CAPTURE_OK, calCaptureFromSnapshot(SENSOR_LIGHT, &snap, 10000, &lim, &x,
  > &age));
529   TEST_ASSERT_EQUAL_FLOAT(1200.0f, x);
530   // TDS: refused without a usable water temperature, then the nominal ppm at 25 degC.
531   snap.readings[METRIC_TDS] = reading(SENSOR_TDS, METRIC_TDS, UNIT_PPM, HEALTH_UNCALIBRATED,
  > 0.0f, 1000, 9500);
532   TEST_ASSERT_EQUAL(CAPTURE_NO_TEMPERATURE, calCaptureFromSnapshot(SENSOR_TDS, &snap, 10000,
  > &lim, &x, &age));
533   snap.readings[METRIC_WATER_TEMPERATURE] =
534       reading(SENSOR_DS18B20, METRIC_WATER_TEMPERATURE, UNIT_DEG_C, HEALTH_OK, 25.0f, 400,
  > 9000);
535   TEST_ASSERT_EQUAL(CAPTURE_OK, calCaptureFromSnapshot(SENSOR_TDS, &snap, 10000, &lim, &x,
  > &age));
536   TEST_ASSERT_FLOAT_WITHIN(0.5f, (133.42f - 255.86f + 857.39f) * 0.5f, x);  // 1 V on the
  > curve
537   TEST_ASSERT_EQUAL(CAPTURE_UNSUPPORTED, calCaptureFromSnapshot(SENSOR_BME280, &snap, 10000,
  > &lim, &x, &age));
538 }
539
540 // ------------------------------------------------------------ journal
541
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 13 OF 16

test/native/test_protocol/test_main.c

Original lines 542-591 | Code and comments unchanged; long lines wrap visually

test_journal_cursors_are_independent (L544-566): Native test: construct a fixed case and assert the named expected behaviour.
test_journal_overrun_reports_gap (L568-582): Native test: construct a fixed case and assert the named expected behaviour.

```
542 static EventJournal s_journal;
543
544 static void test_journal_cursors_are_independent(void) {
545   journalInit(&s_journal);
546   JournalCursor ble, lan;
547   journalCursorInit(&s_journal, &ble);
548   journalCursorInit(&s_journal, &lan);
549   JournalEvent e = {0};
550   for (uint8_t i = 0; i < 3; ++i) {
551     e.ruleId = i;
552     journalAppend(&s_journal, &e);
553   }
554   JournalEvent out;
555   bool gap;
556   TEST_ASSERT_TRUE(journalNext(&s_journal, &ble, &out, &gap));
557   TEST_ASSERT_EQUAL_UINT32(1, out.n);
558   TEST_ASSERT_FALSE(gap);
559   for (uint32_t n = 1; n <= 3; ++n) {
560     TEST_ASSERT_TRUE(journalNext(&s_journal, &lan, &out, &gap));
561     TEST_ASSERT_EQUAL_UINT32(n, out.n);
562   }
563   TEST_ASSERT_FALSE(journalNext(&s_journal, &lan, &out, &gap));
564   TEST_ASSERT_TRUE(journalNext(&s_journal, &ble, &out, &gap));
565   TEST_ASSERT_EQUAL_UINT32(2, out.n);
566 }
567
568 static void test_journal_overrun_reports_gap(void) {
569   journalInit(&s_journal);
570   JournalCursor c;
571   journalCursorInit(&s_journal, &c);
572   JournalEvent e = {0};
573   for (int i = 0; i < EVENT_JOURNAL_LEN + 10; ++i) journalAppend(&s_journal, &e);
574   JournalEvent out;
575   bool gap;
576   TEST_ASSERT_TRUE(journalNext(&s_journal, &c, &out, &gap));
577   TEST_ASSERT_TRUE(gap);
578   TEST_ASSERT_EQUAL_UINT32(11, out.n);  // oldest still stored
579   TEST_ASSERT_TRUE(journalNext(&s_journal, &c, &out, &gap));
580   TEST_ASSERT_FALSE(gap);
581   TEST_ASSERT_EQUAL_UINT32(12, out.n);
582 }
583
584 // ------------------------------------------------------------- backoff
585
586 static void test_backoff_doubles_with_bounded_jitter(void) {
587   Backoff b;
588   backoffInit(&b, 1000, 60000, 20);
589   const uint32_t expected[] = {1000, 2000, 4000, 8000, 16000, 32000, 60000, 60000};
590   for (size_t i = 0; i < sizeof(expected) / sizeof(expected[0]); ++i) {
591     const uint32_t base = b.currentMs;
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 14 OF 16

test/native/test_protocol/test_main.c

Original lines 592-639 | Code and comments unchanged; long lines wrap visually

test_backoff_doubles_with_bounded_jitter (L586-602): Native test: construct a fixed case and assert the named expected behaviour.
airEvent (L618-628): Read this helper with its caller: its return value or copied state supports the module lifecycle described in this chapter.

```
    TEST_ASSERT_EQUAL_UINT32(expected[i], base);
    const uint32_t lo = backoffNext(&b, 0);
    b.currentMs = base;
    const uint32_t hi = backoffNext(&b, 0xFFFFFFFFu);
    TEST_ASSERT_TRUE(lo >= base - base / 5);
    TEST_ASSERT_TRUE(hi <= base + base / 5);
    TEST_ASSERT_TRUE(hi <= 60000);
  }
  backoffReset(&b);
  TEST_ASSERT_EQUAL_UINT32(1000, b.currentMs);
}

// ------------------------------------------------------------ safety

static const SafetyConfig kSafety = {true, {40.0f, 120.0f, 0, 0, 5}, 3000};
static const AlarmRule kRules[2] = {
    {.ruleId = 0, .enabled = true, .metric = METRIC_AIR_TEMPERATURE, .comparison =
COMPARISON_ABOVE,
     .tripThreshold = 40.0f, .clearThreshold = 37.0f, .persistMs = 0, .severity =
SEVERITY_CRITICAL,
     .latching = true, .actuatorId = ACTUATOR_RELAY1, .action = ACTUATOR_ACTION_DEENERGIZE},
    {.ruleId = 1, .enabled = true, .metric = METRIC_TDS, .comparison = COMPARISON_ABOVE,
     .tripThreshold = 1500.0f, .clearThreshold = 1400.0f, .persistMs = 0, .severity =
SEVERITY_WARNING,
     .latching = false, .actuatorId = ACTUATOR_NONE, .action = ACTUATOR_ACTION_NONE},
};

static SafetyCore s_core;

static AlarmEvent airEvent(RuleDecision d, float v, MonoMs end) {
  AlarmEvent e;
  memset(&e, 0, sizeof(e));
  e.ruleId = 0;
  e.decision = d;
  readingReset(&e.sample, SENSOR_BME280, METRIC_AIR_TEMPERATURE, UNIT_DEG_C);
  e.sample.health = HEALTH_OK;
  e.sample.value = v;
  e.sample.intervalEndMs = end;
  return e;
}

static void test_safety_reset_latch_command(void) {
  safetyCoreInit(&s_core, &kSafety, kRules, 2, 0);
  snapshotReset(&s_snap);
  SafetyOutputs out;
  const AlarmEvent trip = airEvent(RULE_DECISION_TRIP, 45.0f, 100);
  SafetyInputs in = {&s_snap, &trip, 1, 100};
  safetyCoreStep(&s_core, &in, &out);
  TEST_ASSERT_EQUAL_HEX16(0x0001, out.latchedMask);

  // Still active: refused.
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 15 OF 16

test/native/test_protocol/test_main.c

Original lines 640-689 | Code and comments unchanged; long lines wrap visually

test_safety_reset_latch_command (L630-669): Native test: construct a fixed case and assert the named expected behaviour.
test_safety_disabled_sensor_cannot_silence_alarm (L671-697): Native test: construct a fixed case and assert the named expected behaviour.

```
640   const SafetyCommand reset = {SAFETY_CMD_RESET_LATCH, 0, 5000, 77};
641   SafetyInputs cmd = {&s_snap, NULL, 0, 200, &reset, 1, 0};
642   safetyCoreStep(&s_core, &cmd, &out);
643   TEST_ASSERT_EQUAL(1, out.resultCount);
644   TEST_ASSERT_EQUAL_UINT32(77, out.results[0].tag);
645   TEST_ASSERT_EQUAL(SAFETY_CMD_STILL_ACTIVE, out.results[0].outcome);
646   TEST_ASSERT_TRUE(out.interlock);
647
648   // Recovery sample clears "active"; an expired reset is still refused.
649   const AlarmEvent clear = airEvent(RULE_DECISION_CLEAR, 30.0f, 300);
650   SafetyInputs rec = {&s_snap, &clear, 1, 300};
651   safetyCoreStep(&s_core, &rec, &out);
652   TEST_ASSERT_EQUAL_HEX16(0x0001, out.latchedMask);
653   SafetyInputs late = {&s_snap, NULL, 0, 6000, &reset, 1, 0};
654   safetyCoreStep(&s_core, &late, &out);
655   TEST_ASSERT_EQUAL(SAFETY_CMD_EXPIRED, out.results[0].outcome);
656   TEST_ASSERT_EQUAL_HEX16(0x0001, out.latchedMask);
657
658   const SafetyCommand fresh = {SAFETY_CMD_RESET_LATCH, 0, 9000, 78};
659   SafetyInputs ok = {&s_snap, NULL, 0, 6100, &fresh, 1, 0};
660   safetyCoreStep(&s_core, &ok, &out);
661   TEST_ASSERT_EQUAL(SAFETY_CMD_APPLIED, out.results[0].outcome);
662   TEST_ASSERT_EQUAL_HEX16(0x0000, out.latchedMask);
663   TEST_ASSERT_FALSE(out.interlock);
664
665   const SafetyCommand bad = {SAFETY_CMD_RESET_LATCH, 5, 9000, 79};
666   SafetyInputs b = {&s_snap, NULL, 0, 6200, &bad, 1, 0};
667   safetyCoreStep(&s_core, &b, &out);
668   TEST_ASSERT_EQUAL(SAFETY_CMD_BAD_RULE, out.results[0].outcome);
669 }
670
671 static void test_safety_disabled_sensor_cannot_silence_alarm(void) {
672   // A latched or active rule blocks disabling its sensor.
673   TEST_ASSERT_TRUE(safetyRulesBlockSensor(kRules, 2, 0x0001, 0, SENSOR_BME280));
674   TEST_ASSERT_TRUE(safetyRulesBlockSensor(kRules, 2, 0, 0x0002, SENSOR_TDS));
675   TEST_ASSERT_FALSE(safetyRulesBlockSensor(kRules, 2, 0x0001, 0, SENSOR_TDS));
676   TEST_ASSERT_FALSE(safetyRulesBlockSensor(kRules, 2, 0, 0, SENSOR_BME280));
677
678   // Disabled and unlatched: no stale report. Latched: still reported, and the
679   // interlock holds, even with the input switched off.
680   safetyCoreInit(&s_core, &kSafety, kRules, 2, 0);
681   snapshotReset(&s_snap);
682   SafetyOutputs out;
683   const SensorMask off = (SensorMask)((1u << SENSOR_BME280) | (1u << SENSOR_TDS));
684   SafetyInputs quiet = {&s_snap, NULL, 0, 100, NULL, 0, off};
685   safetyCoreStep(&s_core, &quiet, &out);
686   TEST_ASSERT_EQUAL_HEX16(0x0000, out.staleMask);
687
688   const AlarmEvent trip = airEvent(RULE_DECISION_TRIP, 45.0f, 200);
689   SafetyInputs t = {&s_snap, &trip, 1, 200, NULL, 0, off};
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 16 OF 16

test/native/test_protocol/test_main.c

Original lines 690-740 | Code and comments unchanged; long lines wrap visually

test_safety_disabled_sensor_cannot_silence_alarm (L671-697): Native test: construct a fixed case and assert the named expected behaviour.
test_sensor_keys_round_trip (L699-706): Native test: construct a fixed case and assert the named expected behaviour.

```
690   safetyCoreStep(&s_core, &t, &out);
691   SafetyInputs later = {&s_snap, NULL, 0, 5000, NULL, 0, off};
692   safetyCoreStep(&s_core, &later, &out);
693   TEST_ASSERT_EQUAL_HEX16(0x0001, out.staleMask);
694   TEST_ASSERT_EQUAL_HEX16(0x0001, out.latchedMask);
695   TEST_ASSERT_TRUE(out.interlock);
696   TEST_ASSERT_FALSE(out.relayOn);
697 }
698
699 static void test_sensor_keys_round_trip(void) {
700   for (uint8_t s = 0; s < SENSOR_COUNT; ++s) {
701     const char* k = sensorKey((SensorId)s);
702     TEST_ASSERT_EQUAL(s, sensorFromKey(k, strlen(k)));
703   }
704   TEST_ASSERT_EQUAL(SENSOR_COUNT, sensorFromKey("ph2", 3));
705   TEST_ASSERT_EQUAL(SENSOR_SGP30, sensorOfMetric(METRIC_CO2EQ));
706 }
707
708 int main(int argc, char** argv) {
709   (void)argc;
710   (void)argv;
711   UNITY_BEGIN();
712   RUN_TEST(test_frame_split_across_packets);
713   RUN_TEST(test_frame_two_in_one_packet);
714   RUN_TEST(test_frame_rejects_zero_and_oversize);
715   RUN_TEST(test_telemetry_value_rules);
716   RUN_TEST(test_telemetry_overflow_returns_zero);
717   RUN_TEST(test_state_escapes_and_hides_ip_unless_connected);
718   RUN_TEST(test_results_and_sensor_list);
719   RUN_TEST(test_parse_valid_commands);
720   RUN_TEST(test_parse_calibration_commands);
721   RUN_TEST(test_encode_calibration_and_capture);
722   RUN_TEST(test_parse_rejects_malformed_and_out_of_range);
723   RUN_TEST(test_parse_keeps_id_for_error_reply);
724   RUN_TEST(test_blob_defaults_seal_and_corruption);
725   RUN_TEST(test_blob_sensor_change_and_slot_pick);
726   RUN_TEST(test_blob_refresh_defaults_follows_compiled_values);
727   RUN_TEST(test_blob_user_calibration_survives_refresh_and_reset);
728   RUN_TEST(test_blob_migrates_schema_1);
729   RUN_TEST(test_calibration_fits);
730   RUN_TEST(test_capture_from_snapshot);
731   RUN_TEST(test_journal_cursors_are_independent);
732   RUN_TEST(test_journal_overrun_reports_gap);
733   RUN_TEST(test_backoff_doubles_with_bounded_jitter);
734   RUN_TEST(test_safety_reset_latch_command);
735   RUN_TEST(test_safety_disabled_sensor_cannot_silence_alarm);
736   RUN_TEST(test_sensor_keys_round_trip);
737   RUN_TEST(test_parse_long_integers_without_overflow);
738   RUN_TEST(test_blob_rejects_non_finite_fit);
739   return UNITY_END();
740 }
```

Return to guided explanation, p. 460

ORIGINAL SOURCE / CHAPTER 16 / 1 OF 1

tools/serial_log.py

Original lines 1-41 | Code and comments unchanged; long lines wrap visually

Host Python/pyserial helper: capture timestamped serial lines without repeated monitor control-line resets.

```
"""Log the ESP serial port to a file without the monitor's DTR/RTS reset toggling.

usage: python tools/serial_log.py PORT OUTFILE SECONDS [--reset]

--reset   one controlled EN pulse (RTS) after opening, so exactly one boot is
          expected; any extra boot in the log is not caused by the monitor.
Run with PlatformIO's Python (it has pyserial):
  %USERPROFILE%\\.platformio\\penv\\Scripts\\python.exe tools/serial_log.py COM4 boot.log 60
--reset
"""
import sys
import time

import serial


def main() -> None:
    port, out, seconds = sys.argv[1], sys.argv[2], float(sys.argv[3])
    ser = serial.Serial()
    ser.port, ser.baudrate, ser.timeout = port, 115200, 0.2
    ser.dtr = False  # IO0 high: normal boot, never download mode
    ser.rts = False  # EN released
    ser.open()
    start = time.monotonic()
    with open(out, "w", encoding="utf-8", errors="replace") as log:
        if "--reset" in sys.argv:
            log.write("[logger] EN reset pulse\n")
            ser.rts = True
            time.sleep(0.15)
            ser.rts = False
        buffer = b""
        while time.monotonic() - start < seconds:
            buffer += ser.read(4096)
            while b"\n" in buffer:
                line, buffer = buffer.split(b"\n", 1)
                log.write(f"{time.monotonic() - start:8.2f} {line.decode('utf-8',
'replace').rstrip()}\n")
                log.flush()
    ser.close()


if __name__ == "__main__":
    main()
```

Return to guided explanation, p. 460

PART IV / INTEGRATION AND VERIFICATION

CHAPTER 17

Complete System Walkthroughs

A full system event crosses several modules. Following it from the first observation to the final action reveals why each boundary exists. These walkthroughs connect the earlier chapters through darkness, an alarm, calibration and failures.

In this chapter

1. Trace a dark light reading through dwell to GPIO output.
2. Follow an enabled temperature rule through retained alarm delivery.
3. Track a saved calibration and resolve partial or failed outcomes.

Chapter contents

17.1 From darkness to a relay command 521
17.2 From an overtemperature sample to an interlock 522
17.3 A saved light calibration through the whole controller 523
17.4 Failure examples explain the architecture's boundaries 524
17.5 Implementation limits to keep visible during development 525

CHAPTER 17

17.1 From darkness to a relay command

Consider a normal powered system with a valid rail, no stored light fit, valid light sampling and no enabled interlock. The example uses the configured 40/120 mV thresholds and 30 s dwell; times are illustrative.

1. Safe boot

GPIO15 is LOW before its output mode is enabled. SafetyCore starts LampControl OFF and records its boot time. No STARTING light result can energise the relay.

2. Measurement

AcqNormal starts the light window during each ordinary frame. Ten taps spaced at least 11 ms are filtered. A dark 20 mV window becomes a fresh UNCALIBRATED Reading with millivolt units, a timestamp and a sequence increment. Acquisition publishes the Snapshot.

3. Policy

Within its next ordinary health wake, Safety peeks the copied state. It recognises a new light intervalEndMs, checks the 3000 ms age limit and feeds the Reading once to LampControl. The millivolt exception makes this UNCALIBRATED light usable for the relative policy.

Before the 30 s OFF dwell expires, lamp.waiting can be true but relayOn remains false. At the first later fresh light result with the dwell elapsed, the core selects ON. With auto enabled, valid rail and no alarm or missing-input hold, relayOn becomes true.

4. Actuation and reporting

Safety notices a changed command and calls relayWrite(true), mapping to GPIO15 HIGH. It publishes SafetyStatus and enqueues a short log. The low-priority logger eventually prints it; printing does not have to finish before the GPIO write.

5. Bright light

A value above 120 mV asks for OFF. Ordinary minimum-ON dwell can delay that change; the policy reduces chatter rather than providing immediate bright-light cutoff. An interlock or stale-data force-off bypasses ordinary dwell.

This complete path explains why simply printing a dark ADC value is not the same as commanding the lamp. Sampling, health, freshness, new-result detection, dwell, automatic enable and interlock all contribute.

Original source in this PDF: src/drivers/light.c (p. 168); src/services/acquisition.c (p. 96); src/services/safety.c (p. 115); src/domain/safety_core.c (p. 307)

CHAPTER 17

17.2 From an overtemperature sample to an interlock

The ordinary build has alarm rules disabled. To understand the implemented event path, suppose a properly commissioned ABOVE air-temperature rule were enabled with trip 40 degC, clear 37 degC, zero persistence and a latched deenergise action.

1. Evaluate at the producer

Assume an externally powered, valid-rail build with the example overtemperature rule explicitly enabled. BME280 completes a fresh HEALTH_OK air-temperature Reading at 41 degC. Acquisition's evaluateRules finds the matching enabled rule. `evaluateRule()` emits TRIP and stores its evaluation state. The full Reading is copied into AlarmEvent.

2. Preserve the event before waking

`safetySubmitEvent()` updates the sticky slot, attempts a zero-wait detail queue send and notifies Safety. A later Snapshot showing cooler air cannot erase the retained TRIP merely by overwriting current state.

3. Apply the decision

Safety drains the inputs and calls SafetyCore. The rule becomes active and latched. Its interlock forces LampControl OFF and sets relayOn false. A changed output writes GPIO15 before logging the result. A subsequent CLEAR can reset active, but the latch remains until an explicit allowed reset.

Separate four delays

Interval	What sets it
Physical event to next sample	BME280 release phase, up to roughly one 250 ms period.
Trigger to accepted result	Device conversion, bus read and scheduling delays.
Accepted sample to GPIO decision	Event submission, scheduling and Safety work.
GPIO decision to load interruption	Transistor, relay mechanics and actual wiring.

The recorded synthetic latency run measures the third path, not all four. A sub-millisecond task wake cannot compensate for a 250 ms sampling interval, and a fast GPIO write is not a measured contact-opening time.

The real rule's validity, thresholds, persistence and latching policy must be commissioned. The latency test's always-trip synthetic threshold exists to measure scheduling and keeps the relay off; it is not an operating overtemperature setting.

Original source in this PDF: src/services/acquisition.c (p. 96); src/services/safety.c (p. 115); src/domain/alarm.c (p. 291); src/domain/safety_core.c (p. 307); README.md

CHAPTER 17

17.3 A saved light calibration through the whole controller

Consider an illustrative gain 3 lux/mV stored for the light sensor. The input remains the same 200 mV analog signal, but the displayed result becomes 600 lux.

From command to applied revision

The BLE host callback copies bytes. BleTransport assembles the frame, validates scaled integer fields and queues a calibration request. Configuration checks the session, deadline, alarm state and gain plausibility. It stores the inactive A/B slot and publishes revision 12, then waits for acquisition.

At a frame boundary acquisition copies revision 12, gives the driver its new fit and acknowledges 12. The next light result retains raw 200 mV, publishes value 600 with unit LUX and calibration revision 12. Its observation sequence advances. Configuration can then return applied true for that revision.

Why the lamp does not suddenly change scale

Safety reads a Snapshot copy. Lamp policy sees LUX and uses raw pin mV for its 40/120 mV thresholds. The same physical input therefore asks for the same relay decision, subject to dwell, freshness, power and interlocks. A display gain change cannot reinterpret 200 as 200 mV in one place and 600 mV in another.

From result to telemetry

The transport peeks the latest Snapshot and gates its rail-sensitive metrics against current power history. Its JSON writer emits the light's declared lux unit, health, age, sequence and calibration revision. If the sample ages past the limit, value becomes null and health STALE rather than retransmitting an old number as fresh.

At reboot the newest valid blob restores the user override. Resetting calibration clears that override and returns light to uncalibrated mV. This example shows why ownership, units, revision and health travel with the number.

Original source in this PDF: src/services/configuration.c (p. 374); src/services/acquisition.c (p. 96); src/drivers/light.c (p. 168); src/domain/lamp_control.c (p. 301); src/domain/protocol_codec.c (p. 402)

CHAPTER 17

17.4 Failure examples explain the architecture's boundaries

These examples connect data validity, scheduling and output policy. Each case shows how a local failure changes the data or decision passed to the next layer.

A sensor disappears

An init error backs off instead of spinning. A completed failure publishes health or the prior sample ages out. Missing light eventually forces OFF. A missing input for an enabled relay-interlock rule also forces OFF; a warning-only rule does not gain actuator authority merely because it is stale.

External power is lost

The first observed edge invalidates the 5V epoch immediately. The power task debounces the level, requests quiescence and stops Wi-Fi. The current worker releases before the low-power worker gets the shared drivers. HC-SR04, pH and TDS are UNPOWERED. The relay is held OFF. BLE and retained 3.3V sensors continue by design.

The pure transition decisions have host tests. Target measurements establish the timing of the divider, noisy edge storms, radio shutdown and restart.

A configuration reply times out

A change can be stored in NVS and published before acquisition acknowledges it. TIMEOUT therefore does not mean the old settings were restored. A later frame or reboot can use the stored revision. Inspect state and revision rather than blindly repeating an operation that already committed.

A BLE frame is cut

Before any telemetry chunk is sent, congestion may drop that frame. After a chunk has gone out, dropping the remainder would corrupt stream alignment. The transport retries, and if still blocked after partial transmission it disconnects to force clean reassembly in a new session.

The relay command says OFF but the lamp remains on

There is no measured contact feedback. A welded contact, transistor fault or wiring error lies outside relayIsOn. Software decision tests do not prove an electrical shutdown path.

Original source in this PDF: src/services/acquisition.c (p. 96); src/services/configuration.c (p. 374); src/transports/ble_transport.c (p. 436); src/domain/safety_core.c (p. 307)

CHAPTER 17

17.5 Implementation limits to keep visible during development

The implementation defines clear boundaries between measurement, policy, transport and hardware. The following table records what each boundary means when integrating the controller.

Boundary	Practical meaning
Ordinary alarm rules disabled	No commissioned temperature trip protection is active by default.
New power/calibration integration not bench-validated	Host tests do not establish actual transition or probe accuracy.
Wi-Fi association only	No MQTT publishing or LAN data server exists.
No tank geometry conversion	HC-SR04 publishes distance, not fill percentage or litres.
No relay feedback	Commanded state cannot detect welded contacts or coil failure.
Disconnected analog inputs can look plausible	Filtering and calibration do not identify every wiring fault.
Driver call budgets are estimates	A blocked function can prevent service-level guards from running.
Watchdog settings belong to the SDK build	The application constant does not configure timeout/panic policy.
Journal is bounded RAM	Old events can be overwritten; reboot loses the journal.

Integration details that need careful reading

NVS writes use a gate against HC-SR04 echo capture, not against every timing-sensitive operation. A successful storage commit can precede a timeout waiting for driver application. The applied configuration mailbox is published before the acquisition acknowledgement; the Reading's own configRevision identifies settings actually stamped by acquisition.

Power restore uses epoch matching, and disabled sensors remain distinct from those excluded by the power lease. The transition kill switch grants all sensors to the normal worker and suppresses power switching; it is a diagnostic fallback rather than evidence of valid 5V supply.

Thresholds, wiring and task placement form part of the complete system. Changing them requires checking the affected paths against the pinned implementation and repeating the relevant target measurements.

Original source in this PDF: include/policy_config.h (p. 286); src/services/configuration.c (p. 374); src/services/power_coordinator.c (p. 334); src/services/acquisition.c (p. 96)

APPENDIX A

References and Source Integrity

The glossary and board reference support the code chapters. The final integrity note identifies the source baseline and the embedded archive, whose manifest records each original Git blob identifier.

All printed file bytes were verified against the pinned repository tree before typesetting. Original code is unchanged; display line numbers, wrap markers and reading notes are editorial additions. No source line is omitted.

The book embeds original_project_source.zip, containing the same controller-source files plus a revision/checksum manifest. Desktop app code, its demo firmware and app-dependent PC scripts are excluded. Framework/dependency source, documentation-generation scripts, README prose and repository PDFs are outside the printed-source scope. Their APIs and architectural meaning are explained in the text.

APPENDIX A

A.1 Board profile and configuration reference

The hardware contract lives in board_profile.h, timing and task budgets in app_config.h, and adjustable lamp/alarm policy in policy_config.h. A configuration field becomes active only when an executing path reads it.

Hardware contract	Assignment
Shared retained I2C	SDA 21, SCL 47; 400000 Hz; BME280 0x76, SGP30 0x58.
Retained 1-Wire	GPIO14; externally powered DS18B20.
ADC signals	Light GPIO1, TDS GPIO6, pH GPIO7.
Ultrasonic	TRIG GPIO17, ECHO GPIO18 through divider.
External supply sense	GPIO8; 220/440 Ohm divider; verified flag true.
Relay	GPIO15; active-high transistor stage.
5V branch feedback	Unassigned, represented by -1.

The N16R8 profile reserves GPIO35-37 for octal PSRAM, GPIO19/20 for native USB, and GPIO0/3/45/46 as strapping pins. Do not infer pin availability from a different ESP32 model's diagram.

Timing and channel constants

Frame and BME280 periods are 1000 and 250 ms. Stale limits are 1000 ms for BME280 and 3000 ms for ordinary metrics. Safety's health wait is 100 ms; Supervisor's period is 1000 ms. The detail and log queues have 16 and 32 entries.

PowerCoordinator passes four active timing constants to the power state machine: 50 ms loss debounce, 500 ms stable return, 200 ms worker-release acknowledgement and 2000 ms Wi-Fi-shutdown acknowledgement. It samples pending transitions every 10 ms and steady armed operation every 100 ms. The separate power-sense diagnostic uses 20 ms level qualification; that diagnostic value does not replace production debounce.

`Rail` identifies retained 3.3V and switched 5V. Rail-mask bits describe which sensor supply a lease may allow. `boardPinAssigned()` checks against -1; `boardProfileValidate()` returns a small status record. `powerModeName()` translates the mode enum; the pure powerModeStep state machine implements transitions and PowerCoordinator executes its actions.

Original source in this PDF: include/board_profile.h (p. 32); include/app_config.h (p. 36); include/policy_config.h (p. 286); src/services/power_coordinator.c (p. 334)

APPENDIX A

A.2 Glossary: language and execution

Term	Meaning in this code
Address	A memory location for a C pointer, or a device selector on I2C; context distinguishes them.
Callback	A function address passed for another component to invoke, such as afterResult.
HAL	Hardware abstraction layer: thin functions hiding target peripheral APIs.
Driver	Stateful protocol adapter that yields Reading records.
Domain	Hardware-independent values, conversion math and decision rules.
Task	Scheduled execution context with its own stack and saved CPU state.
TCB	Task control block holding kernel task bookkeeping.
Stack	Per-task memory for calls, local variables and saved context.
Heap	Runtime allocation pool used by framework and other dynamic code.
Affinity	Restriction on which CPU core may run a task.
Preemption	Pausing a running task so a more urgent eligible task can run.
Blocked	Waiting for an event or timeout without competing for CPU time.
ISR	Interrupt service routine; must keep work short and avoid blocking APIs.
Critical section	Short mutually excluded region protecting shared state or precise timing.
Static lifetime	Storage exists for the program's lifetime, not just one call.
Function table	Set of function pointers implementing the generic driver operations.
Linker symbol	Name used to join a compiled definition with calls from other objects.
Watchdog	Independent timer/subscription mechanism detecting failure to report progress.

A task's priority is a scheduling property. A sensor's sampling rate is a measurement property. An alarm's severity is policy metadata. These values are not interchangeable: raising priority does not create more samples, and a CRITICAL enum does not itself move a GPIO.

APPENDIX A

A.3 Glossary: measurements and state

Term	Meaning in this code
Metric	One published quantity; one sensor can supply several metrics.
Health	Main verdict describing whether the result can be trusted for its purpose.
Quality flag	Additional limitation such as saturation or no compensation.
Freshness	Whether a result is within its allowed age at the time it is consumed.
Calibration	Mapping a measured signal to reference physical values.
Compensation	Correcting a measurement for a known influence, such as temperature.
CRC	Computed byte check for detecting many communication corruptions.
Fixed point	An integer representing a scaled real quantity.
Snapshot	Coherent copy of latest metrics, with potentially different sample times.
Mailbox	Length-one queue holding the most recent state.
Event	A transition worth retaining even if current state later changes.
Notification	Wake signal asking a task to inspect its real data channels.
Hysteresis	Separate trip/clear or ON/OFF thresholds that retain state between them.
Dwell	Minimum time to hold the current output state before ordinary switching.
Persistence	Required duration beyond a threshold before a trip is emitted.
Latch	Retained trip state requiring an explicit allowed reset.
Epoch	Counter identifying supply-state history across transitions.
Lease	Exclusive, generation-numbered permission to use the shared sensor engine.
Jitter	Difference between intended and actual job release times.
Budget	Expected allowed cost; not necessarily an independently enforced limit.

Millivolts, ppm, ppb, Pa and mm are declared units, not generic interchangeable numbers. The Reading's own unit is authoritative, especially for a pH driver that can publish module voltage when no fit exists.

APPENDIX A

A.4 Language reference: keywords and declarations

These entries provide a quick reading aid. The C Foundations sections explain their effects with source examples; synchronization is developed in the RTOS chapter.

Construct	Meaning to retain while reading
`static`	At file scope, a private linked name; for a local object, lifetime across calls.
`const`	Limits modification through a qualified object or pointer path.
`volatile`	Observable accesses under compiler rules; provides no general mutual exclusion.
`typedef`	Adds a name for an existing type, not a new distinct C type.
`struct`	Groups members; padding can affect total storage.
`enum`	Defines named integer constants; the project often stores state in a separate fixed-width alias.
`extern`	Declares an externally linked name; a declaration without initializer need not define storage.
`inline`	Allows inline substitution; does not require it.
`void`	No return value, an explicit no-parameter list, or a generic object-pointer type when followed by a star.
`sizeof`	Reports type or object storage size in bytes.
`_Static_assert`	Checks a constant condition during compilation.
`return`	Leaves the current function and optionally returns a value.
`break` / `continue`	Leave the nearest loop/switch, or advance to the next loop iteration.
`if` / `switch`	Select conditional or state-based execution paths.
`for` / `while` / `do`	Repeat statements; blocking still requires a wait operation.

uint32_t, size_t and bool are type names supplied by the relevant C headers on this build. NULL is a null pointer constant macro. IRAM_ATTR and FreeRTOS critical-section names are SDK constructs. Capitalization alone does not identify a C keyword.

APPENDIX A

A.5 Sources and reproducibility

[R1] Repository source baseline: Yassine511/Hydroponics-Monitoring-System-Firmware, commit `58af0f37c6b7c52ee308f3bf391499698fe93a7e`. The internal listings and source archive reproduce that revision rather than a changing branch.

Open reference online

[R2] Espressif, ESP-IDF FreeRTOS SMP guide, ESP32-S3, v4.4.2. Used for affinity, scheduling and stack-byte conventions.

Open reference online

[R3] Espressif, FreeRTOS API reference, ESP32-S3, v4.4.7. API semantics for task creation and the queue/notification family.

Open reference online

[R4] Espressif, Watchdogs, ESP32-S3, v4.4.7. Task subscription, self-reset and configuration boundary.

Open reference online

[R5] Bosch Sensortec, BME280 datasheet. Register and compensation reference for the driver analysis.

Open reference online

[R6] Analog Devices / Maxim, DS18B20 datasheet. 1-Wire transactions, temperature representation and conversion timing.

Open reference online

All technical behaviour described as implemented is grounded in the inspected repository, not inferred from an architecture diagram alone. API and datasheet references explain the interfaces; repository-specific assumptions and heuristics remain identified as such.

APPENDIX A

A.6 Sources and verification record

[R7] DFRobot, GravityTDS library, GravityTDS.cpp. The manufacturer implementation applies temperature correction to EC after the polynomial, then derives TDS. This book compares that placement with tds_codec.c.

Open reference online

[R8] Sensirion, SGP30 datasheet. IAQ command cadence, reply order, warm-up, humidity and baseline interfaces.

Open reference online

[R9] FreeRTOS_Hydroponics_Guide.pdf. Background teaching reference. The implemented behaviour, pin assignments and statement ordering in this book follow the repository source.

[R10] ThrowTheSwitch, Unity, src/unity.c, unity.h and unity_internals.h. Host runner used with all three original native suites.

Open reference online

[R11] NimBLE-Arduino official API reference. The device adapter uses the pinned 2.5.1 library; current API pages are supplementary rather than a substitute for that pinned source.

Open reference online

Validation scope

The repository was read at the pinned commit. All controller C/C++ modules, headers, build settings, bring-up entry points and native tests were reviewed. The 86 existing tests passed with host GCC, GNU11 and Unity. Desktop app source was excluded. Target compilation and physical hardware validation were not performed for this book.

The README's hardware figures are quoted as project records, including no-radio and radio-loaded alarm latency and injected-stall results. They remain separate from the host test result. Exact physical probe accuracy, actual power-transition timing and worst-case radio interference cannot be established from those host tests.

Reading later revisions

A source line number is meaningful only within the referenced revision. If the branch changes, compare functions and configuration against the commit recorded here. The complete internal source directory and original line numbers make later comparisons precise.

APPENDIX A

A.7 Sources for C, compiler and adapter semantics

[R12] ISO/IEC JTC1/SC22/WG14, N1570 public C11 committee draft. Sections 6.2.1-6.2.4 cover scope, linkage and storage duration; 6.7 covers declarations and qualifiers; 6.5 covers operators; 7.19 defines offsetof and size_t. The GNU toolchain also supports extensions described separately in this book.

Open reference online

[R13] GNU Compiler Collection, Volatiles. Defines GCC's volatile-access behaviour and explains why volatile is not a general ordering barrier for ordinary memory.

Open reference online

[R14] GNU Compiler Collection, Inline Functions. Explains static inline definitions and distinguishes inline syntax from actual inline expansion.

Open reference online

[R15] Espressif, ESP-IDF v4.4.7 interrupt allocation guide for ESP32-S3. Covers core allocation and the complete code/data requirements of IRAM-safe interrupt handlers.

Open reference online

[R16] ISO/IEC JTC1/SC22/WG21, N3337 public C++ working draft. The declarations, classes, lambdas and atomics sections explain the language constructs used by the two C++ adapters. Repository code remains authoritative for their actual callback and session behaviour.

Open reference online

INTERNAL SOURCE INDEX

Complete source directory / 1

Every entry opens its complete original listing inside this PDF. Headers define module contracts; implementations and test entry points remain with their subject chapters.

Original file	Chapter	Pages
platformio.ini (p. 28)	1	28-31
include/board_profile.h (p. 32)	1	32-34
include/app_config.h (p. 36)	1	36-37
.gitignore (p. 38)	1	38
.gitattributes (p. 39)	1	39
src/domain/types.h (p. 44)	2	44-45
src/domain/reading.h (p. 46)	2	46-47
src/domain/reading.c (p. 49)	2	49-51
src/hal/clock.h (p. 52)	2	52
src/hal/clock.c (p. 53)	2	53
src/domain/sensor_set.h (p. 54)	2	54
src/domain/sensor_set.c (p. 55)	2	55-56
src/hal/arduino_entry.h (p. 80)	3	80
src/main.c (p. 82)	3	82-83
src/domain/schedule.h (p. 84)	3	84
src/domain/schedule.c (p. 85)	3	85
src/services/snapshot_store.h (p. 86)	3	86
src/services/snapshot_store.c (p. 88)	3	88
src/services/acquisition.h (p. 89)	3	89-90
src/services/acquisition.c (p. 96)	3	96-110
src/services/safety.h (p. 111)	3	111-112
src/services/safety.c (p. 115)	3	115-122

INTERNAL SOURCE INDEX

Complete source directory / 2

Every entry opens its complete original listing inside this PDF. Headers define module contracts; implementations and test entry points remain with their subject chapters.

Original file	Chapter	Pages
src/diagnostics/logger.h (p. 123)	3	123
src/diagnostics/logger.c (p. 125)	3	125-126
src/diagnostics/progress.h (p. 127)	3	127
src/diagnostics/progress.c (p. 129)	3	129
src/services/supervisor.h (p. 130)	3	130
src/services/supervisor.c (p. 132)	3	132-137
src/drivers/sensor_driver.h (p. 144)	4	144-145
src/drivers/sensor_driver.c (p. 146)	4	146
src/hal/i2c_bus.h (p. 147)	4	147
src/hal/i2c_bus.c (p. 149)	4	149-150
src/hal/adc_channel.h (p. 151)	4	151
src/hal/adc_channel.c (p. 153)	4	153
src/domain/calibration.h (p. 154)	4	154-155
src/domain/calibration.c (p. 158)	4	158-159
src/drivers/light.h (p. 163)	5	163-164
src/drivers/light.c (p. 168)	5	168-171
test/bringup/light_main.c (p. 172)	5	172
src/drivers/bme280.h (p. 177)	6	177-178
src/drivers/bme280.c (p. 182)	6	182-188
test/bringup/bme280_main.c (p. 189)	6	189-190
src/hal/onewire_bus.h (p. 195)	7	195
src/hal/onewire_bus.c (p. 197)	7	197-199

INTERNAL SOURCE INDEX

Complete source directory / 3

Every entry opens its complete original listing inside this PDF. Headers define module contracts; implementations and test entry points remain with their subject chapters.

Original file	Chapter	Pages
src/domain/ds18b20_codec.h (p. 200)	7	200
src/domain/ds18b20_codec.c (p. 201)	7	201
src/drivers/ds18b20.h (p. 202)	7	202-203
src/drivers/ds18b20.c (p. 206)	7	206-209
test/bringup/ds18b20_main.c (p. 210)	7	210
src/drivers/ph.h (p. 215)	8	215-216
src/drivers/ph.c (p. 219)	8	219-222
test/bringup/ph_main.c (p. 223)	8	223-224
src/domain/tds_codec.h (p. 229)	9	229
src/domain/tds_codec.c (p. 230)	9	230
src/drivers/tds.h (p. 231)	9	231-232
src/drivers/tds.c (p. 235)	9	235-238
test/bringup/tds_main.c (p. 239)	9	239-240
src/domain/sgp30_codec.h (p. 246)	10	246
src/domain/sgp30_codec.c (p. 248)	10	248
src/drivers/sgp30.h (p. 249)	10	249-251
src/drivers/sgp30.c (p. 254)	10	254-261
test/bringup/sgp30_main.c (p. 262)	10	262-263
src/hal/pulse_capture.h (p. 268)	11	268-269
src/hal/pulse_capture.c (p. 271)	11	271-272
src/domain/ultrasonic.h (p. 273)	11	273
src/domain/ultrasonic.c (p. 274)	11	274

INTERNAL SOURCE INDEX

Complete source directory / 4

Every entry opens its complete original listing inside this PDF. Headers define module contracts; implementations and test entry points remain with their subject chapters.

Original file	Chapter	Pages
src/drivers/hcsr04.h (p. 275)	11	275-276
src/drivers/hcsr04.c (p. 278)	11	278-280
test/bringup/hcsr04_main.c (p. 281)	11	281
include/policy_config.h (p. 286)	12	286-287
src/domain/alarm.h (p. 288)	12	288-289
src/domain/alarm.c (p. 291)	12	291
src/domain/alarm_sticky.h (p. 292)	12	292
src/domain/alarm_sticky.c (p. 293)	12	293
src/domain/event_journal.h (p. 294)	12	294
src/domain/event_journal.c (p. 296)	12	296-297
src/domain/lamp_control.h (p. 298)	12	298-299
src/domain/lamp_control.c (p. 301)	12	301-302
src/domain/safety_core.h (p. 303)	12	303-305
src/domain/safety_core.c (p. 307)	12	307-310
src/hal/gpio_safe.h (p. 311)	12	311
src/hal/gpio_safe.c (p. 312)	12	312
src/hal/relay.h (p. 313)	12	313
src/hal/relay.c (p. 314)	12	314
test/bringup/lamp_main.c (p. 315)	12	315-316
src/services/rail_state.h (p. 323)	13	323
src/services/rail_state.c (p. 324)	13	324
src/domain/power_mode.h (p. 325)	13	325-326

INTERNAL SOURCE INDEX

Complete source directory / 5

Every entry opens its complete original listing inside this PDF. Headers define module contracts; implementations and test entry points remain with their subject chapters.

Original file	Chapter	Pages
src/domain/power_mode.c (p. 328)	13	328-331
src/services/power_coordinator.h (p. 332)	13	332
src/services/power_coordinator.c (p. 334)	13	334-337
test/bringup/power_sense_main.c (p. 338)	13	338-350
src/domain/config_blob.h (p. 356)	14	356-358
src/domain/config_blob.c (p. 360)	14	360-363
src/domain/cal_capture.h (p. 364)	14	364
src/domain/cal_capture.c (p. 366)	14	366
src/services/flash_gate.h (p. 367)	14	367
src/services/flash_gate.c (p. 369)	14	369
src/services/configuration.h (p. 370)	14	370-371
src/services/configuration.c (p. 374)	14	374-384
src/domain/frame_codec.h (p. 393)	15	393
src/domain/frame_codec.c (p. 395)	15	395-396
src/domain/protocol_codec.h (p. 397)	15	397-400
src/domain/protocol_codec.c (p. 402)	15	402-419
src/domain/backoff.h (p. 420)	15	420
src/domain/backoff.c (p. 421)	15	421
src/transports/protocol.h (p. 422)	15	422
src/transports/protocol.c (p. 423)	15	423
src/transports/frames.h (p. 424)	15	424
src/transports/frames.c (p. 425)	15	425-426

INTERNAL SOURCE INDEX

Complete source directory / 6

Every entry opens its complete original listing inside this PDF. Headers define module contracts; implementations and test entry points remain with their subject chapters.

Original file	Chapter	Pages
src/transports/ble_gatt.h (p. 427)	15	427-428
src/transports/ble_gatt.cpp (p. 429)	15	429-433
src/transports/ble_transport.h (p. 434)	15	434
src/transports/ble_transport.c (p. 436)	15	436-447
src/transports/wifi_lan.h (p. 448)	15	448
src/transports/wifi_lan.cpp (p. 449)	15	449
src/transports/wifi_mqtt.h (p. 450)	15	450-451
src/transports/wifi_mqtt.c (p. 453)	15	453-458
test/bringup/bringup_harness.h (p. 465)	16	465-467
test/bringup/climate_main.c (p. 468)	16	468-472
test/bringup/water_main.c (p. 473)	16	473-479
test/native/test_domain/test_main.c (p. 480)	16	480-495
test/native/test_power/test_main.c (p. 497)	16	497-502
test/native/test_protocol/test_main.c (p. 503)	16	503-518
tools/serial_log.py (p. 519)	16	519

SOURCE INTEGRITY

One source revision, one complete record

A firmware explanation is reproducible when its statements and listings refer to the same source revision. This book pins that relationship to the repository commit below.

Repository

Yassine511/Hydroponics-Monitoring-System-Firmware

58af0f37c6b7c52ee308f3bf391499698fe93a7e

What is preserved

The PDF prints 125 original project files, comprising 14,425 source lines. Editorial line numbers, wrap markers and reading notes sit outside the original code. The source directory identifies the chapter and page range of each file.

The embedded archive, original_project_source.zip, contains those same files as exact bytes. SOURCE_REVISION.txt records the pinned commit and the original Git blob identifier of every included file. These identities allow a later comparison to distinguish an editorial change from a firmware change.

What the archive is for

A PDF reader with attachment support can extract the archive for building, searching or comparing the source. All code needed for the explanations is already printed in the book; extraction is optional.

Reading another revision

Line numbers belong to this commit. When a function changes, compare its inputs, state transitions and callers before carrying a timing or ownership assumption into the new revision.

Return to contents | Open the source directory